

计算机网络原理

徐明伟

清华大学计算机系

课程有关信息

- 教师
 - 吴建平 (jianping@cernet.edu.cn)
 - 徐明伟 (xumw@tsinghua.edu.cn)
 - 崔勇 (cuiyong@tsinghua.edu.cn)
- 助教
 - 陈志鹏 (czp1990k@sina.com)
 - 林恒 (linheng_mail@126.com)
 - 郭迎亚 (guoyingya@csnet1.cs.tsinghua.edu.cn)
 - 杨增印 (zengyiny@163.com)
- 办公室
 - 东主楼9区321
 - 最好通过邮件联系
- 课程主页
 - 网络学堂
 - 每节课后更新课件

课程的任务、目的和基本要求

- 了解计算机网络的基本概念
- 掌握计算机网络的体系结构和参考模型
- 掌握典型计算机网络（Internet）各层协议的基本工作原理及其所采用的技术
- 学会计算机网络的一些基本设计方法
- 通过网络实验，掌握计算机网络协议的基本实现技术
- 为以后计算机网络及其应用的专题学习和研究奠定基础

主要教学内容和学时分配

第一章	引言	3
第二章	计算机网络体系结构	6
第三章	数据通信基本原理	3
第四章	物理层接口及其协议	3
第五章	数据链路控制及其协议	9
第六章	局域网与介质访问子层	6
第七章	网络互联和访问控制	9
第八章	传输层及可靠传输	3
第九章	互联网应用	4
复习		2
共计		48

计算机网络课程体系

计算机网络前沿研究

...

无线网络与移动计算

计算机网络体系结构

研究生阶段

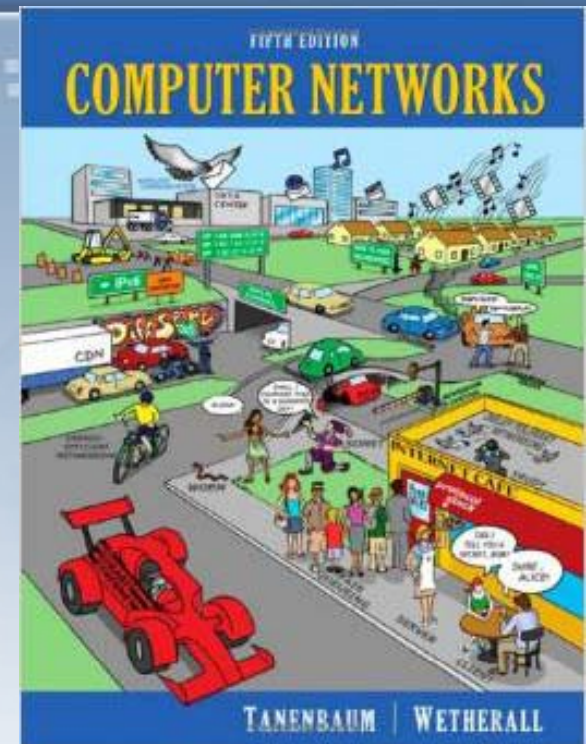
本科生阶段

计算机网络原理

计算机网络
专题训练

主要参考书

- **Andrew S. Tanenbaum, Computer Networks, 5th edition, Prentice-Hall, 2011**
- James F. Kurose and Keith W. Ross, Computer Networking: A Top-Down Approach, Addison Wesley, 6th edition, 2013
- Douglas E. Comer, Computer Networks and Internets, 6th edition, Prentice-Hall, 2015
- Larry L. Peterson and Bruce S. Davie. Computer Networks, A Systems Approach, 4th edition, Morgan Kaufmann, 2007.
- 徐明伟, 崔勇, 徐恪. 计算机网络实验教程, 第二版, 机械工业出版社, **2013**



课程评价

- 作业 (10 %)
 - 4次，两周内完成作业
- 课堂测试 (10%)
 - 次数不定
- 实验 (30 %)
 - 滑动窗口，RIP，FTP
 - 独立完成
 - 实验教材（如何购买见网络学堂的通知）
- 期末考试 (50%) ， 闭卷考试
- **Deadline means deadline**

Q&A

第一章 引言

计算机网络的历史和新进展



引言的引言

- 计算机网络是信息社会的基础设施，已经发展成为网络空间（**Cyberspace**）是陆、海、空、天之后的人类“第五空间”
- 体系结构是计算机网络的骨架
- 协议是计算机网络的心脏、血液和神经



主要内容

- 计算机网络概述
- 互联网的发展和成功经验
- 互联网的核心思想：分组交换
- 国际高速计算机网络研究计划
- 中国高速计算机网络研究计划



什么是网络 --- 从端系统的角度看

- 网络提供的最基本服务：信息传递
 - 信鸽、烽火、信使、卡车、电报、电话、互联网...
 - 类比运输服务：物体的传递
 - 马车、火车、卡车和飞机
- 不同的网络用什么区分？
 - 所提供的服务
- 服务用什么区分？
 - 功能、延迟、带宽、丢失率、端节点数目、服务接口、可靠性、实时 / 非实时等外特性



什么是网络 --- 从网络核心的角度看

- 电子、光子等作为信息载体
- 链路：光纤、电缆、卫星链路等
- 交换节点：机械 / 电 / 光
- 协议：**TCP / IP, ATM, MPLS, SONET, Ethernet, PPP, X.25, FrameRelay, AppleTalk, IPX, SNA**
- 功能：路由，差错控制、拥塞控制、服务质量(QoS)
- 应用：**FTP、HTTP、DNS...**



网络类型

■ 空间距离

- 局域网 (**LAN**): 以太网、令牌环、**FDDI**
- 城域网 (**MAN**): **DQDB, SMDS, RPR**
- 广域网 (**WAN**): **X.25, ATM, Internet**
- 个域网 (**PAN**), 家庭网络 (**HAN**), 星际网络 / 空间网络

■ 信息类型

- 数据网络 **vs.** 电话网络

■ 应用类型

- 专用网络: 飞机订票网, 银行网, 信用卡网
- 通用网络: **Internet**



网络类型 (续)

- 使用权
 - 私有：企业网
 - 公用：电话网、**Internet**
- 协议的所有权
 - 私有：**SNA**
 - 开放：**IP**
- 技术
 - 地面 **vs.** 卫星
 - 有线 **vs.** 无线
 - ...
- 协议
 - **IP, AppleTalk, SNA...**



计算机网络发展历史

- 计算机网络的形成
 - 1940年代，计算机诞生， **ENIAC...**
 - 1950年代，大型机（**Mainframe**），多终端系统
 - 1960年代，计算机网络研究，**Packet Switch vs Circuit Switch.**
 - 1969年，**ARPANET**启动
- 1970年代的计算机网络
 - **X.25** 分组交换网：各国的电信部门建设运行
 - 各种专用的网络体系结构：**SNA, DNA**
 - **Internet** 的前身**ARPANET**进行实验运行
- 1980年代的计算机网络
 - 标准化计算机网络体系结构：**ISO/OSI**
 - 局域网络 **LAN** 技术空前发展
 - 建成**NSFNET**，**Internet** 初具规模



计算机网络发展历史（续）

- **1990年代的计算机网络**
 - **Internet**商业化，空前发展
 - **Web**技术在**Internet**上得到广泛应用
- **2000年以后的计算机网络**
 - 网络应用大发展
 - 搜索引擎：**Google**，百度...
 - 社交网络：**Facebook**，**Twitter**，**QQ**，微博，微信...
 - **P2P**技术昙花一现
 - 移动互联网产业发展迅速
 - **IPv6**网络快速发展
 - 未来互联网研究受到普遍重视



互联网发展历史

- **1969年，ARPANET诞生**
- **1970年代，ARPANET作为研究项目，带宽为56kbps，连接计算机不超过100台**
- **1979年，TCP/IP成熟**
- **1980—83，APPANET和MILNET分开，ARPANET采用TCP/IP协议**
 - **1982年12月31日，美国军方从NCP协议全部改为TCP/IP协议**
- **1983年，BSD UNIX内含TCP/IP**
- **1985—86，NSF开始建设NSFNET，作为骨干网连接6个超级计算机中心，带宽为1.544Mbps，连接10,000台计算机**
 - **NSF在IBM（捐赠50台RISC6000），MCI（捐赠线路），Merit（密西根大学一些人成立的非营利公司）的支持下，建成NSFNET**
 - **1986年Cisco公司成立**



互联网发展历史（续）

- **1987—90**，NSFNET连接地区网络：**NSI**，**ESNET**，**DARTnet**，**TWBNet**，计算机数量超过**10万台**
- **1990—92**，NSFNET网络带宽发展到**45Mbps**，连接**16**个地区网络
- **1994年**，NSFNET骨干网解体，出现多个商用骨干网：**ANS**，**MCI**，**Uunet**，**Sprint...**
- **2004.1**，全球主要学术网宣布开通**IPv6**服务
- **2011.2.3**，全球互联网名称与数字地址分配机构**ICANN**宣布**IPv4**地址耗尽
- 今天：**Internet**骨干网的速率达到**100Gbps**，连接**150**多个国家的计算机



全球互联网用户统计

WORLD INTERNET USAGE AND POPULATION STATISTICS DEC 31, 2014 - Mid-Year Update

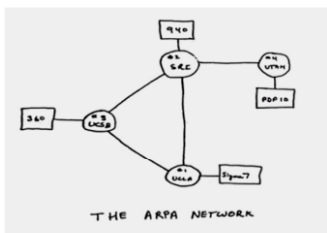
World Regions	Population (2015 Est.)	Internet Users Dec. 31, 2000	Internet Users Latest Data	Penetration (% Population)	Growth 2000-2015	Users % of Table
Africa	1,158,353,014	4,514,400	318,633,889	27.5 %	6,958.2 %	10.3 %
Asia	4,032,654,624	114,304,000	1,405,121,036	34.8 %	1,129.3 %	45.6 %
Europe	827,566,464	105,096,093	582,441,059	70.4 %	454.2 %	18.9 %
Middle East	236,137,235	3,284,800	113,609,510	48.1 %	3,358.6 %	3.7 %
North America	357,172,209	108,096,800	310,322,257	86.9 %	187.1 %	10.1 %
Latin America / Caribbean	615,583,127	18,068,919	322,422,164	52.4 %	1,684.4 %	10.5 %
Oceania / Australia	37,157,120	7,620,480	26,789,942	72.1 %	251.6 %	0.9 %
WORLD TOTAL	7,264,623,793	360,985,492	3,079,339,857	42.4 %	753.0 %	100.0 %

NOTES: (1) Internet Usage and World Population Statistics are preliminary for Dec 31, 2014. (2) CLICK on each world region name for detailed regional usage information. (3) Demographic (Population) numbers are based on data from the [US Census Bureau](#) and local census agencies. (4) Internet usage information comes from data published by [Nielsen Online](#), by the [International Telecommunications Union](#), by [GfK](#), local ICT Regulators and other reliable sources. (5) For definitions, disclaimers, navigation help and methodology, please refer to the [Site Surfing Guide](#). (6) Information in this site may be cited, giving the due credit to www.internetworldstats.com. Copyright © 2001 - 2015, Miniwatts Marketing Group. All rights reserved worldwide.

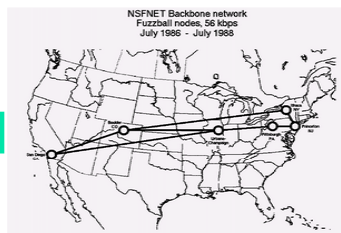


互联网演进路线

1969



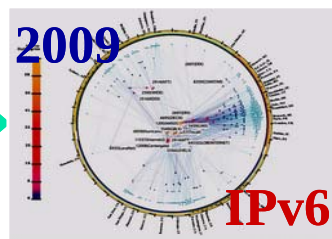
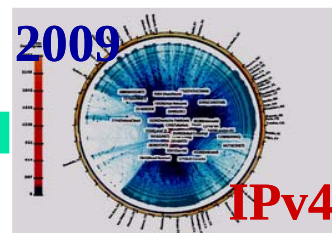
1986



1994

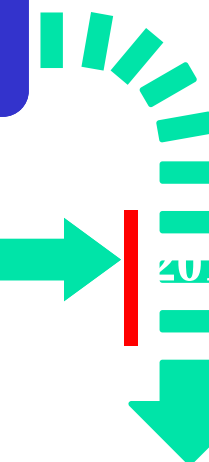


新网络体系结构
GENI和FIND



不断创新和革命

革命性路线



演进性路线





中国计算机网络的发展历史

- **1970年代末开始**
- **1980年代**
 - 局域网
 - **OSI**网络体系结构
 - 低速广域网（电话线）
- **1990年代**
 - 局域网：**Novell, TCP/IP**
 - **X.25**广域网及其应用
 - 国民经济信息化建设高潮：“金”字工程（金关、金卡、金盾、金智、金土...）
 - **Internet**在中国开始大规模发展
 - **1995年, CERNET**建成
 - **1990年代末**，自主研发成功中低端**IPv4**路由器



中国计算机网络的发展历史（续）

■ 2000年以后

- 推动以**IPv6**为基础的下一代互联网，**CNGI**
- **2006年，CNGI-CERNET2**建成，国产设备占**50 %**以上
- **2001年**，自主研发成功**IPv4**核心路由器
- **2004年**，自主研发成功**IPv6**核心路由器
- 积极参与国际标准制订，在**ITU**、**IETF**等标准化组织中影响力不断提高



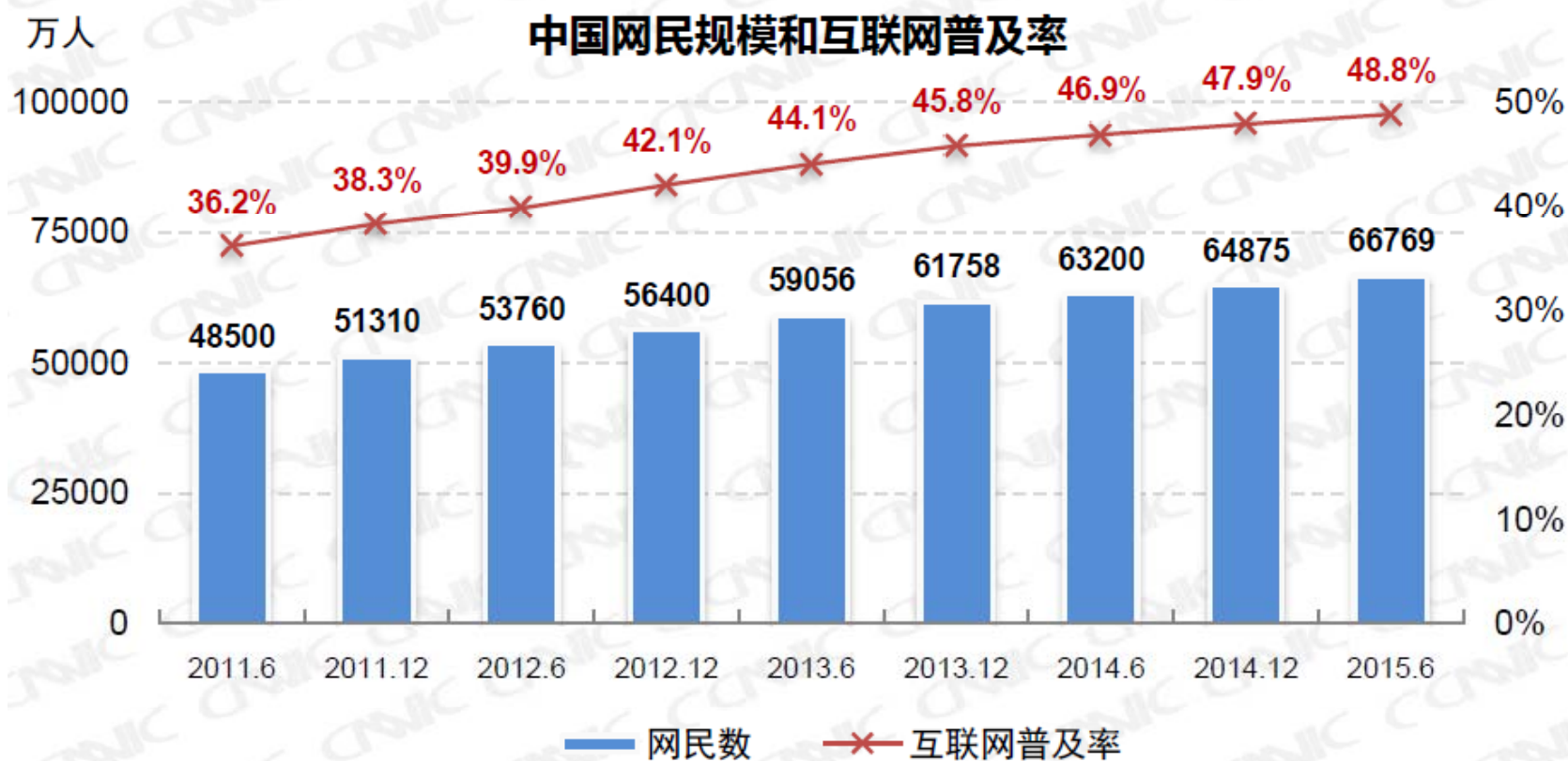
互联网在中国发展迅速

- **1987年**，中国第一个电子邮件发到**Internet**
- **1990—1993**，通过 **X.25**与国际连网
 - **Tunet**建成（清华第一代校园网，用自己研制的**X.25**交换机）
- **1994年**，中科院高能所，**64K**连接日本
- **1995年**
 - **CERNET**：骨干网**64K**专线，国际线路**128K**连接**Sprint**
 - **Chinanet**：**64K + 64K**连接**Sprint**
- **1996年**，**ChinaGBN**：**64K**连接**Sprint**
- 逐渐形成三大运营商网络（电信、联通、移动）、中国教育网**CERNET**和中国科技网**CSTNET**
- 互联网应用发展迅速，**BAT**各领风骚



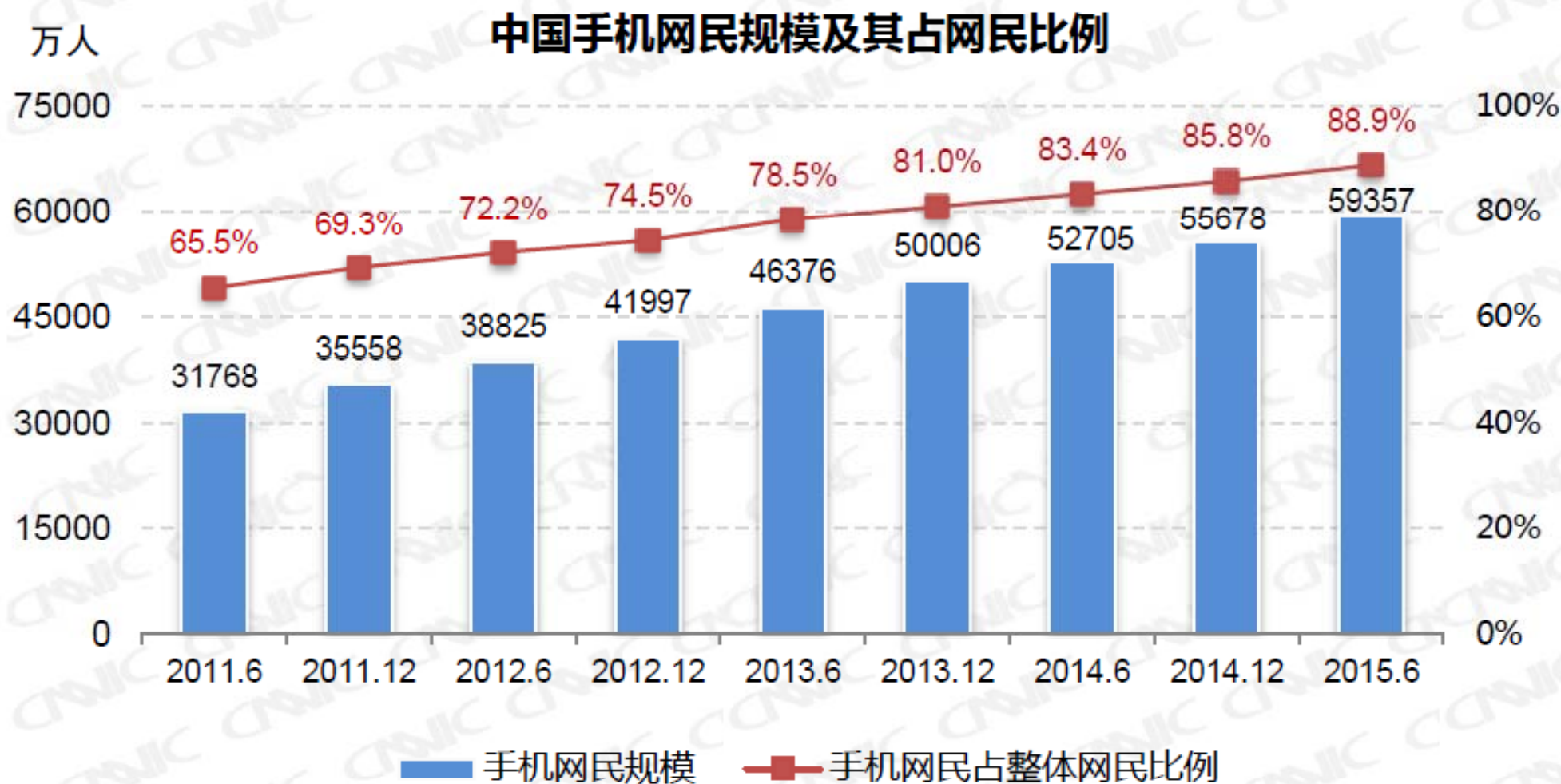
中国互联网用户

- **2008.6**，互联网用户数超过美国，成为全球第一
- **2015.6**，互联网用户近**6.68**亿，普及率**48.8%**





中国手机上网用户





中国互联网IPv4地址统计

中国IPv4地址数量

万个

40000

30000

20000

10000

0

2010.6 2010.12 2011.6 2011.12 2012.6 2012.12 2013.6 2013.12 2014.6 2014.12 2015.6

25045

27764

33163

33044

33047

33053

33062

33031

33041

33199

33554



中国互联网IPv6地址统计



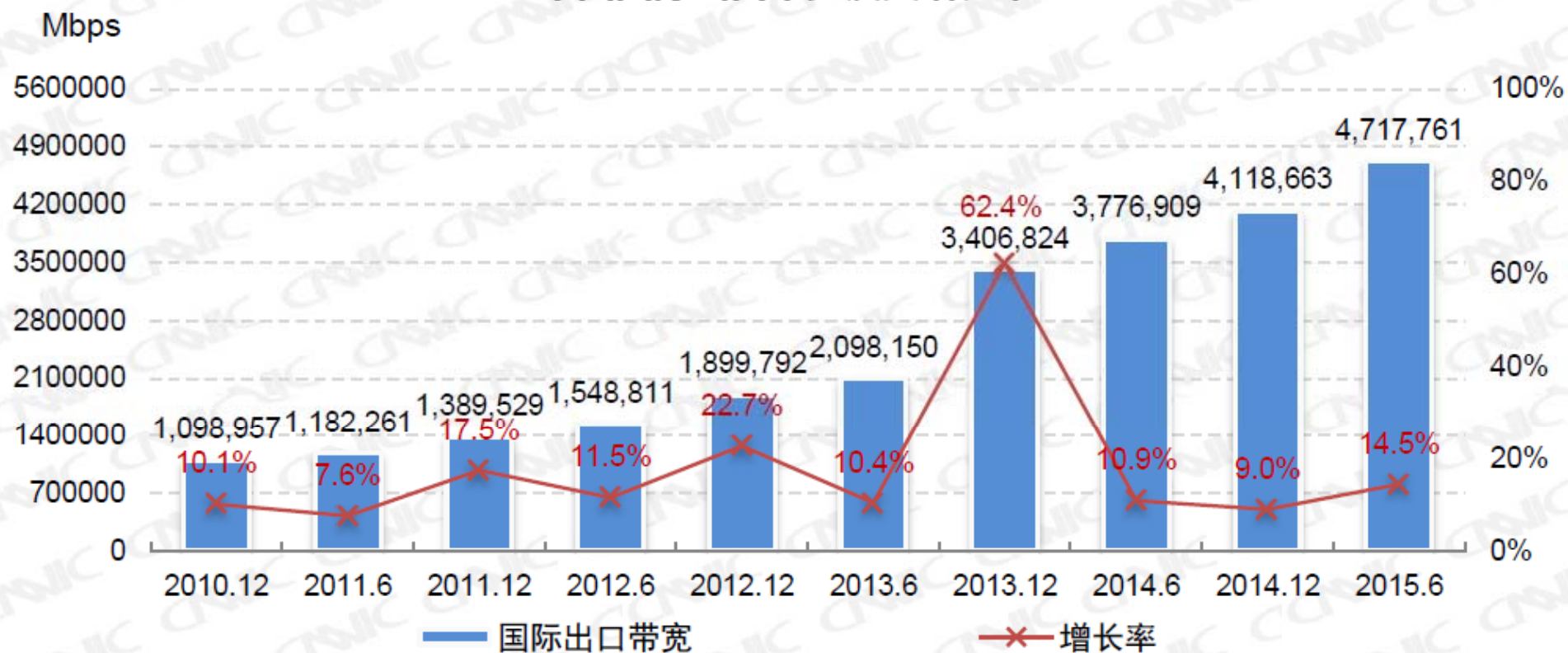
来源：CNIC 中国互联网络发展状况统计调查

2015.6



中国互联网国际出口带宽统计

中国国际出口带宽及其增长率





主要内容

- 计算机网络概述
- 互联网的发展和成功经验
- 互联网的核心思想：分组交换
- 国际高速计算机网络研究计划
- 中国高速计算机网络研究计划

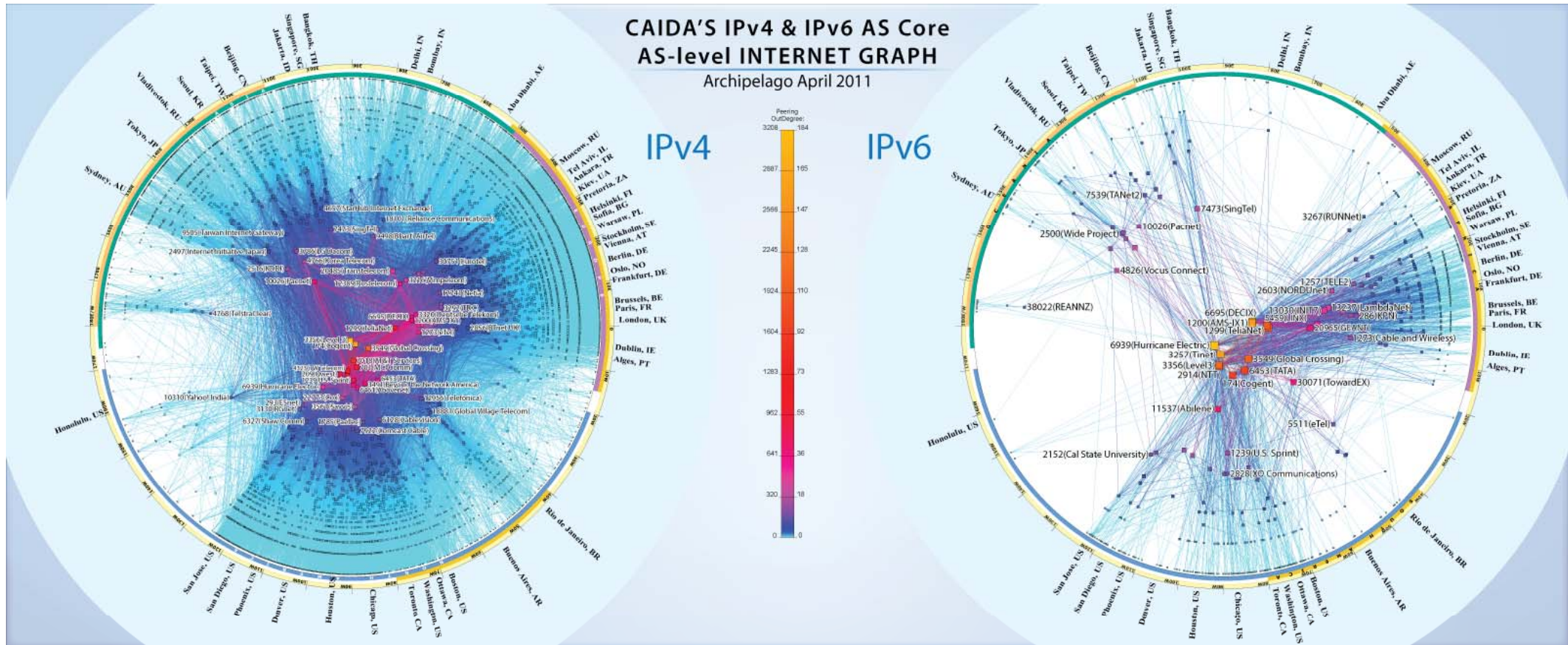


互联网 (Internet)

- 全球范围、通用、异构的公用计算机网络
- 开放的标准
 - **Internet Engineering Task Force (IETF)** 负责标准的制订、维护和协调
- 是其他类型网络的技术基础
 - 企业内部网 (**Intranet**)



IPv4/IPv6网络互联关系图



This visualization represents macroscopic snapshots of IPv4 and IPv6 Internet topology samples captured in 2011. The plotting method illustrates both the extensive geographical scope as well as rich interconnectivity of nodes participating in the global Internet routing system.

For the IPv4 map, CAIDA collected data from 54 monitors located in 29 countries on 6 continents. Coordinated by our active measurement infrastructure, Archipelago (Ark), the monitors probed paths toward 207 million /24 networks that cover 93.3% of the routable prefixes seen in the Route Views' Border Gateway Protocol (BGP) routing tables on 1 April 2011.

For the IPv6 map, CAIDA collected data from 16 IPv6-connected Ark monitors located in 12 countries on 4 continents. This subset of monitors probed paths toward 307,000 IPv6 prefixes which represent 21.9% of the globally routed IPv6 prefixes seen in Route Views BGP tables on 1 April 2011.

We aggregate this IP-level data to construct IPv4 and IPv6 Internet connectivity graphs at the Autonomous System (AS) level. Each AS approximately corresponds to an Internet Service Provider (ISP). We map each observed IP address to the AS responsible for routing traffic to it, i.e., to the origin (end-of-path) AS for the IP prefix representing the best match for this address in BGP routing tables collected from Route Views.

The position of each AS node is plotted in polar coordinates (radius, angle) calculated as follows:

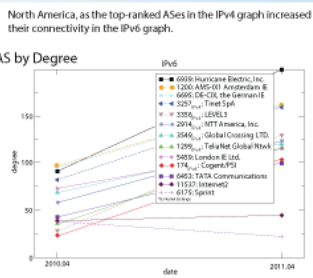
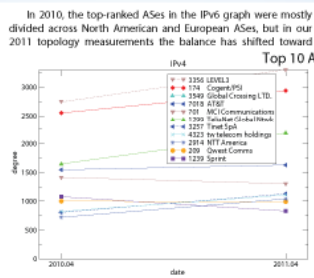
$$\text{radius} = 1 - \log \left(\frac{\text{outdegree}(\text{AS}) + 1}{\text{maximum outdegree} + 1} \right)$$
$$\text{angle} = \left(\frac{\text{length of the AS's BGP prefixes in netes}}{\text{length of the AS's BGP prefixes in netes}} \right)$$

Our IPv6 graph grew from 677 AS nodes in January 2010 to 1,183 nodes in April 2011 (74% growth). Over the same period, the number of ASes in our IPv4 graph grew 14.0% from 28,175 to 29,315.

Most ASes grew their observed peering degree in both our IPv4 and IPv6 graphs, although at different rates, which alters their relative degree-based rank over time. In the IPv4 graph Level 3 (AS 3356) remained dominant with the largest observed degree in both 2010 and 2011. The ASes with the second, third, and fourth largest degrees (Cogent's AS 719, Global Crossing's AS 3549, and AT&T's AS 7018) also maintained their previously observed degree rank. While most ASes increased their degree over this period, the fifth most highly connected AS, Sprint's AS

1299, had its degree decline, dropping its rank to tenth. Note that we rank each AS independently; some network providers have topology spread across multiple ASes. A more accurate topology-based ranking of providers would require a validated list of AS ownership by organization, a data set we are still working to compile.

The observed IPv6 AS ranking experienced greater change. Hurricane's AS 6939 moved up from third place in 2010 to first place in 2011, not surprising given Hurricane's aggressive peering policy. AS 1291 (Amsterdam IX, an exchange point) dropped from first to second place. Sprint's AS 11537 and Sprint's AS 6175 fell out of the top ten, and Sprint's AS 1299 and Cogent's AS 174 moved into the top ten. In the case of the two Sprint ASes, 57% of AS 6175's dropped neighbors were never seen as neighbors of AS 1299, while 64% of AS 1299's new neighbors were never seen as neighbors of AS 6175. The prominence of a European exchange point and a U.S. research network in the observable IPv6 topology is a symptom of the immaturity of the IPv6 infrastructure. As IPv6 deployment progresses we expect increasing congruity between IPv4 and IPv6 topologies, a trend reflected in the increasing presence of the same top-ranked ASes (by degree) in the two peering graphs. This trend also implies increasing congruity in the



ANALYSIS TEAM: Bradley Huffaker, Jo Chilly
SOFTWARE DEVELOPMENT: Young Hyun, Matthew Uske
POSTER DESIGN: Justin Chang

	Number of IP addresses	Number of IP links	Number of ASes	Number of AS links
IPv4	23,037,154	20,187,579	29,315	77,610
IPv6	14,683	33,927	1,183	2,738

ARK HOSTS: AARNET, Acore, AMS-IX, APAN, ARIN, ASTI, CAIDA, Canarie, CENIC, CNIST, Colorado State Univ., DePaul Univ., Evolve Telecom, FORTN, KordiaNet, HET-Net, Hurricane Electric, Indonesian IPv6 Task Force, Internet Systems Consortium, Iowa State Univ., Kantoetsusha, KIXNET, Level 3 Communications, NREN and NREN Research Council Canada, NCAR, NCC Chile, NCC Mexico, NORDUNET, Northwestern Univ., Public Univ. of Navarra, Purdue Univ., Regensburg, RRI, Simula Research Laboratory, StihlNet, THO, TWARON, UCAID, Univ. Leipzig, Univ. Politecnica de Catalunya, Univ. of Cambridge, Univ. of Hawaii, Univ. Melbourne, Univ. of Napoli, Univ. of Nevada at Reno, Univ. of Oregon, Univ. of Waikato, Univ. of Washington, Univ. of Zurich, US Army Research Lab

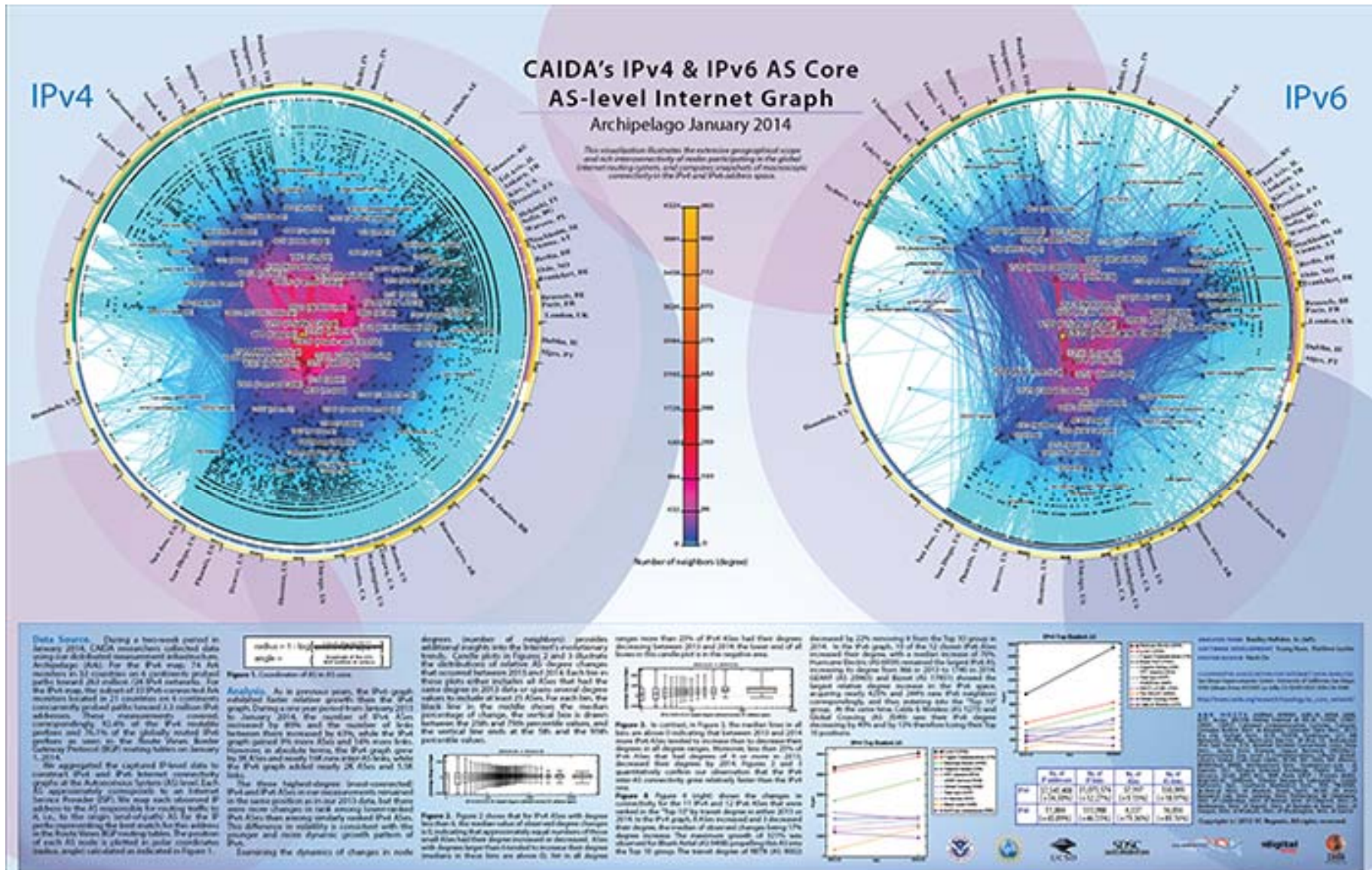
COOPERATIVE ASSOCIATION FOR INTERNET DATA ANALYSIS
San Diego Supercomputer Center, University of California, San Diego
5500 Gilman Drive, #0350, La Jolla, CA 92039-0350, 858-534-5888
http://www.caida.org/research/topology/as_core_network/

Copyright © 2012 UC Regents
All rights reserved.

Logos: UC San Diego, SDSC, digital, caida



IPv4/IPv6 网络互联关系图





互联网发展规模和趋势

- **Internet**的发展速度
 - 是历史上发展最快的一种技术
 - 以商业化后达到 **5000** 万用户为例
 - 电视用了**13**年，收音机用了**38**年，电话更长
 - **Internet** 从商业化后达到 **5000** 万用户用了**4** 年时间
- **Internet** 正在以超过摩尔定理的速度发展



网络时代的三大定律

摩尔定律:

CPU性能18个月翻番,10年100倍。

所有电子系统
(包括电子通信系统, 计算机)
都适用

光纤定律:

超摩尔定律, 骨干网带宽**9个月翻番, 10年**

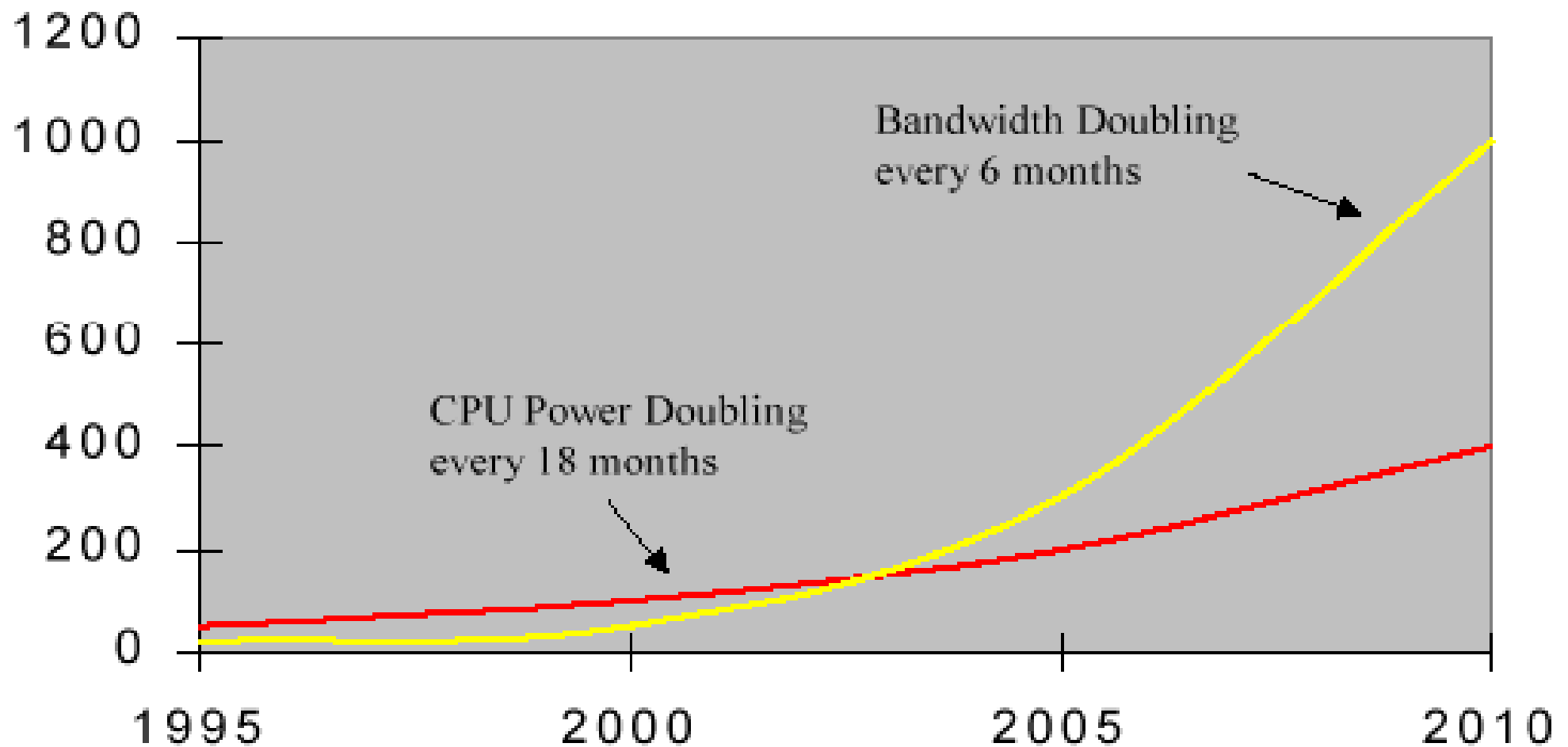
10000倍。带宽需求呈超高速增长的趋势

迈特卡菲定律:

联网定律,
网络价值随用户数平方成正比。
未联网设备增加**N**倍, 效率增加**N**倍。
联网设备增加**N**倍, 效率增加 **N^2** 倍

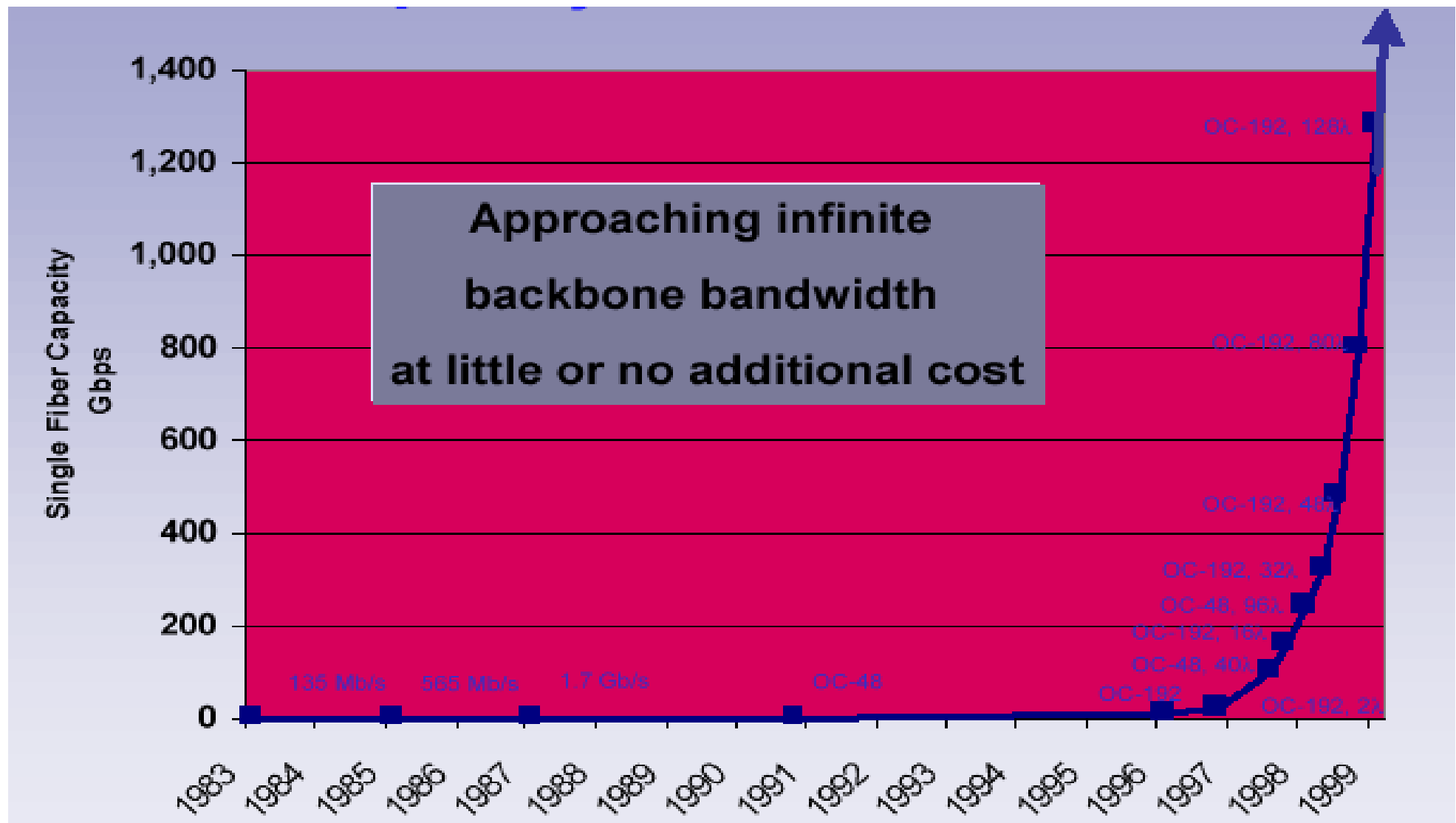


网络带宽与CPU性能





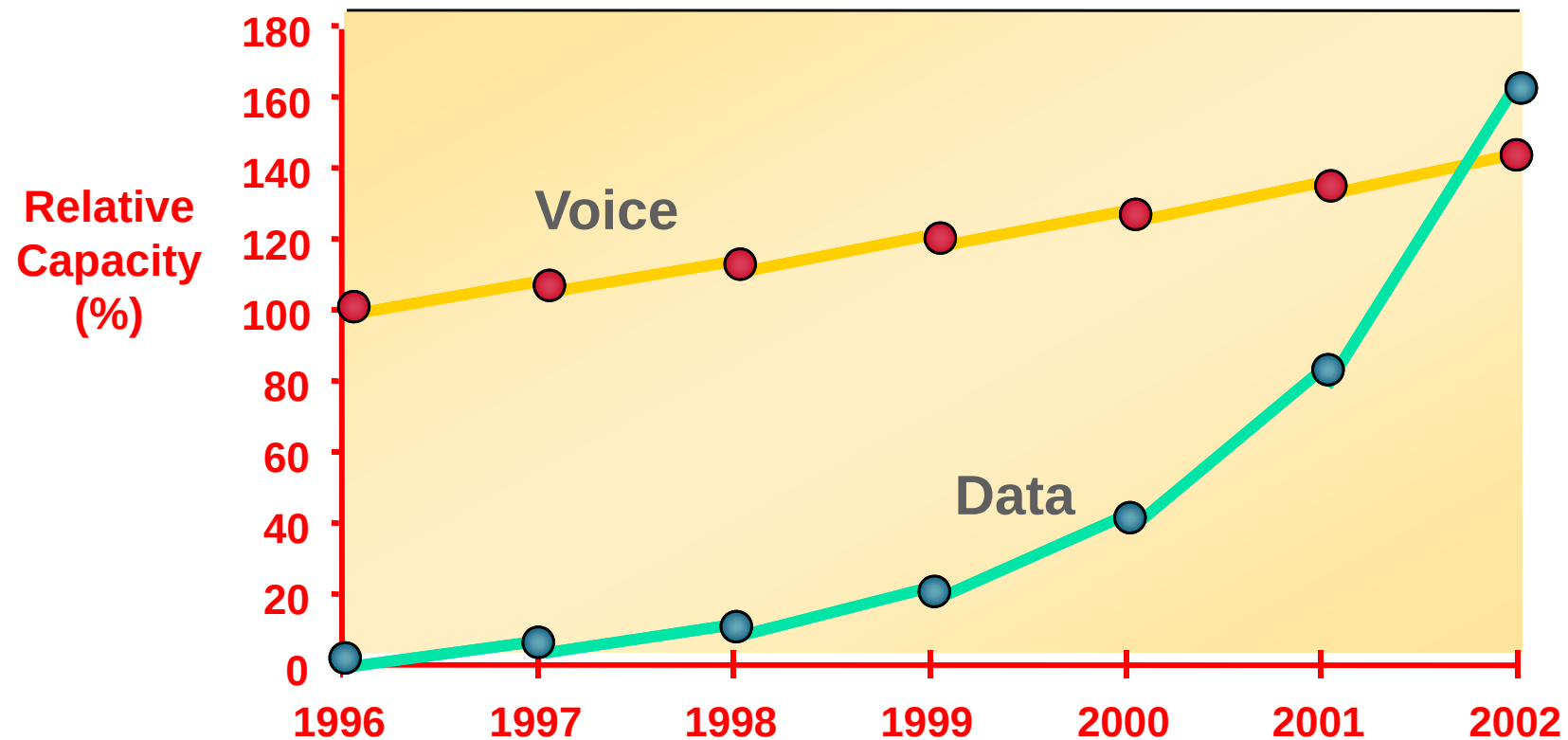
光纤容量





数据流量超过语音

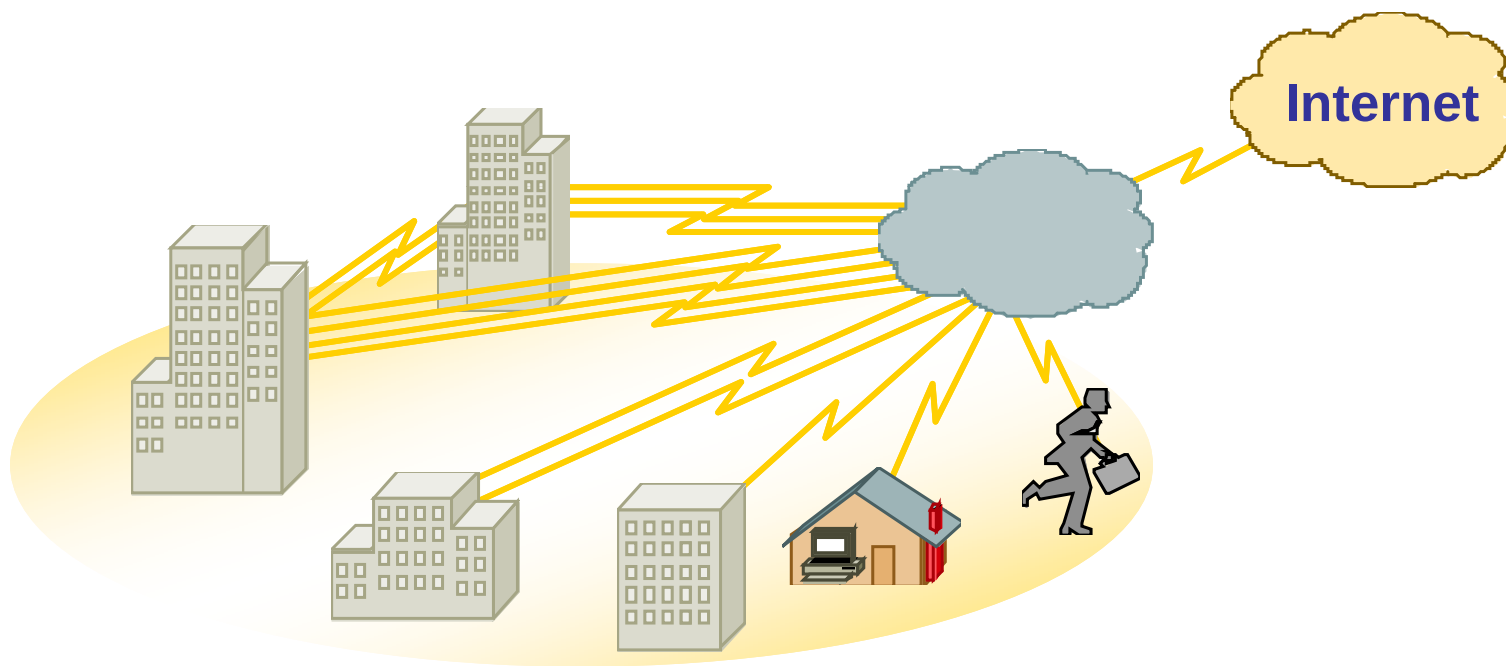
International data traffic already exceeds international voice from Australia and Scandinavia.



Source: MCI (Vint Cerf)



Data/Voice/Video 三网融合





新三网融合:

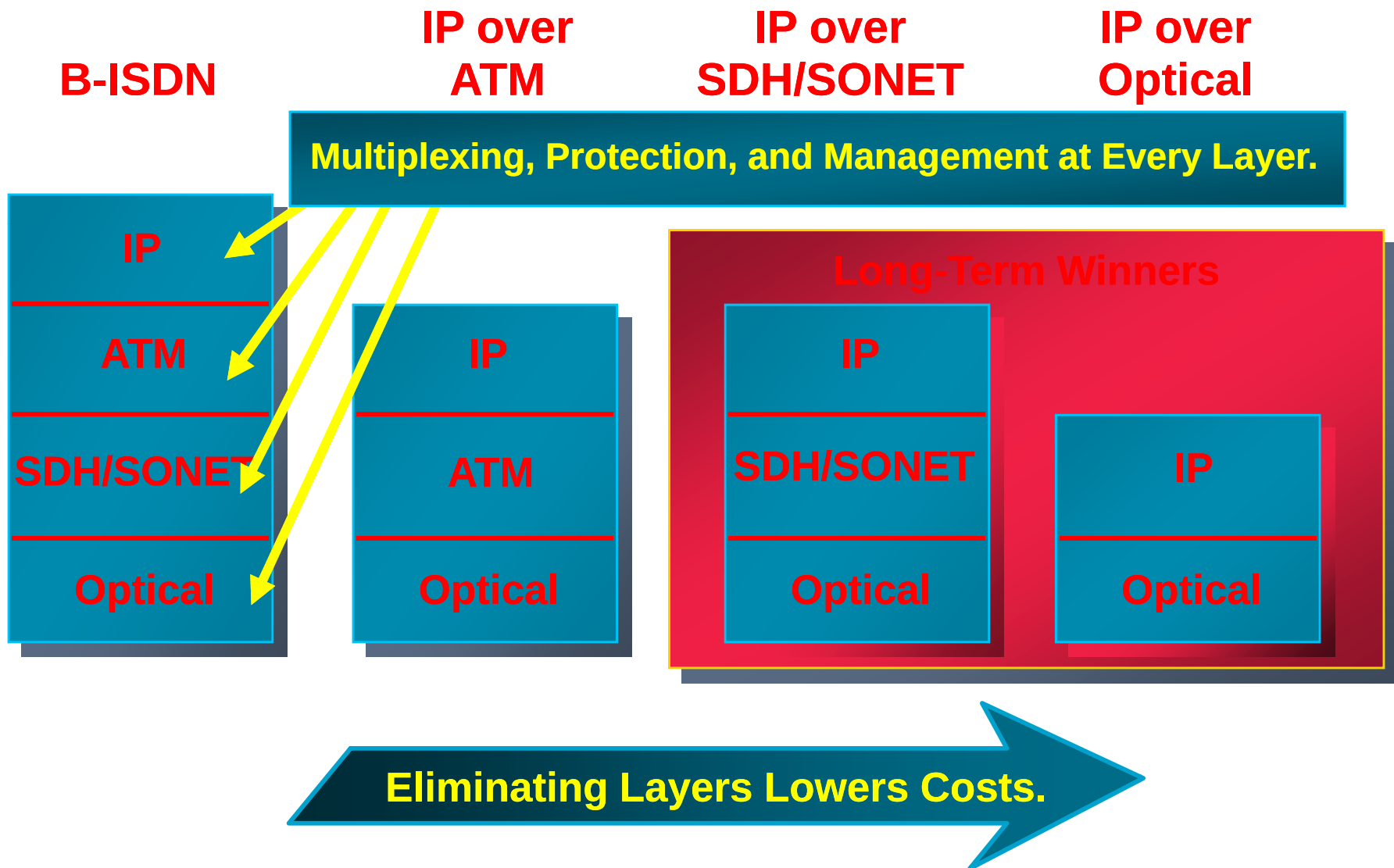
互联网、移动通信网和空间网络



无线移动网络和空间网络正通过互联网技术加速演进，
“新三网融合”的需求将不再局限于业务的互联互通，
而是发展成为体系结构层面的一体化深度融合。



网络体系结构的发展趋势





IP + Optical

- **DWDM transmission**
- **Mesh topology**
- **End-to-end provisioning**



- **Wavelength switching granularity**
 - **Open protocols**



无线技术

Fixed

Broadband

MMDS
2.5GHz

ETSI
3.5 GHz

UNII
5.7GHz

Mobile

Cellular

2G

GSM/GPRS
CDMA/PDSN

3G

UMTS
CDMA 2000

Campus

Wireless LAN

802.11b



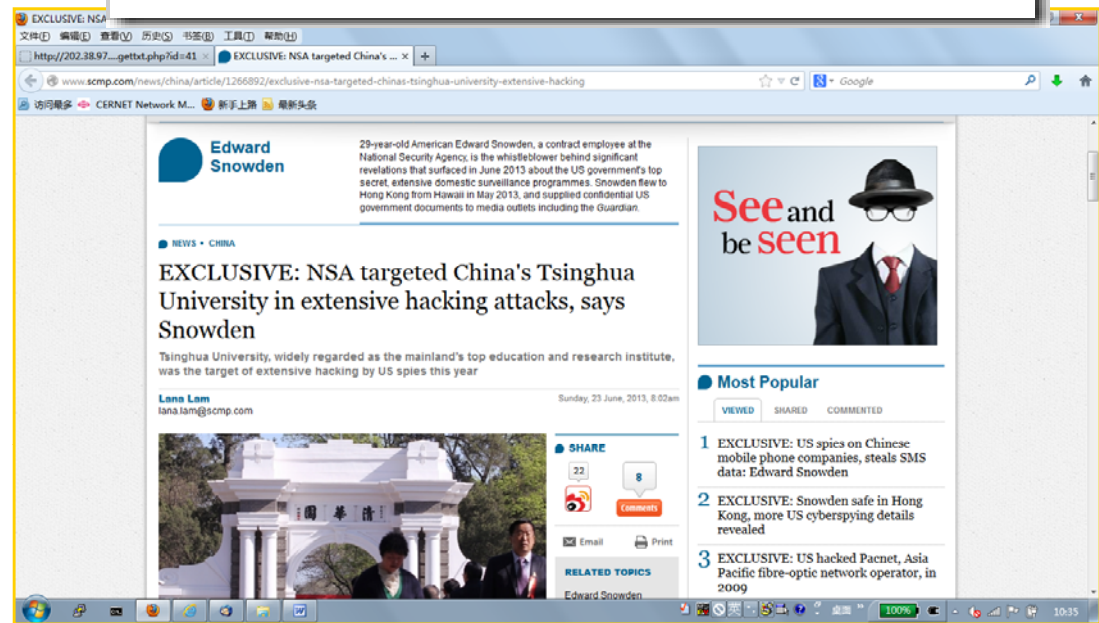
下一代互联网与IPv6

- 下一代互联网试验网络规模不断扩大，全球**IPv6**下一代互联网主干网正在形成
- **2003年1月DoD**宣布全面向**IPv6**过渡，**2008年**完成
- **2005年7月**美国政府决定**2008年6月**过渡到**IPv6**
- **IPv6**网络设备和应用软件不断推出
 - **IPv6**网络设备：**Juniper, Cisco...**
 - **IPv6**应用软件：**Microsoft, SUN...**
- 移动通信和智能电器对**IPv6**的需求越来越迫切
- 下一代互联网基础理论研究得到认可



互联网带来的社会问题

- 病毒 (**Virus**)
- 木马 (**Trojan**)
- 垃圾邮件 (**Spam**)
- 隐私 (**Privacy**)
- 知识产权
(**Intellectual Property**)
- ...
- 网络实名制?





互联网标准化组织

- **Internet Engineering Task Force (IETF)** : **IETF**负责**Internet**协议的研发和改进。**IETF**被分为很多个工作组（**working groups**），他们提交的文档称为**RFC**（**Request For Comments**）
- **IRTF (Internet Research Task Force)** : **IRTF**由一些专注于某个领域长期发展的研究小组组成
- **Internet Architecture Board (IAB)** : **IAB**负责定义**Internet**的整体框架，为**IETF**提供大方向上的指导
- **The Internet Engineering Steering Group (IESG)** : **IESG**在技术方面管理**IETF**的活动，负责**Internet**标准的制定过程



IETF的历史

- **1986年1月16日**，在美国加州圣地亚哥，召开第**1**次会议
- **2010年11月**，在北京，召开第**79**届**IETF**会议
- 现在每年**3**次会议
- **Lixia Zhang, professor at UCLA:**” At the first IETF I was a graduate student. I felt that I had so much to contribute. I had lots of great ideas. As years go by, I have better appreciation of how much I can learn from this community. Each time I come to an IETF Meeting, I learn from others. Now, I feel how little I know. As a graduate student, I felt how much I know.”
-- from IETF Journal, Volume 2, Issue 1, 2006



互联网标准的制定过程

- 所有的标准以**RFC**的形式发布出来，可以从www.ietf.org免费获得。但不是所有的**RFC**都是**Internet**标准
- 标准形成的一般步骤是：
 - **Internet Drafts** (individual draft, WG draft)
 - **RFCs**
 - **Proposed Standard**
 - **Draft Standard** (需要两个可以工作的实现)
 - **Internet Standard** (由**IAB**发布)
- **David Clark, MIT, 1992: "We reject: kings, presidents, and voting. We believe in: rough consensus and running code."**



IETF 文化

VIEWS OF THE FUTURE

The last force on us -- us

The standards elephant of yesterday -- OSI.

The standards elephant of today -- it's right here.

As the Internet and its community grows, how do we manage the process of change and growth?

- Open process -- let all voices be heard.
- Closed process -- make progress.
- Quick process -- keep up with reality.
- Slow process -- leave time to think.
- Market driven process -- the future is commercial.
- Scaling driven process -- the future is the Internet.

We reject: kings, presidents and voting.

We believe in: rough consensus and running code.



VIEWS OF THE FUTURE

A Cloudy Crystal Ball -- Visions of the Future

David D. Clark
M.I.T. Laboratory for Computer Science
IETF, July 1992

Alternate title: Apocalypse Now

DDO 7/16/92 19:39 COPYRIGHT © David Clark 1992

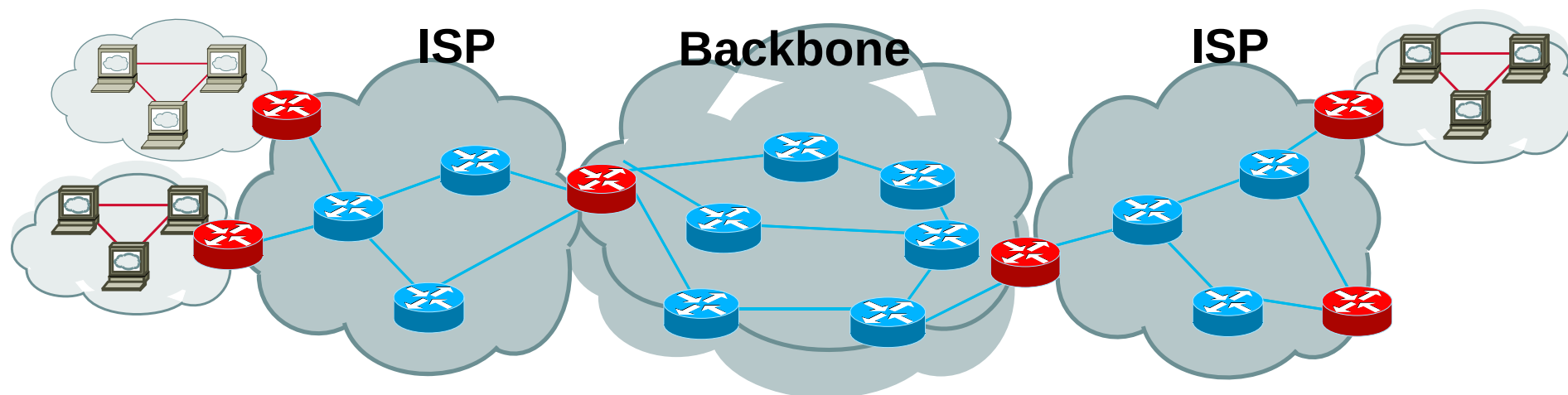


互联网提供的服务

- 计算资源的共享访问: **telnet (1970's)**
- 数据和文件的共享访问: **FTP, NFS (1980's)**
- 人们互相通讯的媒介:
 - **Email (1980's)**, 网上聊天室, 即时消息 (**1990's**)
- 信息分发的媒介
 - **USENET (1980's)**, **WWW (1990's)**, 语音和视频 (**1990's**)
 - **P2P (2000's)**, **MSN、QQ、博客...**
- 学习的媒介: **Google, Baidu, Mooc**
- 社交的媒介: **Facebook**, 微博, 微信
- 网络空间 (**Cyberspace**)
- ...



互联网网络结构



■ 用户接入

- Modem
- DSL
- Cable modem
- Satellite
- LAN

■ ISP主干网

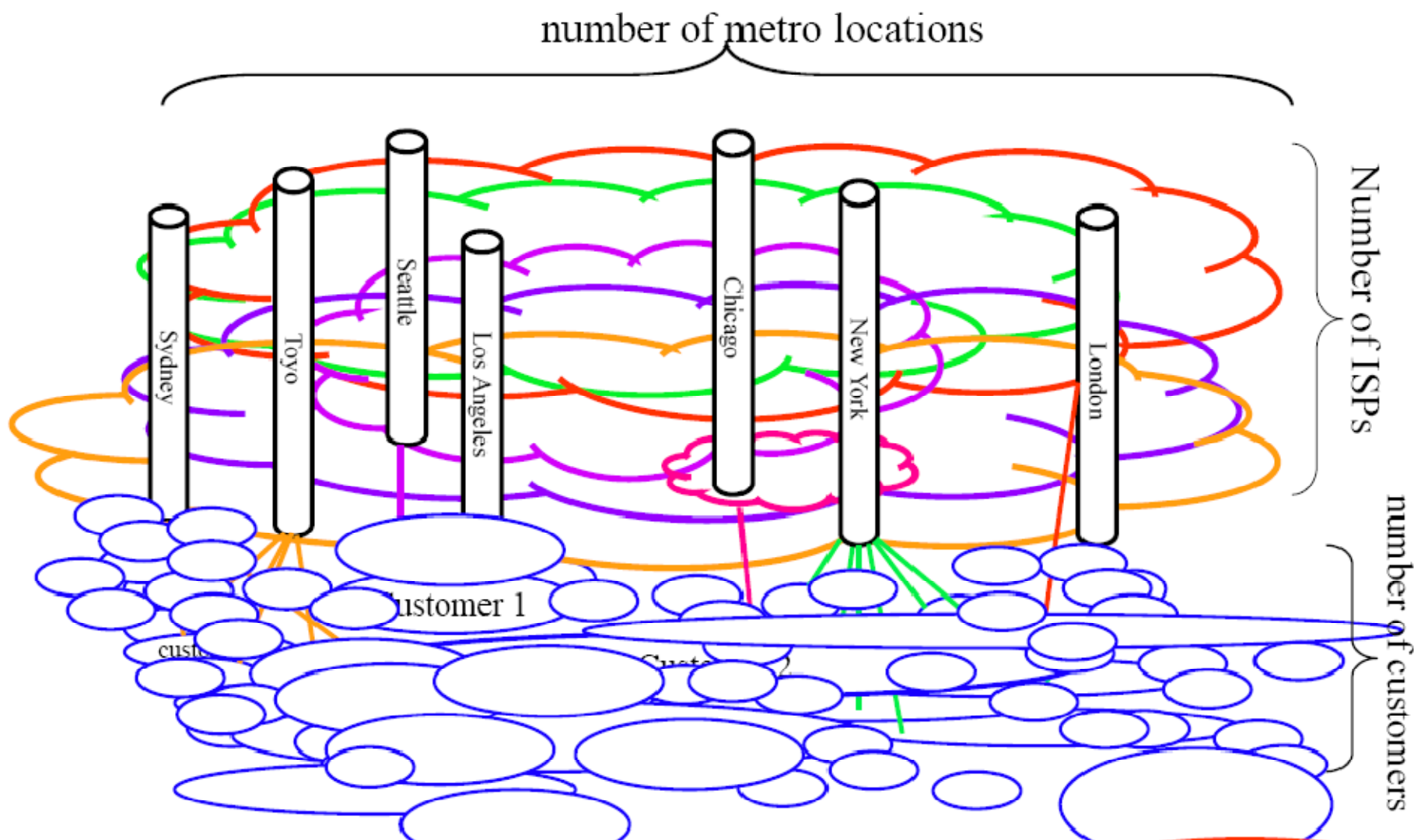
- OC-3
- OC-12
- OC-48
- OC-192

■ 校园网

- Ethernet
- WIFI



实际网络结构要复杂得多





互联网的主要技术特点

- 分层的分布式结构
- 无连接的分组交换技术
- 网络互连协议 **IP** (**IP over everything**)
- 路由器加专线技术
- 可扩展的路由技术
- 端到端的网络连接技术
- 层次结构的域名、网络管理技术
- 通用的应用技术



互联网的成功经验

- 有远见的政府不断支持：**1969—**
- 有风险的企业参与和投入
 - **NSFNET: MCI、IBM**
 - **vBNS: MCI**
 - **Abilene: Qwest, CISCO**
- 联合协作的开放式研究：**IETF/RFC**
- 教育和科研的示范网络为起点
 - 具有实验物理学的研究特点
 - **ARPAnet、NSFNET、ANS、vBNS**
- 简单实用的技术路线：**TCP/IP**



Commercialization

Privatization

21st Century
Networking

Interoperable
High Performance
Research & Education
Networks

SprintLink
InternetMCI

US Govt
Networks
ANS

Active
Nets
wireless
WDM

gigabit
testbeds

ARPAnet

NSFNET

Internet2, Abilene, vBNS
Advanced US Govt Networks

Quality of Service
(QoS)

Research and
Development

Partnerships



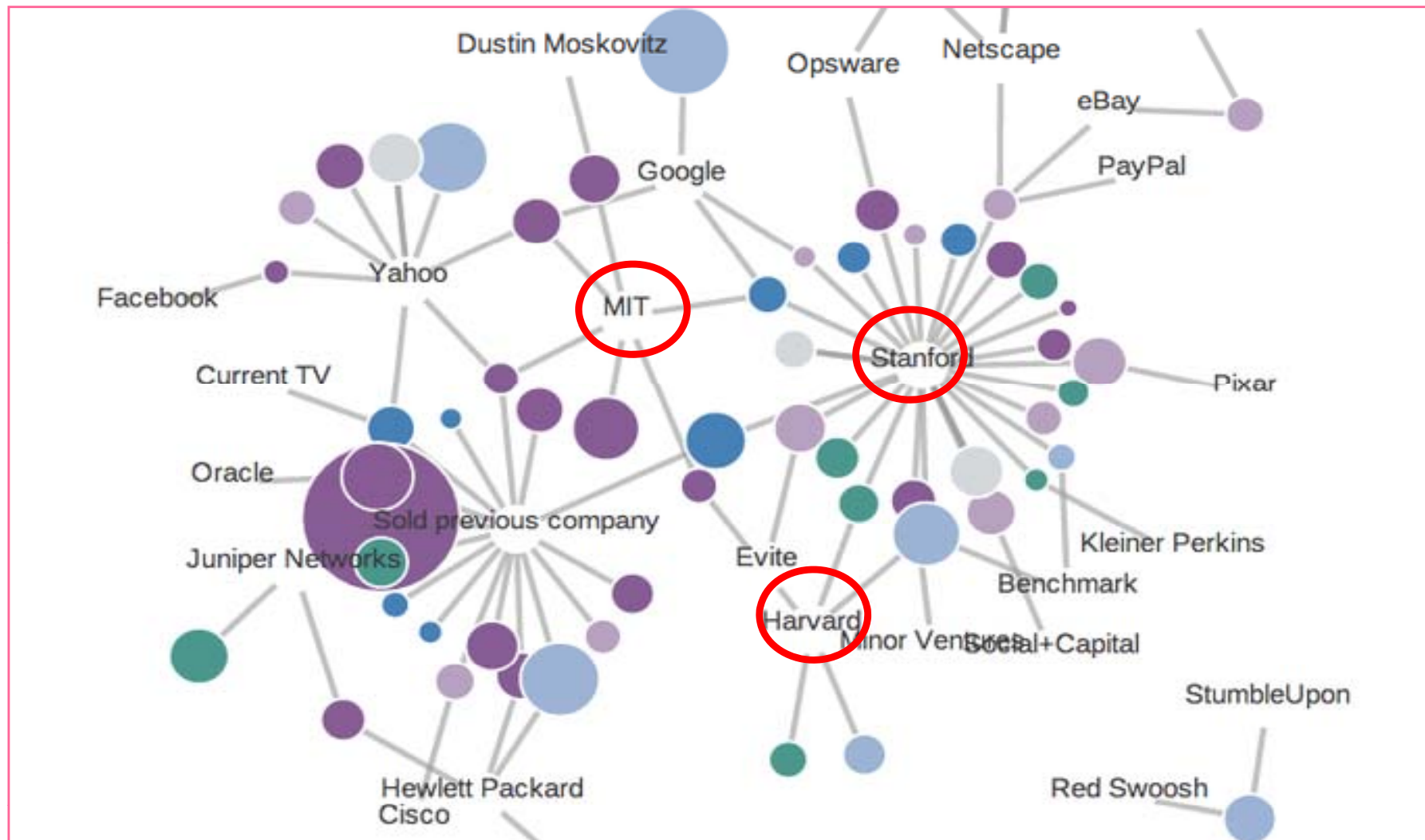
互联网的发展在于人才

互联网企业创始人创业时年龄

- facebook:
mark zuckerberg 19岁
- 微软: bill gates 20岁
- 微软: paul allen 22岁
- 苹果: steve jobs 21岁
- 苹果: steve wozniak 25岁
- google: sergey brin 25岁
- google: larry pages 25岁
- 雅虎: 杨致远 26岁
- 雅虎: david filo 28岁
- skype: janus friis 26岁
- skype: niklas zennstrom 36岁
- youtube: chad hurley 27岁
- myspace: tom anderson 27岁
- myspace: chris dewolfe 36岁
- ebay: pierre omiydar 28岁
- amazon: jeff bezos 30岁
- paypal: max levchin 23岁
- paypal: peter thiel 31岁



英雄也问出处 (美)





英雄也问出处 (中)

1.	张朝阳	清华-MIT	搜狐
2.	王小川	清华	搜狗
3.	史立荣	清华大学	中兴
4.	李彦宏	北大-布法罗	百度
5.	俞敏洪	北大	新东方
6.	杨元庆	上海交通大学	联想
7.	周鸿祎	西交	奇虎
8.	陈天桥	复旦	盛大
9.	曹国伟	复旦	新浪
10.	丁磊	电子科技大	网易
11.	雷军	武大	小米
12.	柳传志	西安电子科技大学	联想
13.	刘强东	人大	京东
14.	马化腾	深大	腾讯
15.	马云	杭州师范	阿里
16.	任正非	重庆建筑工程学院	华为
17.	古永锵	新南威尔士	优酷

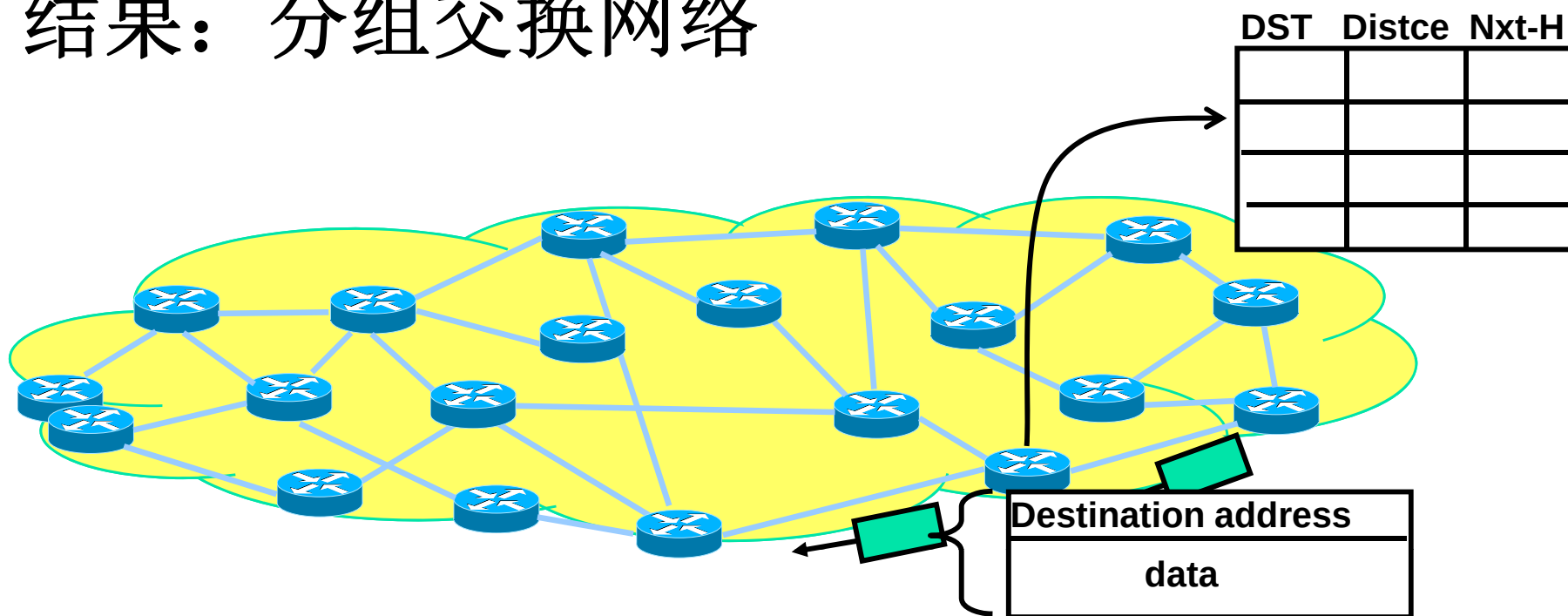


主要内容

- 计算机网络概述
- 互联网的发展和成功经验
- 互联网的核心思想：分组交换
- 国际高速计算机网络研究计划
- 中国高速计算机网络研究计划

Paul Baran在分布式通信方面的贡献

- 时间: **1960 - 1964**
- 目标: 建造一套健壮的通信系统可以承受核攻击
- 结果: 分组交换网络





Baran的设计细节

- 自适应系统：热土豆（**hot potato**）路由策略
 - 如果不知道正确的路由，就把分组转发给所有的邻居节点
 - 通过观察路过的分组更新路由表，旧的路由表项会过期而被删除
 - 尽可能快的转发分组
 - 不需要每次都沿着最短路径转发
- 学习并适应变化的环境



Baran的设计细节（续）

■ 分组发送

- 每个交换节点根据自己的路由表判断如何转发分组
- 每个分组的转发都独立于其他分组
- 交换节点不保存端节点的状态
 - 可扩展性好
 - 不是最有效的网络
 - 发送不是完美的

→ 端节点必须能容忍发送错误并从中恢复



Baran的设计细节（续）

■ 分布式系统

- 所有交换结点是平等的
 - 避免了单一节点失效问题
- 部件可以失效，但系统不会失效
- 系统的健壮性来自于
 - 足够的物理（硬件）冗余
 - 适应性路由

■ 模拟实验表明

- **“extremely survivable networks can be built using a moderately low redundancy of connectivity level”—Paul Baran, 1964**



两种实现可靠系统的思路

■ 电话系统

- 笨终端，聪明的网络
- 确保每个网络部件都是可靠的
 - 系统可靠性=部件可靠性
 - 通过局部冗余实现部件的高可靠性
 - 期望每个部件都能正常工作，部件失败的可能性很低
- 需要人工配置的，高度控制的网络

■ Baran的系统

- 建立在简单的、不可靠部件上的可靠系统
- 自适应的系统
- 聪明的终端，可以修正传输错误

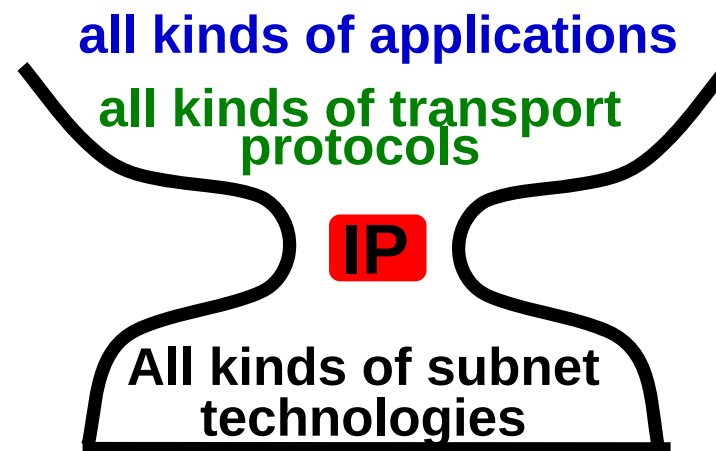


Baran设计思想的一种实现: Internet

IP's view of the world

- 连接异构的子网
- 提供两种基本功能
 - 全球唯一的地址
 - 分组通过动态路由从源节点发送到目的节点

特性: **simple, flexible, scalable, and robust**





分组交换的特点：简单性

- 每个分组携带各自的地址信息
- 一个路由表可以为所有的流量服务
- 可以适应爆炸性的增长
 - 越简单越不容易出错
 - 越简单越容易增长
 - 对基本网络的要求少



分组交换的特点：灵活性

- 可以在各种底层物理网络上运行
 - **IP over everything**
 - **Ethernet, FDDI, Frame Relay, ATM, SONET, DWDM ...**
- 可以支持各种类型应用
 - **Everything over IP**
 - **telnet, ftp, email, 多媒体, web, 电子商务...**



分组交换的特点：可扩展性

- 可扩展的系统必须能应对
 - 端系统的增加
 - 流量的增加
 - 网络规模的增长
 - 大的路由表
 - 路由频繁的变化
- **With IP, “the network knows nothing about individual end applications; end applications know nothing about network internals”—Van Jacobson**



分组交换的特点：健壮性

- 动态路由具有自适应的特性
 - 动态路由和分组转发相辅相成
 - 周期性路由更新
 - 默认：现有的部件会失效，会有新的部件加入，认为变化是正常的
- 牺牲一定的带宽利用率，提高健壮性
 - 分组头开销
 - 路由更新开销



今天的互联网

■ 与**40**年前相比

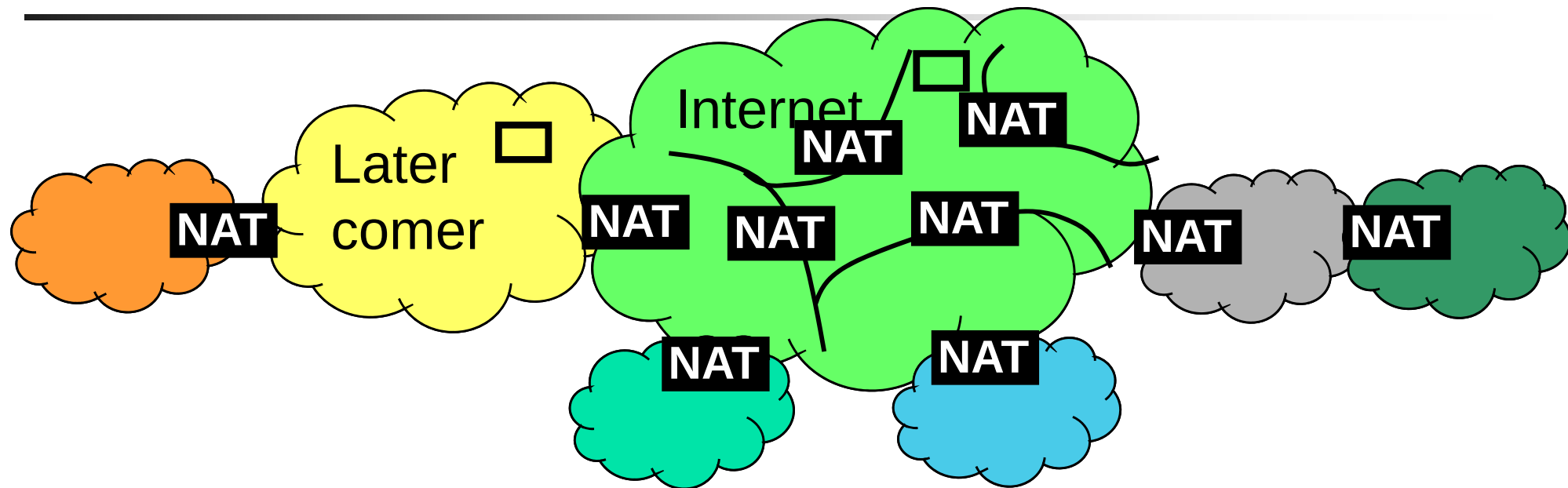
- 规模更大
- 用户更多
- 功能更多，更有价值
- 但是，健壮性、适应性和互联程度都下降了

■ **IPv4**地址空间耗尽

- 越来越多的用户通过**NAT**访问网络



NAT: a feature or a problem?



- **NAT**缓解了地址耗尽的问题，并且增强了安全性和控制性
- **NAT**打破了许多协议和应用基于**IP**地址全球唯一的假设
- 难以很好地支持**peer to peer**的应用
- 端到端的分组传输路径变成多个**NAT**域的级联，**相当于虚电路**



为什么需要IPv6?

- **NAT**导致随着时间推移，**Internet**原有结构遭到破坏
- 为了恢复**Internet**原有结构，必须过渡到**IPv6**
 - 巨大的地址空间：43亿 vs. 3.4×10^{38}
 - 实现无处不在的网络，网络规模可无限扩展
 - 连接所有可能的装置和设备
 - 改善了路由性能
 - 路由聚合减少了路由表的表项
 - 简化的IP头减少了路由器的处理负载
 - 增强了网络安全
 - 强制的IPsec支持安全的IPv6设备
 - 支持大规模移动IP设备



小结

- 基于分组交换的**IP**结构使网络的持续增长成为可能
- **Internet**需要过渡到**IPv6**以阻止目前网络结构的破坏和保证将来的增长
- 过渡到**IPv6**将会是困难和昂贵的，需要一个过程



主要内容

- 计算机网络概述
- 互联网的发展和成功经验
- 互联网的核心思想：分组交换
- 国际高速计算机网络研究计划
- 中国高速计算机网络研究计划



国际高速信息网络技术研究计划

- **1992**年美国政府的“国家信息基础设施 **NII**”
- **1993**年西方七国的“全球信息基础设施 **GII**”
- **1996**年, **NGI** 和 **vBNS**
- **Internet 2** 和 **Abilene**
- **CANARIE** 和 **CA* net3**
- 欧盟下一代学术主干网**GEANT**
- **APAN**
- **STAR TAP**
- 全球**IPv6**下一代互联网主干网**GTRN**正在形成
- 未来互联网研究计划: **FIND, GENI, FIRE**





NGI: 美国下一代互联网研究计划

- **1996.10**, 美国总统和副总统宣布启动**NGI**
- **NGI** 的三个主要目标:
 - 先进网络技术的实验研究
 - 下一代网络测试床
 - 革命性的应用



NGI 目标1: 先进网络技术的实验研究

■ 网络工程

- 规划和模拟, 监视, 集成, 数据传递
- 网络管理, 动态和自适应的网络

■ 服务质量 (端到端)

- 服务质量体系结构, 准入控制, 计费 and 优先权
- 可观察和控制的 **API**

■ 安全

- 用户用安全和公平的方法获取网络资源
- 优越的网络管理, 网络内部的监视
- 移动 / 远程访问
- 公钥基础设施



NGI目标2：下一代网络测试床

- 开发下一代网络测试床，用比当时**Internet**快**100**倍以上的速度连接至少**100**个大学和国家研究实验室
 - 以**1997年1.54Mbps**计，**10**个连接点速度达到比当时**Internet**快**1000**倍
 - 端到端连接速度达到**100Mbps—1Gbps**
- 主要策略：协调建立一个高性能的协作网络
 - **vBNS, ESnet, NREN**
- 评价标准：连接点的数量，端到端的性能
 - 支持目标**1**的研究，支持目标**3**的应用



NGI目标3：革命性的网络应用

- 开发当时互联网没有，对国家重要的网络应用
 - 健康保健：远程医疗、紧急医疗响应支持
 - 教育：远程教育、数字图书馆
 - 科学研究：能源、地理系统、气象、生物
 - 国家安全：高性能全球通信、先进的信息传播
 - 环境：监测、预测、警告、响应
 - 政府：传递政府服务和信息给公民和企业
 - 突发事件：灾难响应、危机管理
 - 设计和制造：制造工程
- 主要策略：重点研究基础性应用
 - 分布式计算应用、协同性应用

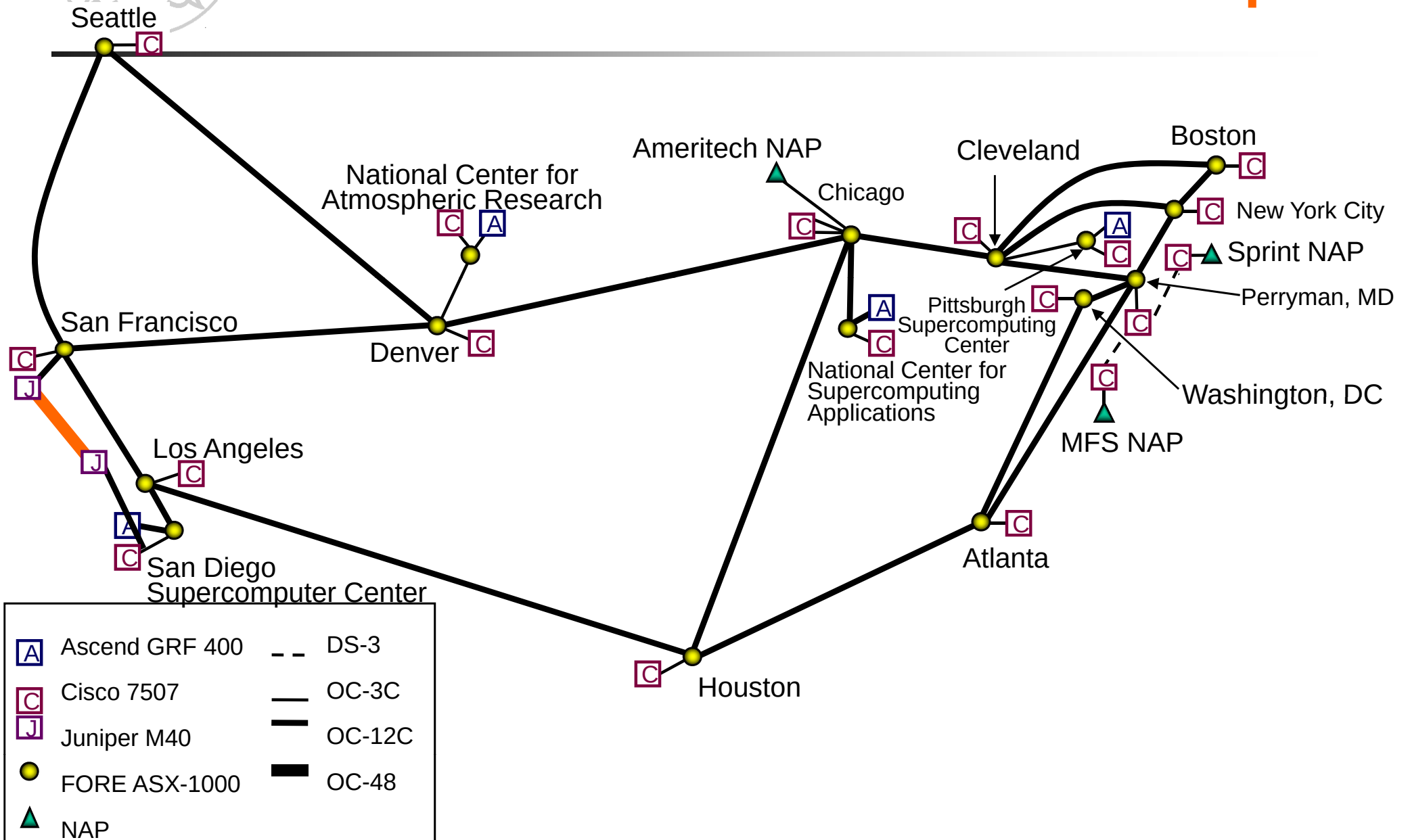


vBNS

- **1995年4月批准**
- 由**NSF**建立，目标是为美国教育科研机构提供高性能的网络资源，并促进网络技术的进步
- **NSF**提供：基金、管理
- **MCI**提供：带宽，设备和工程支持



vBNS Backbone Network Map





Internet 2

- <http://www.internet2.edu>
- **UCAID**（**120**多个大学会员）的一项研究计划
University Corporation for Advanced Internet Development
- 形成大学试验网，开发下一代 **Internet** 技术和应用
 - **IPv6, Multicasting, QOS**
 - 以竞争方式得到 **NGI** 计划的经费支持
- **NGI**是政府计划，**Internet 2** 是大学合作计划
 - 相互补充，相互依靠



Abilene

- **1998年4月14日**美国副总统 **Gore**启动该项目
- 当时世界上最先进的科研教育网络，为参加**Internet2**的大学提供先进的**IP**骨干网络
 - 支持先进的科研项目
 - 整合先进的网络服务
- **UCAID**负责研发
 - **Qwest, Nortel**和**Cisco**等大公司加盟
- **G**比特汇接点（**gigaPoPs**）之间采用**2.5 Gbps (OC48)**的连接，并增加至 **9.6 Gbps (OC192)**。
- 一般连接采用**622 Mbps (OC12)** 或**155 Mbps (OC3)**
- 采用**IP over SONET** 技术

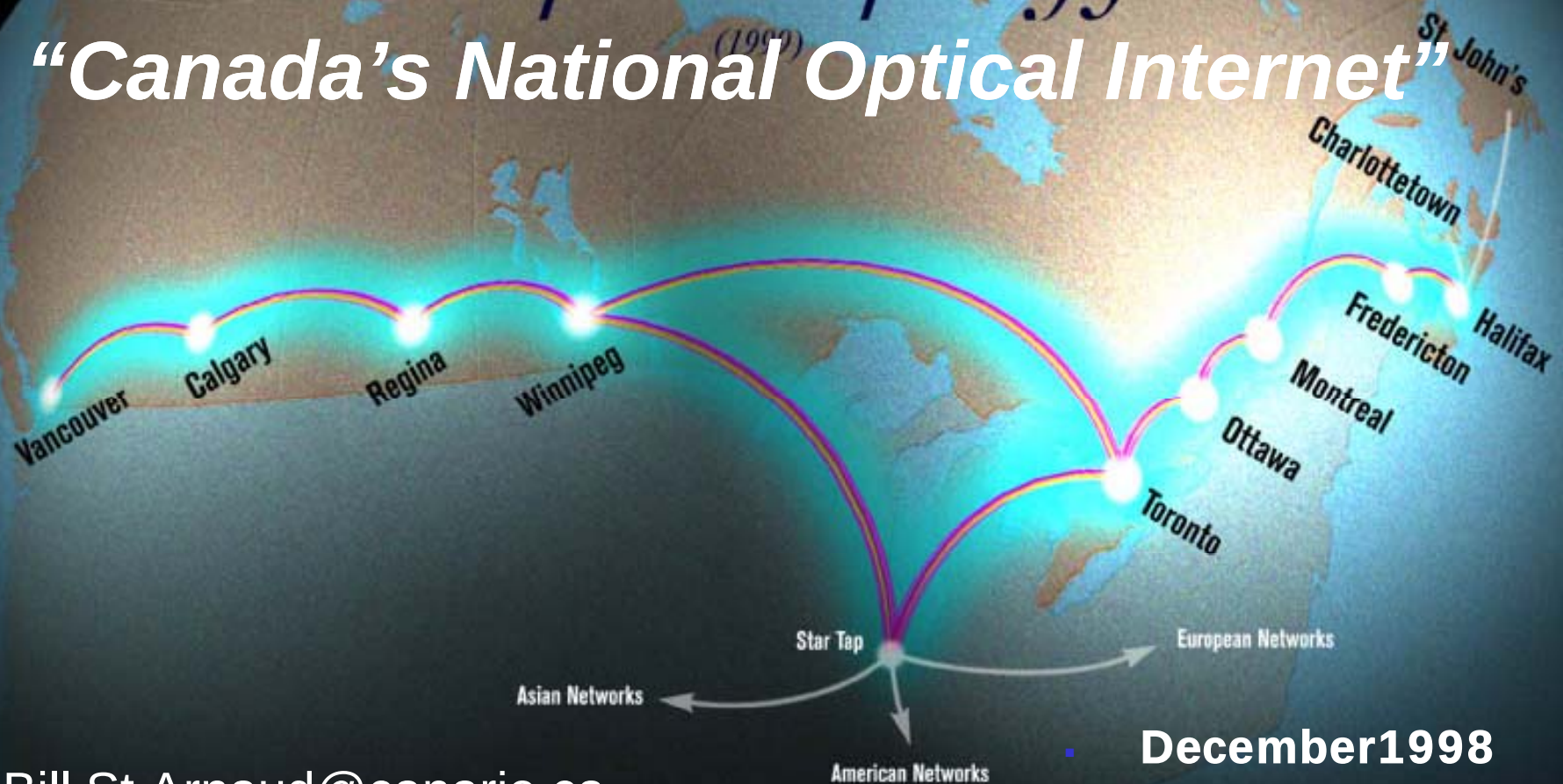
The Abilene Network





Proposed topology

“Canada’s National Optical Internet”



Bill.St.Arnaud@canarie.ca
<http://Tweetie.canarie.ca/~bstarn>
Tel: +1.613.785.0426

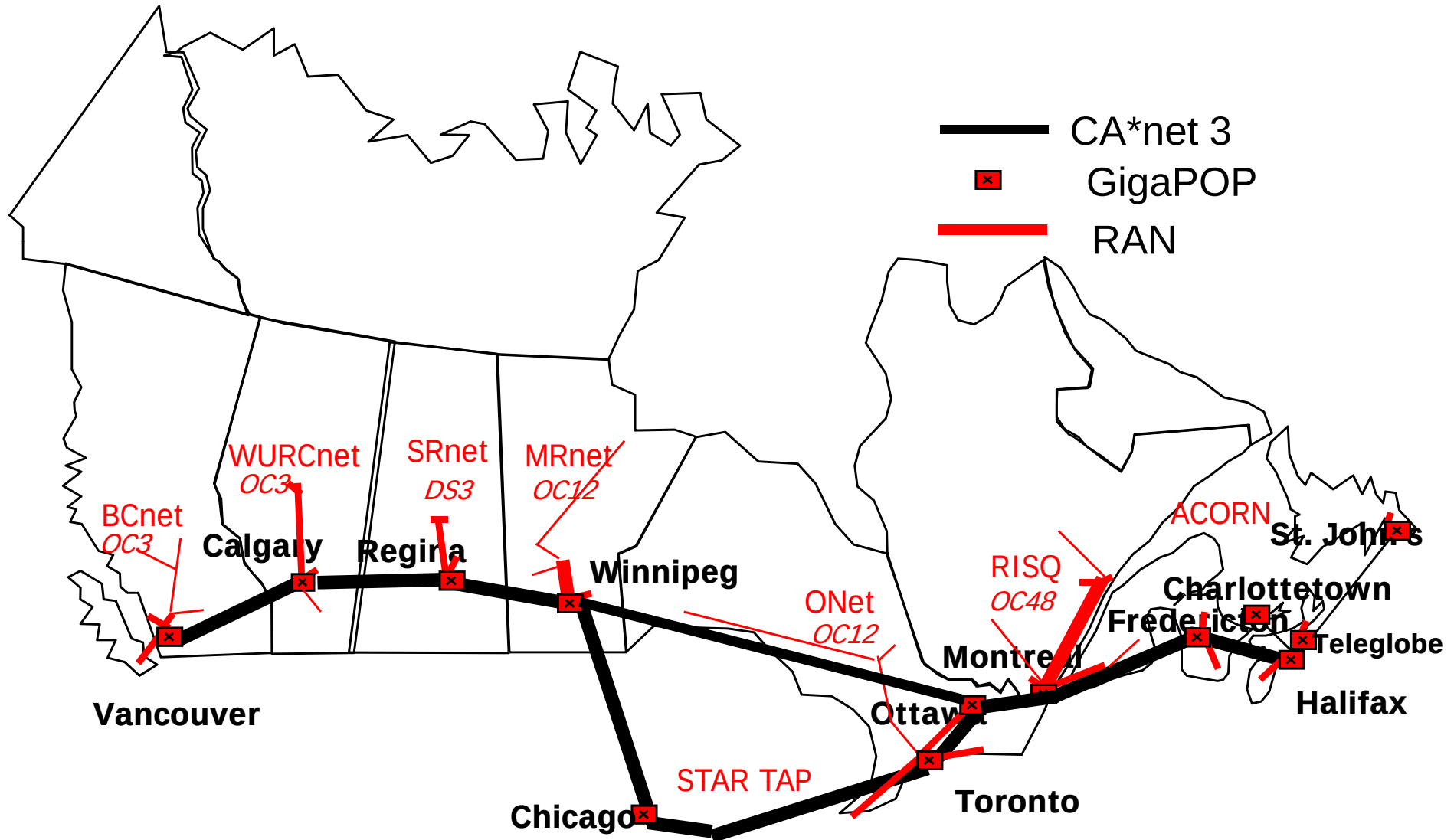
December 1998

<http://www.canarie.ca>

<http://www.canet3.net>



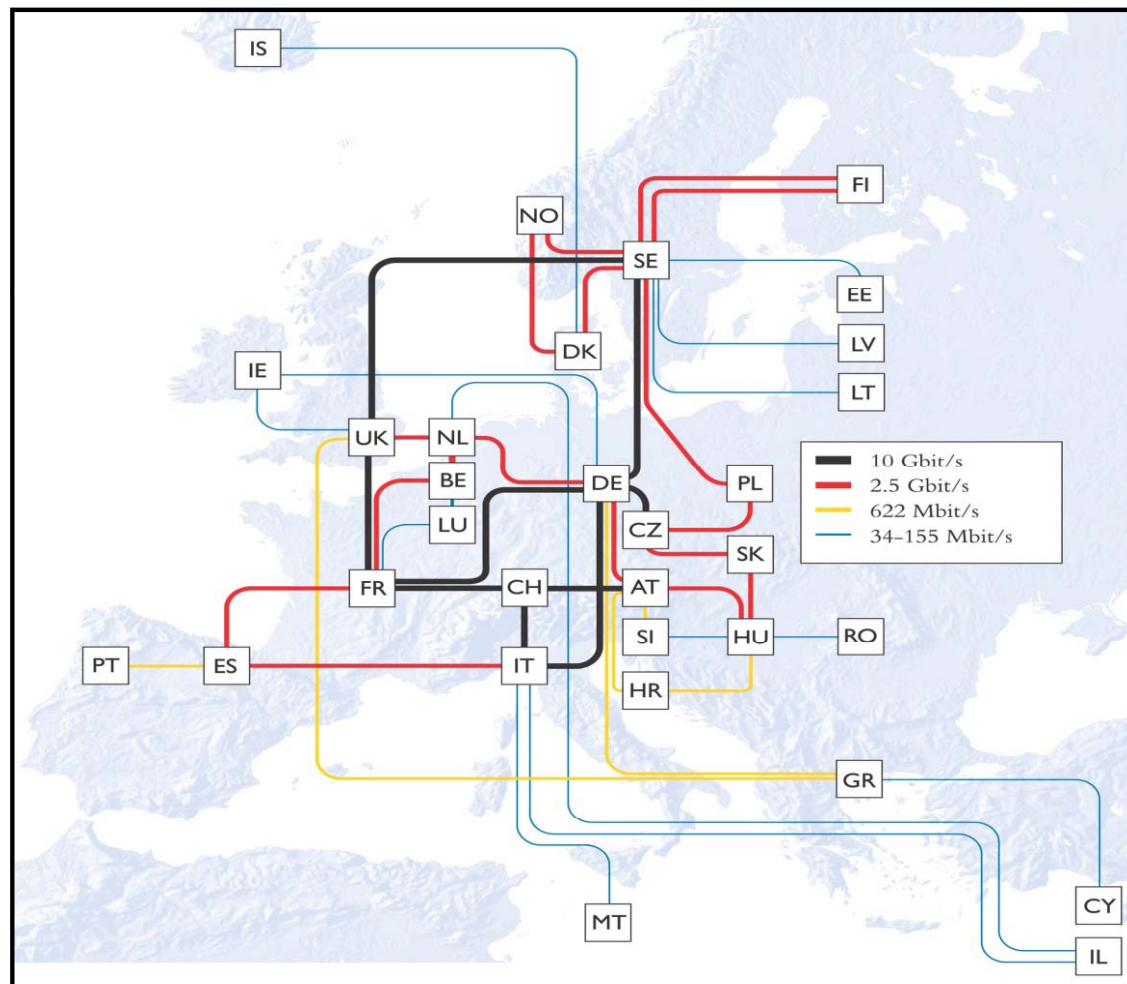
CA*net 3 National Optical Network





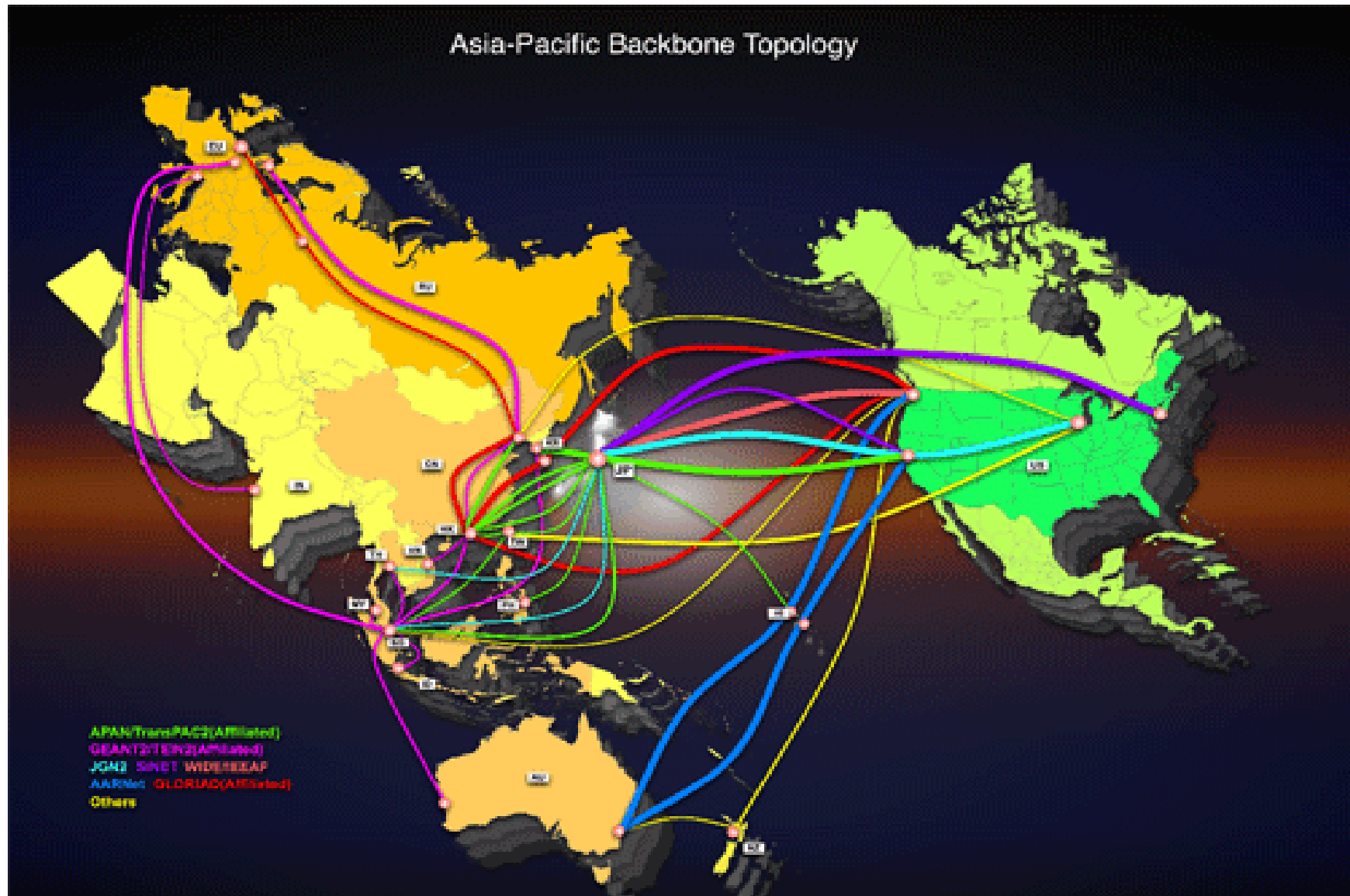
GEANT

- 连接**31**个国家
- 由**DANTE**负责运行
- **10Gbps**核心主干网
- 由**DANTE & EuroLink**提供
4x2.5Gbps + 2x1Gbps 跨过大西洋
- **EuroLink**由**NSF**提供资助





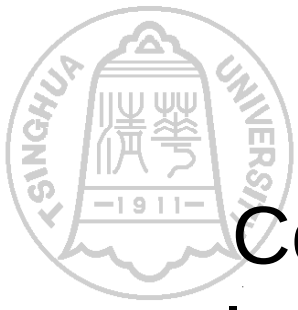
Asia-Pacific Advanced Network





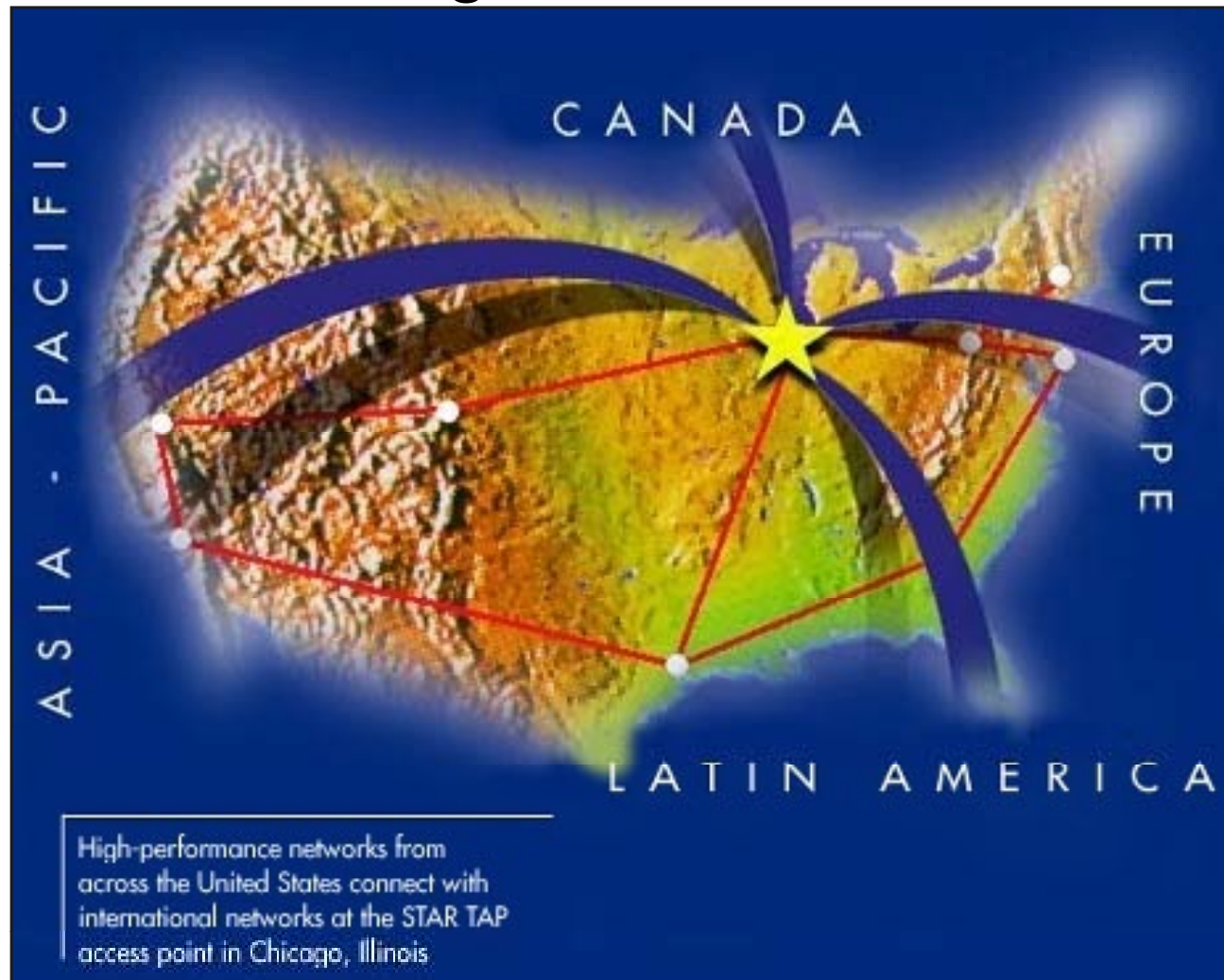
STAR TAP

- **Science, Technology And Research Transit Access Point”**
- 一个永久的基础设施，互联先进的国际网络，用来支持科研和教育
- 由**NSF CISE Networking and Communications Research and Infrastructure division**资助
- **Anchors the international vBNS connections program.**
- **<http://www.startap.net>**



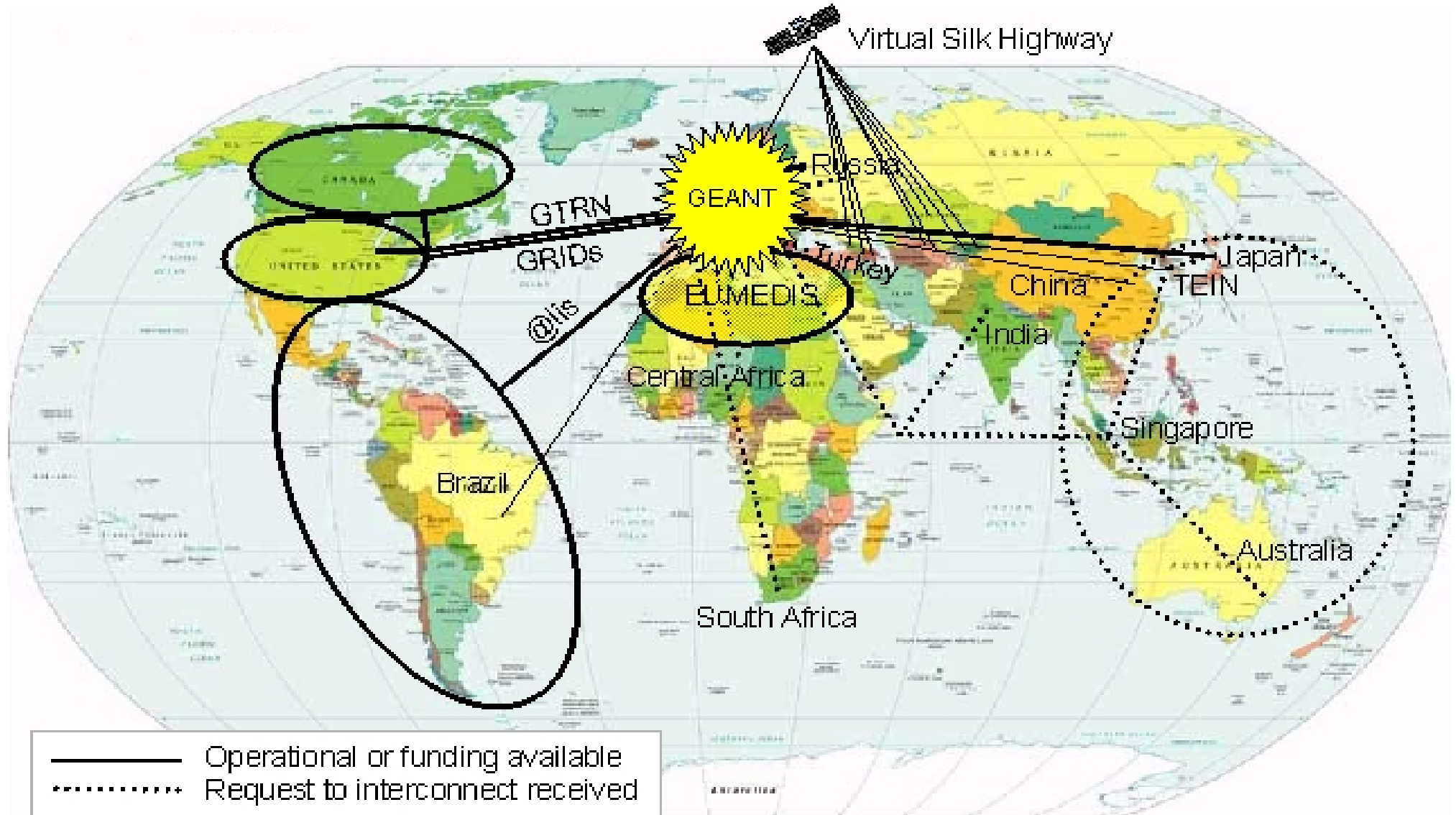
STAR TAP

Common Interconnect for NGI, Internet2,
International High-Performance Networks





GTRN





下一代互联网和IPv6的主要进展

- 下一代互联网和IPv6试验网络规模不断扩大，全球IPv6下一代互联网主干网正在形成
 - CERNET2, Internet2, Geant2, TEIN2...
- 下一代互联网技术和IPv6标准不断完善
 - IETF
 - IPv6网络设备和应用软件不断推出和采用
 - 移动通信和家用电器等对IPv6需求越来越迫切
- 下一代互联网基础研究正在得到重视
 - GENI、FIND、FIRE、CNGI
- **2011.2.3, ICANN宣布IPv4地址耗尽**



FIND: Future INternet Design

- 美国**NSF**的研究计划 **NeTS**:

- 传感器系统网络 **NOSS**
- 可编程的无线通信 **ProWin**
- 广义联网 **NBD**
- 未来互联网设计 **FIND**

- **FIND**

- 面向端到端的网络体系结构和设计
- 不面向单元技术和子网

- **Accelerate innovations and maintain US leadership in this vital area**



FIND Challenged the Research Community to:

Create the Future Internet you want to have in 10-15 years

30 Sep 2007

2



GENI: Global Environment for Networking Innovation

- 发现和评估可以作为**21**世纪互联网基础的新的革命性概念、示范和技术
- 建立一个支持新网络体系结构探险和评估的大规模试验环境
- 未来的互联网:
 - 值得社会信任
 - 激发科学和工程革命
 - 支持新技术融合
 - 支持普适计算
 - 成为真实世界和虚拟世界的桥梁
 - 支持革命性服务和应用



欧盟新一代互联网研究计划

The European FIRE
Future Internet Research and Experimentation Activities
Recommendations prepared by
the FIRE Scientific and Industrial Preparatory Expert Groups
DRAFT

1. CONTEXT – WHY FIRE ?	2
1.1. Evolution of the Internet – growth and problems - patches	2
1.2. The starting point - EU research in the field	2
1.3. Need for experimentally-driven research on the Future Internet – definition and methodology	3
1.4. Why should the Commission fund FIRE?	4
2. THE FIRE VISION	5
3. HOW TO IMPLEMENT IT	6
3.1. Experimentally-driven long-term research related to the Future Internet	6
3.1.1. Openness of the approach	6
3.1.2. Multidisciplinarity	6
3.1.3. Experimentally-driven Research	7
3.2. Experimental testbeds and their federation into FIRE	7
3.2.1. Research on methodologies and tools for testing	8
3.2.2. Operation and management of federated testbeds	8
3.2.3. Building the FIRE experimental facilities	9
4. SOCIO-ECONOMIC ISSUES	10
5. STRATEGIC ISSUES	11
5.1. Sustainability of the testbeds, beyond the lifetime of the projects	11
5.2. European strengths and weaknesses - competitiveness issues (SWOT-type)	12
5.3. Analysis of the constituency and their interests	13
5.4. Positioning of FIRE in the European and international contexts	14
ANNEXES:	16

Disclaimer:
This paper reports the consolidated ideas of many committed individuals but does not prejudge the individual opinions of any of its contributor or their employers.

Neither the European Commission nor any person acting on its behalf is responsible for the use which might be made of the information contained in the present publication. The European Commission is not responsible for the external web sites referred to in the present publication. The views expressed in this publication are those of the authors and do not necessarily reflect the official European Commission's view on the subject.

Draft Version 14 June 2007

The European FIRE

Future Internet Research and Experimentation Activities

■ FIRE 科学和产业预备专家组

■ 2007年6月14日草案



主要内容

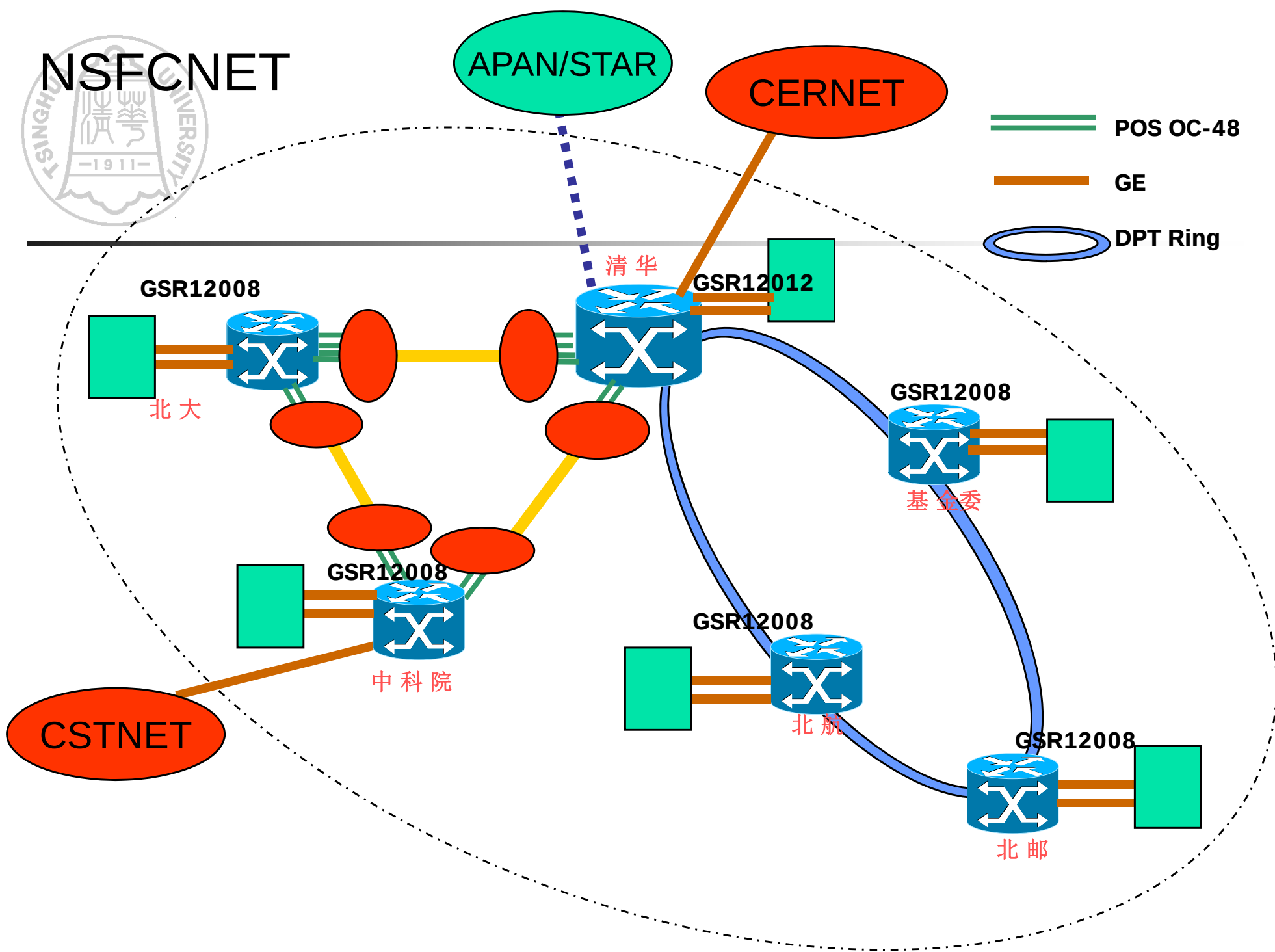
- 计算机网络概述
- 互联网的发展和成功经验
- 互联网的核心思想：分组交换
- 国际高速计算机网络研究计划
- 中国高速计算机网络研究计划



中国的互联网研究历程

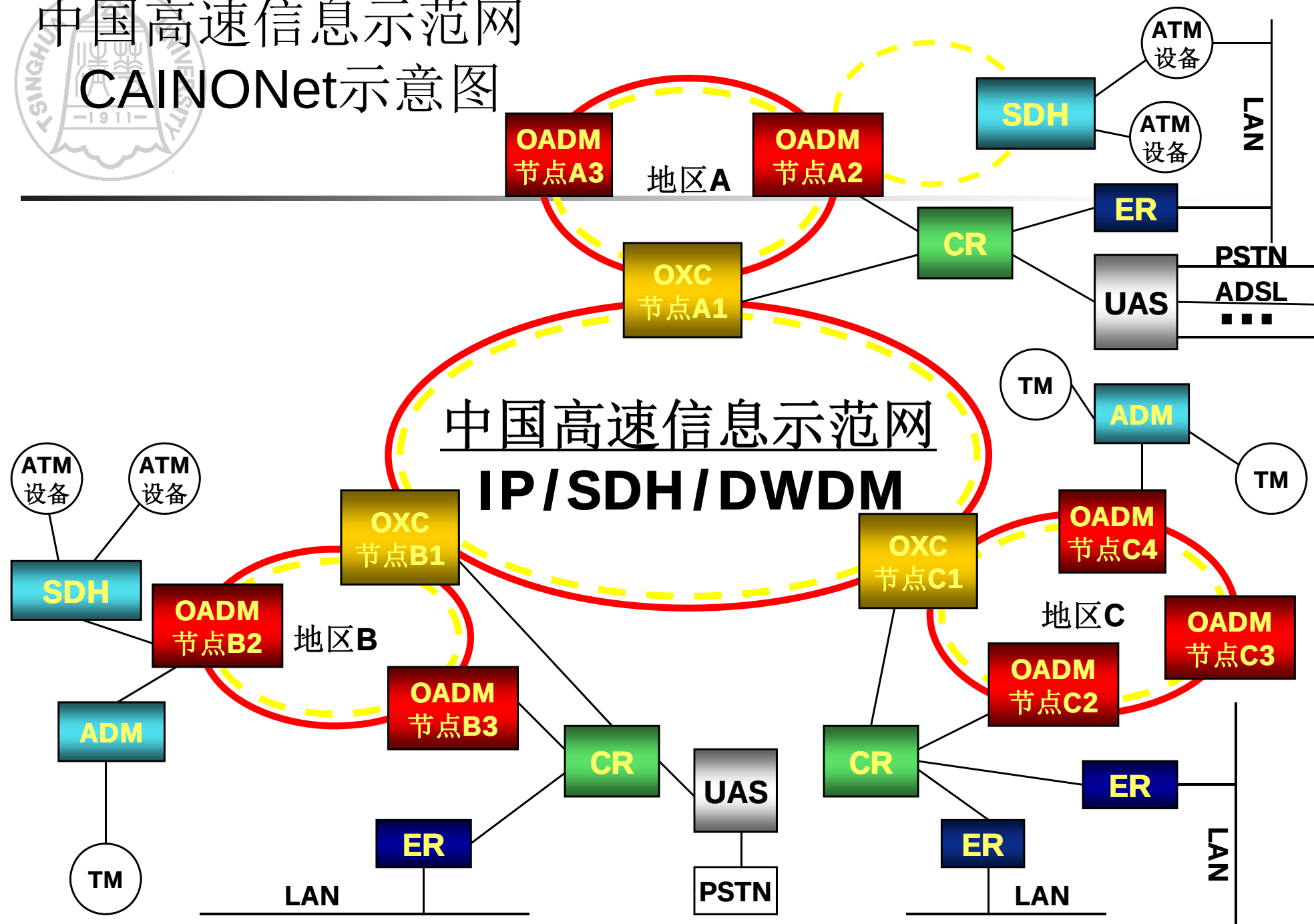
- 1994—1999：学习互联网建设应用
- 2000—2004：攻克关键技术：核心路由器
 - IPv4/IPv6核心路由器
 - 下一代互联网的研究：NSFCNET
 - 国家863计划：相关课题，CAINONET和3TNet
- 2005—2010：研究互联网体系结构
 - 国家自然科学基金重点项目
 - 973计划项目：新一代互联网体系结构基础研究
 - 863计划：新一代高可信网络
 - 支撑计划：可信任互联网
 - 国务院批准，国家发改委、科技部等八部委组织实施：中国下一代互联网示范工程CNGI

NSFCNET



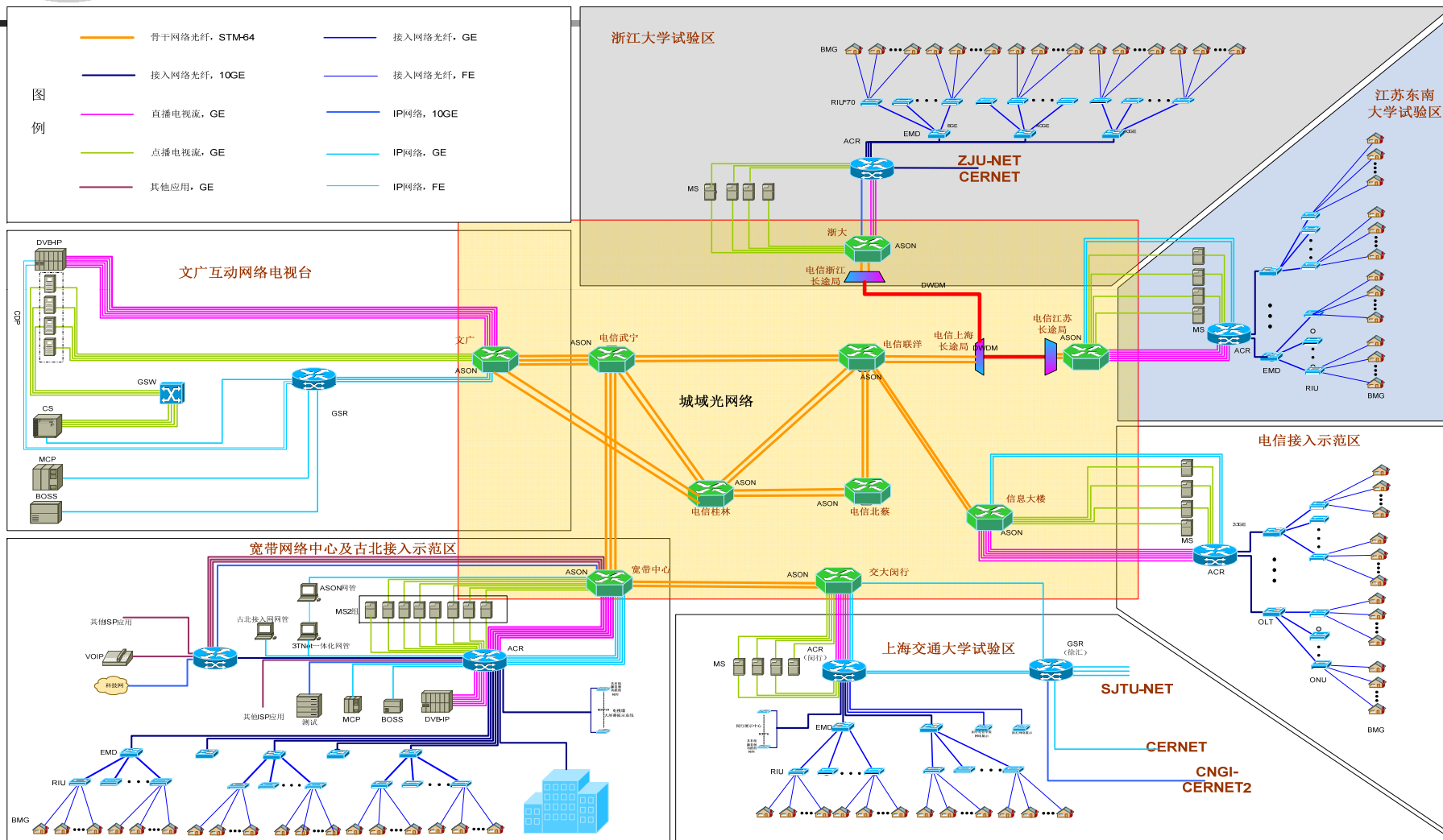


中国高速信息示范网 CAINONet示意图





3TNet拓扑图



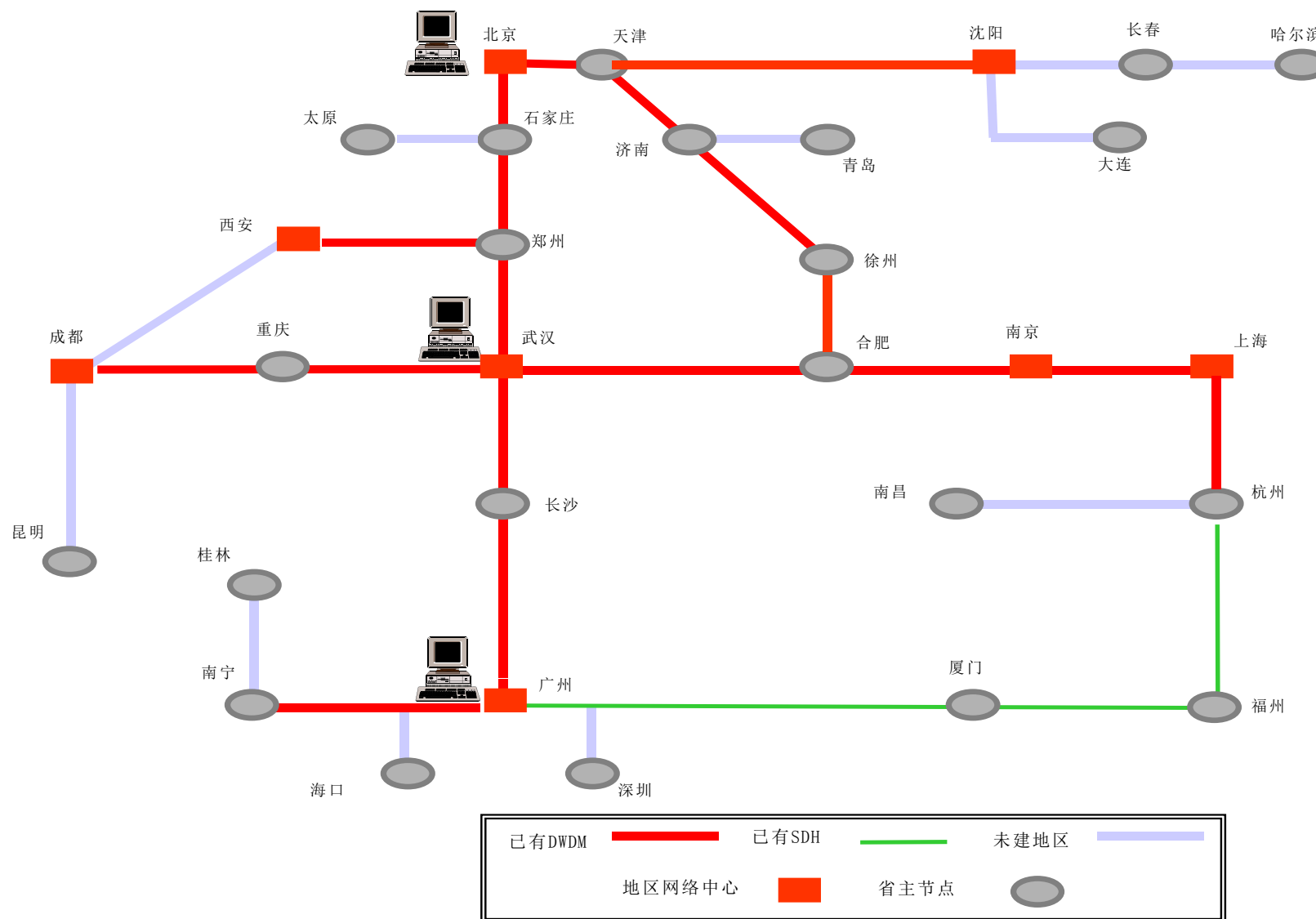


CERNET的发展现状

- 我国第一个全国性计算机互联网络（**1994**），目前我国第二大计算机互联网
- 全国主干网**2.5-10Gbps**,国际带宽**2.5Gbps**以上；地区网**155M-2.5Gbps**；**300**多高校**100M**以上的接入速度
- 国家网络中心、地区网络中心和省主节点分布在全国**36**个城市的**38**所高校，通达全国**200**多个城市
- 联网单位超过**1500**个，大学**800**多个；网络用户达**2000**万人

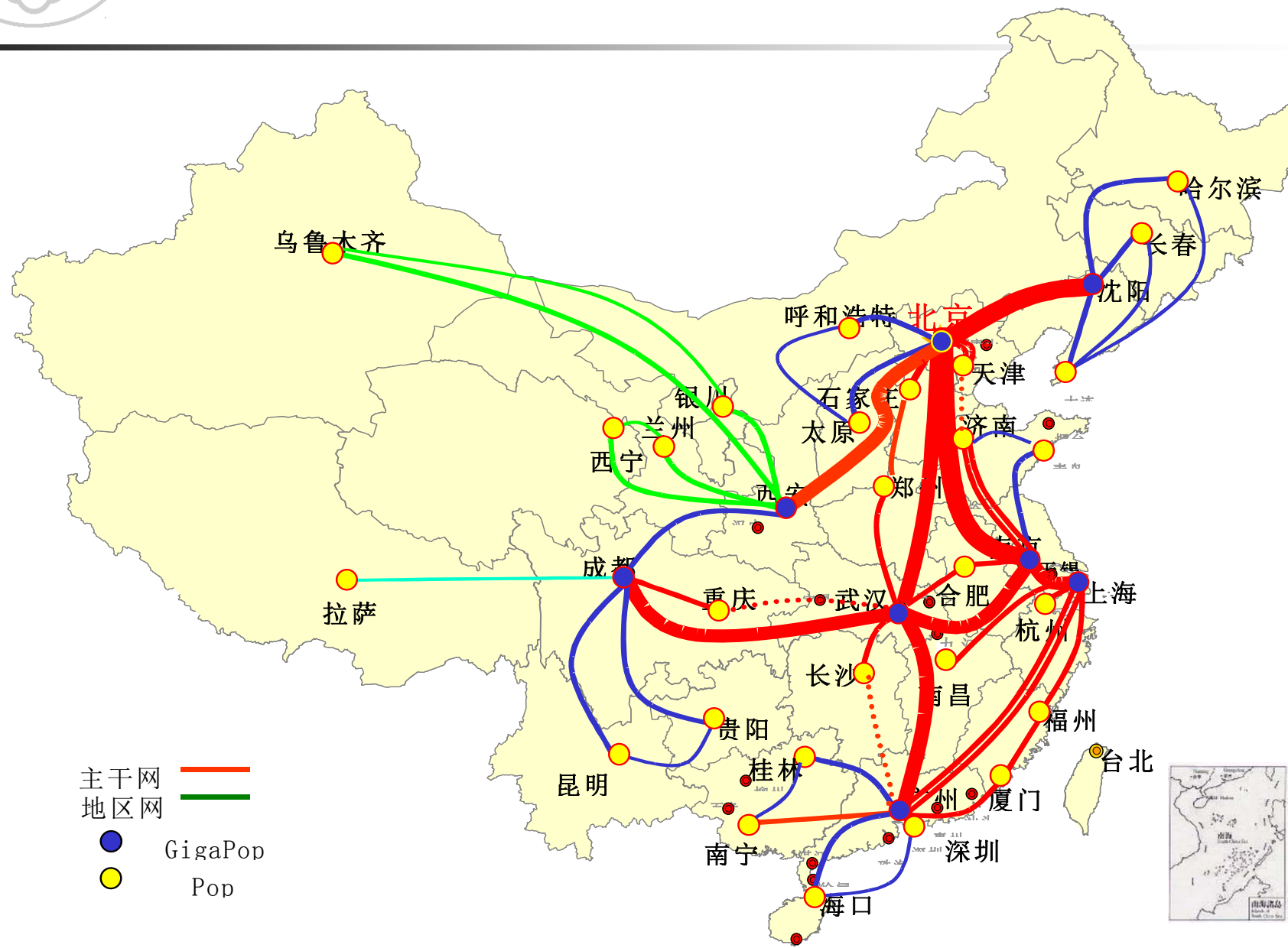


CERNET 光纤传输网络



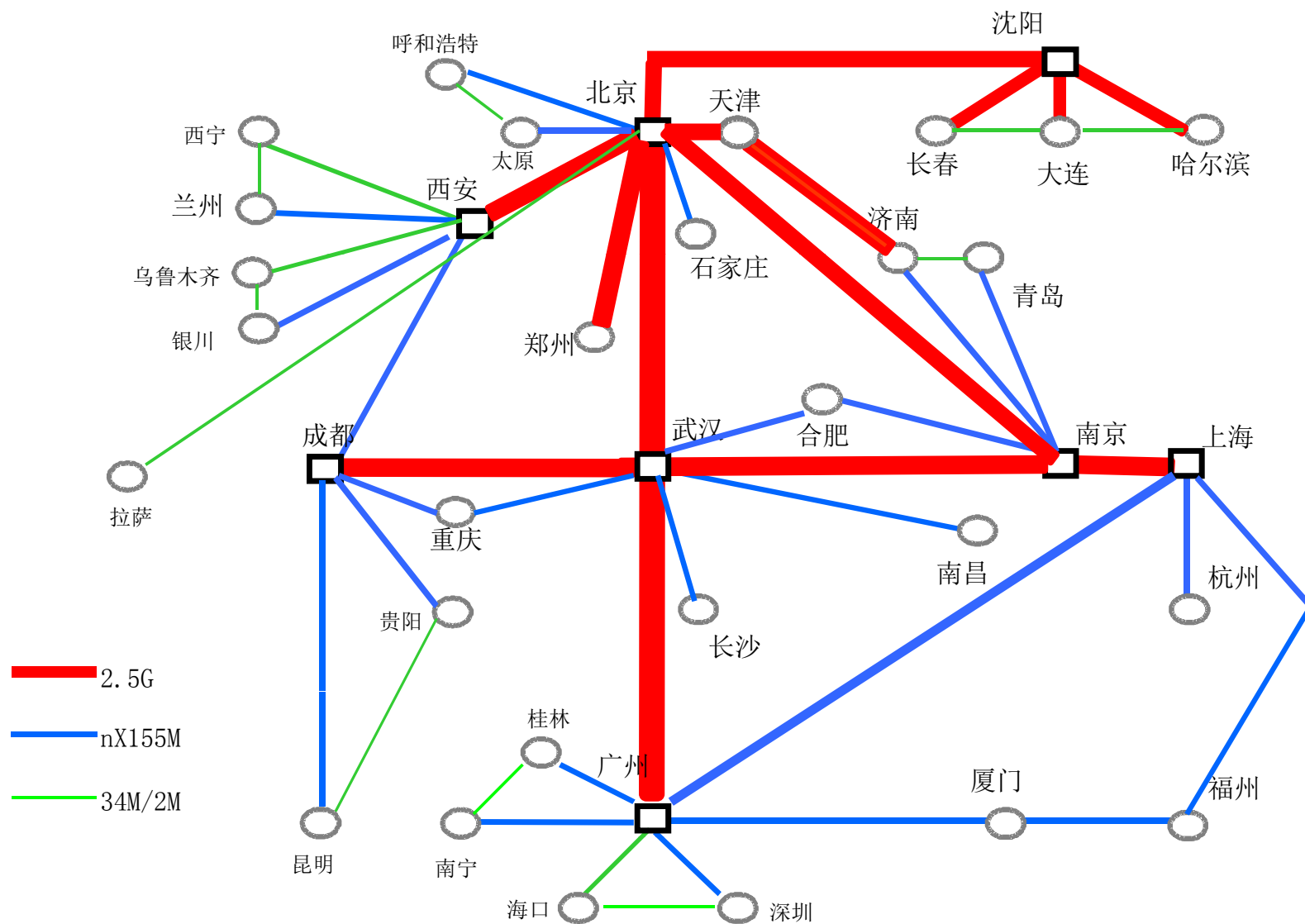


CERNET 主干网





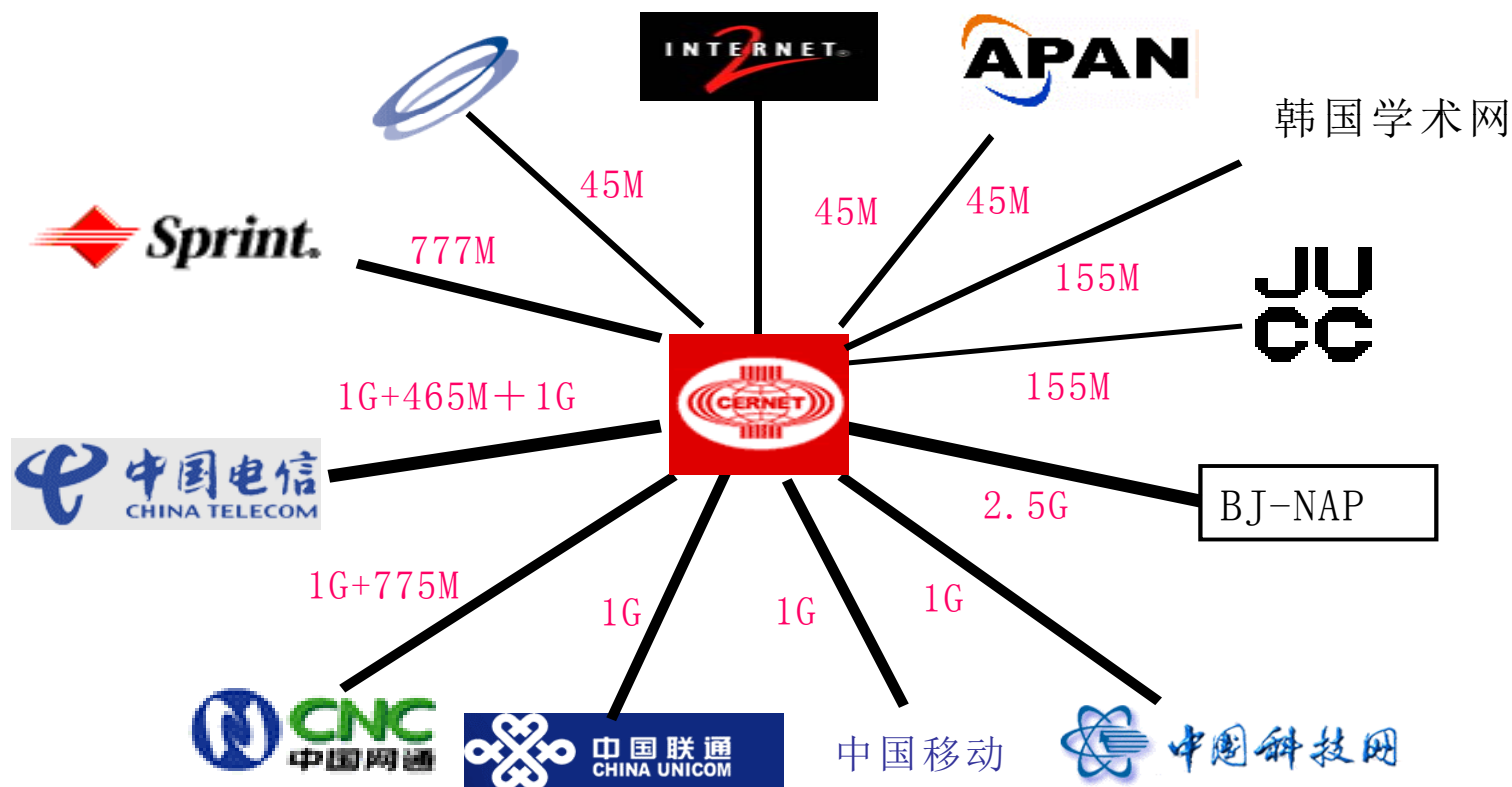
CERNET 主干网





CERNET 国际国内互联

CERNET 国际出入口及国内互联图





中国下一代互联网发展战略研究

- **2002年8月1日**，原国家计委组织中国“下一代互联网发展战略研究”
- **2003年8月**，国务院批复同意国家发改委等八部委“关于推动我国下一代互联网发展有关工作的请示”，正式启动“中国下一代互联网示范工程 **CNGI**”
- **2003年8月**，国家发改委委托中国工程院对“中国下一代互联网示范工程”中的示范网络核心网络承担单位进行了招标。**5+1**参加**CNGI**核心网建设
- **2004年7月**，**CNGI**领导小组、协调小组、专家委员会成立，项目全面开始实施



CNGI核心网：CERNET2

- 中国下一代互联示范网络**CNGI**最大的主干网
- 连接分布在**20**个主要城市核心节点，传输速率**2.5—10Gbps**, **25**所高校成为核心节点
- 与北美、欧洲、亚太地区国际下一代互联网实现实现高速互联
- 与其他**CNGI**主干网实现高速互联
- 全国**300**余所高校、科研单位和企业研发中心以**1G~10G**高速接入
- 成为我国研究下一代互联网技术、开发重大应用、推动下一代互联网产业发展的关键性基础设施
- **2004年12月25日**正式开通**CNGI—CERNET2**主干网



CERNET2主干网





CERNET2 的主要技术特点

- **2004年底初步建成世界上最大规模的纯IPv6网络**
 - 覆盖全国**20**个主要城市，连接**100**所以上高校和科研单位
 - 进行大规模纯**IPv6**网络验证和试验：**IPv4 Over IPv6**等
- **大规模采用国产设备，成为国产设备验证试验基地**
 - **IPv6**核心路由器，接入路由器和三层路由交换机
 - 清华比威，华为的**IPv6**核心路由器
- **建成可信任的下一代互联网**
 - 进行真实**IPv6**地址网络相关技术的试验研究
 - 为构件安全可信的下一代互联网奠定基础
- **开发下一代互联网的关键应用**
 - 基于**SIP**的大规模点到点综合通信系统
 - **IPv6**网络



课后阅读

- <http://www.internetsociety.org/internet/what-internet/history-internet/brief-history-internet>
- 纪录片，互联网时代

The background of the slide features a large, light gray watermark of the Tsinghua University seal. The seal is circular and contains the university's name in English, "TSINGHUA UNIVERSITY", around the perimeter. In the center is a shield with a bell and the year "1911".

第二章

计算机网络体系结构



主要内容

- 计算机网络的定义和组成
 - 计算机网络的定义
 - 计算机网络的组成
- 计算机网络体系结构
 - 计算机网络功能的分层
 - 协议和协议的分层结构
 - 计算机网络的体系结构
 - 分层原则
 - 端到端原则
- 典型计算机网络参考模型
 - 计算机网络的标准化
 - **OSI**参考模型
 - **TCP/IP**参考模型
- 其他网络体系结构
 - **Novell NetWare**
 - X.25分组交换网
 - **B-ISDN** 和 **ATM**



计算机网络的定义

■ 计算机网络的定义

- 定义：一批独立自治的计算机系统的互连集合体
- 说明：独立自治的计算机系统，互连的手段是各种各样的，依据协议进行工作

■ 计算机网络和通信网络

- 通信网络：重点研究通信终端（电话等）与通信网络，以及通信网络内部的通信问题
- 计算机网络：重点研究计算机联网

■ 计算机网络和分布式系统

- 分布式系统是一种建立在计算机网络之上的、具有高度内聚性 (**Cohesiveness**) 和透明性 (**Transparency**) 的系统，呈现给用户的是一个统一的系统，好像是一台计算机
- 计算机网络是独立自治的计算机系统的互连集合体，用户看到的还是不同的计算机
- 发展趋势是计算机网络与分布式系统逐渐统一



计算机网络的组成

- 计算机网络的组成
 - 两级结构的计算机网络
 - 资源子网（或用户子网）和通信子网

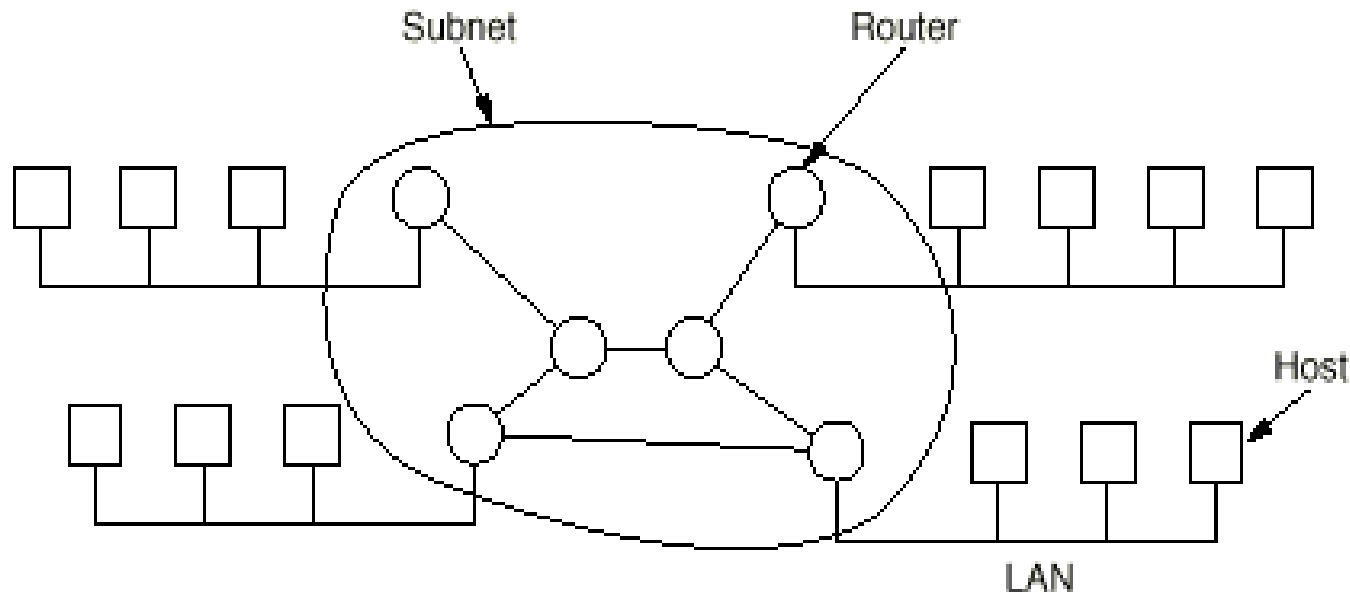


Fig. 1-5. Relation between hosts and the subnet.



计算机网络的组成（续）

- 资源子网

- 服务器
- 客户计算机

- 通信子网

- 通信线路（或称通道）
- 网络互连设备（路由器、交换机、**HUB**等）

- **Note:** 子网（**subnet**）有两种含义

- 含义1：物理网络的一部分，例如通信子网是通信线路和网络设备的集合。
- 含义2：与网络编址有关



计算机网络的构成（续）

■ 基本通信方式

- 交换式通信
- 广播式通信

■ 交换式通信

■ 基本特点

- 需要经过交换设备进行转发
- 交换设备根据需要选择输出

■ 典型拓扑结构

- **star, ring (loop), tree, complete, intersecting rings, irregular**

■ 关键技术：路由选择（**Routing**）

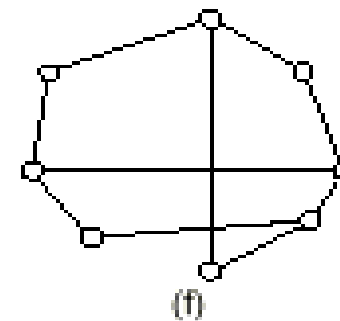
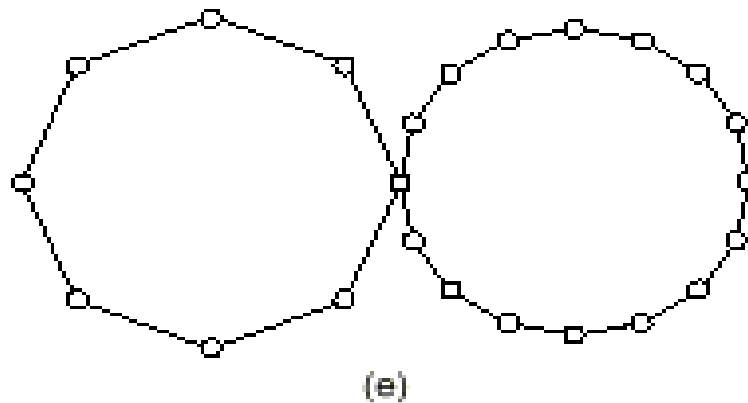
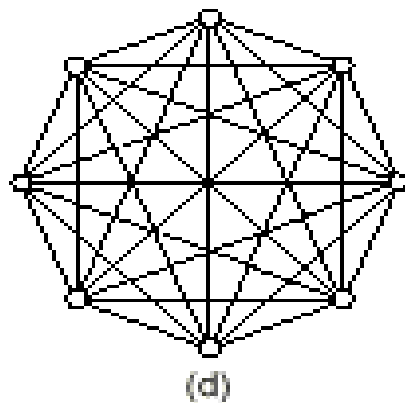
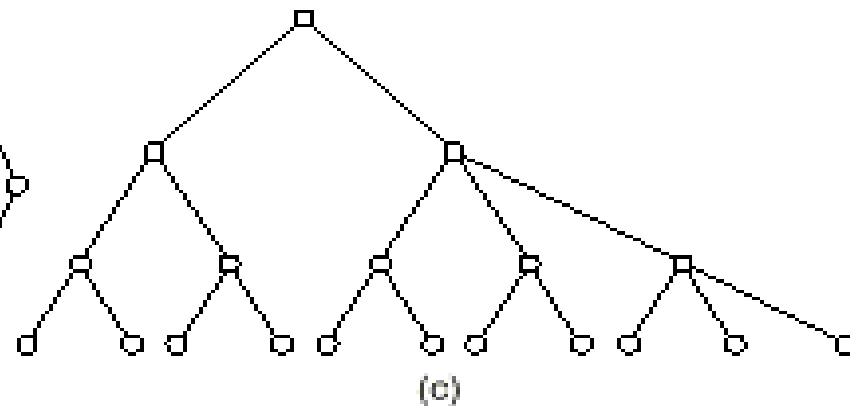
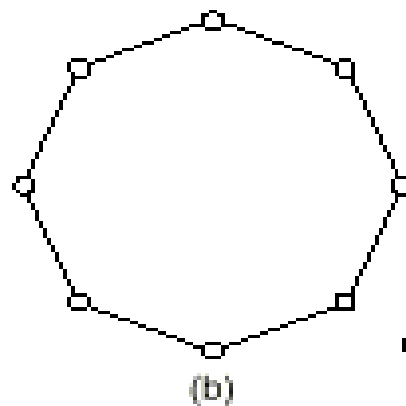
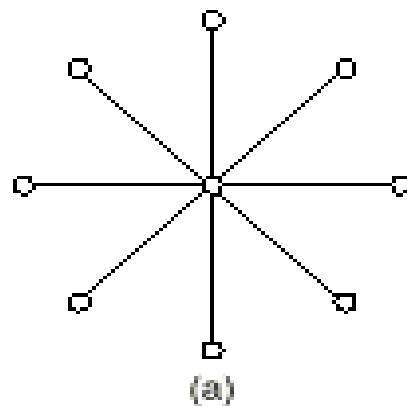


Fig. 1-6. Some possible topologies for a point-to-point subnet.
(a) Star. (b) Ring. (c) Tree. (d) Complete. (e) Intersecting rings. (f) Irregular.



计算机网络的构成（续）

■ 广播式通信

■ 基本特点

- 多台计算机共享通信线路
- 任一台计算机发出的信息可以直接被其它计算机接收

■ 典型拓扑结构

- bus, ring

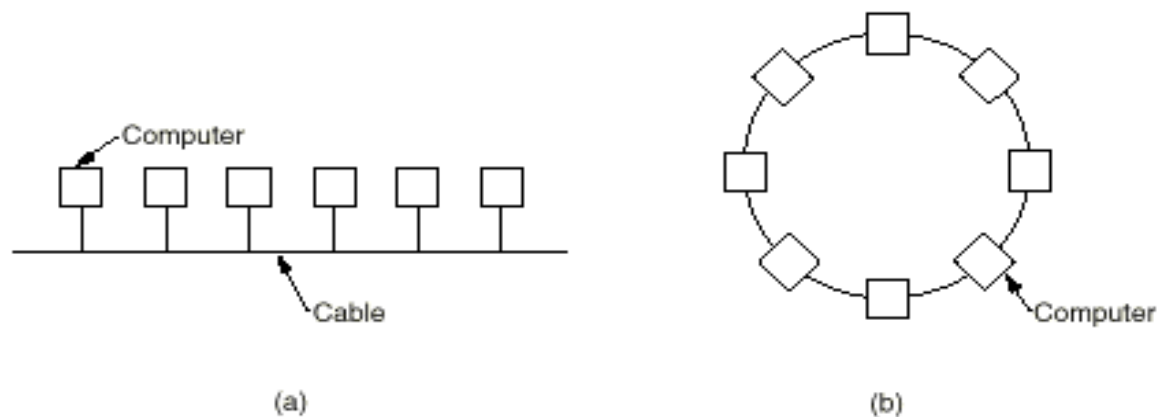


Fig. 1-3. Two broadcast networks. (a) Bus. (b) Ring.



计算机网络的构成（续）

■ 关键技术：通道分配

■ 静态分配：分时间片

- 特点：控制简单，通道利用率低

■ 动态分配：各站点动态使用通道

- 特点：控制复杂，通道利用率高

- 通道分配方法：

- 集中式：只有一个仲裁机构
- 分布式：各站点均有仲裁机构



计算机网络的构成（续）

- 局域网络（**Local Area Networks**）
 - 主要采用广播式通信技术
- 城域网络（**Metropolitan Area Networks**）
 - 主要采用交换式通信技术
- 广域网络（**Wide Area Networks**）
 - 主要采用交换式通信技术



主要内容

- 计算机网络的定义和组成
 - 计算机网络的定义
 - 计算机网络的组成
- 计算机网络体系结构
 - 计算机网络功能的分层
 - 协议和协议的分层结构
 - 计算机网络的体系结构
 - 分层原则
 - 端到端原则
- 典型计算机网络参考模型
 - 计算机网络的标准化
 - **OSI**参考模型
 - **TCP/IP**参考模型
- 其他网络体系结构
 - **Novell NetWare**
 - **X.25**分组交换网
 - **B-ISDN** 和 **ATM**



计算机网络的体系结构

- **计算机网络的体系结构**：对计算机网络及其部件所完成功能的比较精确的定义，即从**功能**的角度描述计算机网络的**结构**，是**层次和层间关系**的集合
- **注意**：计算机网络体系结构仅仅定义了网络及其部件通过协议应完成的功能，不定义协议的实现细节和各层协议之间的接口关系
- 网络功能的分层 → 协议的分层 → 体系结构的分层
- 协议分层易于协议的设计、分析、实现和测试



计算机网络功能的分层

- 计算机网络的基本功能是为地理位置不同的计算机用户之间提供访问通路
- 下述功能是必须提供的：
 - 连接源结点和目的结点的物理传输线路，可以经过中间结点
 - 每条线路两端的结点利用信号进行二进制通信
 - 无差错的信息传送
 - 多个用户共享一条物理线路
 - 按照地址信息，进行路由选择
 - 信息缓冲和流量控制
 - 会话控制
 - 满足各种用户、各种应用的访问要求



计算机网络功能的分层（续）

- 上述功能有三个显著特点
 - 上述功能必须同时满足一对用户
 - 用户之间的通信功能是相互的
 - 这些功能分散在各个网络设备和用户设备中
- 一般人们采用“层次结构”的方法来描述计算机网络，即：计算机网络中提供的功能是分成层次的



协议和协议的分层结构

■ 协议的定义和组成

- 层次结构的计算机网络功能中，最重要的功能是通信功能
- 通信功能主要涉及同一层次中通信双方的相互作用
- 位于不同计算机上进行对话的第N层通信各方可分别看成是一种进程，称为对等（同等）进程
- 协议（**Protocol**）： 计算机网络同等层次中，通信双方进行信息交换时必须遵守的规则
- 协议的组成
 - 语法（**syntax**）： 以二进制形式表示的命令和相应的结构
 - 语义（**semantics**）： 由发出的命令请求，完成的动作和回送的响应组成的集合
 - 定时关系（**timing**）： 有关事件顺序的说明



协议和协议的分层结构（续）

- 协议的分层和层间结构
 - 协议分层
 - 目的主机第N层收到的报文与源主机第N层发出的报文相同
 - 洋葱结构
 - 协议分层要保证整个通信系统功能完备、高效
 - 相邻层之间都有一个接口（**Interface**），它定义了下层向上层提供的原语操作和服务
 - 对于第N层协议来说，它有如下特性
 - 不知道上、下层的内部结构
 - 独立完成某种功能
 - 为上层提供服务
 - 使用下层提供的服务



计算机网络体系结构

■ 基本术语与分层结构

- 接口：定义了下层向上层提供的原语操作和服务
- 协议：计算机网络同等层次中，通信双方进行信息交换时必须遵守的规则
- 服务：层间交换信息时必须遵守的规则
- 服务和协议的关系
- 服务提供者，服务用户

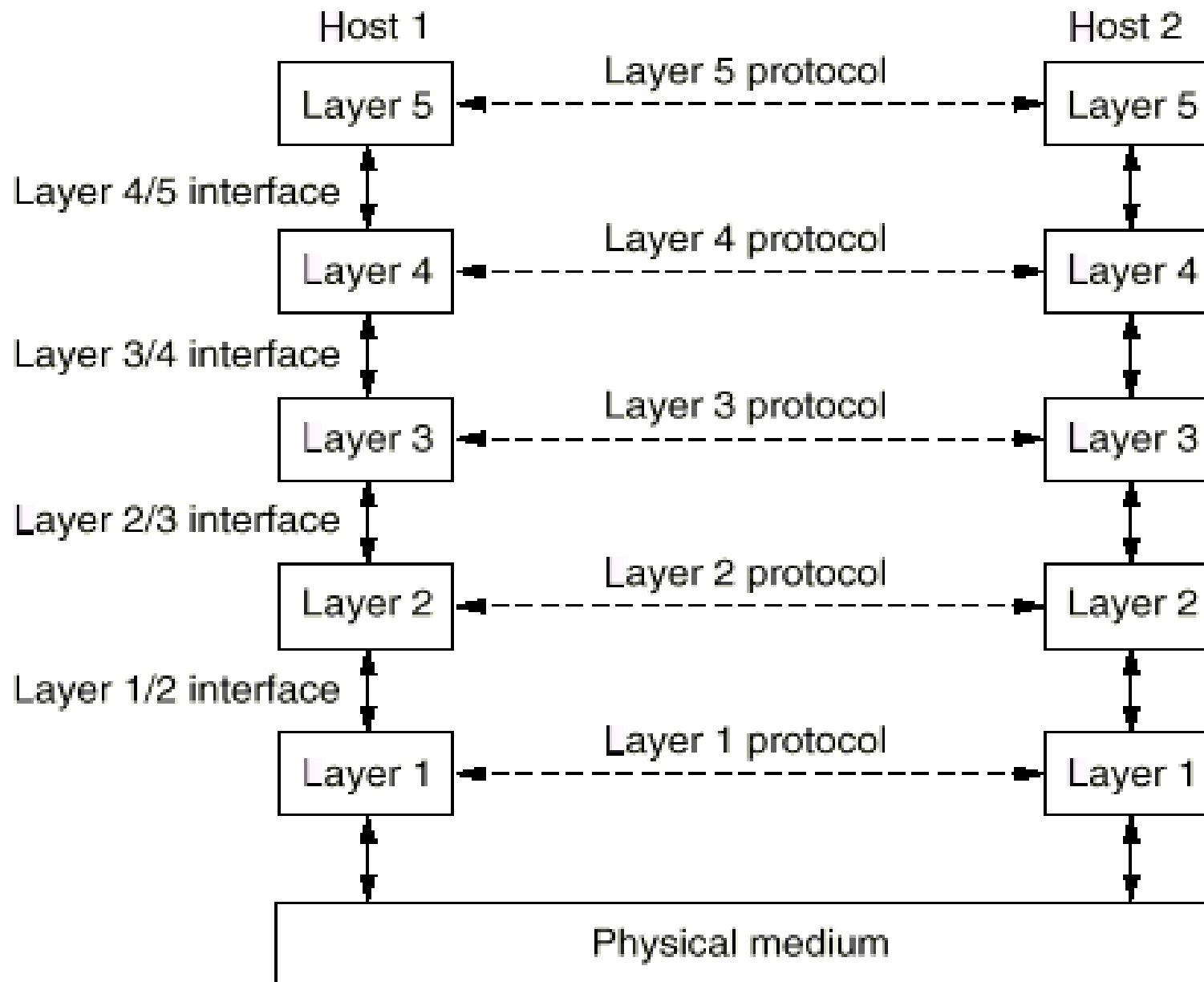


Fig. 1-9. Layers, protocols, and interfaces.



计算机网络体系结构（续）

- 服务访问点**SAP**（**Service Access Point**）
 - 任何层间服务是在接口的**SAP**上进行的
 - 每个**SAP**有唯一的识别地址
 - 每个层间接口可以有多个**SAP**
- 接口数据单元**IDU**（**Interface Data Unit**）
 - **IDU**是通过**SAP**进行传送的层间信息单元
 - **IDU**由上层的服务数据单元**SDU**（**Service Data Unit**）和接口控制信息**ICI**（**Interface Control Information**）组成
- 协议数据单元**PDU**（**Protocol Data Unit**）
 - 第**N**层实体通过网络传送给它的对等实体的信息单元
 - **PDU**由上层的服务数据单元**SDU**或其分段和协议控制信息**PCI**（**Protocol Control Information**）组成
 - 分段和重组

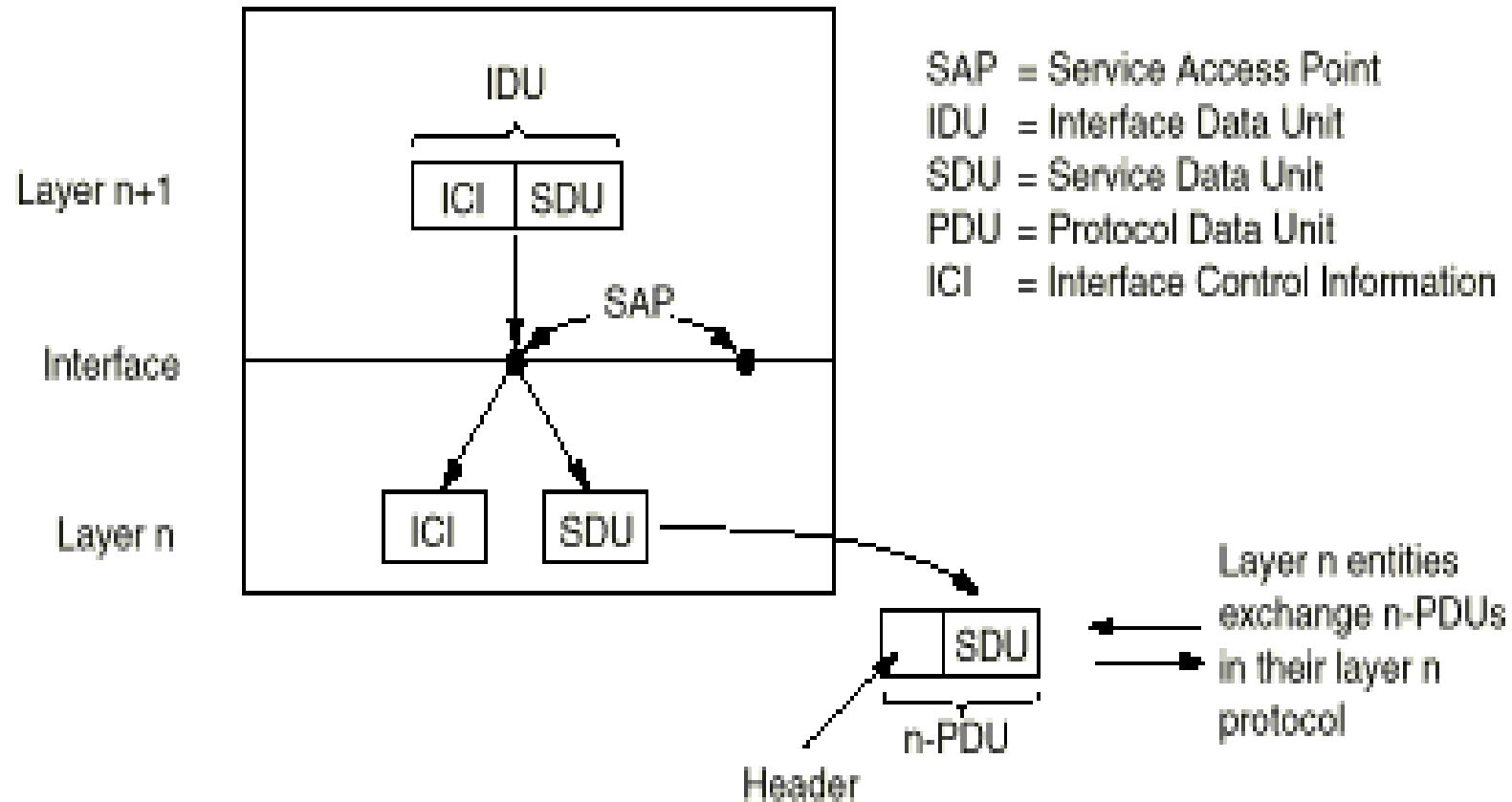


Fig. 1-12. Relation between layers at an interface.



Layer

5

4

3

2

1

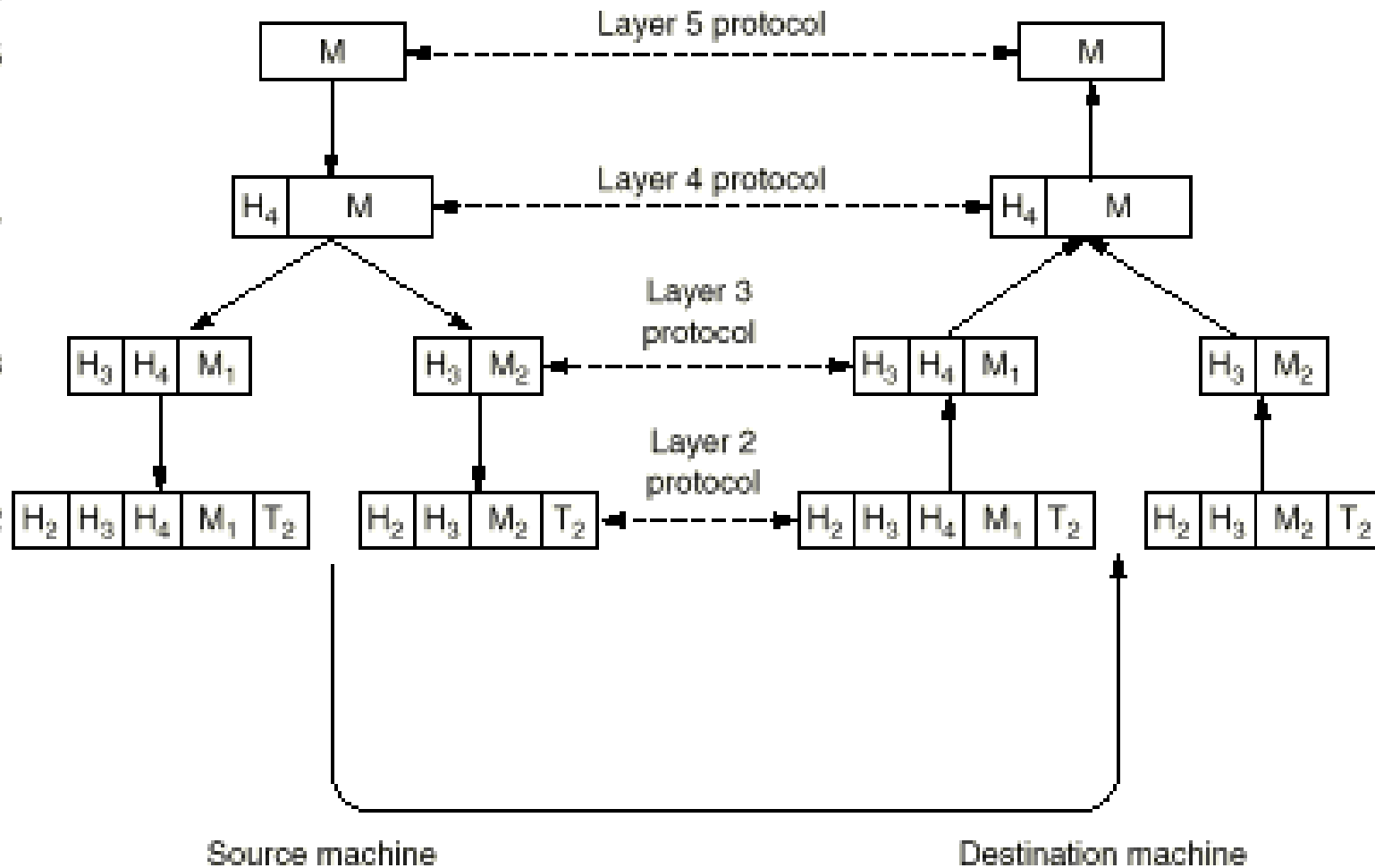


Fig. 1-11. Example information flow supporting virtual communication in layer 5.



计算机网络体系结构（续）

■ 服务分类和服务原语（**primitives**）

■ 面向连接的服务和无连接服务

■ 面向连接的服务

- 当使用服务传送数据时，首先建立连接，然后使用该连接传送数据。使用完后，关闭连接
- 特点：顺序性好

■ 无连接服务

- 直接使用服务传送数据，每个包独立进行路由选择
- 特点：顺序性差

- 注意：连接并不意味着可靠，可靠要通过确认、重传等机制来保证



计算机网络体系结构（续）

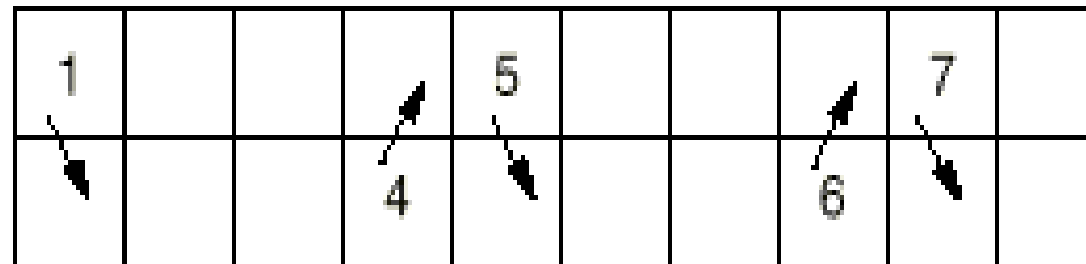
■ 服务原语

- 服务在形式上是由一组接口原语（或操作）来描述的
- 服务原语可分为四种类型
 - 请求（Request）:An entity wants the service to do some work
 - 指示（Indication）:An entity is to be informed about an event
 - 响应（Response）:An entity wants to respond to an event
 - 确认（Confirm）:The response to an earlier request has come back



Layer N + 1

Layer N

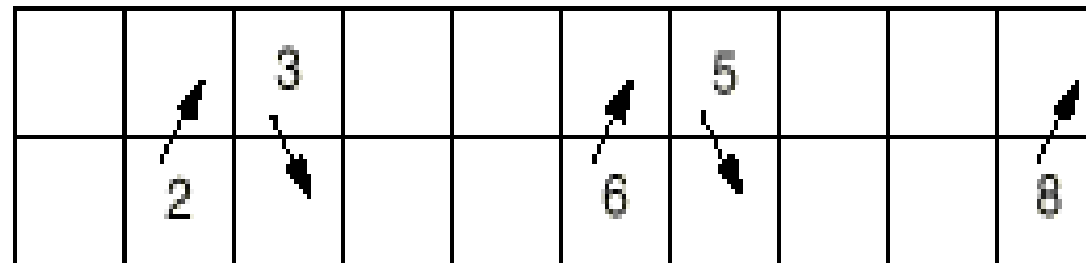


Computer 1

1 2 3 4 5 6 7 8 9 10 Time →

Layer N + 1

Layer N

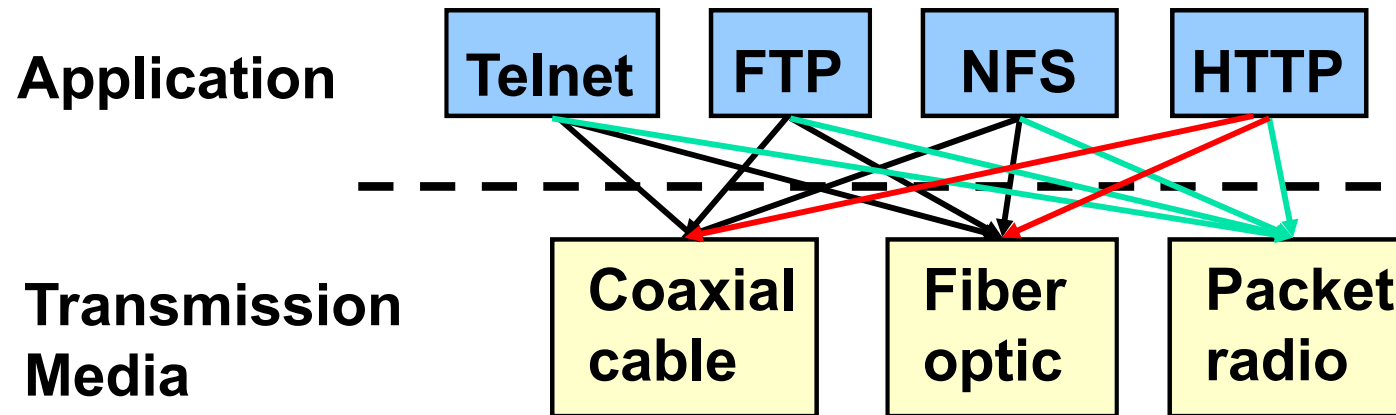


Computer 2

Fig. 1-15. How a computer would invite its Aunt Millie to tea. The numbers near the tail end of each arrow refer to the eight service primitives discussed in this section.



Why Layering?

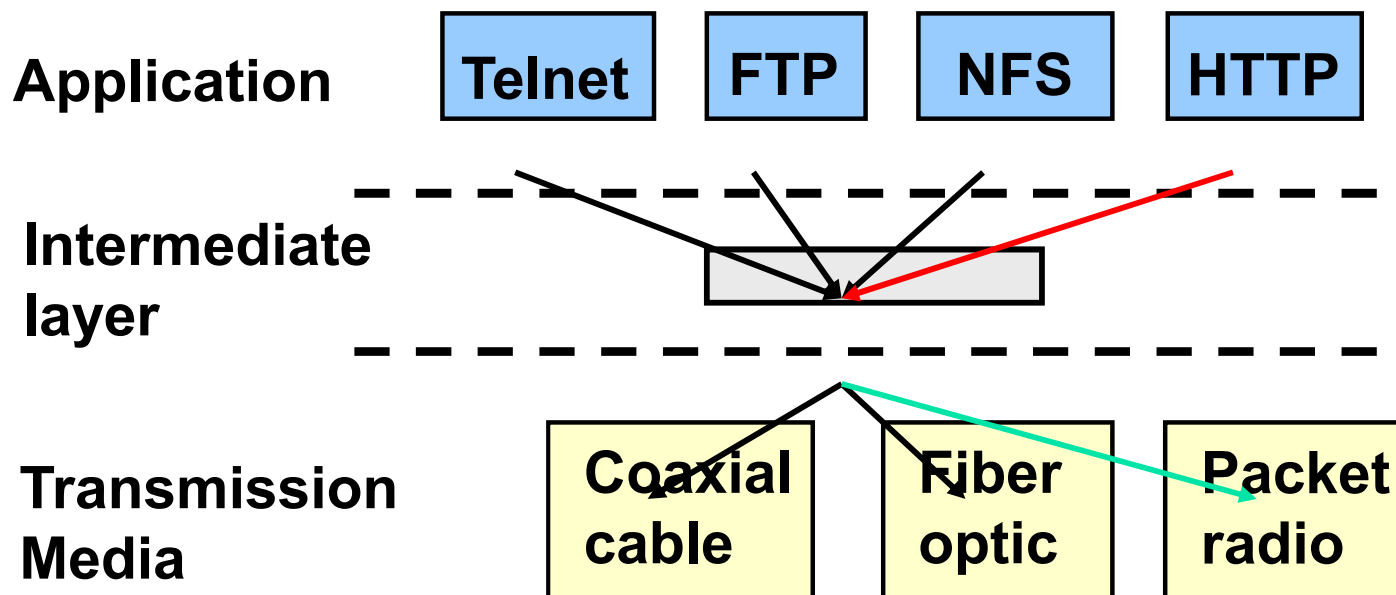


- **No layering: each new application has to be **re**-implemented for every network technology!**



Why Layering?

- **Solution: introduce an intermediate layer that provides a **unique** abstraction for various network technologies**





Layering

■ Advantages

- **Modularity** – protocols easier to manage and maintain
- **Abstract functionality** – lower layers can be changed **without** affecting the upper layers
- **Reuse** – upper layers can reuse the functionality provided by lower layers

■ Disadvantages

- **Information hiding** – inefficient implementations

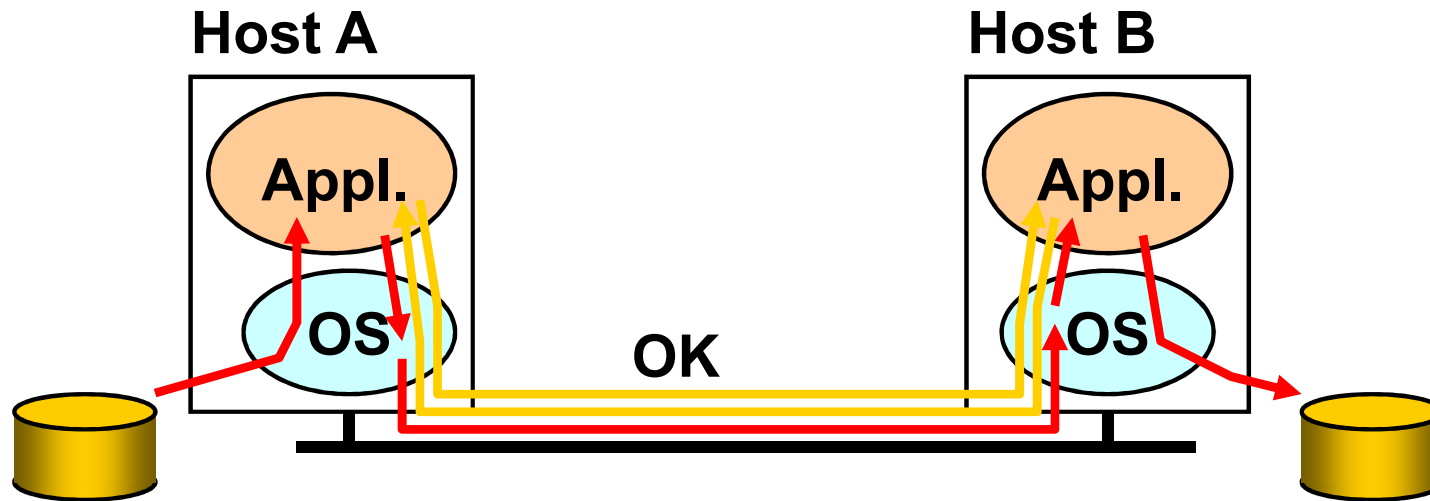


End-to-End Argument

- **Think twice before implementing a functionality that you believe that is useful to an application at a lower layer**
- **If the application can implement a functionality correctly, implement it a lower layer only as a performance enhancement**



Example: Reliable File Transfer



- **Solution 1: make each step reliable, and then concatenate them**
- **Solution 2: end-to-end check and retry**



Discussion

- **The receiver has to do the check anyway!**
- **Thus, full functionality can be entirely implemented at application layer; **no** need for reliability from lower layers**
- **Is there any need to implement reliability at lower layers?**
- **Yes, but only to improve performance**
- **Example:**
 - **Assume a high error rate on communication network**
 - **Then, a reliable communication service at data link layer might help**



Trade-offs

- **Application has more information about the data and the semantic of the service it requires (e.g., can check only at the end of each data unit)**
- **A lower layer has more information about constraints in data transmission (e.g., packet size, error rate)**
- **Note: these trade-offs are a direct result of layering!**



Rule of Thumb

- **Implementing a functionality at a lower level should have minimum performance impact on the application that do not use the functionality**



主要内容

- 计算机网络的定义和组成
 - 计算机网络的定义
 - 计算机网络的组成
- 计算机网络体系结构
 - 计算机网络功能的分层
 - 协议和协议的分层结构
 - 计算机网络的体系结构
 - 分层原则
 - 端到端原则
- 典型计算机网络参考模型
 - 计算机网络的标准化
 - **OSI**参考模型
 - **TCP/IP**参考模型
- 其他网络体系结构
 - **Novell NetWare**
 - **X.25**分组交换网
 - **B-ISDN** 和 **ATM**



计算机网络的标准化

■ 电信标准

- **1865**年成立国际电信联盟**ITU** (**International Telecommunication Union**)
- **1947**年 **ITU** 成为联合国的一个组织，由三部分组成
 - **ITU- R**: 无线通信
 - **ITU- T**: 电信标准，**1956 - 1993** 年称为**CCITT**，
下设许多研究组**SG**，研究组下设专题，例如：
Q42/SG VII 专门研究 **OSI** 参考模型。
 - **ITU- D**: 开发



计算机网络的标准化（续）

■ 国际标准

- **1946**年成立的国际标准化组织 **ISO** 负责制定各种国际标准，**ISO** 有**89**个成员国家，**85** 个其他成员
 - **ISO** 有**200** 多个技术委员会**TC**，每个技术委员会下设若干分委员会**SC**，每个分委员会由若干工作组**WG** 组成。
 - 例如：**TC97** - 计算机和信息处理，**TC97/SC21/WG1** - **OSI** 体系结构、概念性方案和形式描述
 - 一个国际标准的形成：**CD**（**Committee Draft**）- **DIS**（**Draft International Standard**）- **IS**（**International Standard**）



计算机网络的标准化（续）

- 其它标准化组织：
 - **ANSI**: 美国国家标准研究所，**ISO** 的美国代表
 - **NIST**: 美国国家标准和技术研究所，美国商业部的标准化机构
 - **IEEE**: 发布行业标准。例如**IEEE 802**，后成为**ISO 8802**。
 - **OIF** (**Optical Internetworking Forum**)
 - **CCSA**: 中国通信标准化协会，行业标准
- 值得注意的是，**ITU - T** 和 **ISO** 之间有很好的合作和协调。



计算机网络的标准化（续）

■ Internet 标准

- **Internet** 的标准是自发而非政府干预的，称为 **RFC（Request For Comments）**。
- **1969** 年 **ARPANET** 时就开始发布 **RFC**，**1969.4** 产生 **RFC0001**，至今已超过 **6000** 个
- **1983** 年成立 **IAB（Internet Architecture Board）**
- **1986** 年在 **IAB** 下成立 **IETF**
 - **1986** 年 **1** 月，在美国圣地亚哥召开第一次 **IETF** 会议，**21** 人参加
- **1989** 年在 **IAB** 下又成立了 **IRTF**



OSI参考模型

- **1983年ISO 的 OSI 模型正式成为国际标准**
 - **物理层 (The Physical Layer)**：在物理线路上传输原始的二进制数据位（基本网络硬件）
 - **数据链路层 (The Data Link Layer)**：在有差错的物理线路上提供无差错的数据传输（**Frame**）
 - **网络层 (The Network Layer)**：控制通信子网提供源点到目的点的数据传送（**Packet**）
 - **运输层 (The Transport Layer)**：为用户提供端到端的数据传送服务



OSI参考模型（续）

- 会话层（**The Session Layer**）：为用户提供会话控制服务（安全认证）
 - 令牌管理和同步，例如，在数据流中插入检查点（**checkpoint**）
- 表示层（**The Presentation Layer**）：为用户提供数据转换和表示服务
- 应用层（**The Application Layer**）

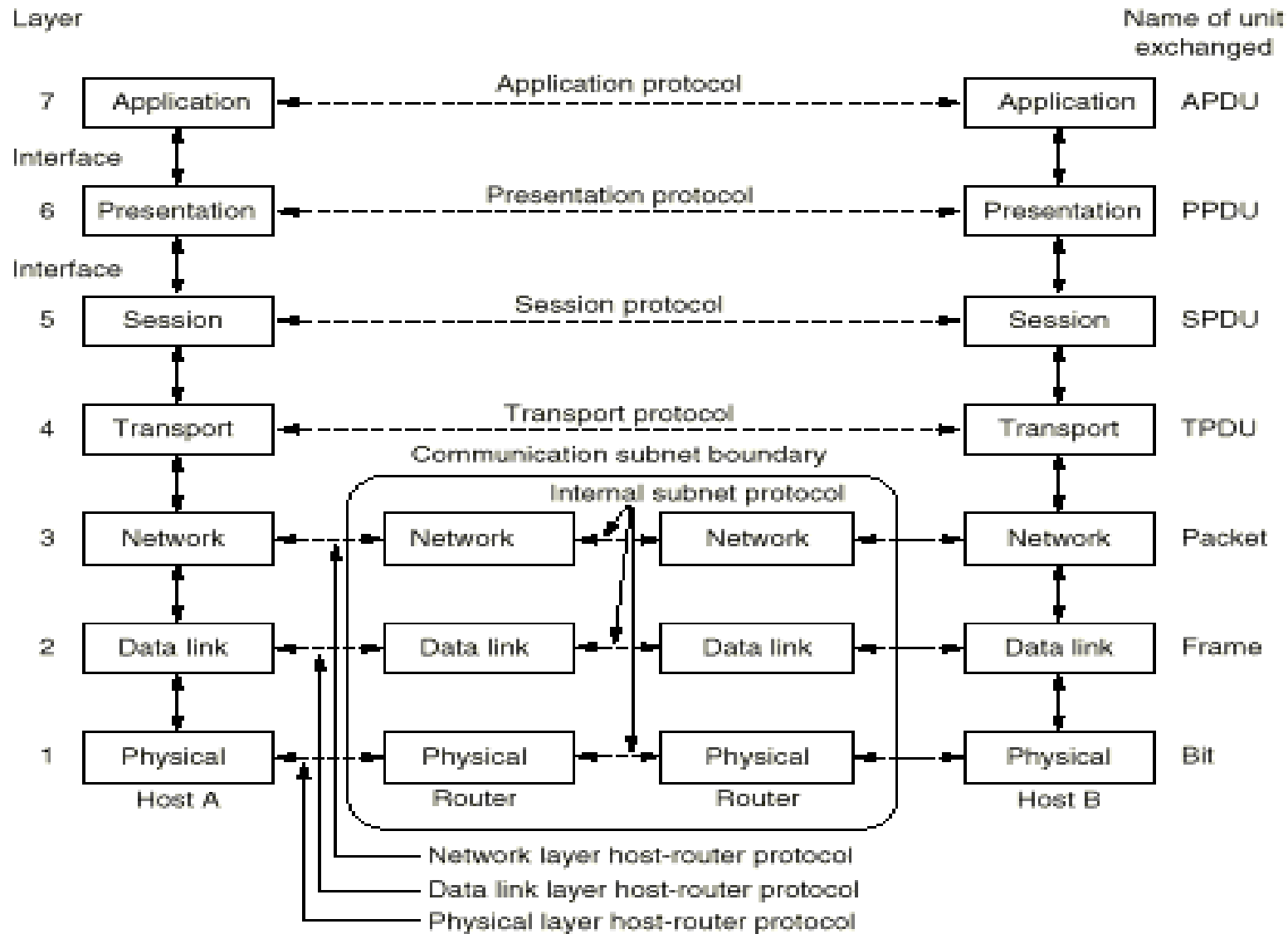


Fig. 1-16. The OSI reference model.

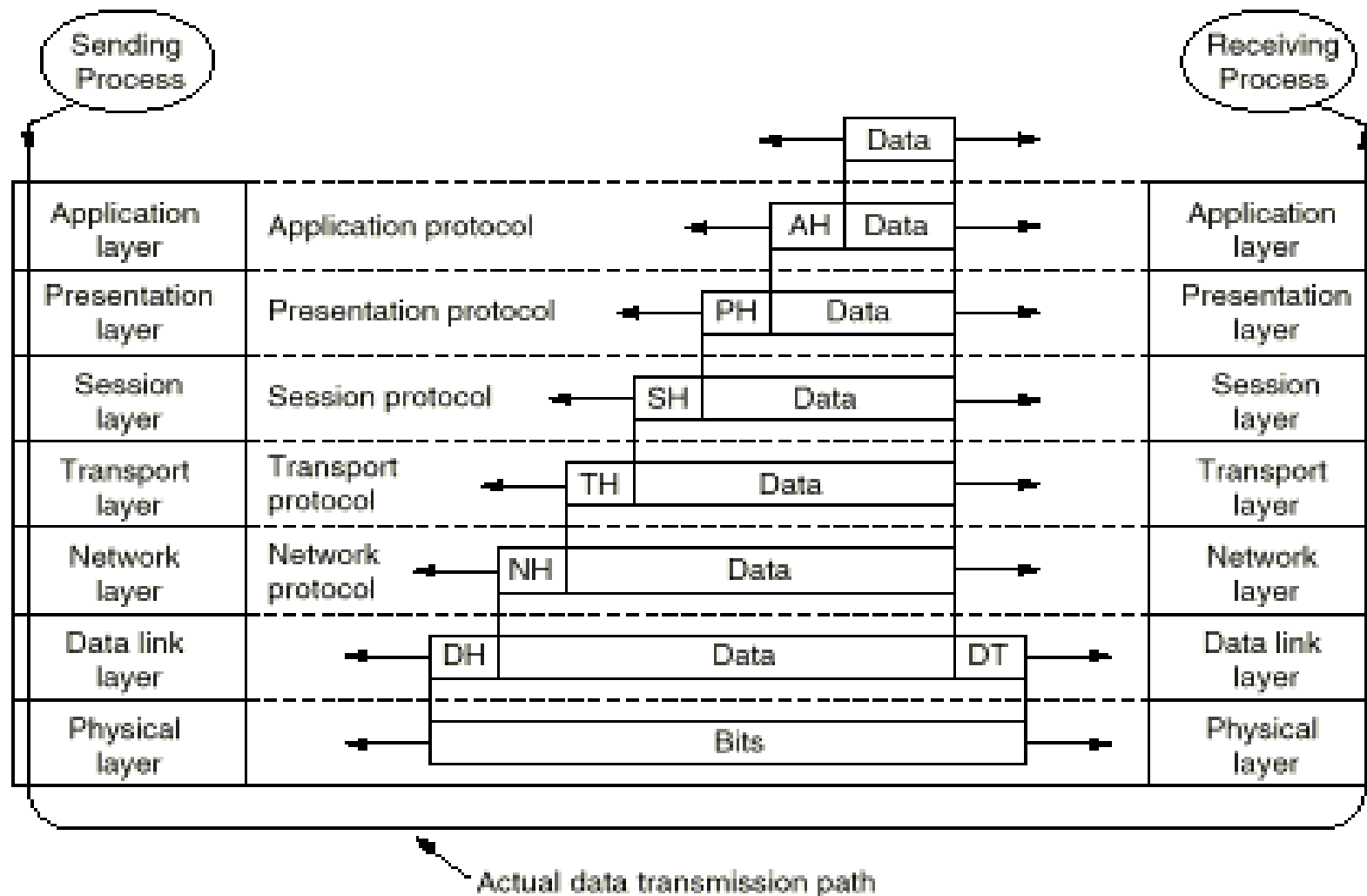


Fig. 1-17. An example of how the OSI model is used. Some of the headers may be null. (Source: H.C. Folts. Used with permission.)



TCP/IP 参考模型

- 以 **TCP/IP** 协议为核心的 **Internet** 网络体系结构
 - **TCP/IP** 参考模型把物理层和数据链路层合起来称为：**Host-to- Network**
 - 物理层：在物理线路上传输原始的二进制数据位
 - 数据链路层：在有差错的物理线路上提供无差错的数据传输
 - **Internet**层（网络层）：控制通信子网提供源点到目的点的 **IP** 包传送，实现异构网络互联
 - 运输层：提供端到端的数据传送服务。**TCP** 和 **UDP**
 - 应用层：提供各种 **Internet** 管理和应用服务功能



TCP/IP 参考模型（续）

■ TCP/IP 与 OSI 的比较

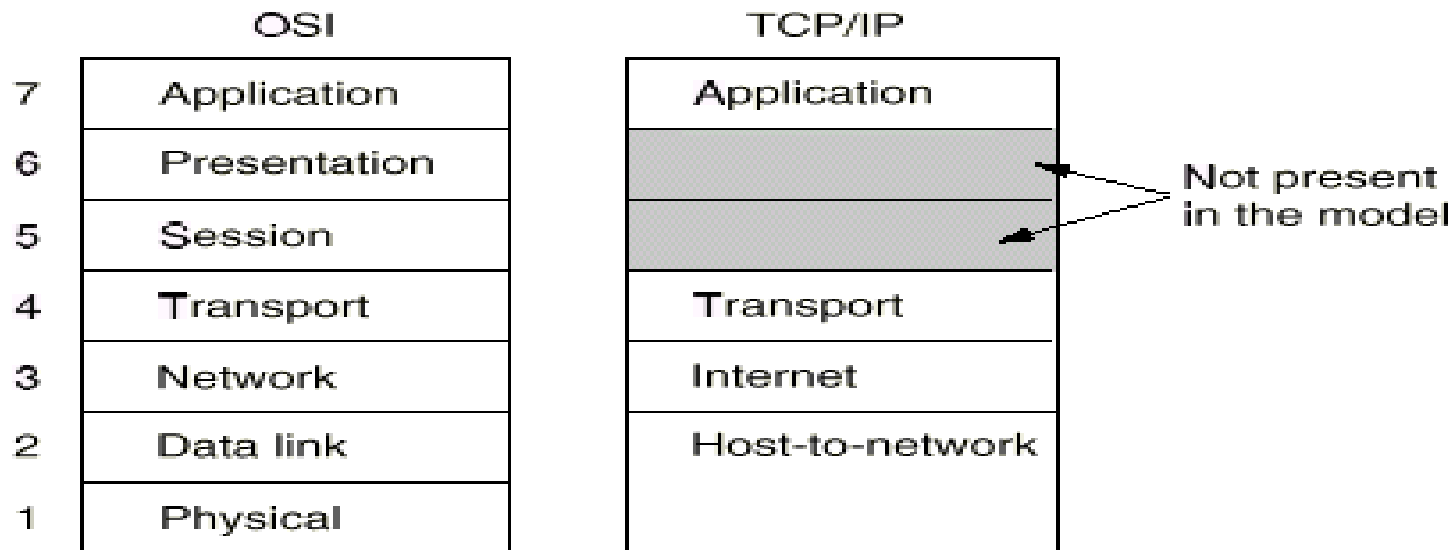


Fig. 1-18. The TCP/IP reference model.

- 思考：OSI参考模型中的会话层和表示层的功能在TCP/IP参考模型的哪层实现？

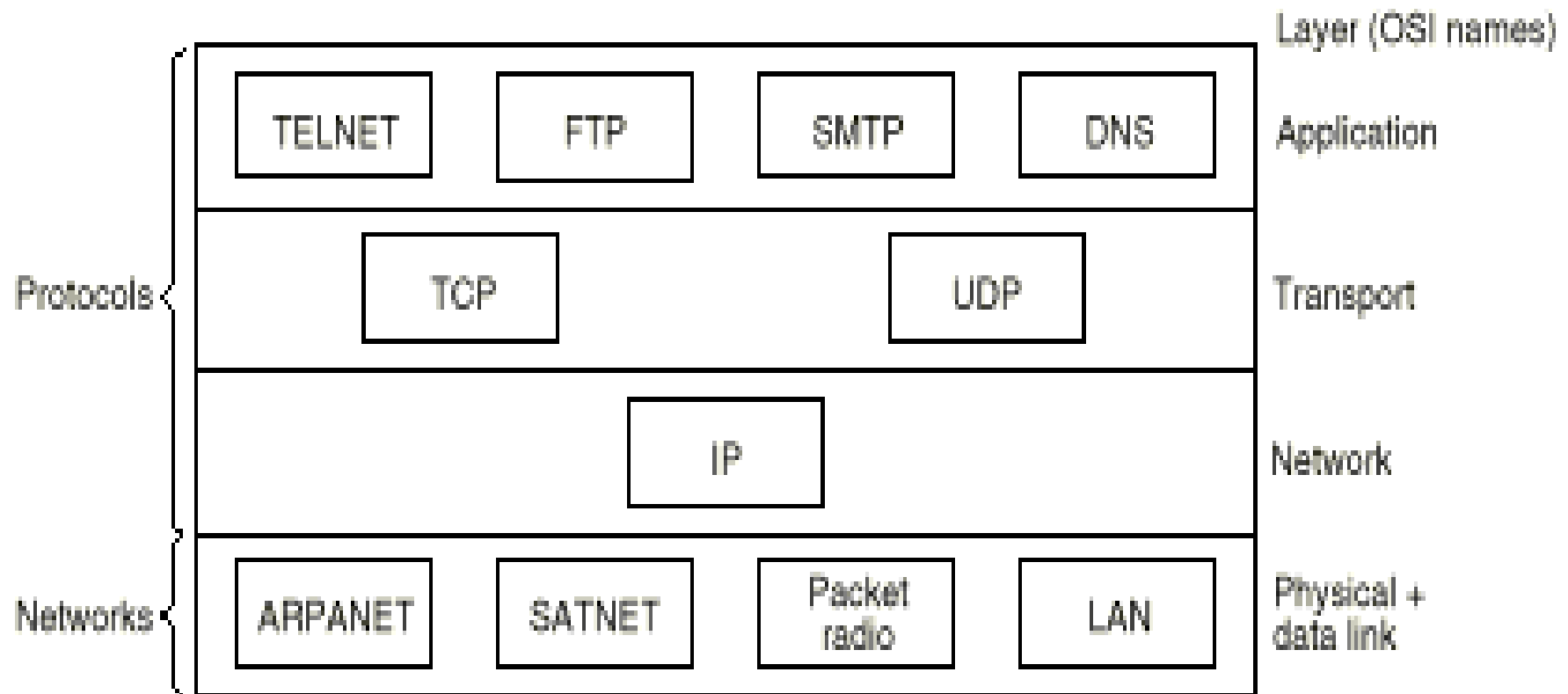


Fig. 1-19. Protocols and networks in the TCP/IP model initially.



5	Application layer
4	Transport layer
3	Network layer
2	Data Link layer
1	Physical layer

Fig. 1-21. The hybrid reference model to be used in this book.



OSI的历史经验和教训

- **OSI**是80年代计算机网络技术，网络体系结构的主流
- **OSI**网络体系结构的核心和贡献：
 - 分层模型
 - 服务、接口、协议
- **Andrew S. Tanenbaum** 在 “**Computer Networks**” 中评价**OSI**:
 - **Bad timing (too late)**
 - **Bad technology (both the model and the protocol are flawed)**
 - **Bad implementations (huge, unwieldy, and slow)**
 - **Bad politics (government and organizations bureaucrats)**
- 其他评价



主要内容

- 计算机网络的定义和组成
 - 计算机网络的定义
 - 计算机网络的组成
- 计算机网络体系结构
 - 计算机网络功能的分层
 - 协议和协议的分层结构
 - 计算机网络的体系结构
 - 分层原则
 - 端到端原则
- 典型计算机网络参考模型
 - 计算机网络的标准化
 - **OSI**参考模型
 - **TCP/IP**参考模型
- 其他网络体系结构
 - **Novell NetWare**
 - X.25分组交换网
 - **B-ISDN** 和 **ATM**



Novell NetWare

- **NetWare**是**Novell**公司开发的**PC**上的网络操作系统，**client-server**结构
- **1983**年发布，**2005**年终止开发
- **NetWare**的基本思想：文件共享
 - 而当时（**1983**年）其他系统采用磁盘共享



Novell NetWare (续)

- 基于**Xeror Network System (XNS)**，但有很多改进
 - 网络层协议，**IPX (Internetwork Packet eXchange)**：不可靠无连接协议，与**IP**类似，地址长度不同：**IPX**，**10**字节（**4**字节网络号，**6**字节机器号（**MAC**地址））；**IP**，**4**字节
 - 传输层协议，**NCP (NetWare Core Protocol)**，**SPX (Sequenced Packet eXchange)**：面向连接的协议

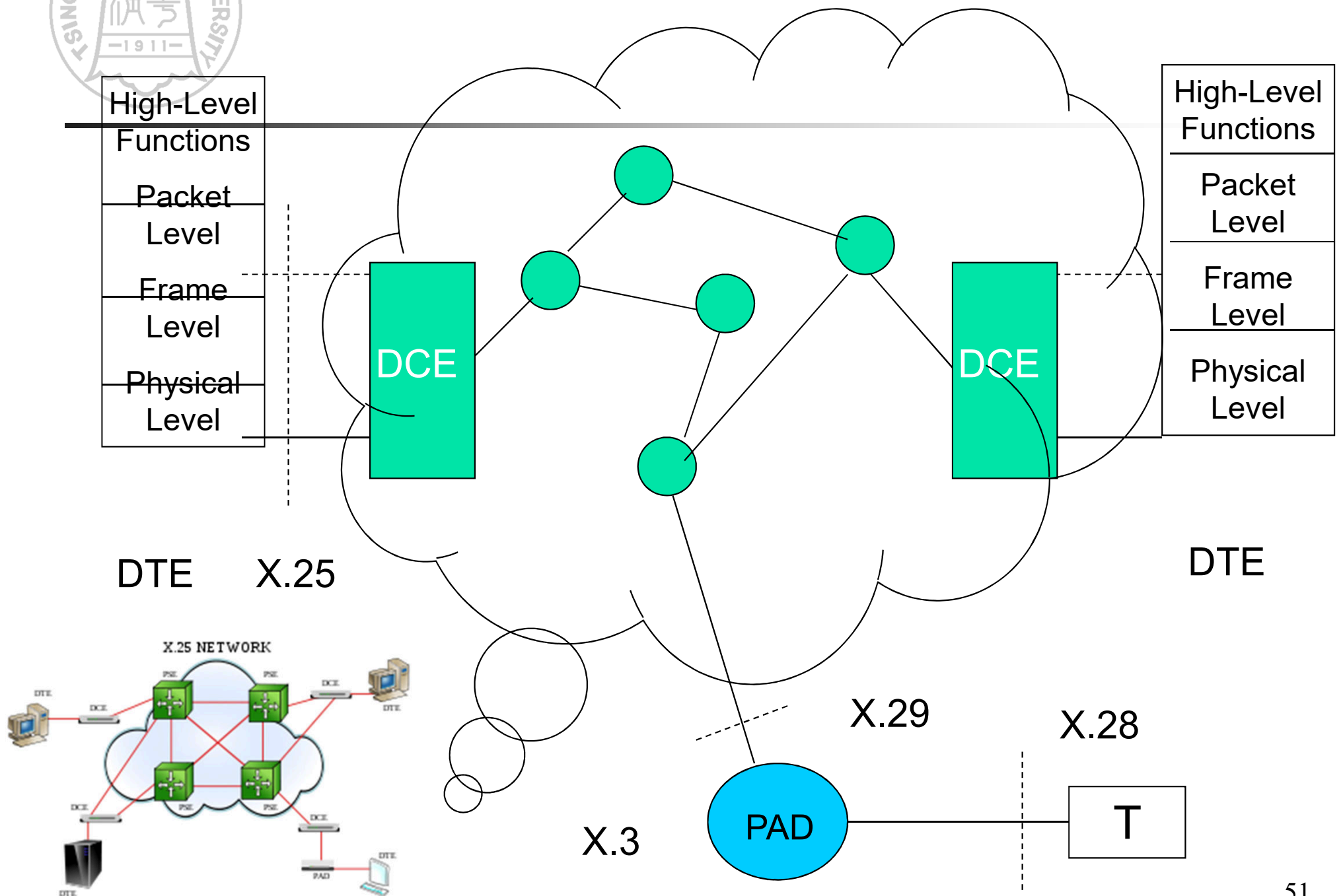
Layer			
Application	SAP	File server	...
Transport	NCP		SPX
Network	IPX		
Data link	Ethernet	Token ring	ARCnet
Physical	Ethernet	Token ring	ARCnet

Fig. 1-22. The Novell NetWare reference model.



X.25 分组交换网

- **70年代，CCITT推出X.25标准，为公用包交换网和用户之间提供接口，早于ISO/OSI参考模型**
- **X.25面向连接，支持交换虚电路和永久虚电路**
- **定义了三层协议**
 - 物理层协议：**X.21, X.3 / X.28 / X.29**
 - 数据链路层协议：**LAP, LAPB**
 - 网络层协议：**PLP**
- **DTE: Digital Terminal Equipment**
- **DCE: Digital Circuit Terminating Equipment**
- **PAD: Packet Assembler and Disassembler**





B-ISDN 和 ATM

- 宽带综合业务数字网 **B-ISDN**
(**Broadband Integrated Services Digital Network**)
- 产生背景
 - 多种网络共存 (**POTS, Telex, SMDS, DQDB, Frame Relay, ...**)，电信公司想统一成一个网络**B-ISDN**
- **B-ISDN**的技术基础是异步传输模式**ATM**
(**Asynchronous Transfer Mode**)



B-ISDN 和 ATM (续)

■ ATM

- 异步传输，没有主时钟
- 传输单元是短的、定长的包，称为信元 (**cell**)
- 面向连接
- 速率主要有两种：**155M，622M**



Fig. 1-29. An ATM cell.



B-ISDN 和 ATM (续)

■ B-ISDN ATM参考模型

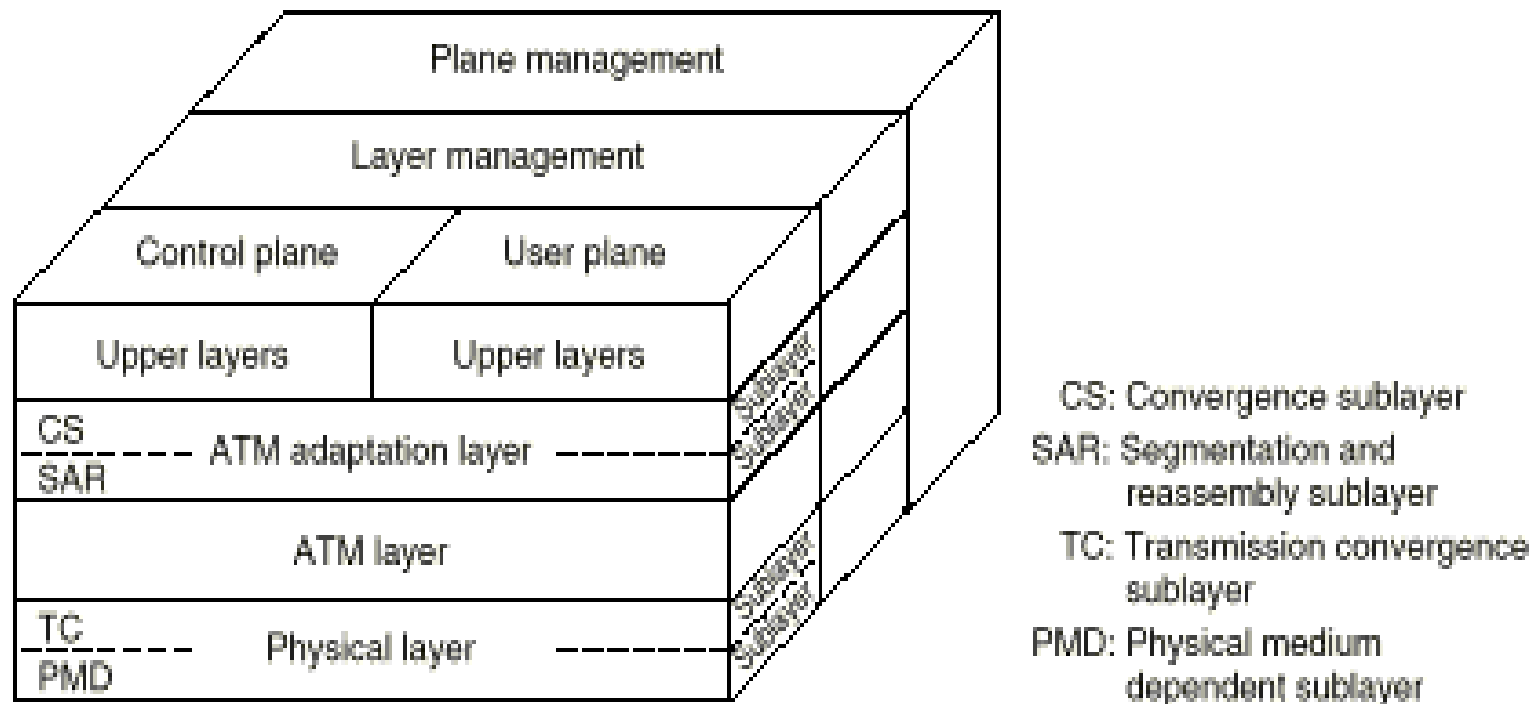
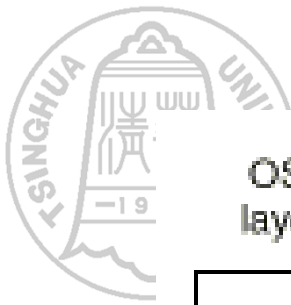


Fig. 1-30. The B-ISDN ATM reference model.



OSI layer	ATM layer	ATM sublayer	Functionality
3/4	AAL	CS	Providing the standard interface (convergence)
		SAR	Segmentation and reassembly
2/3	ATM		Flow control Cell header generation/extraction Virtual circuit/path management Cell multiplexing/demultiplexing
2	Physical	TC	Cell rate decoupling Header checksum generation and verification Cell generation Packing/unpacking cells from the enclosing envelope Frame generation
1		PMD	Bit timing Physical network access

Fig. 1-31. The ATM layers and sublayers, and their functions.



总结

- 计算机网络的构成：资源子网和通信子网
 - 通信子网：点到点通道，关键技术是路由选择；广播通道，关键技术是通道分配
- 计算机网络体系结构
 - 功能的分层，层次结构
 - 对等实体、协议、服务、接口、服务原语
 - **SAP, SDU, IDU, PDU**
 - 分层原则和端到端原则



总结（续）

- 典型计算机网络参考模型
 - 标准化组织
 - **OSI参考模型**
 - **TCP/IP参考模型**
- 其它网络体系结构
 - **Novell NetWare**
 - **X.25**
 - **B-ISDN和ATM**

第三章

数据通信基本原理



主要内容

■ 数据通信基础理论

- 傅立叶分析
- 有限带宽信号
- 信道的最大数据传输速率

■ 数据通信技术

- 数据通信系统的基本结构
- 传输和传输方式
- 数据编码技术
- 多路复用技术
- 交换技术



数据通信基础理论

■ 主要内容

- 研究信号在通信信道上传输时的数学表示及其所受到的限制

■ 傅立叶分析

- 在网络通信中，信息是以电磁信号（或简称信号）的形式传输的
- 电磁信号是时间的函数（时域观）
- 也可以表示成频率的函数（频域观）
- 对于理解数据传输来讲，信号的频域观比时域观更重要



数据通信基础理论（续）

■ 时域观

- 从时间函数的角度来看，电磁信号分为模拟信号和数字信号
- 模拟信号的信号强度随着时间平滑变化，或者说信号中没有突变或不连续的地方。
- 数字信号的信号强度在一段时间内保持一个恒定值，然后又变成另外一个恒定值。

■ 频域观

■ 基本定义

- 当一个信号的所有频率成分是某一个频率的整数倍时，该频率被称为基本频率
- 信号的周期等于基本频率的周期

■ 傅立叶分析



傅立叶分析

■ 傅立叶分析

- 任何一个周期为**T**的有理周期性函数 **g(t)** 可分解为若干项（可能无限多项）正弦和余弦函数之和

$$g(t) = \frac{1}{2} c + \sum_{n=1}^{\infty} a_n \sin(2\pi nft) + \sum_{n=1}^{\infty} b_n \cos(2\pi nft)$$

$$f = 1/T$$

基本频率

$$a_n, b_n$$

n次谐波项的正弦和余弦振幅值



傅立叶分析 (续)

- 已知 $g(t)$, 求 c , a_n , b_n

- 将等式两边从 0 到 T 积分可得 c

$$c = \frac{2}{T} \int_0^T g(t) dt$$

- 用 $\sin(2\pi kft)$ 乘等式两边, 并从 0 到 T 积分, 可得 a_n

$$a_n = \frac{2}{T} \int_0^T g(t) \sin(2\pi nft) dt$$

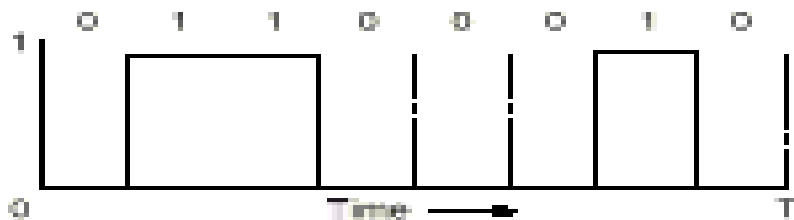
- 用 $\cos(2\pi kft)$ 乘等式两边, 并从 0 到 T 积分, 可得 b_n

$$b_n = \frac{2}{T} \int_0^T g(t) \cos(2\pi nft) dt$$



傅立叶分析（续）

- 对于二进制编码 0 1 1 0 0 0 1 0，其输出电压波形为：



$$g(t) = \begin{cases} 0 & 0 < t \leq \frac{T}{8} \\ 1 & \frac{T}{8} < t \leq \frac{3T}{8} \\ 0 & \frac{3T}{8} < t \leq \frac{6T}{8} \\ 1 & \frac{6T}{8} < t \leq \frac{7T}{8} \\ 0 & \frac{7T}{8} < t < T \end{cases}$$



傅立叶分析 (续)

- 其傅立叶分析的系数为

- $a_n = \frac{1}{\pi n} [\cos(\pi n/4) - \cos(3 \pi n/4) + \cos(6 \pi n/4) - \cos(7 \pi n/4)]$

- $b_n = \frac{1}{\pi n} [\sin(3\pi n/4) - \sin(\pi n/4) + \sin(7 \pi n/4) - \sin(6 \pi n/4)]$

- $c = 3/8$



傅立叶分析（续）

- 根据傅立叶分析，任何电磁信号可以由若干具有不同振幅、频率和相位的周期模拟信号（正弦波）组成
- 反过来，只要有足够的具有适当振幅、频率和相位的正弦波，就可以构造任何一个信号

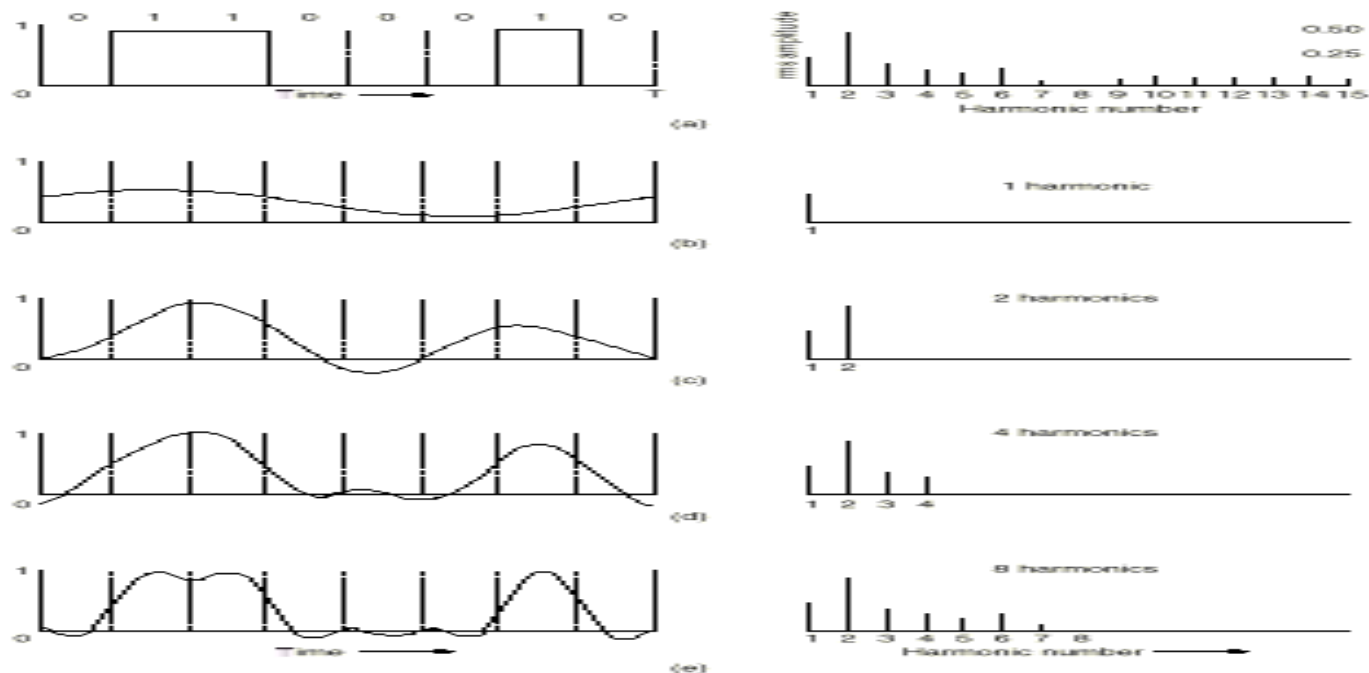


Fig. 2-1. (a) A binary signal and its root-mean-square Fourier amplitudes. (b)-(e) Successive approximations to the original signal.



有限带宽信号

- 频谱（spectrum）是一个信号所包含的频率的范围
 - 图2-1 (e) 中信号的频谱从 f 到 $8f$
- 信号的绝对带宽等于频谱的宽度
 - 图2-1 (e) 中信号的带宽为 $8f - f = 7f$
- 许多信号的带宽是无限的，然而信号的主要能量集中在相对窄的频带内，这个频带被称为有效带宽，或带宽（bandwidth）
- 信号的信息承载能力与带宽有直接关系，带宽越宽，信息承载能力越强



有限带宽信号（续）

- 信号在信道上传输时的特性
 - 对不同傅立叶分量的衰减不同，引起输出失真
 - 信道有截止频率 f_c , $0 \sim f_c$ 的振幅衰减较弱, f_c 以上的振幅衰减厉害，这主要由信道的物理特性决定, $0 \sim f_c$ 是信道的有限带宽
 - 实际使用时，可以接入滤波器，限制用户的带宽
 - 通过信道的谐波次数越多，信号越逼真



有限带宽信号（续）

■ 波特率（**baud**）和比特率（**bit**）的关系

- 波特率：每秒钟信号变化的次数，也称调制速率
- 比特率：每秒钟传送的二进制位数
- 波特率与比特率的关系取决于信号值与比特位的关系
 - 例：每个信号值可表示**3**位，则比特率是波特率的**3**倍；
每个信号值可表示**1**位，则比特率和波特率相同



有限带宽信号（续）

- 对于比特率为 B bps的信道，发送8位所需的时间为 $8/B$ 秒，若8位为一个周期 T ，则一次谐波的频率是： $f_1 = B/8$ Hz
- 能通过信道的最高次谐波数目为： $N = f_c / f_1$
 - 音频线路的截止频率为3000Hz
$$N = f_c / f_1 = 3000 / (B/8) = 24000/B$$



有限带宽信号 (续)

Bps	T (msec)	First harmonic (Hz)	# Harmonics sent
300	26.67	37.5	80
600	13.33	75	40
1200	6.67	150	20
2400	3.33	300	10
4800	1.67	600	5
9600	0.83	1200	2
19200	0.42	2400	1
38400	0.21	4800	0

Fig. 2-2. Relation between data rate and harmonics.

- **结论：** 即使对于完善的信道，有限的带宽限制了数据的传输速率



信道的最大数据传输速率

- **1924年，奈魁斯特(H. Nyquist)推导出无噪声有限带宽信道的最大数据传输率公式**
- **奈魁斯特定律**
 - **最大数据传输率 = $2H\log_2 V$ (bps)**
 - **任意信号通过一个带宽为H的低通滤波器，则每秒采样 $2H$ 次就能完整地重现该信号，信号电平分为 V 级**



信道的最大数据传输速率（续）

- 1948年，香农(C. Shannon)把奈魁斯特的的工作扩大到信道受到随机（热）噪声干扰的情况
- 随机噪声出现的大小用信噪比（信号功率**S**与噪声功率**N**之比）来衡量， $10\log_{10}S/N$ ，单位：分贝
 - 电话系统的典型信噪比为**30db**
- 香农定律
 - 带宽为 **H** 赫兹，信噪比为**S/N**的任意信道的最大数据传输率为： **$H\log_2(1 + S/N)$ (bps)**
 - 此式是利用信息论得出的，具有普遍意义
 - 与信号电平级数、采样速度无关
 - 此式仅是上限，难以达到



主要内容

■ 数据通信基础理论

- 傅立叶分析
- 有限带宽信号
- 信道的最大数据传输速率

■ 数据通信技术

- 数据通信系统的基本结构
- 传输和传输方式
- 数据编码技术
- 多路复用技术
- 交换技术

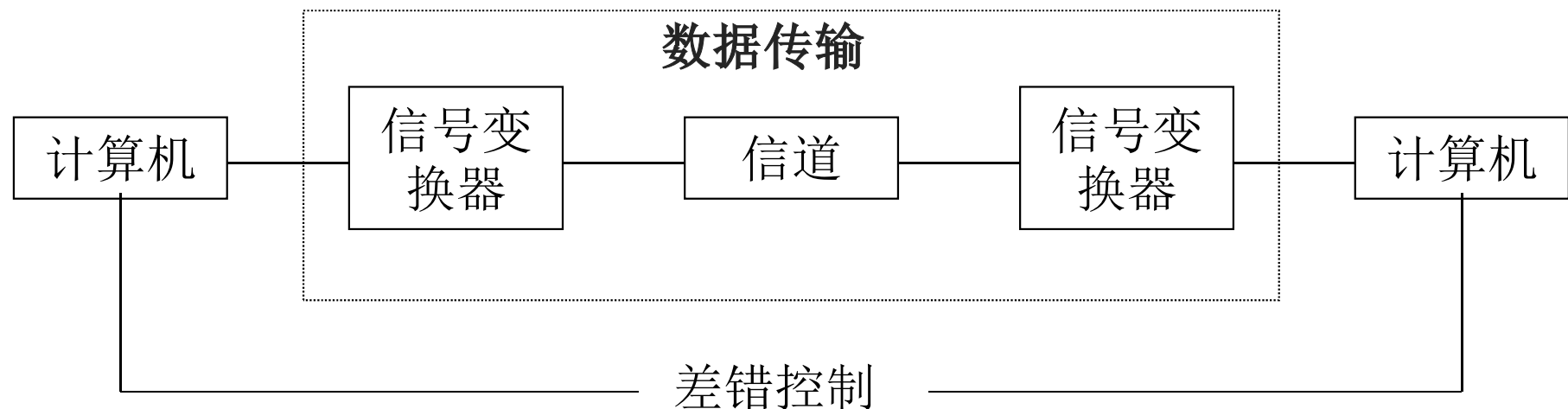


数据通信技术

■ 主要内容

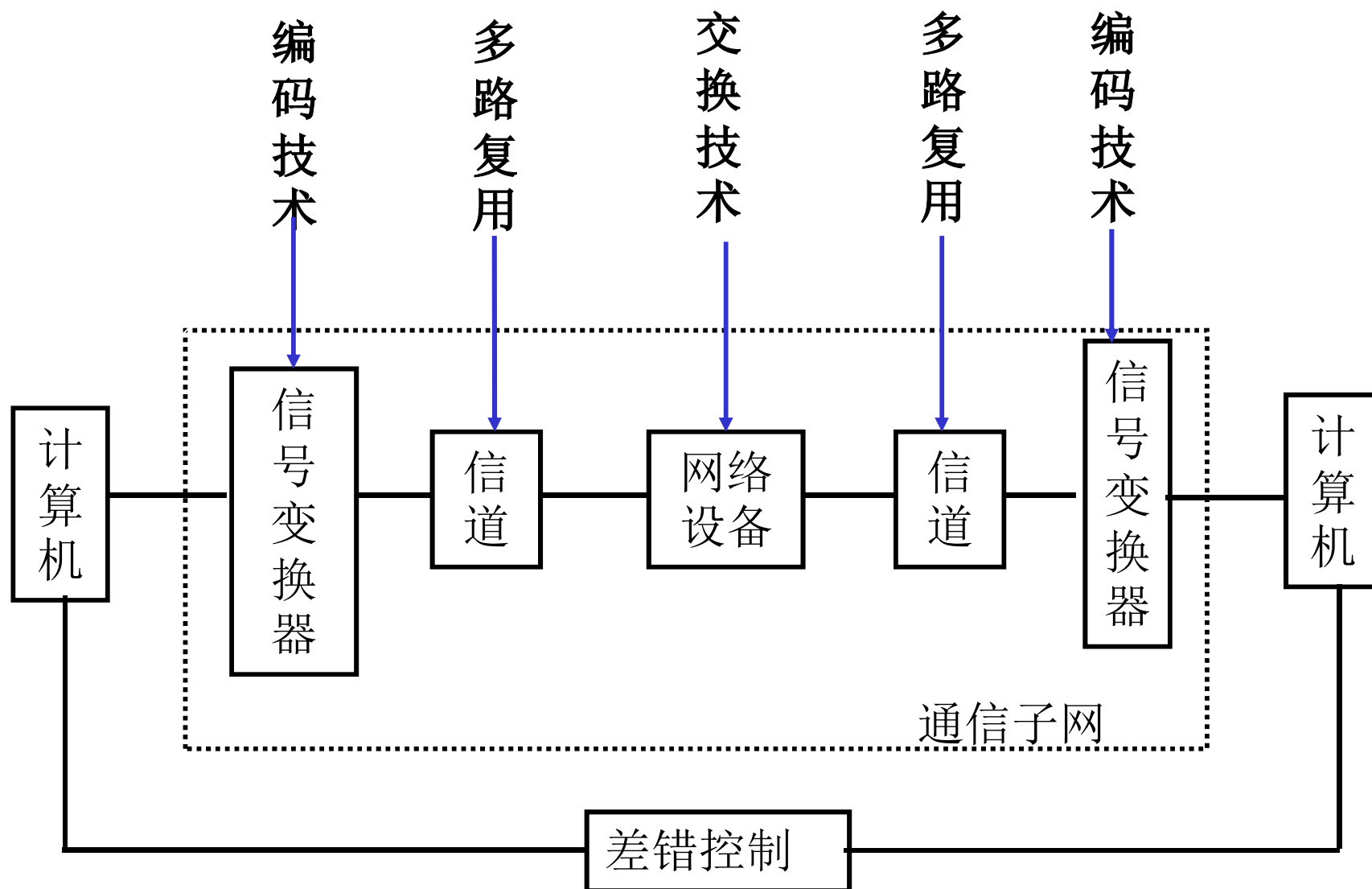
- 研究数据在通信信道上的各种传输方式及其所采用的技术

■ 数据通信系统的基本结构





数据通信技术





传输和传输方式

- 数字传输 / 模拟传输
- 并行传输 / 串行传输
- 点到点传输 / 点到多点传输
- 单工、半双工和全双工传输
- 同步传输 / 异步传输

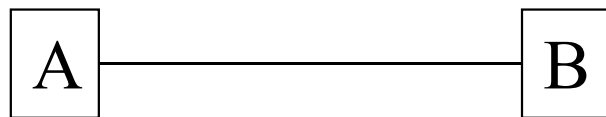


点到点传输/点到多点传输

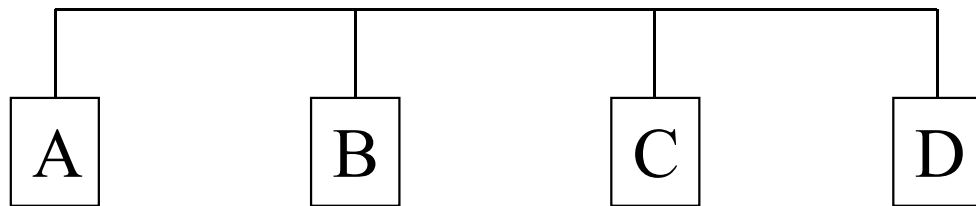
■ 连接方式

- 为适应不同的需要，通信线路采用不同的连接方式

- 点到点传输



- 点到多点传输

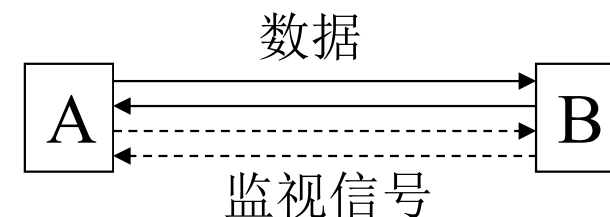
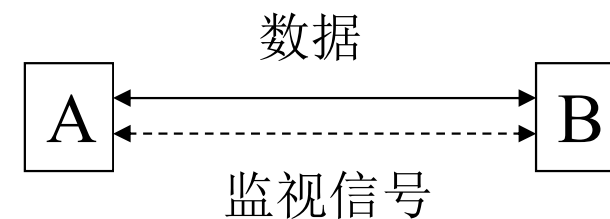
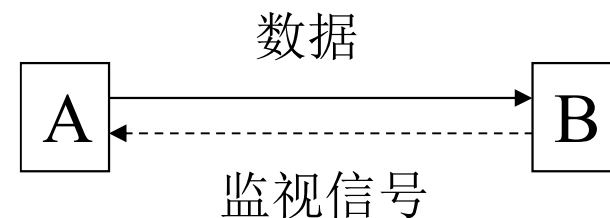




单工、半双工和全双工传输

■ 从信息传送方向和时间的关系角度研究

- 单工传输：信息只能单向传输，监视信号可回送
- 半双工传输：信息可以双向传输，但在某一时刻只能单向传输
- 全双工传输：信息可以同时双向传输





同步传输/异步传输

■ 同步方式

- 目的：接收方必须知道每一位信号的开始及其持续时间，以便正确的采样接收
- 以字符传输（字符为基本传输单位）为例，在基于字符的信息传送中，可以采用异步传输，也可以采用同步传输



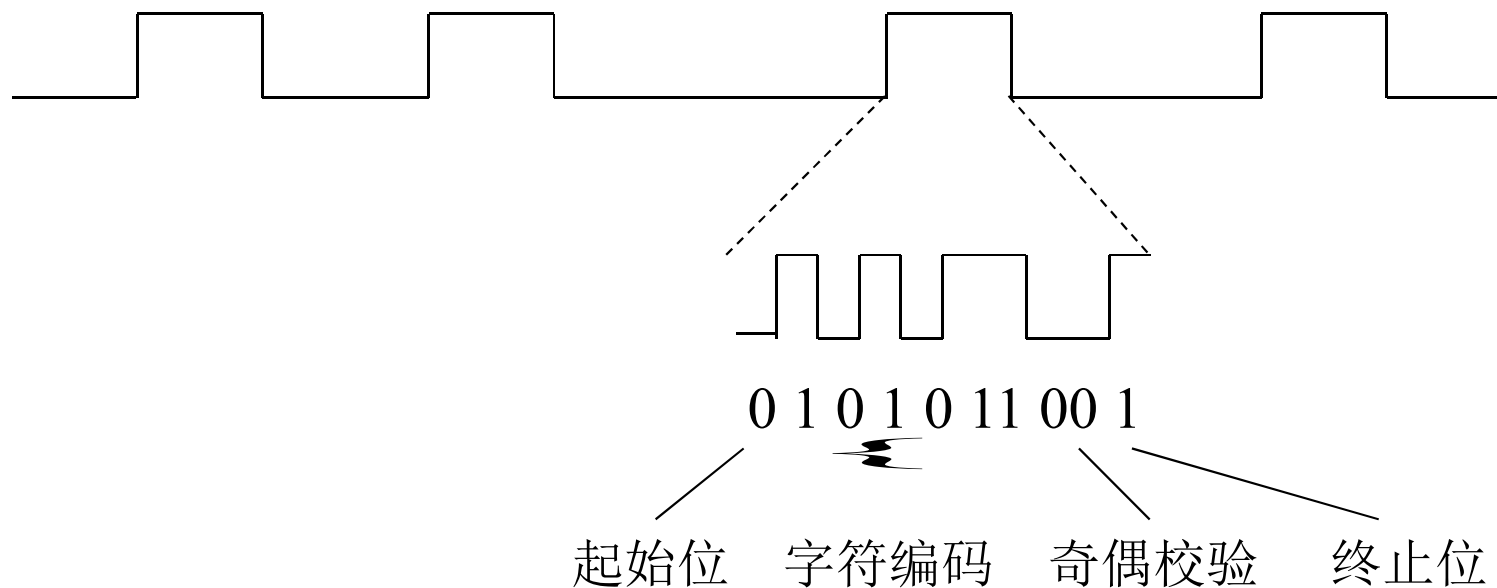
同步传输/异步传输（续）

■ 异步传输（以字符传输为例）

- 信息传送以字符为单位
- 每个字符由发送方异步产生，有随机性
- 字符一般采用5，6，7或8位二进制编码
- 需要辅助位，每个字符可能需要用10位或11位才能传送，例如：
 - 起始位，**1**位
 - 字符编码，**7**位
 - 奇偶校验位，**1**位
 - 终止位，**1 ~ 2**位
- 特点
 - 传输效率低
 - 主要用于字符终端与计算机之间的通信



同步传输/异步传输（续）





同步传输/异步传输（续）

■ 同步传输（以字符传输为例）

- 信息传送以报文为单位
- 传输开始时，以同步字符使收发双方同步
- 从传输信息中抽取同步信息，修正同步，保证正确采样
- 特点
 - 可以不间断地传输信息，传输效率较高
 - 字符间减少了辅助信息
 - 传输的信息中不能有同步字符出现，需要透明传输处理

SYN	SYN	信	息	SYN	SYN
-----	-----	---	---	-----	-----



同步传输/异步传输（续）

- 基于位的传输，一般采用同步传输
 - 信息以二进制位流为单位传送
 - 传输过程中收发双方以位为单位同步
 - 传输的开始和结束由特定的八位二进制位同步
 - 特点：传输效率高

标记	二进制位流	标记
----	-------	----



数据编码技术

■ 数据表示

- 模拟数据 (**Analog Data**), 连续值
- 数字数据 (**Digital Data**), 离散值

■ 数据传输方式

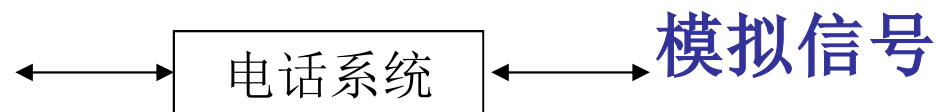
- 以信号作为载体
- 模拟信号 (**Analog Signals**)
- 数字信号 (**Digital Signals**)



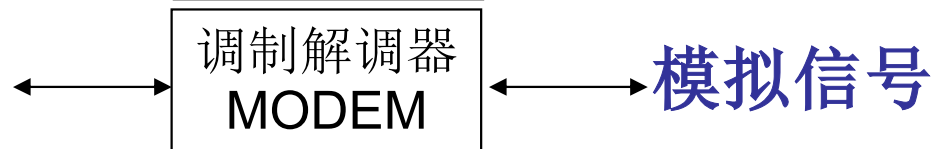
信号发送方式

- 模拟信号发送（模拟信道）

- 模拟数据(声音)

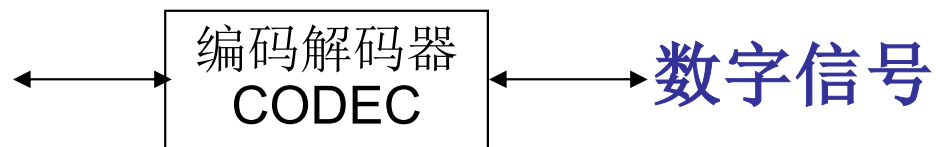


- 数字数据(二进制脉冲)

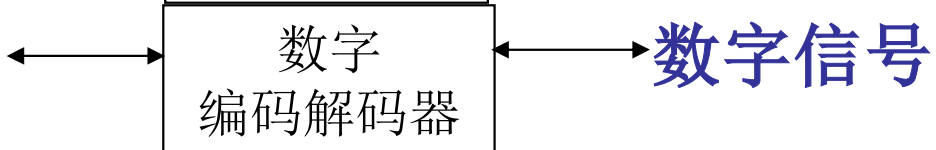


- 数字信号发送（数字信道）

- 模拟数据



- 数字数据(二进制脉冲)



- 数字信号发送的优点是：价格便宜，对噪声不敏感；缺点是：易受衰减，频率越高，衰减越厉害



数据编码技术

- 研究数据在信号传输过程中如何进行编码(变换)
- 数字数据的数字传输（基带传输）
 - 基带：基本频带，指传输变换前所占用的频带，是原始信号所固有的频带
 - 基带传输：在传输时直接使用基带信号
 - 基带传输是一种最简单最基本的传输方式，一般用低电平表示“0”，高电平表示“1”
 - 适用范围：低速和高速的各种情况
 - 限制：因基带信号所带的频率成分很宽，所以对传输线有一定的要求

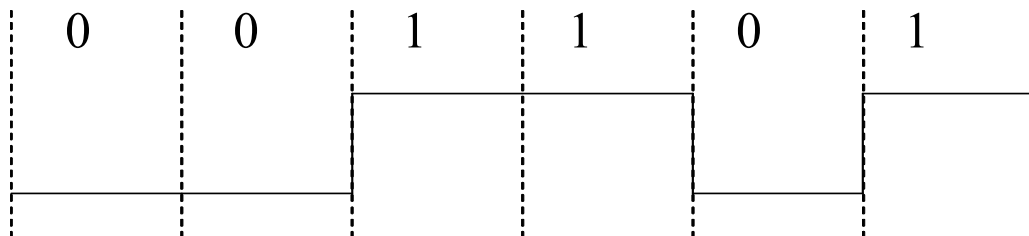


数据编码技术（续）

■ 常用的几种编码方式

■ 不归零制码（**NRZ: Non-Return to Zero**）

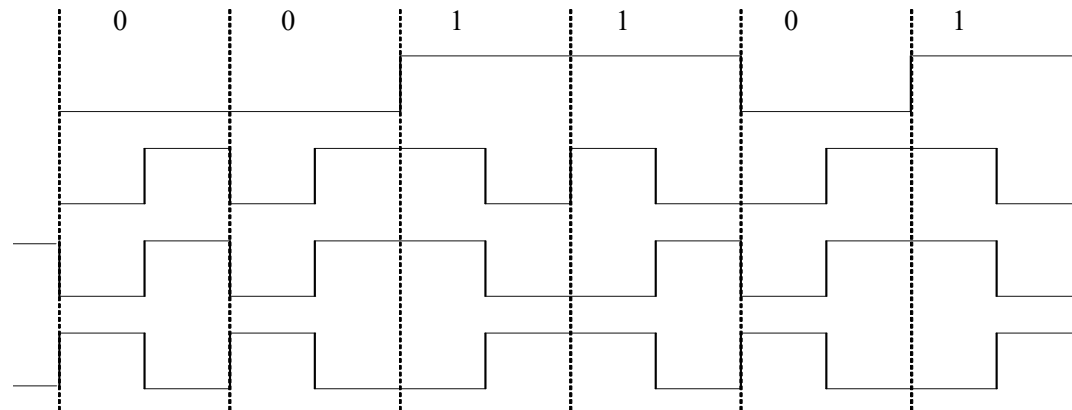
- 原理：用两种不同的电平分别表示二进制信息“0”和“1”，低电平表示“0”，高电平表示“1”
- 缺点：
 - 难以分辨一位的结束和另一位开始
 - 发送方和接收方必须有时钟同步
 - 若信号中“0”或“1”连续出现，信号直流分量将累加
- 结论：容易产生传播错误





数据编码技术（续）

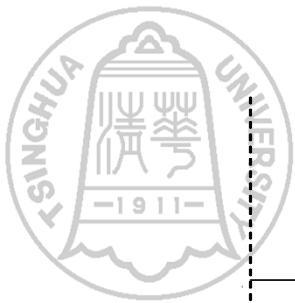
- 曼彻斯特码（**Manchester**），也称相位编码
 - 原理：每一位中间都有一个跳变，从低跳到高表示“0”，从高跳到低表示“1”
 - 优点：克服了NRZ码的不足。每位中间的跳变即可作为数据，又可作为时钟，能够自同步
- 差分曼彻斯特码（**Differential Manchester**)
 - 原理：每一位中间都有一个跳变，每位开始时有跳变表示“0”，无跳变表示“1”。位中间跳变表示时钟，位前跳变表示数据
 - 优点：时钟、数据分离，便于提取





数据编码技术（续）

- 逢“**1**”变化的**NRZ**码
 - 原理：在每位开始时，逢“**1**”电平跳变，逢“**0**”电平不跳变
- 逢“**0**”变化的**NRZ**码
 - 原理：在每位开始时，逢“**0**”电平跳变，逢“**1**”电平不跳变



0

0

1

1

0

1

NRZ

曼彻斯特

差分曼彻斯特

逢“1”变化NRZ

逢“0”变化NRZ



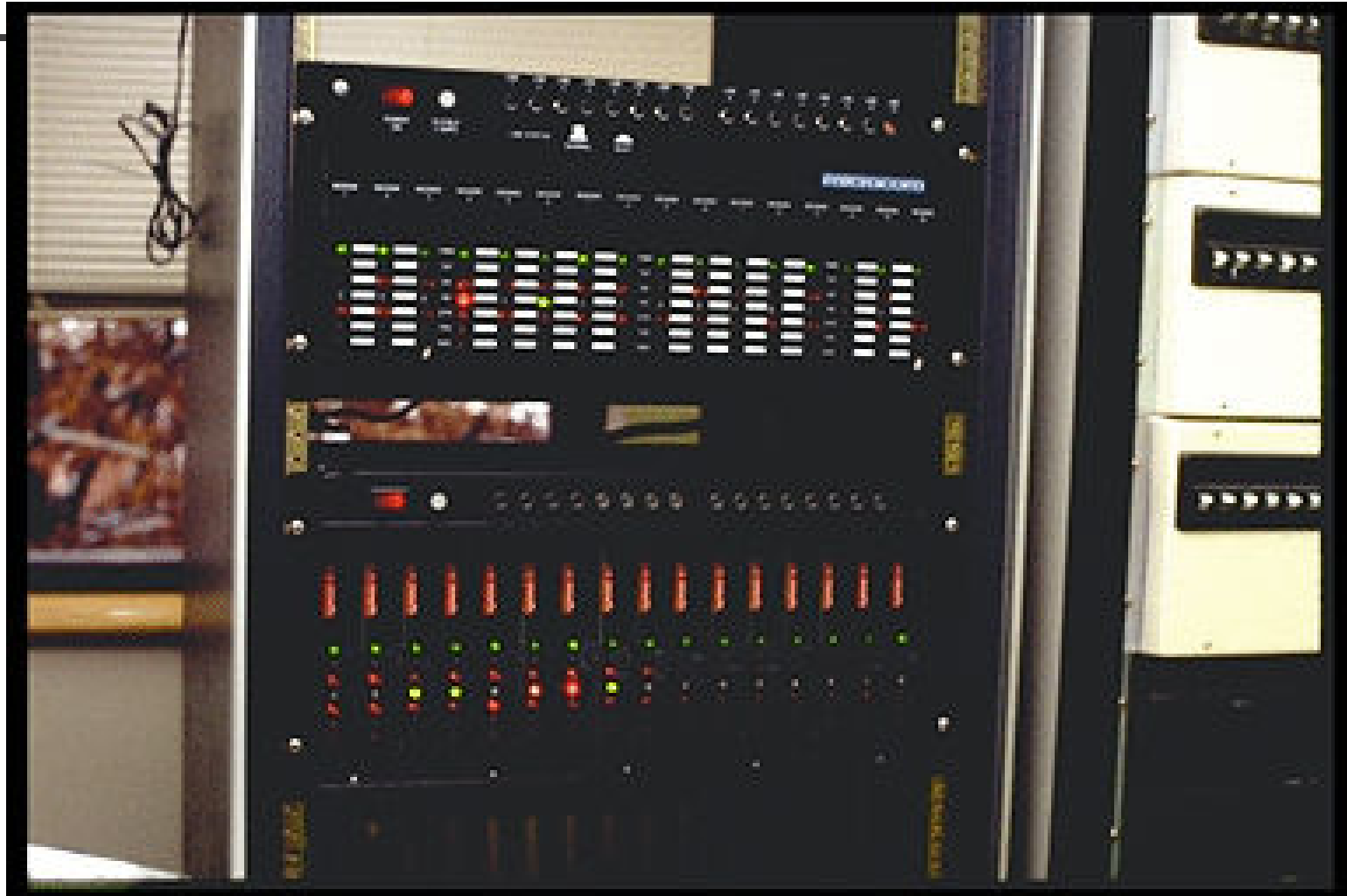
数字数据的模拟传输

- 数字数据的模拟传输，也称频带传输
 - 指在一定频率范围内的线路上，进行载波传输。用基带信号对载波进行调制，使其变为适合于线路传送的信号
 - 调制（**Modulation**）：用基带脉冲对载波信号的某些参量进行控制，使这些参量随基带脉冲变化
 - 解调（**Demodulation**）：调制的反变换
 - 调制解调器**MODEM**（**m**odulation-**d**emodulation）



- **Modem: RS-232**接口用来连接电脑，**RJ-11**用来连接电话线











数字数据的模拟传输（续）

- 根据载波 $A\sin(\omega t + \varphi)$ 的三个特性：幅度、频率、相位，产生常用的三种调制技术：

- 幅移键控法（调幅） **Amplitude-shift keying (ASK)**

- 幅移就是把频率、相位作为常量，而把振幅作为变量，即：

$$\begin{cases} \omega(t) = \omega_0 \\ \varphi(t) = \varphi_0 \\ A(t) = A_1, A_2, \dots, A_N \end{cases}$$

- $A(t)$ 取不同的值表示不同的信息码。例如： $A(t)$ 取 A_1 ， A_2 ， A_1 表示“0”， A_2 表示“1”



数字数据的模拟传输（续）

■ 频移键控法（调频） **Frequency-shift keying (FSK)**

- 频移就是把振幅、相位作为常量，而把频率作为变量，即：

$$\begin{cases} A(t) = A_0 \\ \varphi(t) = \varphi_0 \\ \omega(t) = \omega_1, \omega_2, \Lambda \omega_N \end{cases}$$

- $\omega(t)$ 取不同的值表示不同的信息码。例如： $\omega(t)$ 取 ω_1 , ω_2 , ω_1 表示“0”， ω_2 表示“1”



数字数据的模拟传输（续）

■ 相移键控法（调相） Phase-shift keying (PSK)

- 相移就是把振幅、频率作为常量，而把相位作为变量，即：

$$\begin{cases} A(t) = A_0 \\ \omega(t) = \omega_0 \\ \varphi(t) = \varphi_1, \varphi_2, \dots, \varphi_N \end{cases}$$

- $\varphi(t)$ 取不同的值表示不同的信息码。例如： $\varphi(t)$ 取 φ_1 , φ_2 , φ_1 表示“0”， φ_2 表示“1”

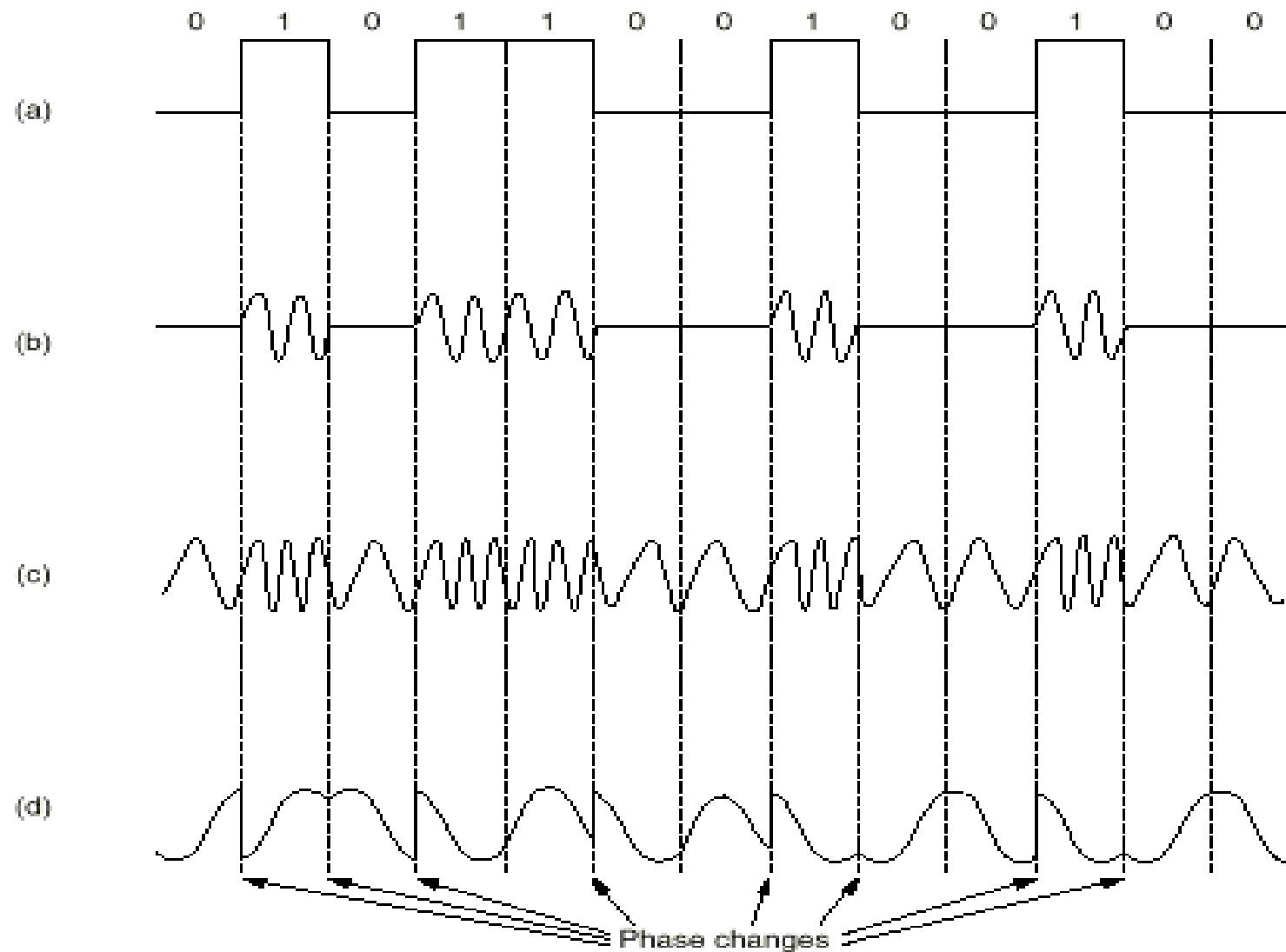


Fig. 2-18. (a) A binary signal. (b) Amplitude modulation. (c) Frequency modulation. (d) Phase modulation.



模拟数据的数字传输

- 解决模拟数据数字化问题，也称为脉冲代码调制**PCM**（**Pulse Code Modulation**）
- 根据**Nyquist**原理进行采样
- 常用的**PCM**技术
 - 将模拟信号振幅分成多级（ 2^n ），每一级用 n 位表示
 - 例如：贝尔系统的 **T1** 载波将模拟信号分成**128**级，每次采样用**7**位二进制数表示



模拟数据的数字传输（续）

■ 差分脉冲代码调制

- 原理：不是将振幅值数字化，而是根据前后两个采样值的差进行编码，输出二进制数字

■ δ 调制

- 原理：根据每个采样值与前一个值之间的差来决定输出二进制“1”或“0”
- 缺点：编码速度跟不上变化太快的信号

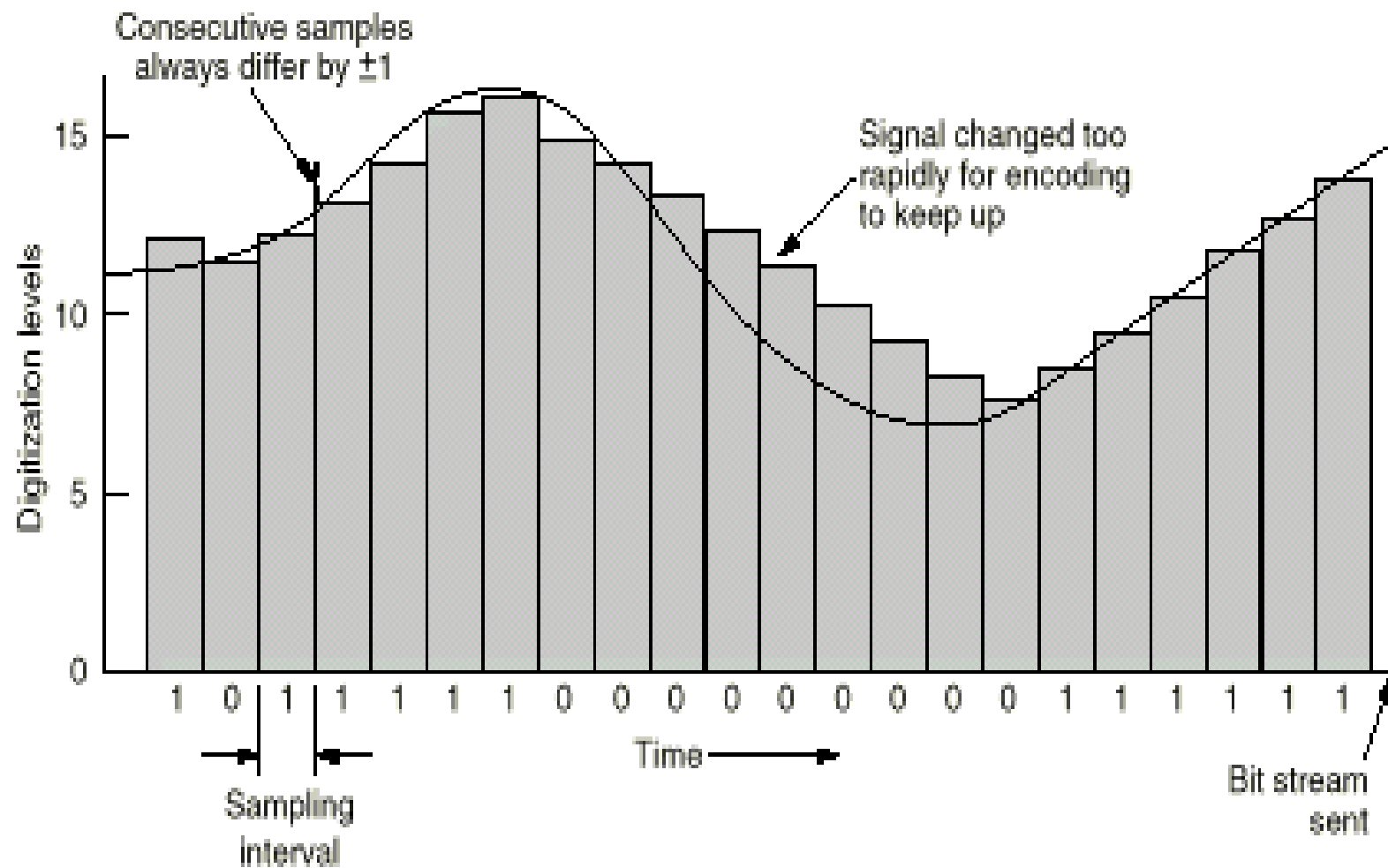


Fig. 2-27. Delta modulation.



Customer premises
equipment

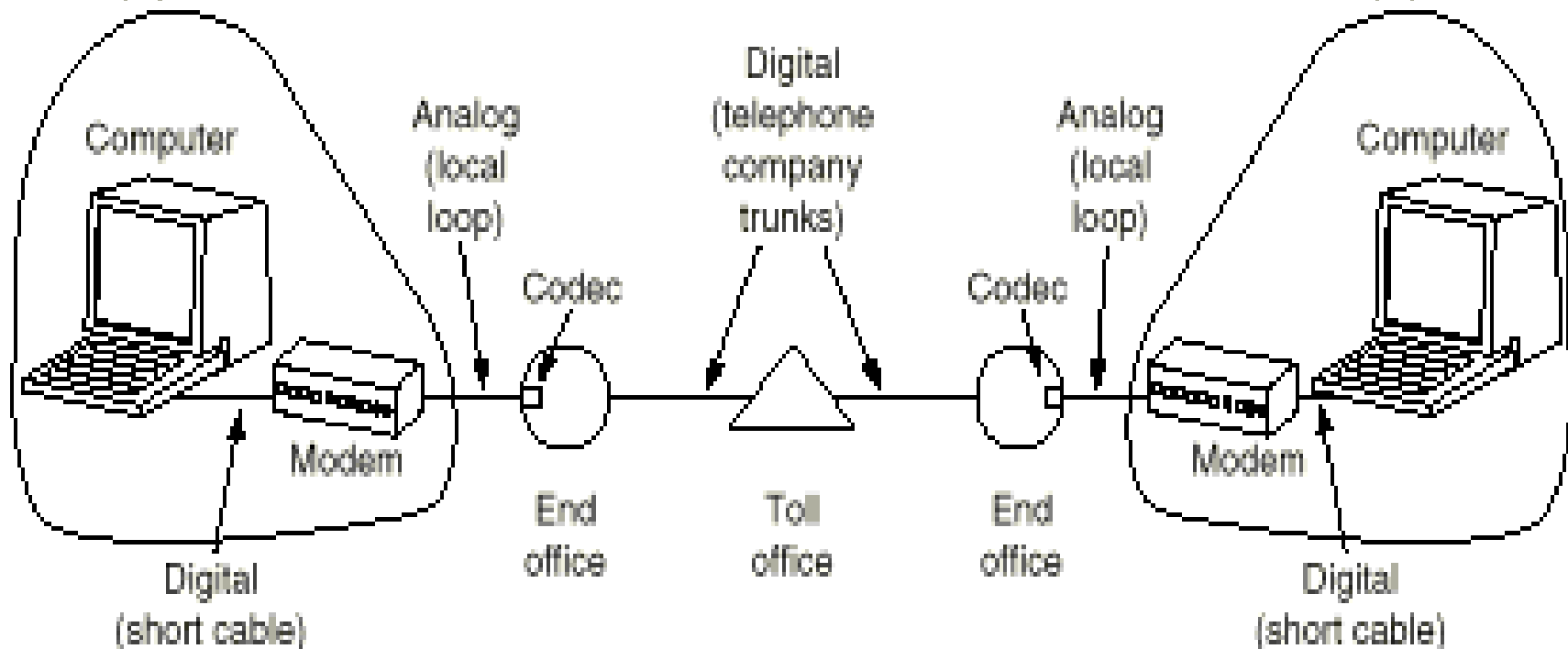


Fig. 2-17. The use of both analog and digital transmission for a computer to computer call. Conversion is done by the modems and codecs.



多路复用技术

- 由于一条传输线路的能力远远超过传输一个用户信号所需的能力，为了提高线路利用率，经常让多个信号同时共用一条物理线路
- 常用的有三种方法
 - 时分复用 **TDM (Time Division Multiplexing)**
 - T1载波，分成 24 个信道
 - 频分复用 **FDM (Frequency Division Multiplexing)**
 - 波分复用 **WDM (Wavelength Division Multiplexing)**



时分复用 TDM

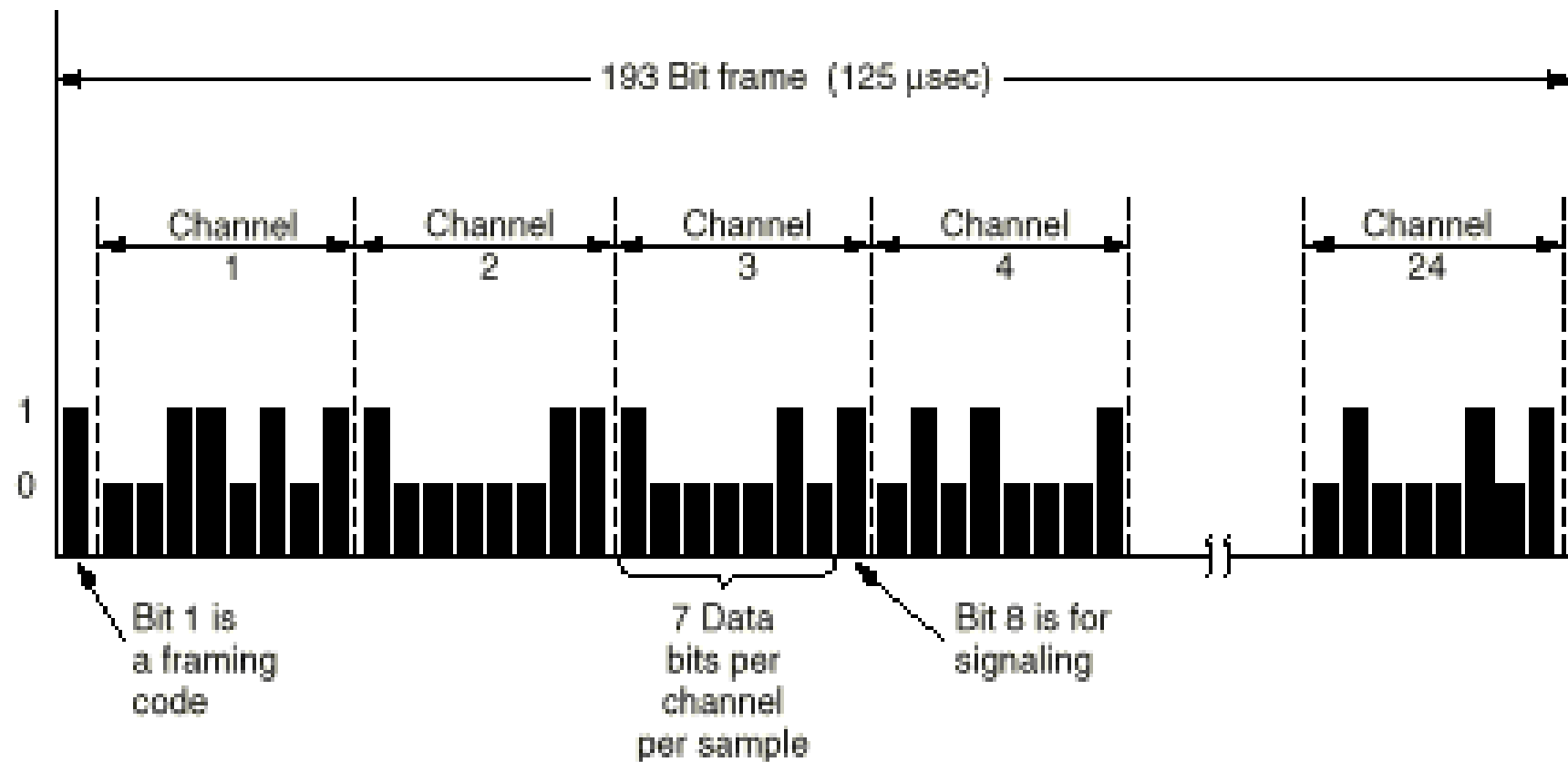


Fig. 2-26. The T1 carrier (1.544 Mbps).



频分复用 FDM

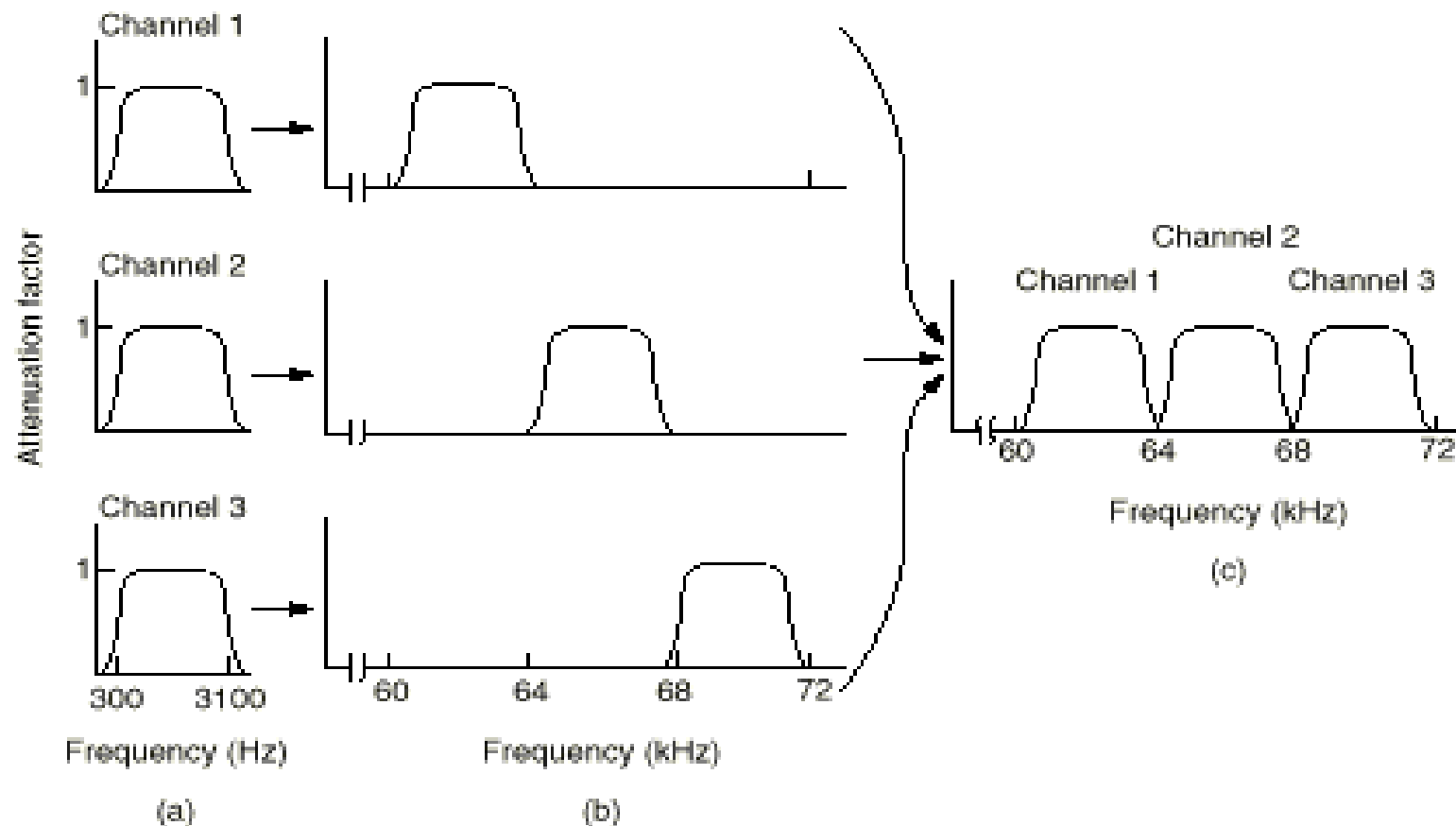


Fig. 2-24. Frequency division multiplexing. (a) The original bandwidths. (b) The bandwidths raised in frequency. (c) The multiplexed channel.



波分复用 WDM

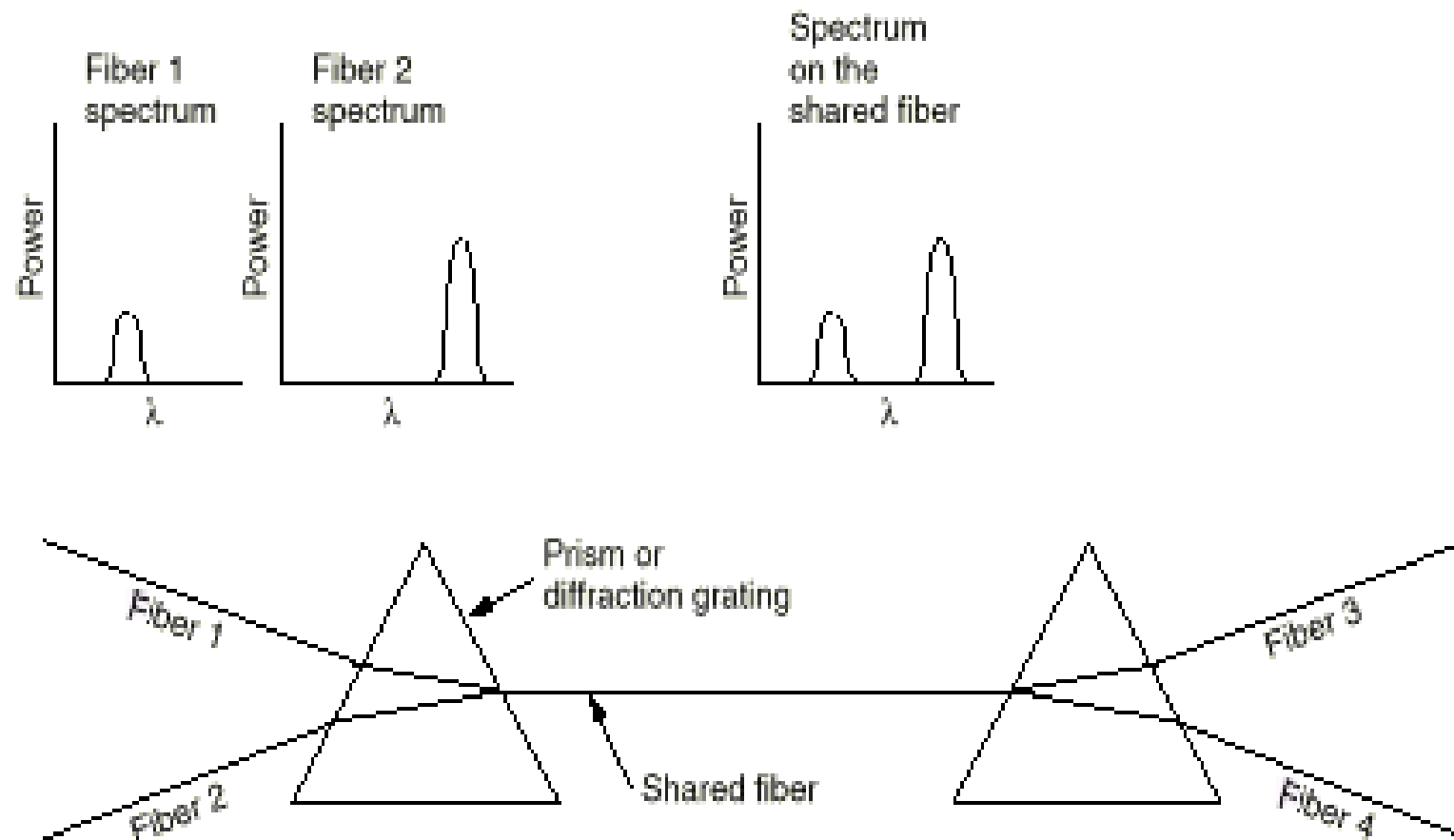
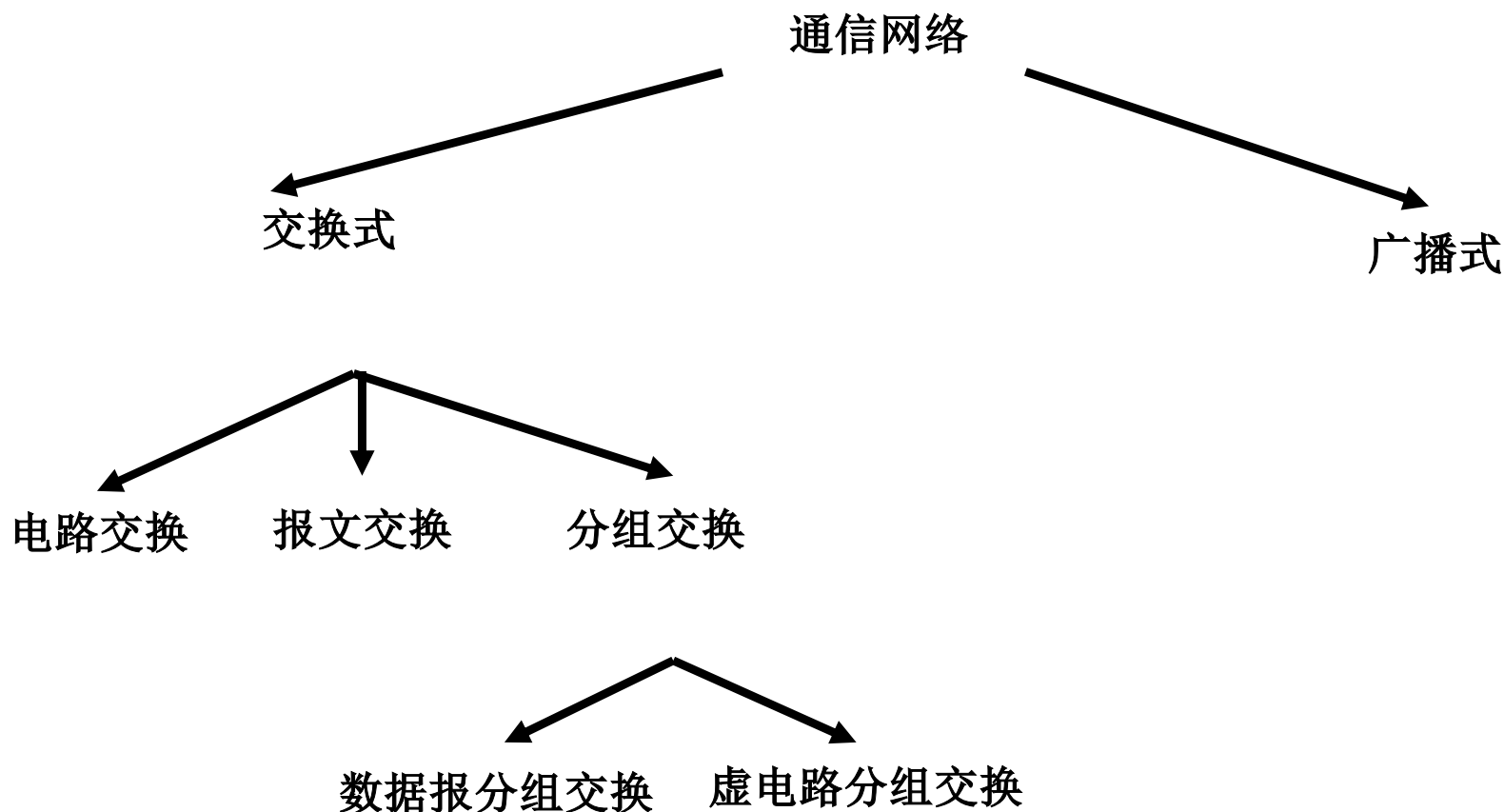


Fig. 2-25. Wavelength division multiplexing.



交换技术

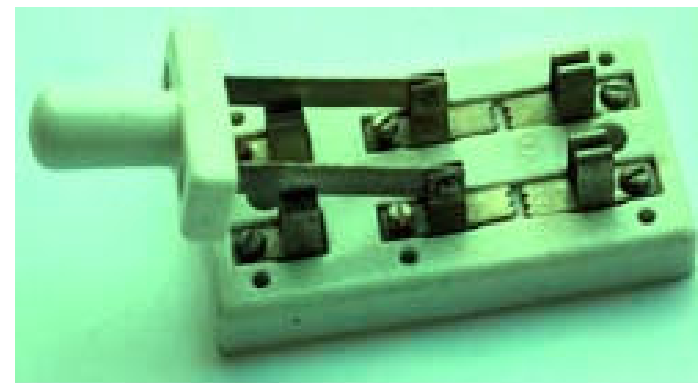
- 通信网络可以根据其结点交换信息的方式进行分类





交换技术（续）

- 在多结点通信网络中，为有效利用通信设备和线路，一般希望动态地设定通信双方间的线路。
- 动态地接通或断开通信线路，称为“交换”
- 交换方式分类：
 - 电路交换
 - 报文交换，存储转发方式
 - 分组交换（包交换），存储转发方式





电路交换 (circuit switching)

■ 原理

- 直接利用可切换的物理通信线路，连接通信双方

■ 三个阶段

- 建立电路，传输数据，拆除电路

■ 特点

- 在发送数据前，必须建立起点到点的物理通路
- 建立物理通路时间较长，数据传送延迟较短

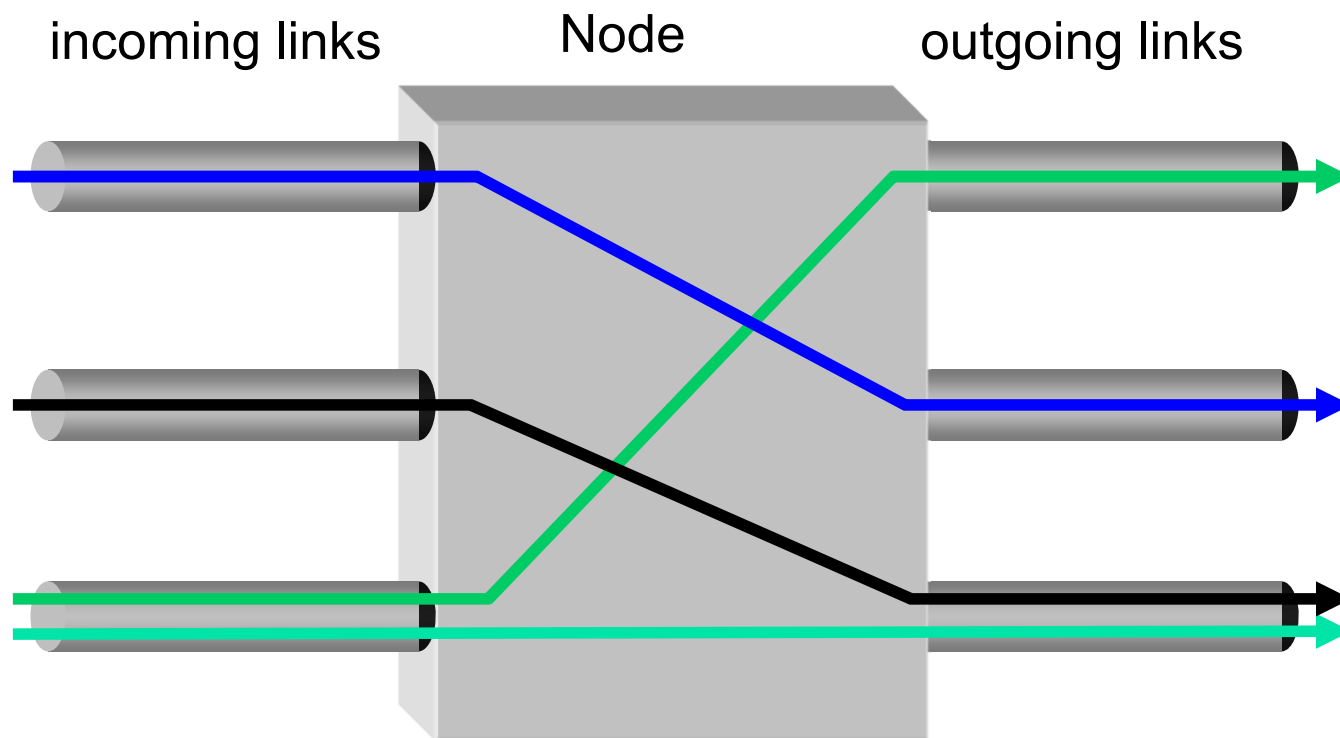
■ 例

- 电话网
- **ISDN (Integrated Services Digital Networks)**



电路交换 (续)

■ 电路交换网络中的结点（交换机）工作方式

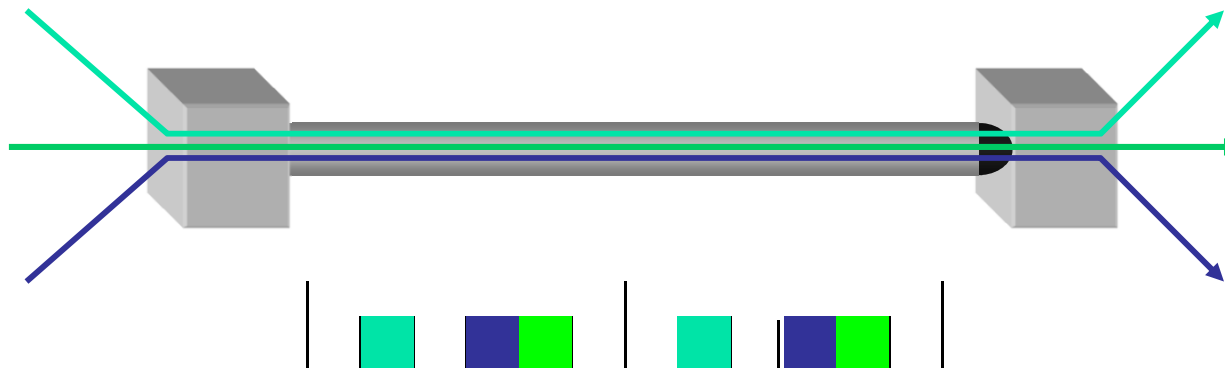




电路交换（续）

■ 复用/解复用

- 一般采用时分复用
- 时间被分为帧（**frame**），帧被分为时槽（**slot**）
- 时槽在帧内的相对位置决定这个槽所传输数据所属的会话
- 发送方和接收方需要同步
- 非永久会话需要动态绑定时槽到一个会话





报文交换 (message switching)

■ 原理

- 信息以报文（逻辑上完整的信息段）为单位进行存储转发

■ 特点

- 线路利用率高
- 要求中间结点（网络通信设备）缓冲大
- 延迟时间长



分组交换 (packet switching)

■ 原理

- 分组：比报文还小的信息段，可定长，也可变长
- 信息以分组为单位进行存储转发。源结点把报文分为分组，在中间结点存储转发，目的结点把分组合成报文

■ 特点

- 每个分组头包括源地址和目的地址，独立进行路由选择
- 网络结点设备中不预先分配资源
- 线路利用率高
- 易于重传，可靠性高
- 易于开始新的传输，让紧急信息优先通过
- 额外信息增加

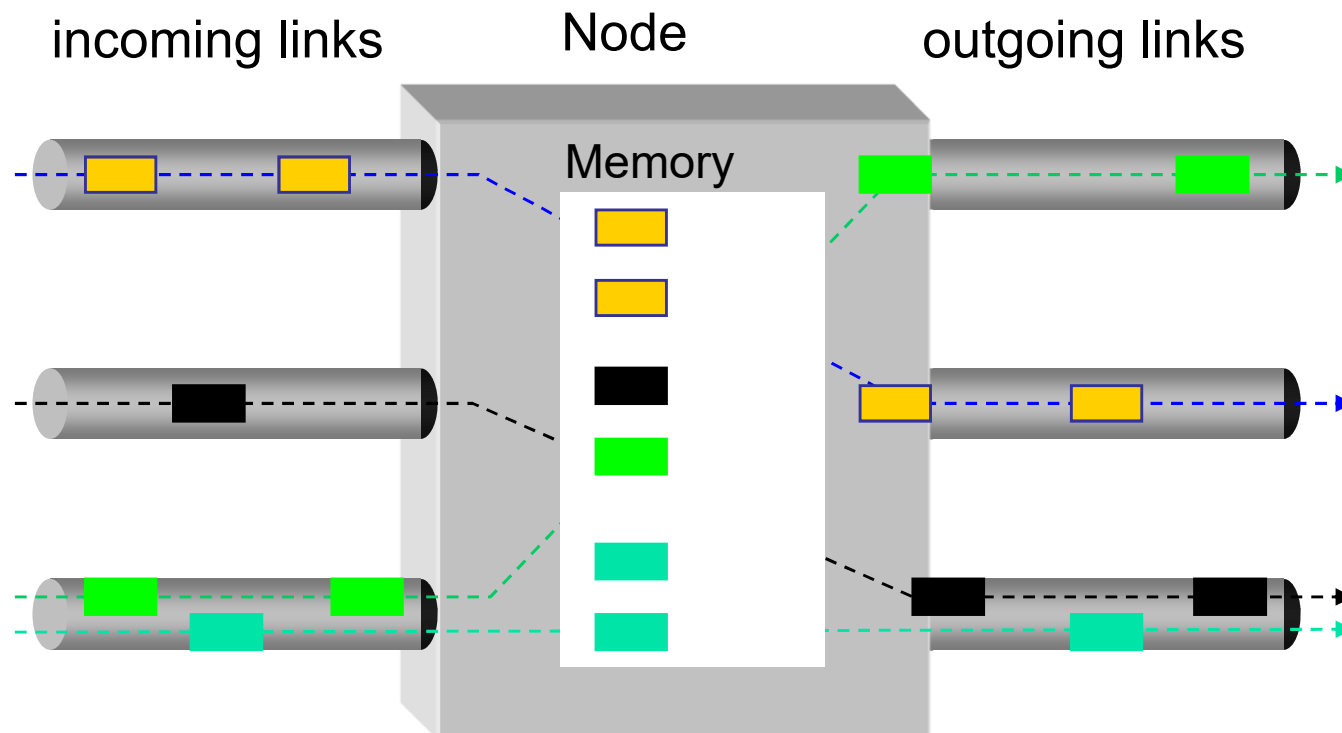
■ 分组交换分为

- 数据报 (**datagram**) 分组交换
- 虚电路 (**virtual circuit**) 分组交换



分组交换（续）

- 分组交换网中的结点（交换机/路由器）工作方式

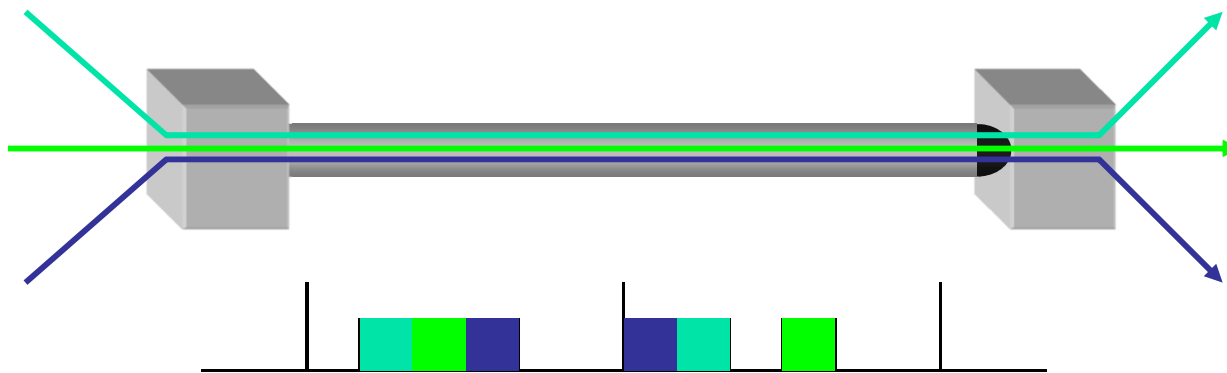




分组交换（续）

■ 复用/解复用

- 采用统计复用
- 来自任意会话的数据可以立即发送，不需要等待时槽
- 用附加的分组头来区分数据



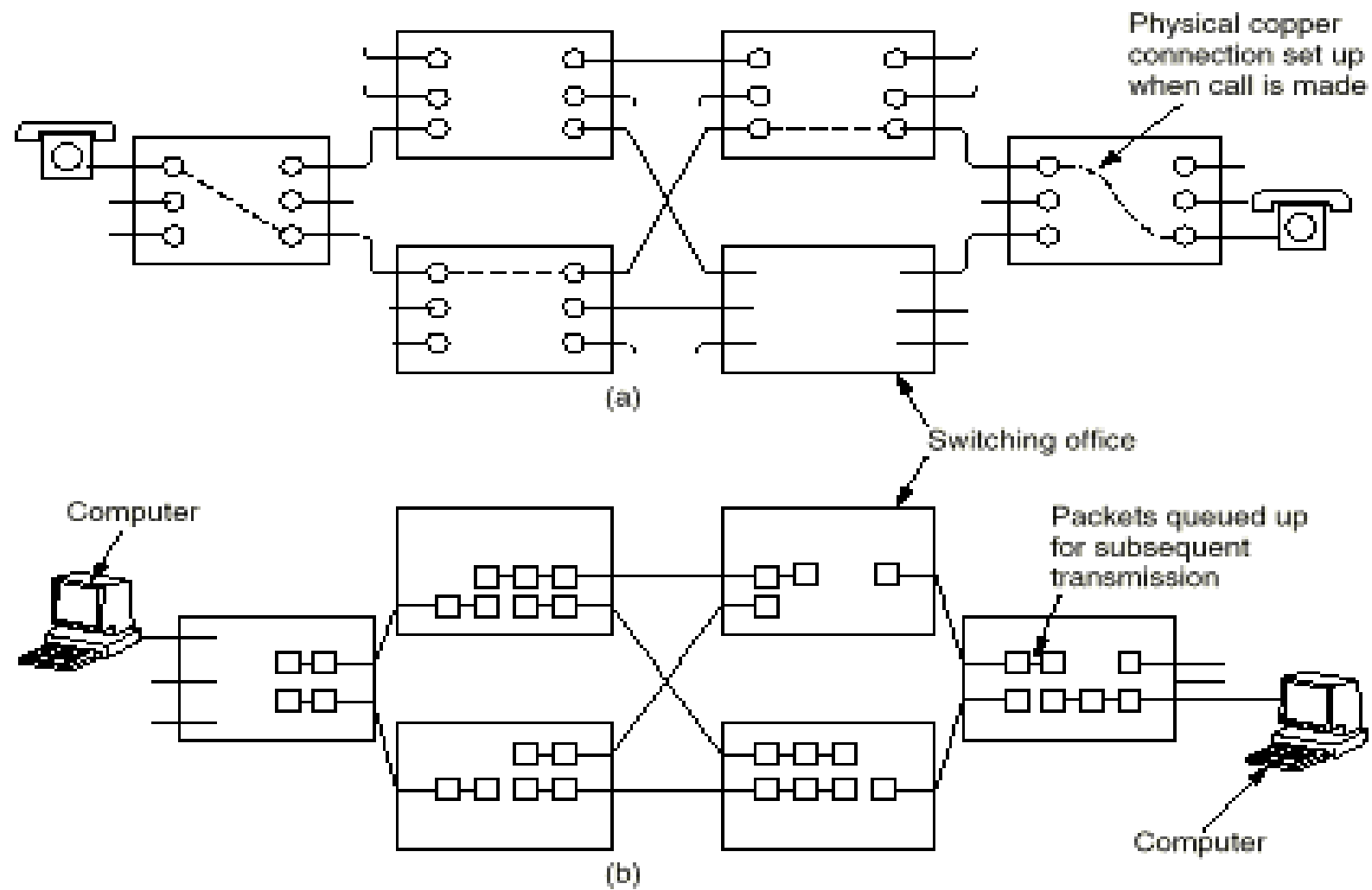


Fig. 2-34. (a) Circuit switching. (b) Packet switching.

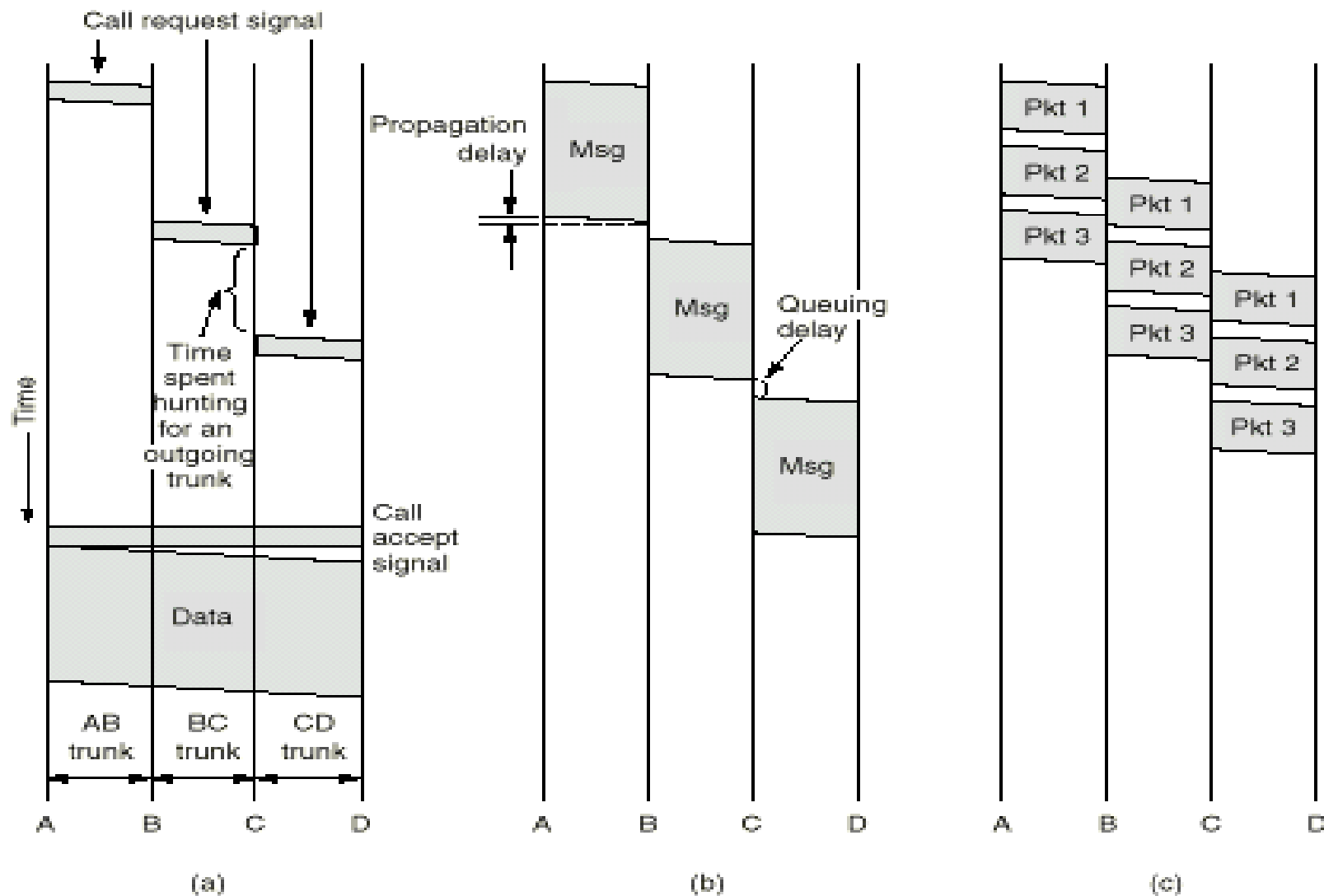
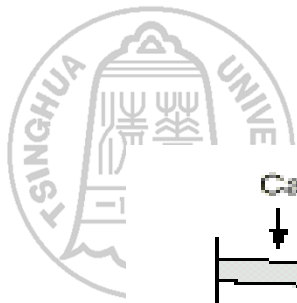


Fig. 2-35. Timing of events in (a) circuit switching, (b) message switching, (c) packet switching.



分组交换（续）

- 数据报分组交换
 - 每个分组均带有网络地址（源、目的），可走不同的路径
 - 例: **IP networks**



分组交换（续）

■ 虚电路分组交换

■ 电路交换和分组交换的结合

- 数据以分组形式传输
- 来自同一流的分組通过一个预先建立的路径（虚电路）传输
- 确保分组的顺序
- 但是来自不同虚电路的分組可能会交错在一起

■ 分三个阶段

- 建立：发带有全称网络地址的呼叫分組，建立虚电路
- 传输：沿建立好的虚电路传输数据；
- 拆除：拆除虚电路。

■ 注意：分組头不需要包含完整的地址信息

■ 例：ATM networks

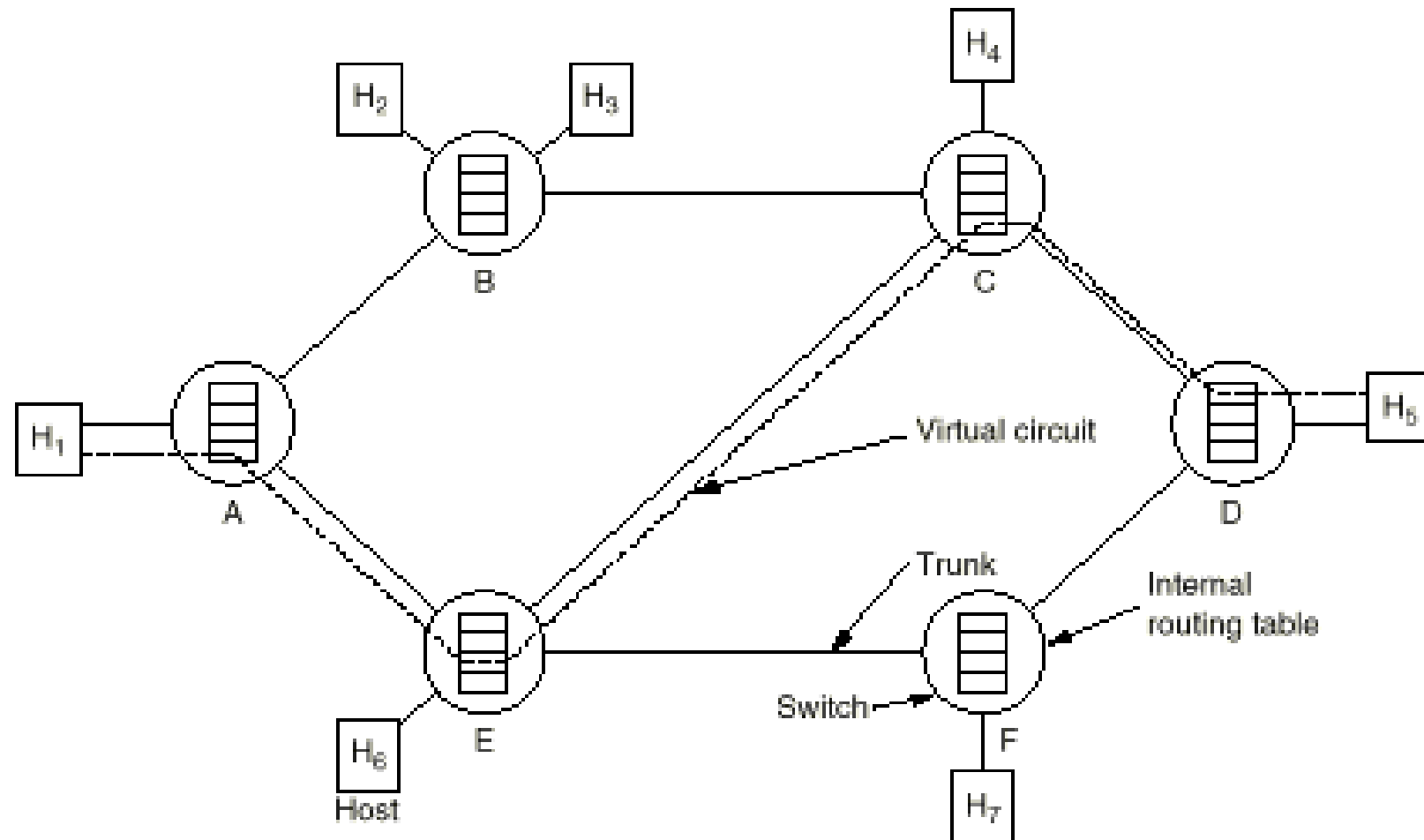


Fig. 2-43. The dotted line shows a virtual circuit. It is simply defined by table entries inside the switches.



分组交换（续）

- 电路交换与分组交换的比较
 - 分组交换相比电路交换的最大优势是可以实现统计复用，有效的利用带宽
 - 但是分组交换需要处理拥塞，因此：
 - 需要复杂的路由器
 - 难以提供好的服务质量（延迟和带宽的保证）
 - 实际应用中，这两种方式可以结合在一起
 - **IP over SONET, IP over Frame Relay**



Item	Circuit-switched	Packet-switched
Dedicated "copper" path	Yes	No
Bandwidth available	Fixed	Dynamic
Potentially wasted bandwidth	Yes	No
Store-and-forward transmission	No	Yes
Each packet follows the same route	Yes	No
Call setup	Required	Not needed
When can congestion occur	At setup time	On every packet
Charging	Per minute	Per packet

Fig. 2-36. A comparison of circuit-switched and packet-switched networks.



不同交换技术的比较

- 电路交换适用于实时信息和模拟信号传送，在线路带宽比较低的情况下使用比较经济
- 报文交换适用于线路带宽比较高的情况，可靠灵活，但延迟大
- 分组交换缩短了延迟，也能满足一般的实时信息传送。在高带宽的通信中更为经济、合理、可靠。是目前公认较（最）好的一种交换技术



交换结构

- 交换结构 (**switch fabric**)
 - **crossbar** 交换
 - 空分交换
 - 时分交换



crossbar 交换

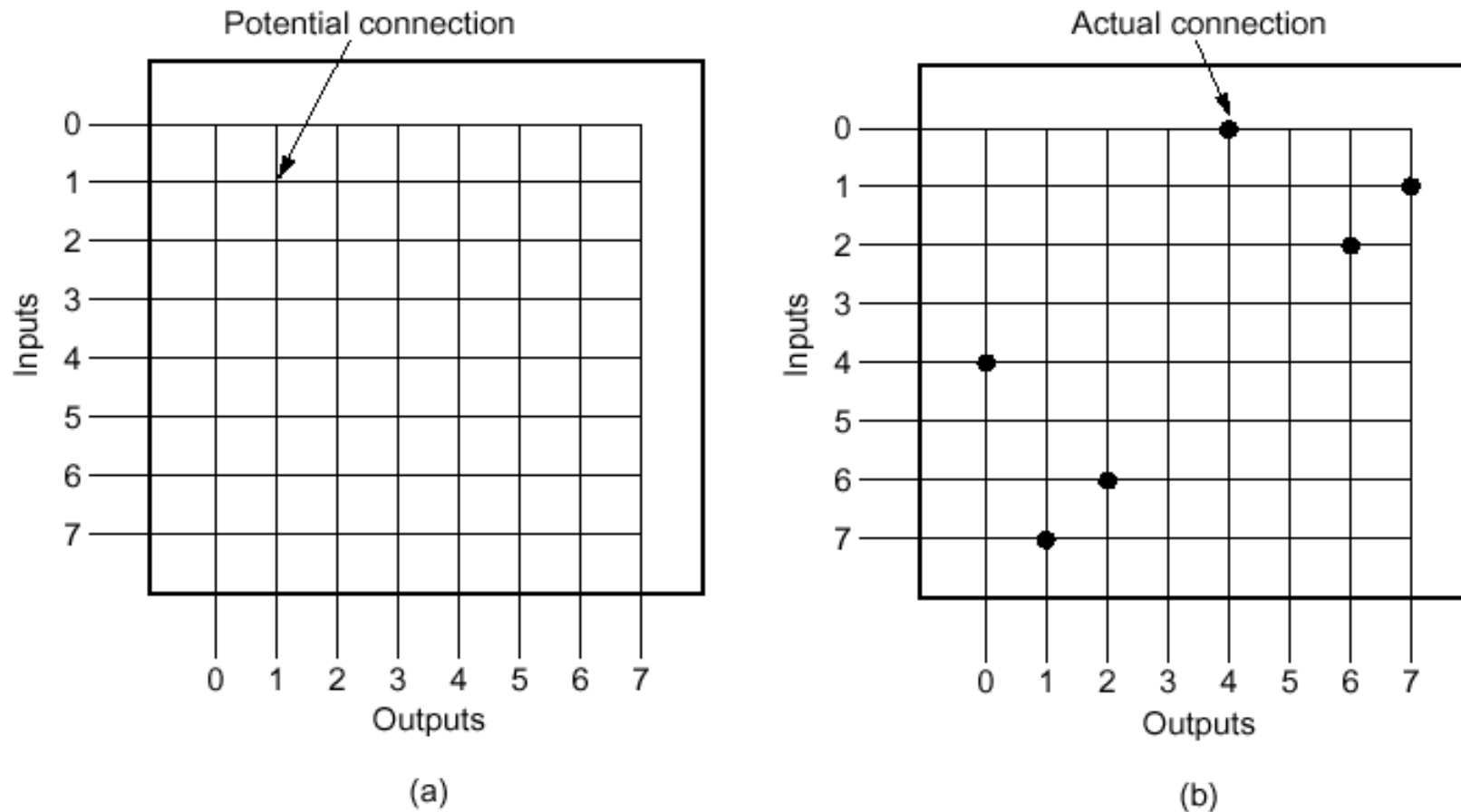


Fig. 2-38. (a) A crossbar switch with no connections. (b) A crossbar switch with three connections set up: 0 with 4, 1 with 7, and 2 with 6.



空分交换

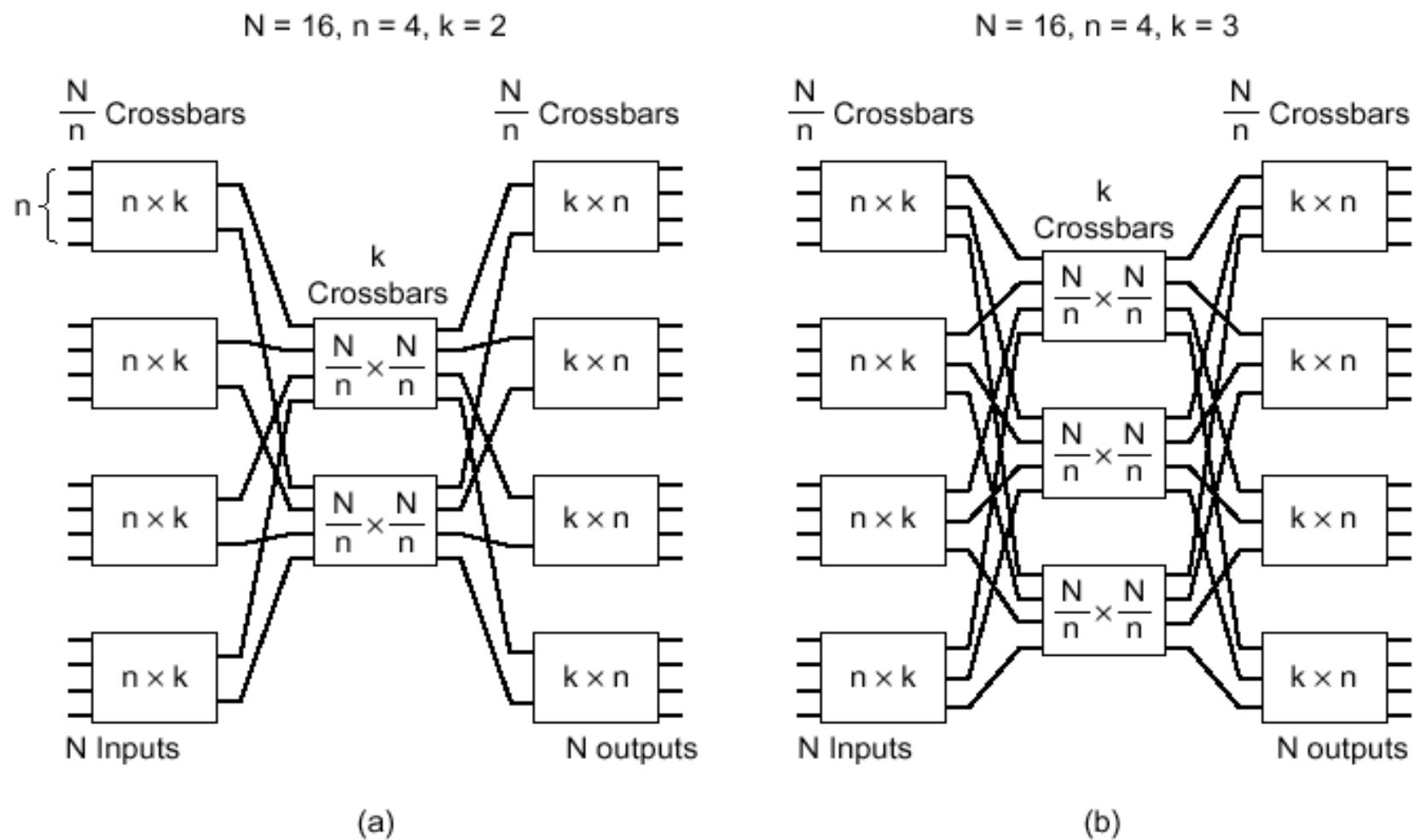


Fig. 2-39. Two space division switches with different parameters.



时分交换

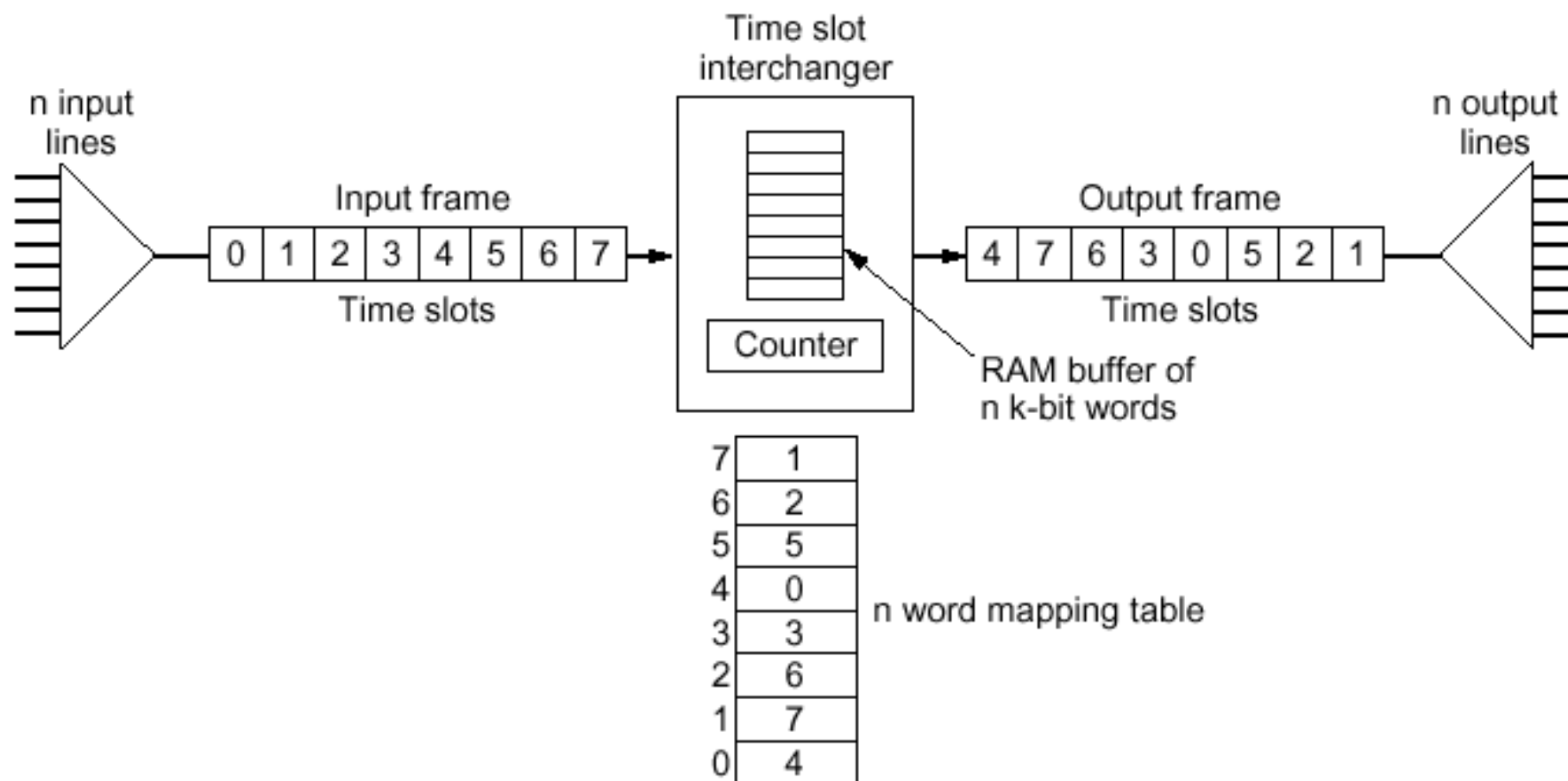


Fig. 2-40. A time division switch.



总结

■ 数据通信基础理论

- 信号，信号的时域观和频域观，傅立叶分析
- 有限带宽信号
- 奈魁斯特定律、香农定律和信道的最大数据传输速率

■ 数据通信技术

- 数据通信系统的基本结构
- 传输和传输方式
- 数据编码技术
- 多路复用技术



总结（续）

■ 交换技术

■ 电路交换

■ 报文交换

■ 分组交换

■ 数据报分组交换

■ 虚电路分组交换

■ 交换结构

■ 思考：查阅资料，总结电路交换与分组交换的区别

The background of the slide features a large, light gray watermark of the Tsinghua University seal. The seal is circular, with the university's name in English, "TSINGHUA UNIVERSITY", around the top and "1911" at the bottom. In the center is a shield-like emblem with traditional Chinese architectural and symbolic elements.

第四章

物理层接口及其协议



主要内容

- 物理层的定义和功能
- 物理层的特性
- 典型的物理层标准接口
- 传输介质
- 网络传输技术



物理层的定义和功能

■ 物理层的定义

- **ISO/OSI** 关于物理层的定义：物理层提供机械的、电气的、功能的和规程的特性，目的是启动、维护和关闭数据链路实体之间进行比特传输的物理连接。这种连接可能通过中继系统，在中继系统内的传输也是在物理层

■ 物理层的功能

- 在两个网络设备之间提供透明的比特流传输

■ 研究内容

- 物理连接的启动和关闭，正常数据的传输，以及维护管理



物理层的定义和功能（续）

- 物理层有关的传输方式
 - 连接方式（点到点，点到多点）
 - 通信方式（单工，半双工，全双工）
 - 位传输方式（串行，并行）
- 物理层的四个重要特性
 - 机械特性 (**mechanical characteristics**)
 - 电气特性 (**electrical characteristics**)
 - 功能特性 (**functional characteristics**)
 - 规程特性 (**procedural characteristics**)



物理层的特性

■ 机械特性

- 主要定义物理连接的边界点，即接插装置。规定物理连接时所采用的规格、引脚的数量和排列情况
- 标准接口举例
 - **ISO 2110, 25芯连接器, EIA RS-232-C, EIA RS-366-A**
 - **ISO 2593, 34芯连接器, V.35宽带MODEM**
 - **ISO 4902, 37芯和9芯连接器, EIA RS-449**
 - **ISO 4903, 15芯连接器, X.20、X.21、X.22**



物理层的特性（续）

■ 电气特性

- 规定传输二进制位时，线路上信号的电压高低、阻抗匹配、传输速率和距离限制
- 早期的标准是在边界点定义电气特性，例如**EIA RS-232-C、V.28**；最近的标准则说明了发送器和接收器的电气特性，而且给出了有关对连接电缆的控制
- **CCITT** 制订的电气特性标准
 - **CCITT V.10/X.26**：新的非平衡型电气特性，**EIA RS-423-A**
 - **CCITT V.11/X.27**：新的平衡型电气特性，**EIA RS-422-A**
 - **CCITT V.28**：非平衡型电气特性，**EIA RS-232-C**
 - **CCITT X.21/EIA RS-449**



物理层的特性（续）

■ 功能特性

- 主要定义各条物理线路的功能
- 线路的功能分为四大类
 - 数据
 - 控制
 - 定时
 - 地

■ 规程特性

- 主要定义各条物理线路的工作规程和时序关系



EIA RS-232-C

- **1960年**美国电子工业协会**EIA**提出**RS-232**，**1963年**提出**RS-232-A**，**1965年**提出**RS-232-B**，**1969年**提出**RS-232-C**。用于**DTE/DCE**之间的接口
- 机械特性
 - **25芯连接器**，**DTE**为插头，**DCE**为插座
- 电气特性
 - 采用非平衡型电气特性，低于**-3V**为“**1**”，高于**+4V**为“**0**”，最大**20Kbps**，最长**15m**
 - 非平衡传输（**unbalanced transmission**）：所有电路共享一个公用的地线
 - 平衡传输（**balanced transmission**）：每个主要电路需要两根线，没有公用的地线



EIA RS-232-C (续)

■ 功能特性

- 定义了**21**条线，许多子集，基本与**CCITT V.24**兼容

■ 规程特性

- 对不同的功能子集，有不同的规程
- **RS-232-C** 有**14**中不同的接口类型，适合于单工，半双工，全双工，同步，异步

■ **RS-232-C**的不足与改进

- 不足
 - 传输性能低，距离短，速率低
- 改进
 - 重新设计，**X.21**
 - 以**RS-232-C**为基础改进，**1977**年提出**RS-449**

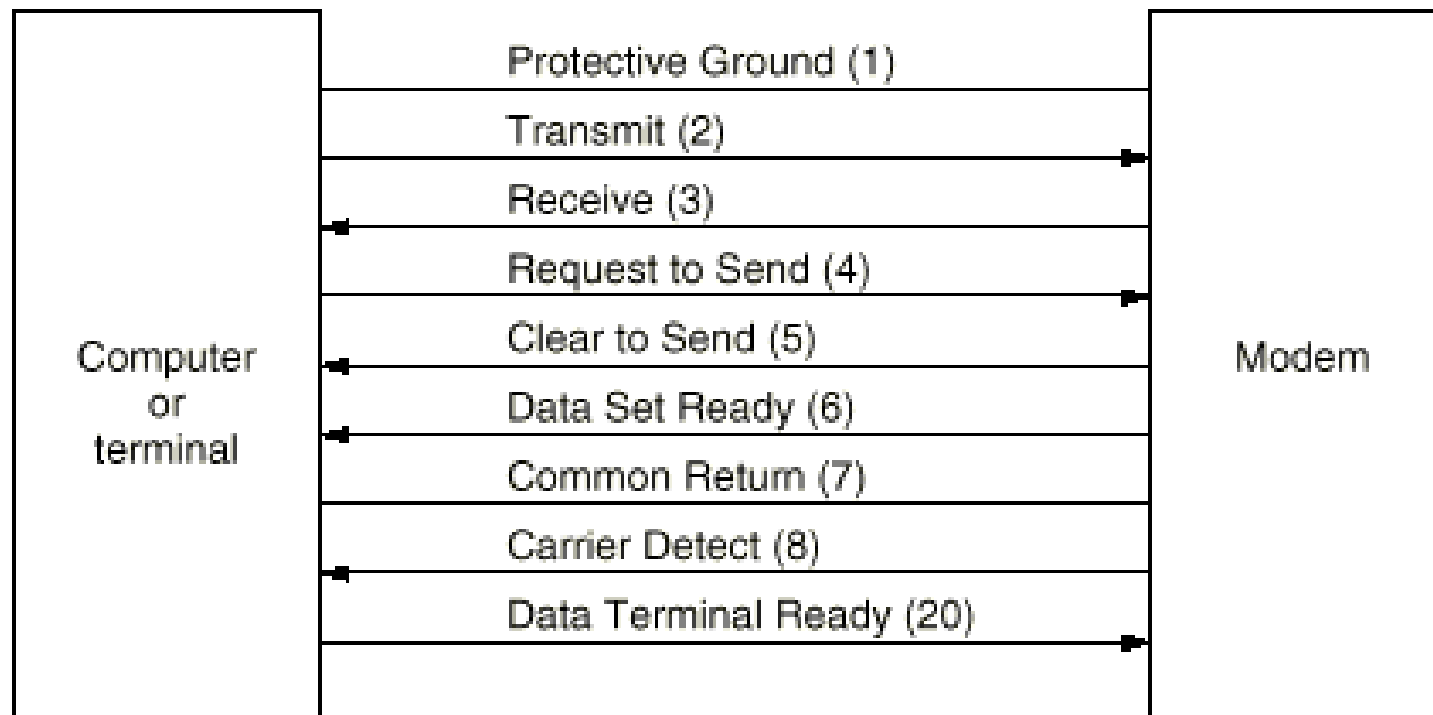
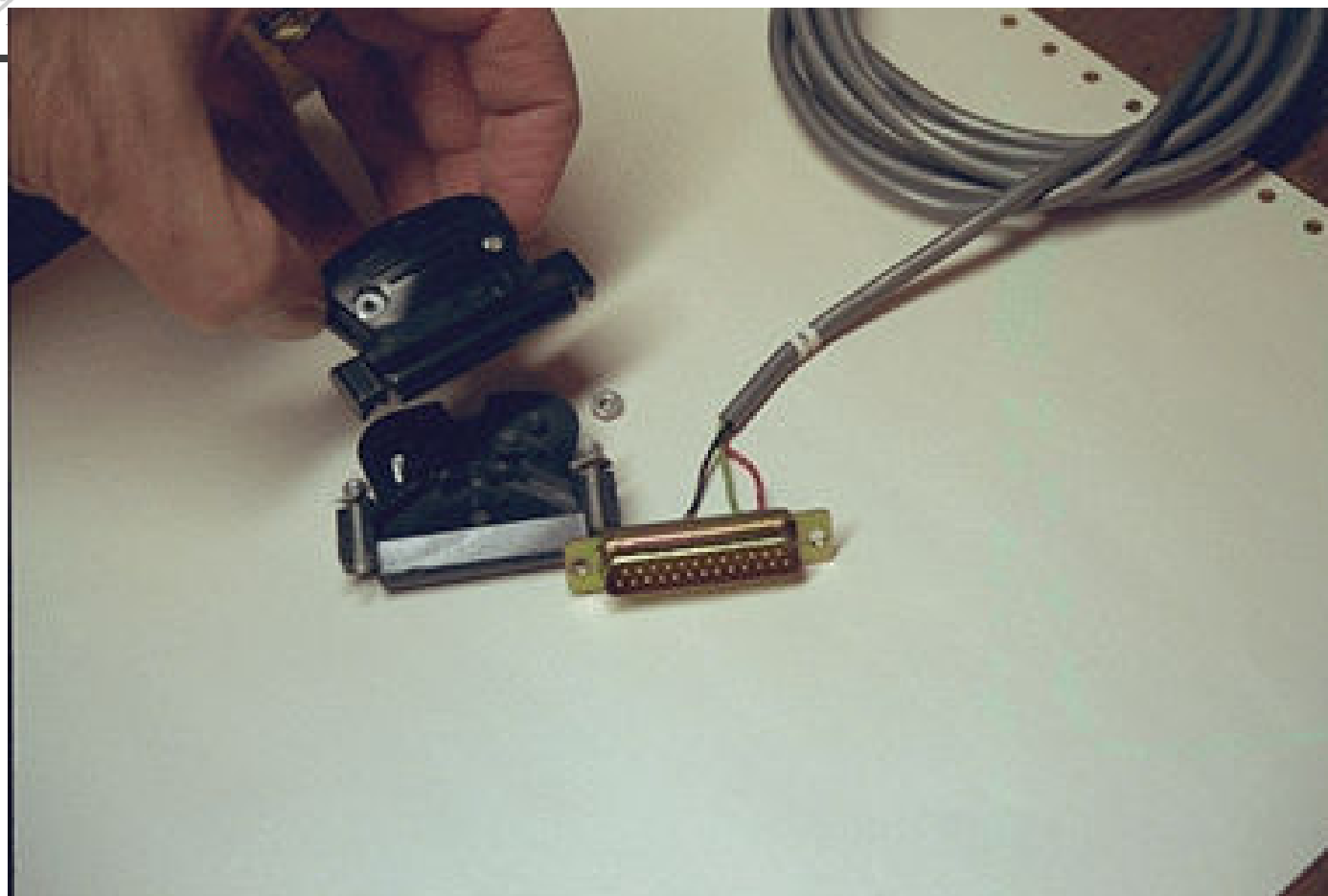


Fig. 2-21. Some of the principal RS-232-C circuits. The pin numbers are given in parentheses.



RS232接头



EIA RS-449/422-A/423-A

- **EIA RS-449** 是为替代**RS-232-C**而提出的物理层标准接口。实际上是一体化的三个标准
- 主要改进
 - 改善了性能，加长了接口电缆距离，加大了数据传输率
 - 增加了新的接口功能，例如，回送检查
 - 解决了机械接口问题
- 机械特性
 - **37芯或9芯**连接器



EIA RS-449/422-A/423-A (续)

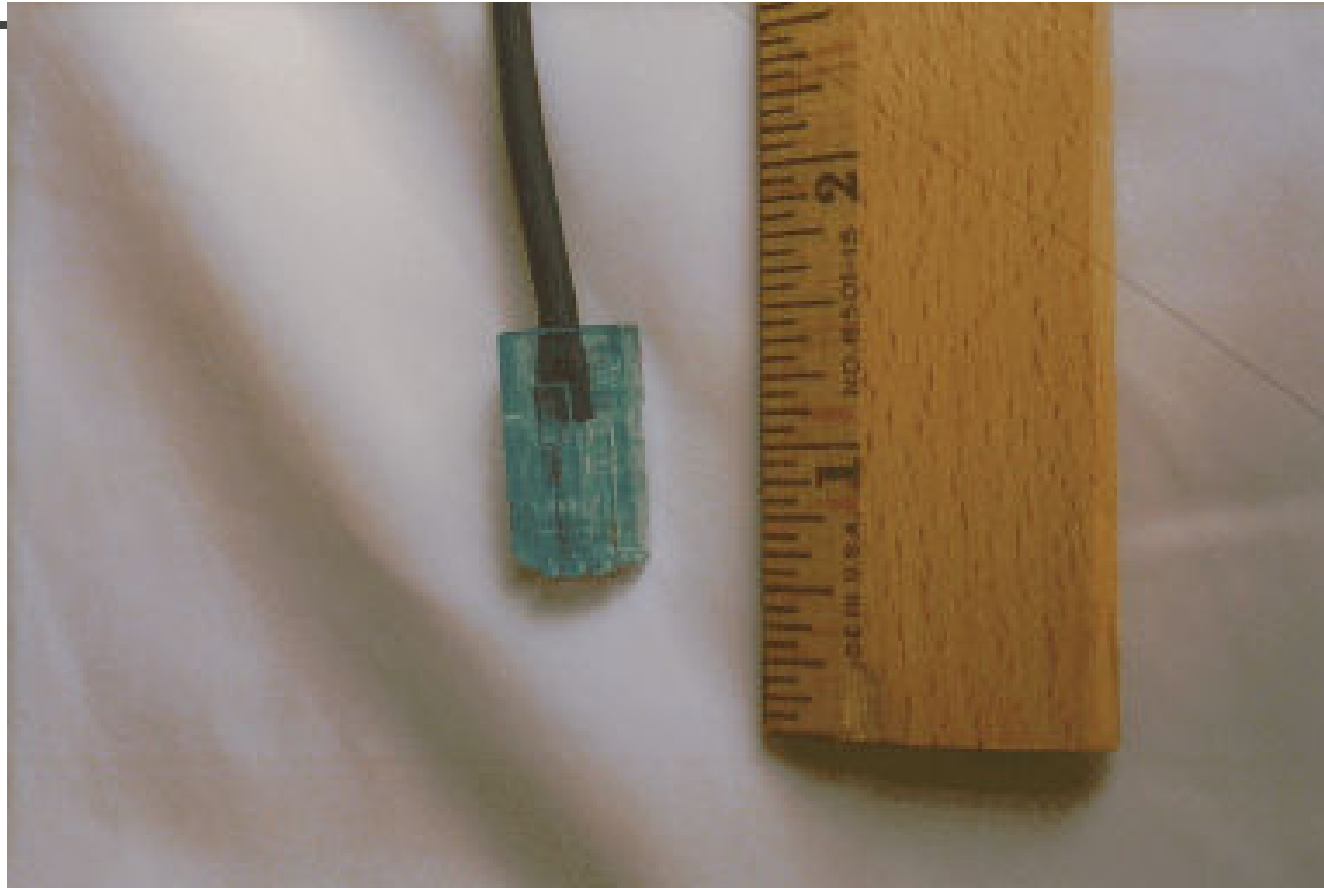
- 电气特性
 - 与**RS-232-C**相连，采用非平衡型电气特性 **RS-423-A**，**20Kbps**以下
 - 其他情况，采用平衡型电气特性 **RS-422-A**，**20Kbps ~ 2Mbps**
- 功能特性
 - 定义了**30**条功能线
- 规程特性
 - 基本上以**RS-232-C**为基础



传输介质

■ 双绞线

- 既可用于模拟传输，也可用于数据传输
- 带宽依赖于线的类型和传输距离
- **3类线，5类线，增强型5类线、6类线、7类线**
- **非屏蔽双绞线UTP（Unshielded Twisted Pair），屏蔽双绞线STP（Shielded Twisted Pair）**



帶有**RJ—45**接头的双绞线



具有以太网的计算机背面



一个**10/100M**双绞线以太网接口，指示灯的状态显示接口连接在一个**10M**以太网上



传输介质（续）

- 基带同轴电缆
 - **50欧姆**，用于数据传输
- 宽带同轴电缆
 - **75欧姆**，用于模拟传输
 - **Cable TV技术**，**300MHz或450MHz**

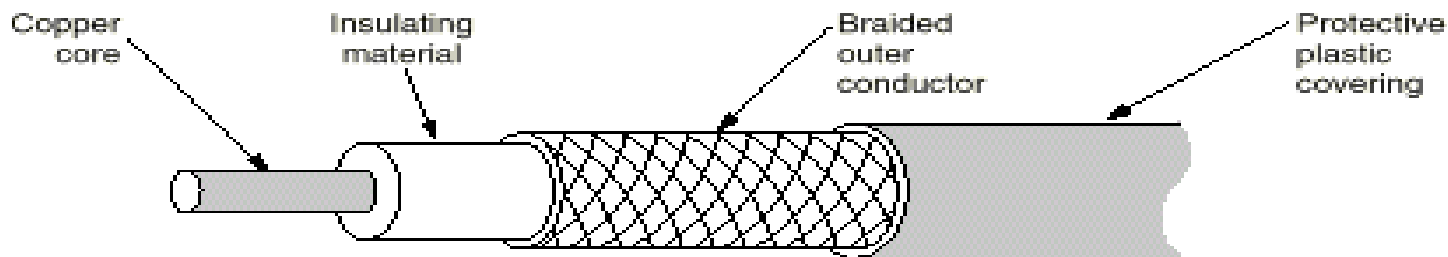


Fig. 2-3. A coaxial cable.



传输介质（续）

■ 光纤

- 目前，在试验室中光纤带宽超过**70Tbps**； **8×2.5 Gbps**， **8×10 Gbps**， **32×10 Gbps**， **32×40 Gbps**的光纤传输系统已经实用
- 光纤分类：单模光纤和多模光纤
 - 光源发出的光进入光纤后，入射角度小的光被反射，并沿着光纤传播，其他光被周围介质吸收，能够反射的角度有多个，这种形式的传播称为多模。多模光纤适于短距离传输
 - 当光纤半径减小到波长的数量级时，只有一个角度（或者一个模式）的光线可以进入，这种形式的传播称为单模。单模光纤适于长距离传输
 - 单模和多模都支持同时传输几个不同波长的光线，支持波分复用



传输介质（续）

- 常用的三个波长窗口（光纤波段）
 - **850 nm**: 衰减 (**attenuation**) 大, 传输速率和距离受限制, 但价格便宜
 - **1310 nm**: 衰减小, 无色散 (**dispersion**) 补偿、功率放大情况下, 最大传**40km** (最坏情况)
 - **1550 nm**: 衰减小, 无色散补偿、功率放大情况下, 最大传**80km** (最坏情况)

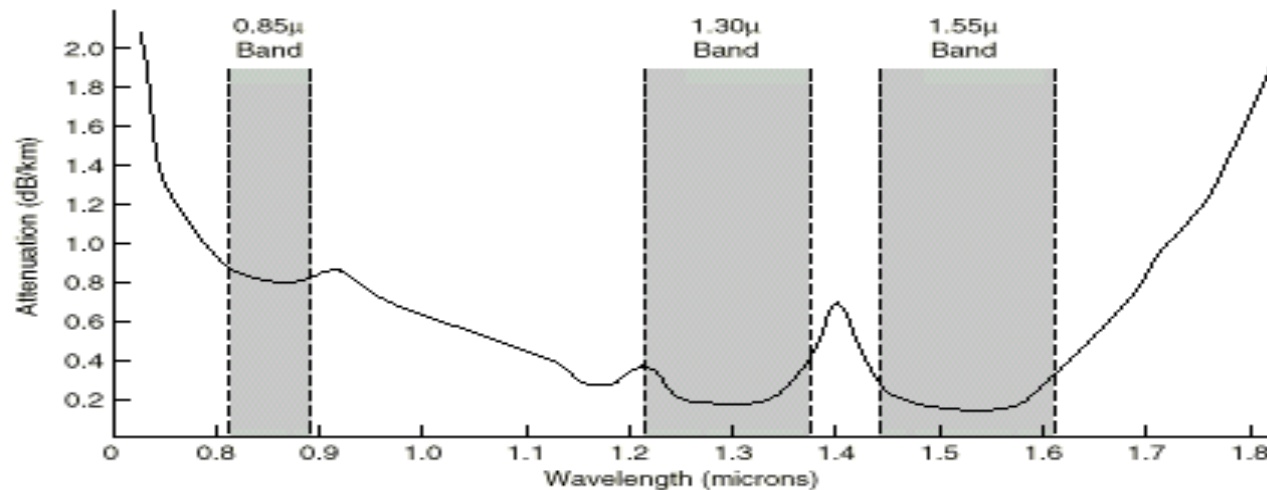


Fig. 2-6. Attenuation of light through fiber in the infrared region.



光纤和光缆

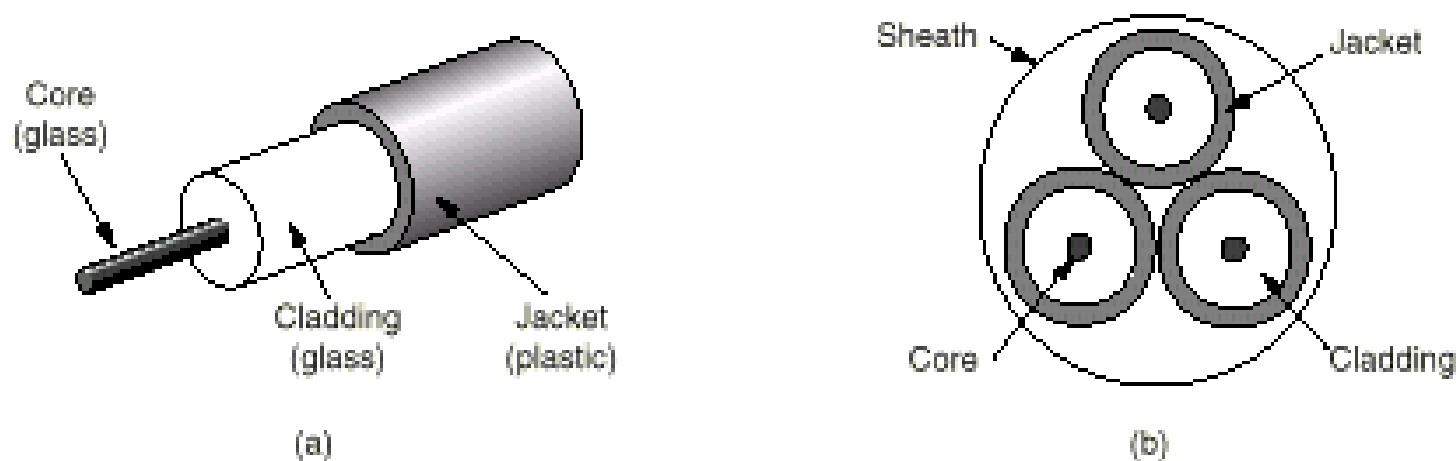


Fig. 2-7. (a) Side view of a single fiber. (b) End view of a sheath with three fibers.



传输介质（续）

■ 光网络

■ 组网方式

- 点到点：四根线（两根用于保护倒换）
- 环：两根线（一根用于保护倒换）

■ 中继器：光 — 电 — 光，全光

■ 全光网，光因特网论坛 **OIF**

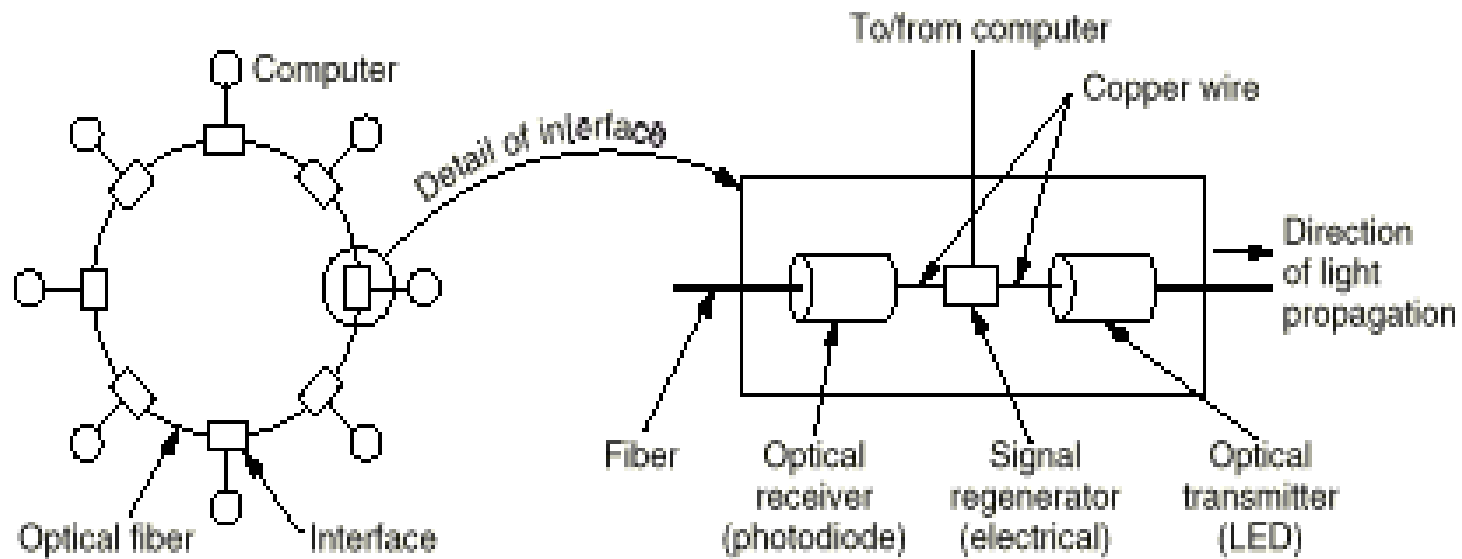
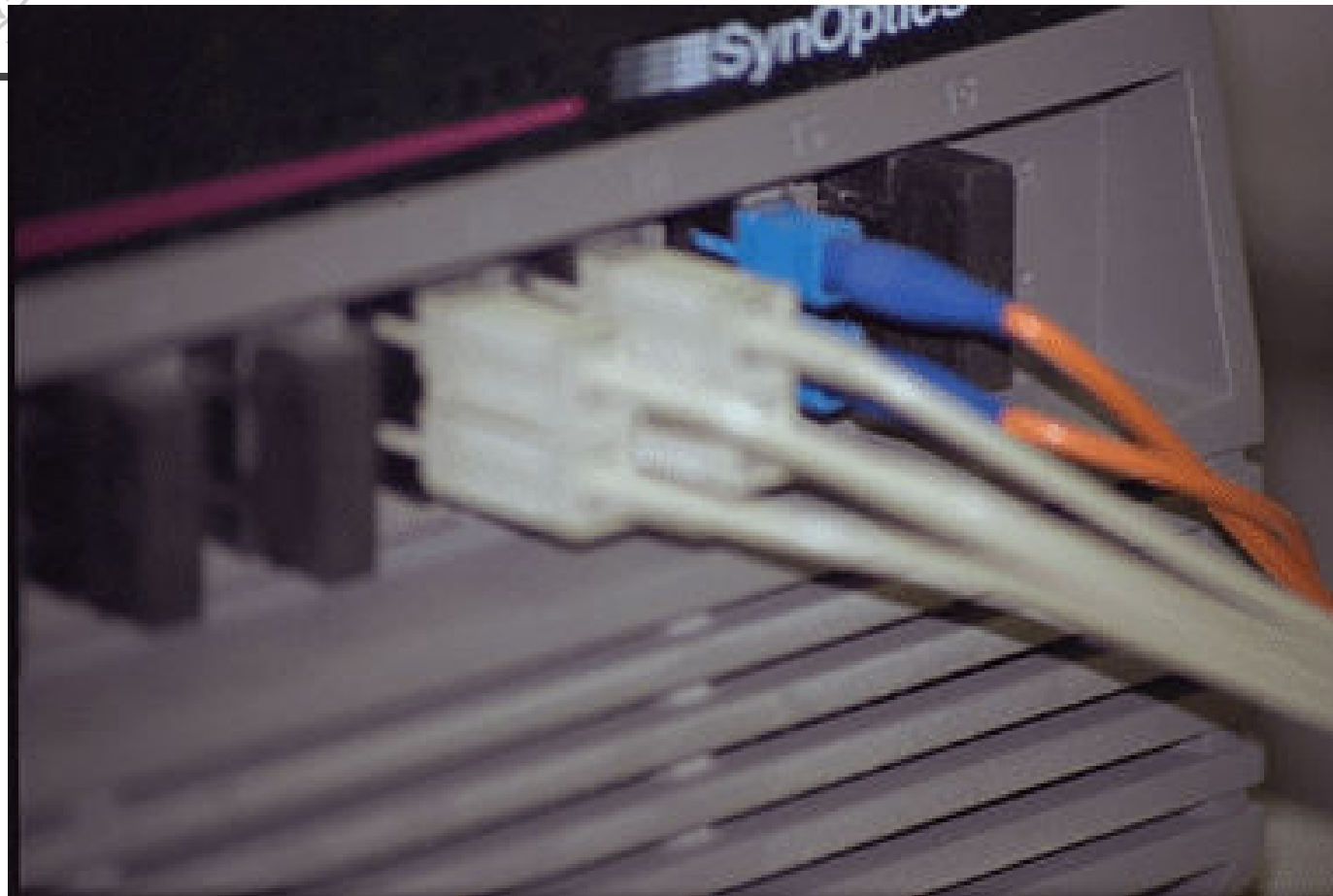


Fig. 2-9. A fiber optic ring with active repeaters.



Caption: Optical fiber cables connected to an ATM switch.



接入**ATM**网络接口的光纤



光纤和连接器，黑色套子在连接器不用的时候保护连接器



网络传输技术

- 光纤传输
- 移动电话网络
- 无线传输
- 通信卫星
- 公共交换电话网络
- 有线电视网络



SONET/SDH

■ 标准

- 1985年，Bellcore提出**SONET**（**Synchronous Optical NETwork**）标准
- 1989年，CCITT提出**SDH**（**Synchronous Digital Hierarchy**）标准，与 **SONET** 有微小差别
- **SONET**主要用于北美和日本，**SDH**主要用于欧洲和中国

■ **SONET/SDH**，采用**TDM**技术，是同步系统，由主时钟控制，时钟精度 10^{-9} 秒

■ **SONET**路径

- 路径（**path**），线路（**line**），段（**section**）



SONET/SDH 结构

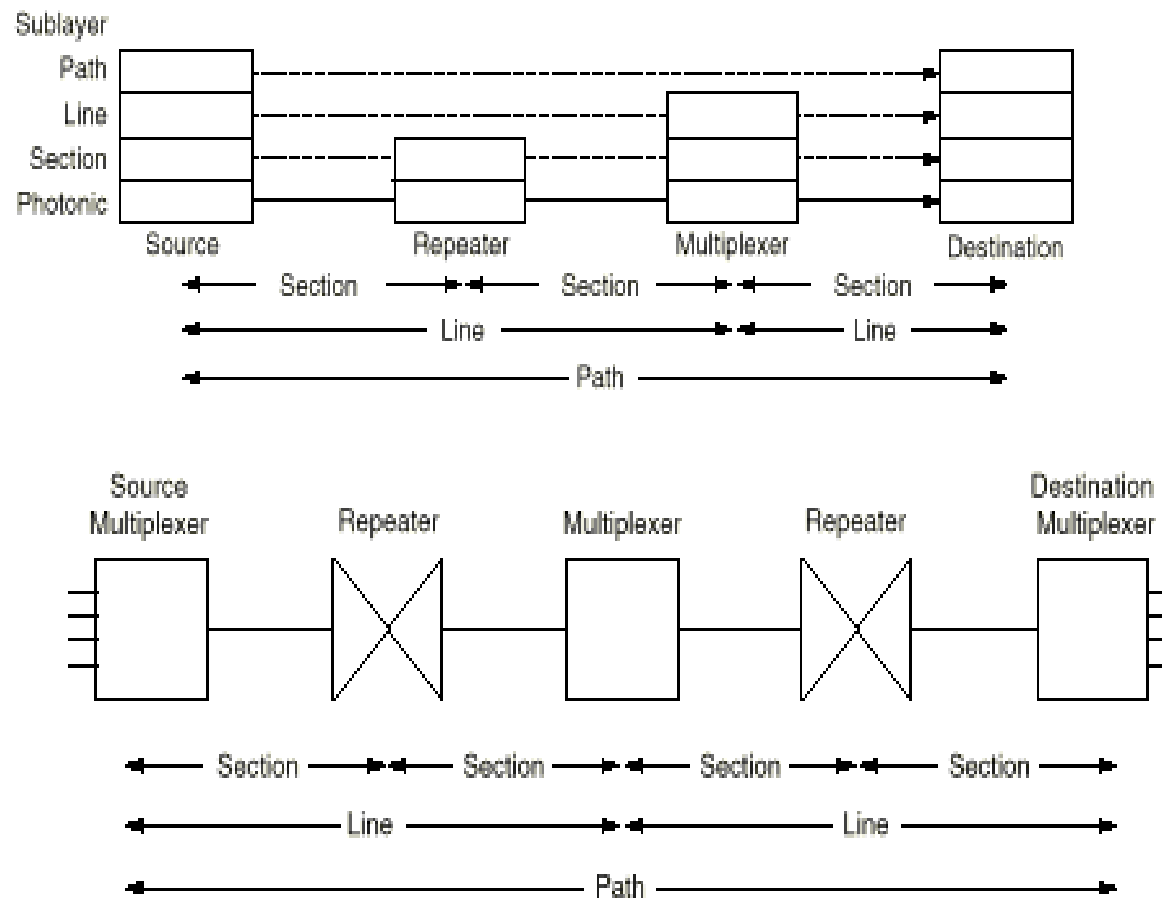


Fig. 2-29. A SONET path.



SONET/SDH 帧

■ 基本SONET帧

- **810 字节/125us**，所以传输速率为 **$810 \times 8 / (125 \times 10^{-6}) = 51.84 \text{ Mbps}$**
- 基本SONET信道称为**STS-1 (Synchronous Transport Signal-1)**
- **SONET帧格式**



SONET/SDH 帧 (续)

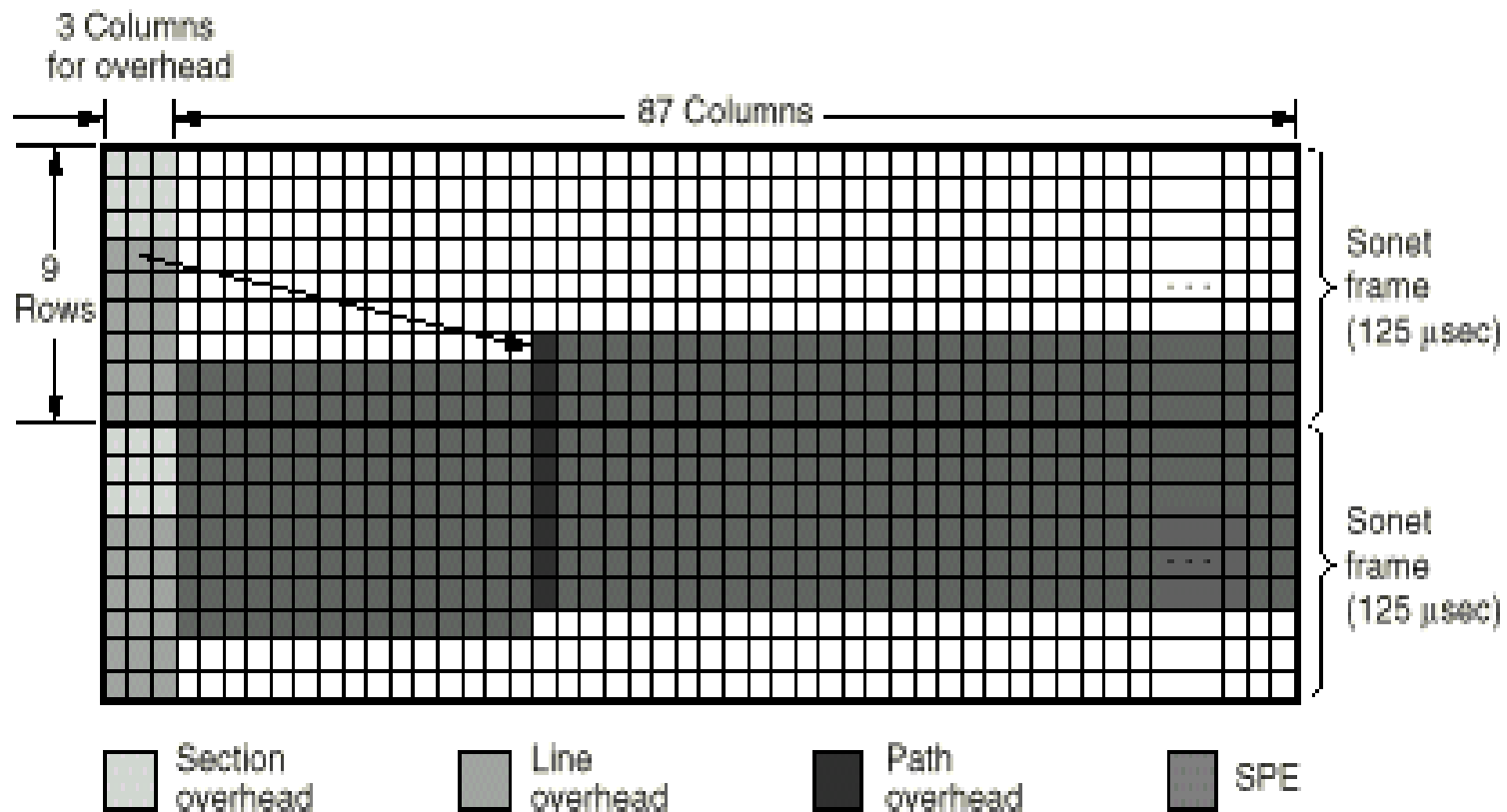


Fig. 2-30. Two back-to-back SONET frames.



SONET/SDH 复用

■ 复用

■ 复用是基于字节的

■ OC-3 与 OC-3c的区别

- c (concatenated) 表示级联, 非复用
- OC-3 表示一个155.52 Mbps的载波是由三个单独的OC-1载波复用构成的
- OC-3c 表示一个单独的155.52 Mbps的载波

■ SONET/SDH复用速率



SONET/SDH复用 (续)

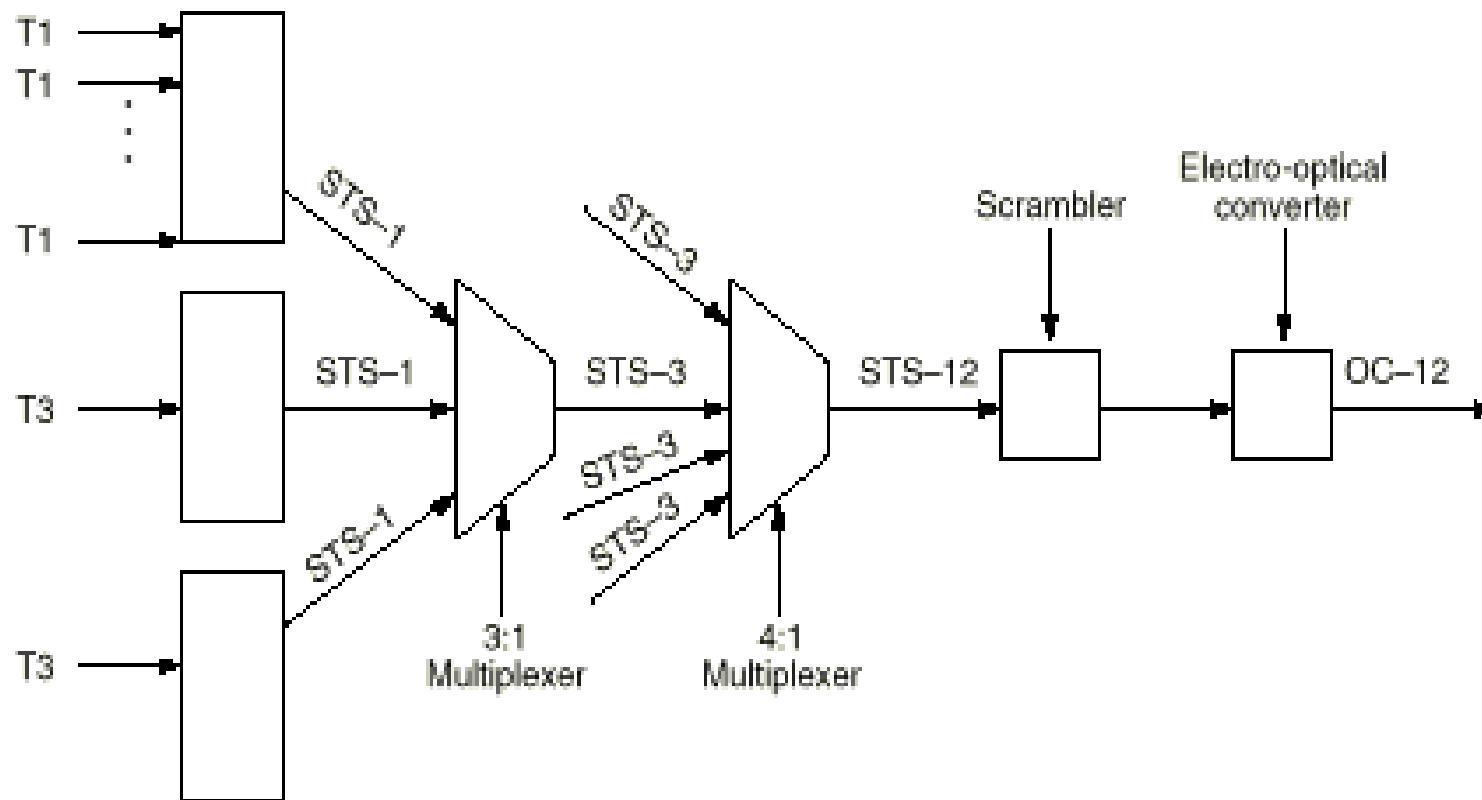


Fig. 2-31. Multiplexing in SONET.



SONET/SDH复用 (续)

SONET		SDH	Data rate (Mbps)		
Electrical	Optical	Optical	Gross	SPE	User
STS-1	OC-1		51.84	50.112	49.536
STS-3	OC-3	STM-1	155.52	150.336	148.608
STS-9	OC-9	STM-3	466.56	451.008	445.824
STS-12	OC-12	STM-4	622.08	601.344	594.432
STS-18	OC-18	STM-6	933.12	902.016	891.648
STS-24	OC-24	STM-8	1244.16	1202.688	1188.864
STS-36	OC-36	STM-12	1866.24	1804.032	1783.296
STS-48	OC-48	STM-16	2488.32	2405.376	2377.728

Fig. 2-32. SONET and SDH multiplex rates.



移动电话网络

■ 单方向的寻呼系统

■ 寻呼过程

- 打电话给寻呼公司，输入寻呼机号码
- 寻呼公司的计算机收到请求，通过线路传到高处（山顶）的天线
- 天线直接广播信号（本地寻呼），或传递给卫星（异地寻呼），卫星再广播

■ 单向系统

- 需要很小的带宽

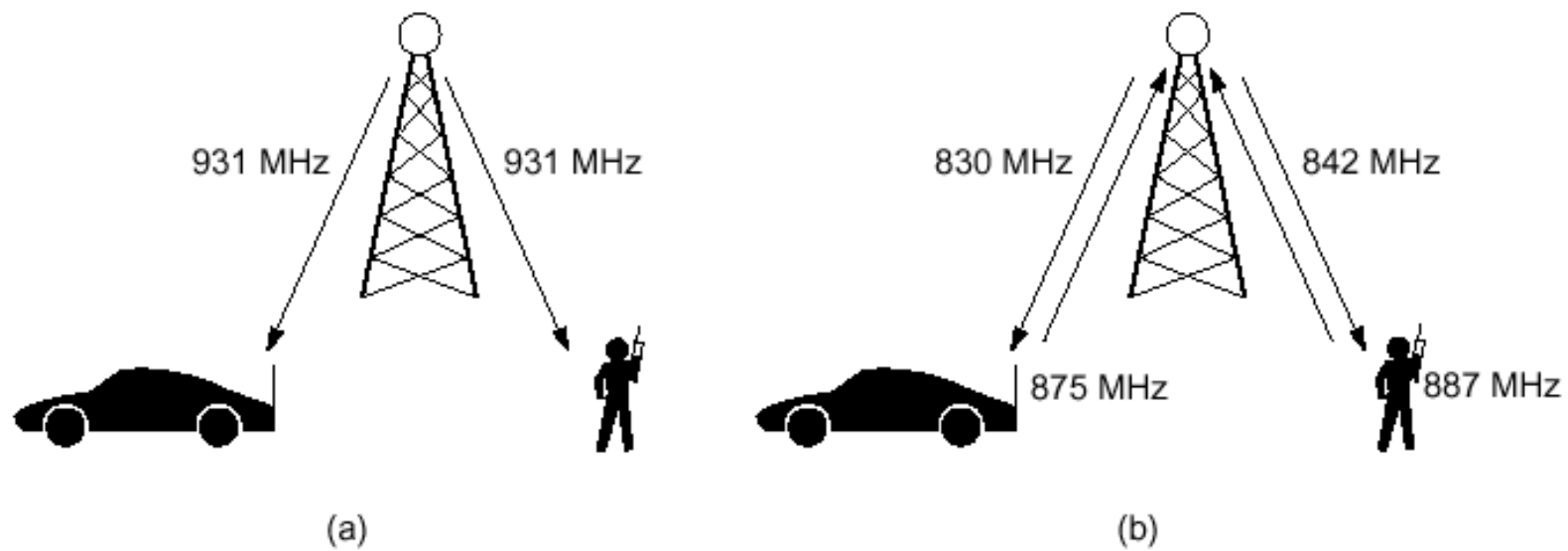


Fig. 2-53. (a) Paging systems are one way. (b) Mobile telephones are two way.



移动电话网络（续）

■ 蜂窝电话

- 第一代：模拟蜂窝电话，只能传送语音
- 第二代：数字蜂窝电话，主要传送语音，**GSM, CDMA**
- **3G / 4G**：可以传送语音和数据

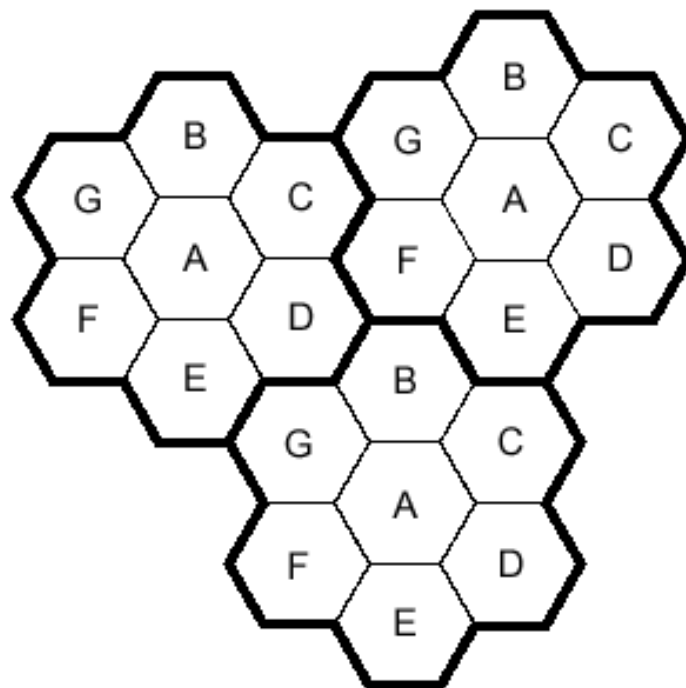
■ 模拟蜂窝电话

- 早期用于军事通信，**push-to-talk system**，一个信道，半双工
- **60年代, IMTS (Improved Mobile Telephone System)**，双频，全双工

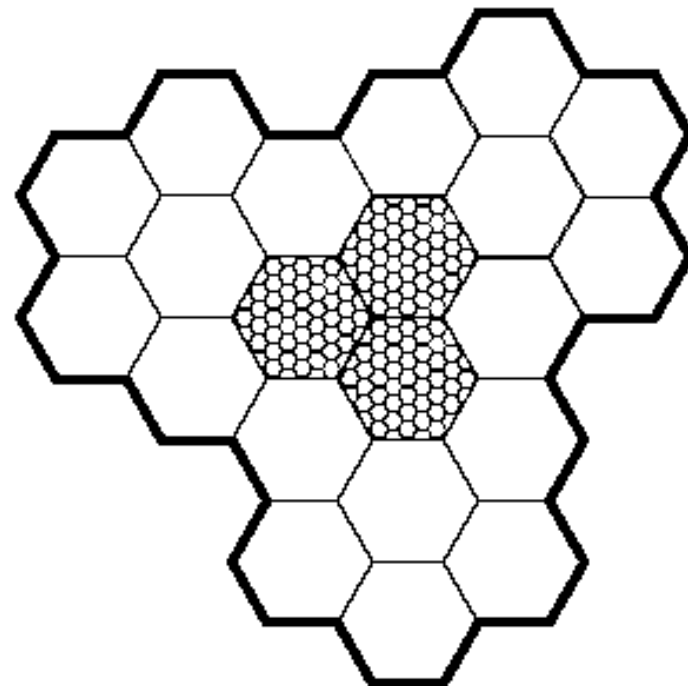


移动电话网络（续）

- **1982年，AMPS（Advanced Mobile Phone System）**
 - 使用小的蜂窝（**cell**）
 - 在附近（不相邻）的蜂窝中重用传输频率
 - 发射功率小，设备小而便宜
 - 当某个蜂窝内的用户超过系统容量时，将蜂窝划分成几个更小的蜂窝，以便重用频率，并将发射功率减弱
 - 在蜂窝中心，有一个基站（**base station**），基站包括一个计算机和与天线相连的收发器
 - 所有的基站通过包交换网络与**MTSO（Mobile Telephone Switching Office）**或**MSC（Mobile Switching Center）**相连，**MTSO**与电话系统相连
 - 安全问题：窃听、盗用



(a)



(b)

Fig. 2-54. (a) Frequencies are not reused in adjacent cells. (b) To add more users, smaller cells can be used.



总结

■ 物理层的定义

- 物理层提供机械的、电气的、功能的和规程的特性，目的是启动、维护和关闭数据链路实体之间进行比特传输的物理连接。

■ 物理层的特性

- 机械特性，电气特性，功能特性，规程特性

■ 传输介质

- 双绞线，同轴电缆，光纤

■ 网络传输技术

- 光纤传输，移动电话网络

The background of the slide features a large, light gray watermark of the Tsinghua University seal. The seal is circular and contains the university's name in English, 'TSINGHUA UNIVERSITY', around the perimeter. In the center, there is a traditional Chinese architectural structure, likely a gate or pavilion, with the year '-1911-' inscribed below it.

第五章

数据链路控制及其协议



主要内容

- 数据链路层概述
 - 定义和功能
 - 为网络层提供的服务
- 组帧
- 错误检测和纠正
 - 纠错码
 - 检错码
- 基本的数据链路层协议
 - 无约束单工协议
 - 单工停等协议
 - 有噪声信道的单工协议
- 滑动窗口协议
 - 一比特滑动窗口协议
 - 退后n帧重传协议
 - 选择重传协议
- 协议说明与验证
 - 协议形式化描述技术
 - 有限状态机模型
 - Petri网模型
- 常用的数据链路层协议
 - 高级数据链路控制规程HDLC
 - X.25的链路层协议LAPB
 - PPP协议



定义和功能

- 要解决的问题
 - 如何在有差错的线路上，进行无差错传输
- **ISO**关于数据链路层的定义
 - 数据链路层的目的是为了提供功能上和规程上的方法，以便建立、维护和释放网络实体间的数据链路



基本概念

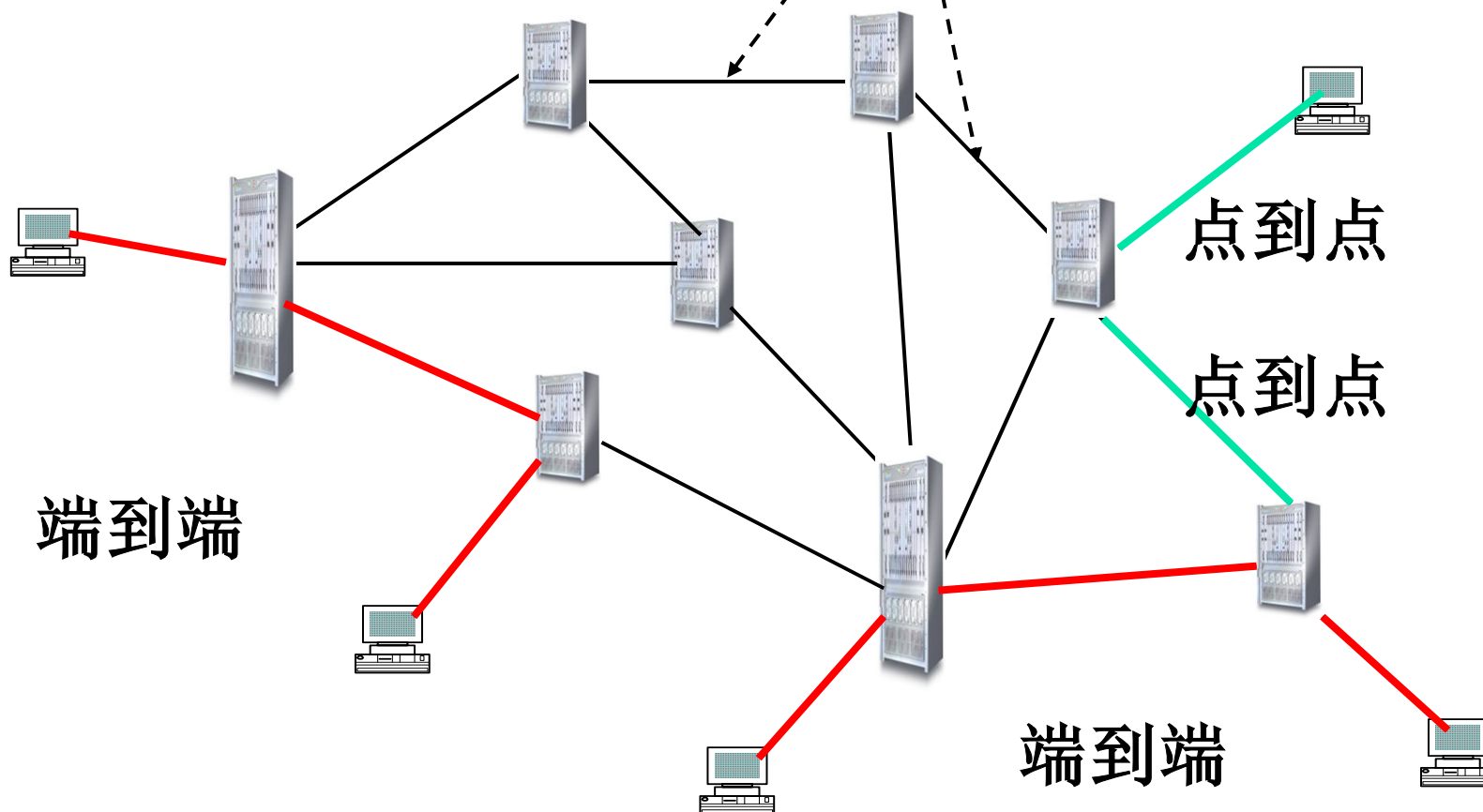
- 结点 (**node**)：网络中的主机和网络设备（路由器、交换机等）
- 链路 (**link**)：通信路径上连接相邻结点的通信信道
 - 数据链路层协议定义了一条链路的两个结点间交换的数据单元格式，以及结点发送和接收数据单元的动作
- 点到点 (**point to point**) 通信：一条链路上两个相邻结点间的通信
- 端到端 (**end to end**) 通信：从源结点到目的结点的通信，通信路径 (**path**) 可能由多个链路组成
- 实际数据通路与虚拟数据通路



结点

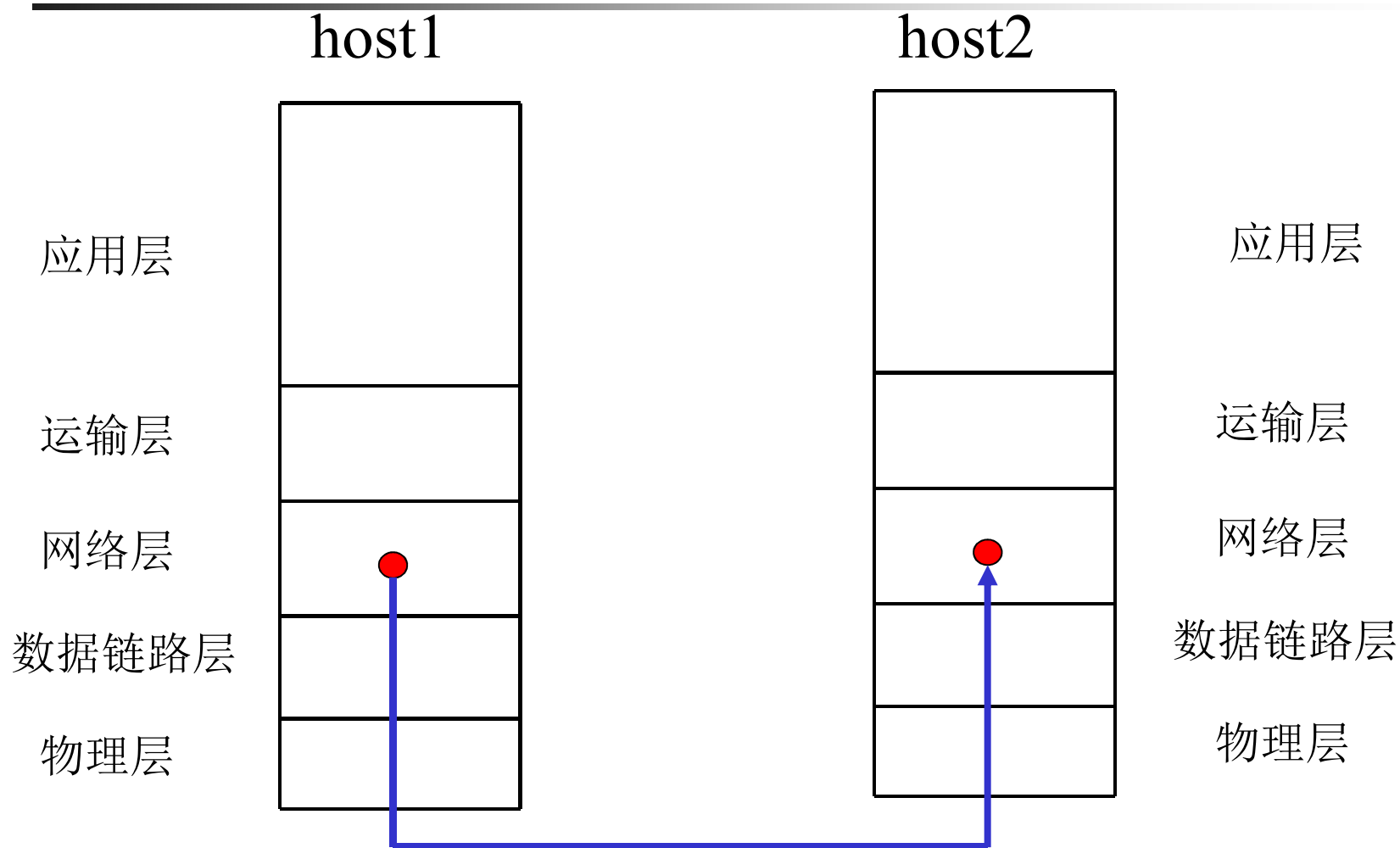


链路



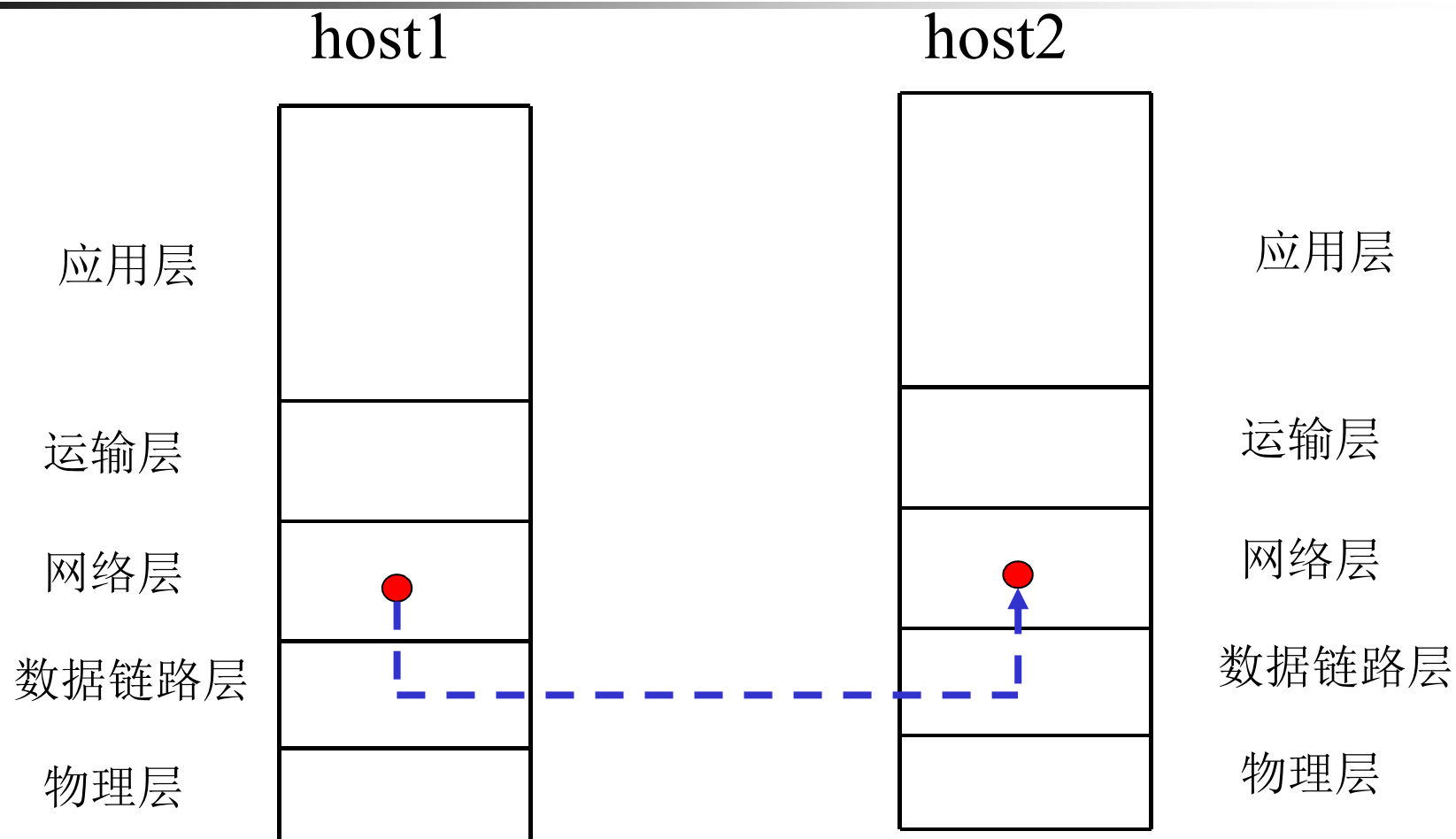


实际数据通路





虚拟数据通路





定义和功能（续）

- 数据链路控制规程
 - 为使数据能迅速、正确、有效地从发送点到达接收点所采用的控制方式
- 数据链路层协议应提供的基本功能
 - 数据在数据链路上的正常传输（建立、维护和释放）
 - 组帧：定界与同步，处理透明传输问题
 - 差错控制：检错和纠错
 - 顺序控制（可选）
 - 流量控制（可选）：基于反馈机制



为网络层提供的服务

- 为网络层提供三种合理的服务
 - 无确认无连接服务，适用于
 - 误码率很低的线路，错误恢复留给高层
 - 实时业务
 - 大部分局域网
 - 有确认无连接服务，适用于不可靠的信道，如无线网
 - 有确认有连接服务



主要内容

- 数据链路层概述
 - 定义和功能
 - 为网络层提供的服务
- 组帧
- 错误检测和纠正
 - 纠错码
 - 检错码
- 基本的数据链路层协议
 - 无约束单工协议
 - 单工停等协议
 - 有噪声信道的单工协议
- 滑动窗口协议
 - 一比特滑动窗口协议
 - 退后n帧重传协议
 - 选择重传协议
- 协议说明与验证
 - 协议形式化描述技术
 - 有限状态机模型
 - Petri网模型
- 常用的数据链路层协议
 - 高级数据链路控制规程HDLC
 - X.25的链路层协议LAPB
 - PPP协议



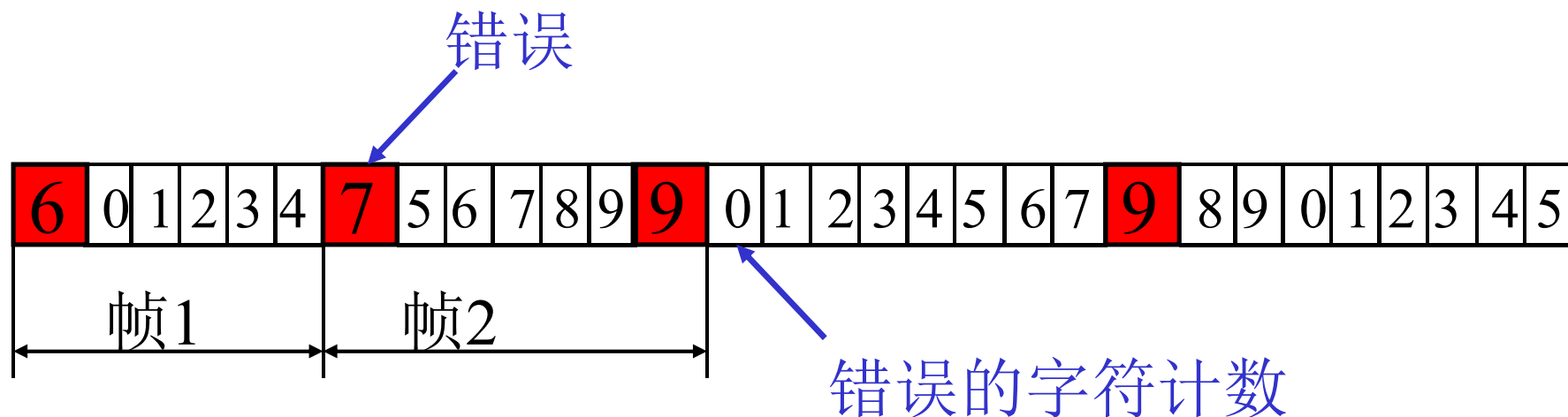
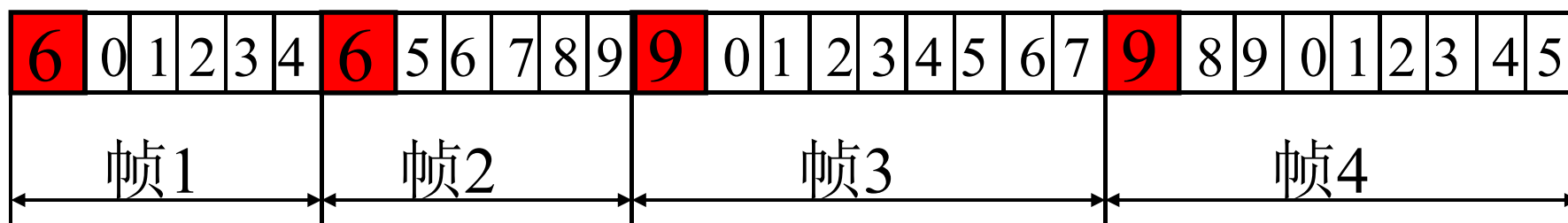
组帧 (Framing)

- 将比特流分成离散的帧，并计算每个帧的校验和
- 组帧方法
 - 字符计数法
 - 带字符填充的字符定界法
 - 带位填充的标记定界法
 - 物理层编码违例法
- 注意：在很多数据链路协议中，使用字符计数法和一种其它方法的组合



字符计数法

- 在帧头中用一个域来表示整个帧的字符个数
- 缺点：若计数出错，对本帧和后面的帧有影响



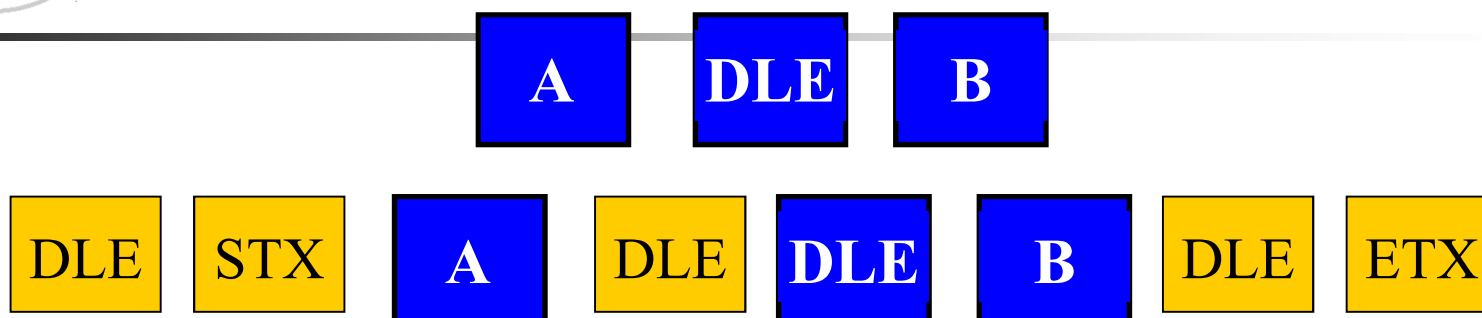


带字符填充的字符定界法

- 起始字符 **DLE STX**，结束字符**DLE ETX**
 - **DLE:Data Link Escape**
 - **STX:Start of Text**
 - **ETX:End of Text**
- 字符填充
- 缺点： 局限于**8**位字符和**ASCII**字符传送

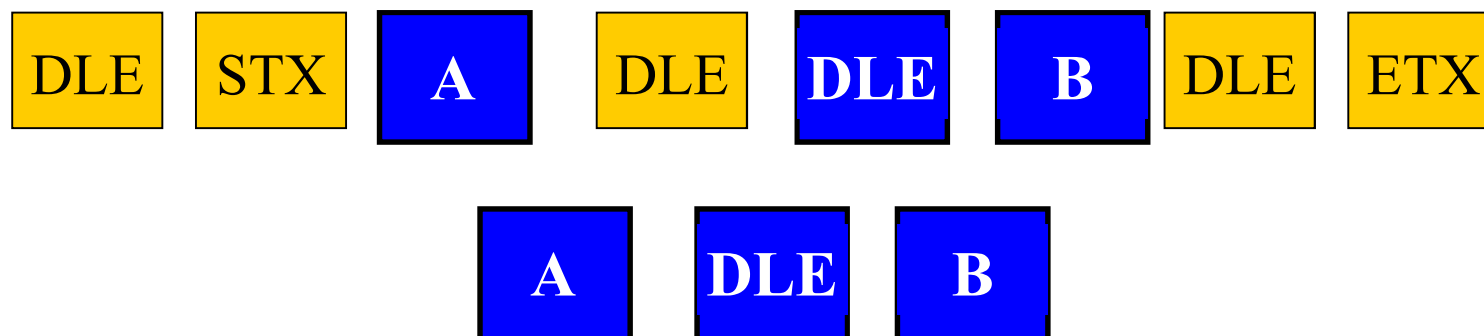


发送方



填充DLE

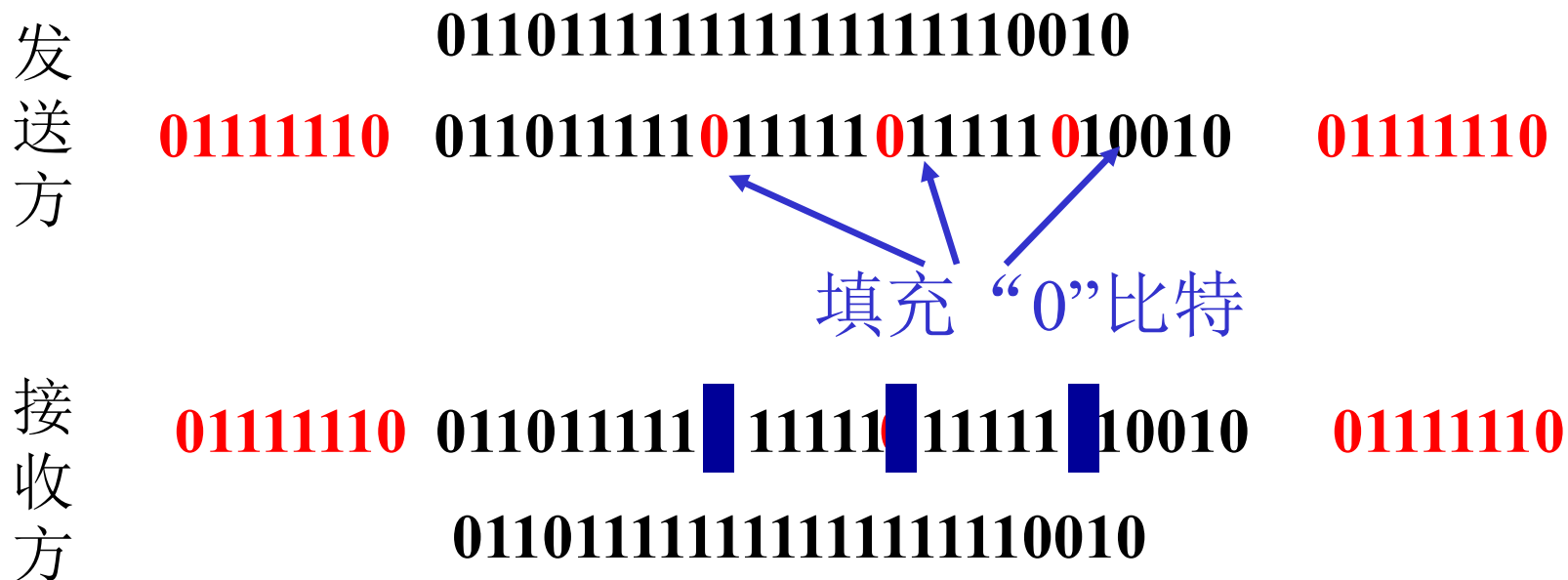
接收方





带位填充的标记定界法

- 帧的起始和结束都用一个特殊的位串“**01111110**”，称为标记(flag)
- “0”比特插入删除技术





物理层编码违例法

- 只适用于物理层编码有冗余的网络
- **802 LAN:** 曼彻斯特编码用**high-low pair/low-high pair**表示**1/0**, **high-high/low-low**不表示数据, 可以用来做定界符



主要内容

- 数据链路层概述
 - 定义和功能
 - 为网络层提供的服务
- 成帧
- 错误检测和纠正
 - 纠错码
 - 检错码
- 基本的数据链路层协议
 - 无约束单工协议
 - 单工停等协议
 - 有噪声信道的单工协议
- 滑动窗口协议
 - 一比特滑动窗口协议
 - 退后n帧重传协议
 - 选择重传协议
- 协议说明与验证
 - 协议形式化描述技术
 - 有限状态机模型
 - Petri网模型
- 常用的数据链路层协议
 - 高级数据链路控制规程HDLC
 - X.25的链路层协议LAPB
 - PPP协议



错误检测和纠正

- 差错出现的特点：随机，连续突发 (**burst**)
- 处理差错的两种基本策略
 - 使用纠错码：发送方在每个数据块中加入足够的冗余信息，使得接收方能够判断接收到的数据是否有错，并能纠正错误
 - 使用检错码：发送方在每个数据块中加入足够的冗余信息，使得接收方能够判断接收到的数据是否有错，但不能判断哪里有错



纠错码

- 码字 (**codeword**) : 一个帧包括 m 个数据位, r 个校验位, $n = m + r$, 则此 n 比特单元称为 n 位码字
- 海明距离 (**Hamming distance**) : 两个码字之间不同的对应比特位数目
 - 例: **0000000000** 与 **0000011111** 的海明距离为 **5**
 - 如果两个码字的海明距离为 d , 则需要 d 个单比特错就可以把一个码字转换成另一个码字
 - 为了检查出 d 个错 (比特错), 可以使用海明距离为 $d + 1$ 的编码
 - 为了纠正 d 个错, 可以使用海明距离为 $2d + 1$ 的编码



纠错码（续）

- 最简单的例子是奇偶校验，在数据后添加一个奇偶位

- 例：使用偶校验（“1”的个数为偶数）

10110101 —> 101101011

10110001 —> 101100010

- 奇偶校验可以用来检查奇数个错误

- 设计纠错码

- 要求： m 个信息位， r 个校验位，纠正单比特错
- 对 2^m 个有效信息中任何一个，有 n 个与其距离为1的无效码字，因此有： $(n + 1) 2^m \leq 2^n$
- 利用 $n = m + r$ ，得到 $(m + r + 1) \leq 2^r$ 。给定 m ，利用该式可以得出校正单比特误码的校验位数目的下界



纠错码（续）

■ 海明码

- 码位从左边开始编号，从“**1**”开始
- 位号为**2**的幂的位是校验位，其余是信息位
- 每个校验位使得包括自己在内的一些位的奇偶值为偶数（或奇数）
- 为看清数据位**k**对哪些校验位有影响，将**k**写成**2**的幂的和。例： **$11 = 1 + 2 + 8$**

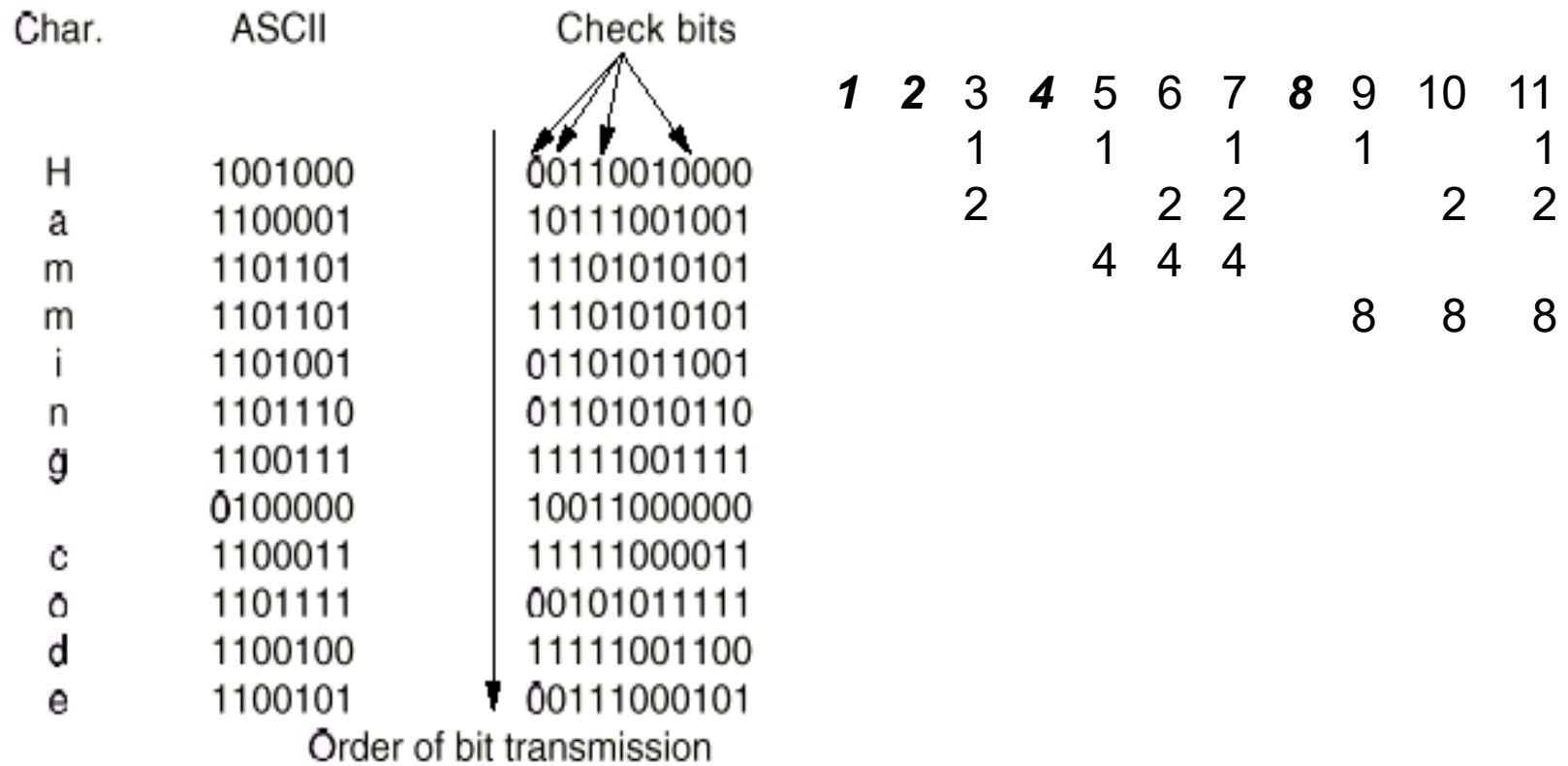


Fig. 3-6. Use of a Hamming code to correct burst errors.



纠错码 (续)

■ 海明码工作过程

- 每个码字到来前，接收方计数器清零
- 接收方检查每个校验位 k ($k = 1, 2, 4 \dots$) 的奇偶值是否正确
- 若第 k 位奇偶值不对，计数器加 k
- 所有校验位检查完后，若计数器值为 0 ，则码字有效；若计数器值为 m ，则第 m 位出错。例：若校验位 1 、 2 、 8 出错，则第 11 位变反

■ 使用海明码纠正突发错误

- 可采用 k 个码字 ($n = m + r$) 组成 $k \times n$ 矩阵，按列发送，接收方恢复成 $k \times n$ 矩阵
- kr 个校验位， km 个数据位，可纠正最多为 k 个的突发性连续比特错



检错码

- 使用纠错码传数据，效率低，适用于不可能重传的场合；大多数情况采用检错码加重传
- 循环冗余码（**CRC**码，多项式编码）
 - **110001**，表示成多项式 $x^5 + x^4 + 1$
- 生成多项式 **$G(x)$**
 - 发方、收方事前商定
 - 生成多项式的高位和低位必须为**1**
 - 生成多项式必须比传输信息对应的多项式短



检错码（续）

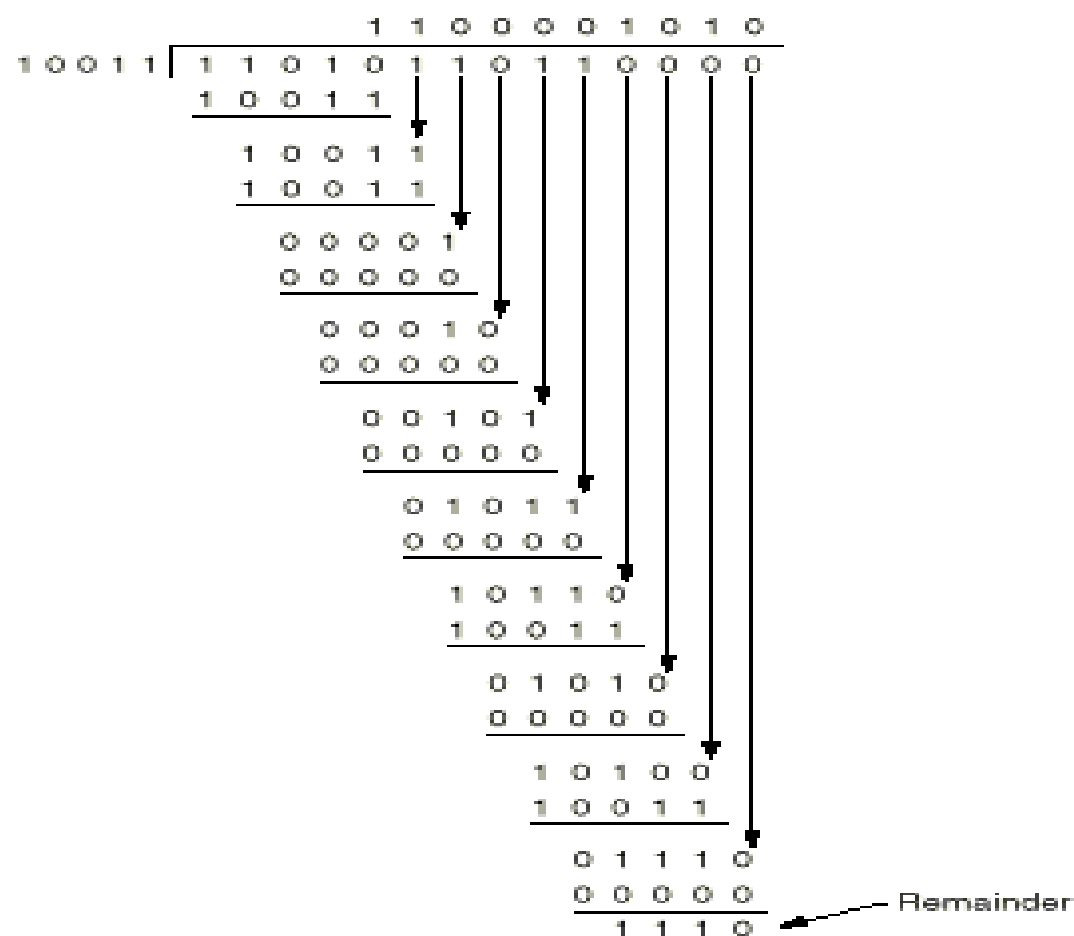
■ CRC码基本思想

- 校验和（**checksum**）加在帧尾，使带校验和的帧的多项式能被 **$G(x)$** 除尽；收方接收时，用 **$G(x)$** 去除它，若有余数，则传输出错

■ 校验和计算算法

- 设 **$G(x)$** 为 r 阶，在帧的末尾加 r 个0，使帧为 $m + r$ 位，相应多项式为 **$x^r M(x)$**
- 按模2除法用对应于 **$G(x)$** 的位串去除对应于 **$x^r M(x)$** 的位串
- 按模2减法从对应于 **$x^r M(x)$** 的位串中减去余数(等于或小于 r 位)，结果就是要传送的带校验和的多项式 **$T(x)$**

Message after appending 4 zero bits: 1 1 0 1 0 1 1 0 0 0 0



Transmitted frame: 1 1 0 1 0 1 1 0 1 1 1 1 0

Fig. 3-7. Calculation of the polynomial code checksum.



检错码 (续)

■ CRC的检错能力

- 发送: $T(x)$
- 接收: $T(x) + E(x)$, $E(x) \neq 0$
- 余数 $((T(x) + E(x)) / G(x)) = 0 + \text{余数}(E(x) / G(x))$
- 若余数 $(E(x) / G(x)) = 0$, 则差错不能发现; 否则, 可以发现



检错码 (续)

■ CRC检错能力的几种情况分析

- 如果只有单比特错, 即 $E(x) = x^i$, 而 $G(x)$ 中至少有两项, 余数 $(E(x) / G(x)) \neq 0$, 所以可以查出单比特错
- 如果发生两个孤立单比特错, 即 $E(x) = x^i + x^j = x^j (x^{i-j} + 1)$, $G(x)$ 不能被 x 整除, 那么能够发现两个比特错的充分条件是: $x^k + 1$ 不能被 $G(x)$ 整除 ($k \leq i - j$)
- 如果有奇数个比特错, 即 $E(x)$ 包括奇数个项, $G(x)$ 选 $(x + 1)$ 的倍数就能查出奇数个比特错
- 具有 r 个校验位的多项式能检查出所有长度 $\leq r$ 的突发性差错。长度为 k 的突发性连续差错可表示为 $x^i (x^{k-1} + \dots + 1)$, 若 $G(x)$ 包括 x^0 项, 且 $k - 1$ 小于 $G(x)$ 的阶, 则余数 $(E(x) / G(x)) \neq 0$
- 如果突发差错长度为 $r + 1$, 当且仅当突发差错和 $G(x)$ 一样时, 余数 $(E(x) / G(x)) = 0$, 概率为 $1/2^{r-1}$
- 长度大于 $r + 1$ 的突发差错或几个较短的突发差错发生后, 坏帧被接收的概率为 $1/2^r$



检错码 (续)

- 四个国际标准生产多项式

- **CRC-12** $= x^{12} + x^{11} + x^3 + x^2 + x + 1$

- **CRC-16** $= x^{16} + x^{15} + x^2 + 1$

- **CRC-CCITT** $= x^{16} + x^{12} + x^5 + 1$

- **CRC-32** $= x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$

- 硬件实现**CRC**校验

- 网卡**NIC** (**N**etwork **I**nterface **C**ard)



主要内容

- 数据链路层概述
 - 定义和功能
 - 为网络层提供的服务
- 成帧
- 错误检测和纠正
 - 纠错码
 - 检错码
- 基本的数据链路层协议
 - 无约束单工协议
 - 单工停等协议
 - 有噪声信道的单工协议
- 滑动窗口协议
 - 一比特滑动窗口协议
 - 退后n帧重传协议
 - 选择重传协议
- 协议说明与验证
 - 协议形式化描述技术
 - 有限状态机模型
 - Petri网模型
- 常用的数据链路层协议
 - 高级数据链路控制规程 HDLC
 - X.25的链路层协议LAPB
 - PPP协议

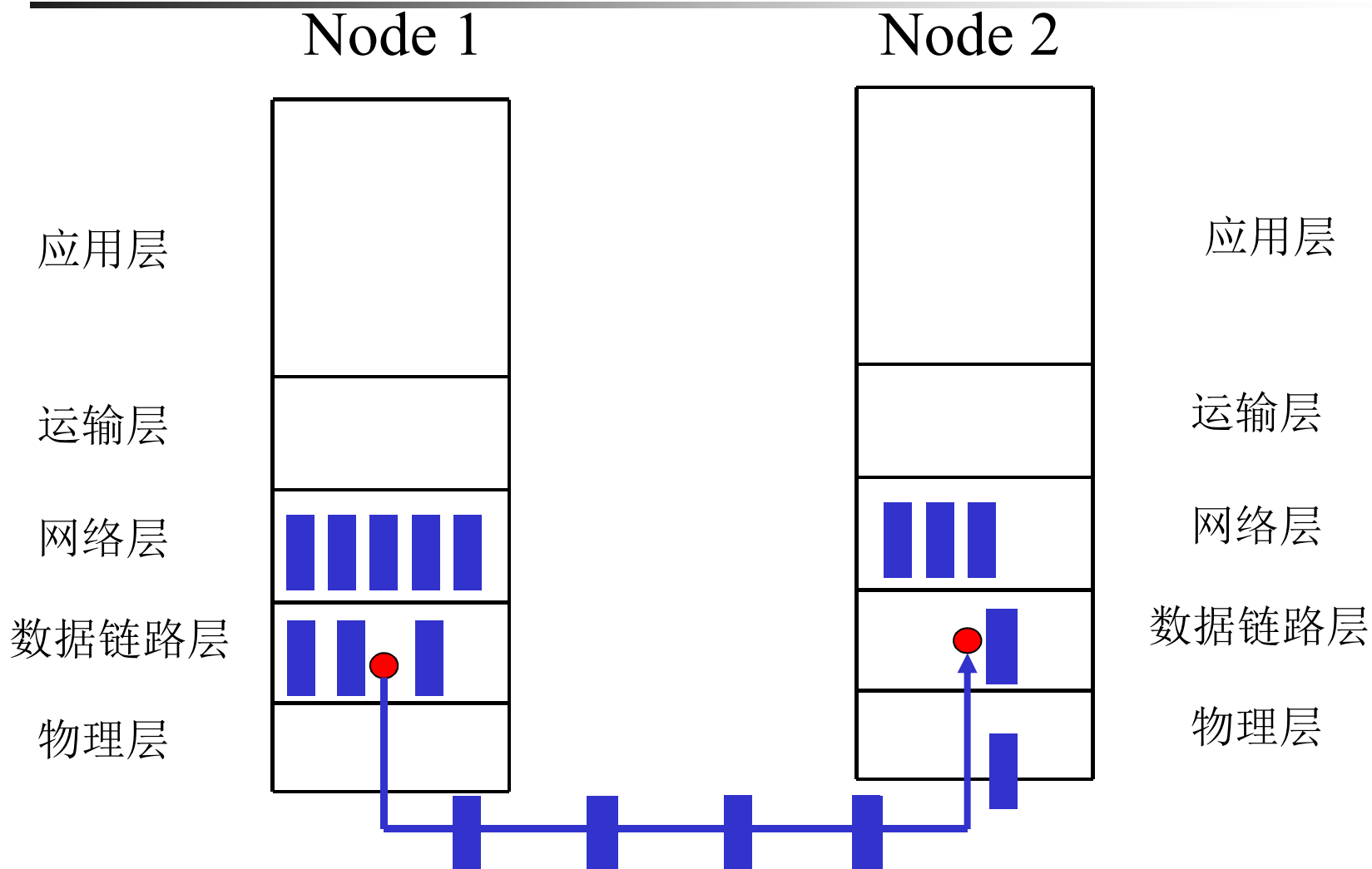


无约束单工协议

- **An Unrestricted Simplex Protocol**
- 工作在理想情况，几个前提
 - 单工传输
 - 发送方无休止工作（要发送的信息无限多）
 - 接收方无休止工作（缓冲区无限大）
 - 通信线路（信道）不损坏或丢失信息帧
- 工作过程
 - 发送程序：取数据，构成帧，发送帧
 - 接收程序：等待，接收帧，送数据给高层



无约束单工协议（续）





/* Protocol 1 (utopia) provides for data transmission in one direction only, from sender to receiver. The communication channel is assumed to be error free, and the receiver is assumed to be able to process all the input infinitely fast. Consequently, the sender just sits in a loop pumping data out onto the line as fast as it can. */

```
typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender1(void)
{
    frame s;                /* buffer for an outbound frame */
    packet buffer;          /* buffer for an outbound packet */

    while (true) {
        from_network_layer(&buffer);    /* go get something to send */
        s.info = buffer;                /* copy it into s for transmission */
        to_physical_layer(&s);          /* send it on its way */
    }
    /* Tomorrow, and tomorrow, and tomorrow,
       Creeps in this petty pace from day to day
       To the last syllable of recorded time
       - Macbeth, V, v */
}

void receiver1(void)
{
    frame r;
    event_type event;        /* filled in by wait, but not used here */

    while (true) {
        wait_for_event(&event); /* only possibility is frame_arrival */
        from_physical_layer(&r); /* go get the inbound frame */
        to_network_layer(&r.info); /* pass the data to the network layer */
    }
}
```

Fig. 3-9. An unrestricted simplex protocol.

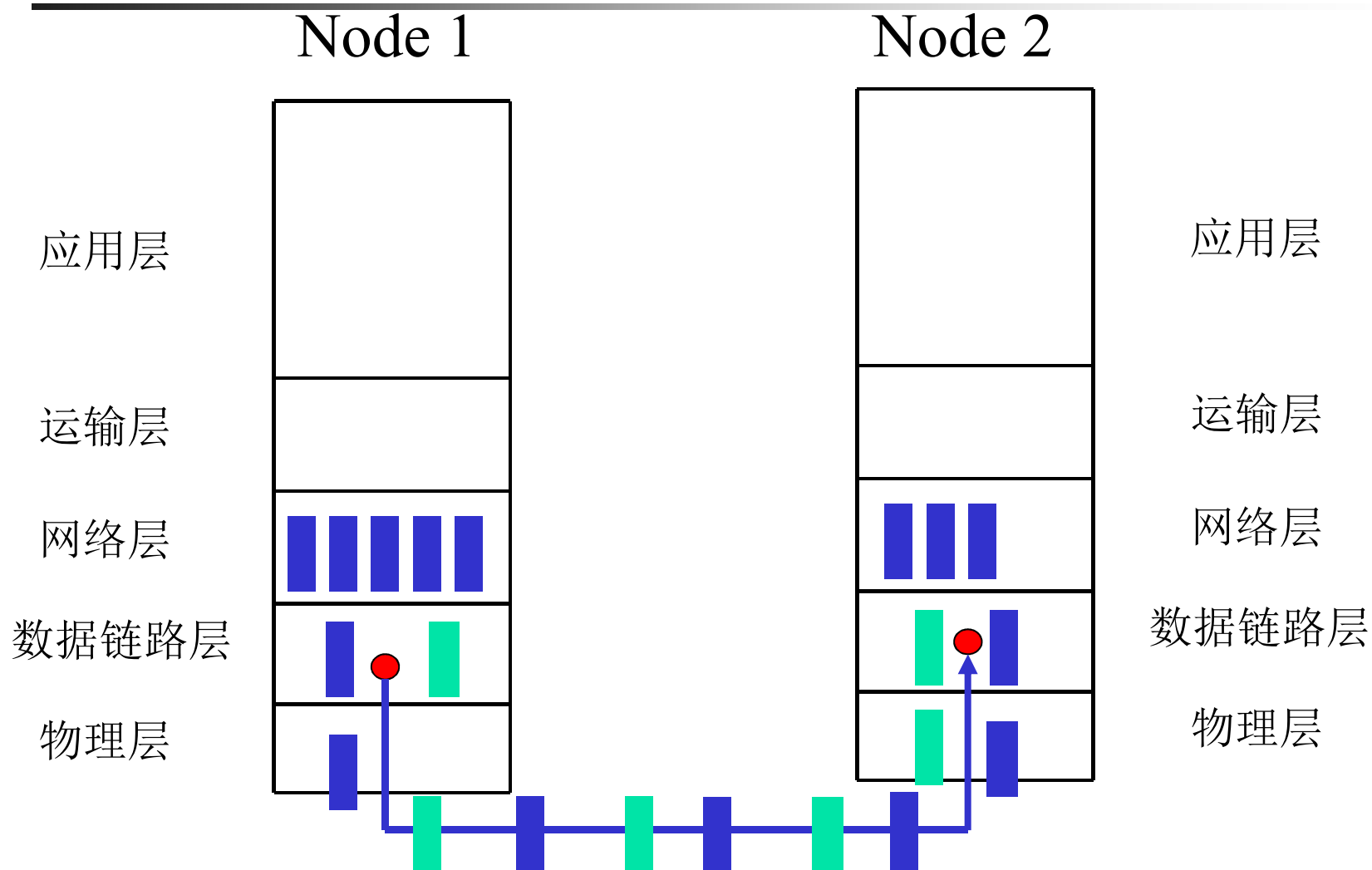


单工停等协议

- **A Simplex Stop-and-Wait Protocol**
- 增加约束条件：接收方不能无休止接收
- 解决办法：接收方每收到一个帧后，给发送方回送一个响应
- 工作过程
 - 发送程序：取数据，组帧，发送帧，等待响应帧
 - 接收程序：等待，接收帧，送数据给高层，回送响应帧



单工停等协议 (续)





/* Protocol 2 (stop-and-wait) also provides for a one-directional flow of data from sender to receiver. The communication channel is once again assumed to be error free, as in protocol 1. However, this time, the receiver has only a finite buffer capacity and a finite processing speed, so the protocol must explicitly prevent the sender from flooding the receiver with data faster than it can be handled. */

```
typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender2(void)
{
    frame s;                /* buffer for an outbound frame */
    packet buffer;          /* buffer for an outbound packet */
    event_type event;       /* frame_arrival is the only possibility */

    while (true) {
        from_network_layer(&buffer);          /* go get something to send */
        s.info = buffer;                      /* copy it into s for transmission */
        to_physical_layer(&s);                /* bye bye little frame */
        wait_for_event(&event);               /* do not proceed until given the go ahead */
    }
}

void receiver2(void)
{
    frame r, s;                /* buffers for frames */
    event_type event;          /* frame_arrival is the only possibility */
    while (true) {
        wait_for_event(&event); /* only possibility is frame_arrival */
        from_physical_layer(&r); /* go get the inbound frame */
        to_network_layer(&r.info); /* pass the data to the network layer */
        to_physical_layer(&s);     /* send a dummy frame to awaken sender */
    }
}
```

Fig. 3-10. A simplex stop-and-wait protocol.



有噪声信道的单工协议

- **A Simplex Protocol for a Noisy Channel**
- 增加约束条件：信道（线路）有差错，信息帧可能损坏或丢失
- 解决办法：出错重传
- 带来的问题：
 - 什么时候重传 —— 定时
 - 响应帧损坏怎么办（重复帧） —— 发送帧头中放入序号
 - 为了使帧头精简，序号取多少位 —— **1**位

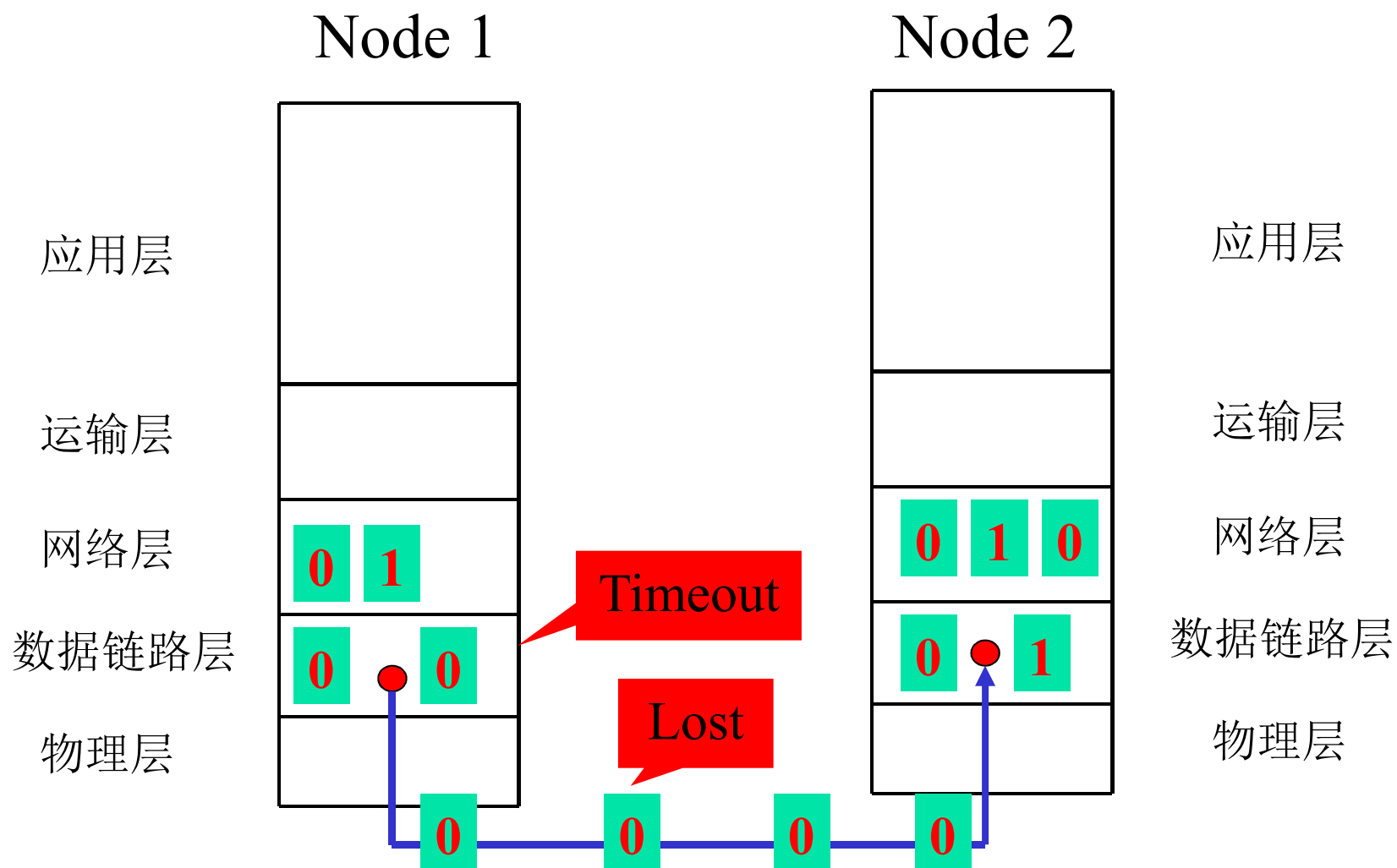


有噪声信道的单工协议（续）

- 发方在发下一个帧之前等待一个肯定确认的协议叫做**PAR**或**ARQ**
 - **PAR: Positive Acknowledgement with Retransmission**
 - **ARQ: Automatic Repeat reQuest**
- 工作过程



有噪声信道的单工协议（续）





```

/* Protocol 3 (par) allows unidirectional data flow over an unreliable channel. */
#define MAX_SEQ 1 /* must be 1 for protocol 3 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void sender3(void)
{
    seq_nr next_frame_to_send; /* seq number of next outgoing frame */
    frame s; /* scratch variable */
    packet buffer; /* buffer for an outbound packet */
    event_type event;

    next_frame_to_send = 0; /* initialize outbound sequence numbers */
    from_network_layer(&buffer); /* fetch first packet */
    while (true) {
        s.info = buffer; /* construct a frame for transmission */
        s.seq = next_frame_to_send; /* insert sequence number in frame */
        to_physical_layer(&s); /* send it on its way */
        start_timer(s.seq); /* if (answer takes too long, time out */
        wait_for_event(&event); /* frame_arrival, cksum_err, timeout */
        if (event == frame_arrival) {
            from_physical_layer(&s); /* get the acknowledgement */
            if (s.ack == next_frame_to_send) {
                from_network_layer(&buffer); /* get the next one to send */
                inc(next_frame_to_send); /* increment next_frame_to_send */
            }
        }
    }
}

void receiver3(void)
{
    seq_nr frame_expected;
    frame r, s;
    event_type event;
    frame_expected = 0;
    while (true) {
        wait_for_event(&event); /* possibilities: frame_arrival, cksum_err */
        if (event == frame_arrival) { /* a valid frame has arrived. */
            from_physical_layer(&r); /* go get the newly arrived frame */
            if (r.seq == frame_expected) { /* this is what we have been waiting for. */
                to_network_layer(&r.info); /* pass the data to the network layer */
                inc(frame_expected); /* next time expect the other sequence nr */
            }
            s.ack = 1 - frame_expected; /* tell which frame is being acked */
            to_physical_layer(&s); /* none of the fields are used */
        }
    }
}

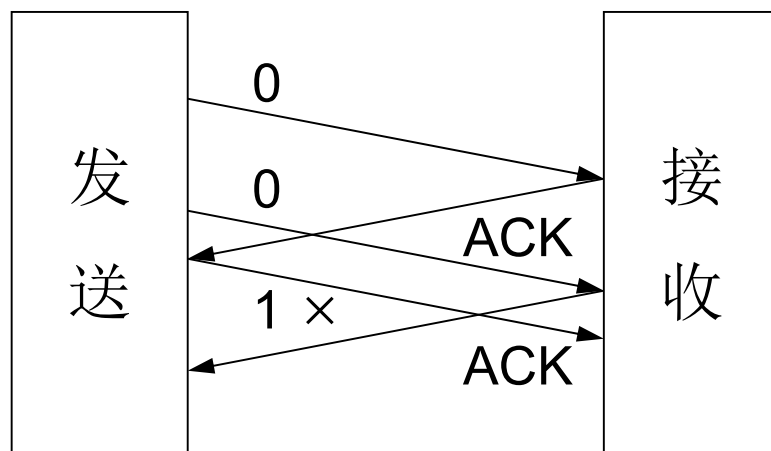
```

Fig. 3-11. A positive acknowledgement/retransmission protocol.



有噪声信道的单工协议（续）

- 如果确认帧没有序号，则协议**3**有漏洞
 - 由于确认帧中没有序号，超时时间不能太短，否则协议失败





主要内容

- 数据链路层概述
 - 定义和功能
 - 为网络层提供的服务
- 成帧
- 错误检测和纠正
 - 纠错码
 - 检错码
- 基本的数据链路层协议
 - 无约束单工协议
 - 单工停等协议
 - 有噪声信道的单工协议
- 滑动窗口协议
 - 一比特滑动窗口协议
 - 退后n帧重传协议
 - 选择重传协议
- 协议说明与验证
 - 协议形式化描述技术
 - 有限状态机模型
 - Petri网模型
- 常用的数据链路层协议
 - 高级数据链路控制规程 HDLC
 - X.25的链路层协议LAPB
 - PPP协议



滑动窗口协议

- 单工 ——> 全双工
- 捎带/载答 (**piggybacking**)：暂时延迟待发确认，以便附加在下一个待发数据帧的技术
 - 优点：充分利用信道带宽，减少帧的数目意味着减少“帧到达”中断
 - 带来的问题：复杂
- 本节的三个协议统称滑动窗口协议，都能在实际（非理想）环境下正常工作，区别仅在于效率、复杂性和对缓冲区的要求



滑动窗口协议（续）

- 滑动窗口协议（**Sliding Window Protocol**）工作原理
 - 发送的信息帧都有一个序号，从**0**到某个最大值，**0 ~ $2^n - 1$** ，一般用**n**个二进制位表示
 - 发送端始终保持一个已发送但尚未确认的帧的序号表，称为发送窗口。发送窗口的上界表示要发送的下一个帧的序号，下界表示未得到确认的帧的最小序号。发送窗口大小 = 上界 - 下界，大小可变
 - 发送端每发送一个帧，序号取上界值，上界加**1**；每接收到一个确认序号 = 发送窗口下界的正确响应帧，下界加**1**；若确认序号落在发送窗口之内，则发送窗口下界连续加**1**，直到发送窗口下界 = 确认序号 + **1**（累计确认）

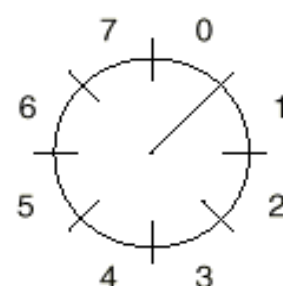
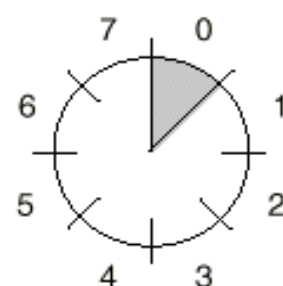
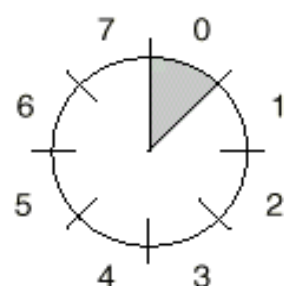
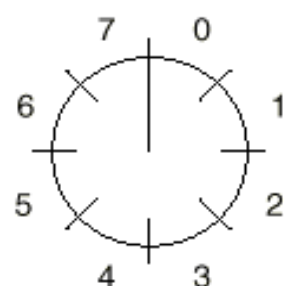


滑动窗口协议（续）

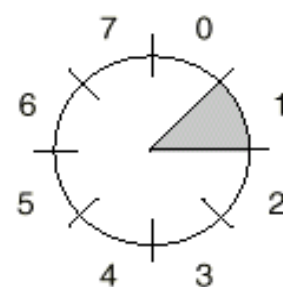
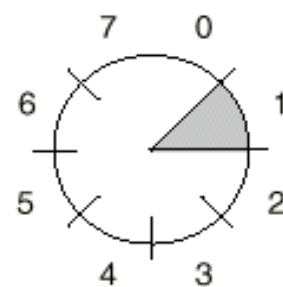
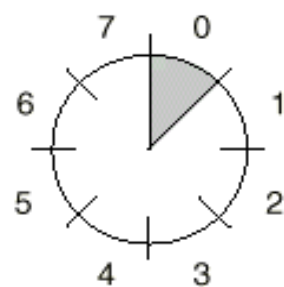
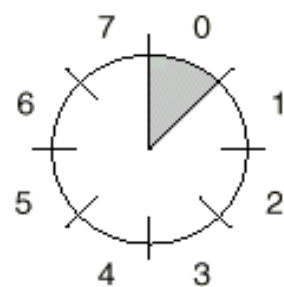
- 接收端有一个接收窗口，大小固定，但不一定与发送窗口相同。接收窗口的上界表示允许接收的最大序号，下界表示希望接收的序号
- 接收窗口容纳允许接收的信息帧，落在窗口外的帧均被丢弃。序号等于下界的帧被正确接收，并产生一个响应帧，上界、下界都加**1**。接收窗口大小不变



Sender



Receiver



(a)

(b)

(c)

(d)

Fig. 3-12. A sliding window of size 1, with a 3-bit sequence number. (a) Initially. (b) After the first frame has been sent. (c) After the first frame has been received. (d) After the first acknowledgement has been received.



一比特滑动窗口协议

- 协议特点
 - 窗口大小: $N = 1$
 - 发送序号和接收序号的取值范围: 0, 1
 - 可进行数据双向传输, 信息帧中可含有确认信息 (**piggybacking** 技术)
 - 信息帧中包括两个序号域: 发送序号和确认序号 (已经正确收到的帧的序号)
- 工作过程



/* Protocol 4 (sliding window) is bidirectional and is more robust than protocol 3. */

#define MAX_SEQ 1 /* must be 1 for protocol 4 */

— typedef enum {frame_arrival, cksum_err, timeout} event_type;

#include "protocol.h"

void protocol4 (void)

{

seq_nr next_frame_to_send; /* 0 or 1 only */

seq_nr frame_expected; /* 0 or 1 only */

frame r, s; /* scratch variables */

packet buffer; /* current packet being sent */

event_type event;

next_frame_to_send = 0; /* next frame on the outbound stream */

frame_expected = 0; /* number of frame arriving frame expected */

from_network_layer(&buffer); /* fetch a packet from the network layer */

s.info = buffer; /* prepare to send the initial frame */

s.seq = next_frame_to_send; /* insert sequence number into frame */

s.ack = 1 - frame_expected; /* piggybacked ack */

to_physical_layer(&s); /* transmit the frame */

start_timer(s.seq); /* start the timer running */



```

while (true) {
    wait_for_event(&event);          /* frame_arrival, cksum_err, or timeout */
    if (event == frame_arrival) { /* a frame has arrived undamaged. */
        from_physical_layer(&r);    /* go get it */

        if (r.seq == frame_expected) {
            /* Handle inbound frame stream. */
            to_network_layer(&r.info); /* pass packet to network layer */
            inc(frame_expected); /* invert sequence number expected next */
        }

        if (r.ack == next_frame_to_send) { /* handle outbound frame stream. */
            from_network_layer(&buffer); /* fetch new pkt from network layer */
            inc(next_frame_to_send); /* invert sender's sequence number */
        }
    }
    s.info = buffer; /* construct outbound frame */
    s.seq = next_frame_to_send; /* insert sequence number into it */
    s.ack = 1 - frame_expected; /* seq number of last received frame */
    to_physical_layer(&s); /* transmit a frame */
    start_timer(s.seq); /* start the timer running */
}
}

```

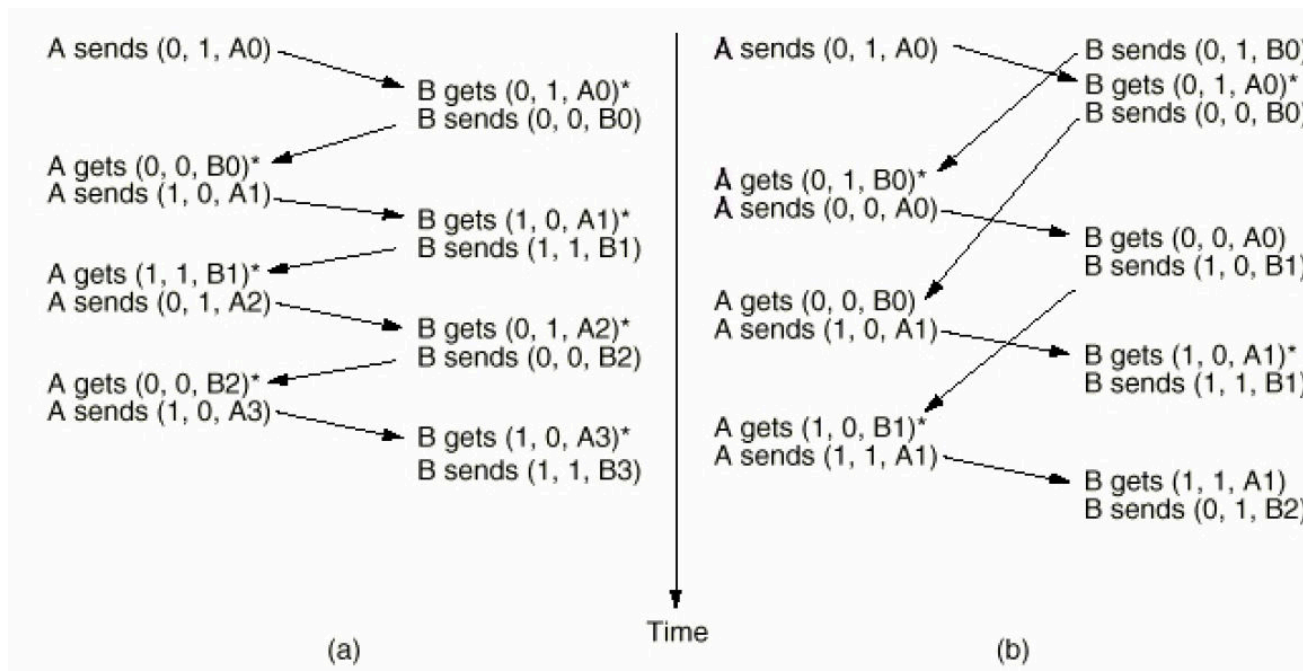
Fig. 3-13. A 1-bit sliding window protocol.



一 比特滑动窗口协议 (续)

■ 存在问题

- 能保证无差错传输，但是基于停等方式
- 若双方同时开始发送，则会有有一半重复帧
- 效率低，传输时间长





滑动窗口协议（续）

- 一比特滑动窗口协议传输效率低
- 例：卫星信道传输速率**50kbps**，往返传输延迟**500ms**，若传**1000bit**的帧，使用协议**4**
 - 则传输一个帧所需时间 = 发送时间 + 信息信道延迟 + 确认信道延迟（确认帧很短，忽略发送时间） = $1000\text{bit} / 50\text{kbps} + 500\text{ms} = 520\text{ms}$
 - 信道利用率 = $20 / 520 \approx 4\%$



滑动窗口协议（续）

- 一般情况，信道带宽 **b** 比特/秒，帧长度 **L** 比特，往返传输延迟 **R** 秒
 - 则信道利用率为 $(L/b) / (L/b + R) = L / (L + Rb)$
- 结论
 - 传输延迟大，信道带宽高，帧短时，信道利用率低
- 解决办法
 - 流水线技术：连续发送多帧后再等待确认
- 带来的问题
 - 信道误码率高时，对损坏帧和非损坏帧的重传非常多



滑动窗口协议（续）

■ 两种改进方法

■ 退后n帧重传（go back n）

- 发送窗口大于1，接收窗口为1
- 接收方从坏帧起丢弃所有后继帧，发送方从坏帧开始重传
- 对于出错率较高的信道，浪费带宽

■ 选择重传（selective repeat）

- 发送窗口大于1，接收窗口大于1
- 接收方可暂存坏帧的后继帧，发送方只重传坏帧
- 接收窗口较大时，需较大缓冲区

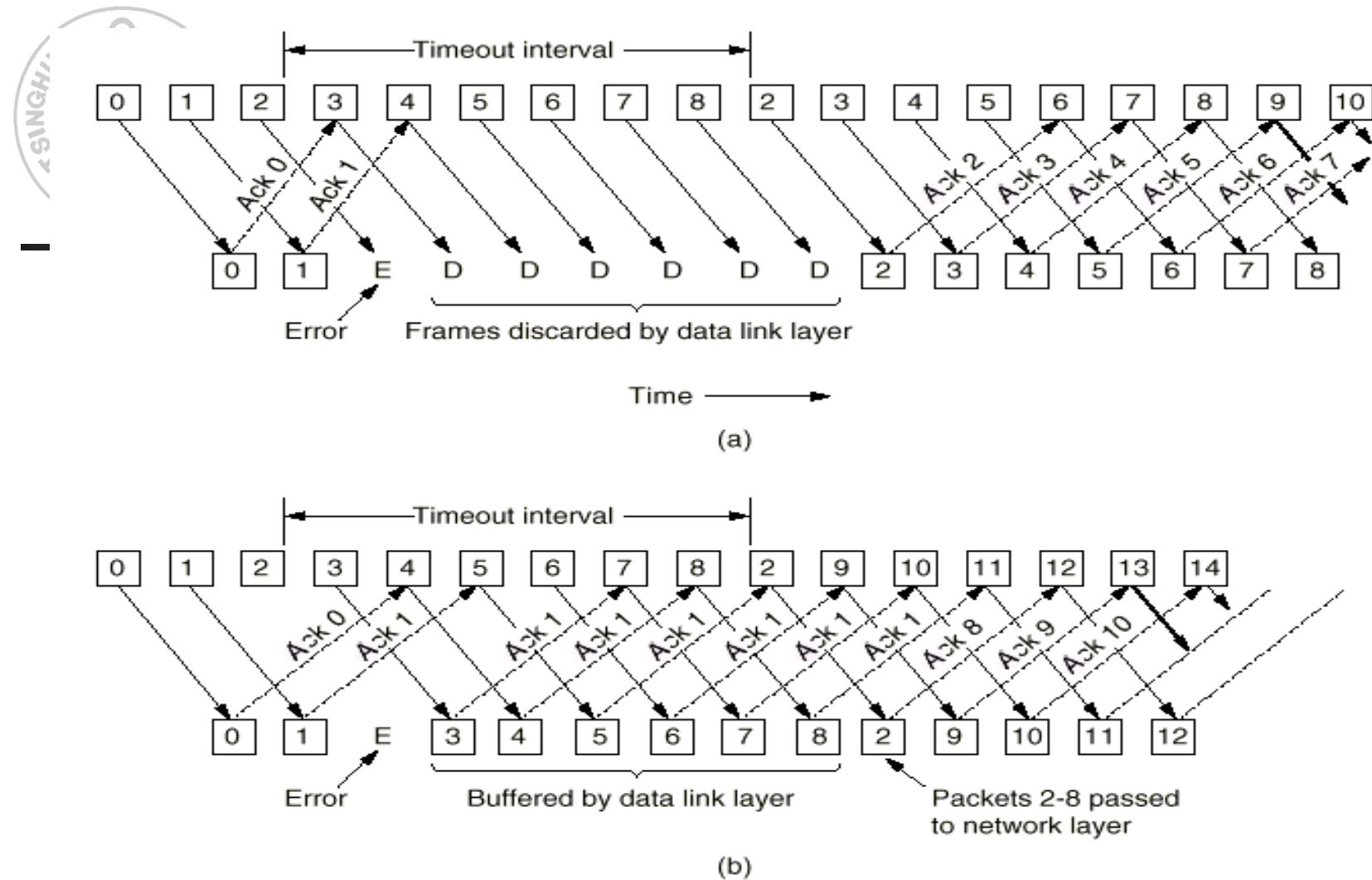


Fig. 3-15. (a) Effect of an error when the receiver window size is 1. (b) Effect of an error when the receiver window size is large.



退后n帧重传协议

- 发送方有流量控制，为重传设缓冲
 - 发送窗口未满，**EnableNetworkLayer**
 - 发送窗口满，**DisableNetworkLayer**
- 发送窗口尺寸 $< \text{序号个数} (\text{MaxSeq} + 1)$
 - 考虑**MaxSeq = 7**的情况
 - 发送方发送帧 0 ~ 7
 - 序号为 7 的帧的确认被捎带回发送方
 - 发送方发送另外 8 个帧，序号为 0 ~ 7
 - 另一个对帧 7 的捎带确认返回
 - 问题：第二次发送的 8 个帧成功了还是丢失了？（累计确认）

/* Protocol 5 (pipelining) allows multiple outstanding frames. The sender may transmit up to MAX_SEQ frames without waiting for an ack. In addition, unlike the previous protocols, the network layer is not assumed to have a new packet all the time. Instead, the network layer causes a network_layer_ready event when there is a packet to send. */

```
#define MAX_SEQ 7          /* should be  $2^n - 1$  */
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready} event_type;
#include "protocol.h"
```

```
static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
    /* Return true if (a <= b < c circularly; false otherwise. */
    if (((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a)))
        return(true);
    else
        return(false);
}
```

```
static void send_data(seq_nr frame_nr, seq_nr frame_expected, packet buffer[])
{
    /* Construct and send a data frame. */
    frame s;          /* scratch variable */

    s.info = buffer[frame_nr];          /* insert packet into frame */
    s.seq = frame_nr;          /* insert sequence number into frame */
    s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1); /* piggyback ack */
    to_physical_layer(&s);          /* transmit the frame */
    start_timer(frame_nr);          /* start the timer running */
}
```



```
void protocol5(void)
{
    seq_nr next_frame_to_send;    /* MAX_SEQ > 1; used for outbound stream */
    seq_nr ack_expected;          /* oldest frame as yet unacknowledged */
    seq_nr frame_expected;        /* next frame expected on inbound stream */
    frame r;                      /* scratch variable */
    packet buffer[MAX_SEQ + 1];   /* buffers for the outbound stream */
    seq_nr nbuffered;             /* # output buffers currently in use */
    seq_nr i;                     /* used to index into the buffer array */
    event_type event;

    enable_network_layer();        /* allow network_layer_ready events */
    ack_expected = 0;             /* next ack expected inbound */
    next_frame_to_send = 0;       /* next frame going out */
    frame_expected = 0;           /* number of frame expected inbound */
    nbuffered = 0;                /* initially no packets are buffered */
}
```

```

while (true) {
    wait_for_event(&event);          /* four possibilities: see event_type above */

    switch(event) {
        case network_layer_ready:    /* the network layer has a packet to send */
            /* Accept, save, and transmit a new frame. */
            from_network_layer(&buffer[next_frame_to_send]); /* fetch new packet */
            nbuffered = nbuffered + 1; /* expand the sender's window */
            send_data(next_frame_to_send, frame_expected, buffer); /* transmit the frame */
            inc(next_frame_to_send); /* advance sender's upper window edge */
            break;

        case frame_arrival:          /* a data or control frame has arrived */
            from_physical_layer(&r); /* get incoming frame from physical layer */

            if (r.seq == frame_expected) {
                /* Frames are accepted only in order. */
                to_network_layer(&r.info); /* pass packet to network layer */
                inc(frame_expected); /* advance lower edge of receiver's window */
            }

            /* Ack n implies n - 1, n - 2, etc. Check for this. */
            while (between(ack_expected, r.ack, next_frame_to_send)) {
                /* Handle piggybacked ack. */
                nbuffered = nbuffered - 1; /* one frame fewer buffered */
                stop_timer(ack_expected); /* frame arrived intact; stop timer */
                inc(ack_expected); /* contract sender's window */
            }
            break;
    }
}

```




```
case cksum_err: break;          /* just ignore bad frames */

case timeout:                   /* trouble; retransmit all outstanding frames */
    next_frame_to_send = ack_expected; /* start retransmitting here */
    for (i = 1; i <= nbuffered; i++) {
        send_data(next_frame_to_send, frame_expected, buffer); /* resend 1 frame */
        inc(next_frame_to_send); /* prepare to send the next one */
    }

}

if (nbuffered < MAX_SEQ)
    enable_network_layer();
else
    disable_network_layer();
}
```

From: *Computer Networks*, 3rd ed. by Andrew S. Tanenbaum, © 1996 Prentice Hall

Fig. 3-16. A sliding window protocol using go back n.



计时器

- 由于有多个未确认帧，需要设多个计时器

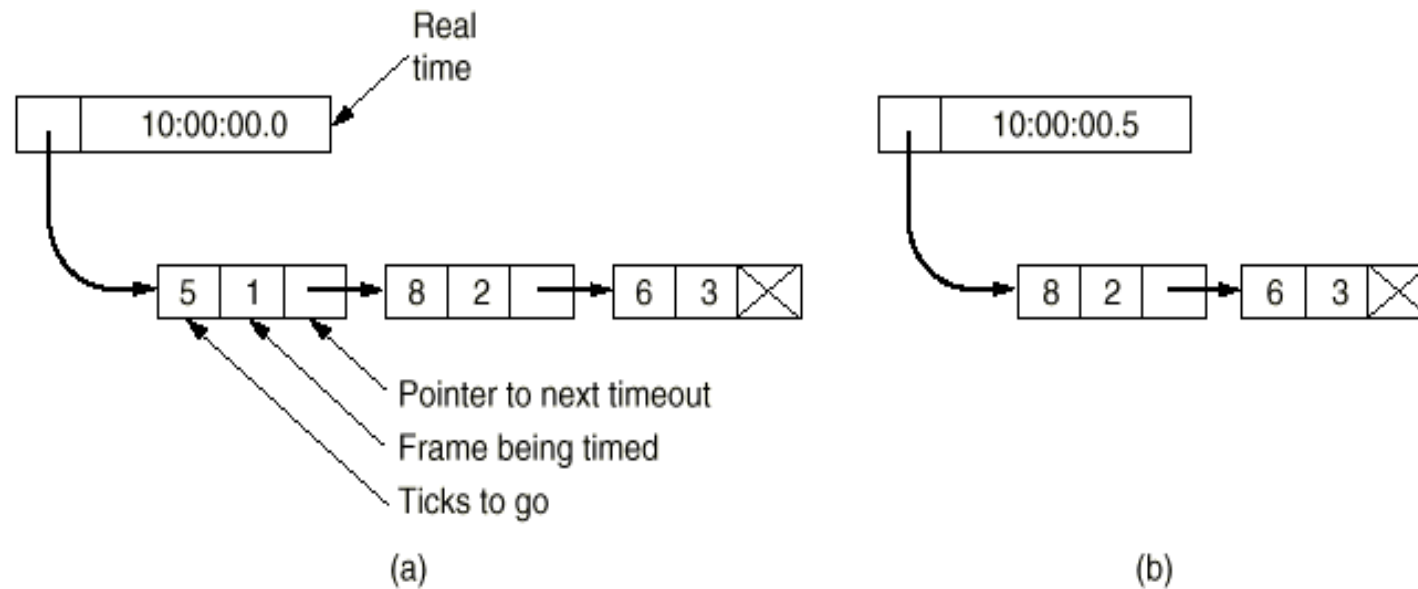


Fig. 3-17. Simulation of multiple timers in software.



退后n帧重传协议（续）

■ 协议实现分析

■ 事件驱动

■ Network_layer_ready（内部事件）

- 发送帧（帧序号，确认序号，数据）

■ Frame_arrival（外部事件）

- 检查帧序号，落在接收窗口内则接收，否则丢弃
- 检查确认序号，落在发送窗口内则移动发送窗口，否则不做处理

■ Cksum_err（外部事件）

- 丢弃

■ timeout（内部事件）

- 退后n帧重传



退后n帧重传协议（续）

■ 计时器处理

- 启动，发送帧时启动
- 停止，收到正确确认时停止
- 超时则产生**timeout**事件



选择重传协议

■ 目的

- 采用选择重传技术在不可靠信道上传输时，不会因不必要的重传而浪费信道资源

■ 窗口设置

- 保证接收窗口移动前后，接收窗口内的帧没有重叠
- 设 **MaxSeq = 7**, 若接收窗口尺寸 = **7**, 发方发帧 **0 ~ 6**, 收方全部收到, 接收窗口前移 (**7 ~ 5**), 确认帧丢失, 发方重传帧**0**, 收方作为新帧接收, 并对帧**6**确认, 发方发新帧 **7 ~ 5**, 收方已收过帧 **0**, 丢弃新帧 **0**, 协议出错
- 当发送窗口尺寸 $\geq$ 接收窗口尺寸时, 接收窗口尺寸最大值为 **$(\text{MaxSeq} + 1) / 2$**
 - 例如: 发送窗口尺寸: **MaxSeq**, 接收窗口尺寸: **$(\text{MaxSeq} + 1) / 2$**
- 当发送窗口尺寸 $<$ 接收窗口尺寸时, 要求发送窗口尺寸 + 接收窗口尺寸 $\leq$ 序号个数

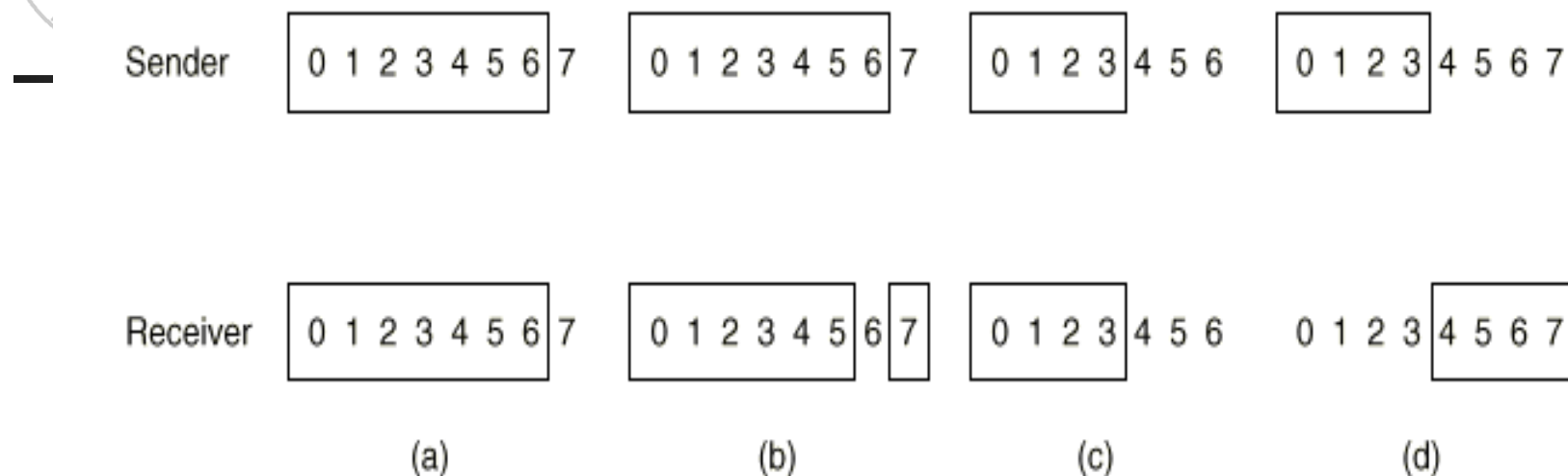


Fig. 3-19. (a) Initial situation with a window of size seven. (b) After seven frames have been sent and received but not acknowledged. (c) Initial situation with a window size of four. (d) After four frames have been sent and received but not acknowledged.



选择重传协议（续）

- 窗口参数
 - 发送窗口下界: **AckExpected**
 - 发送窗口上界: **NextFrameToSend**
 - 接收窗口下界: **FrameExpected**
 - 接收窗口上界: **TooFar**
- 缓冲区设置
 - 发送方和接收方的缓冲区大小应等于各自窗口大小
- 增加确认计时器，解决两个方向负载不平衡带来的阻塞问题
- 可随时发送否定性确认帧**NAK**

/* Protocol 6 (nonsequential receive) accepts frames out of order, but passes packets to the network layer in order. Associated with each outstanding frame is a timer. When the timer goes off, only that frame is retransmitted, not all the outstanding frames, as in protocol 5. */

```
#define MAX_SEQ 7 /* should be  $2^n - 1$  */
```

```
#define NR_BUFS ((MAX_SEQ + 1)/2)
```

— typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready, ack_timeout} event_type;

```
#include "protocol.h"
```

```
boolean no_nak = true; /* no nak has been sent yet */
```

```
seq_nr oldest_frame = MAX_SEQ + 1; /* initial value is only for the simulator */
```

```
static boolean between(seq_nr a, seq_nr b, seq_nr c)
```

```
{
```

```
/* Same as between in protocol5, but shorter and more obscure. */
```

```
return ((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a));
```

```
}
```

```
static void send_frame(frame_kind fk, seq_nr frame_nr, seq_nr frame_expected, packet buffer[])
```

```
{
```

```
/* Construct and send a data, ack, or nak frame. */
```

```
frame s; /* scratch variable */
```

```
s.kind = fk; /* kind == data, ack, or nak */
```

```
if (fk == data) s.info = buffer[frame_nr % NR_BUFS];
```

```
s.seq = frame_nr; /* only meaningful for data frames */
```

```
s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);
```

```
if (fk == nak) no_nak = false; /* one nak per frame, please */
```

```
to_physical_layer(&s); /* transmit the frame */
```

```
if (fk == data) start_timer(frame_nr % NR_BUFS);
```

```
stop_ack_timer(); /* no need for separate ack frame */
```

```
}
```



```

void protocol6(void)
{
    seq_nr ack_expected;           /* lower edge of sender's window */
    seq_nr next_frame_to_send;     /* upper edge of sender's window + 1 */
    seq_nr frame_expected;         /* lower edge of receiver's window */
    seq_nr too_far;                /* upper edge of receiver's window + 1 */
    int i;                         /* index into buffer pool */
    frame r;                       /* scratch variable */
    packet out_buf[NR_BUFS];       /* buffers for the outbound stream */
    packet in_buf[NR_BUFS];        /* buffers for the inbound stream */
    boolean arrived[NR_BUFS];      /* inbound bit map */
    seq_nr nbuffered;              /* how many output buffers currently used */
    event_type event;

    enable_network_layer();         /* initialize */
    ack_expected = 0;               /* next ack expected on the inbound stream */
    next_frame_to_send = 0;         /* number of next outgoing frame */
    frame_expected = 0;
    too_far = NR_BUFS;
    nbuffered = 0;                  /* initially no packets are buffered */
    for (i = 0; i < NR_BUFS; i++) arrived[i] = false;
}

```

```

while (true) {
    wait_for_event(&event);          /* five possibilities: see event_type above */
    switch(event) {
        case network_layer_ready:    /* accept, save, and transmit a new frame */
            nbuffered = nbuffered + 1; /* expand the window */
            from_network_layer(&out_buf[next_frame_to_send % NR_BUFS]); /* fetch new packet */
            send_frame(data, next_frame_to_send, frame_expected, out_buf); /* transmit the frame */
            inc(next_frame_to_send); /* advance upper window edge */
            break;

        case frame_arrival:          /* a data or control frame has arrived */
            from_physical_layer(&r); /* fetch incoming frame from physical layer */
            if (r.kind == data) {
                /* An undamaged frame has arrived. */
                if ((r.seq != frame_expected) && no_nak)
                    send_frame(nak, 0, frame_expected, out_buf); else start_ack_timer();
                if (between(frame_expected, r.seq, too_far) && (arrived[r.seq%NR_BUFS] == false)) {
                    /* Frames may be accepted in any order. */
                    arrived[r.seq % NR_BUFS] = true; /* mark buffer as full */
                    in_buf[r.seq % NR_BUFS] = r.info; /* insert data into buffer */
                    while (arrived[frame_expected % NR_BUFS]) {
                        /* Pass frames and advance window. */
                        to_network_layer(&in_buf[frame_expected % NR_BUFS]);
                        no_nak = true;
                        arrived[frame_expected % NR_BUFS] = false;
                        inc(frame_expected); /* advance lower edge of receiver's window */
                        inc(too_far); /* advance upper edge of receiver's window */
                        start_ack_timer(); /* to see if a separate ack is needed */
                    }
                }
            }
    }
}

```



—

```
    if((r.kind==nak) && between(ack_expected,(r.ack+1)%(MAX_SEQ+1),next_frame_to_send))
        send_frame(data, (r.ack+1) % (MAX_SEQ + 1), frame_expected, out_buf);

    while (between(ack_expected, r.ack, next_frame_to_send)) {
        nbuffered = nbuffered - 1; /* handle piggybacked ack */
        stop_timer(ack_expected % NR_BUFS);/* frame arrived intact */
        inc(ack_expected); /* advance lower edge of sender's window */
    }
    break;

case cksum_err:
    if (no_nak) send_frame(nak, 0, frame_expected, out_buf);/* damaged frame */
    break;

case timeout:
    send_frame(data, oldest_frame, frame_expected, out_buf);/* we timed out */
    break;

case ack_timeout:
    send_frame(ack,0,frame_expected, out_buf);/* ack timer expired; send ack */
}

if (nbuffered < NR_BUFS) enable_network_layer(); else disable_network_layer();
}
}
```



选择重传协议（续）

■ 协议实现分析

■ 事件驱动

■ Network_layer_ready（内部事件）

- 发送帧（帧类型，帧序号，确认序号，数据）

■ Frame_arrival（外部事件）

- 若是数据帧，则检查帧序号，落在接收窗口内则接收，否则丢弃；不等于接收窗口下界还要发NAK
- 若是NAK，则选择重传
- 检查确认序号，落在发送窗口内则移动发送窗口，否则不做处理

■ Cksum_err（外部事件）：发送NAK

■ timeout（内部事件）：选择重传

■ Ack_timeout（内部事件）：发送确认帧ACK



选择重传协议（续）

■ 计时器处理

- 启动，发送数据帧时启动
- 停止，收到正确确认时停止
- 超时则产生**timeout**事件

■ Ack计时器处理

- 启动，收到帧的序号等于接收窗口下界或已经发过**NAK**时启动
- 停止，发送帧时停止
- 超时则产生**ack_timeout**事件



小结

- 可靠传输
 - 纠错编码
 - 检错码、确认和重传机制
- 传输层协议**TCP**也提供可靠传输服务
- 链路层的可靠传输服务通常用于高误码率的连路上，如无线链路
- 对于误码率低的链路，链路层协议可以不实现可靠传输功能



主要内容

- 数据链路层概述
 - 定义和功能
 - 为网络层提供的服务
- 成帧
- 错误检测和纠正
 - 纠错码
 - 检错码
- 基本的数据链路层协议
 - 无约束单工协议
 - 单工停等协议
 - 有噪声信道的单工协议
- 滑动窗口协议
 - 一比特滑动窗口协议
 - 退后n帧重传协议
 - 选择重传协议
- 协议说明与验证
 - 协议形式化描述技术
 - 有限状态机模型
 - Petri网模型
- 常用的数据链路层协议
 - HDLC
 - X.25的链路层协议LAPB
 - PPP协议



协议说明与验证

- 协议工程与协议的形式化描述技术
- 协议工程
 - 协议说明 (**Protocol Specification**)
 - 协议验证 (**Protocol Verification**)
 - 协议实现 (**Protocol Implementation**)
 - 协议测试 (**Protocol Testing**)
 - 一致性测试 (**Conformance Testing**)
 - 互操作性测试 (**Interoperability Testing**)
 - 性能测试 (**Performance Testing**)



协议工程

■ 协议说明

- 既定义一个协议实体提供给它的用户的服务，又定义该协议实体的内部操作

■ 协议验证

- 验证协议说明是否完整正确
- 用于系统实现前的设计阶段，避免可能出现的设计错误
- 以协议说明为基础，涉及逻辑证明，原则上验证协议所有可能的状态
- 可达性分析是一种常用的验证方法
 - 利用图论知识可以解决状态的可达性问题
 - 可达性分析能够用来解决协议的不完整性、死锁和无关变迁等问题



协议工程（续）

- 协议实现
 - 用硬件和/或软件实现协议说明中规定的功能
- 协议测试
 - 用测试的方法来检查协议实现是否满足要求，包括
 - 协议实现是否与协议说明一致（一致性测试）
 - 协议实现之间的互操作能力（互操作性测试）
 - 协议实现的性能（性能测试）等



形式化描述技术

- 在协议的说明、验证、实现和测试过程中使用形式化描述技术，不仅可以比较容易地理解协议，而且可以使协议描述更加精确，大大简化了协议的研究工作
- 形式化描述的意义
 - 实际使用的协议非常复杂，给协议的理解、验证、实现和测试等工作带来困难，需要采用形式化的、数学的描述方法来描述协议
 - 但是目前大多数协议还是采用自然语言描述
- 自然语言描述协议的缺点
 - 冗余
 - 多义性
 - 结构性不好
 - 不便于自动验证、测试、实现



形式化描述技术（续）

- 形式化描述技术**FDT**（**Formal Description Technique**）/形式化方法**FM**（**Formal Method**）广泛应用于协议工程研究中
- 一种形式化方法总是以一种形式体系为基础，只是在具体应用时，大都做了便于描述的改进和扩充
- 常用的形式化方法
 - 有限状态机**FSM**（**Finite State Machine**）
 - 扩展：**EFSM**
 - 形式化语言模型
 - **LOTOS, Estelle, SDL**
 - 都有相应扩展
 - **Petri网**
 - 扩展：时间**Petri网**，随机**Petri网**，高级**Petri网**
 - 进程代数（**Process Algebra**）
 - 扩展：随机进程代数



有限状态机模型

■ 定义

- 一个有限状态机是一个四元组(**S, M, I, T**)，其中**S**是状态的集合，**M**是标号的集合，**I**是初始状态的集合，**T**是变迁的集合

■ 网络协议建模

- 基本出发点：认为通信协议主要是由响应多个“事件”的相对简单的处理过程组成

■ 事件

- 命令（来自用户）
- 信息到达（来自底层）
- 内部超时

■ 优点：简单明了，比较精确

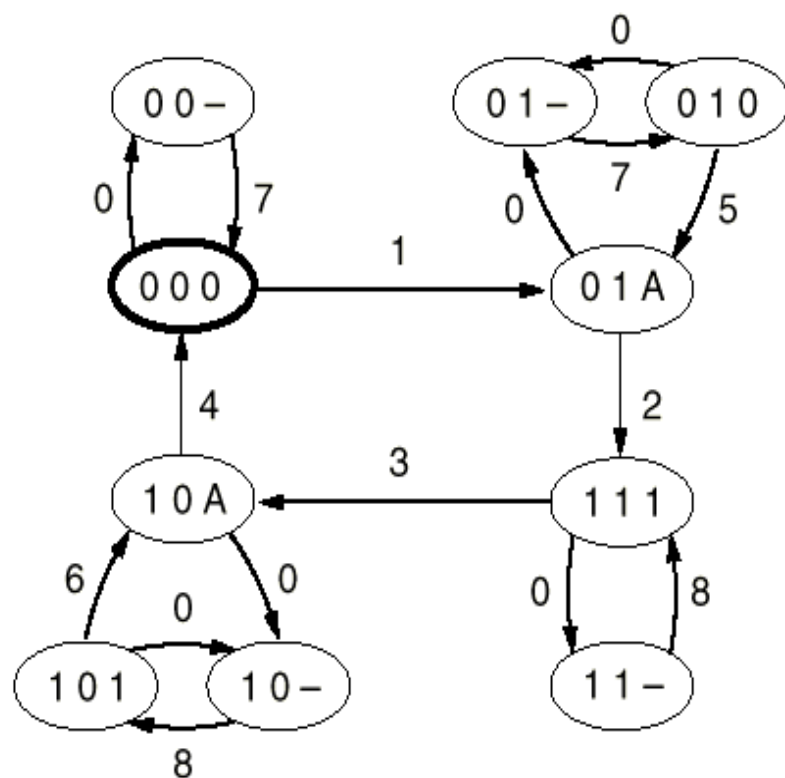
■ 缺点：复杂协议的事件数和状态数会剧增，状态爆炸



有限状态机模型（续）

■ 例，协议3

- 每个状态用三个字母表示：**XYZ**
 - **X**: 发送方发送的帧序号，为**0**或**1**
 - **Y**: 接收方准备接收的帧序号，为**0**或**1**
 - **Z**: 信道状态，为**0**，**1**，**A**或**-**（空）
- 初始状态为（**000**）
- 半双工信道
- 全双工信道



(a)

Transition	Who runs?	Frame accepted	Frame emitted	To network layer
0	—	(frame lost)		—
1	R	0	A	Yes
2	S	A	1	—
3	R	1	A	Yes
4	S	A	0	—
5	R	0	A	No
6	R	1	A	No
7	S	(timeout)	0	—
8	S	(timeout)	1	—

(b)

Fig. 3-20. (a) State diagram for protocol 3. (b) Transitions.



Petri网模型

- Petri网模型最早在**1962年 Carl Adam Petri**的博士论文中提出来，主要特性是具有较强的对并行、不确定性、异步和分布的描述能力和分析能力
- Petri网研究的系统模型行为特性包括
 - 状态的可达 (**reachability**)
 - 位置的限界 (**boundedness**)
 - 变迁的活性 (**liveness**)
 - 初始状态的可逆达 (**reversibility**)
 - 标识 (**marking**) 之间的可达 (**reachability**)
 - 事件之间的同步距离 (**synchronic distance**)
 - 公平性 (**fairness**)



Petri网模型（续）

■ Petri网定义

■ 结构元素

- 位置（**place**）：描述系统状态，用一个圆圈表示
- 变迁（**transition**）：描述修改系统状态的事件，用一个长方形或线段表示
- 弧（**arc**）：描述状态与事件之间的关系，包括输入弧和输出弧，用有向弧表示

■ 活动元素 —— 标记（**token**）

- 包含在位置中，用点表示
- 用来表示处理的信息单元、资源单元和顾客、用户等对象
- 如果位置用来描述条件，它可以包含一个标记或不包含标记，当包含标记时，条件为真，否则，为假
- 如果位置用来定义状态，位置中的标记个数用于规定这个状态



Petri网模型（续）

■ 变迁实施规则（**firing rule**）

- 如果一个变迁的所有输入位置至少包含一个标记，则这个变迁可能实施
- 一个可实施变迁的实施导致从它所有输入位置中都清除一个标记，在它所有输出位置中产生一个标记
- 当使用大于**1**的弧权（**weight**）时，在变迁的所有输入位置中都要包含至少等于连接弧权的标记个数它才可实施，并根据弧权，在每个输出位置中产生相应标记个数
- 变迁的实施是一个原子操作，输入位置清除标记和输出位置产生标记是一个不可分割的完整操作

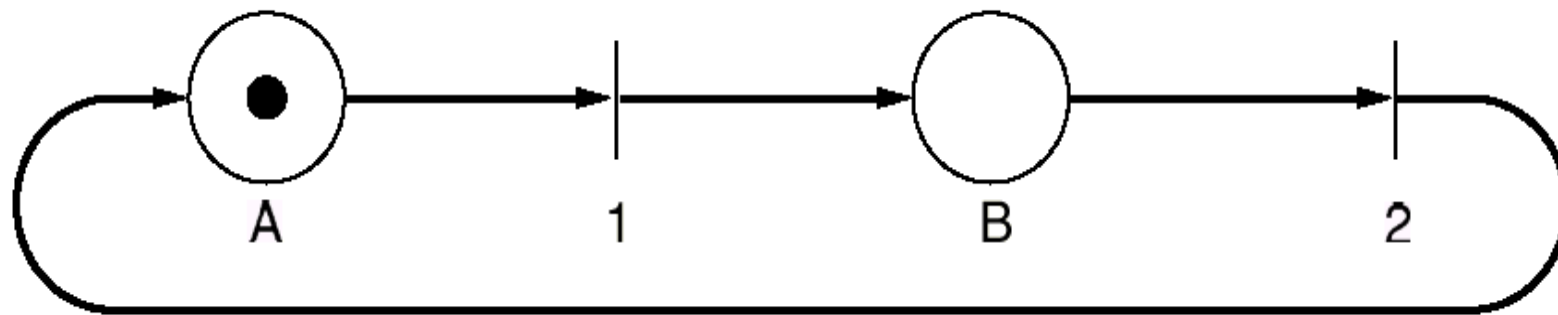


Fig. 3-22. A Petri net with two places and two transitions.



Petri网模型（续）

■ Petri网的组成

- 条件/事件（C/E）网
 - 每个位置最多一个标记，表示条件
- 位置/变迁（P/T）网
 - 每个位置中的标记可以有多个

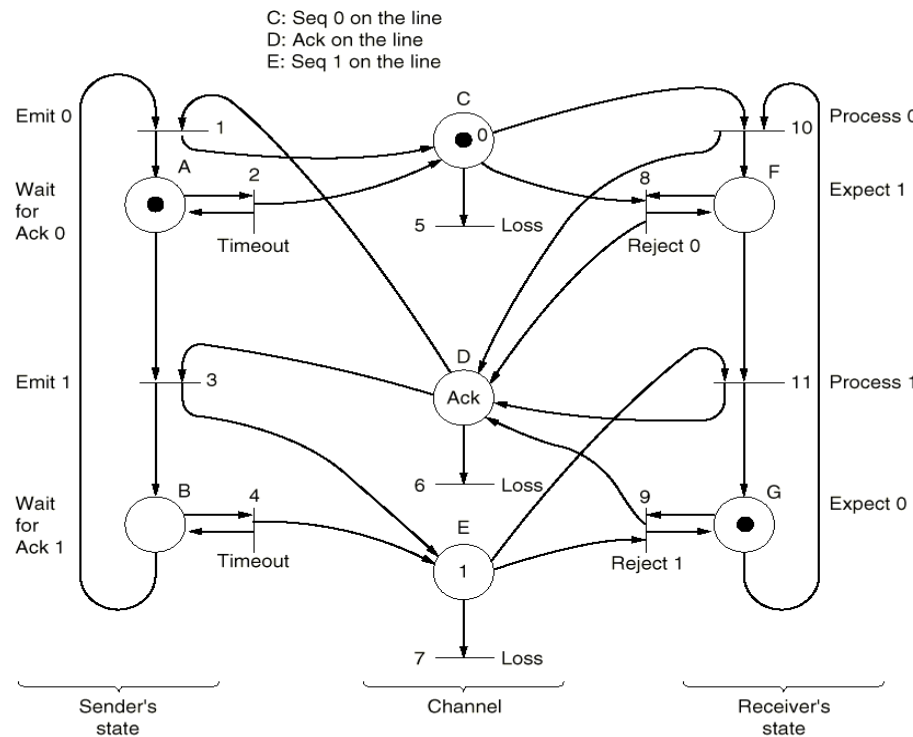
■ 高级Petri网

- 包括谓词/变迁网、着色网等
- 给标记以属性，即标记有区别
- 从没有参数的网，发展到时间Petri网和随机Petri网
- 从一般有向弧发展到可变弧
- 从自然数标记个数发展到概率标记个数
- 从原子变迁发展到谓词变迁和子网变迁



Petri网模型 (续)

■ 例：协议3（半双工信道）



$M_0 = ACG \ (000)$

ADF (0A1)

BEF (111)

BDG (1A0)

1

Fig. 3-23. A Petri net model for protocol 3.



形式化方法的补充说明

- 模型描述能力的增强会在某种程度上增加模型分析的难度
- 模型仅仅是手段，不是目的
 - **Modeling is a bridge**



主要内容

- 数据链路层概述
 - 定义和功能
 - 为网络层提供的服务
- 成帧
- 错误检测和纠正
 - 纠错码
 - 检错码
- 基本的数据链路层协议
 - 无约束单工协议
 - 单工停等协议
 - 有噪声信道的单工协议
- 滑动窗口协议
 - 一比特滑动窗口协议
 - 退后n帧重传协议
 - 选择重传协议
- 协议说明与验证
 - 协议形式化描述技术
 - 有限状态机模型
 - Petri网模型
- 常用的数据链路层协议
 - HDLC
 - X.25的链路层协议LAPB
 - PPP协议



常用的数据链路层协议

- **ISO和CCITT**在数据链路层协议的标准制定方面做了大量工作，各大公司也形成了自己的标准
- 数据链路层协议分类
 - 面向字符的链路层协议
 - **ISO的IS1745**，基本型传输控制规程及其扩充部分（**BM**和**XBM**）
 - **IBM**的二进制同步通信规程（**BSC**）
 - **DEC**的数字数据通信报文协议（**DDCMP**）
 - **PPP**
 - 面向比特的链路层协议
 - **IBM**的**SNA**使用的数据链路协议**SDLC**
 - **ANSI**修改**SDLC**，提出**ADCCP**
 - **ISO**修改**SDLC**，提出**HDLC**
 - **CCITT**修改**HDLC**，提出**LAP**和**LAPB**



高级数据链路控制规程HDLC

- **1976年，ISO提出HDLC（High-level Data Link Control）**
- **HDLC的组成**
 - 帧结构 }
 - 规程元素 } 语法
 - 规程类型 语义
 - 使用**HDLC**的语法可以定义多种具有不同操作特点的链路层协议
- **HDLC的适用范围**
 - 计算机与计算机通信
 - 计算机与终端通信
 - 终端与终端通信



HDLC (续)

- 数据站（简称站 **station**），由计算机和终端组成，负责发送和接收帧。**HDLC**涉及三种类型的站
 - 主站（**primary station**）
 - 主要功能是发送命令（包括数据），接收响应，负责整个链路的控制（如系统的初始、流控、差错恢复等）
 - 次站（**secondary station**）
 - 主要功能是接收命令，发送响应，配合主站完成链路的控制
 - 组合站（**combined station**）
 - 同时具有主、次站功能，既发送又接收命令和响应，并负责整个链路的控制



HDLC (续)

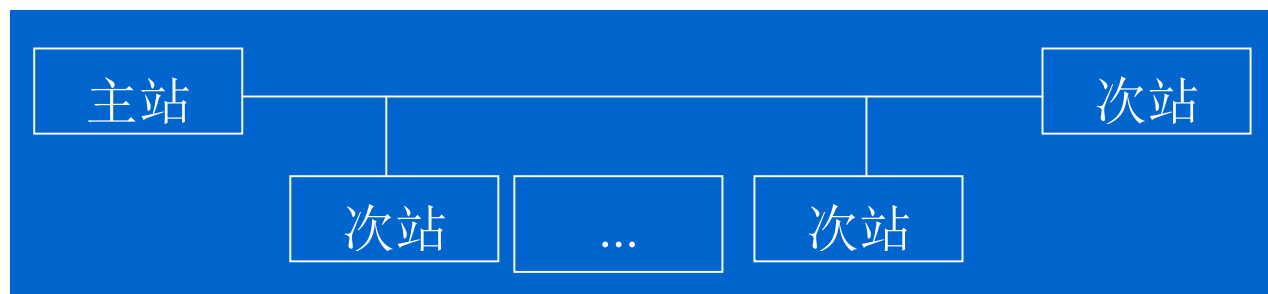
■ HDLC适用的链路构型

■ 非平衡型

■ 点到点式



■ 点到多点式



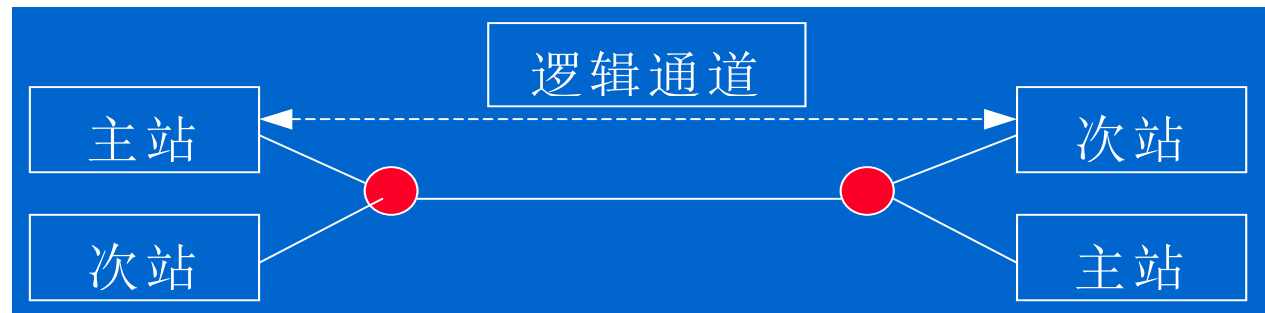
■ 适合把智能和半智能的终端连接到计算机



HDLC (续)

- 平衡型

- 主站 — 次站式



- 组合式



- 适合于计算机和计算机之间的连接



HDLC的基本操作模式

■ 正规响应模式 **NRM**

- 适用于点到点式和多点式两种非平衡构型。只有当主站向次站发出探测后，次站才能获得传输帧的许可

■ 异步响应模式 **ARM**

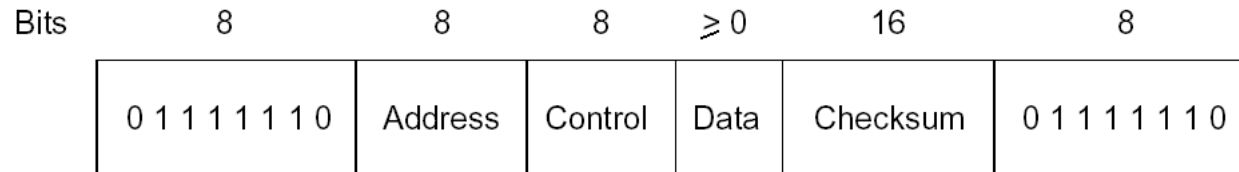
- 适用于点到点式非平衡构型和主站 — 次站式平衡构型。次站可以随时传输帧，不必等待主站的探测

■ 异步平衡模式 **ABM**

- 适用于通信双方都是组合站的平衡构型，也采用异步响应，双方具有同等能力



HDLC 帧 结构



■ 定界符

- **01111110**

- 空闲的点到点链路上连续传定界符

■ 地址域 (**Address**)

- 多终端线路，用来区分终端

- 点到点线路，有时用来区分命令和响应

- 若帧中的地址是接收该帧的站的地址，则该帧是命令帧

- 若帧中的地址是发送该帧的站的地址，则该帧是响应帧



HDLC帧结构（续）

- 控制域（**Control**）
 - 序号：使用滑动窗口技术
 - 确认
 - 其它
- 数据域（**Data**）
 - 任意信息，任意长度（上层协议**SDU**有上限）
- 校验和（**Checksum**）
 - **CRC**校验
 - 生成多项式：**CRC-CCITT**



HDLC 帧结构（续）

■ 帧类型

- 信息帧（**Information**）
- 监控帧（**Supervisory**）
- 无序号帧（**Unnumbered**）
 - 可以用来传控制信息，也可在无连接不可靠服务中传数据

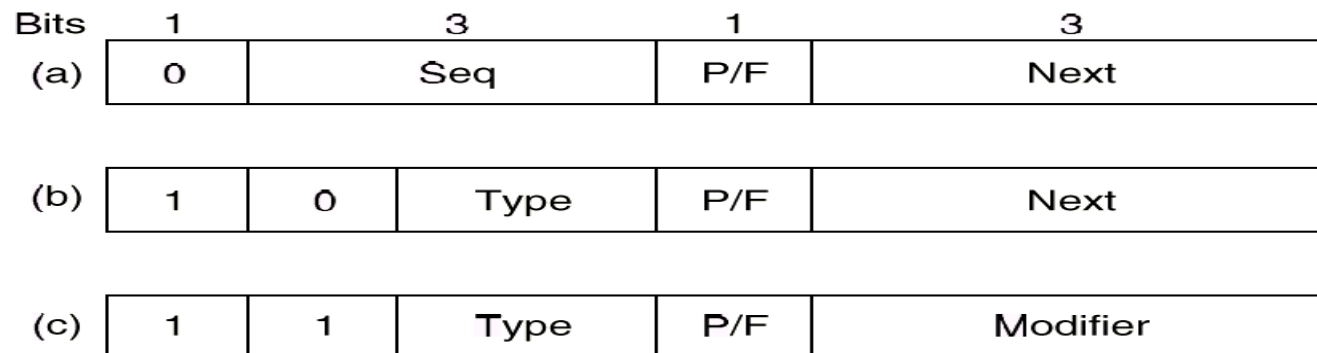


Fig. 3-25. Control field of (a) an information frame, (b) a supervisory frame, (c) an unnumbered frame.



HDLC帧结构 — 控制域

- 序号 (**Seq**)：使用滑动窗口技术，**3**位序号，发送窗口大小为**7**
- 确认 (**Next**)：捎带第一个未收到的帧序号
- 探测/结束 **P/F**位 (**Poll/Final**)
 - 命令帧置 “**P**”，响应帧置 “**F**”
 - 有些协议，**P/F**位用来强迫对方机器立刻发控制帧
 - 多终端系统中，计算机置 “**P**”，允许终端发送数据；终端发向计算机的帧中，最后一个帧置为 “**F**”，其它置为 “**P**”
- 类型 (**Type**)
 - “**0**”表示确认帧 **RR (RECEIVE READY)**
 - “**1**”表示否定性确认帧 **REJ (REJECT)**
 - “**2**”表示接收未准备好 **RNR (RECEIVE NOT READY)**
 - “**3**”表示选择拒绝 **SREJ (SELECTIVE REJECT)**
 - **HDLC**和**ADCCP**允许选择拒绝，**SDLC**和**LAPB**不允许



HDLC命令

- **DISC (DISConnect)**
- **SNRM (Set Normal Response Mode)**
- **SARM (Set Asynchronous Response Mode)**
- **SABM (Set Asynchronous Balanced Mode)**
 - HDLC和LAPB使用
- **SABME SABM的扩展**
- **SNRME SNRM的扩展**
- **FRMR (FRaMe Reject)**
 - 校验和正确，语义错误
- **无序号确认UA (Unnumbered Acknowledgement)**
 - 对控制帧进行确认，用于确认模式建立和接受拆除命令
- **UI (Unnumbered Information)**



HDLC的功能组合

- 三种站，两种构型，三种操作模式，以及规程元素中定义的各种帧的各种组合产生多种链路层协议
- 构造链路层协议的过程
 - 选择站构型 ——> 基本操作模式 ——> 基本帧种类 ——> **12种**任选功能 ——> 得到协议



X.25的链路层协议LAPB

- **X.25协议**
 - 分组级: **PLP**
 - 帧级: **LAP (Link Access Procedure)**, **LAPB (Balanced)**
 - 物理级: **X.21**
- **X.25协议规程使用HDLC规程的原理和术语**
- **X.25 LAP: HDLC非平衡规程帧的基本清单 + 任选功能2、8、12, 也可组成主站 — 次站式平衡规程**
- **X.25 LAPB: HDLC组合站平衡规程帧的基本清单 + 任选功能2、8、11、12**
- **因此, X.25 LAP、LAPB是HDLC的子集**



LAPB的命令和响应

■ LAPB的帧格式与HDLC完全相同

格式	命令	响应	控制域编码			
信息帧	I(信息)		0	N(S)	P	N(R)
监控帧	RR(接收准备好)	RR(接收准备好)	1	000	P/F	N(R)
	RNR(接收未准备好)	RNR(接收未准备好)	1	010	P/F	N(R)
	REJ(拒绝)	REJ(拒绝)	1	011	P/F	N(R)
无序号帧	SARM(置异步响应模式)	DM(拆除模式)	1	111	P/F	000
	SABM(置异步平衡模式)		1	111	P	100
	DISC(拆除)		1	100	P	010
		UA(无序号确认)	1	100	F	110
		CMDR(命令拒绝) FRMR(帧拒绝)	1	110	F	001



LAPB的检错和纠错方法

- 帧格式
 - 采用**CRC**校验，只检错，不纠错，丢弃出错帧
- 超时机制
 - 计时器超时重传，重传**N**次，则向上层协议报告
 - 超时机制用来检错，重传用来纠错
- 帧序号
 - 若接收方发现帧序号错，就发拒绝帧给发送方，发送方重传，既检错也纠错
- 采用**P/F**位来进行校验指示
 - 发送置为 **P** 的命令帧，等待置为 **F** 的响应帧，能及时发现远程数据站是否收到命令帧
- 规程规定：第一项必须使用；第二、三、四项组合使用



Internet数据链路层协议

- 点到点通信的两种场景
 - 路由器到路由器
 - 通过**modem**拨号上网，连到路由器或接入服务器

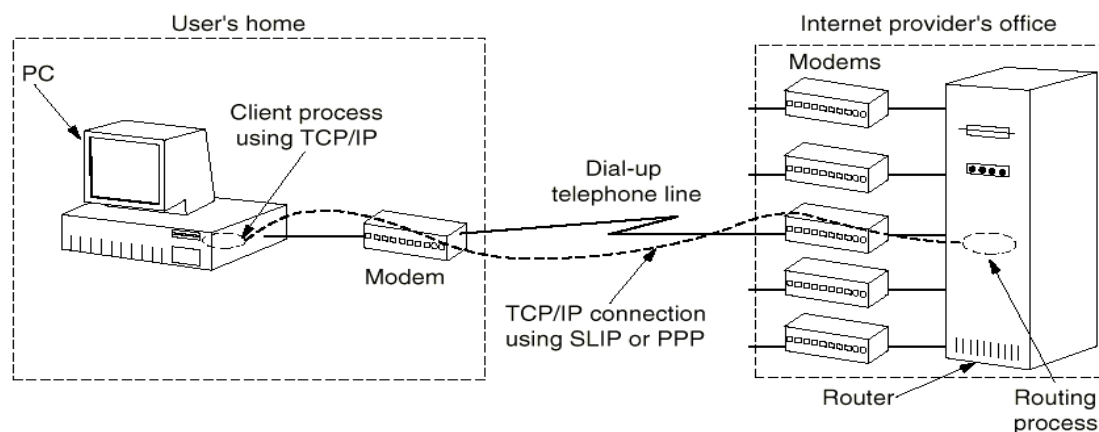


Fig. 3-26. A home personal computer acting as an Internet host.



SLIP: Serial Line IP

- **1984年，Rick Adams提出SLIP，RFC1055**
- **基本思想：发送原始IP包，用一个标记字节来定界，采用字符填充技术**
- **新版本提供TCP和IP头压缩技术，RFC 1144**
- **存在的问题**
 - 不提供差错校验
 - 只支持IP
 - IP地址不能动态分配
 - 不提供认证
 - 多种版本并存，互连困难



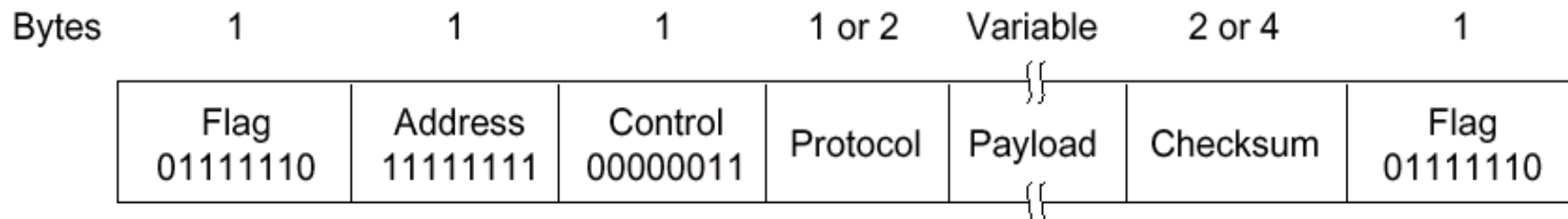
PPP: Point-to-Point Protocol

- **RFC 1661, RFC 1662, RFC 1663**
- **PPP**改进了**SLIP**，提供差错校验、支持多种协议、允许动态分配**IP**地址、支持认证等
- 以帧为单位发送，而不是原始**IP**包
- 协议包括两部分
 - 链路控制协议**LCP**（**Link Control Protocol**）
 - 可使用多种物理层服务：modem，HDLC串线，SDH/SONET等
 - 网络控制协议**NCP**（**Network Control Protocol**）
 - 可支持多种网络层协议



PPP 帧格式

- 帧格式与**HDLC**相似，区别在于**PPP**是面向字符的，采用字符填充技术



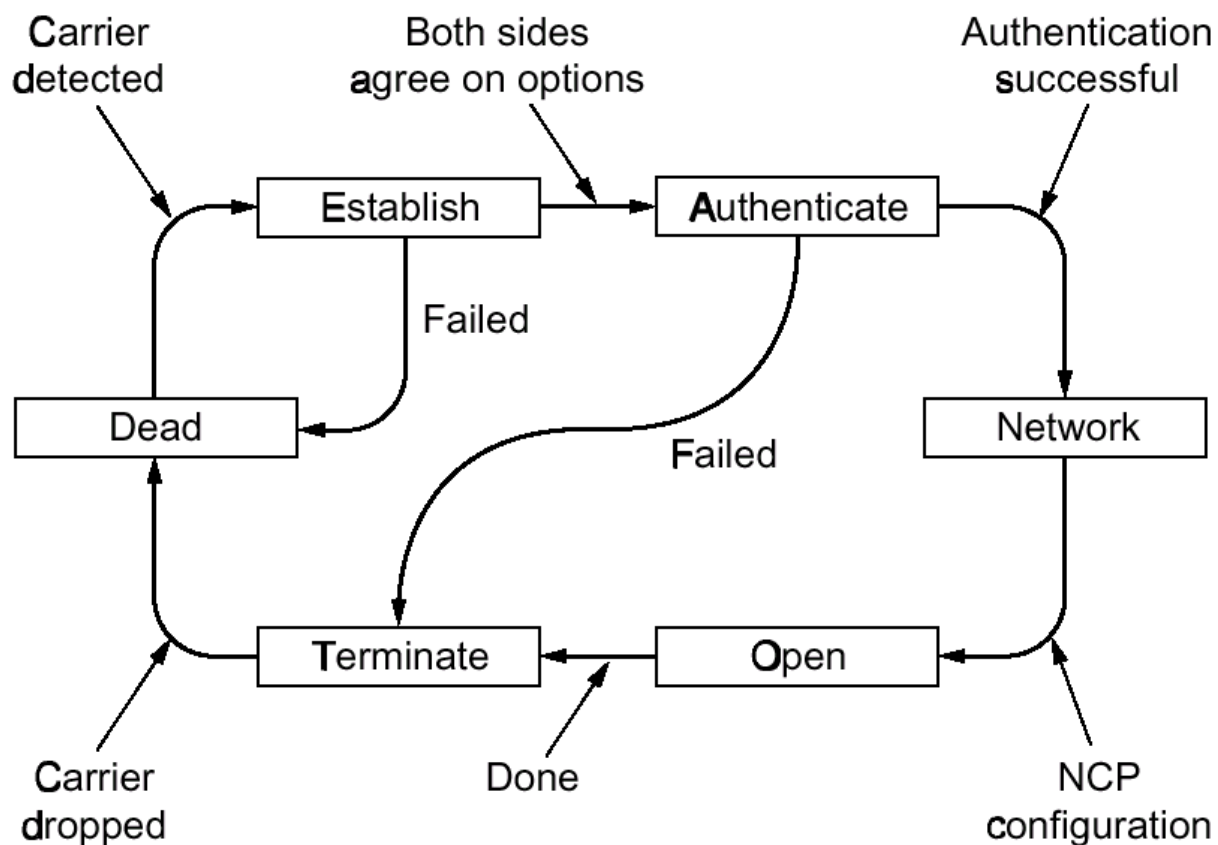


PPP 帧格式 (续)

- 标记域: **01111110**, 字符填充
- 地址域: **11111111**
- 控制域
 - 缺省值为**00000011**, 表示无序号帧, 不提供使用序号和确认的可靠传输
 - 不可靠线路上, 也可使用有序号的可靠传输
- 协议域: 指示净负荷中包的类型, 缺省为**2**字节
- 净负荷域: 变长, 缺省为**1500**字节
- 校验和域: **2或4**个字节



PPP链路 up / down 过程



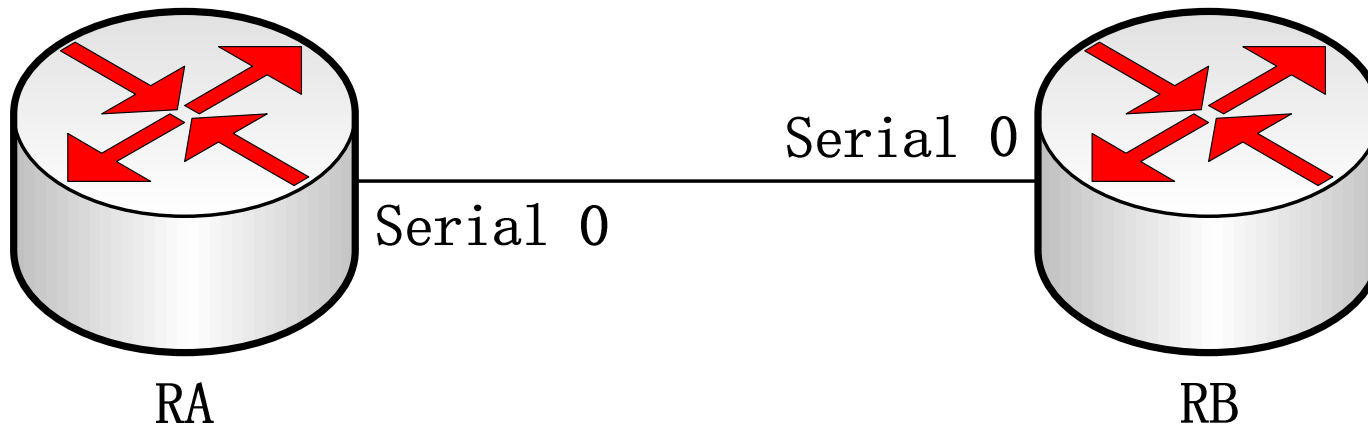


小结

- 三种数据链路层协议：**HDLC**、**LAPB**（面向比特）和**PPP**（面向字符）
- **HDLC**具有三种站，两种构型，三种操作模式
- **X.25 LAPB**是**HDLC**的子集
- **PPP**
 - 具有多协议成帧机制
 - 可以在**modem, HDLC bit-serial lines, SDH/SONET**等物理层上运行
 - 支持差错检测、选项协商、包头压缩、动态分配**IP**地址和认证等
 - 包括两部分协议：**LCP**和**NCP**
 - 通常不使用滑动窗口技术，但是也具有利用**HDLC**帧进行可靠传输的可选功能



在路由器上简单配置HDLC

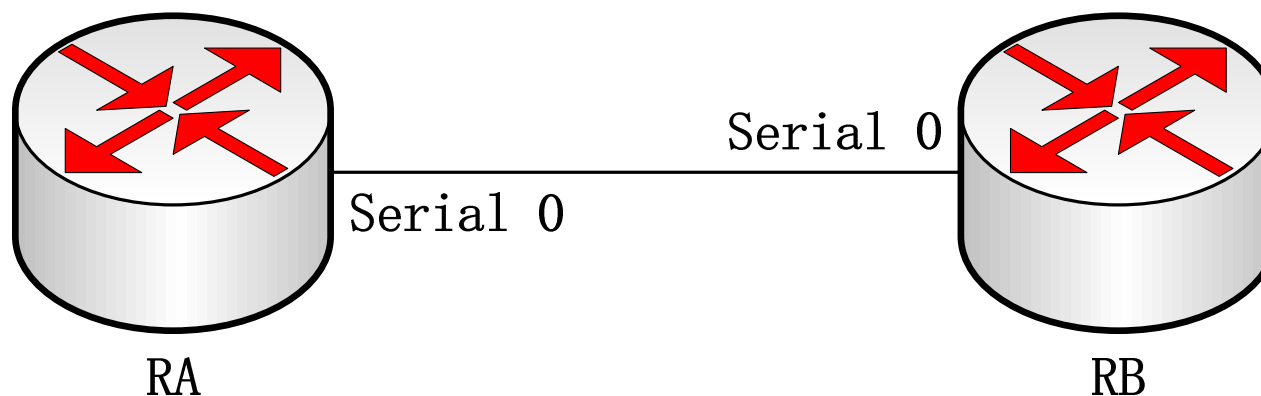


```
interface serial 0  
encapsulation hdlc
```

```
interface serial 0  
encapsulation hdlc
```



在路由器上简单配置LAPB



```
interface serial 0  
encapsulation lapb dce
```

```
interface serial 0  
encapsulation lapb dte
```




在路由器上简单配置PPP



```
interface serial 0  
encapsulation ppp  
ppp authentication chap
```

```
interface serial 0  
encapsulation ppp  
ppp authentication chap
```

The background of the slide features a large, faint, light-gray seal of Tsinghua University. The seal is circular, with the university's name in English, "TSINGHUA UNIVERSITY", around the perimeter. In the center is a shield-like emblem containing stylized Chinese characters and the founding year "1911".

第六章

局域网与介质访问子层



主要内容

- 局域网概述
- 局域网拓扑结构和传输介质
- 局域网技术
 - 信道分配
 - 多路访问协议
- **IEEE 802系列标准**
 - 逻辑链路控制**LLC**
 - **IEEE 802.3** 和 **Ethernet**
 - 快速以太网
 - 千兆以太网



主要内容（续）

- 其他局域网技术
 - IEEE 802.5和Token Ring
 - 光纤分布式数据接口FDDI
 - DPT
- 网桥技术
 - 连接 802.X和 802.Y的网桥
 - 透明网桥/生成数网桥
 - 源路由网桥



局域网概述

■ 局域网产生的原因

- **1980**年代，微型机发展迅速，彼此需要相互通信（近距离），共享资源；
- 分布式的网络应用：分布式计算，分布式数据库

■ 定义

- 局域网是一种将小区域内的各种通信设备互连在一起的通信网络。



局域网概述（续）

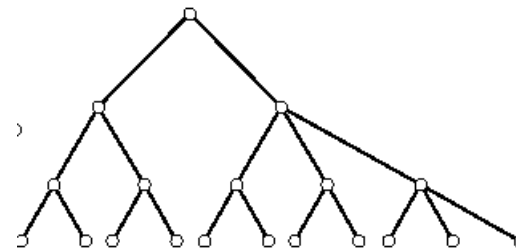
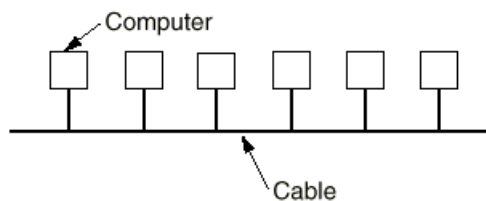
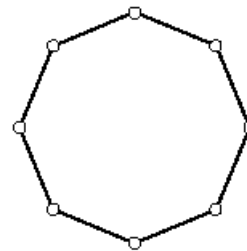
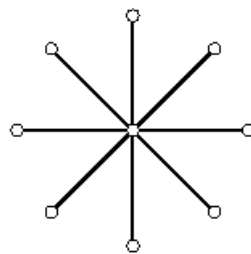
- 局域网的基本特点
 - 高传输率（**10 Mbps ~ 100 Gbps**）
 - 短距离（**0.1 km ~ 10 km**）
 - 低出错率（ **$10^{-8} \sim 10^{-11}$** ）
- 局域网发展趋势
 - 高速，**100G Ethernet**
 - 无线，无线局域网 **IEEE 802.11**



局域网拓扑结构和传输介质

■ 局域网拓扑结构

- 星型结构
- 环型结构
- 总线型结构
- 树型结构



■ 传输介质

- 双绞线
- 基带同轴电缆
- 光纤
- 无线



无线网卡和桌面天线



无线网卡和天线



无线**PCMCIA**网卡



无线 **PCMCIA** 网卡和发送/接收器



信道分配

- 计算机网络可以分成两类
 - 使用点到点连接的网络 —— 广域网
 - 使用广播信道（多路访问信道，随机访问信道）的网络 —— 局域网
 - 关键问题：如何解决对信道争用
- 解决信道争用的协议称为介质访问控制协议 **MAC (Medium Access Control)**，是数据链路层协议的一部分
- 信道分配方法有两种
 - 静态分配
 - 动态分配



信道分配（续）

■ 静态分配

■ 频分多路复用 **FDM**（波分复用**WDM**）

- 原理：将频带平均分配给每个要参与通信的用户
- 优点：适合于用户较少，数目基本固定，各用户的通信量都较大的情况
- 缺点：无法灵活地适应站点数及其通信量的变化

■ 时分多路复用 **TDM**

- 原理：每个用户拥有固定的信道传送时槽
- 优点：适合于用户较少，数目基本固定，各用户的通信量都较大的情况
- 缺点：无法灵活地适应站点数及其通信量的变化



信道分配 (续)

■ 动态分配

■ 信道分配模型

- 独立站点假设：每个站点是独立的，并以统计固定的速率产生帧，一帧产生后到被发送走之前，站点被封锁
- 单信道假设：所有的通信都是通过单一的信道来完成的，各个站点都可以从信道上收发信息
- 冲突假设：若两帧同时发出，会相互重叠，结果使信号无法辨认，称为冲突。所有的站点都能检测到冲突，冲突帧必须重发
- 确定何时发送：连续时间/时间分槽
- 确定能否发送：载波监听/非载波监听



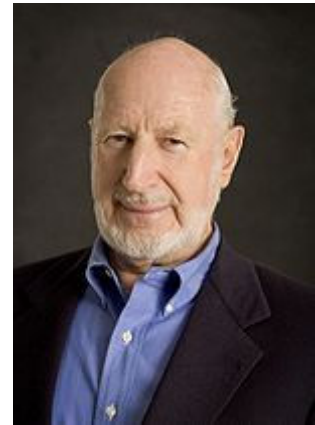
多路访问协议

- 定义：控制多个用户共用一条信道的协议
- 多路访问协议
 - **ALOHA**协议
 - 载波监听多路访问协议**CSMA**
 - 带冲突检测的载波监听多路访问协议**CSMA/CD**
 - 无冲突协议
 - 有限竞争协议
 - 无线局域网协议



ALOHA协议

- 70年代，夏威夷大学的**Norman Abramson**设计了**ALOHA**协议
- 目的：解决信道的动态分配
 - 基本思想可用于任何无协调关系的用户争用单一共享信道使用权的系统
- 分类
 - 纯**ALOHA**协议
 - 分槽**ALOHA**协议





纯ALOHA协议

- 基本思想：用户有数据要发送时，可以直接发至信道；然后监听信道看是否产生冲突，若产生冲突，则等待一段随机的时间重发
- 多用户共享单一信道，并由此产生冲突，这样的系统称为竞争系统

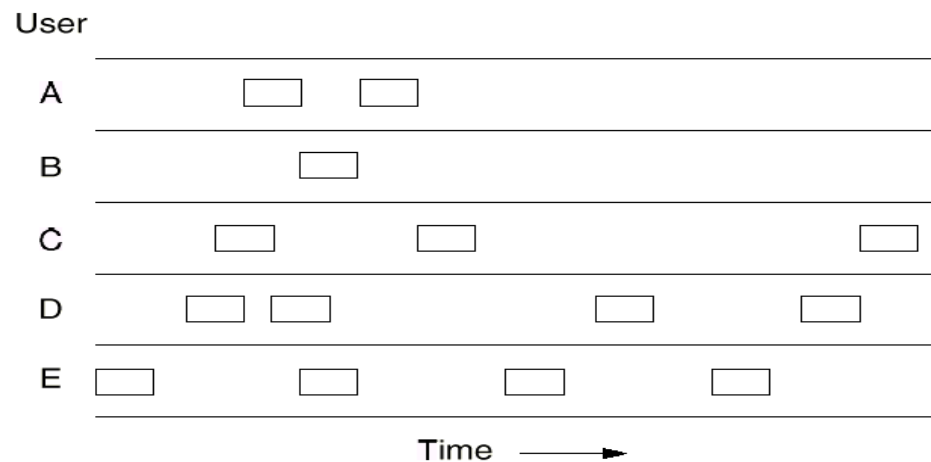


Fig. 4-1. In pure ALOHA, frames are transmitted at completely arbitrary times.



纯ALOHA协议（续）

■ 信道效率

- 假设：帧长固定，无限个用户，按泊松分布产生新帧，平均每个帧时（**frame time**）产生**S**帧（ $0 < S < 1$ ）；发生冲突重传，新旧帧共传**k**次，遵从泊松分布，平均每个帧时产生**G**帧
- 吞吐率 $S = GP_0$ ， P_0 为发送一帧不受冲突影响的概率
- 冲突危险区
- 一个帧时内产生**k**帧的概率： $\Pr[k] = \frac{G^k e^{-G}}{k!}$ ，两个帧时平均产生**2G**个帧，在冲突危险区内无其它帧产生的概率为： $P_0 = e^{-2G}$ ，所以 $S = Ge^{-2G}$
- 效率：信道利用率最高只有**18.4%**

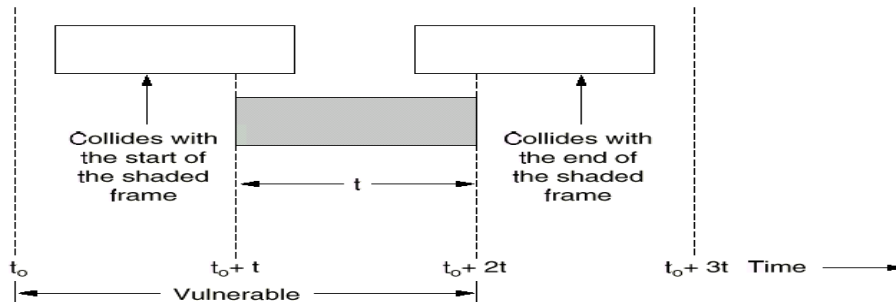


Fig. 4-2. Vulnerable period for the shaded frame.

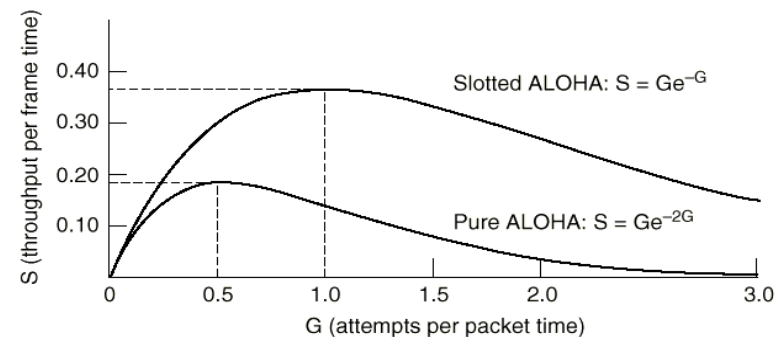


Fig. 4-3. Throughput versus offered traffic for ALOHA systems.



分槽 ALOHA 协议

- 基本思想：把信道时间分成离散的时间槽，槽长为一个帧所需的发送时间。每个站点只能在时槽开始时才允许发送。其他过程与纯**ALOHA**协议相同
- 信道效率
 - 冲突危险区是纯**ALOHA**的一半，所以 $P_0 = e^{-G}$ ， $S = Ge^{-G}$
 - 与纯**ALOHA**协议相比，降低了产生冲突的概率，信道利用率最高为**36.8%**

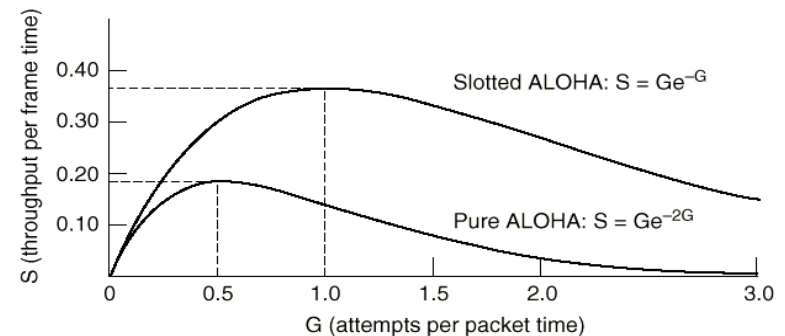


Fig. 4-3. Throughput versus offered traffic for ALOHA systems.



载波监听多路访问协议CSMA

- **CSMA (Carrier Sense Multiple Access Protocols)**
- **载波监听 (Carrier Sense)**
 - 站点在为发送帧而访问传输信道之前，首先监听信道有无载波
 - 若有载波，说明已有用户在使用信道，则不发送帧以避免冲突
- **多路访问 (Multiple Access)**
 - 多个用户共用一条线路



1-坚持型CSMA

■ 1-persistent CSMA

■ 原理

- 若站点有数据发送，先监听信道
- 若站点发现信道空闲，则发送
- 若信道忙，则继续监听直至发现信道空闲，然后完成发送
- 若产生冲突，等待一随机时间，然后重新开始发送过程

■ 优点：减少了信道空闲时间

■ 缺点：增加了发生冲突的概率

■ 广播延迟对协议性能的影响

- 广播延迟越大，发生冲突的可能性越大，协议性能越差



非坚持型CSMA

- **nonpersistent CSMA**

- 原理

- 若站点有数据发送，先监听信道
- 若站点发现信道空闲，则发送
- 若信道忙，等待一随机时间，然后重新开始发送过程
- 若产生冲突，等待一随机时间，然后重新开始发送过程

- 优点：减少了冲突的概率

- 缺点：增加了信道空闲时间，数据发送延迟增大

- 随着每个帧时发送帧数量（**G**）的增加，非坚持型**CSMA**信道效率比 **1-坚持CSMA**高，传输延迟比 **1-坚持CSMA**大



p-坚持型CSMA

- **p-persistent CSMA**
- 适用于分槽信道
- 原理
 - 若站点有数据发送，先监听信道
 - 若站点发现信道空闲，则以概率 p 发送数据，以概率 $q = 1 - p$ 延迟至下一个时槽发送。若下一个时槽仍空闲，重复此过程，直至数据发出或时槽被其他站点所占用
 - 若信道忙，则等待下一个时槽，重新开始发送
 - 若产生冲突，等待一随机时间，然后重新开始发送



五种多路访问协议性能比较

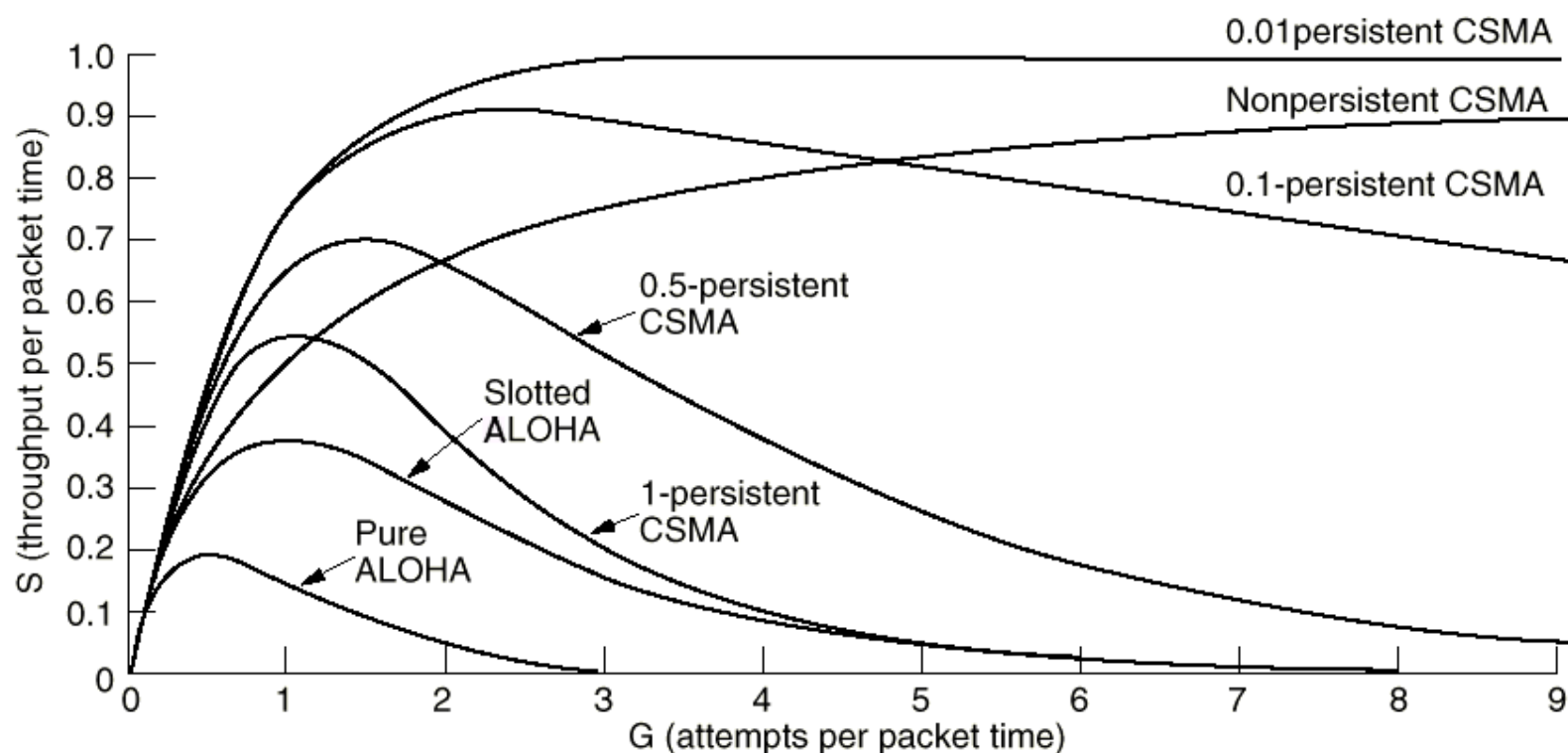


Fig. 4-4. Comparison of the channel utilization versus load for various random access protocols.



带冲突检测的载波监听多路访问协议 CSMA/CD

■ 引入原因

- 当两个帧发生冲突时，两个被损坏帧继续传送毫无意义，而且信道无法被其他站点使用，浪费信道
- 如果站点边发送边监听，并在监听到冲突之后立即停止发送，可以提高信道的利用率，因此产生了**CSMA/CD**

■ 原理

- 站点使用**CSMA**协议进行数据发送
- 在发送期间如果检测到冲突，立即终止发送，并发出一个瞬间干扰信号，使所有的站点都知道发生了冲突
- 在发出干扰信号后，等待一段随机时间，再重新开始发送

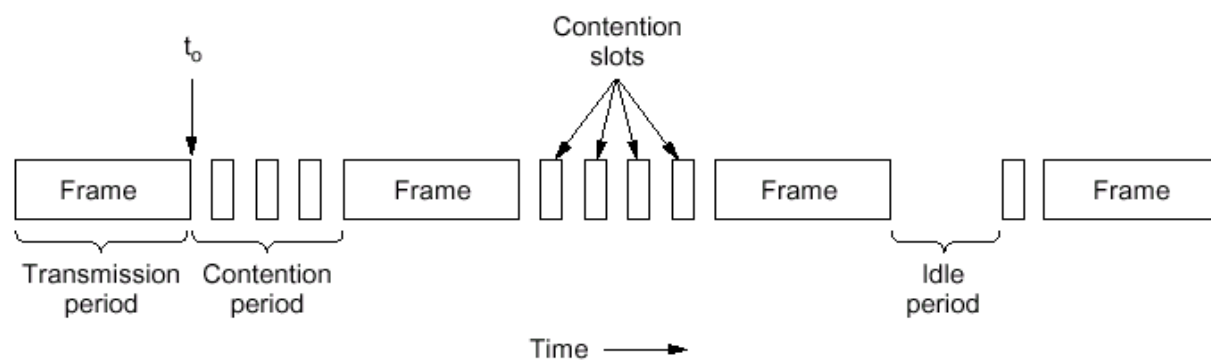
■ 问题：一个站点确定发生冲突要花多少时间？

- 最坏情况下，**2倍**电缆传输时间



无冲突协议

- 工作状态
 - 传输周期
 - 竞争周期
 - 空闲周期



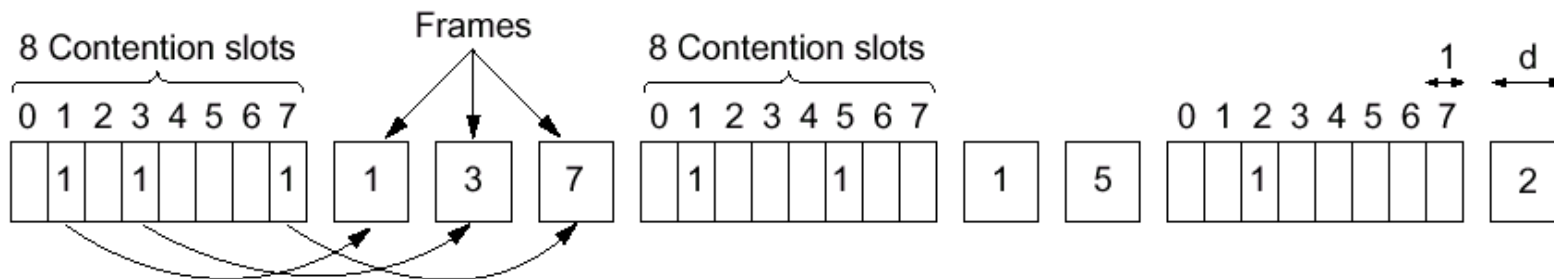


无冲突协议（续）

■ 基本位图协议（A Bit-Map Protocol）

■ 工作原理

- 共享信道上共有N个站，竞争周期分为N个时槽，如果一个站有帧发送，则在对应的时槽内发送比特1
- N个时槽之后，每个站都知道哪个站要发送帧，这时按站序号发送





无冲突协议（续）

- 象这样在实际发送信息前先广播发送请求的协议称为预留协议（**reservation protocol**）
- 效率
 - 轻负载下，效率为 $d / (N + d)$ ，数据帧由 d 个时间单位组成
 - 重负载下，效率为 $d / (d + 1)$
- 缺点
 - 与站序号有关的不平等性，序号大的站得到的服务好
 - 每个站都有 **1** 比特的开销



无冲突协议（续）

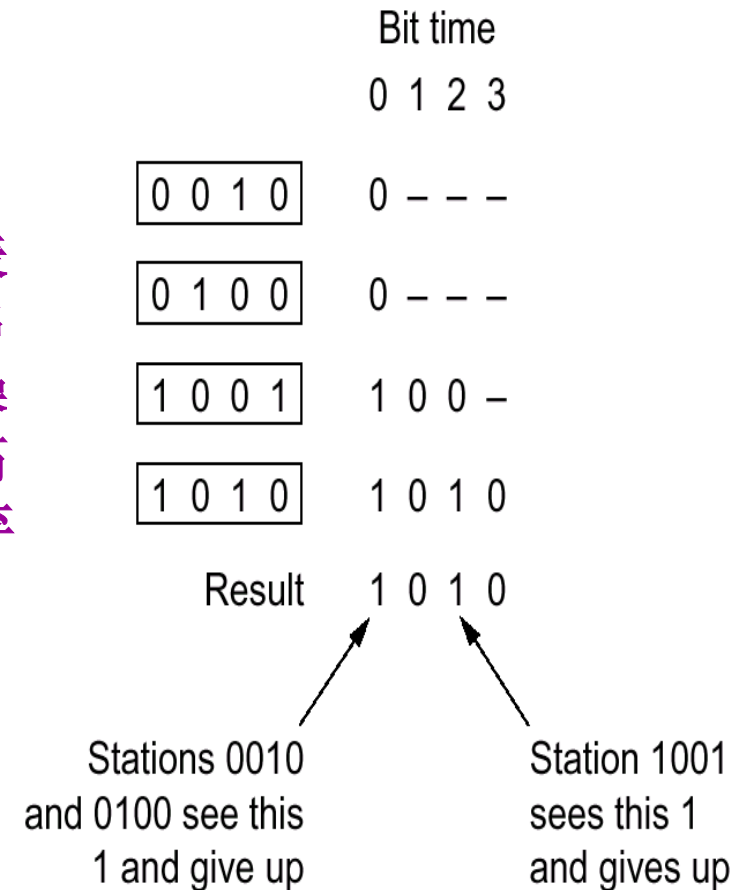
■ 二进制下数法 (Binary Countdown)

■ 工作原理

- 所有站的地址用等长二进制位串表示，若要占用信道，则广播该位串
- 不同站发的地址中的位做“或”操作，一旦某站了解到比本站地址高位更高的位置被置为“1”，便放弃发送请求

■ 效率

- $d / (d + \log_2 N)$





有限竞争协议

■ 占用信道的策略

■ 竞争方法

■ 例，CSMA

- 轻负载下，发送延迟小；重负载下，信道效率低

■ 无冲突方法

■ 例，基本位图法

- 轻负载下，发送延迟大；重负载下，信道效率高

■ 有限竞争方法

- 结合以上两种方法，轻负载下使用竞争，重负载下使用无冲突方法
- 减少竞争的站的数目可以增加获取信道的概率
- 基本思路：将站分组，组内竞争
- 问题：如何分组？



适应树搜索协议

■ 工作原理

- 站点组织成二叉树
- 一次成功传输之后，第0槽全部站可竞争信道，只有一个站要使用信道则发送；有冲突则在第1槽内半数站（2以下站）参与竞争。如其中之一获得信道，本帧后的时槽留给3以下的站；如发生冲突，继续折半搜索

- 当系统负载很重时，从根结点开始竞争发生冲突的概率非常大。为提高效率，可以从中间结点开始竞争
 - 问题：搜索应该从树的哪一级开始？

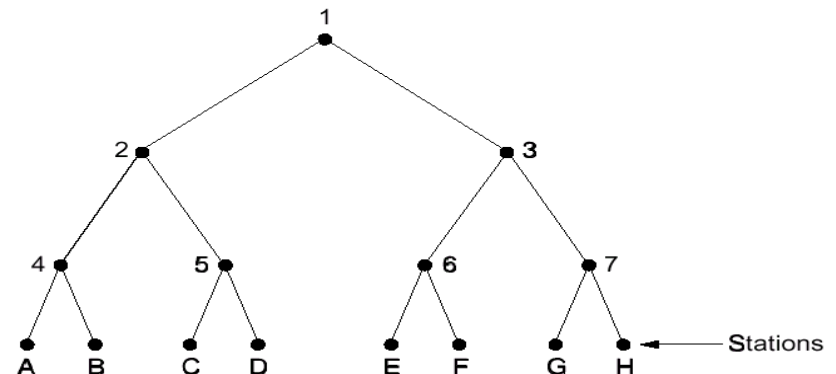
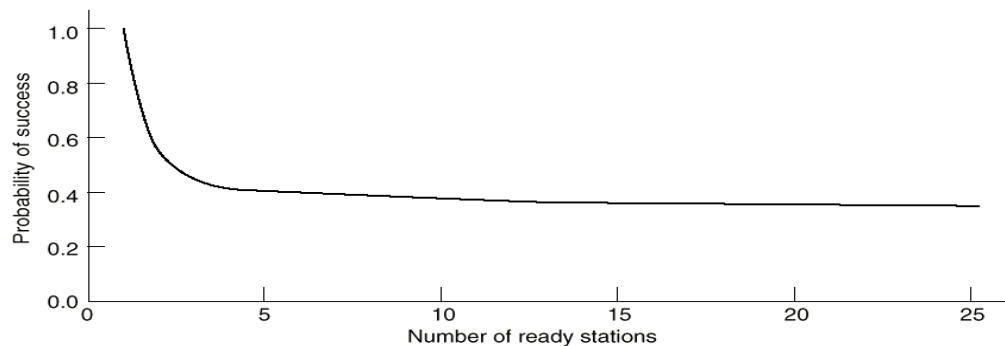


Fig. 4-8. Acquisition probability for a symmetric contention channel.



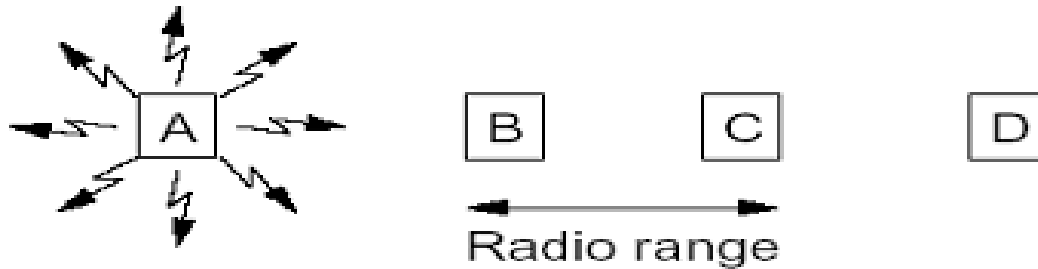
无线局域网协议

- 无线局域网产生背景
 - 笔记本电脑的普及促进了无线局域网的发展
 - **portable $\neq$ mobile**
 - 要做到真正的移动，需要使用无线信号进行通信
- 无线局域网的特点
 - 只有一个信道（与蜂窝电话不同）
 - 短距离传输
 - 一个站点发送的信号，只能被它周围一定范围内的站点接收到



无线局域网协议（续）

- 无线局域网与有线局域网不同
 - 隐藏站点问题（**hidden station problem**）
 - 由于站点距离竞争者太远，从而不能发现潜在介质竞争者的问题称为隐藏站点问题
 - **A**向**B**发送数据的过程中，**C**由于收不到**A**的数据，也可以向**B**发送数据，导致**B**接收时发生冲突

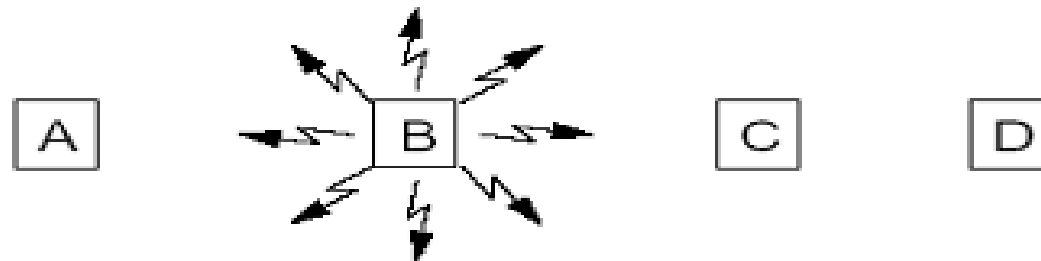




无线局域网协议（续）

■ 暴露站点问题（exposed station problem）

- 由于非竞争者距离发送站点太近，从而导致介质非竞争者不能发送数据的问题称为暴露站点问题
- **B**向**A**发送数据，被**C**监听到，导致**C**不能向**D**发送数据





无线局域网协议（续）

- 传统的**CSMA**协议不适合于无线局域网，需要特殊的**MAC**子层协议
 - **CSMA**
 - 电缆上的信号传播给所有站点
 - **CSMA**只判断发送站点周围是否有其它活跃发送站点
 - 冲突被发送站点发现
 - 某一时刻，信道上只能有一个有效数据帧
 - 无线局域网
 - 信号只能被发送站点周围一定范围内的站点接收
 - 需要尽量保证接收站点周围一定范围内只有一个发送站点
 - 冲突被接收站点发现
 - 某一时刻，信道上可以有多个有效数据帧



无线局域网协议（续）

■ MACA（Multiple Access with Collision Avoidance）

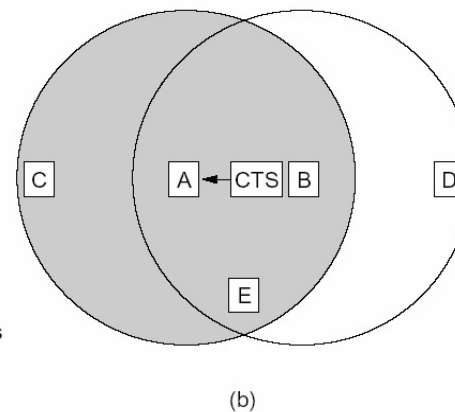
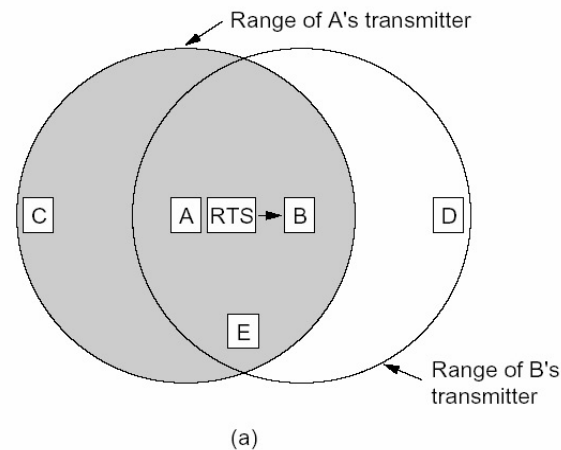
- 是**IEEE 802.11**无线局域网标准的基础
- 基本思想：发送站点刺激接收站点发送应答短帧，从而使得接收站点周围的站点监听到该帧，并在一定时间内避免发送数据



无线局域网协议（续）

■ 基本过程

- **A向B发送RTS(Request To Send)帧，A周围的站点在一定时间内不发送数据，以保证CTS帧返回给A**
- **B向A回答CTS(Clear To Send)帧，B周围的站点在一定时间内不发送数据，以保证A发送完数据**
- **A开始发送**
- **若发生冲突，采用二进制指数后退算法等待随机时间，再重新开始**





无线局域网协议（续）

■ MACAW

- 对**MACA**协议做了改进，提高了性能
- 主要改进
 - 对每个成功传输的数据帧，都要产生确认帧
 - 增加了发送站点的载波监听
 - 发生冲突后，针对每个数据流（相同源和目的地址）执行后退算法，而不是针对每个站点
 - 发生拥塞时，站点间交互信息



小结

- 关键问题：在**MAC**子层如何解决多个站点争用共享信道？
- 信道分配方式：静态（**FDM**、**WDM**、**TDM**），动态
- 多路访问协议
 - 竞争协议
 - **ALOHA**：纯**ALOHA**、分槽**ALOHA**
 - **CSMA**：1-坚持型**CSMA**、非坚持型**CSMA**、p-坚持型**CSMA**（分槽协议）
 - **CSMA/CD**
 - **Ethernet**采用哪种**CSMA**协议？
 - 无冲突协议
 - 有限竞争协议
 - 无线局域网协议（**MACA**, **MACAW**）

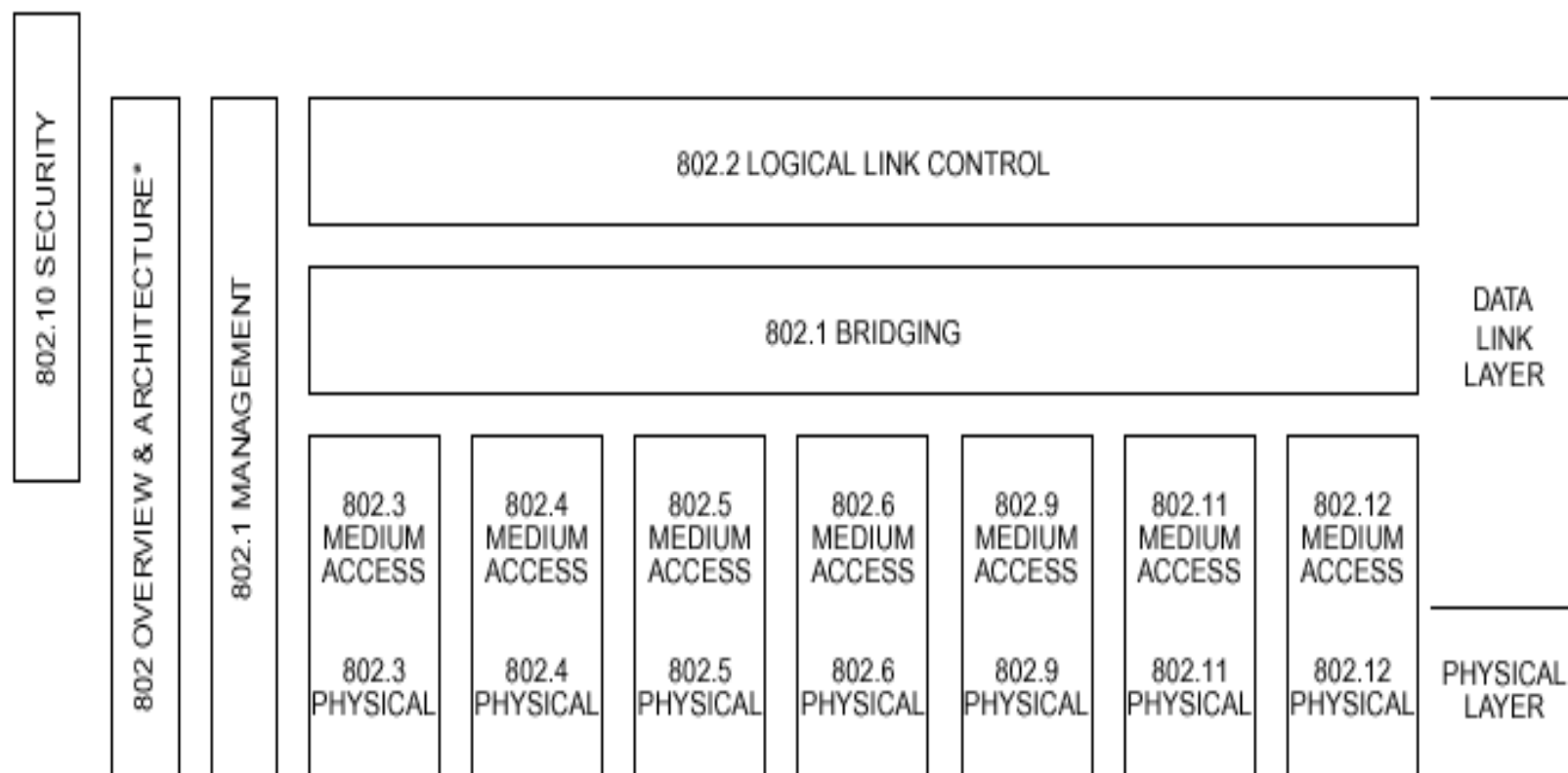


IEEE 802协议

- **IEEE 802**系列标准定义了若干种**LAN**，包括对物理层、**MAC**子层的定义和描述。它的组成如下：
 - **802.1** 基本介绍和接口原语定义
 - **802.2** 逻辑链路控制（**LLC**）子层
 - **802.3** 采用**CSMA/CD**技术的局域网
 - **802.4** 采用令牌总线（**Token Bus**）技术的局域网
 - **802.5** 采用令牌环（**Token Ring**）技术的局域网
 - ...



802标准在网络体系结构中的位置



* Formerly IEEE Std 802.1A.



IEEE 802 family

- IEEE Std 802 *Overview and Architecture.* This standard provides an overview to the family of IEEE 802 Standards.
- ANSI/IEEE Std 802.1B and 802.1k [ISO/IEC 15802-2] *LAN/MAN Management.* Defines an OSI management-compatible architecture, and services and protocol elements for use in a LAN/MAN environment for performing remote management.
- ANSI/IEEE Std 802.1D [ISO/IEC 15802-3] *Media Access Control (MAC) Bridges.* Specifies an architecture and protocol for the interconnection of IEEE 802 LANs below the MAC service boundary.
- ANSI/IEEE Std 802.1E [ISO/IEC 15802-4] *System Load Protocol.* Specifies a set of services and protocol for those aspects of management concerned with the loading of systems on IEEE 802 LANs.
- ANSI/IEEE Std 802.1F *Common Definitions and Procedures for IEEE 802 Management Information*
- ANSI/IEEE Std 802.1G [ISO/IEC 15802-5] *Remote Media Access Control (MAC) Bridging.* Specifies extensions for the interconnection, using non-LAN communication technologies, of geographically separated IEEE 802 LANs below the level of the logical link control protocol.
- ANSI/IEEE Std 802.2 [ISO/IEC 8802-2] *Logical Link Control*
- ANSI/IEEE Std 802.3 [ISO/IEC 8802-3] *CSMA/CD Access Method and Physical Layer Specifications*
- ANSI/IEEE Std 802.4 [ISO/IEC 8802-4] *Token Passing Bus Access Method and Physical Layer Specifications*



IEEE 802 family (续)

- ANSI/IEEE Std 802.5 [ISO/IEC 8802-5] *Token Ring Access Method and Physical Layer Specifications*
- ANSI/IEEE Std 802.6 [ISO/IEC 8802-6] *Distributed Queue Dual Bus Access Method and Physical Layer Specifications*
- ANSI/IEEE Std 802.9 [ISO/IEC 8802-9] *Integrated Services (IS) LAN Interface at the Medium Access Control (MAC) and Physical (PHY) Layers*
- ANSI/IEEE Std 802.10 *Interoperable LAN/MAN Security*
- ANSI/IEEE Std 802.11 [ISO/IEC DIS 8802-11] *Wireless LAN Medium Access Control (MAC) and Physical Layer Specifications*
- ANSI/IEEE Std 802.12 [ISO/IEC DIS 8802-12] *Demand Priority Access Method, Physical Layer and Repeater Specifications*

In addition to the family of standards, the following is a recommended practice for a common Physical Layer technology:

- IEEE Std 802.7 *IEEE Recommended Practice for Broadband Local Area Networks*

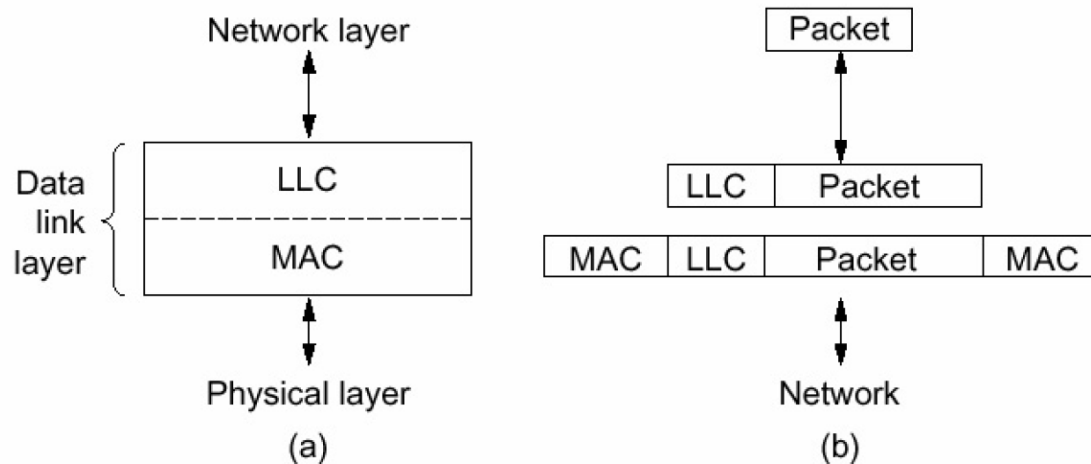
The following additional working group has authorized standards projects under development:

- IEEE 802.14 *Standard Protocol for Cable-TV Based Broadband Communication Network*



LAN的参考模型

- 引入**LLC**子层的原因：
 - **MAC**子层只提供尽力而为的数据报服务，不提供确认机制和流量控制（滑动窗口）
 - 有些情况下，这种服务足够，如支持**IP**协议；当需要确认和流控的时候，这种服务就不能满足，需要**LLC**
 - 对于同一个**LLC**，可以提供多个**MAC**选择





逻辑链路控制子层 LLC

- **LLC: Logical Link Control**
- **LLC子层提供确认机制和流量控制**
- **LLC隐藏了不同802MAC子层的差异，为网络层提供单一的格式和接口**
- **LLC提供三种服务选项：**
 - 不可靠数据报服务，有确认数据报服务，可靠的面相连接的服务
- **LLC帧头基于HDLC协议**



介质访问控制子层 MAC

- **MAC: Medium Access Control**
- **MAC子层的功能**
 - 数据帧封装、发送和接收
 - 组帧（帧定界、帧同步）
 - 寻址（源和目的**MAC**地址处理）
 - 差错检测
 - 介质访问管理
 - 介质分配（避免冲突）
 - 冲突解决（处理冲突）



IEEE 802.3和Ethernet

■ 历史

- **ALOHA**系统
- **ALOHA + 载波监听**
- **Xerox** 设计了**2.94Mbps**的采用**CSMA/CD**协议的**Ethernet**
- **Xerox, DEC, Intel**共同制定了**10Mbps**的**CSMA/CD**以太网标准
- **IEEE**定义了采用**1-坚持型CSMA/CD**技术的**802.3**局域网标准，速率从**1M**到**10Mbps**

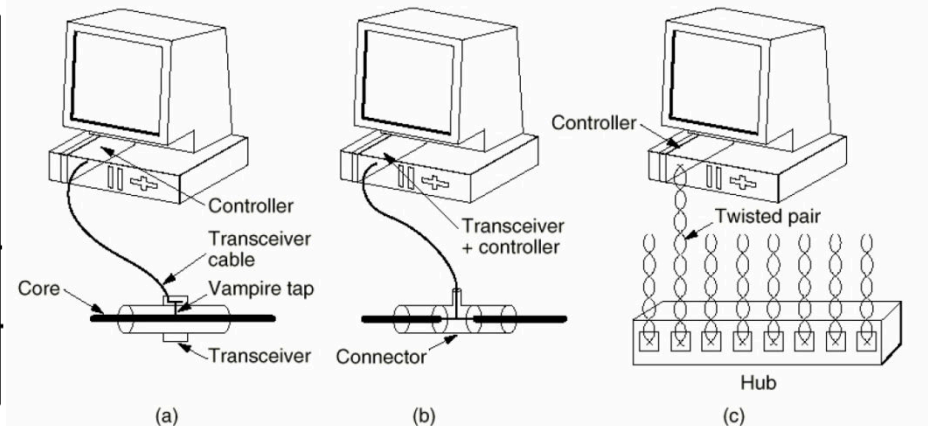
■ **IEEE 802.3**标准与以太网协议略有差别



IEEE 802.3和Ethernet（续）

- **802.3采用的电缆标准**
 - **10Base5**: 粗缆, **AUI**接口
 - **10Base2**: 细缆, **BNC**接口, **T型头**
 - **10Base-T**: **RJ-45**接口
 - **10Base-F**: 光纤接口

Name	Cable	Max. segment	Nodes/seg.	Advantages
10Base5	Thick coax	500 m	100	Good for backbones
10Base2	Thin coax	200 m	30	Cheapest system
10Base-T	Twisted pair	100 m	1024	Easy maintenance
10Base-F	Fiber optics	2000 m	1024	Best between buildings





IEEE 802.3和Ethernet（续）

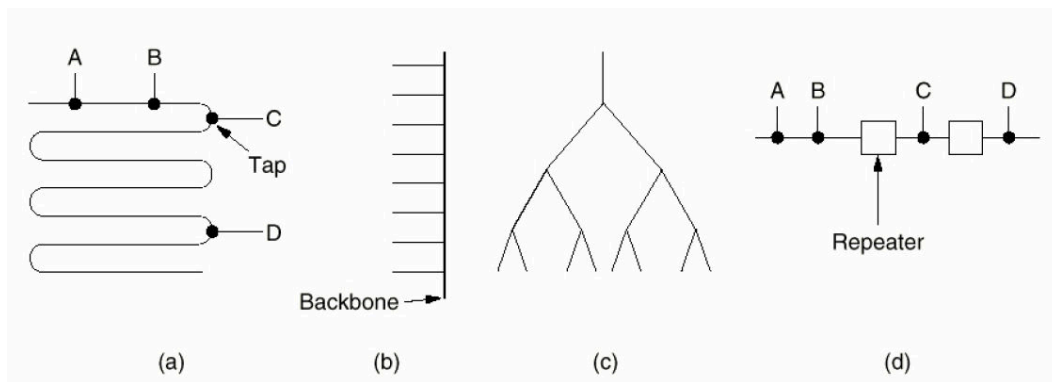
- 物理层类型用以下域表示
 - **<data rate in Mb/s> <medium type>**
<maximum segment length (*100m)>
 - **10Base5**含义
 - **10: 10Mbps**
 - **Base: 基带传输 (baseband medium)**
 - **5: 500米**
- **收发器 (transceiver) : 处理载波监听和冲突检测**



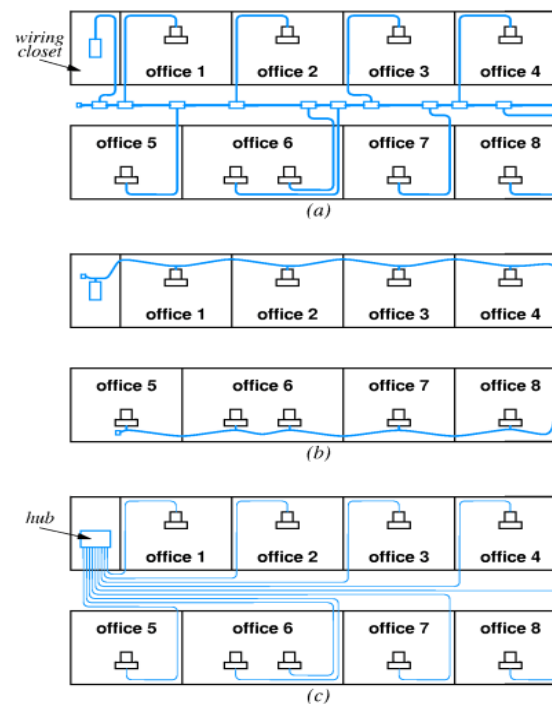
IEEE 802.3和Ethernet（续）

■ 布线拓扑结构

■ 总线形，脊椎形，树形，分段



■ 三种电缆布线



8个办公室的网络连接 (a)
粗缆(b) 细缆 (c) 10Base-T 双绞线



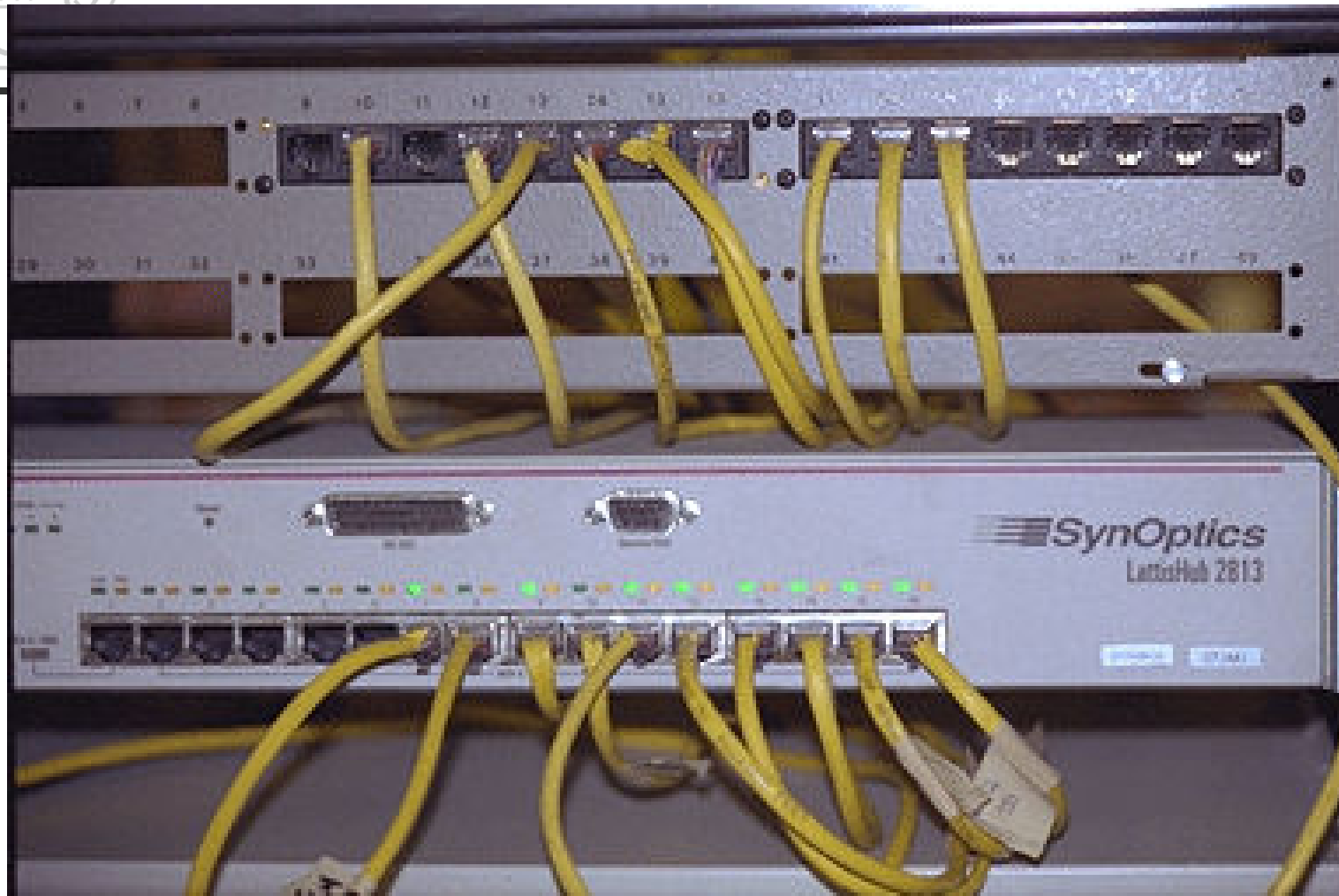
IEEE 802.3 和 Ethernet (续)

■ 扩展网段长度

- 中继器：物理层设备，只对信号进行接收、放大和双向重传
- 两个收发器之间最多使用4个中继器，最长2500米

■ 802.3的信号编码

- 由于曼彻斯特编码简单，所有的802.3基带系统都使用曼彻斯特编码

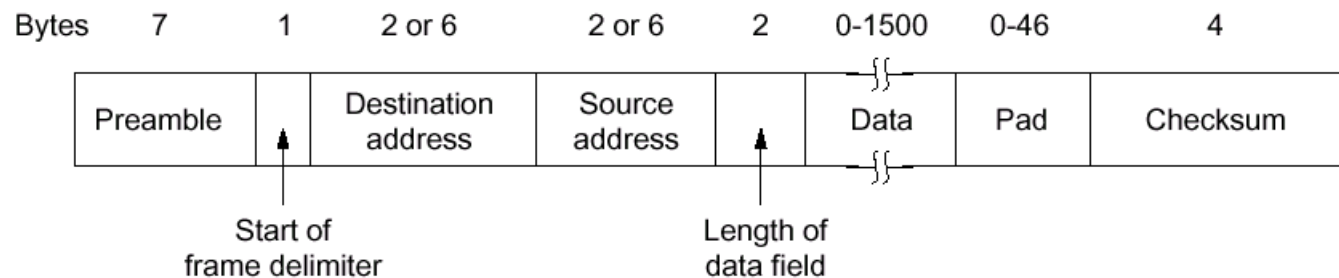


10Base-T hub



IEEE 802.3 和 Ethernet (续)

■ 802.3的MAC子层帧格式



- 前导序列 (7个字节 **10101010**)
- 帧开始标志 (1字节, **10101011**)



IEEE 802.3 和 Ethernet (续)

- 目标地址和源地址
 - 2 或 6 个字节，以太网为 6 个字节
 - 目的地址第一位 (**LSB: Least Significant Bit**) 为 **0**，表示单地址 (**individual address**)；为 **1**，表示组地址 (**group address**)，支持 **multicast**，目的地址全 **1**，为广播地址。源地址第一位 (**LSB**) 为 **0**
 - 地址中的第二位 (**LSB**) 用来区分本地地址和全球地址。
- 帧长度域 (**2 字节**，取值在 **0-1500** 之间)
- 数据 (**0-1500 个字节**)
- 填充 (**0-46 字节**)
- 校验和: **CRC 校验 (4 个字节)**



I/G	U/L	46-BIT ADDRESS
-----	-----	----------------

I/G = 0 INDIVIDUAL ADDRESS

I/G = 1 GROUP ADDRESS

U/L = 0 GLOBALLY ADMINISTERED ADDRESS

U/L = 1 LOCALLY ADMINISTERED ADDRESS

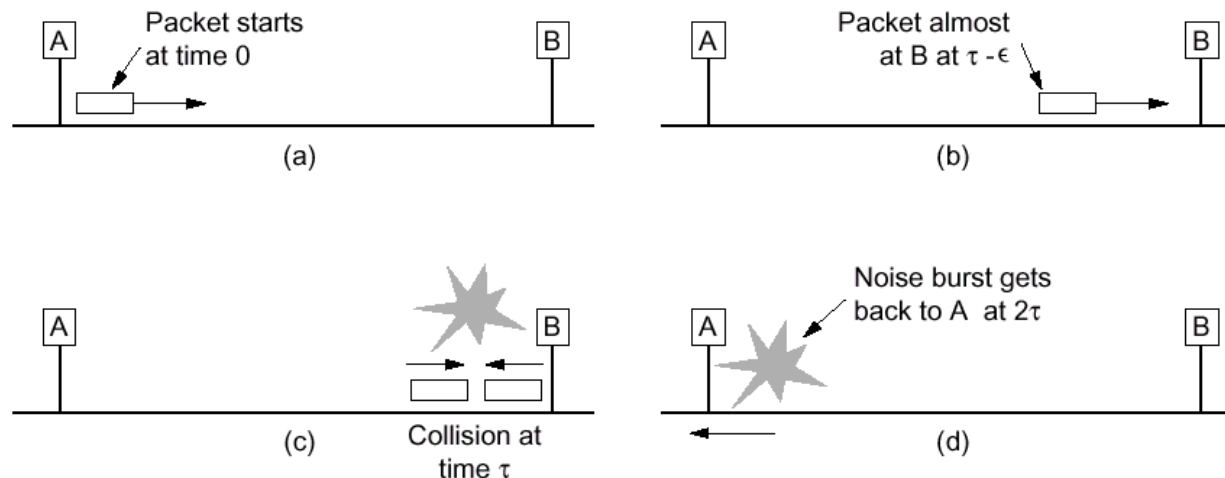
Figure 3-2—Address field format



IEEE 802.3 和 Ethernet (续)

■ 最短帧长

- 避免帧的第一个比特到达电缆的远端前帧已经发完，帧发送时间应该大于 2τ ;
- **10Mbps LAN**，最大冲突检测时间为**51.2**微秒，最短帧长为**64**字节;
- 网络速度提高，最短帧长也应该增大或者站点间的距离要减小。





IEEE 802.3 和 Ethernet (续)

- 二进制指数后退算法 (**binary exponential backoff**)
 - 将冲突发生后的时间划分为长度为**51.2**微秒的时槽
 - 发生第一次冲突后, 各个站点等待 **0** 或 **1** 个时槽再开始重传;
 - 发生第二次冲突后, 各个站点随机地选择等待**0,1, 2**或**3**个时槽再开始重传;
 - 第 **i** 次冲突后, 在 **0** 至 **2^i-1** 间随机地选择一个等待的时槽数, 再开始重传;
 - **10**次冲突后, 选择等待的时槽数固定在**0**至 **$2^{10}-1$** 间;
 - **16**次冲突后, 发送失败, 报告上层。

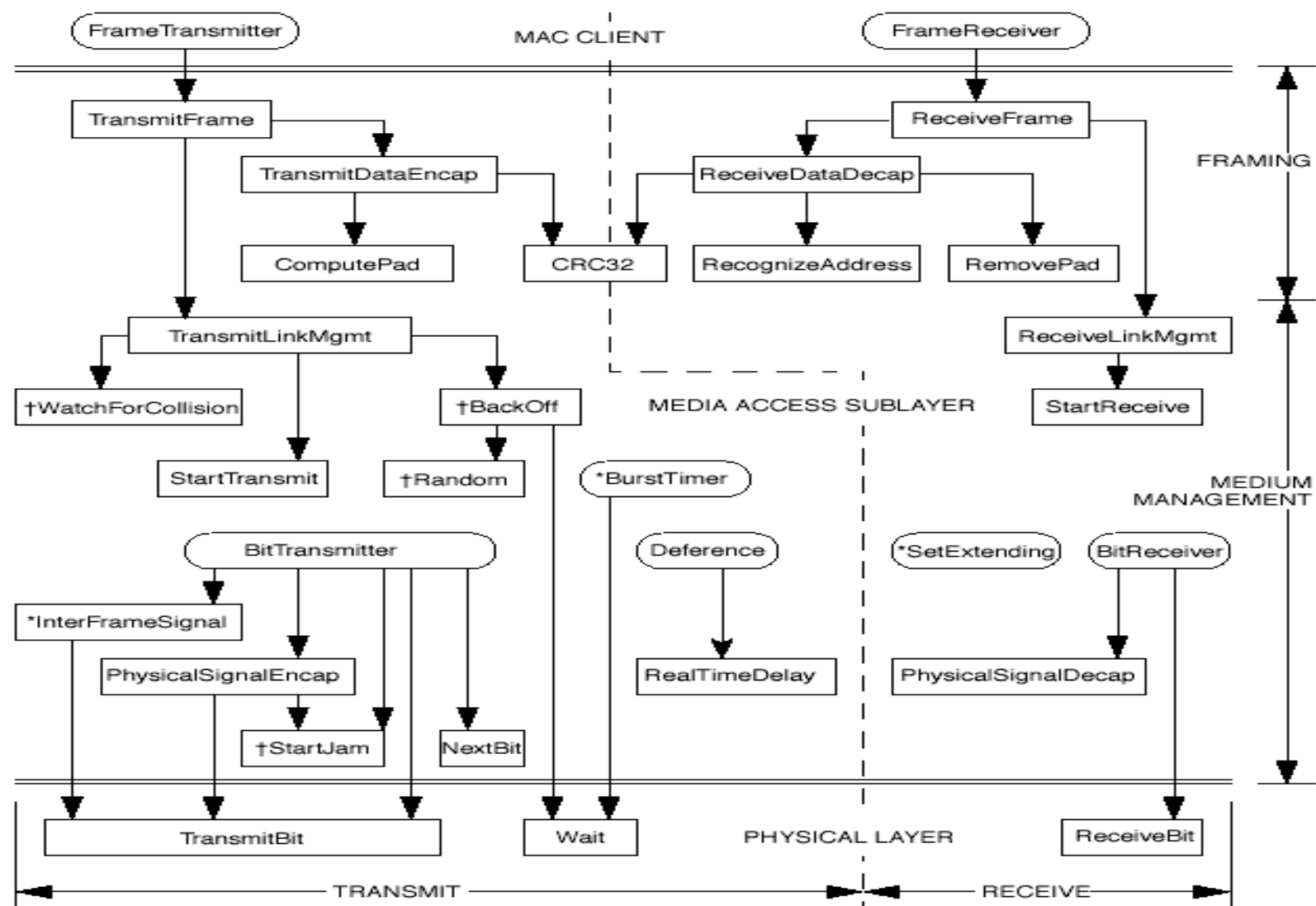


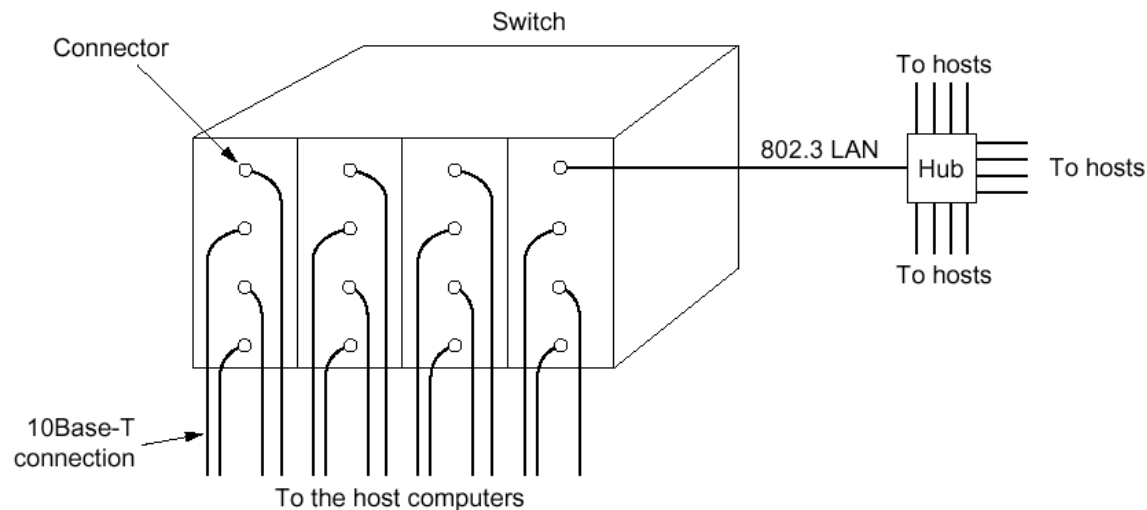
Figure 4-3—Relationship among CSMA/CD procedures



IEEE 802.3 和 Ethernet (续)

■ 交换式802.3 LAN

- 目的：减少冲突；
- 两种实现方法
 - 一个卡内是一个**802.3LAN**，构成自己的冲突域，卡间并行
 - 使用端口缓存，无冲突发生





IEEE 802.3 和 Ethernet (续)

This standard provides for two modes of operation of the MAC sublayer:

- a) In *half duplex* mode, stations contend for the use of the physical medium, using the CSMA/CD algorithms specified. Bidirectional communication is accomplished by rapid exchange of frames, rather than full duplex operation. Half duplex operation is possible on all supported media; it is required on those media that are incapable of supporting simultaneous transmission and reception without interference, for example, 10BASE2 and 100BASE-T4.
- b) The *full duplex* mode of operation can be used when all of the following are true:
 - 1) The physical medium is capable of supporting simultaneous transmission and reception without interference (e.g., 10BASE-T, 10BASE-FL, and 100BASE-TX/FX).
 - 2) There are exactly two stations on the LAN. This allows the physical medium to be treated as a full duplex point-to-point link between the stations. Since there is no contention for use of a shared medium, the multiple access (i.e., CSMA/CD) algorithms are unnecessary.
 - 3) Both stations on the LAN are capable of and have been configured to use full duplex operation.

The most common configuration envisioned for full duplex operation consists of a central bridge (also known as a switch) with a dedicated LAN connecting each bridge port to a single device.



快速以太网

■ Fast Ethernet

■ 标准

- **1995年，IEEE通过802.3u标准，实际上是802.3的一个补充。原有的帧格式、接口、规程不变，只是将比特时间从100ns缩短为10ns**
- **已合并到IEEE 802.3中**

■ 对**10 Mbps 802.3 LAN**的改进

- **一种方法是改进10Base-5 或 10Base-2，采用CSMA/CD，最大电缆长度减为1/10，未被采纳**
- **另一种方法是改进10Base-T，使用HUB，被采纳**



Name	Cable	Max. segment	Advantages
100Base-T4	Twisted pair	100 m	Uses category 3 UTP
100Base-TX	Twisted pair	100 m	Full duplex at 100 Mbps
100Base-FX	Fiber optics	2000 m	Full duplex at 100 Mbps; long runs

Fig. 4-47. Fast Ethernet cabling.



快速以太网（续）

■ 100Base-T4

- 使用4对 **ISO/IEC 11801**定义的3、4、5类平衡双绞线
- 3类非屏蔽双绞线（**UTP**），使用**25MHz**的信号
（**802.3 10M LAN**使用**20MHz**的信号，由于使用**Manchester**编码，波特率=2*比特率）
- 4对双绞线，1对 **to the hub**，1对 **from the hub**，另外2对根据数据传输方向变换
- **8B6T**（**8 bits map to 6 trits**）编码，使用三进制信号（**ternary signals**），1对双绞线的比特率为 **$25 * 8/6 = 33.3 \text{ Mbps}$** ，正向**100M**，反向**33.3M**

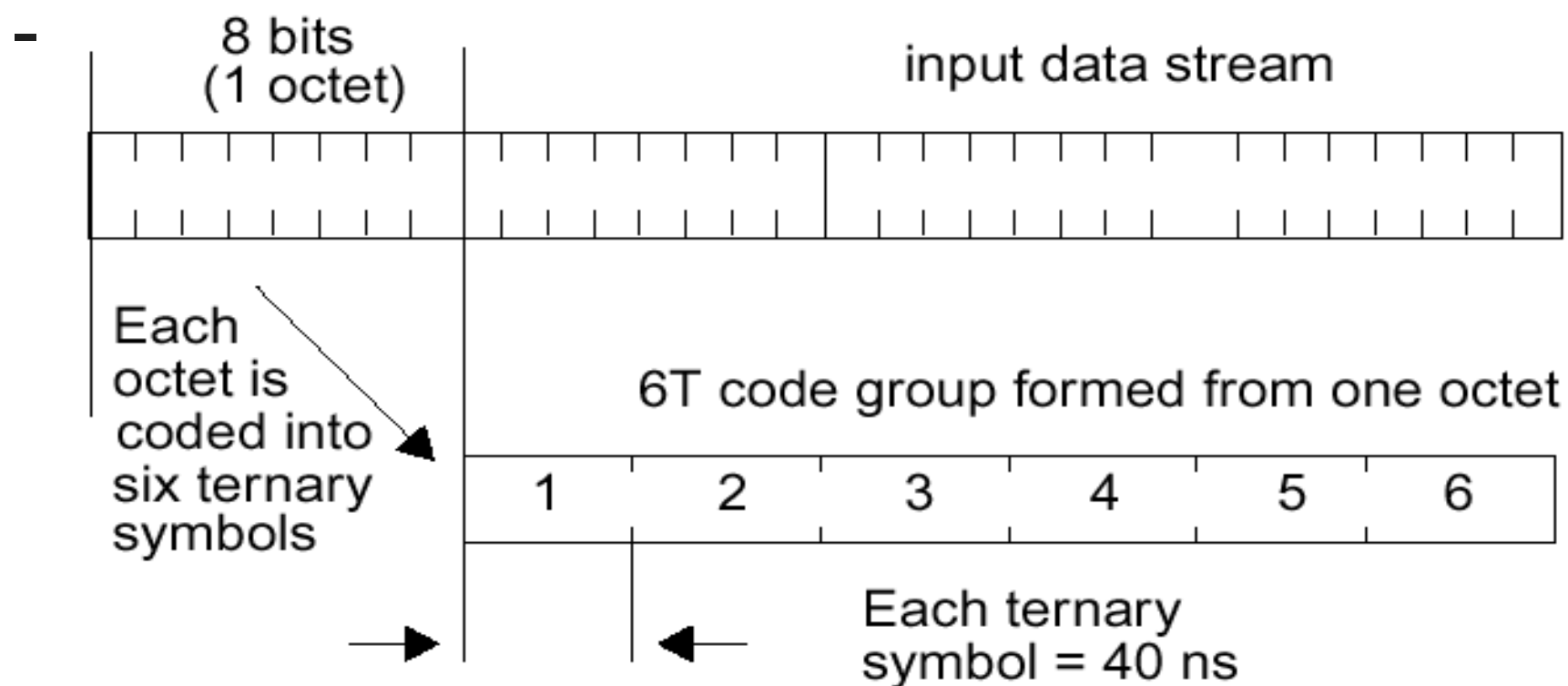


Figure 23-4—8B6T coding



快速以太网（续）

■ 100Base-TX

- 使用**2对5类平衡双绞线**或**150Ω屏蔽平衡电缆**，**1对 to the hub**，**1对 from the hub**，全双工
- **5类双绞线使用125 MHz的信号**
- **4B5B编码**，**5个时钟周期发送4个比特**，物理层与**FDDI兼容**，比特率为 $125 * 4/5 = 100 \text{ Mbps}$

■ 100Base-FX

- 使用**2根多模光纤**，全双工

■ 100Base-T4 和 100Base-TX 统称 100Base-T



快速以太网（续）

■ 两种类型的HUB

- 共享式 **HUB**，一个冲突域，工作方式与**802.3**相同，**CSMA/CD**，二进制指数后退算法，半双工 ...
- 交换式**HUB**，输入帧被缓存，一个端口构成一个冲突域



千兆以太网

- **Gigabit Ethernet**
- **标准：802.3z**
 - 已合并到**IEEE 802.3**中
- **Gigabit Ethernet** 使用扩展的 **802.3 MAC** 子层接口，通过**GMII**（**Gigabit Media Independent Interface**）与物理层相连



千兆以太网（续）

- 物理层实体
 - **1000BASE-LX, 1000BASE-SX, 1000BASE-CX, 1000BASE-T**
- **1000BASE-X**（包括**1000BASE-SX, 1000BASE-LX, 和 1000BASE-CX**）物理层标准符合 **ANSI X3.230-1994 (Fibre Channel) FC-0 和 FC-1**，采用**8B/10B**编码
- **1000BASE-T** 采用**4B/5B**编码
- 在一个冲突域内，只允许一个**repeater**
- 帧格式

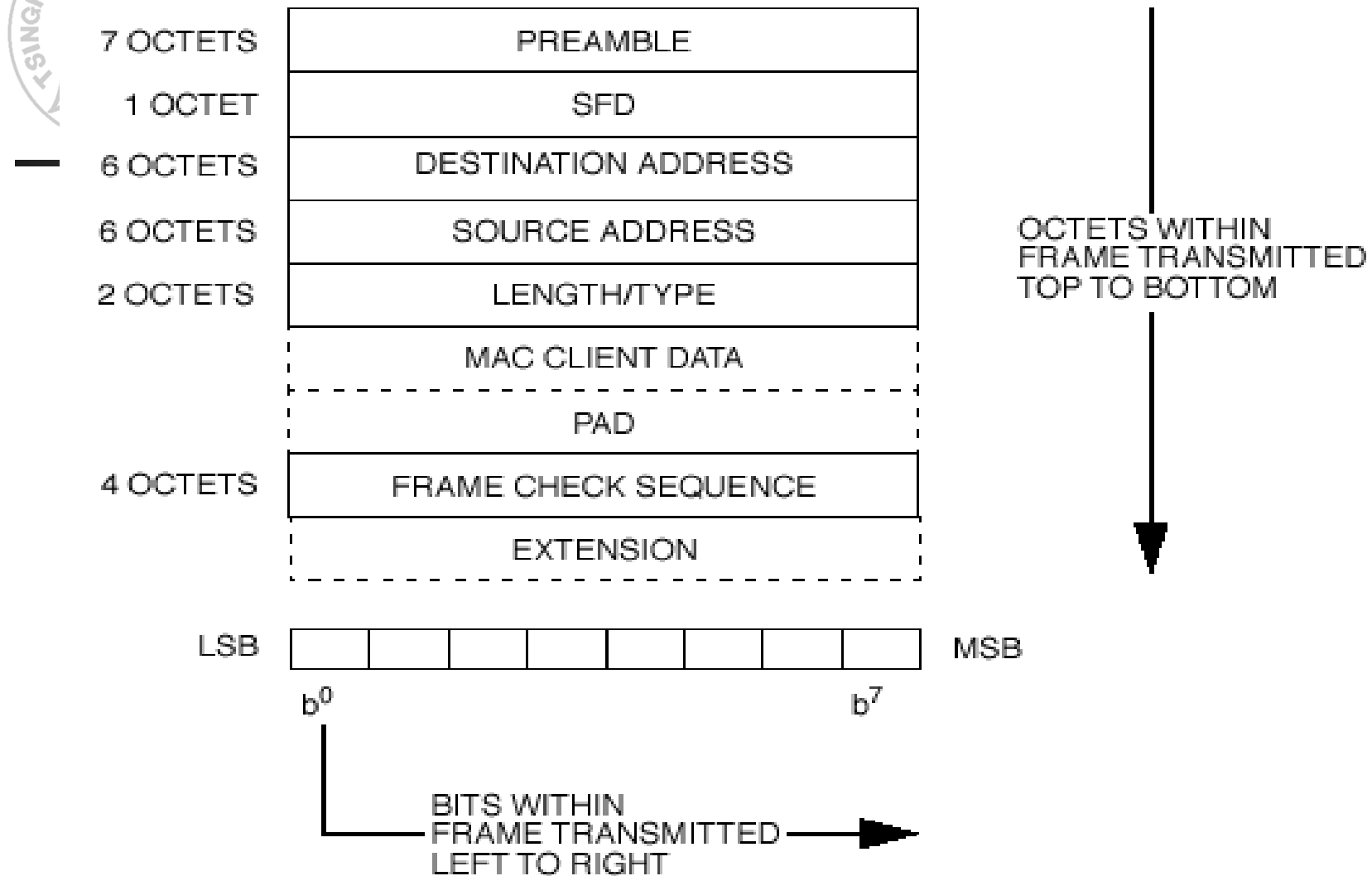


Figure 3-1 – MAC frame format



1000BASE-SX Short Wave Length Optical	Duplex multimode fibers	Clause 38
1000BASE-LX Long Wave Length Optical	Duplex single-mode fibers or Duplex multimode fibers	Clause 38
1000BASE-CX Shielded Jumper Cable	Two pairs of specialized balanced cabling	Clause 39
1000BASE-T Category 5 UTP	Advanced multilevel signaling over four pairs of Category 5 balanced copper cabling.	Clause 40 (under development in IEEE P802.3ab)



万兆以太网

- **10 Gigabit Ethernet / 10GE**
- 标准: **IEEE 802.3ae**, 2002年6月完成
- **10GE**帧格式与**10/100/1000M**以太网的帧格式完全相同
- **10GE**不再使用铜线而只使用光纤作为传输介质
 - 使用单模光纤, 可以工作在广域网和城域网
 - 也可使用多模光纤, 但传输距离为**65~300m**
- **10GE**只工作在全双工方式, 不使用**CSMA/CD**协议, 传输距离大大提高



万兆以太网（续）

- **10GE**有两种不同的物理层
 - 局域网物理层**LAN PHY**, **10Gb/s**
 - 可选的广域网物理层**WAN PHY**, 使用**SONET/SDH**传输, **OC-192/STM-64**的速率是**9.95328Gb/s**, 有效载荷的速率是**9.58464Gb/s**



以太网与Metcalfe

- **1972 – 1975年**（通常认为**1973年**），**施乐公司（Xerox）的Robert Metcalfe**等人发明了**Ethernet**
- **1979年**，**Metcalfe**离开施乐，成立了**3Com**（**Computer, Communication, Compatibility**）
- **3Com**一度是**Cisco**的强大竞争对手
- **2010.4**，**3Com**为惠普收购，退出市场
- **Metcalfe**
 - 以太网发明人
 - 创建**3Com**公司
 - 提出**Metcalfe's Law** ($V \sim N^2$)



3Com



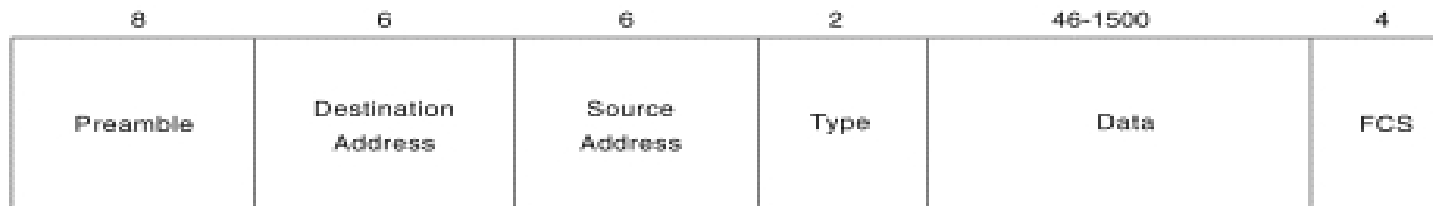
IEEE 802.3 与 Ethernet 的区别

■ 帧格式

- **IEEE 802.3 MAC**地址长度为**2或6**个字节
- **Ethernet MAC**地址长度为**6**个字节

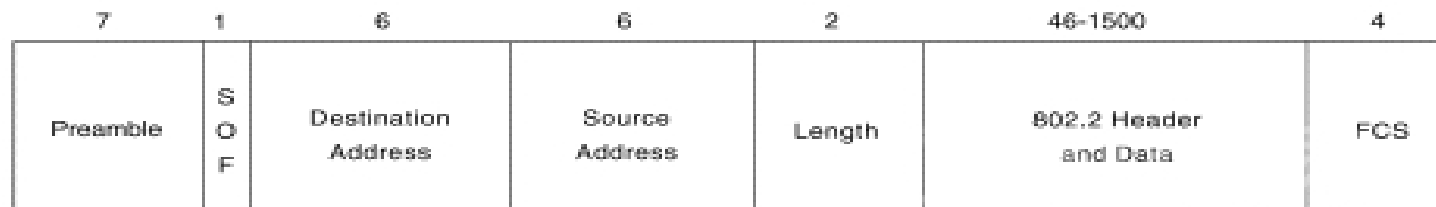
Field Length,
in Bytes

Ethernet



Field Length,
in Bytes

IEEE 802.3



SOF = Start-of-Frame Delimiter
FCS = Frame Check Sequence



IEEE 802.5 令牌环

- **Token Ring**
- **1980年代，IBM**开发出令牌环作为它的**LAN**技术
- **IEEE 802.5**标准是主要基于**IBM**的令牌环网络的，但是也有一些细微的差别
- 令牌环技术提出时声称理论上比**Ethernet**好，但是随着交换以太网的出现和以太网速率不断提高，令牌环技术落后于以太网
- 技术特点
 - 环实际上并不是一个广播介质，而是不同的点到点链路组成的环，点到点链路有很多技术优势
 - 各个站点是公平的，获得信道的时间有上限，避免冲突发生



IEEE 802.5 令牌环 (续)

■ 基本思想

- 令牌 (**Token**) 是一种特殊的比特组合模式，一个站要发送帧时，需要抓住令牌，并将其移出环
- 环本身必须有足够的时延容纳一个完整的令牌，时延由两部分组成：每站的1比特延迟和信号传播延迟。对于短环，必要时需要插入人工延迟
- 环接口有两种操作模式：监听模式和传输模式

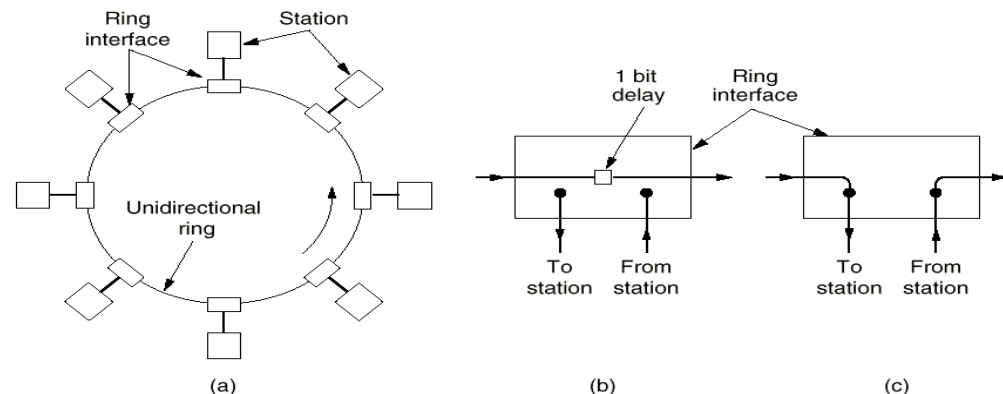


Fig. 4-28. (a) A ring network. (b) Listen mode. (c) Transmit mode.



IEEE 802.5 令牌环（续）

- 当一个站点有数据发送时，在令牌通过此站点时，将令牌从环上取下，发送自己的数据，发送站负责将发出的帧从环上移去，然后重新生成令牌，并转入监听模式
- 确认：帧内一个比特域，初值为**0**，目的站收到后，将其变为**1**；对广播的确认比较复杂
- 重负载下，效率接近**100%**
- 环网设计分析的一个主要问题是 **1** 比特的“物理长度”，数据传输速率为 **R Mbps**，典型信号传播速率为**200米/微秒**，则**1** 比特的“物理长度”为 **200/R米**
- 环接口引入了**1**比特的传输延迟



IEEE 802.5 令牌环（续）

■ 802.5的布线

- 屏蔽双绞线，速率为**1/4/16M**，采用差分曼彻斯特编码传输
- 为解决环断裂导致整个环无法工作的问题，使用线路中心（**Wire Center**）进行布线，线路中心设有旁路中继器

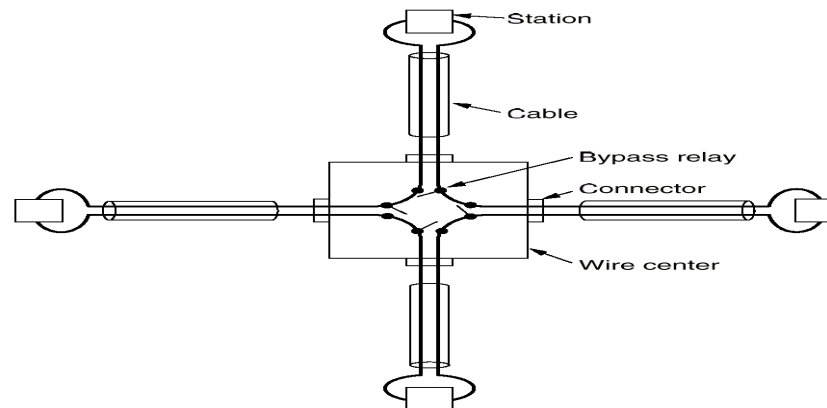
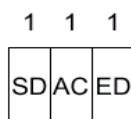


Fig. 4-29. Four stations connected via a wire center.

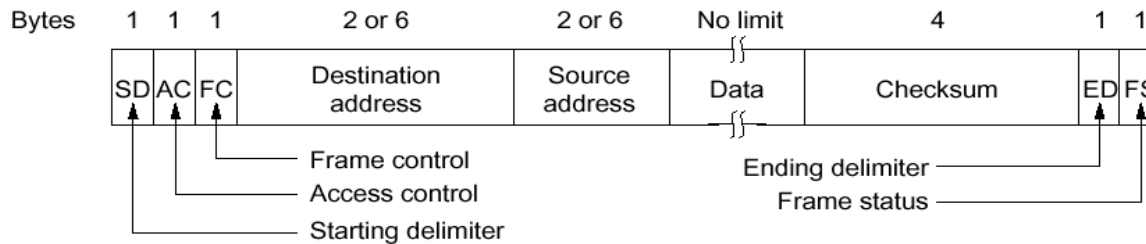


IEEE 802.5 令牌环 (续)

■ 令牌环MAC子层协议



(a)



(b)

- 协议基本操作：无信息传输时，**3**字节的令牌在环上循环；有信息要发送时，站获得令牌，并将第二个字节的某一位由 **0** 变成 **1**，将令牌的前两个字节变成帧的起始序列，然后输出帧的其它部分



IEEE 802.5 令牌环 (续)

- 开始定界符**SD**和结束定界符**ED**标志着帧的开始和结束,使用差分曼彻斯特编码模式(**HH**和**LL**, 物理层编码违例法)
- 访问控制域 **AC** 包括令牌位、监视位、优先级位和保留位
- 帧控制域 **FC** 用于将数据帧和控制帧区别开来和进行环的维护; 帧状态字节**FS**用于报告帧的传送情况, 包括地址位**A**和拷贝位**C**, 帧经过目的站, **A**置为“**1**”, 帧被接收, **C**置为“**1**”。**A**、**C**位提供了自动确认。为增加可靠性, **A**、**C**在 **FS**中出现两次
 - **A = 0, C = 0**, 目的站不存在或未加电
 - **A = 1, C = 0**, 目的站存在但帧未被接收
 - **A = 1, C = 1**, 目的站存在且帧被复制
- 令牌持有时间 (**token-holding time**), 一般为**10**毫秒
- 提供优先级控制: 访问控制域中的优先级位给出令牌的优先级, 只有当要发送的帧的优先级大于等于令牌的优先级时才能获得令牌, 站还可以预约某个优先级的令牌



IEEE 802.5 令牌环（续）

■ 环的维护

- 环上存在一个监控站，负责环的维护，通过站的竞争产生
- 监控站的职责
 - 保证令牌不丢失
 - 处理环断开情况
 - 清除坏帧，检查无主帧



光纤分布式数据接口

■ FDDI (Fiber Distributed Data Interface)

■ 特征

- 使用多模光纤作为传输介质
- MAC协议与 Token Ring 类似
- 100M的速率
- 采用4B5B编码方法
 - 32中组合中的16种表示数据，3种表示定界符，2种表示控制，3种表示硬件信号，8种保留
- 最大距离200公里
- 最多1000个站点

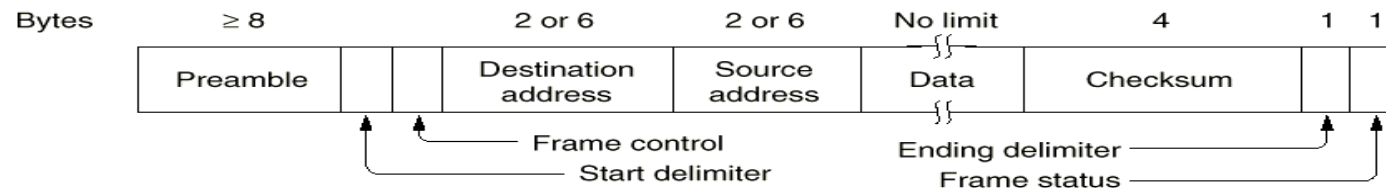


Fig. 4-46. FDDI frame format.



光纤分布式数据接口（续）

- 通常作为连接**LAN**的主干网络
- **FDDI**的双环操作
- **FDDI**定义了两类站：**A**类站连接双环，**B**类站连接单环。
- 为提高信道利用率，站点发完数据后立即产生新令牌，环上可能同时存在多个帧

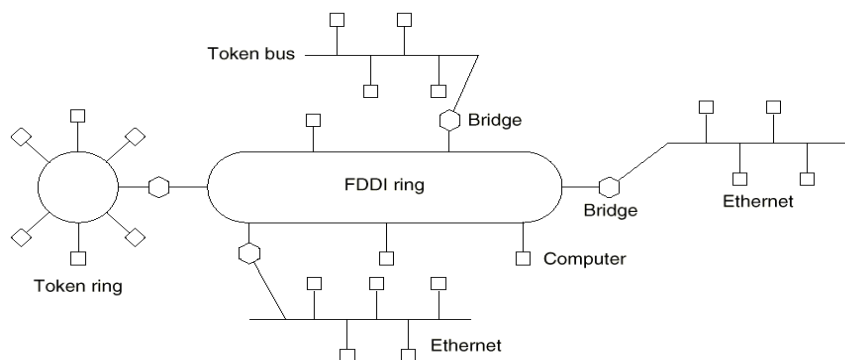


Fig. 4-44. An FDDI ring being used as a backbone to connect LANs and computers.

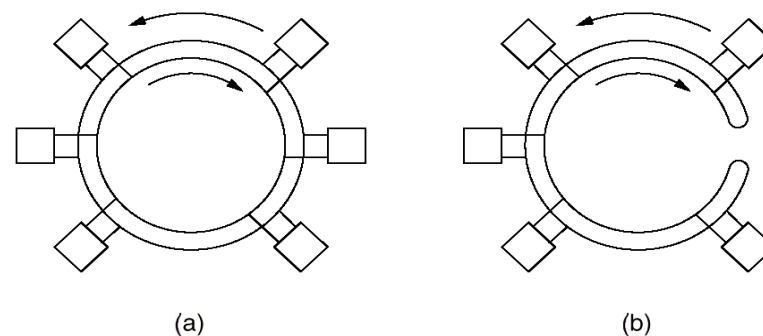


Fig. 4-45. (a) FDDI consists of two counterrotating rings. (b) In the event of failure of both rings at one point, the two rings can be joined together to form a single long ring.



- **FDDI的hub**

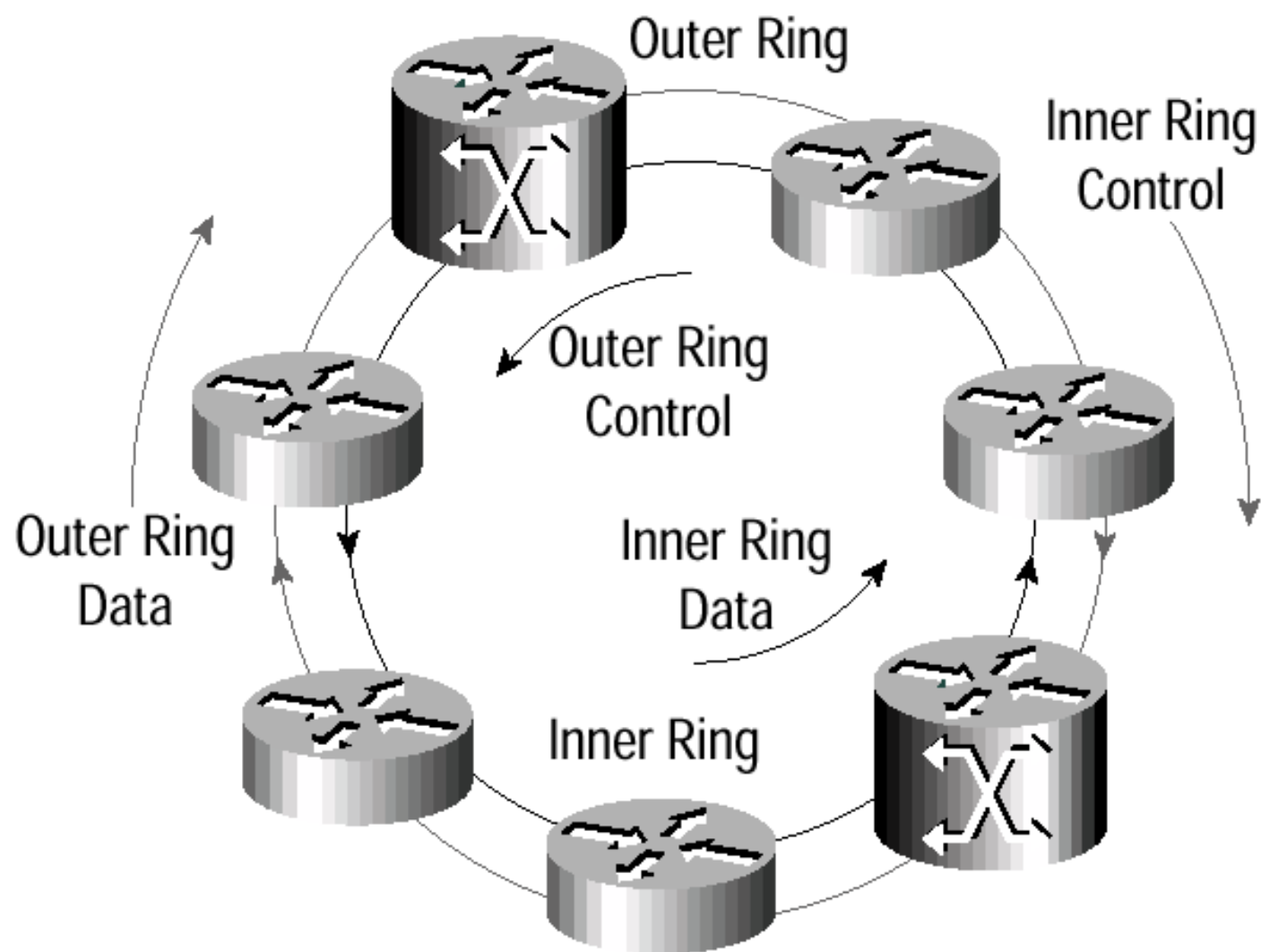


DPT/RPR

- **Dynamic Packet Transport**
- **Cisco** 的技术，主要用于城域网
- **OC-12 (622 Mbps), OC-48 (2.488 Gbps)**
- 结合了**IP**带宽利用率高、服务种类丰富的特点和光纤环高带宽、自愈的特点
- **DPT**环是双环，每个环都同时用于用户数据和控制数据的传输
- **Spatial Reuse Protocol (SRP)**
 - **SRP**是一个媒介无关的**MAC**层协议，用来实现**DPT**在光纤环情况上的功能
 - **SRP**提供基本的寻址，报文封装，带宽控制和控制信息的传输机制



—



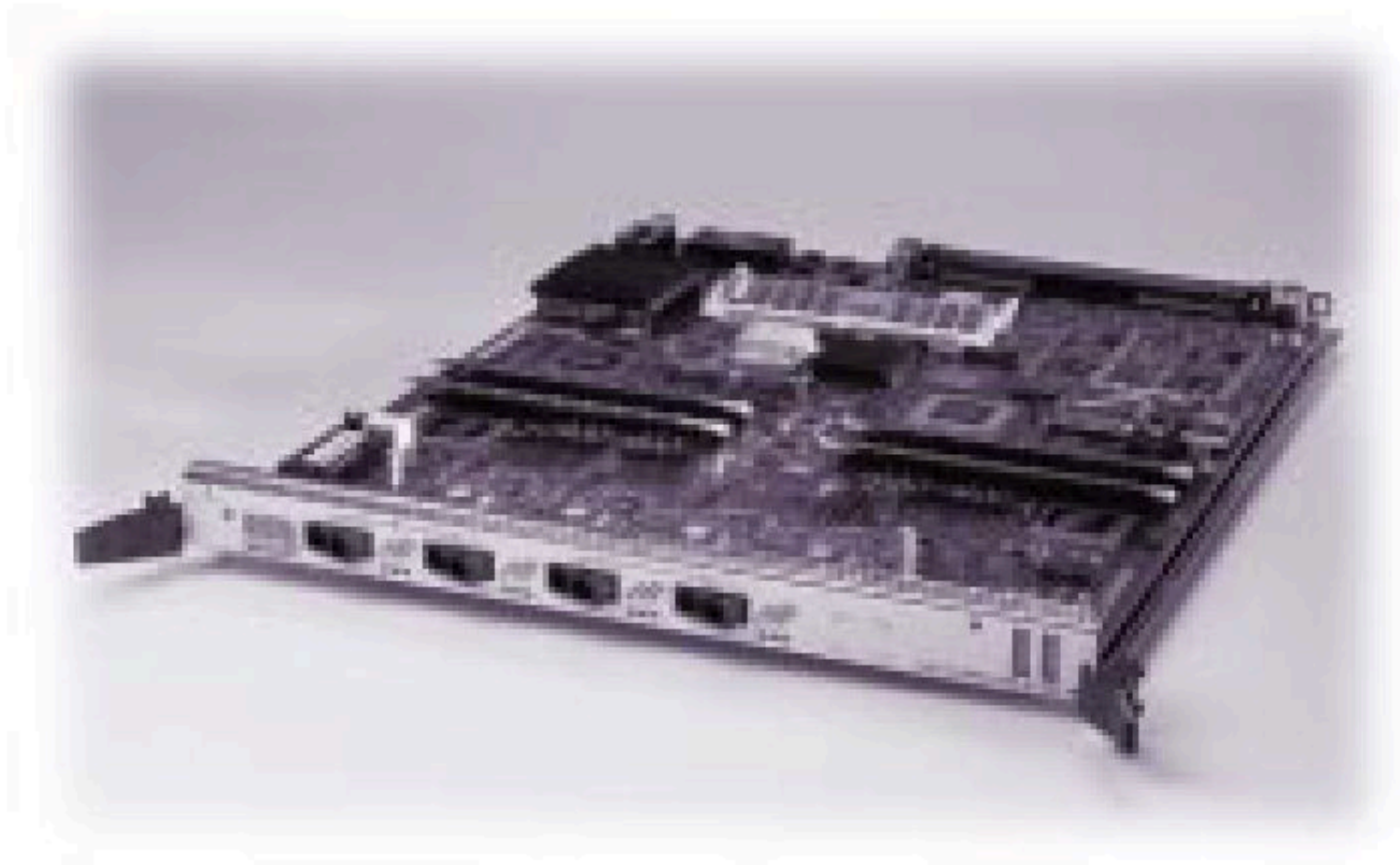


DPT (续)

- 目的地提取报文: 报文被目的节点从环上取下, 不继续占用带宽。这样**DPT**环可以提供空间复用, 使得多个不同网段可以同时全速使用带宽
- **DPT**结合了 **SONET/SDH**的处理能力和第二层的管理能力, 来实现多层性能监视, 错误检查和错误隔离功能
- **Reference**
 - [Http://www.cisco.com/](http://www.cisco.com/), 搜索关键词 “**DPT**”



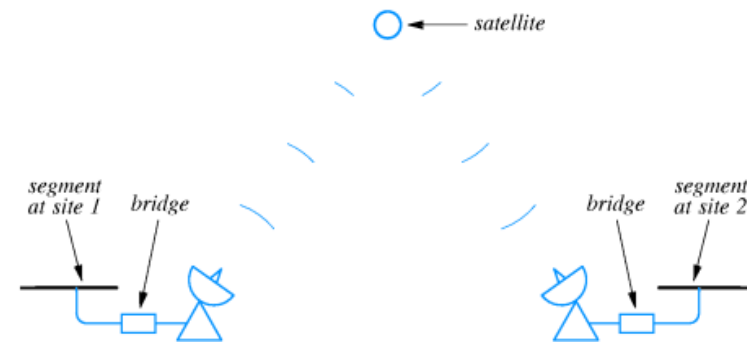
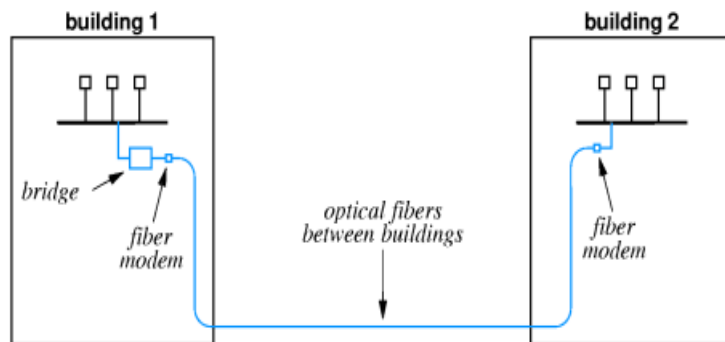
Figure 2 Cisco DPT Line Card





网桥技术

- 定义：网桥（**bridge**）是工作在数据链路层的一种网络互连设备，它在互连的**LAN**之间实现帧的存储和转发
- 为什么使用网桥？
 - 两个**LAN**之间的距离超过**2500**米，一个比较经济的方案是使用桥将它们连接起来





网桥技术（续）

- 将一个负载很重的大**LAN**分隔成使用网桥互连的几个**LAN**以减轻负担，防止出故障的站点损害全网
 - 冲突域：在使用**CSMA/CD**协议的以太网中，如果两个站点同时发送帧会产生冲突，则这个**CSMA/CD**网络就是一个冲突域
 - 中继器不能隔离冲突域，网桥/交换机可以隔离冲突域

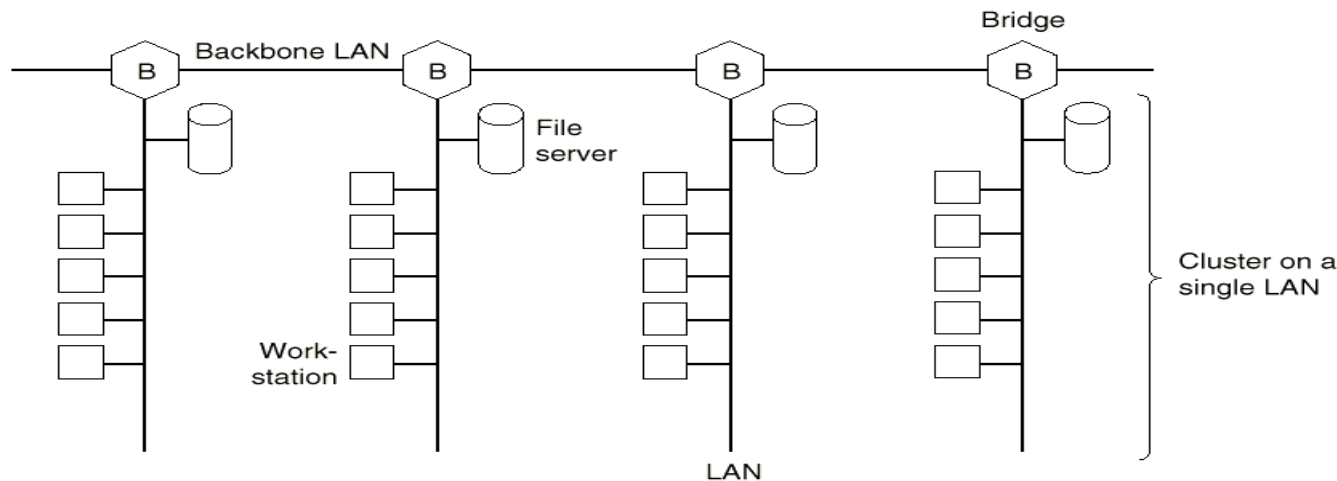


Fig. 4-34. Multiple LANs connected by a backbone to handle a total load higher than the capacity of a single LAN.



网桥技术 (续)

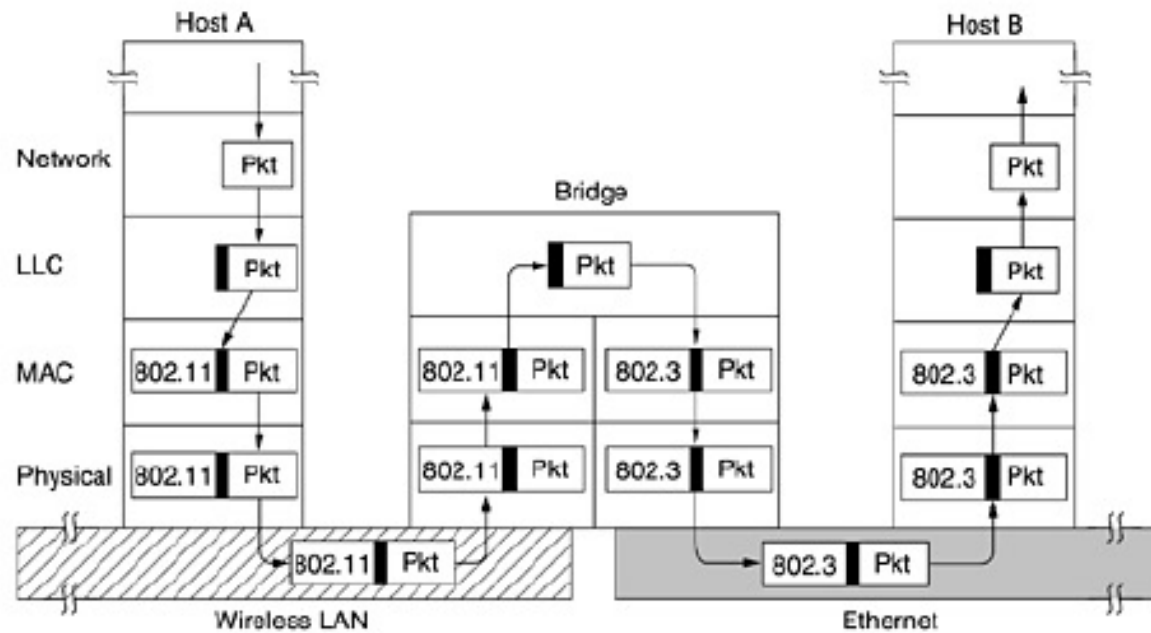
- 网桥可以互连不同类型的**LAN**
- 网桥可以有助于安全保密



网桥技术（续）

■ 网桥的工作原理

- 连接**k**个不同**LAN**的网桥具有**k**个**MAC**子层和**k**个物理层





连接 802.X 和 802.Y 的网桥

- 互连时需要解决的不同问题
 - 不同**LAN**帧格式的转换
 - 不同的**LAN**速率不同，网桥要有缓存能力
 - 高层协议的计时器设置
 - 不同的**LAN**支持的最大帧长度不同，分别为**1500**，**8191**，**5000**。解决办法：丢弃无法转发的帧

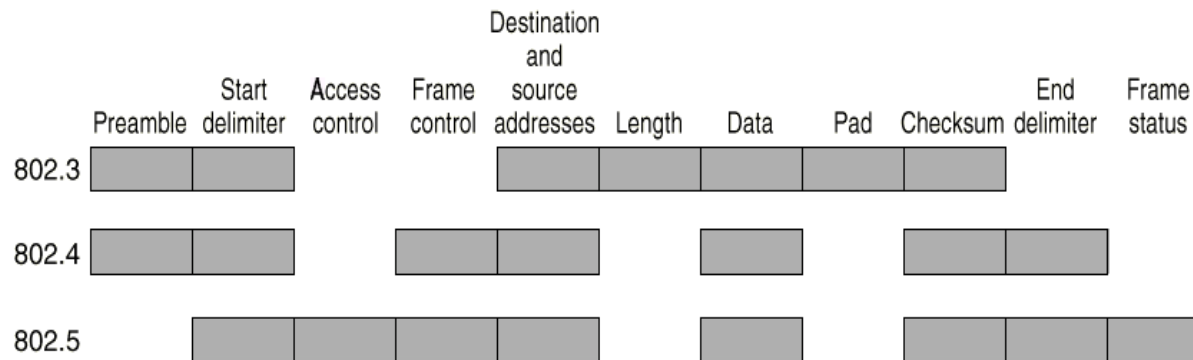


Fig. 4-36. The IEEE 802 frame formats.



连接 802.X 和 802.Y 的网桥 (续)

- 三种不同的**LAN**互连共有九种组合
 - 操作
 - 格式转换和重新计算校验和
 - 变换位的顺序
 - 复制优先级
 - 产生一个虚拟的优先级
 - 放弃优先级
 - 把环排空
 - 设置**A/C**位
 - 解决速率快慢问题
 - 处理帧太长的问題



Source
LAN

	Destination LAN		
	802.3 (CSMA/CD)	802.4 (Token bus)	802.5 (Token ring)
802.3		1, 4	1, 2, 4, 8
802.4	1, 5, 8, 9, 10	9	1, 2, 3, 8, 9, 10
802.5	1, 2, 5, 6, 7, 10	1, 2, 3, 6, 7	6, 7

Actions:

1. Reformat the frame and compute new checksum
2. Reverse the bit order.
3. Copy the priority, meaningful or not.
4. Generate a fictitious priority.
5. Discard priority.
6. Drain the ring (somehow).
7. Set A and C bits (by lying).
8. Worry about congestion (fast LAN to slow LAN).
9. Worry about token handoff ACK being delayed or impossible.
10. Panic if frame is too long for destination LAN.

Parameters assumed:

802.3:	1500-byte frames,	10 Mbps (minus collisions)
802.4:	8191-byte frames	10 Mbps
802.5:	5000-byte frames	4 Mbps

Fig. 4-37. Problems encountered in building bridges from 802.x to 802.y.



透明网桥/生成树网桥

■ 工作原理

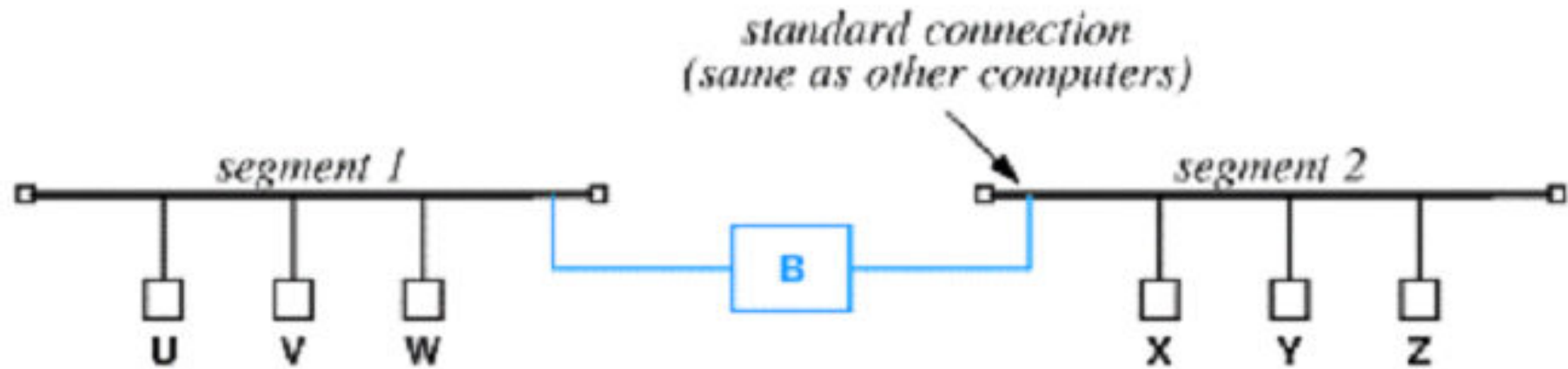
- 网桥工作在混杂（**promiscuous**）方式，接收所有的帧
- 网桥接收到一帧后，通过查询地址/端口对应表来确定是丢弃还是转发
- 网桥刚启动时，地址/端口对应表为空，采用洪泛（**flooding**）方法转发帧
- 在转发过程中采用逆向学习(**backward learning**)算法收集**MAC**地址。网桥通过分析帧的源**MAC**地址得到**MAC**地址与端口的对应关系,并写入地址/端口对应表
- 网桥软件对地址/端口对应表进行不断的更新，并定时检查，删除在一段时间内没有更新的地址/端口项



透明网桥/生成树网桥（续）

■ 帧的路由过程

- 目的**LAN**与源**LAN**相同，则丢弃帧
- 目的**LAN**与源**LAN**不同，则转发帧
- 目的**LAN**未知，则洪泛帧



Event	Segment 1 List	Segment 2 List
Bridge boots	—	—
U sends to V	U	—
V sends to U	U, V	—
Z broadcasts	U, V	Z
Y sends to V	U, V	Z, Y
Y sends to X	U, V	Z, Y
X sends to W	U, V	Z, Y, X
W sends to Z	U, V, W	Z, Y, X



透明网桥/生成树网桥（续）

- 多个网桥（并行网桥）可能产生回路

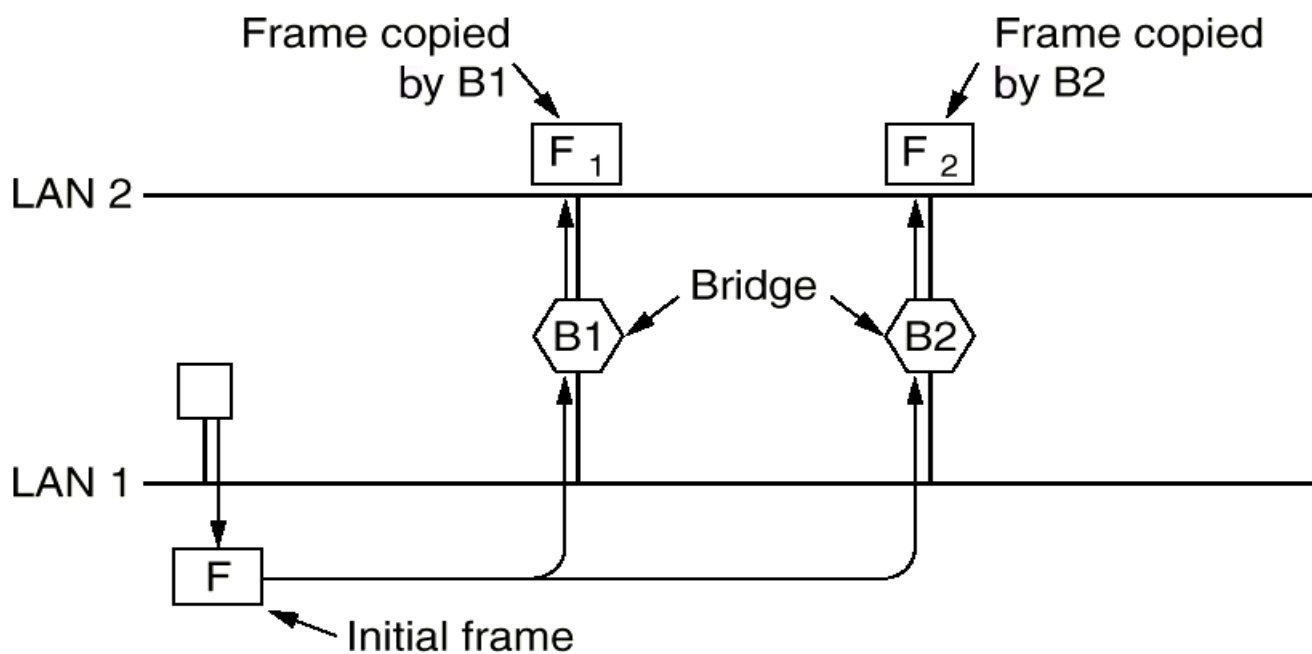
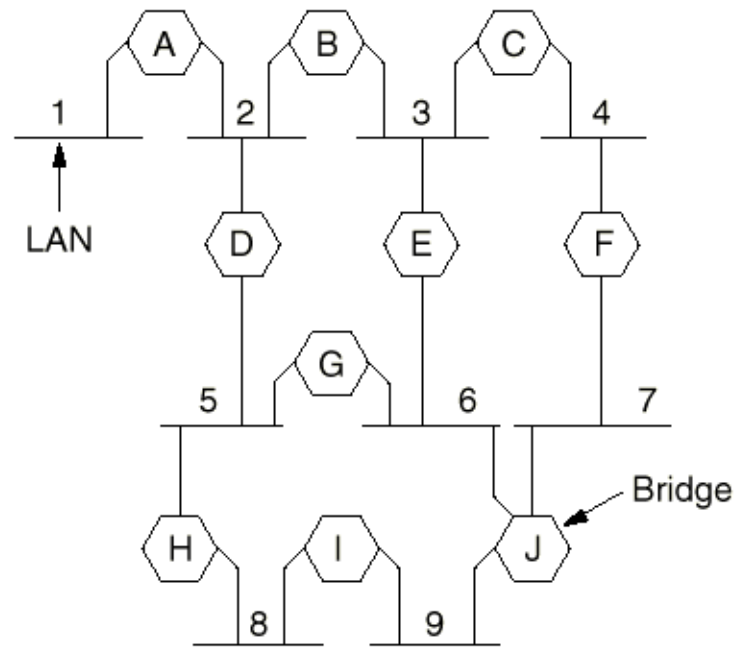


Fig. 4-39. Two parallel transparent bridges.

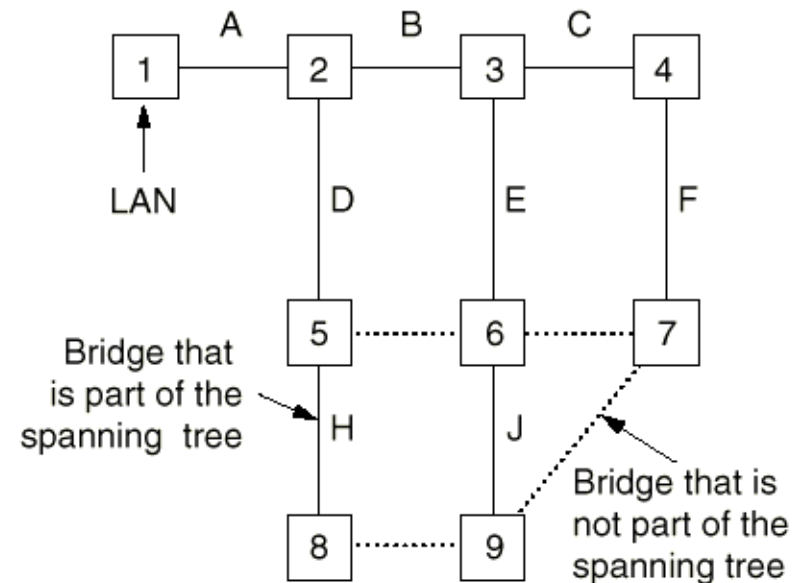


透明网桥/生成树网桥（续）

- 解决多个网桥产生回路的问题
 - 思想
 - 让网桥之间互相通信，用一棵连接每个**LAN**的生成树（**Spanning Tree**）覆盖实际的拓扑结构
 - 构造生成树
 - 每个桥广播自己的桥编号，号最小的桥称为生成树的根
 - 每个网桥计算自己到根的最短路径，构造出生成树，使得每个**LAN**和桥到根的路径最短
 - 当某个**LAN**或网桥发生故障时，要重新计算生成树
 - 生成树构造完后，算法继续执行以便自动发现拓扑结构变化，更新生成树



(a)



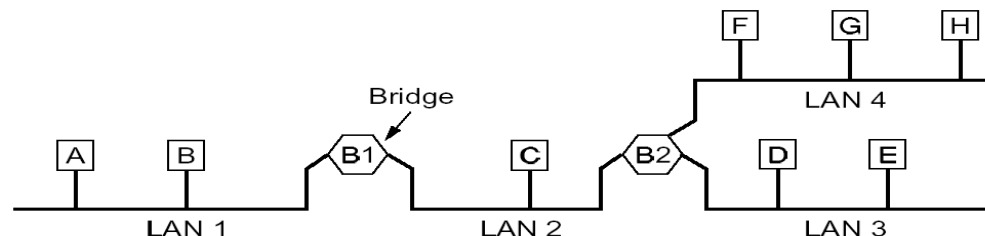
(b)

Fig. 4-40. (a) Interconnected LANs. (b) A spanning tree covering the LANs. The dotted lines are not part of the spanning tree.



源路由网桥

- **CSMA/CD和Token Bus**选择了透明网桥，**Token Ring**选择了源路由网桥
- 源路由网桥的原理
 - 帧的发送者知道目的主机是否在自己的**LAN**内
 - 如果不在，在发出的帧头内构造一个准确的路由序列，包含要经过的网桥、**LAN**的编号。并将发出的帧的源地址的最高位置**1**
 - 例：图中**A**到**D**的路由为:(**L1, B1, L2, B2, L3**)





源路由网桥（续）

- 每个**LAN**有一个**12**位的编号，每个网桥有一个**4**位的编号
- 网桥只接收源地址的最高位为**1**的帧，判定是转发还是丢弃
- 源路由的产生：每个站点通过广播“发现帧” (**discovery frame**) 来获得各个站点的最佳路由
 - 若目的地址未知，源站发送“发现帧”，每个网桥收到后广播，目的站收到后发应答帧，该帧经过网桥时被加上网桥的标识，源站收到后就知道了到目的站的最佳路由
- 优点
 - 对带宽进行最优的使用
- 缺点
 - 网桥的插入对于网络是不透明的，需要人工干预



网桥的比较

Issue	Transparent bridge	Source routing bridge
Orientation	Connectionless	Connection-oriented
Transparency	Fully transparent	Not transparent
Configuration	Automatic	Manual
Routing	Suboptimal	Optimal
Locating	Backward learning	Discovery frames
Failures	Handled by the bridges	Handled by the hosts
Complexity	In the bridges	In the hosts

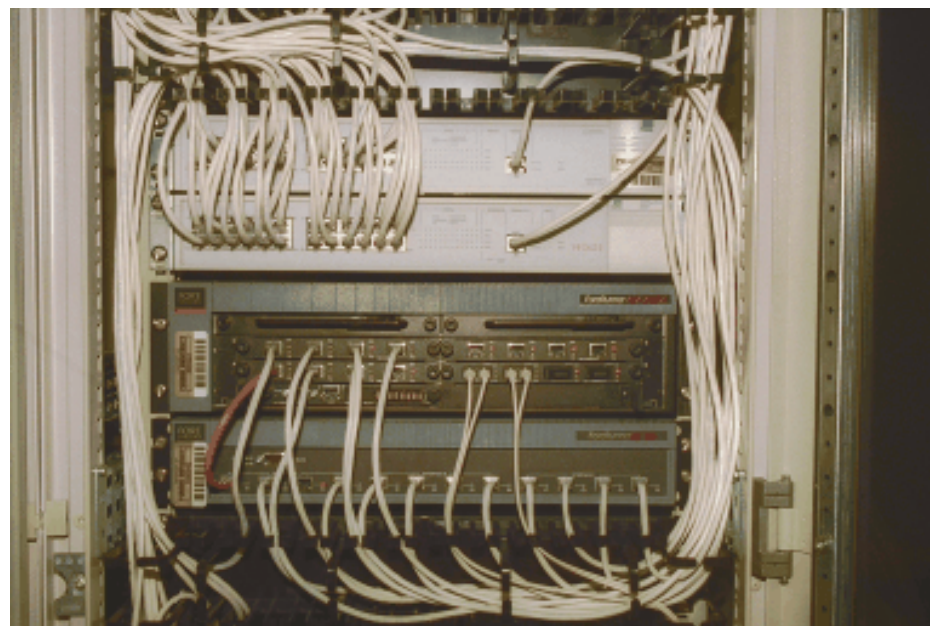
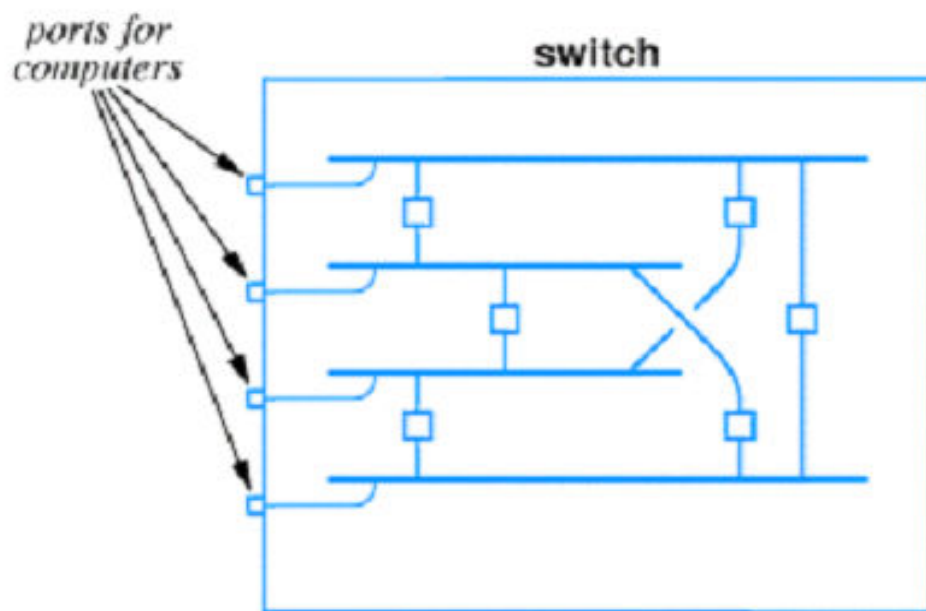
Fig. 4-42. Comparison of transparent and source routing bridges.



- 无线网桥，用于连接无线局域网和一般的局域网



交换机



- 交换机内的电路让每个计算机位于单独的局域网网段上并与其他网段通过网桥连接

1.4.1 10BASE2: IEEE 802.3 Physical Layer specification for a 10 Mb/s CSMA/CD local area network over RG 58 coaxial cable. (See IEEE 802.3 Clause 10.)

1.4.2 10BASE5: IEEE 802.3 Physical Layer specification for a 10 Mb/s CSMA/CD local area network over coaxial cable (i.e., thicknet). (See IEEE 802.3 Clause 8.)

1.4.3 10BASE-F: IEEE 802.3 Physical Layer specification for a 10 Mb/s CSMA/CD local area network over fiber optic cable. (See IEEE 802.3 Clause 15.)

1.4.9 10BASE-T: IEEE 802.3 Physical Layer specification for a 10 Mb/s CSMA/CD local area network over two pairs of twisted-pair telephone wire. (See IEEE 802.3 Clause 14.)

1.4.10 100BASE-FX: IEEE 802.3 Physical Layer specification for a 100 Mb/s CSMA/CD local area network over two optical fibers. (See IEEE 802.3 Clauses 24 and 26.)

1.4.11 100BASE-T: IEEE 802.3 Physical Layer specification for a 100 Mb/s CSMA/CD local area network. (See IEEE 802.3 Clauses 22 and 28.)

1.4.12 100BASE-T2: IEEE 802.3 specification for a 100 Mb/s CSMA/CD local area network over two pairs of Category 3 or better balanced cabling. (See IEEE 802.3 Clause 32.)

1.4.13 100BASE-T4: IEEE 802.3 Physical Layer specification for a 100 Mb/s CSMA/CD local area network over four pairs of Category 3, 4, and 5 unshielded twisted-pair (UTP) wire. (See IEEE 802.3 Clause 23.)

1.4.14 100BASE-TX: IEEE 802.3 Physical Layer specification for a 100 Mb/s CSMA/CD local area network over two pairs of Category 5 unshielded twisted-pair (UTP) or shielded twisted-pair (STP) wire. (See IEEE 802.3 Clauses 24 and 25.)



1.4.15 100BASE-X: IEEE 802.3 Physical Layer specification for a 100 Mb/s CSMA/CD local area network that uses the Physical Medium Dependent (PMD) sublayer and Medium Dependent Interface (MDI) of the ISO/IEC 9314 group of standards developed by ASC X3T12 (FDDI). (See IEEE 802.3 Clause 24.)

1.4.16 1000BASE-CX: 1000BASE-X over specialty shielded balanced copper jumper cable assemblies. (See IEEE 802.3 Clause 39.)

1.4.17 1000BASE-LX: 1000BASE-X using long wavelength laser devices over multimode and single-mode fiber. (See IEEE 802.3 Clause 38.)

1.4.18 1000BASE-SX: 1000BASE-X using short wavelength laser devices over multimode fiber. (See IEEE 802.3 Clause 38.)

1.4.19 1000BASE-T: IEEE 802.3 Physical Layer specification for a 1000 Mb/s CSMA/CD LAN using four pairs of Category 5 balanced copper cabling. (See IEEE 802.3 Clause 40.)

1.4.20 1000BASE-X: IEEE 802.3 Physical Layer specification for a 1000 Mb/s CSMA/CD LAN that uses a Physical Layer derived from ANSI X3.230-1994 (FC-PH) [B19]⁹. (See IEEE 802.3 Clause 36.)

1.4.46 bridge: A layer 2 interconnection device that does not form part of a CSMA/CD collision domain but conforms to the ISO/IEC 15802-3: 1998 [ANSI/IEEE 802.1D, 1998 Edition] International Standard. A bridge does not form part of a CSMA/CD collision domain but, rather appears as a Media Access Control (MAC) to the collision domain. (See also IEEE Std 100-1996.)



1.4.50 carrier sense: In a local area network, an ongoing activity of a data station to detect whether another station is transmitting. *Note*—The carrier sense signal indicates that one or more DTEs are currently transmitting.

1.4.74 collision: A condition that results from concurrent transmissions from multiple data terminal equipment (DTE) sources within a single collision domain.

1.4.75 collision domain: A single, half duplex mode CSMA/CD network. If two or more Media Access Control (MAC) sublayers are within the same collision domain and both transmit at the same time, a collision will occur. MAC sublayers separated by a repeater are in the same collision domain. MAC sublayers separated by a bridge are within different collision domains. (See IEEE 802.3.)

1.4.120 Fibre Distributed Data Interface (FDDI): A 100 Mb/s, fiber optic-based, token-ring local area network standard (ISO/IEC 9314, formerly X3.237-1995).

1.4.124 full duplex: A mode of operation of a network, DTE, or Medium Attachment Unit (MAU) that supports duplex transmission as defined in IEEE Std 100-1996. Within the scope of this standard, this mode of operation allows for simultaneous communication between a pair of stations, provided that the Physical Layer is capable of supporting simultaneous transmission and reception without interference. (See IEEE 802.3.)

1.4.128 half duplex: A mode of operation of a CSMA/CD local area network (LAN) in which DTEs contend for access to a shared medium. Multiple, simultaneous transmissions in a half duplex mode CSMA/CD LAN result in interference, requiring resolution by the CSMA/CD access control protocol. (See IEEE 802.3.)



1.4.130 header hub (HH): The highest-level hub in a hierarchy of hubs. The HH broadcasts signals transmitted to it by lower-level hubs or DTEs such that they can be received by all DTEs that may be connected to it either directly or through intermediate hubs. (See IEEE 802.3, 12.2.1 for details.)

1.4.131 hub: A device used to provide connectivity between DTEs. Hubs perform the basic functions of restoring signal amplitude and timing, collision detection, and notification and signal broadcast to lower-level hubs and DTEs. (See IEEE 802.3 Clause 12.)

1.4.133 in-band signaling: The transmission of a signal using a frequency that is within the bandwidth of the information channel. *Contrast with:* **out-of-band signaling**. *Syn:* **in-channel signaling**. (From IEEE Std 610.7-1995 [B25].)

1.4.134 intermediate hub (IH): A hub that occupies any level below the header hub in a hierarchy of hubs. (See IEEE 802.3, 12.2.1 for details.)

1.4.135 Inter-Packet Gap (IPG): A delay or time gap between CSMA/CD packets intended to provide interframe recovery time for other CSMA/CD sublayers and for the Physical Medium. (See IEEE 802.3,

4.2.3.2.1 and 4.2.3.2.2.) For example, for 10BASE-T, the IPG is 9.6 μ s (96 bit times); for 100BASE-T, the IPG is 0.96 μ s (96 bit times).

1.4.227 segment: The medium connection, including connectors, between Medium Dependent Interfaces (MDIs) in a CSMA/CD local area network.

1.4.246 switch: A layer 2 interconnection device that conforms to the ISO/IEC 10038 [ANSI/IEEE 802.1D-1990] International Standard. *Syn:* **bridge**.



1.4.154 Media Access Control (MAC): The data link sublayer that is responsible for transferring data to and from the Physical Layer.

1.4.155 Media Independent Interface (MII): A transparent signal interface at the bottom of the Reconciliation sublayer. (See IEEE 802.3 Clause 22.)

1.4.156 Medium Attachment Unit (MAU): A device containing an Attachment Unit Interface (AUI), Physical Medium Attachment (PMA), and Medium Dependent Interface (MDI) that is used to connect a repeater or data terminal equipment (DTE) to a transmission medium.

1.4.157 Medium Dependent Interface (MDI): The mechanical and electrical interface between the transmission medium and the Medium Attachment Unit (MAU) (10BASE-T) or PHY (100BASE-T).

1.4.215 repeater: Within IEEE 802.3, a device as specified in Clauses 9 and 27 that is used to extend the length, topology, or interconnectivity of the physical medium beyond that imposed by a single segment, up to the maximum allowable end-to-end transmission line length. Repeaters perform the basic actions of

restoring signal amplitude, waveform, and timing applied to the normal data and collision signals. For wired star topologies, repeaters provide a data distribution function. In 100BASE-T, a device that allows the interconnection of 100BASE-T Physical Layer (PHY) network segments using similar or dissimilar PHY implementations (e.g., 100BASE-X to 100BASE-X, 100BASE-X to 100BASE-T4, etc.). Repeaters are only for use in half duplex mode networks. (See IEEE 802.3, Clauses 9 and 27.)



1.4.256 twisted pair: A cable element that consists of two insulated conductors twisted together in a regular fashion to form a balanced transmission line. (From ISO/IEC 11801: 1995.)

1.4.257 twisted-pair cable: A bundle of multiple twisted pairs within a single protective sheath. (From ISO/IEC 11801: 1995.)

1.4.232 shielded twisted-pair (STP) cable: An electrically conducting cable, comprising one or more elements, each of which is individually shielded. There may be an overall shield, in which case the cable is referred to as shielded twisted-pair cable with an overall shield (from ISO/IEC 11801: 1995). Specifically for IEEE 802.3 100BASE-TX, 150 Ω balanced inside cable with performance characteristics specified to 100 MHz (i.e., performance to Class D link standards as per ISO/IEC 11801: 1995). In addition to the requirements specified in ISO/IEC 11801: 1995, IEEE 802.3 Clauses 23 and 25 provide additional performance requirements for 100BASE-T operation over STP.

1.4.262 unshielded twisted-pair cable (UTP): An electrically conducting cable, comprising one or more pairs, none of which is shielded. There may be an overall shield, in which case the cable is referred to as unshielded twisted-pair with overall shield. (From ISO/IEC 11801: 1995.)



第七章

路由选择和网络层



主要内容

- 网络层概述
- 路由算法
 - 最优化原则
 - 最短路径路由算法
 - 洪泛算法
 - 基于流量的路由算法
 - 距离向量路由算法
 - 链路状态路由算法
 - 分层路由
 - 移动主机的路由



主要内容 (续)

- 拥塞控制算法
 - 拥塞控制的基本原理
 - 拥塞控制算法
- 网络互连
 - 级联虚电路
 - 无连接网络互连
 - 隧道技术
 - 互联网路由
 - 分片
 - 防火墙



主要内容（续）

- 互联网网络层协议
 - **IP**协议
 - **Internet**控制协议**ICMP**
 - 内部网关路由协议**OSPF**
 - 外部网关路由协议**BGP**
- 路由器体系结构和关键技术



网络层概述

■ ISO 定义

- 网络层为一个网络连接的两个传送实体间交换网络服务数据单元提供功能和规程的方法，它使传送实体独立于路由选择和交换的方式

■ 网络层的地位

- 位于数据链路层和传输层之间，使用数据链路层提供的服务，为传输层提供服务
- 通信子网的最高层（传统意义）

■ 网络层的功能

- 屏蔽各种不同类型网络之间的差异，实现互连
- 了解通信子网的拓扑结构，选择路由，实现报文的网络传输



网络层提供的服务

■ 为传输层提供服务

■ 面向连接服务

- 传统电信的观点：通信子网应该提供可靠的、面向连接的服务
- 将复杂的功能放在网络层(通信子网)

■ 无连接服务

- 互联网的观点：通信子网无论怎么设计都是不可靠的，因此网络层只需提供无连接服务
- 将复杂的功能放在传输层



IP over ATM

Email	FTP	...
TCP		
IP		
ATM		
Data link		
Physical		

Fig. 5-1. Running TCP/IP over an ATM subnet.



数据报子网与虚电路子网

- 网络层的内部结构
 - 数据报 (**datagram**) 子网
 - 采用数据报分组交换
 - 每个分组被独立转发，分组带有全网唯一的地址
 - 虚电路 (**virtual circuit**) 子网
 - 采用虚电路分组交换
 - 先在源结点和目的结点之间建立一条虚电路，所有分组沿虚电路按次序存储转发，最后拆除虚电路



数据报子网与虚电路子网（续）

- 虚电路子网与数据报子网的比较
 - 带宽与状态的权衡
 - 数据报子网中，每个数据报都携带完整的目的/源地址，开销大，浪费带宽
 - 虚电路子网中，路由器需要维护虚电路的状态信息；
 - 地址查找时间与连接建立时间的权衡
 - 数据报子网对每个分组的路由查找过程复杂
 - 虚电路子网需要在建立连接时花费较长的路由查找时间
 - 可靠性与服务质量的权衡
 - 数据报不太容易保证服务质量QoS(Quality of Service)，但是对于通信线路的故障，适应性很强
 - 虚电路方式很容易保证服务质量，适用于实时操作，但比较脆弱
 - 可扩展性
 - 数据报子网具有更好的可扩展性



数据报子网与虚电路子网（续）

Issue	Datagram subnet	VC subnet
Circuit setup	Not needed	Required
Addressing	Each packet contains the full source and destination address	Each packet contains a short VC number
State information	Subnet does not hold state information	Each VC requires subnet table space
Routing	Each packet is routed independently	Route chosen when VC is set up; all packets follow this route
Effect of router failures	None, except for packets lost during the crash	All VCs that passed through the failed router are terminated
Congestion control	Difficult	Easy if enough buffers can be allocated in advance for each VC

Fig. 5-2. Comparison of datagram and virtual circuit subnets.



路由算法

- 路由算法是网络层协议的一部分
 - 通信子网采用数据报分组交换方式，每个分组都要做路由选择
 - 通信子网采用虚电路分组交换方式，只需在建立连接时做一次路由选择
- 路由算法应具有的特性
 - 正确性 (correctness)
 - 简单性 (simplicity)
 - 健壮性 (robustness)
 - 稳定性 (stability)
 - 公平性 (fairness)
 - 最优性 (optimality)
 - 可扩展性 (scalability)



路由算法分类

- 路由算法分类
 - 静态路由算法（非自适应算法）
 - 动态路由算法（自适应算法）
- 最优化原则（**optimality principle**）
 - 如果路由器 **J** 在路由器 **I** 到 **K** 的最优路由上，那么从 **J** 到 **K** 的最优路由会落在同一路由上



汇集树

- 汇集树 (**sink tree**)
 - 从所有的源结点到一个给定的目的结点的最优路由的集合形成了一个以目的结点为根的树，称为汇集树
- 路由算法的目的是找出并使用汇集树。

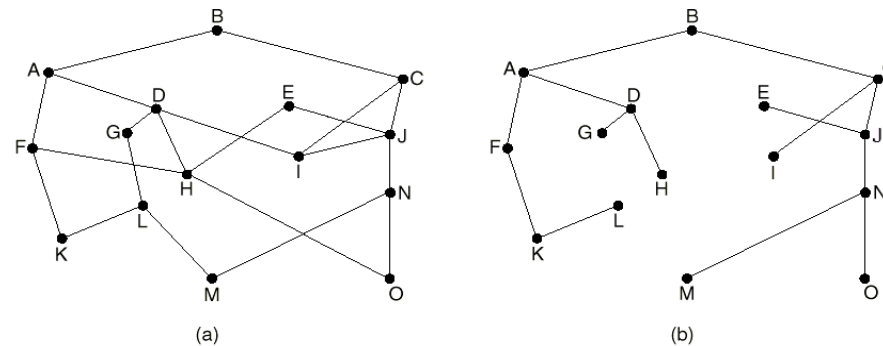


Fig. 5-5. (a) A subnet. (b) A sink tree for router B.



最短路径路由算法

- 最短路径路由算法 (**Shortest Path Routing**)
- 基本思想
 - 构建子网的拓扑图，图中的每个结点代表一个路由器，每条弧代表一条通信线路
 - 为了选择两个路由器间的路由，算法在图中找出最短路径
- 测量路径长度的方法
 - 结点数量 (**hop**)
 - 信道带宽
 - 地理距离
 - 传输延迟
 - 距离、信道带宽等参数的加权函数



Dijkstra 算法

- 每个结点用从源结点沿已知最佳路径到本结点的距离来标注，标注分为临时性标注和永久性标注
- 初始时，所有结点都为临时性标注，标注为无穷大
- 将源结点标注为**0**，且为永久性标注，并令其为工作结点
- 检查与工作结点相邻的临时性结点，若该结点到工作结点的距离与工作结点的标注之和小于该结点的标注，则用新计算得到的和重新标注该结点
- 在整个图中查找具有最小值的临时性标注结点，将其变为永久性结点，并成为下一轮检查的工作结点
- 重复第四、五步，直到目的结点成为工作结点



Edsger Wybe
Dijkstra,
(1930.5.11 –
2002.8.6), 荷
兰计算机科学家,
1972年获得图灵
奖

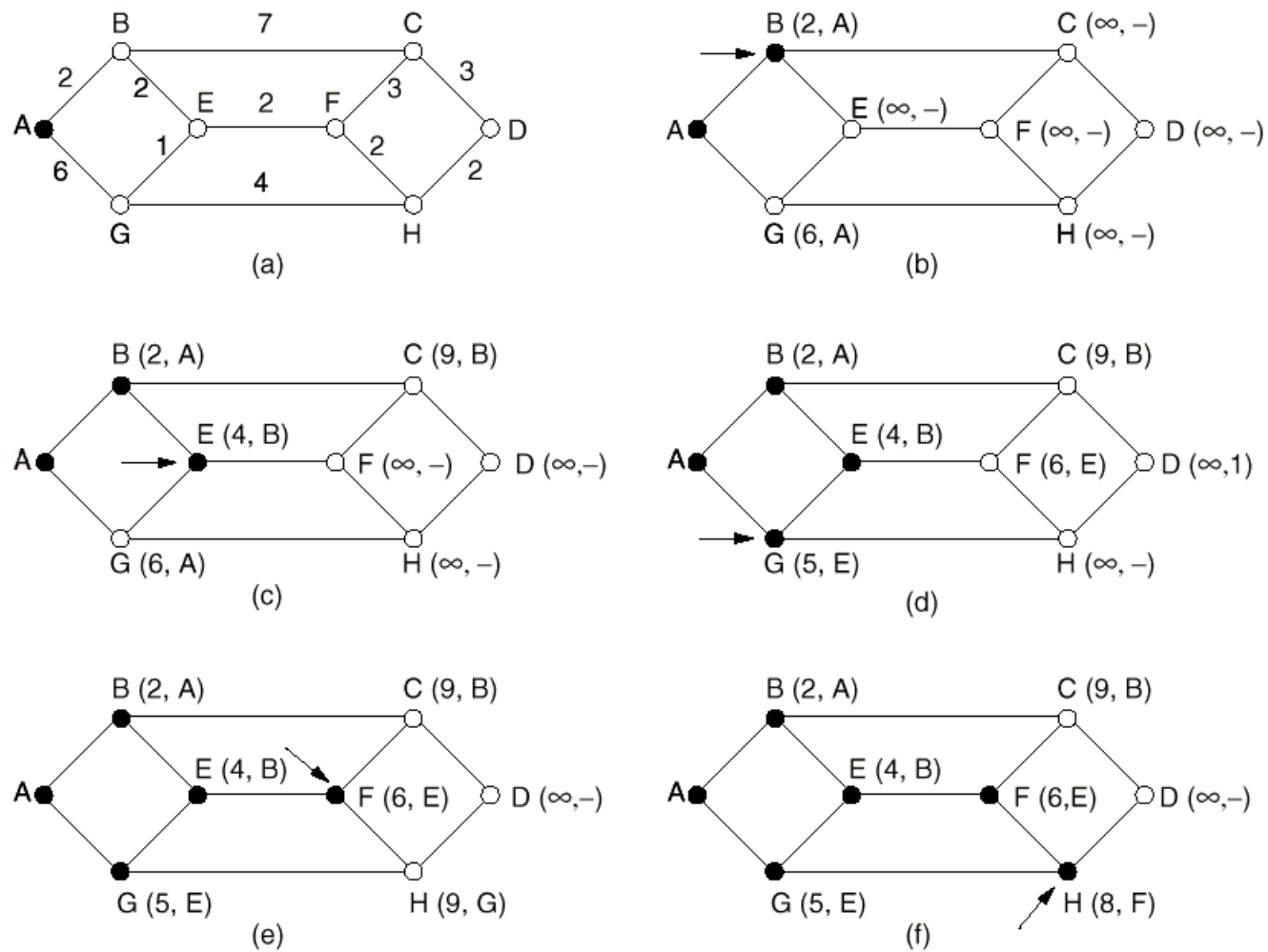


Fig. 5-6. The first five steps used in computing the shortest path from *A* to *D*. The arrows indicate the working node.



洪泛算法

- 洪泛算法（**Flooding**）属于静态路由算法
- 基本思想
 - 把收到的每一个分组，向除了该分组到来的线路外的所有输出线路发送
- 主要问题
 - 洪泛要产生大量重复分组，可能产生回路（**loop**）
- 解决措施
 - 每个分组头含站点计数器，每经过一站计数器减**1**，为**0**时则丢弃该分组
 - 记录分组经过的路径



洪泛算法（续）

- 选择性洪泛算法（**selective flooding**）
 - 洪泛法的一种改进，将进来的每个分组仅发送到与正确方向接近的线路上
- 算法特点
 - 对路由器和线路的资源过于浪费，实际很少直接采用
 - 具有极好的健壮性，可用于军事应用
 - 作为衡量标准评价其它路由算法



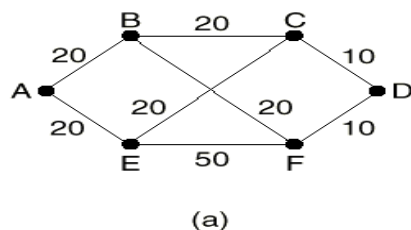
基于流量的路由算法

- 基于流量的路由算法（**Flow-Based Routing**）属于静态路由算法
- 基本思想
 - 既考虑拓扑结构，又兼顾网络负载
 - 前提：每对结点间平均数据流相对稳定和可预测
 - 根据网络带宽和平均流量，可得出平均分组延迟，因此路由选择问题归结为找产生网络最小延迟的路由选择算法
 - 提前离线（**off-line**）计算
 - 可用于流量工程（**traffic engineering**）



基于流量的路由算法（续）

- 需要预知的信息
 - 网络拓扑结构
 - 通信量矩阵 F_{ij}
 - 线路带宽矩阵 C_{ij}
 - 路由算法（可能是临时的）



	Destination					
	A	B	C	D	E	F
A		9 AB	4 ABC	1 ABFD	7 AE	4 AEF
B	9 BA		8 BC	3 BFD	2 BFE	4 BF
C	4 CBA	8 CB		3 CD	3 CE	2 CEF
D	1 DFBA	3 DFB	3 DC		3 DCE	4 DF
E	7 EA	2 EFB	3 EC	3 ECD		5 EF
F	4 FEA	4 FB	2 FEC	4 FD	5 FE	

(b)

Fig. 5-8. (a) A subnet with line capacities shown in kbps. (b) The traffic in packets/sec and the routing matrix.



i	Line	λ_i (pkts/sec)	C_i (kbps)	μC_i (pkts/sec)	T_i (msec)	Weight
1	AB	14	20	25	91	0.171
2	BC	12	20	25	77	0.146
3	CD	6	10	12.5	154	0.073
4	AE	11	20	25	71	0.134
5	EF	13	50	62.5	20	0.159
6	FD	8	10	12.5	222	0.098
7	BF	10	20	25	67	0.122
8	EC	8	20	25	59	0.098

Fig. 5-9. Analysis of the subnet of Fig. 5-8 using a mean packet size of 800 bits. The reverse traffic (*BA*, *CB*, etc.) is the same as the forward traffic.

- **$1/\mu = 800$ bits**
- **根据排队论，平均延迟 $T = 1/(\mu C - \lambda)$**



距离向量路由算法

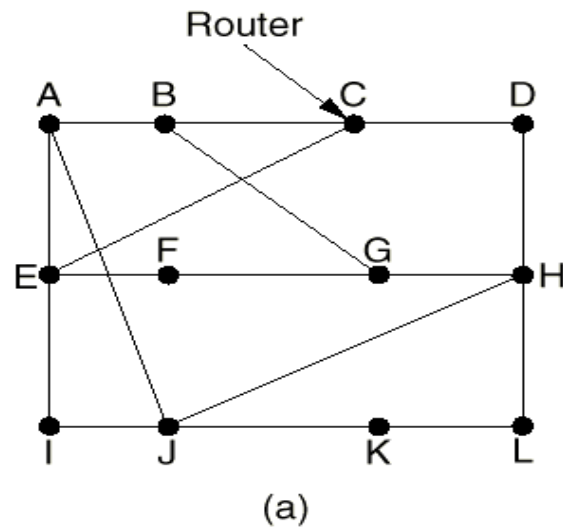
- 距离向量路由算法 (**Distance Vector Routing**)
- 属于动态路由算法
- 也称**Bellman-Ford**路由算法或**Ford-Fulkerson**算法
- 最初用于**ARPANET**，被**RIP**协议采用



距离向量路由算法（续）

■ 算法步骤

- 每个路由器维护一张表，通过与相邻路由器交换距离信息来更新表
- 以子网中其它路由器为表的索引，表项分组括两部分：到达目的结点的最佳输出线路，和到达目的结点所需时间或距离
- 每隔一段时间，路由器向所有邻居结点发送它到每个目的结点的距离表，同时它也接收每个邻居结点发来的距离表
- 邻居结点 X 发来的表中， X 到路由器 i 的距离为 X_i ，本路由器到 X 的距离为 m ，则路由器经过 X 到 i 的距离为 $X_i + m$ 。根据不同邻居发来的信息，计算 $X_i + m$ ，并取最小值，更新本路由器的路由表
- 注意：本路由器中的老路由表在计算中不被使用



(b)

To	A	I	H	K	New estimated delay from J	
					Line	
A	0	24	20	21	8	A
B	12	36	31	28	20	A
C	25	18	19	36	28	I
D	40	27	8	24	20	H
E	14	7	30	22	17	I
F	23	20	19	40	30	I
G	18	31	6	31	18	H
H	17	20	0	19	12	H
I	21	0	14	22	10	I
J	9	11	7	10	0	—
K	24	22	22	0	6	K
L	29	33	9	9	15	K

JA delay is 8	JI delay is 10	JH delay is 12	JK delay is 6
---------------	----------------	----------------	---------------

Vectors received from J's four neighbors

New routing table for J

Fig. 5-10. (a) A subnet. (b) Input from A, I, H, K, and the new routing table for J.



距离向量路由算法 (续)

- 无穷计算问题(count-to-infinity problem)
 - 对好消息反应迅速，对坏消息反应迟钝

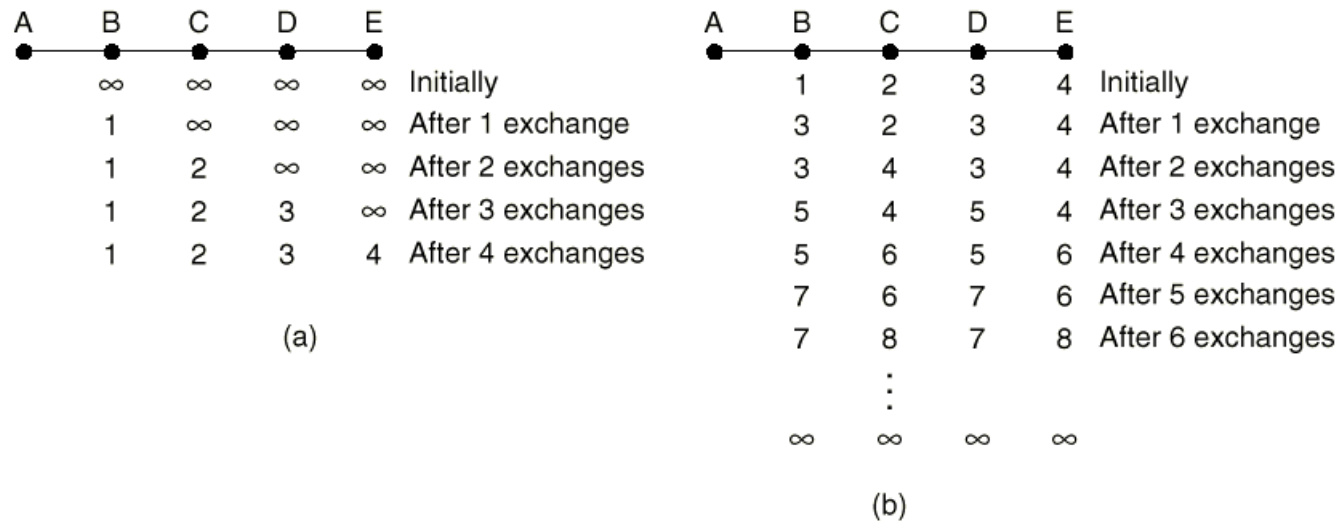


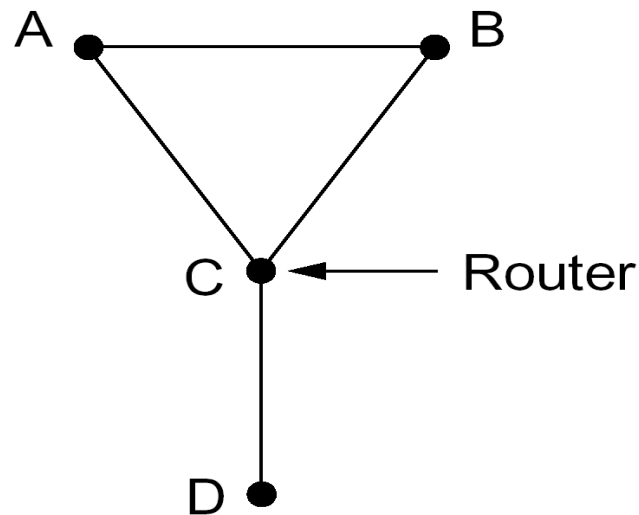
Fig. 5-11. The count-to-infinity problem.



距离向量路由算法（续）

■ 水平分裂算法

- 工作过程与距离向量算法相同，区别在于到X的距离不向真正通向X的邻居结点报告，使得坏消息传播的也快
- 虽然广泛使用，但有时候会失败





链路状态路由算法

- 链路状态路由算法（**Link State Routing**）
- 距离向量路由算法的主要问题
 - 选择路由时，没有考虑链路带宽
 - 路由收敛速度慢
 - 存在无穷计算问题
 - 路由报文开销大（不是增量更新）
 - 不适合用于大规模网络（**RIP**协议最大支持**15**跳）



链路状态路由算法（续）

■ 算法步骤

- 发现邻居结点，并学习它们的网络地址
 - 路由器启动后，通过发送**HELLO**分组发现邻居结点
 - 两个或多个路由器连在一个**LAN**时，引入人工结点（代表路由器**DR**, **Designated Router**）
- 测量到每个邻居结点的延迟或开销
 - 一种直接的方法是：发送一个要对方立即响应的**ECHO**分组，往返时间除以2即为延迟
 - 另一种方法是根据带宽来设定

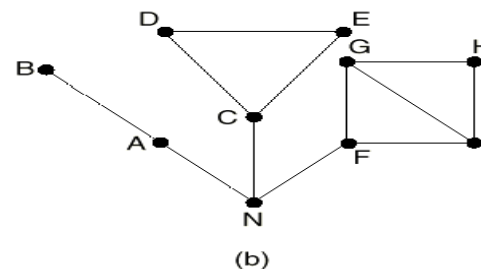
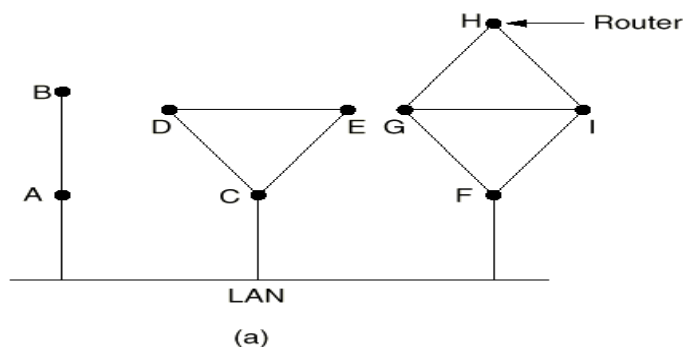
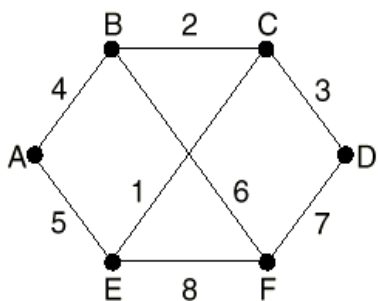


Fig. 5-13. (a) Nine routers and a LAN. (b) A graph model of (a).



链路状态路由算法（续）

- 将所有学习到的内容封装成一个分组
 - 分组以发送方的标识符开头，后面是序号、年龄和一个邻居结点列表
 - 列表中对每个邻居结点，都有发送方到它们的延迟或开销
 - 链路状态分组定期创建或发生重大事件时创建



(a)

		Link	State			Packets	
A		B	C	D		E	F
Seq.		Seq.	Seq.	Seq.		Seq.	Seq.
Age		Age	Age	Age		Age	Age
B	4	A	B	C	3	A	B
E	5	C	D	F	7	C	D
		F	E			F	E

(b)

Fig. 5-15. (a) A subnet. (b) The link state packets for this subnet.



链路状态路由算法（续）

- 将这个分组发送给所有其它路由器
 - 洪泛链路状态分组，为控制洪泛，每个分组分含一个序号，每次发送新分组时加1
 - 路由器记录信息对(源路由器，序号)，当一个链路状态分组到达时，若是新分组，则分发；若是重复分组或过时分组，则丢弃
 - 问题
 - 序号循环使用会混淆
 - 路由器崩溃后，序号重置
 - 序号出错



链路状态路由算法（续）

■ 改进

- 第一个问题的解决办法：使用32位序号
- 第二、三个问题的解决办法：增加年龄（age）域，每秒钟年龄减1，为零则丢弃（OSPF协议规定，age初始值为0，递增，超过MaxAge（一般为3600秒）要丢弃）
- 链路状态分组到达后，延迟一段时间，并与其它已到达的来自同一路由器的链路状态分组比较序号，丢弃重复分组，保留新分组
- 链路状态分组需要应答

■ 计算到所有其它路由器的最短路径

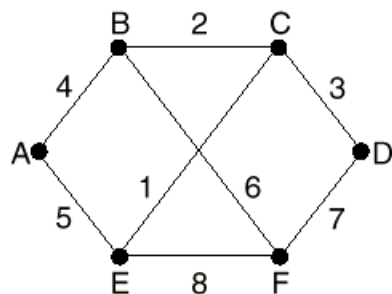
- 根据Dijkstra算法计算最短路径

■ 实用协议

- OSPF, IS-IS



例子：路由器B中的链路状态信息



Source	Seq.	Age	Send flags			ACK flags			Data
			A	C	F	A	C	F	
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

Fig. 5-16. The packet buffer for router *B* in Fig.5-15.

- 从E发来的链路状态分组有两个，一个经过**EAB**，另一个经过**EFB**
- 从D发链路状态分组有两个，一个经过**DCB**，另一个经过**DFB**



链路状态算法 (LS) 和距离向量算法 (DV) 的比较

■ 路由信息的复杂性

■ LS

- 路由信息向全网发送
- N 个节点, E 个链路的情况下, 发送 $O(NE)$ 个报文

■ DV

- 仅在邻居节点之间交换

■ 注意

- **LS**发送的是链路信息, **DV**发送的是到所有结点的向量信息
- **LS**信息定期创建 (**30分钟**) 或发生重大事件时创建, **DV**定期创建 (**30秒钟**)
- **LS**发布增量信息, **DV**发布全部信息



链路状态算法 (LS) 和距离向量算法 (DV) 的比较 (续)

■ 收敛 (Convergence) 速度

■ LS

- 使用最短路径优先算法，算法复杂度为 $O(n^2)$
 - n 个结点（不分组括源结点），需要 $n(n+1)/2$ 次比较
 - 使用更有效的实现方法，算法复杂度可以达到 $O(n \log n)$
- 可能存在路由振荡 (oscillations)

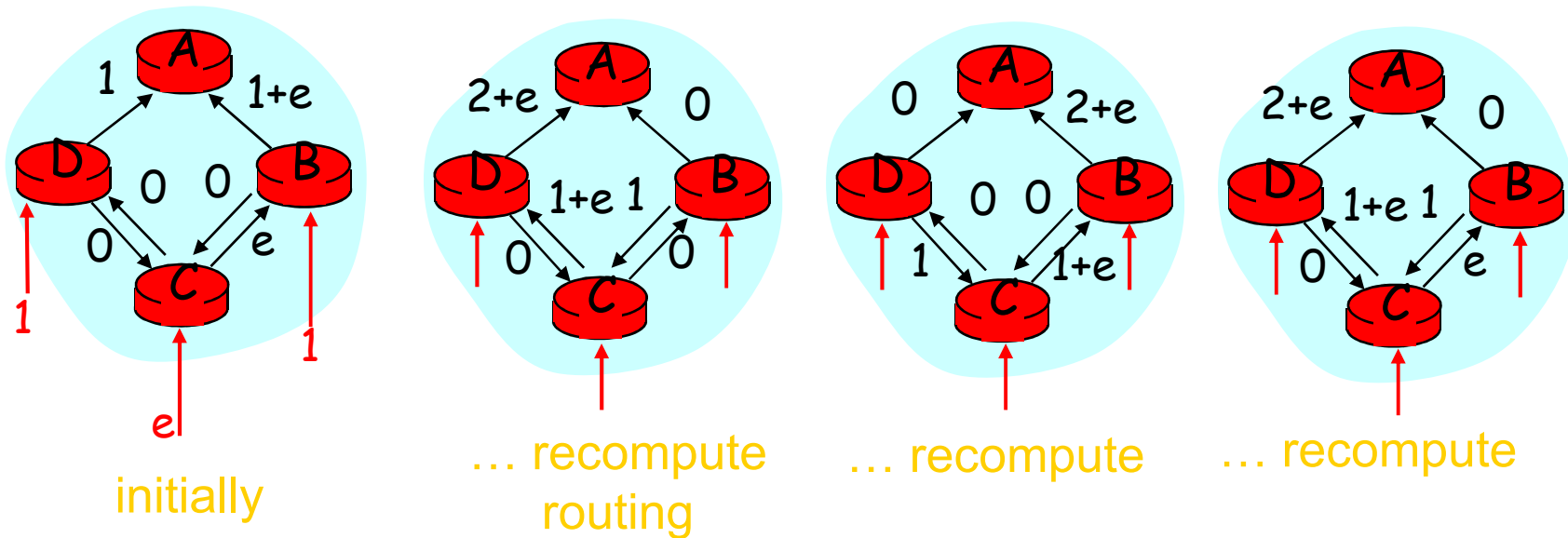
■ DV

- 收敛时间不确定
 - 可能会出现路由循环
 - count-to-infinity问题



路由振荡 (oscillations)

- 例如, **link cost = amount of carried traffic**





链路状态算法 (LS) 和距离向量算法 (DV) 的比较 (续)

- 健壮性: 如果路由器不能正常工作会发生什么?
 - LS
 - 结点会广播错误的链路开销
 - 每个结点只计算自己的路由表
 - DV
 - 结点会广播错误的路径开销
 - 每个结点的路由表被别的结点使用, 错误会传播到全网

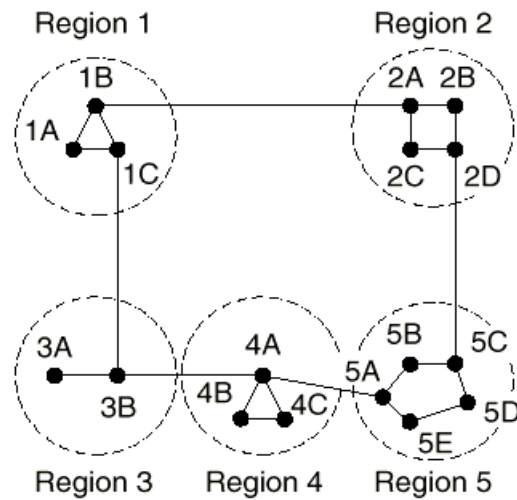


分层路由

- 网络规模增长带来的问题
 - 路由器中的路由表增大
 - 路由器为选择路由而占用的内存、CPU时间和网络带宽增大
 - 路由收敛慢
- 分层路由
 - 分而治之的思想
 - 根据需要，将网络分成若干域（domain）
 - 路由表规模大幅减少
 - Fig. 5-17，路由表由17项减为7项。
- 分层路由带来的问题
 - 分层后计算得到的路由不一定是最优路由



分层路由 (续)



(a)

Full table for 1A

Dest.	Line	Hops
1A	—	—
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

(b)

Hierarchical table for 1A

Dest.	Line	Hops
1A	—	—
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

(c)

Fig. 5-17. Hierarchical routing.



移动主机的路由

- 需要解决的问题
 - 为了能够将分组转发给移动主机，网络必须首先要找到移动的主机
- 网络结构示意图

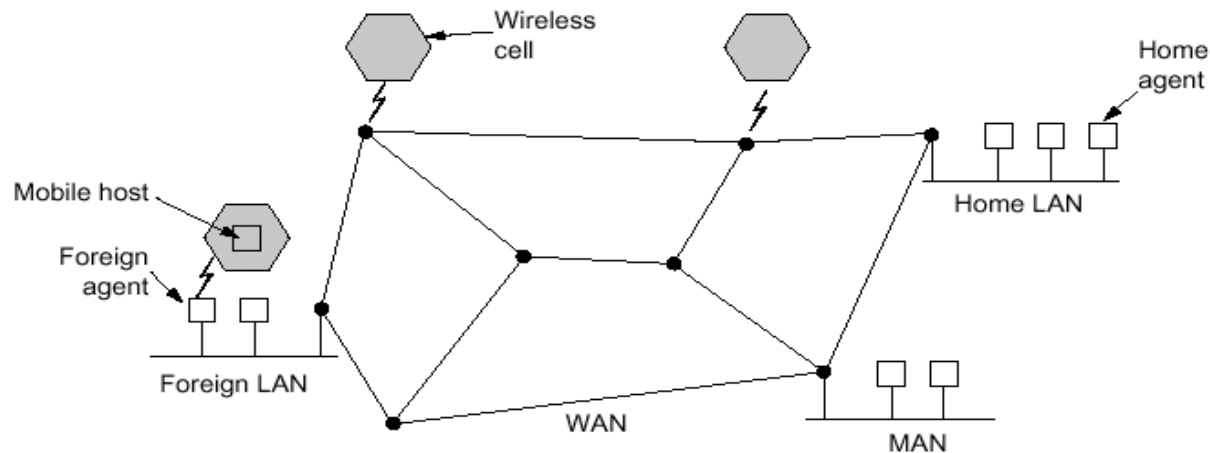


Fig. 5-18. A WAN to which LANs, MANs, and wireless cells are attached.



移动主机的路由（续）

■ 一些基本概念

- 移动用户（**mobile users**）：位置发生变化，包括通过固定方式或移动方式与网络连接的两类用户
- 家乡位置（**home location**）：所有用户都有一个永久的家乡位置，用一个地址来标识
- 家乡代理（**home agent**）：每个区域有一个家乡代理，负责记录家乡在该区域，但是目前正在访问其它区域的用户
- 外部代理（**foreign agent**）：每个区域（一个**LAN**或一个**wireless cell**）有一个或多个外部代理，它们记录正在访问该区域的移动用户



移动主机的路由（续）

- 移动用户进入一个新区域时，必须首先向外部代理注册
 - 外部代理定期广播声明自己的存在和地址的分组，新到达的移动主机接收该信息；若移动用户未能收到该信息，则移动主机广播分组，询问外部代理的地址
 - 移动主机向外部代理注册，告知其家乡地址、目前的数据链路层地址和一些安全信息
 - 外部代理与移动主机的家乡代理联系，告知移动主机的目前位置、自己的网络地址和一些安全信息
 - 家乡代理检查安全信息，通过，则给外部代理确认
 - 外部代理收到确认后，在登记表中加入一项，并通知移动主机注册成功



移动主机的路由（续）

■ 移动用户的路由转发过程

- 当一个分组发给移动用户时，首先被转发到用户的家乡局域网
- 该分组到达用户的家乡局域网后，被家乡代理接收，家乡代理查询移动用户的新位置和与其对应的外部代理的地址
- 家乡代理采用隧道技术，将收到的分组作为净负荷封装到一个新分组中，发给外部代理
- 家乡代理告诉发送方，发给移动用户的后续分组作为净负荷封装成分组直接发给外部代理（发送方要修改协议栈）
- 外部代理收到分组后，将净负荷封装成数据链路帧发给移动用户



三角路由 (Triangle Routing)

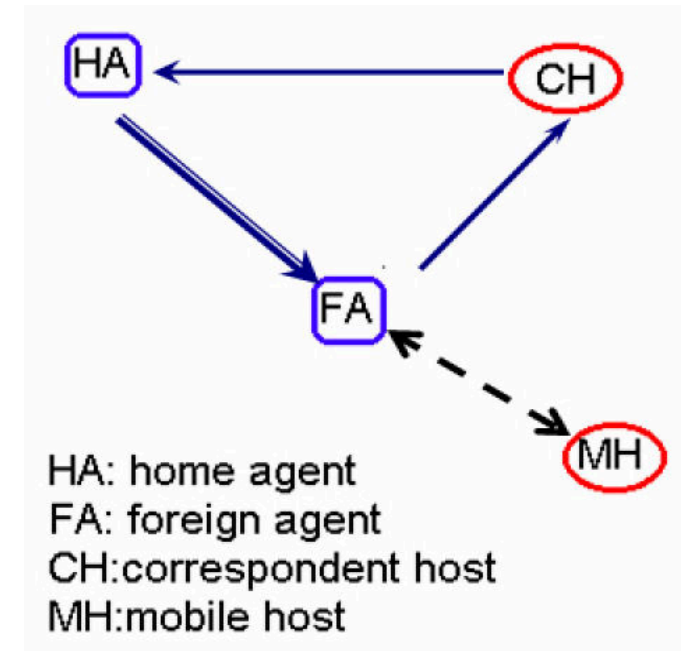
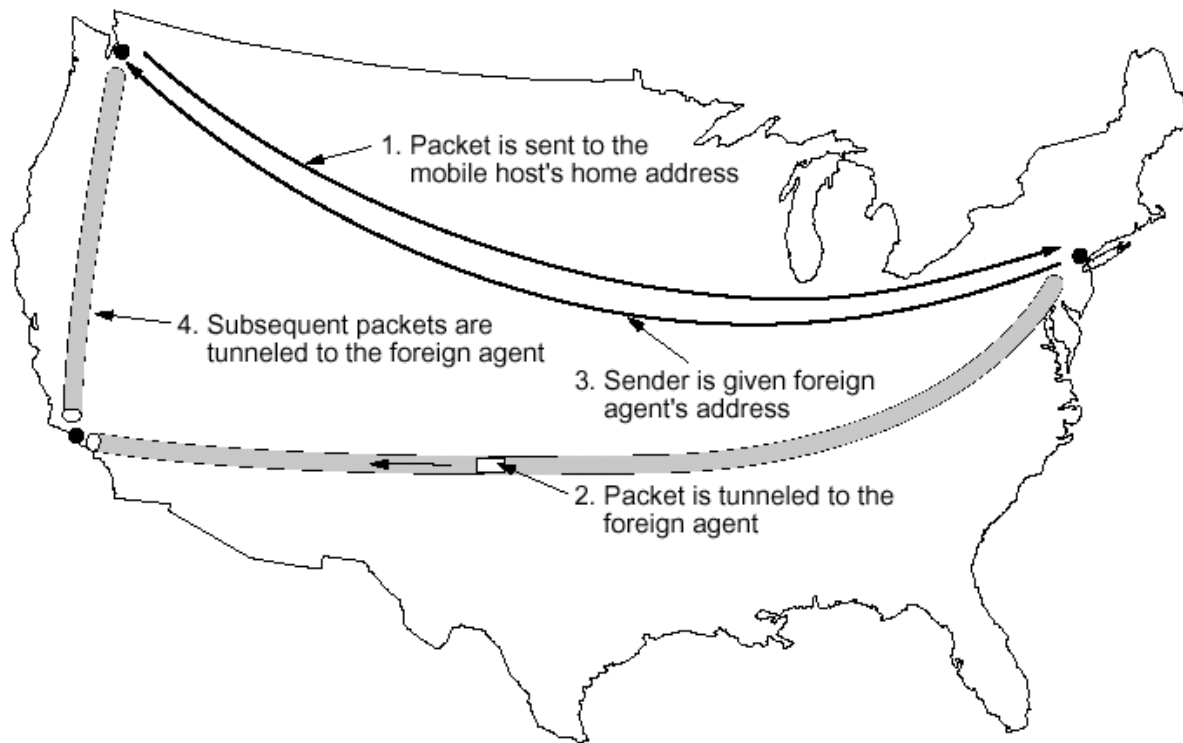


Fig. 5-19. Packet routing for mobile users.



路由算法小结

- 最优化原则
 - 路由算法的目的是找出并使用汇集树。
- 静态路由算法
 - 最短路径路由算法
 - 洪泛算法
 - 基于流量的路由算法



路由算法小结（续）

- 动态路由算法
 - 距离向量路由算法
 - 将自己（路由结点）对全网拓扑结构的认识告诉给邻居
 - 无穷计算问题，水平分裂算法
 - 链路状态路由算法
 - 将自己（路由结点）对邻居的认识洪泛给全网
- 分层路由
- 移动主机的路由



拥塞控制算法

- 拥塞 (**congestion**)
 - 网络上有太多的分组时，性能会下降，这种情况称为拥塞
- 拥塞产生的原因
 - 网络设备处理器性能低
 - 高速端口输入，低速端口输出
 - 多个输入对应一个输出

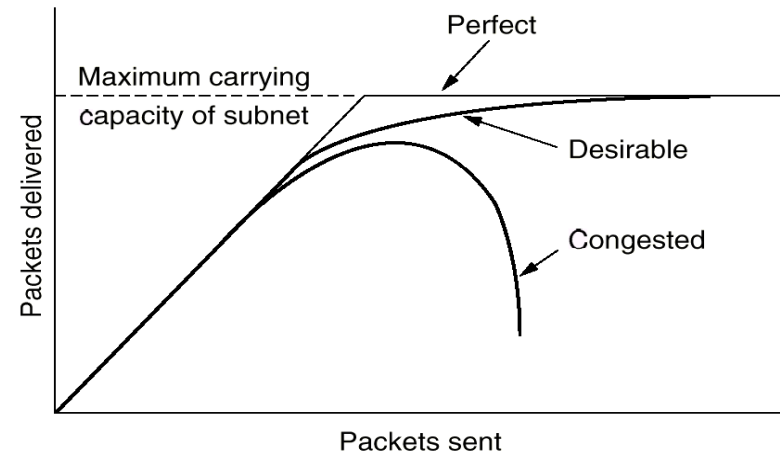


Fig. 5-22. When too much traffic is offered, congestion sets in and performance degrades sharply.



拥塞控制算法（续）

■ 解决办法

- 针对某个因素的解决方案，只能对提高网络性能起到一点点好处，甚至可能仅仅是转移了影响性能的瓶颈，因此需要全面考虑各个因素
- 目前，主要在网络层和传输层（**TCP**）进行控制

■ 拥塞控制与流控制的差别

- 拥塞控制（**congestion control**）需要确保通信子网能够承载用户提交的数据流，是一个全局性问题，涉及主机、路由器等很多因素
- 流控制（**flow control**）与点到点的传输有关，主要解决快速发送方与慢速接收方的问题，是局部问题，一般都是基于反馈进行控制的



拥塞控制的基本原理

- 根据控制论，拥塞控制方法分为两类
 - 开环控制
 - 通过好的设计来解决问题，避免拥塞发生
 - 拥塞控制时，不考虑网络当前状态
 - 闭环控制
 - 基于反馈机制
 - 工作过程
 - 监控系统，发现何时何地发生拥塞
 - 把发生拥塞的消息传给能采取动作的站点
 - 调整系统操作，解决问题



拥塞控制的基本原理（续）

- 衡量网络是否拥塞的参数
 - 缺乏缓冲区造成的分组丢失率
 - 平均队列长度
 - 超时重传的分组的数目
 - 平均分组延迟
 - 分组延迟变化 (**Jitter**)
- 反馈方法
 - 向负载发生源发送一个告警分组
 - 分组结构中保留一个位或域用来表示发生拥塞，一旦发生拥塞，路由器将所有的输出分组置位，向邻居告警
 - 主机或路由器主动地、周期性地发送探报 (**probe**)，查询是否发生拥塞



拥塞预防策略

- 开环控制
- 影响拥塞的网络设计策略

Layer	Policies
Transport	• Retransmission policy • Out-of-order caching policy • Acknowledgement policy • Flow control policy • Timeout determination
Network	• Virtual circuits versus datagram inside the subnet • Packet queueing and service policy • Packet discard policy • Routing algorithm • Packet lifetime management
Data link	• Retransmission policy • Out-of-order caching policy • Acknowledgement policy • Flow control policy

Fig. 5-23. Policies that affect congestion.



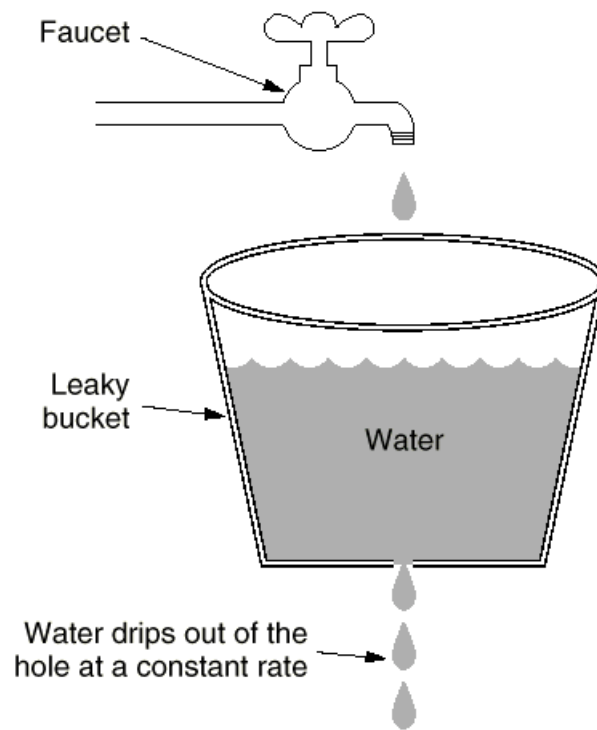
流量整形 (Traffic Shaping)

- 开环控制
- 基本思想
 - 造成拥塞的主要原因是网络流量通常是突发性的
 - 强迫分组以一种可预测的速率发送
- 典型算法
 - 漏桶算法
 - 令牌桶算法

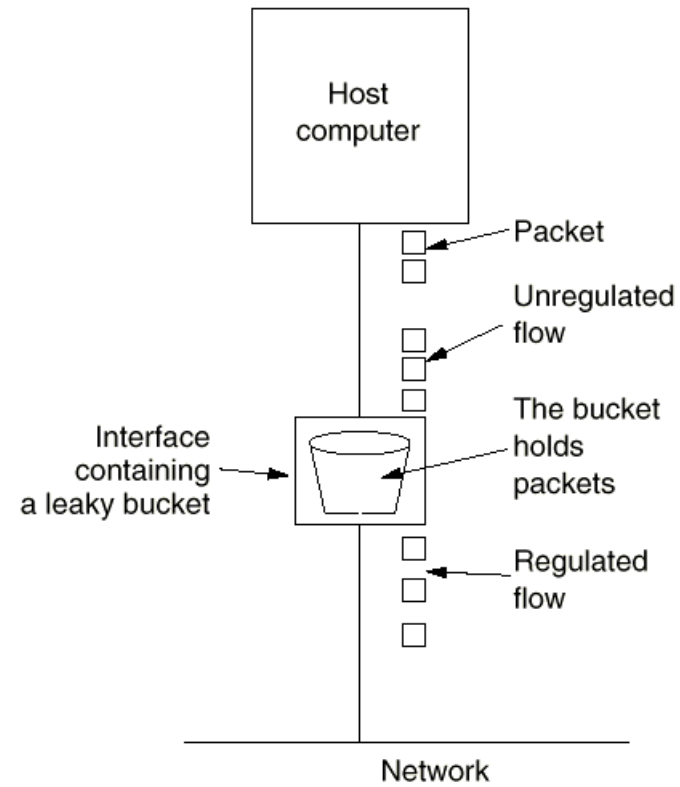


漏桶算法(Leaky Bucket)

- 将用户发出的不平滑的数据分组流转变成网络中平滑的数据分组流
- 可用于固定分组长的协议，如**ATM**；也可用于可变分组长的协议，如**IP**，使用字节计数
- 无论负载突发性如何，漏桶算法强迫输出按平均速率进行，不灵活



(a)



(b)

Fig. 5-24. (a) A leaky bucket with water. (b) A leaky bucket with packets.



令牌桶算法(Token Bucket)

- 漏桶算法不够灵活，因此加入令牌机制
- 基本思想：漏桶存放令牌，每 ΔT 秒产生一个令牌，令牌累积到超过漏桶上界时就不再增加。分组传输之前必须获得一个令牌，传输之后删除该令牌

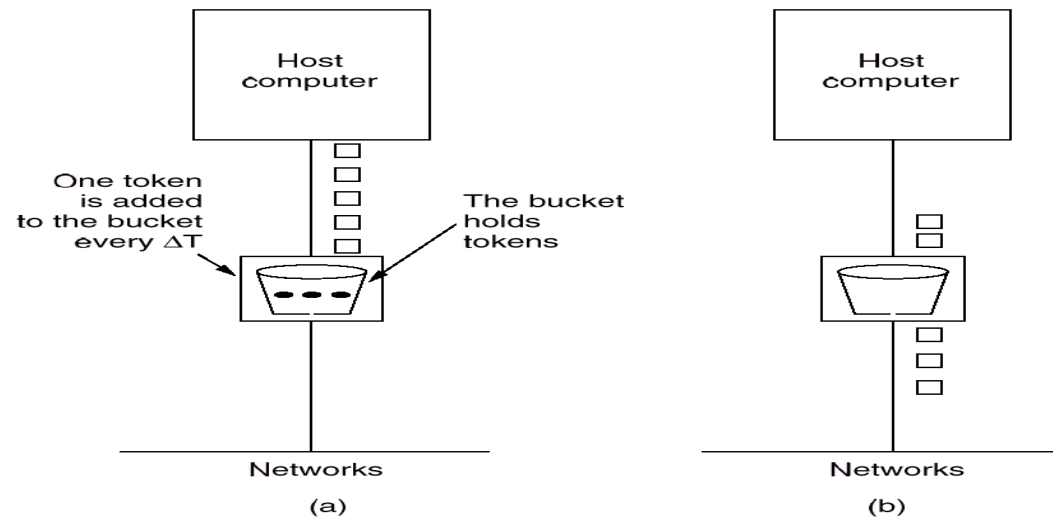


Fig. 5-26. The token bucket algorithm. (a) Before. (b) After.



漏桶算法与令牌桶算法的区别

- 流量整形策略不同
 - 漏桶算法不允许空闲主机积累发送权，以便以后发送大的突发数据
 - 令牌桶算法允许空闲主机积累发送权，最大为桶的大小
- 漏桶维护策略不同
 - 漏桶算法中，漏桶存放的是数据分组，桶满了丢弃数据分组
 - 令牌桶算法中，漏桶存放的是令牌，桶满了丢弃令牌

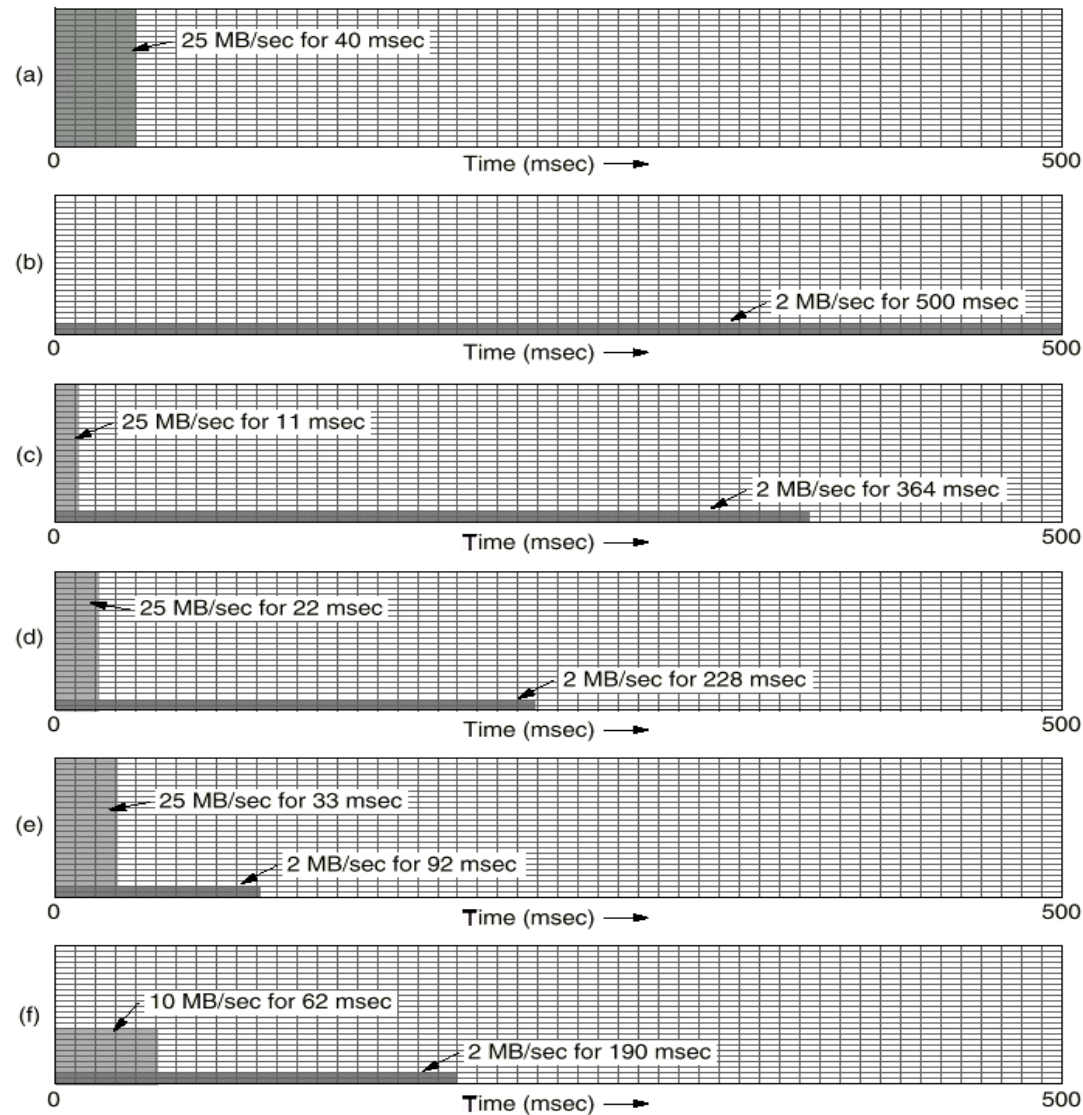


Fig. 5-25. (a) Input to a leaky bucket. (b) Output from a leaky bucket. (c) - (e) Output from a token bucket with capacities of 250KB, 500KB, and 750KB. (f) Output from a 500KB token bucket feeding a 10 MB/sec leaky bucket.



流说明 (Flow Specification)

- 一个数据流的发送方、接收方和通信子网三方认可的、描述发送数据流的模式和希望得到的服务质量的数据结构，称为流说明
- 对发送方的流说明，子网和接收方可以做出三种答复：同意、拒绝、其它建议

Characteristics of the Input	Service Desired
Maximum packet size (bytes)	Loss sensitivity (bytes)
Token bucket rate (bytes/sec)	Loss interval (μ sec)
Token bucket size (bytes)	Burst loss sensitivity (packets)
Maximum transmission rate (bytes/sec)	Minimum delay noticed (μ sec)
	Maximum delay variation (μ sec)
	Quality of guarantee

Fig. 5-27. An example flow specification.



虚电路子网中的拥塞控制

■ 准入控制 (admission control)

- 根据流说明和网络资源分配情况, 进行准入控制
- 一旦发生拥塞, 在问题解决之前, 不允许建立新的虚电路
- 另一种方法是发生拥塞后可以建立新的虚电路, 但要绕开发生拥塞的地区

■ 资源预留: 建立虚电路时, 主机与子网达成协议, 子网根据协议在虚电路上为此连接预留资源

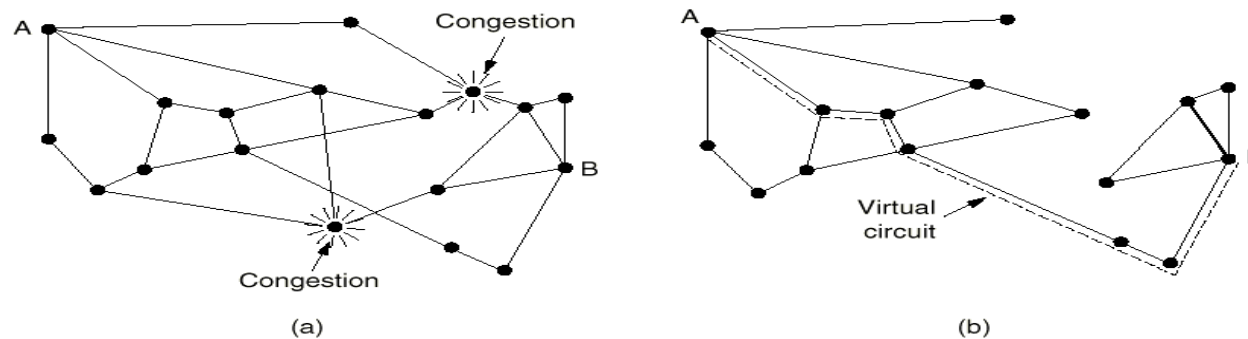


Fig. 5-28. (a) A congested subnet. (b) A redrawn subnet that eliminates the congestion and a virtual circuit from A to B.



抑制分组(Choke Packets)

■ 基本思想

- 路由器监控输出线路及其它资源的利用情况，超过某个阈值，则此资源进入警戒状态
- 每个新分组到来，检查它的输出线路是否处于警戒状态
- 若是，则向源主机发送抑制分组，分组中指出发生拥塞的目的地址。同时将原分组打上标记（为了以后不再产生抑制分组），正常转发
- 源主机收到抑制分组后，按一定比例减少发向特定目的地的流量，并在固定时间间隔内忽略指示同一目的地的抑制分组。然后开始监听，若此线路仍然拥塞，则主机在固定时间内减轻负载、忽略抑制分组；若在监听周期内没有收到抑制分组，则增加负载
- 通常采用的流量增减策略是：减少时，按一定比例减少，保证快速解除拥塞；增加时，以常量增加，防止很快导致拥塞(AIMD)



逐跳抑制分组 (Hop-by-Hop Choke Packets)

- 在高速、长距离的网络中，由于源主机响应太慢，抑制分组算法对拥塞控制的效果并不好，可采用逐跳抑制分组算法
- 基本思想
 - 抑制分组对它经过的每个路由器都起作用
 - 能够迅速缓解发生拥塞处的拥塞
 - 上游路由器要求有更多的缓冲区

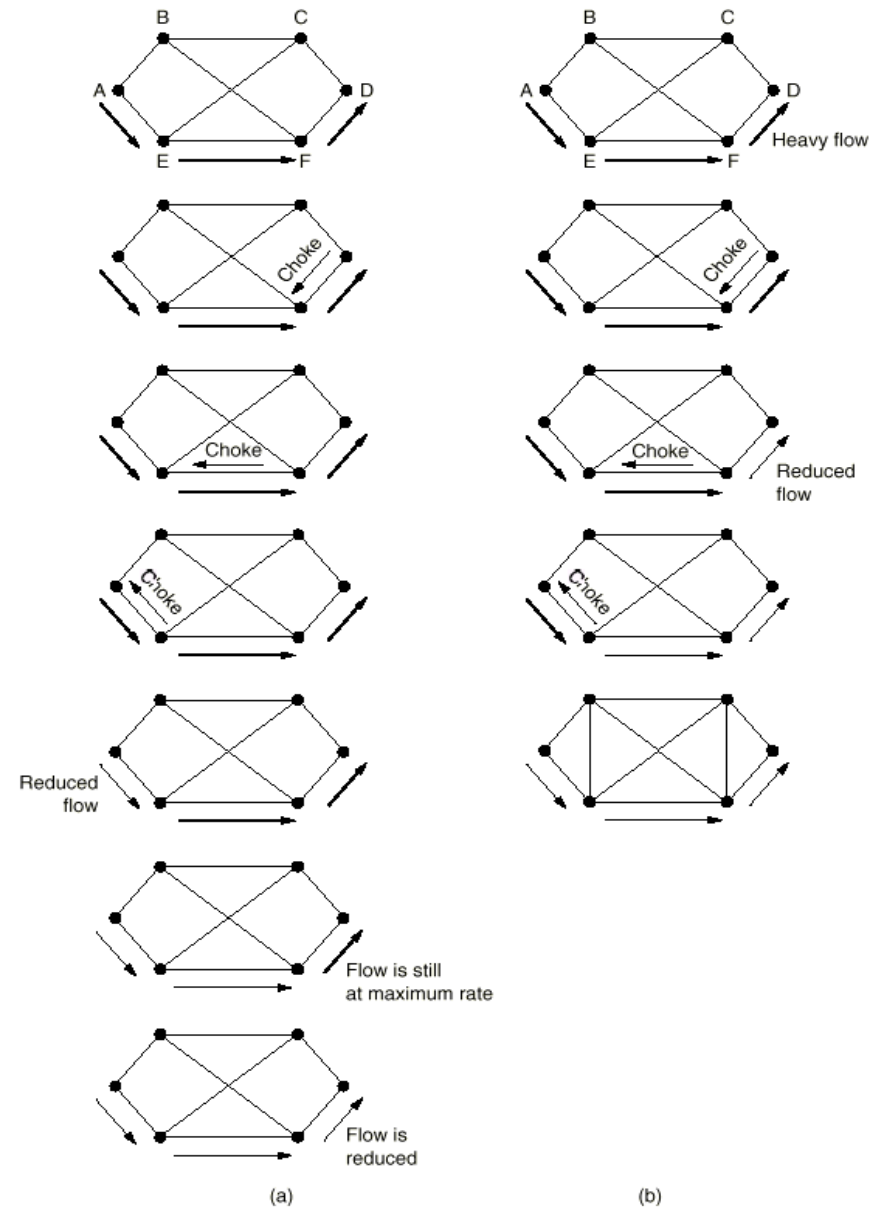


Fig. 5-30. (a) A choke packet that affects only the source. (b) A choke packet that affects each hop it passes through.



公平队列算法 (Fair Queueing)

- 路由器的每个输出线路有多个队列
- 路由器循环扫描各个队列，发送队头的分组
- 所有队列具有相同优先级
- 一些**ATM**交换机、路由器使用这种算法
- 一种改进：对于变长分组，由逐分组轮讯改为逐字节轮讯

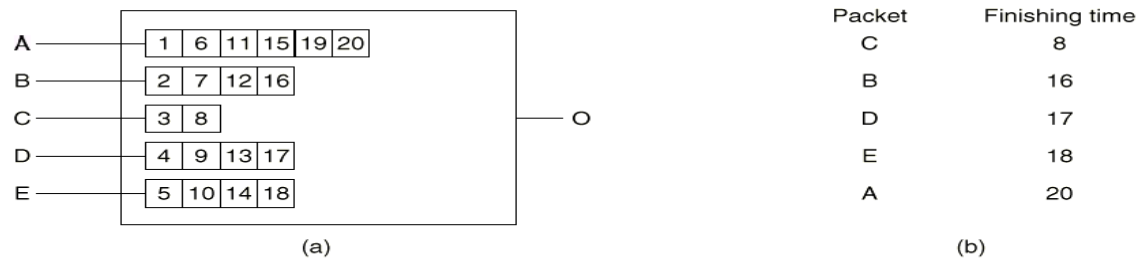


Fig. 5-29. (a) A router with five packets queued for line O. (b) Finishing times for the five packets.



加权公平队列 (Weighted Fair Queueing)

- 给不同队列以不同的优先级
- 优先级高的队列在一个轮询周期内获得更多的时间片



负载丢弃 (Load Shedding)

- 上述算法都不能消除拥塞时，路由器只得将分组丢弃
- 针对不同服务，可采取不同丢弃策略
 - 文件传输，优先丢弃新分组，**wine**策略
 - 多媒体服务，优先丢弃旧分组，**milk**策略
- 早期丢弃分组，会减少拥塞发生的概率，提高网络性能



网络互联

- 互连网络（**internet**）：两个或多个网络构成互连网络

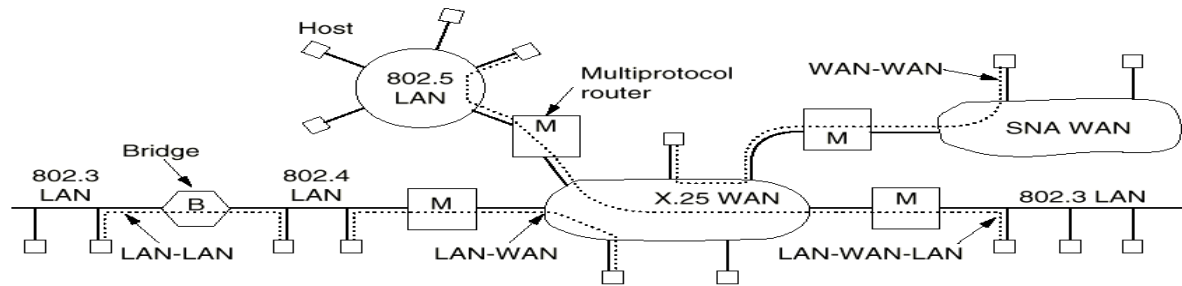


Fig. 5-33. Network interconnection.

- 多种不同网络（协议）存在的原因
 - 历史原因：不同公司的网络产品大量使用
 - 价格原因：网络产品价格低，更多的人有权决定使用何种网络
 - 技术原因：不同网络采用不同技术、不同硬件、不同协议



网络互连设备

■ 网络互连设备

■ 中继器（**repeater**）

- 物理层设备，在电缆段之间拷贝比特
- 对弱信号进行放大或再生，以便延长传输距离

■ 网桥（**bridge**）

- 数据链路层设备，在局域网之间存储转发帧
- 网桥可以改变帧格式

■ 多协议路由器（**multiprotocol router**）

- 网络层设备，在网络之间存储转发分组
- 必要时，做网络层协议转换



网络互连设备（续）

- 传输网关（**transport gateway**）
 - 传输层设备，在传输层转发字节流
- 应用网关（**application gateway**）
 - 应用层设备，在应用层实现互连



网络之间的区别

Item	Some Possibilities
Service offered	Connection-oriented versus connectionless
Protocols	IP, IPX, CLNP, AppleTalk, DECnet, etc.
Addressing	Flat (802) versus hierarchical (IP)
Multicasting	Present or absent (also broadcasting)
Packet size	Every network has its own maximum
Quality of service	May be present or absent; many different kinds
Error handling	Reliable, ordered, and unordered delivery
Flow control	Sliding window, rate control, other, or none
Congestion control	Leaky bucket, choke packets, etc.
Security	Privacy rules, encryption, etc.
Parameters	Different timeouts, flow specifications, etc.
Accounting	By connect time, by packet, by byte, or not at all

Fig. 5-35. Some of the many ways networks can differ.



无连接网络互连

- 无连接网络互连的工作过程与数据报子网的工作过程相似
- 每个分组单独路由，提高网络利用率
- 根据需要，连接不同子网的多协议路由器做协议转换，分组包括分组格式转换和地址转换等

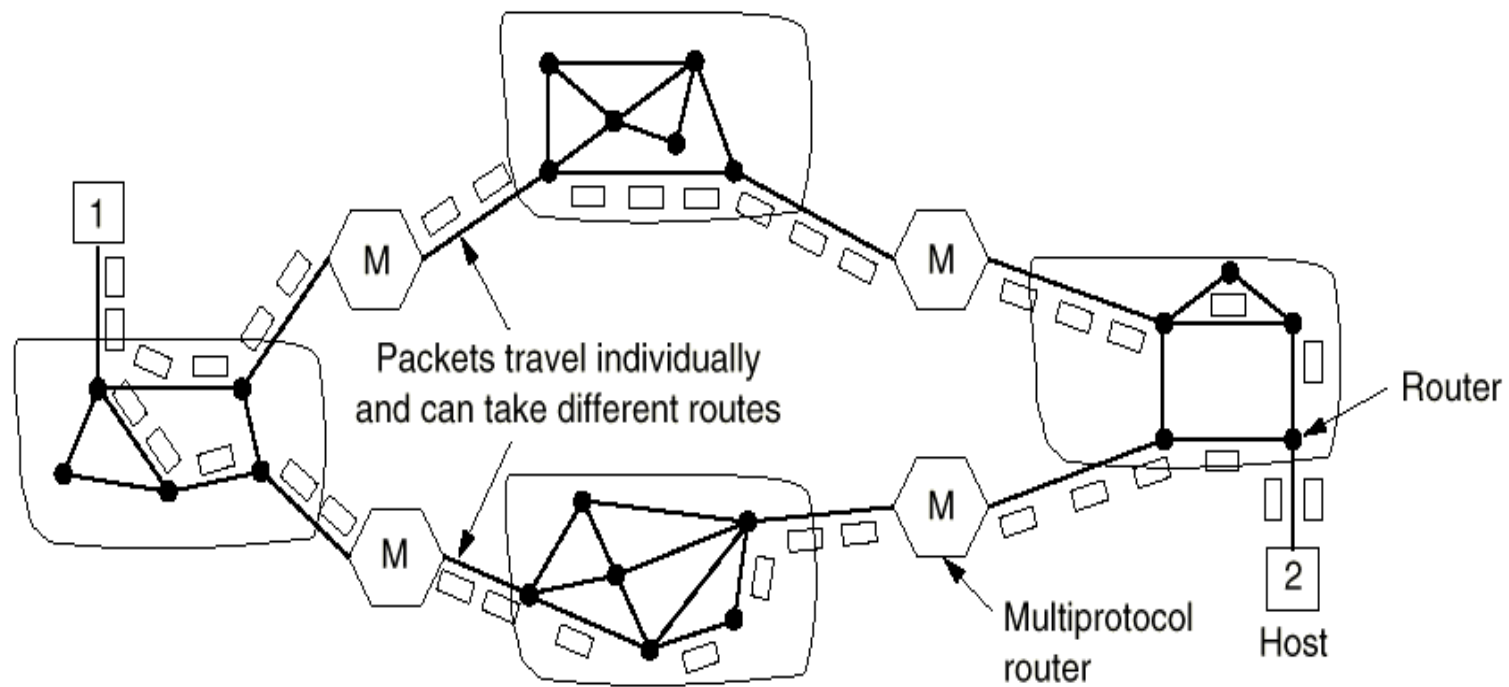
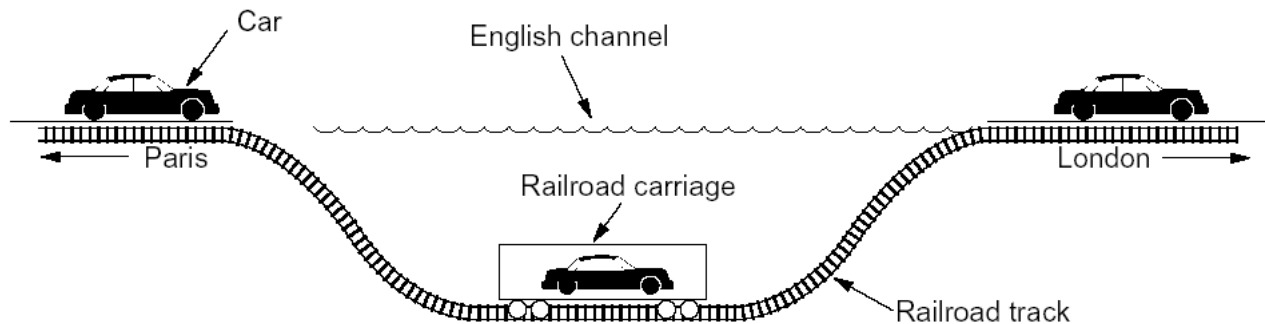


Fig. 5-37. A connectionless internet.



隧道技术

- 源和目的主机所在网络类型相同，连接它们的是一个不同类型的网络，这种情况下可以采用隧道技术





隧道技术 (续)

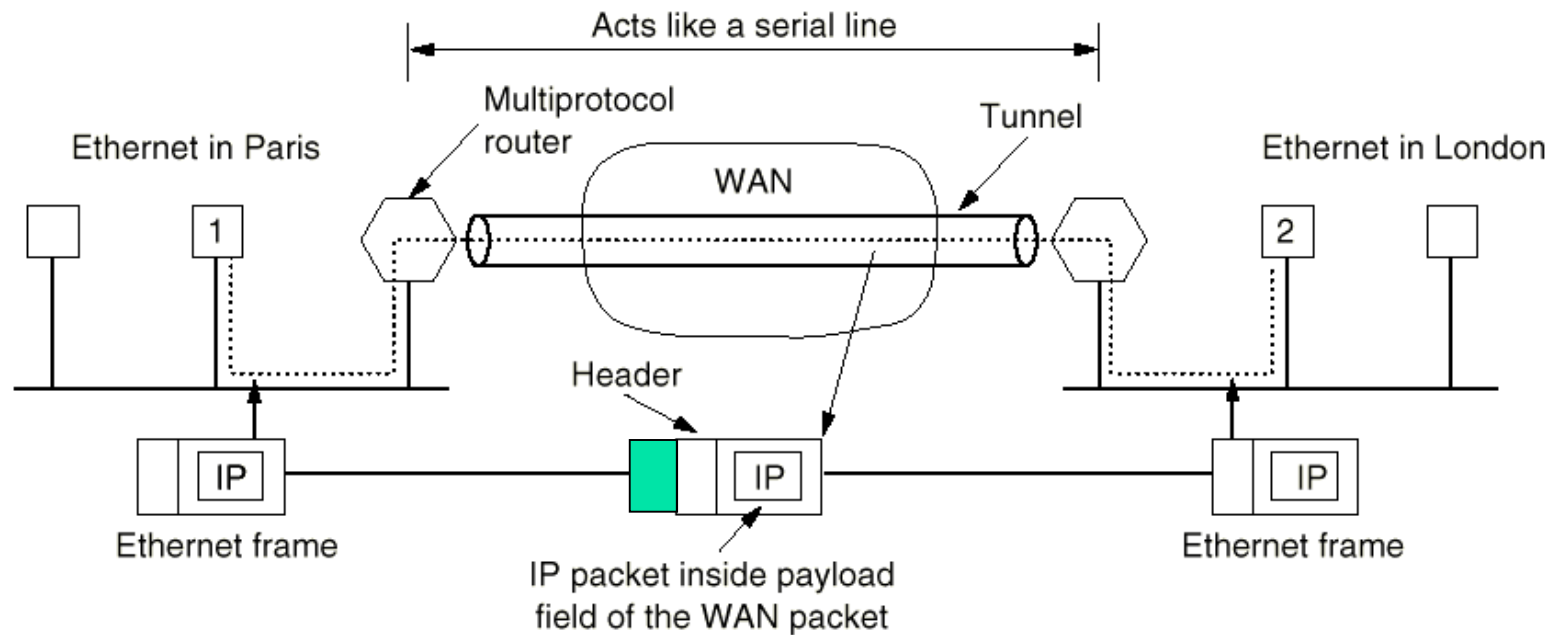


Fig. 5-38. Tunneling a packet from Paris to London.



隧道技术（续）

- 工作过程（以**Fig. 5-38**为例）
 - 主机**1**发送一个分组，目的**IP**地址 = 主机**2-IP**，将分组封装到局域网帧中，帧目的**MAC**地址 = 路由器**1-MAC**
 - 局域网传输
 - 路由器**1**剥掉局域网帧头、帧尾，将得到的**IP**分组封装到广域网**网络层分组**中，分组目的**IP**地址 = 路由器**2-IP**
 - 广域网传输
 - 路由器**2**剥掉广域网分组头，将得到的**IP**分组封装到局域网帧中，帧目的**MAC**地址 = 主机**2-MAC**地址
 - 局域网传输
 - 主机**2**接收



互连网络路由

■ 互连网络路由工作过程

- 互连网络的路由与单独子网的路由过程相似，只是复杂性增加

■ 两级路由算法

- 自治系统**AS**
- 内部网关协议**IGP**
 - **RIP, OSPF**
- 外部网关协议**EGP**
 - **BGP**

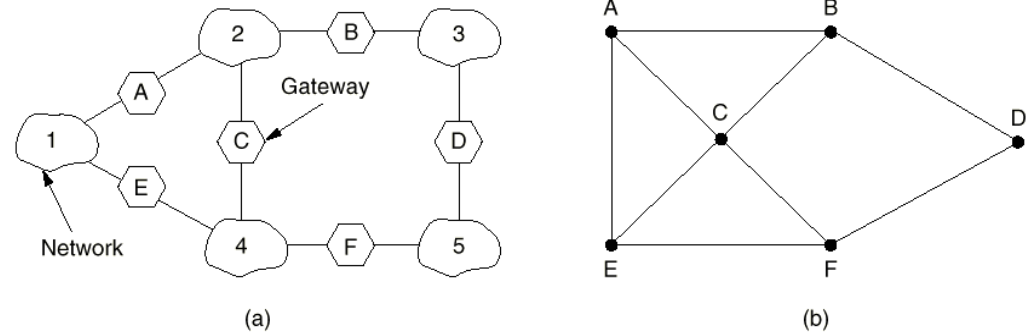


Fig. 5-40. (a) An internetwork. (b) A graph of the internetwork.



分片 (Fragmentation)

- 每种网络都对最大分组长度有限制，有以下原因
 - 硬件，例如 **TDM** 的时槽限制
 - 操作系统
 - 协议，例如分组长度域的比特个数
 - 与标准的兼容性
 - 希望减少传输出错的概率
 - 希望避免一个分组占用信道时间过长
- 大分组经过小分组网络时，网关要将大分组分成若干片段（**fragment**），每个片段作为独立的分组传输



分片（续）

■ 分片重组策略

■ 分片重组过程对其它网络透明

- 网关将大分组分片后，每个片段都要经过同一出口网关，并在那里重组
- 带来的问题
 - 出口网关需要知道何时所有片段都到齐
 - 所有片段必须从同一出口网关离开
 - 大分组经过一系列小分组网络时，需要反复分片重组，开销大

■ 分片重组过程对其它网络不透明

- 中间网关不做重组，而由目的主机做
- 带来的问题
 - 对主机要求高，能够重组
 - 每个片段都要有一个分组头，网络开销增大



分片 (续)

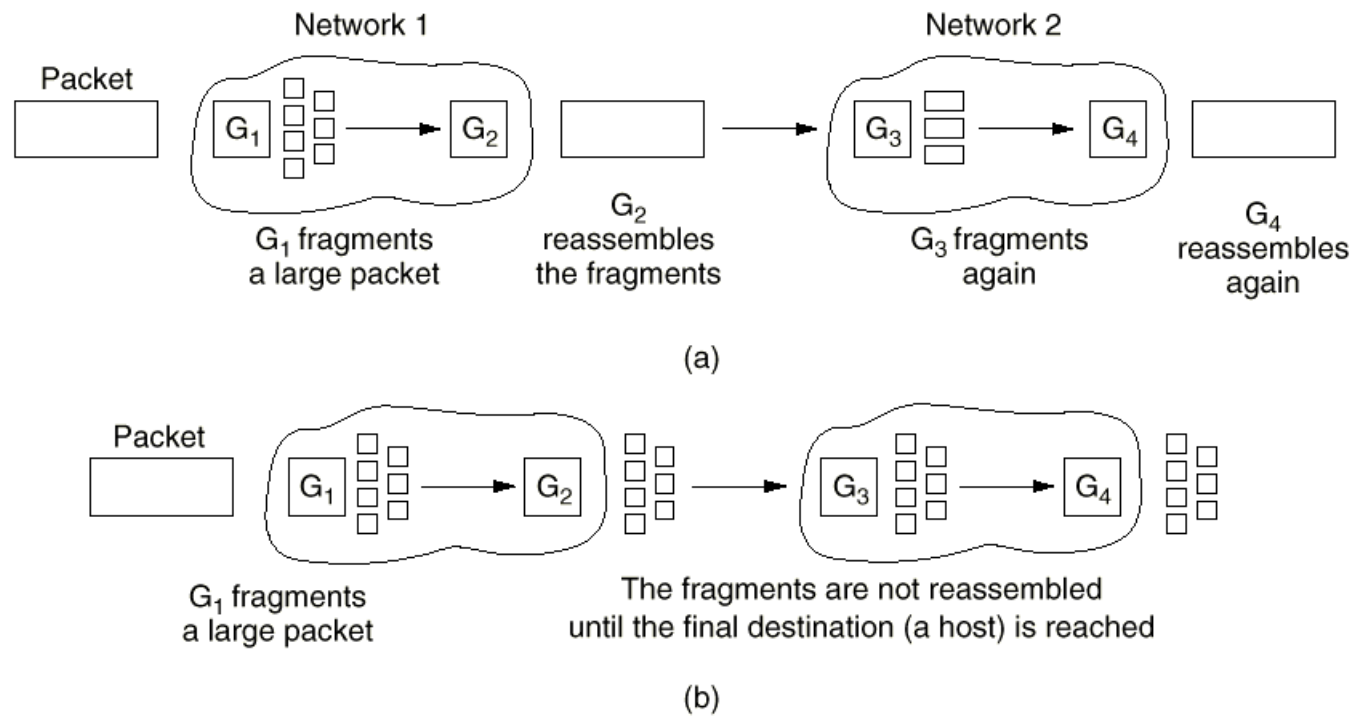


Fig. 5-41. (a) Transparent fragmentation. (b) Nontransparent fragmentation.



分片（续）

■ 标记片段

■ 树型标记法

- 分组0分成三段，分别标记为0.0, 0.1, 0.2，片段0.0构成的分组被分成三片，分别标记为0.0.0, 0.0.1, 0.0.2
- 存在的问题
 - 段标记域要足够长
 - 分片长度前后要一致

■ 偏移量法

- 定义一个基本片段长度，使得基本片段能够通过所有网络
- 分片时，除最后一个片段小于等于基本片段长度外，所有片段长度都等于基本片段长度
- 分片后的分组头中包括；原始分组序号，分组中第一个基本片段的偏移量，最后片段指示位

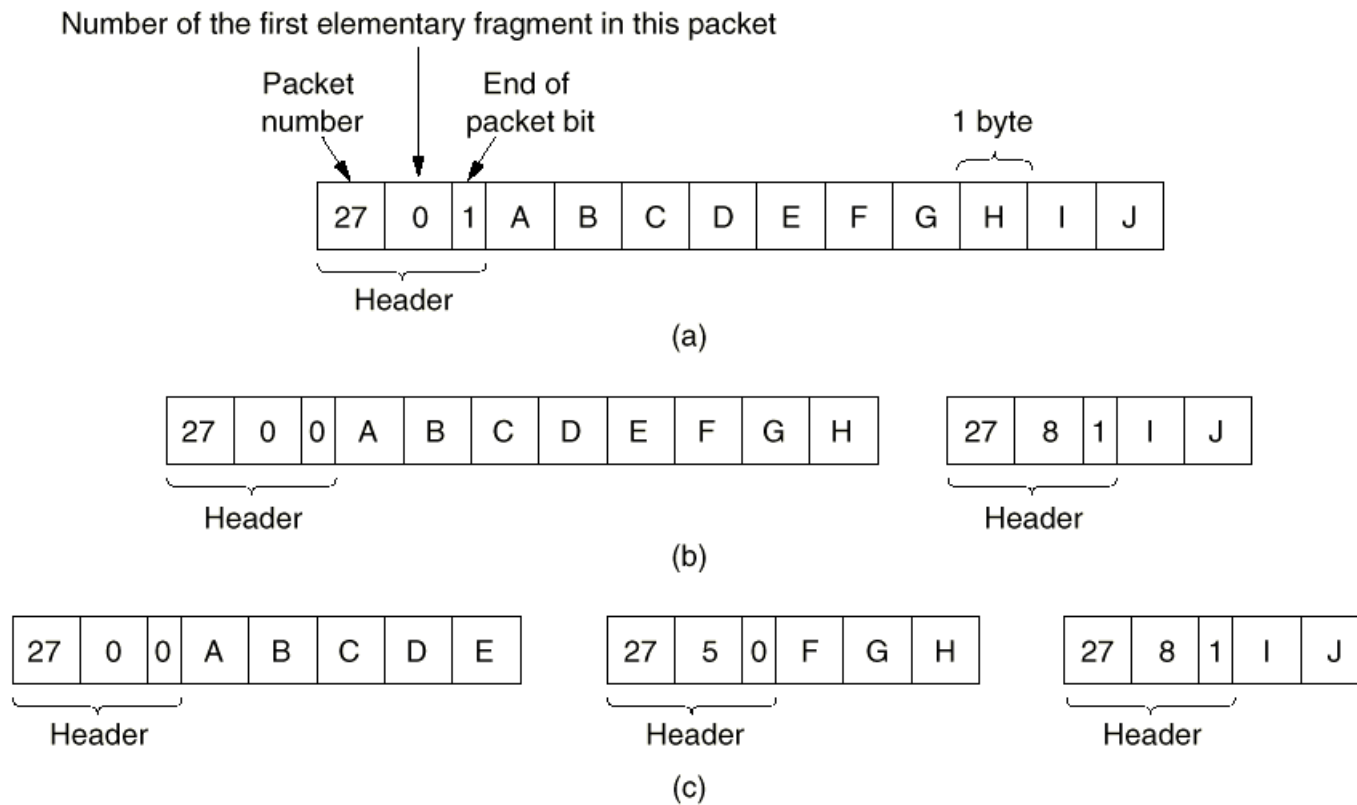


Fig. 5-42. Fragmentation when the elementary data size is 1 byte. (a) Original packet, containing 10 data bytes. (b) Fragments after passing through a network with maximum packet size of 8 bytes. (c) Fragments after passing through a size 5 gateway.



防火墙(Firewall)

- 什么情况下使用防火墙？
 - 为防止网络中的信息泄露出去或不好的信息渗透进来，在网络边缘设置防火墙
- 防火墙的一种早期配置
 - 两个路由器，根据某种规则表，进行分组过滤
 - 一个应用网关，审查应用层信息

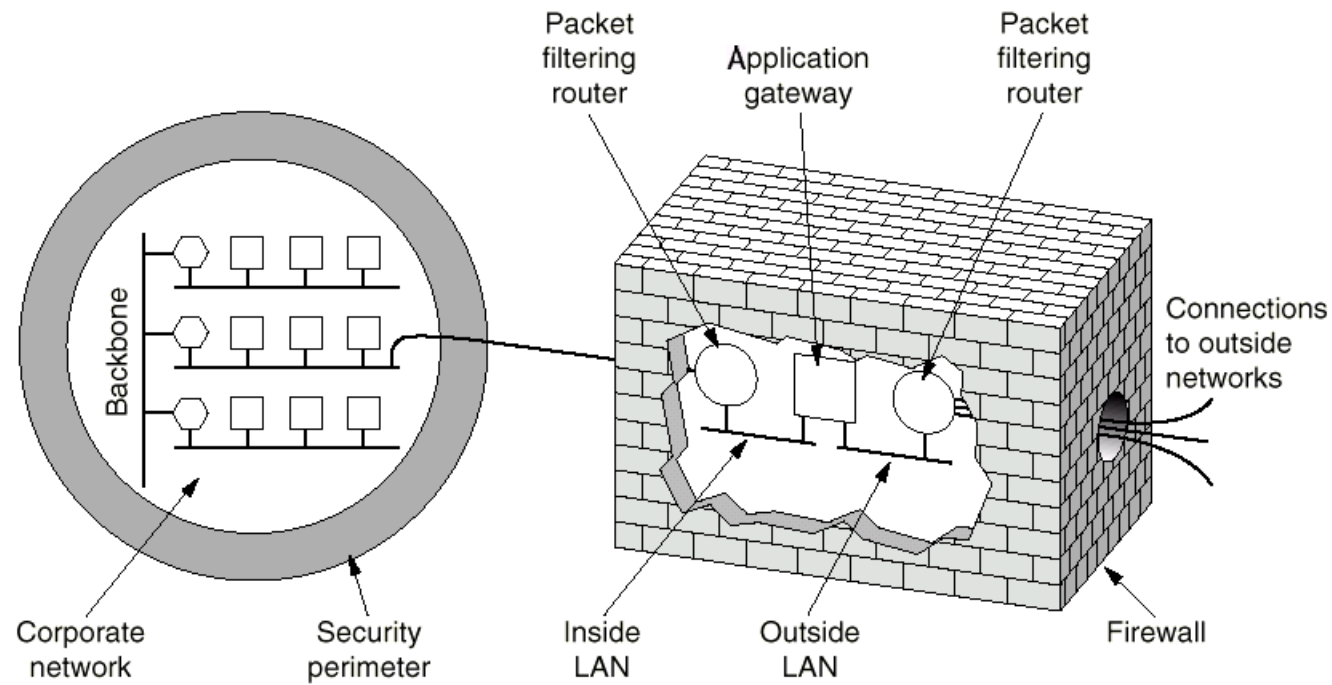


Fig. 5-43. A firewall consisting of two packet filters and an application gateway.



网络互连小结

- 互联网：两个或多个网络构成互联网
- 网络互连设备：中继器、网桥、路由器、传输网关、应用网关
- 隧道技术
- 分片和重组
- 防火墙



网络层协议

- 在网络层，互联网可以看成是自治系统的集合，是由网络组成的网络
- 网络之间互连的纽带是**IP（Internet Protocol）**协议

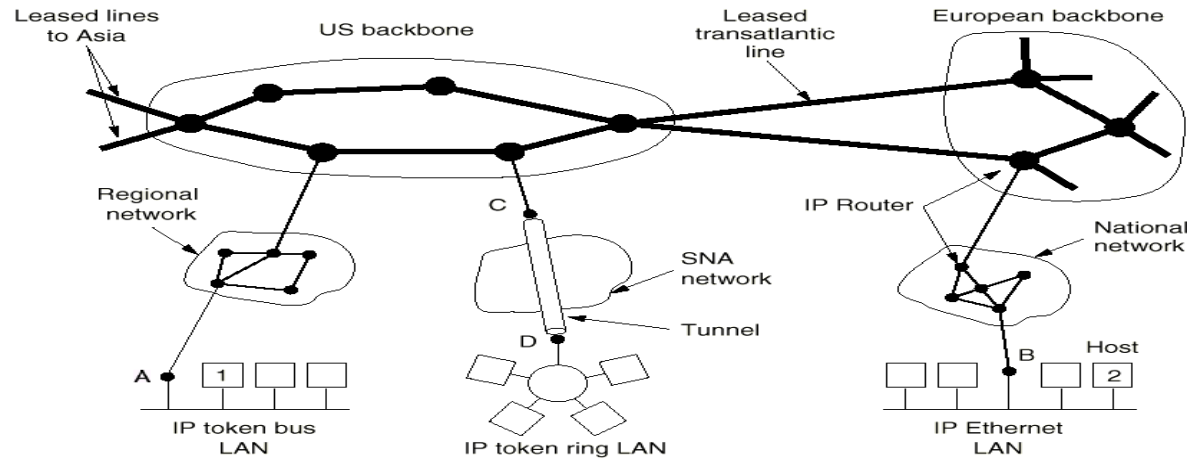


Fig. 5-44. The Internet is an interconnected collection of many networks.



IP协议

■ IP头格式

- IP头包括20个字节的固定部分和变长（最长40字节）的可选部分，从左到右传输

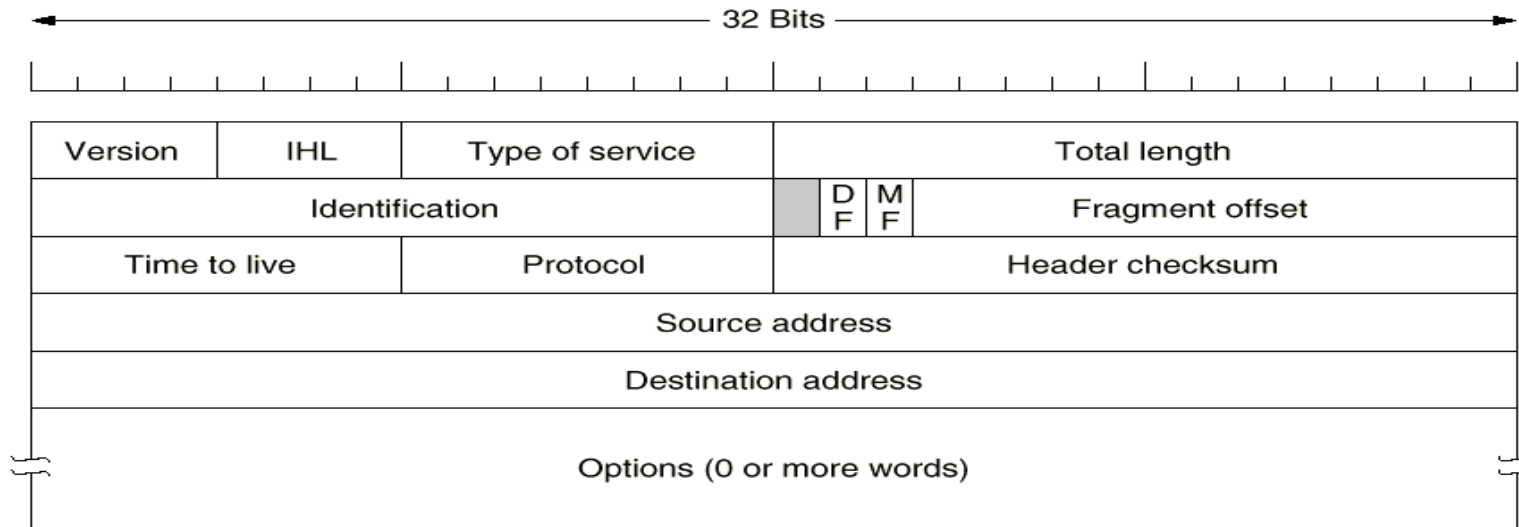


Fig. 5-45. The IP (Internet Protocol) header.



IP协议（续）

- 版本域（**version**）
- **IHL**: IP分组头长度，最小为**5**，最大为**15**，单位为**32-bit word**
- 服务类型域（**Type of Service**）
 - 3个优先级位
 - 3个标志位: **D**（**Delay**）、**T**（**Throughput**）、**R**（**Reliability**）
 - 2个保留位
 - 目前，很多路由器都忽略服务类型域
- 总长度域（**Total length**）
- 标识域（**Identification**）



IP协议（续）

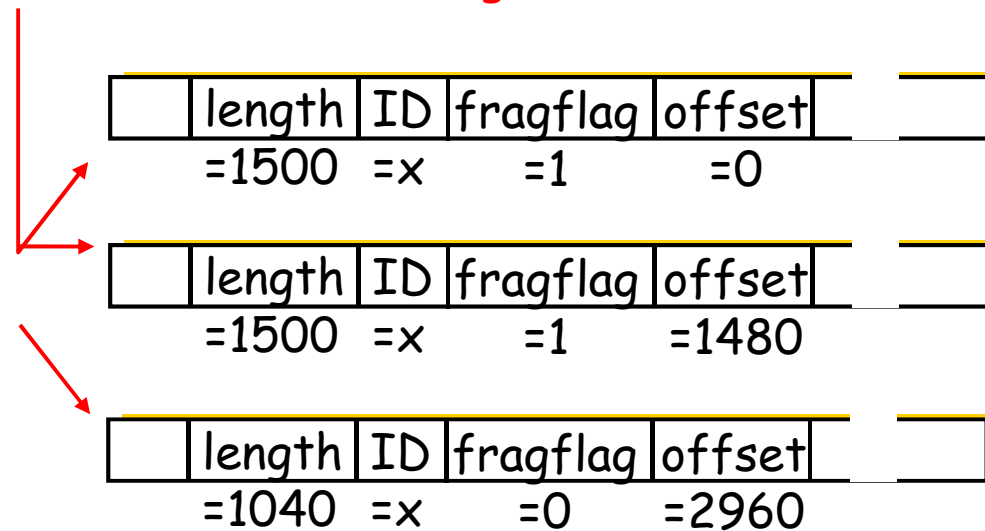
- **DF: Don't Fragment**
 - 所有机器必须能够接收小于等于**576**字节的片段
- **MF: More Fragments**
 - 除最后一个片段外的所有片段都要置**MF**位
- **片段偏移量（Fragment offset）**
 - 除最后一个片段外的所有片段的长度必须是**8**字节的倍数



IP协议 (续)

	length	ID	fragflag	offset	
	=4000	=x	=0	=0	

One large datagram becomes
several smaller datagrams





IP协议（续）

- 生存期（**Time to live**）
 - 实际实现中，**IP**分组每经过一个路由器**TTL**减**1**，为**0**则丢弃，并给源主机发送一个告警分组
 - 最大值为**255**。源主机设定初始值，**Windows**操作系统一般为**128**，**UNIX**操作系统一般为**255**，**Linux**一般为**64**
- 协议域（**Protocol**）：上层为哪种传输协议，**TCP**、**UDP**...
- 头校验和（**Header checksum**）
 - 只对**IP**分组头做校验
 - 算法：每**16**位求反，循环相加（进位加在末尾），和再求反
- 源地址(**Source address**)和目的地址(**Destination address**)



IP协议（续）

■ 选项（Options）

- 变长，长度为4字节的倍数，不够则填充，最长为40字节

Option	Description
Security	Specifies how secret the datagram is
Strict source routing	Gives the complete path to be followed
Loose source routing	Gives a list of routers not to be missed
Record route	Makes each router append its IP address
Timestamp	Makes each router append its address and timestamp

Fig. 5-46. IP options.



IP地址

- 地址组成：网络号 + 主机号
- 有类地址划分(classful addressing)

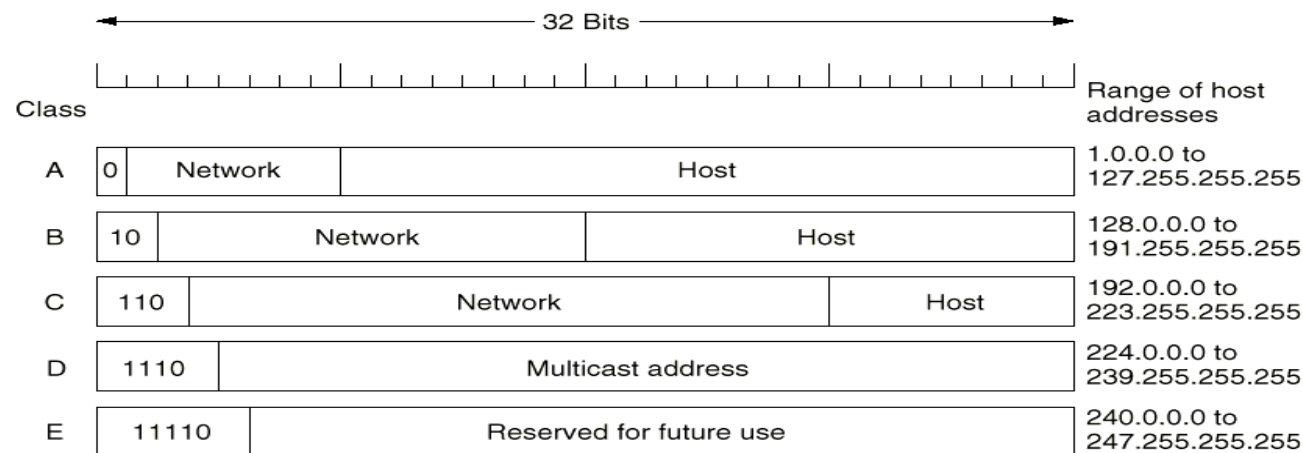


Fig. 5-47. IP address formats.

- 地址表示采用用点分隔的十进制表示法
 - 例如**166.111.68.3**



IP地址（续）

- 全**0**和全**1**有特殊含义
 - 全**0**：表示本网络或本主机
 - 全**1**：表示广播地址

0 0																														This host										
0 0										...										0 0										Host	A host on this network									
1 1																														Broadcast on the local network										
Network										1 1 1 1										...										1 1 1 1										Broadcast on a distant network
127					(Anything)																									Loopback										

Fig. 5-48. Special IP addresses.



子网

■ 子网(Subnet)

- 分而治之的思想：为了便于管理和使用，可以将网络分成若干供内部使用的部分，称为子网

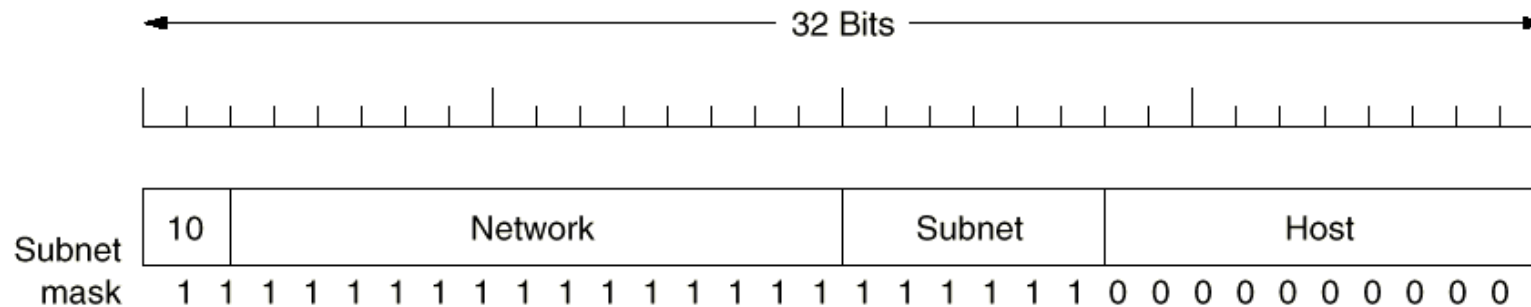


Fig. 5-49. One of the ways to subnet a class B network.



ICMP协议

- 互联网控制消息协议**ICMP(Internet Control Message Protocol)**
 - 主要用来报告错误和测试
 - 报文类型
 - **ICMP**报文封装在**IP**分组中

Message type	Description
Destination unreachable	Packet could not be delivered
Time exceeded	Time to live field hit 0
Parameter problem	Invalid header field
Source quench	Choke packet
Redirect	Teach a router about geography
Echo request	Ask a machine if it is alive
Echo reply	Yes, I am alive
Timestamp request	Same as Echo request, but with timestamp
Timestamp reply	Same as Echo reply, but with timestamp

Fig. 5-50. The principal ICMP message types.



ARP协议

- 地址解析协议**ARP** (**Address Resolution Protocol**)
 - 解决网络层地址 (**IP地址**) 与数据链路层地址 (**MAC地址**) 的映射问题
 - 工作过程
 - 建立一个**ARP**表, 表中存放(**IP地址**, **MAC地址**)对
 - 若目的主机在同一子网内, 用目的**IP地址**在**ARP**表中查找, 否则用缺省网关的**IP地址**在**ARP**表中查找。
 - 若未找到, 则发送广播分组, 目的主机收到后给出应答, **ARP**表增加一项
 - 每个主机启动时, 广播它的(**IP地址**, **MAC地址**)映射
 - **ARP**表中的表项有生存期, 超时则删除



ARP协议 (续)

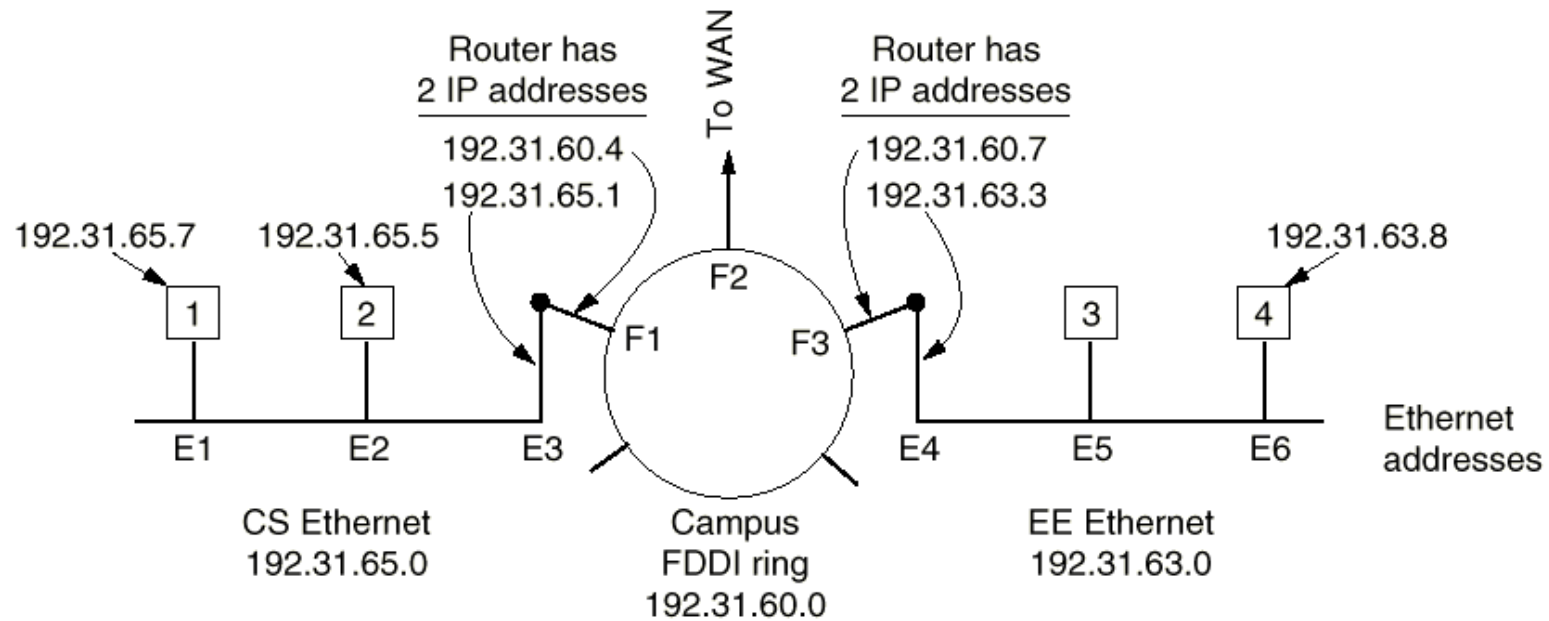


Fig. 5-51. Three interconnected class C networks: two Ethernet and an FDDI ring.



RARP 协议

- 反向地址解析协议**RARP**（**Reverse Address Resolution Protocol**）
- 解决数据链路层地址（**MAC**地址）与网络层地址（**IP**地址）的映射问题
- 主要用于无盘工作站启动
- 缺点：由于路由器不转发广播帧，**RARP**服务器必须与无盘工作站在同一子网内
- 一种替代协议**BOOTP**（使用**UDP**）

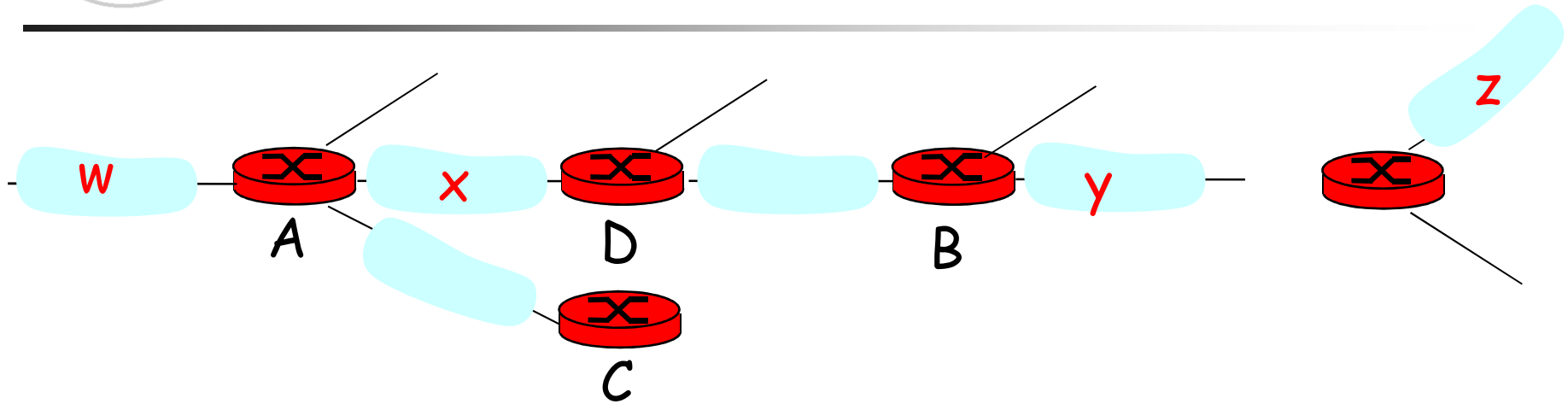


RIP协议

- **RIP (Routing Information Protocol)**
- 属于内部网关协议**IGP**
- **1982年，BSD-UNIX实现了RIP协议**
- **RIPv1, RFC1058; RIPv2, RFC1723**
- 采用距离向量算法
- 距离的衡量采用跳数 (**max = 15 hops**)
- 距离向量：每**30**秒交换一次



RIP协议 (续)



Destination Network	Next Router	Num. of hops to dest.
W	A	2
Y	B	2
Z	B	7
X	--	1
....		

Routing table in D



RIP协议 (续)

■ 故障处理

- 如果**180秒**（**6个路由声明周期**）内没有收到来自邻居的路由声明，则认为邻居/链路失效
- 经过该邻居的路由无效
- 新的路由声明发往其他邻居
- 邻居依次发出新的路由声明
- 链路失效的信息迅速传播到全网
- 使用**poison reverse**避免**ping-pong loops**
(**infinite distance = 16 hops**)



OSPF协议

- 开放最短路径优先**OSPF**（**Open Shortest Path First**）
- 属于内部网关协议**IGP**
- 支持多种距离衡量尺度，例如，物理距离、延迟、带宽等
- 采用链路状态算法
- 支持基于服务类型的路由
- 支持负载平衡
- 支持分层路由
- 适量的安全措施
- 支持隧道技术



OSPF协议 (续)

- 分层路由
 - 自治系统可以划分成区域 (**areas**)
 - 每个**AS**有一个主干 (**backbone**) 区域, 称为区域**0**
 - 所有其它区域都与主干区域相连
 - 其它区域之间不能相连
 - 一般情况下, 有三种路由
 - 区域内
 - 区域间
 - 从源路由器到主干区域
 - 穿越主干区域到达目的区域
 - 到达目的路由器
 - 自治系统间



OSPF协议（续）

■ 四类路由器，允许重叠

- 完全在一个区域内的内部路由器
- 连接多个区域的区域边界路由器**ABR**
- 主干路由器
- 自治系统边界路由器**ASBR**

■ 信息类型

Message type	Description
Hello	Used to discover who the neighbors are
Link state update	Provides the sender's costs to its neighbors
Link state ack	Acknowledges link state update
Database description	Announces which updates the sender has
Link state request	Requests information from the partner

Fig. 5-54. The five types of OSPF messages.

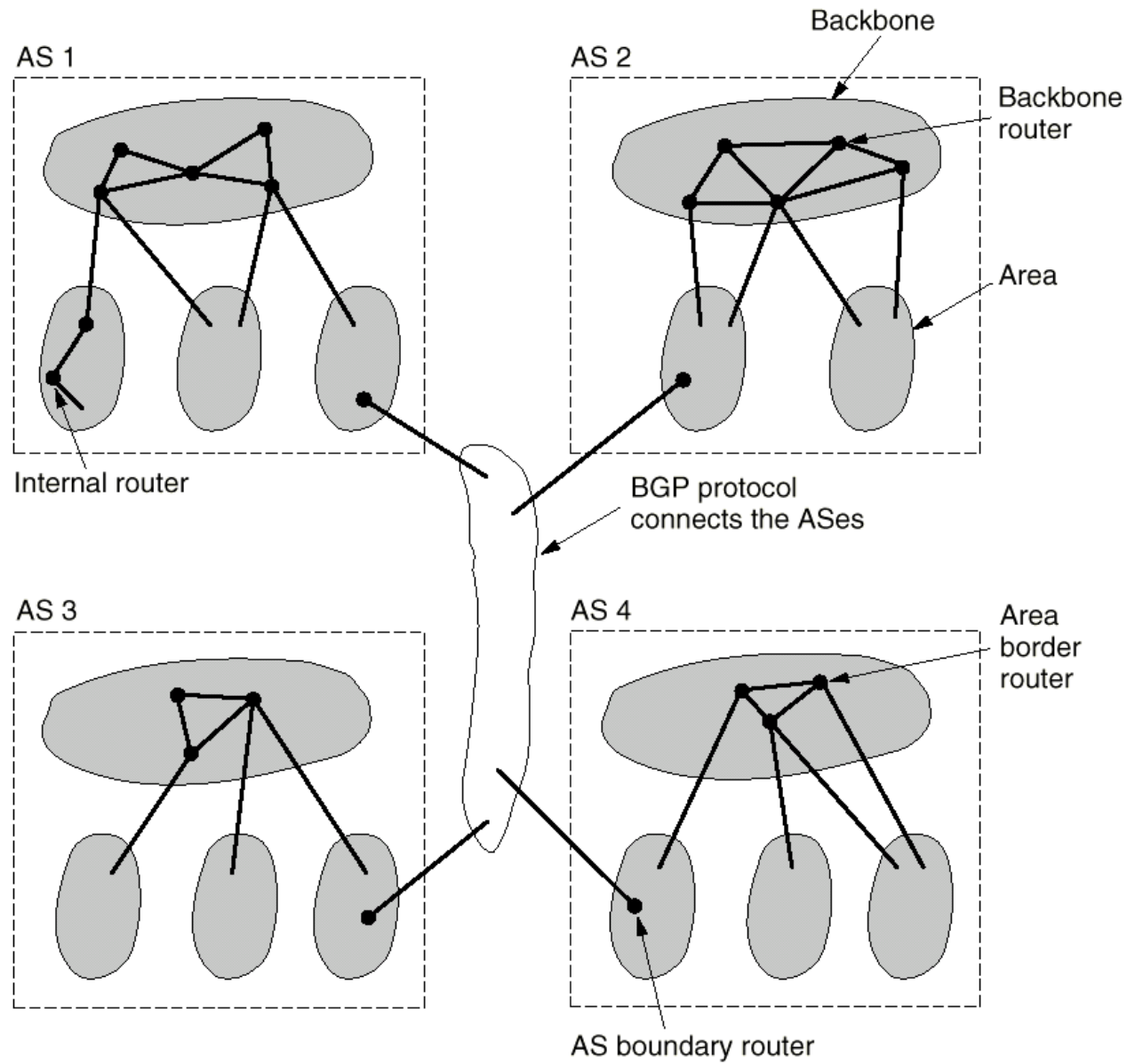


Fig. 5-53. The relation between ASes, backbones, and areas in OSPF.



BGP 协议

- 为什么域间和域内的路由有所不同？
 - 策略
 - 域间路由跨越不同管理域，要控制流量如何路由
 - 域内路由属于同一管理域，不需要定义策略
 - 规模
 - 分层路由降低了路由表的大小，减小了路由更新的流量
 - 性能
 - 域内路由：着重于性能
 - 域间路由：策略更为重要



BGP协议 (续)

- 边界网关协议**BGP** (**B**order **G**ateway **P**rotocol)
- 通过**TCP**连接传送路由信息
- 采用路径向量 (**path vector**) 算法, 路由信息中记录路径的轨迹
 - 与距离向量协议相似
 - 每个**BGP**网关向邻居广播所有通往目的地的路径
 - 比如, **Gateway X** 发送它通往**Z**的路径
 - **Path (X,Z) = X,Y1,Y2,Y3,...,Z**

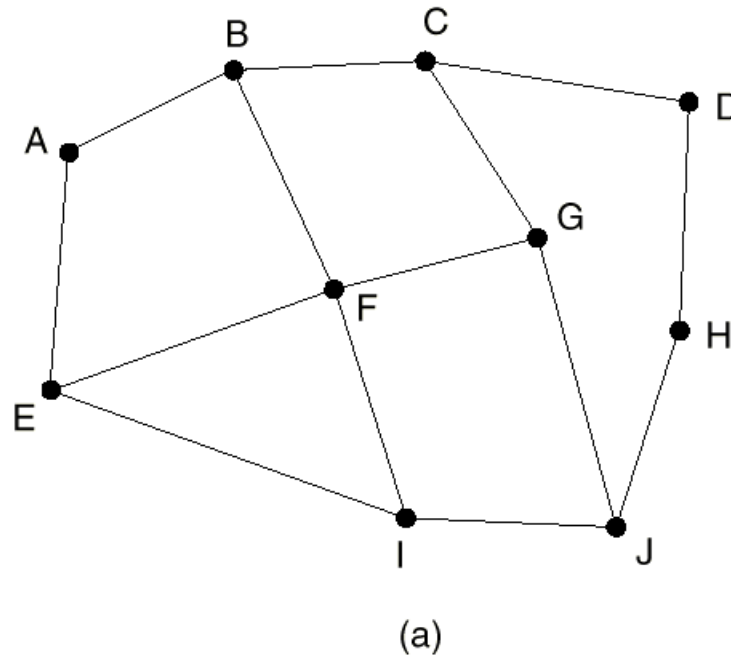


BGP协议 (续)

- **假设:** 网关**X**把它的路径发给**peer**网关**W**
- **W**可以选择是否使用**X**提供的路径, 依据是开销、策略 (例如: 不使用通过竞争对手网络的路径)、防止出现路由循环等
- 如果**W**使用**X**提供的路径, 则
 - **Path (W,Z) = w, Path (X,Z)**
- **注意:** **X**能通过其发往**peers**的路径声明来控制进入的流量。不想转发到**Z**的流量, 可以通过不通告通往**Z**的路径来实现



BGP协议 (续)



Information F receives
from its neighbors about D

From B: "I use BCD"
From G: "I use GCD"
From I: "I use IFGCD"
From E: "I use EFGCD"

(b)

Fig. 5-55. (a) A set of BGP routers. (b) Information sent to *F*.



BGP协议 (续)

■ BGP 消息

- **OPEN:** 建立与对等方的**TCP**连接并认证身份
- **UPDATE:** 声明新的路径或撤销旧的路径
- **KEEPALIVE:** 在没有**UPDATE**消息的时候保持连接有效。也用来回应**OPEN**请求
- **NOTIFICATION:** 报告上一个消息的错误，也用来关闭连接



无类域间路由CIDR

- **CIDR (Classless InterDomain Routing) 的提出**
 - **Internet**指数增长, **IPv4**地址已经分配完毕
 - **IP is rapidly becoming a victim of its own popularity**
 - 基于分类的**IP**地址空间的组织浪费了大量的地址
 - 罪魁祸首是**B**类地址
- **CIDR**
 - **RFC 1519**
 - 基本思想: 将剩余的**C**类地址分成大小可变的地址空间
 - 目前单播地址都采用**CIDR**



无类域间路由CIDR（续）

- 例如，需要**2000**个地址，则分配一个有**2048**个地址（**8个C类地址**）的连续地址块，而不是一个**B类地址**
- **RFC 1519** 改变了过去**C类地址**的分配规则，将世界分成**4个区**，每个区分配一块连续的**C类地址空间**
 - 欧洲：**194.0.0.0 ~ 195.255.255.255**
 - 北美：**198.0.0.0 ~ 199.255.255.255**
 - 中、南美：**200.0.0.0 ~ 201.255.255.255**
 - 亚太：**202.0.0.0 ~ 203.255.255.255**

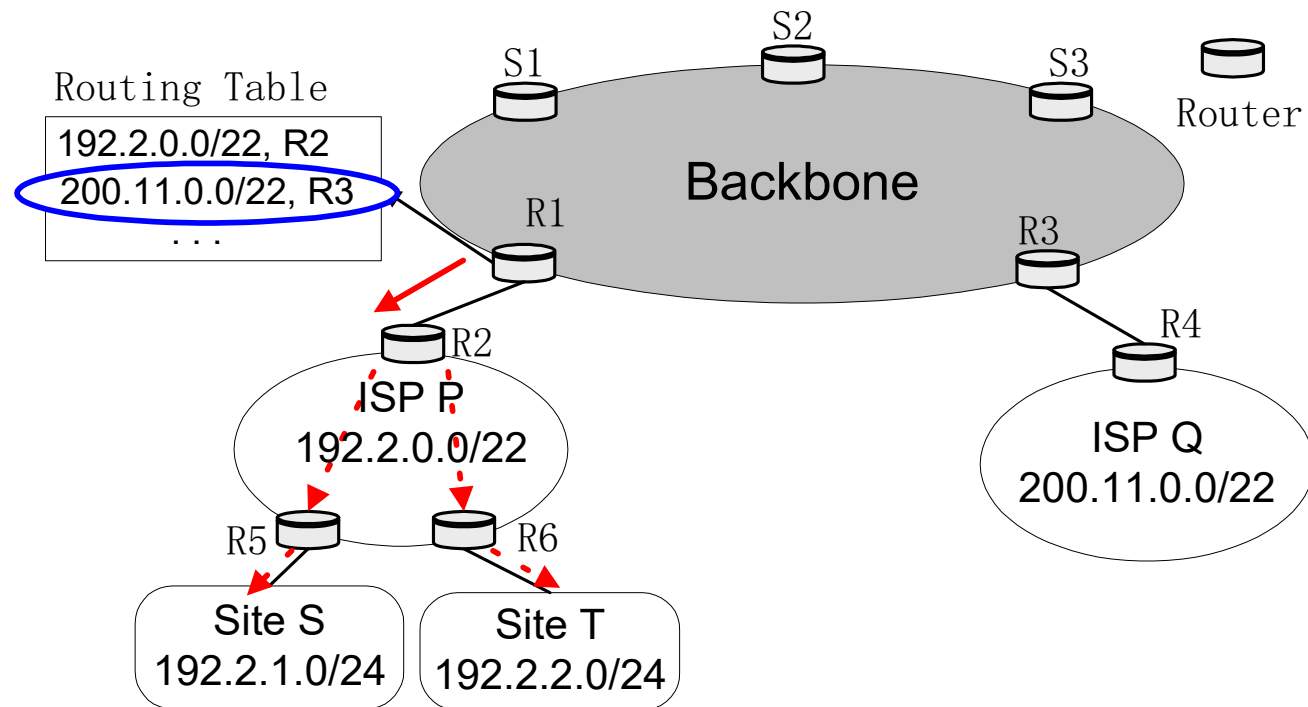


无类域间路由CIDR（续）

- 路由表中增加一个**32**位的掩码（**mask**）域
- 最长前缀匹配原则：路由查找时，若多个路由表项匹配成功，选择掩码最长（**1**比特数多）的路由表项
- **CIDR**思想可用于所有**IP**地址，没有**A**、**B**、**C**类之分。
- 地址格式：**a.b.c.d/x**，其中**x** 地址中网络号的位数

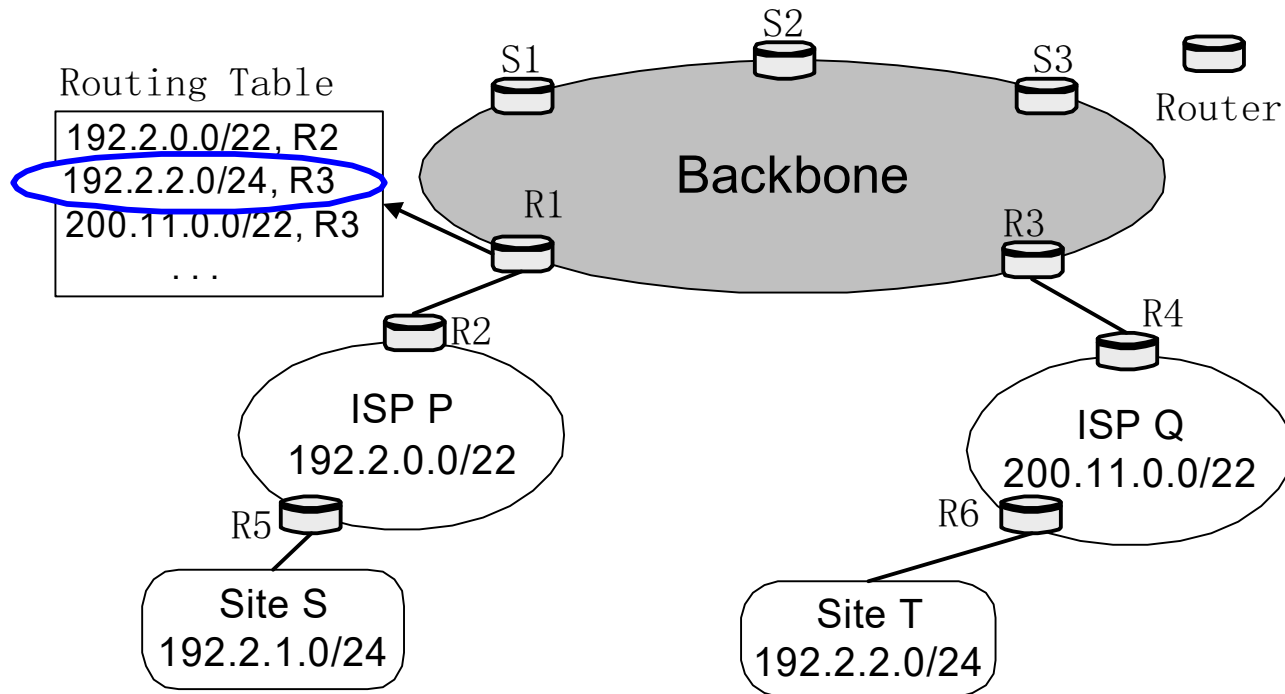


最长前缀匹配



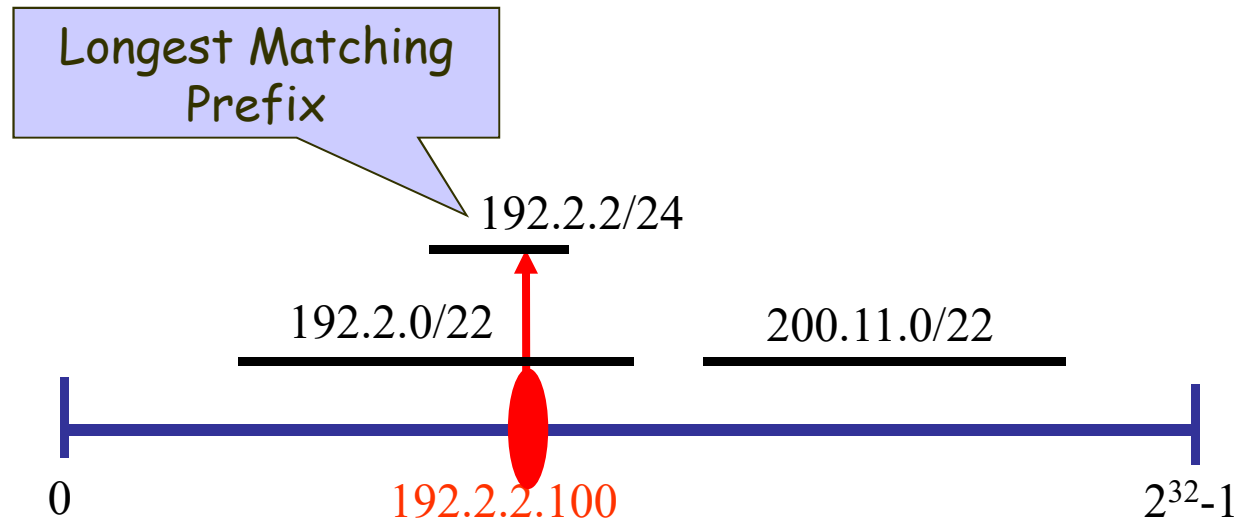


最长前缀匹配 (续)





最长前缀匹配 (续)



路由查找: 在所有前缀中寻找最长匹配



无类域间路由CIDR (续)

■ 例

- **Cambridge University** 需要**2048**个地址,
194.24.0.0 ~ 194.24.7.255, 掩码**255.255.248.0**
- **Oxford University** 需要**4096**个地址, **194.24.16.0**
~ 194.24.31.255, 掩码**255.255.240.0**;
- **Edinburgh University** 需要**1024**个地址,
194.24.8.0 ~ 194.24.11.255, 掩码**255.255.252.0**

■ 路由表内容

地址	掩码
11000010 00011000 00000000 00000000	11111111 11111111 11111000 00000000
11000010 00011000 00010000 00000000	11111111 11111111 11110000 00000000
11000010 00011000 00001000 00000000	11111111 11111111 11111100 00000000

■ 或者表示成

- **194.24.0.0/21, 194.24.16.0/20, 194.24.8.0/22**



无类域间路由CIDR（续）

- 一个目的地址为**194.24.17.4**的分组到达路由器，路由表查找过程如下：
 - **194.24.17.4**的二进制表示为
11000010 00011000 00010001 00000100
 - 该地址与第一项**Cambridge**的掩码做 **AND** 操作，得到
11000010 00011000 00010000 00000000
与**Cambridge**的地址不同，匹配不成功
 - 该地址与下一项**Oxford**的掩码做**AND**操作，得到
11000010 00011000 00010000 00000000
与**Oxford**的地址相同，匹配成功
 - 继续匹配，最后选择前缀最长的路由表项



IPv6

■ IPv6的提出

- **CIDR**仅仅是一种临时补救措施，不能从根本上解决**IP**地址空间匮乏的问题
- 移动电话、家电上网需要大量的**IP**地址
- **1990**年，**IETF**开始**IPv6**的工作，收到**21**个建议
- **1992**年，**7**个建议被讨论
- **1993**年，**3**个比较好的建议发表在**IEEE Network**上，进一步讨论、修改、结合后，形成**IPv6**
- **IPv6**与**IPv4**不兼容，但与其它**Internet**协议兼容，如**TCP**、**UDP**、**OSPF**、**BGP**、**DNS**等，但实际上还是需要开发另外一套协议栈



IPv6 (续)

■ IPv6的目标

- 即使在不能有效分配地址空间的情况下，也能支持数十亿的主机
- 减少路由表的大小
- 简化协议，使得路由器能够更快的处理分组
- 提供比**IPv4**更好的安全性
- 更多的关注服务类型，特别是实时数据
- 支持**Multicast**
- 支持移动功能
- 协议具有很好的可扩展性
- 增强安全性
- 在一段时间内，允许**IPv4**与**IPv6**共存



IPv6 vs IPv4

- 与**IPv4**相比，**IPv6**的主要变化
 - 地址变长，由**32**位变成**128**位
 - **IP**头简化，由**13**个域减少为**7**个域，提高路由器处理速度
 - 由于**IPv6**分组头定长，取消**IHL**域
 - **Protocol**域取消，用**Next header**域表示
 - 取消与分片有关的域，**IPv6**的分片方法：所有主机和路由器必须支持**1280**字节的分组，当主机发送一个大分组时，路由器不做分片，而是给主机发一个错误信息，由主机做分片
 - 取消**Checksum**域
 - 更好的支持选项功能
 - 安全性提高



IPv6 分组头

■ IPv6 分组头

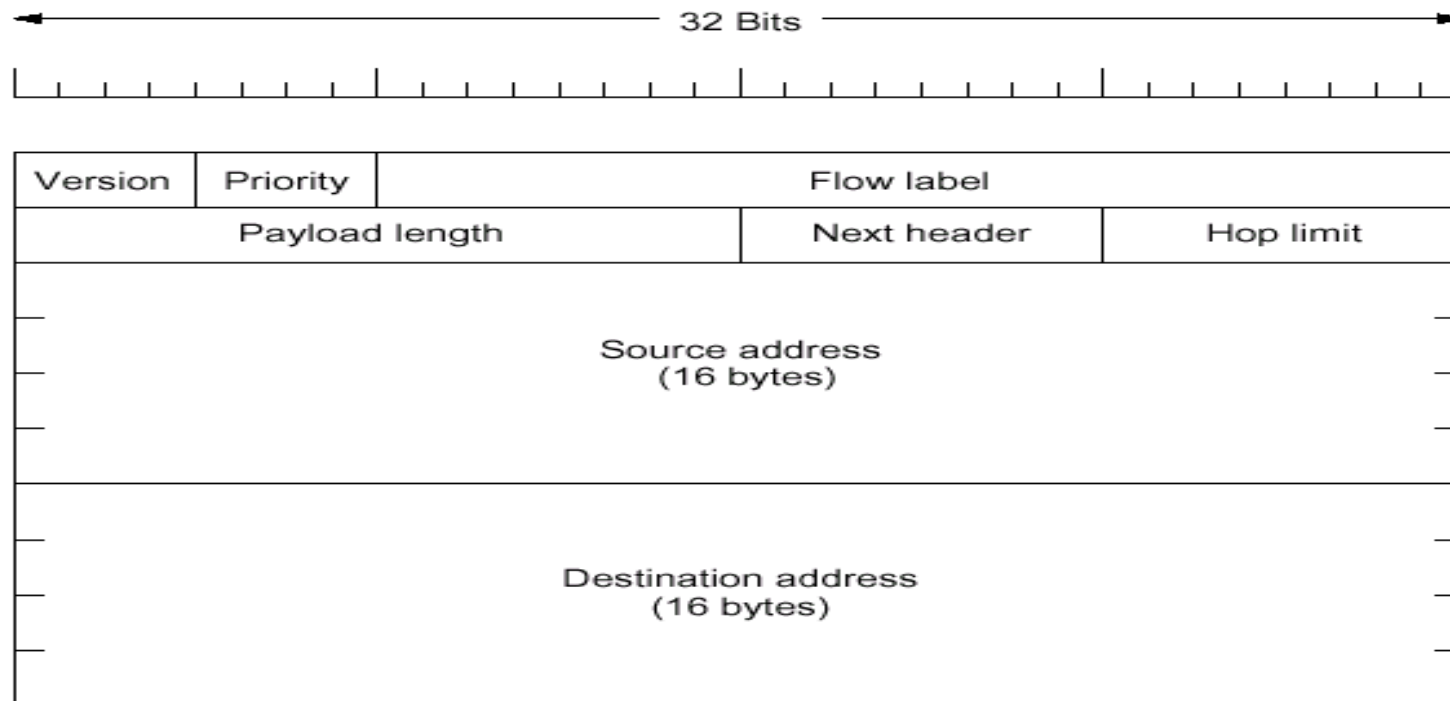


Fig. 5-56. The IPv6 fixed header (required).



IPv6 分组头 (续)

- **Version**, 值为6
- **Priority**, 用来区分源端可以流控或不能流控的分组, 值越小优先级越低
- **Flow label**, 用来允许源和目的建立一条具有特殊属性和需求的伪连接
- **Payload length**, 用来指示IP分组中40字节分组头后面部分的长度, 与IPv4的total length域不同
- **Next header**, 指示扩展分组头, 若是最后一个分组头, 则指示传输协议类型 (**TCP/UDP**)
- **Hop limit**, IP分组的最大跳数
- **Source address, destination address**, 16字节定长地址



IPv6 地址表示

- **16字节地址表示成用冒号（:）隔开的8组，每组4个16进制位，例如，**
8000:0000:0000:0000:0123:4567:89AB:CD
EF
- 由于有很多“0”，有三种优化表示
 - 打头的“0”可以省略，**0123**可以写成**123**
 - 一组或多组**16**个“0”可以被一对冒号替代，但是一对冒号只能出现一次。上面的地址可以表示成**8000::123:4567:89AB:CDEF**
 - **IPv4**地址可以写成一对冒号和用“.”分隔的十进制数，例如**::192.31.20.46**



IPv6扩展分组头

- 目前定义了六种类型的扩展分组头
- **hop-by-hop header**，用来指示路径上所有路由器必须检查的信息
- **routing header**，列出路径上必须要经过的路由器
- **fragmentation header**，与IPv4相似，扩展头中分组包括IP分组标识号、片段号和判断是否还有片段的位，只有源主机可以分片

Extension header	Description
Hop-by-hop options	Miscellaneous information for routers
Routing	Full or partial route to follow
Fragmentation	Management of datagram fragments
Authentication	Verification of the sender's identity
Encrypted security payload	Information about the encrypted contents
Destination options	Additional information for the destination

Fig. 5-58. IPv6 extension headers.



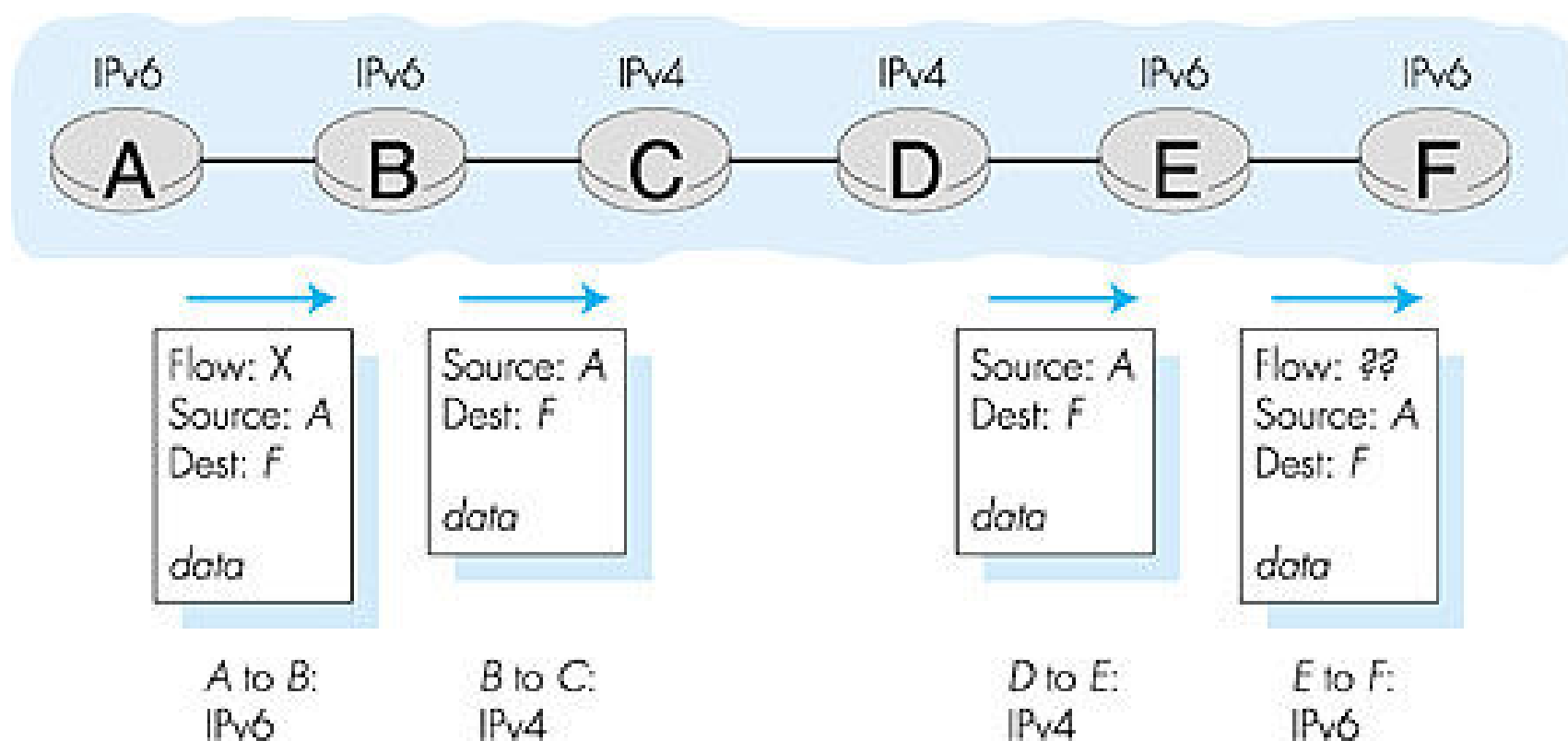
从IPv4往IPv6的过渡

- 所有路由器不可能同时升级
 - 没有一个确定的期限 (**not like Y2K**)
 - **IPv4和IPv6**路由器混合的网络如何操作?
- 有三种方案（两种技术）
 - 双栈：实现**IPv4/v6**两套协议栈，主机根据**DNS**返回的结果或对方发来报文的版本号决定采用哪个协议，路由器根据收到**IP**分组的版本号决定采用哪个协议
 - 翻译：有些路由器同时运行两套协议栈，可以在这两套之间进行协议翻译和地址翻译，例如**NAT-PT**（已废弃）
 - 隧道：**IPv6**的报文作为**IPv4**报文的净负荷在**IPv4**网络中传输



翻译

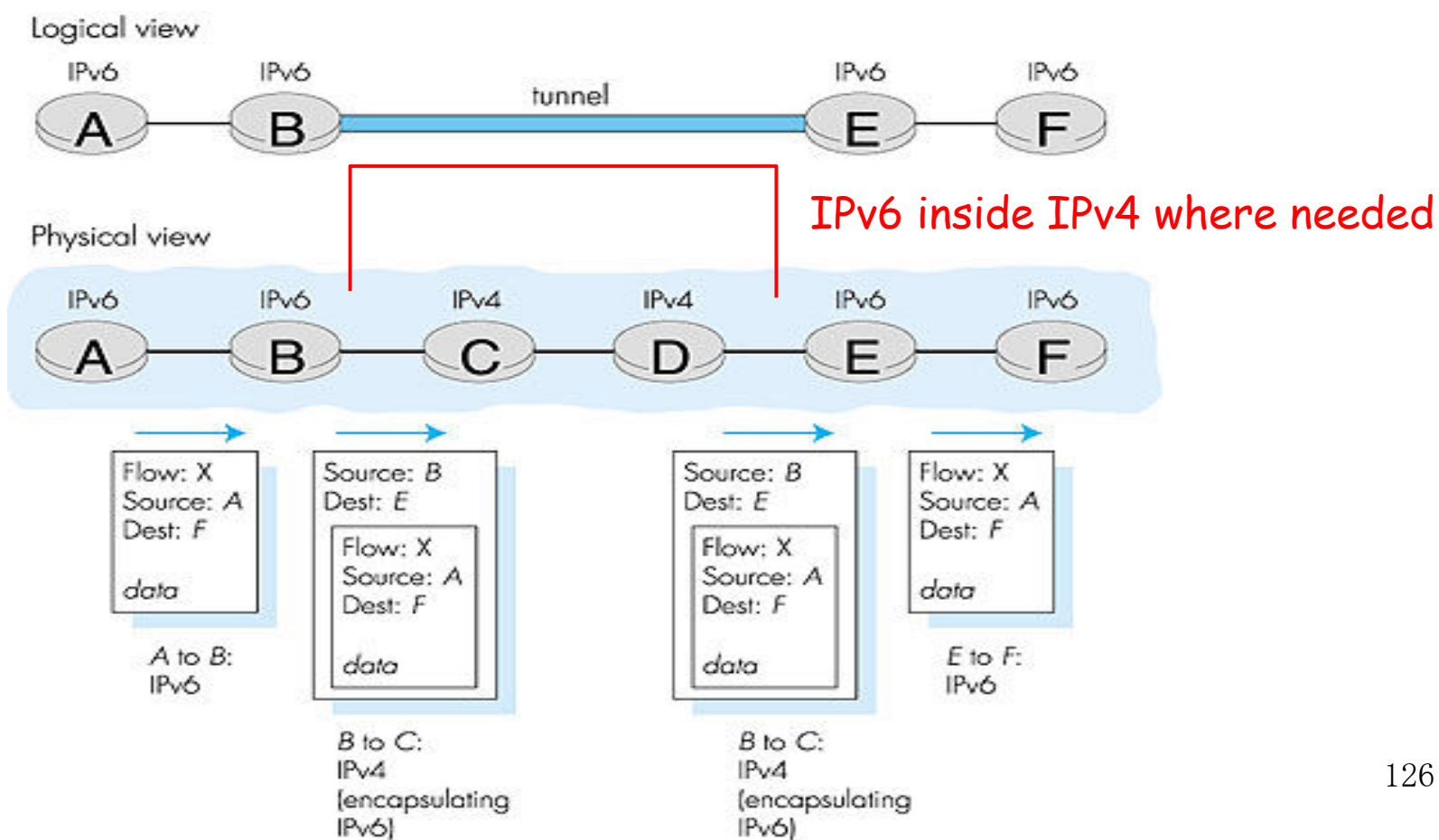
■ NAT-PT





隧道

■ 隧道





互联网协议小结

- **IPv4协议**
 - 分片与重组
 - **IPv4地址**
- **ICMP协议**
 - **ICMP**报文封装在**IP**分组中
- **ARP协议**
 - **ARP**协议工作过程
- **RARP协议**
 - **RARP**与**BOOTP**的区别
- **RIP**
 - 内部网关协议
 - 距离向量算法
 - 基于**UDP**
- **OSPF**
 - 内部网关协议
 - 链路状态算法
 - 分层思想
 - 四种类型路由器
- **BGP**
 - 外部网关协议
 - 路径向量算法
 - 基于**TCP**
- **CIDR**
 - **CIDR**的思想
 - **IP**地址分配和掩码配置
 - 最长匹配原则
- **IPv6**
 - **IPv6**与**IPv4**的主要区别
 - **IPv6**地址
 - **IPv4/IPv6**过渡

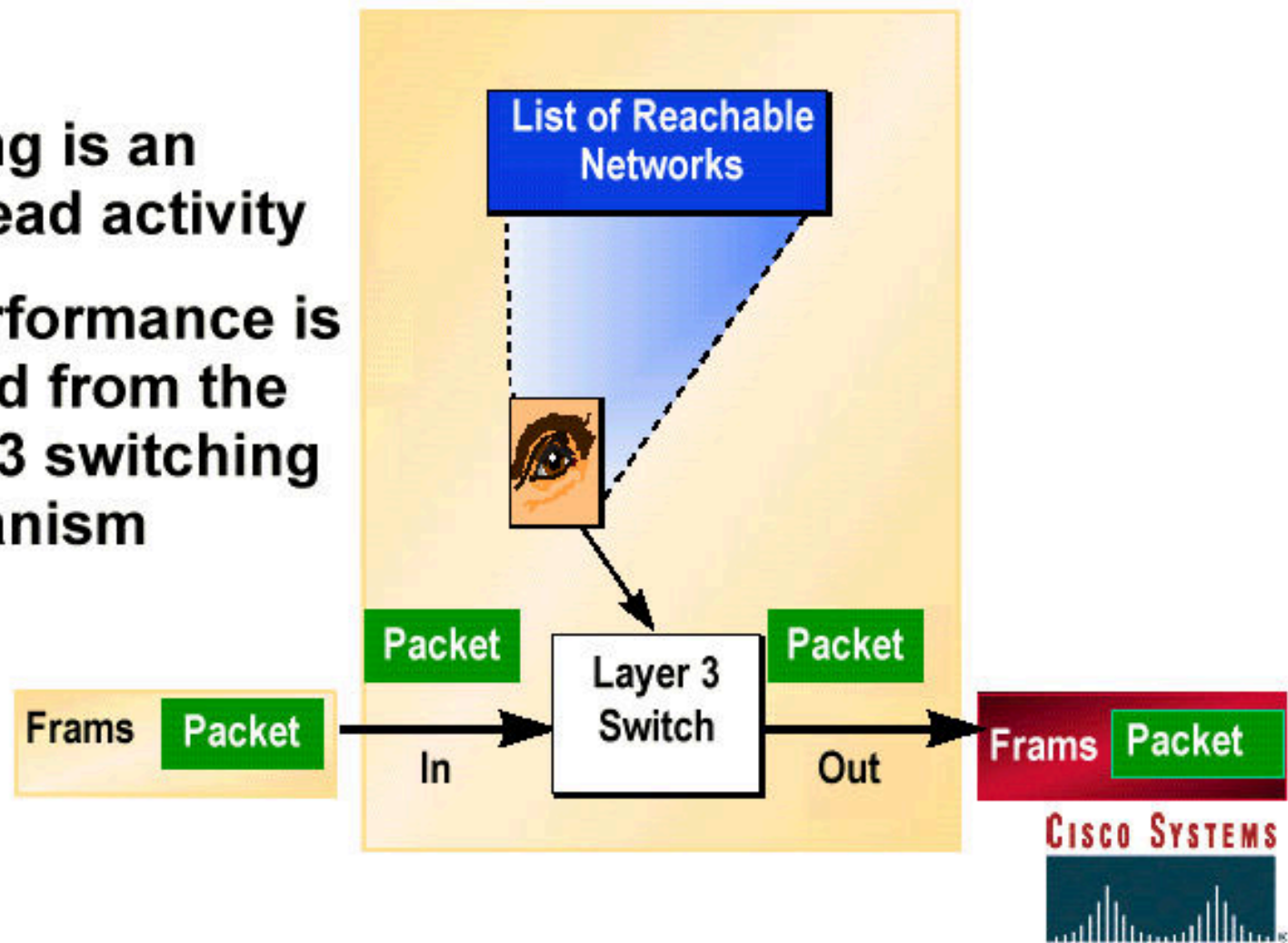
路由器体系结构和关键技术





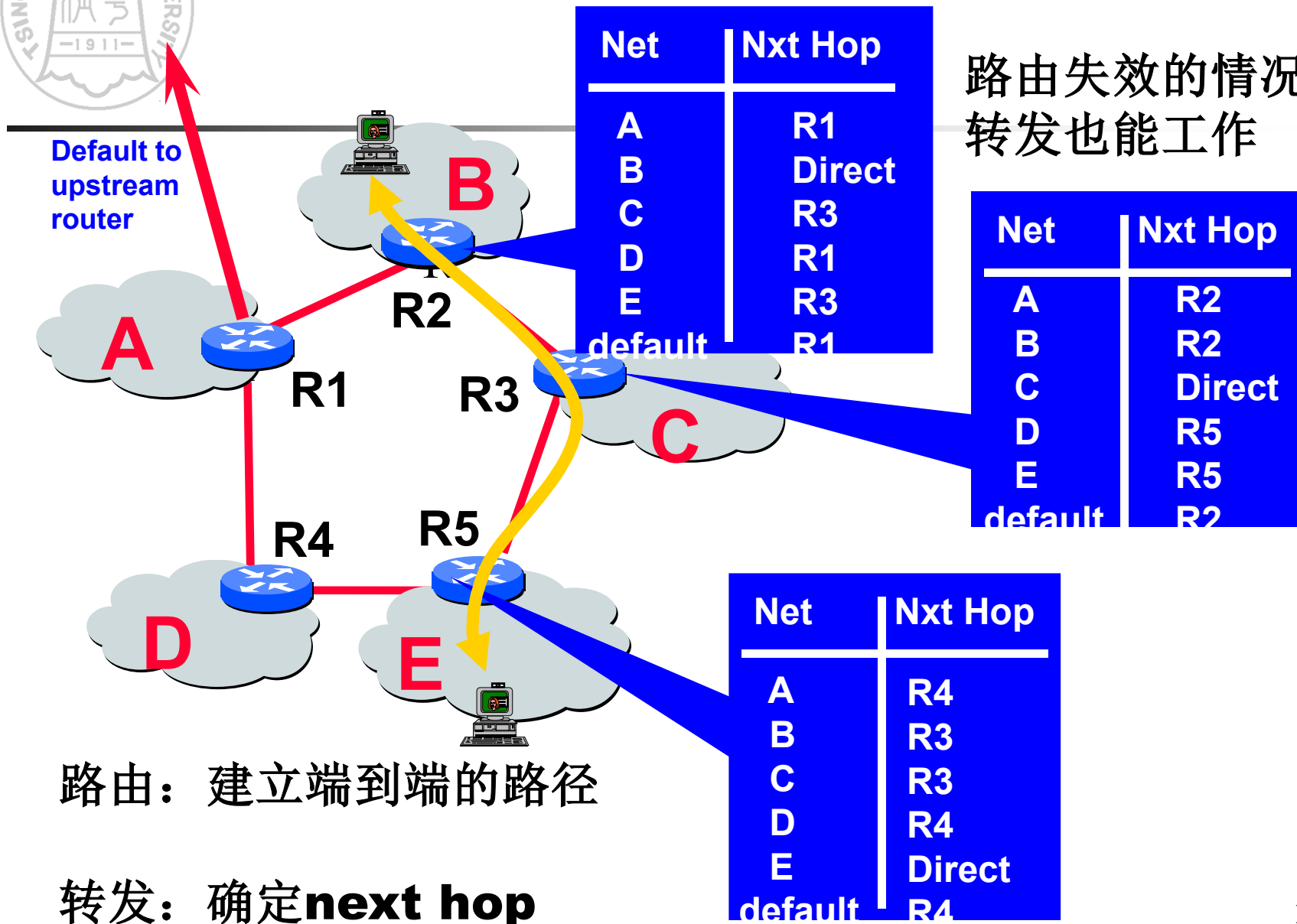
What Is a Router?

- Routing is an overhead activity
- All performance is derived from the Layer 3 switching mechanism



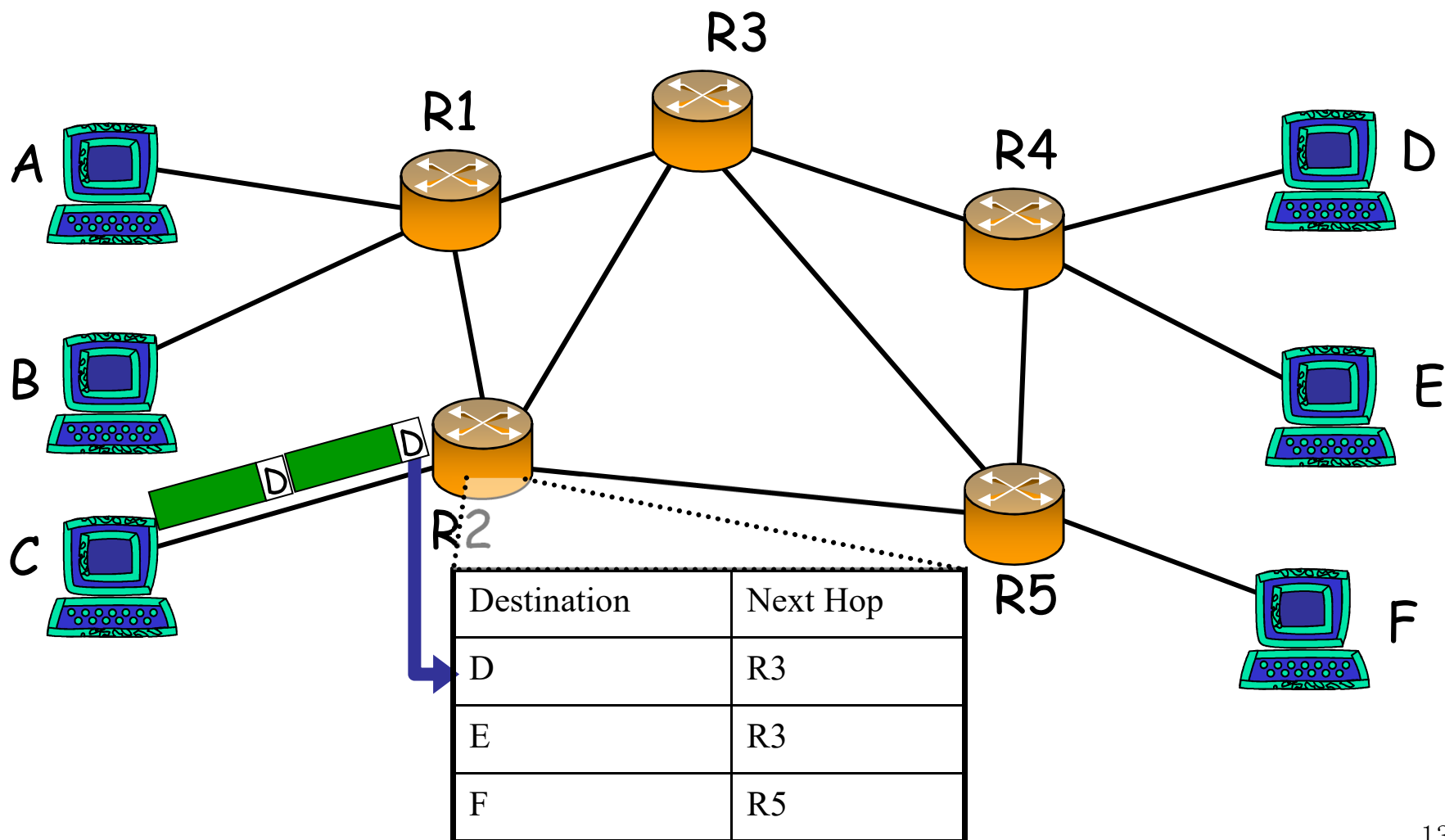


什么是路由？什么是转发？



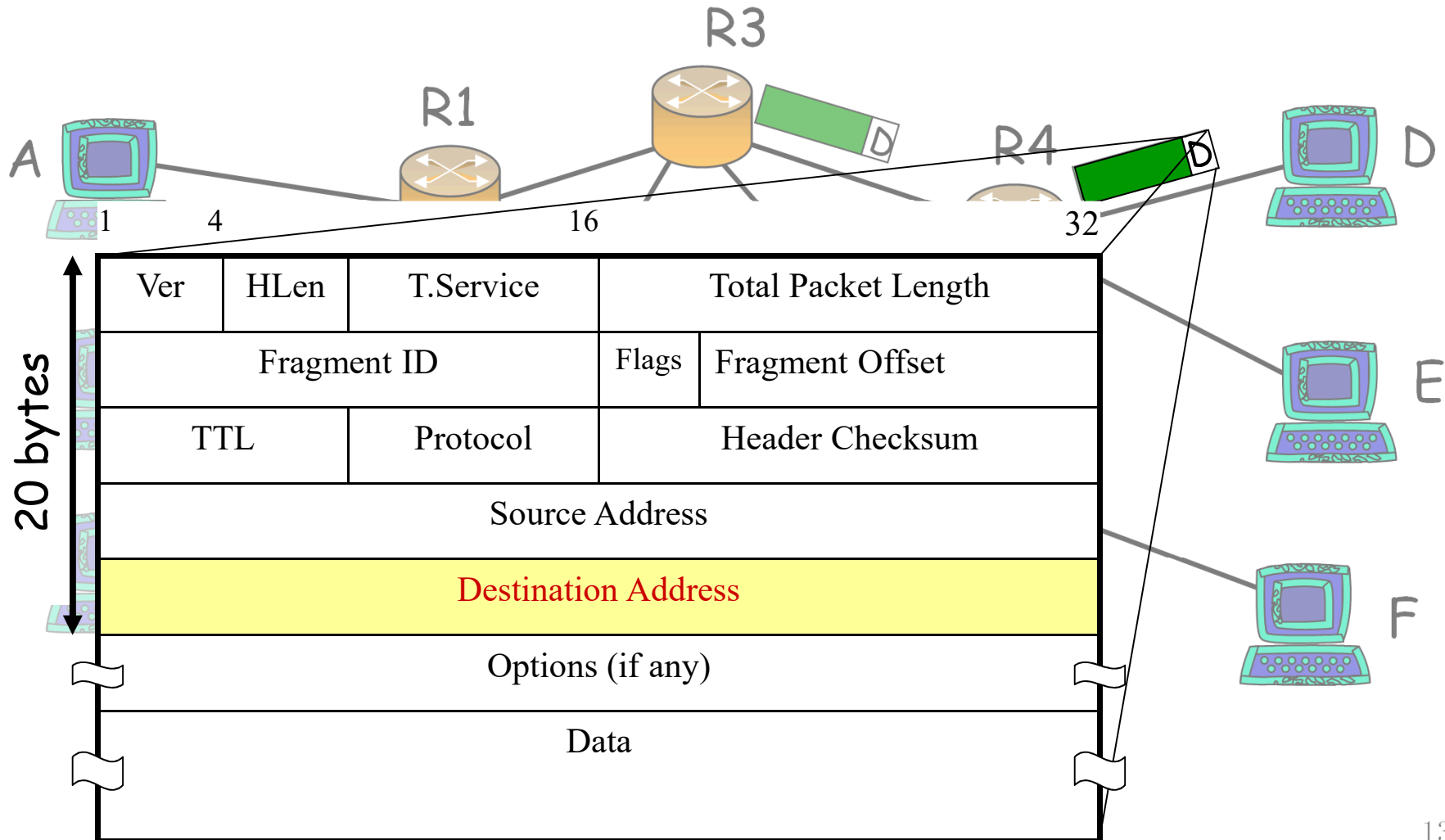


什么是路由？什么是转发？（续）



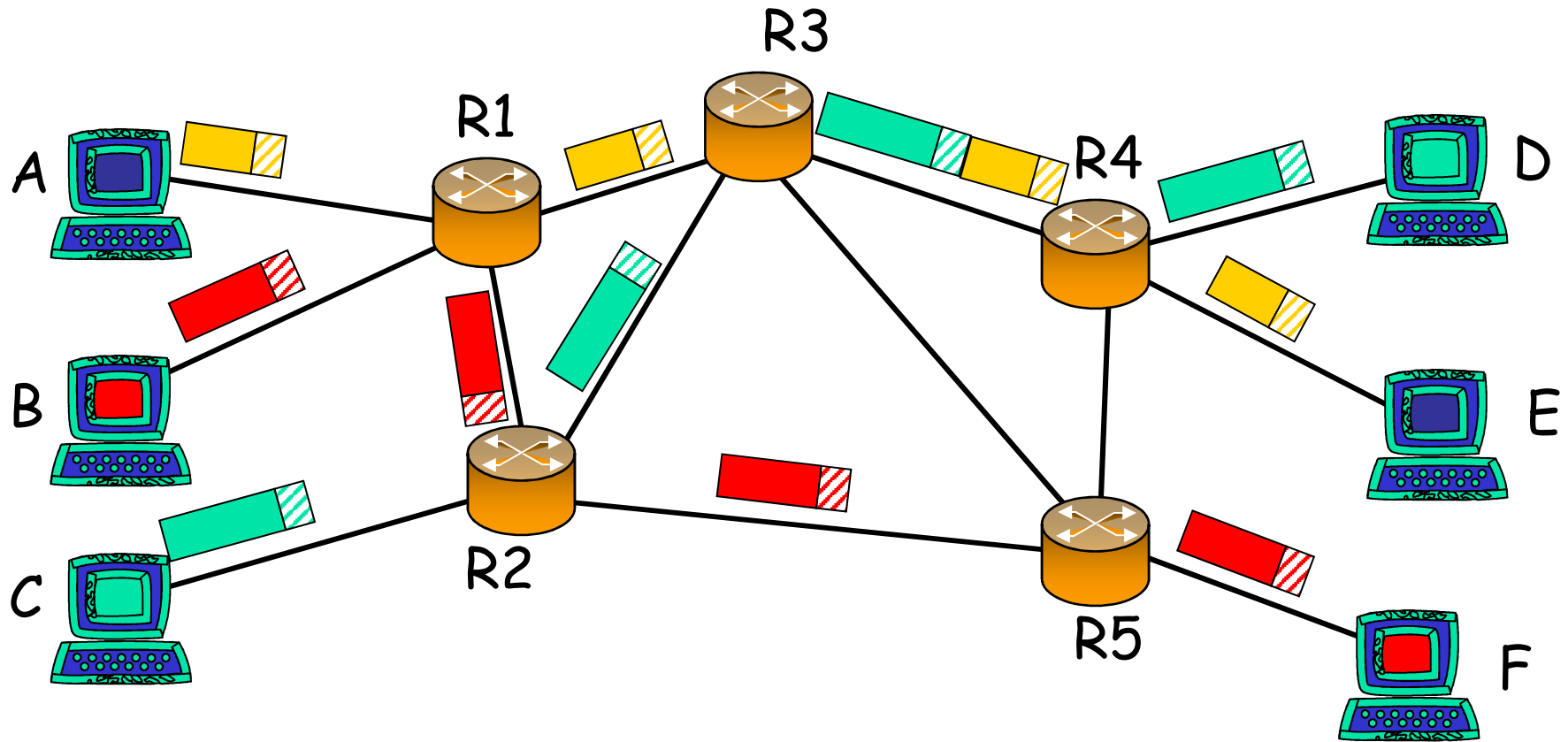


什么是路由？什么是转发？（续）





什么是路由？什么是转发？（续）





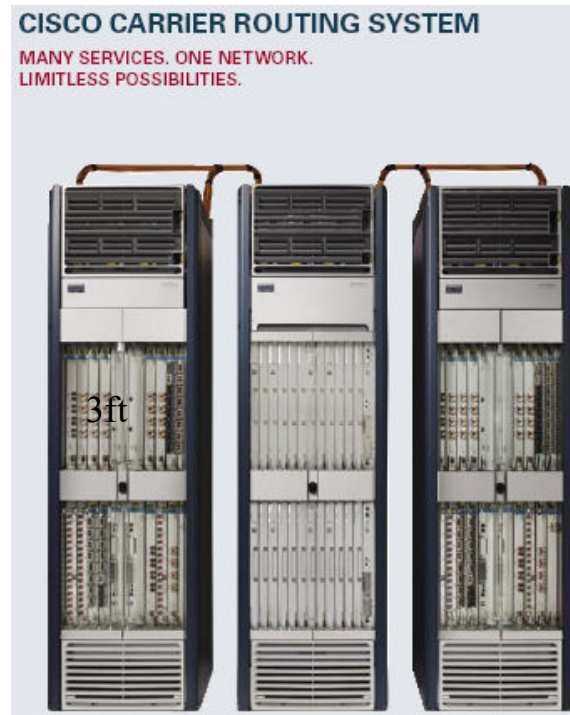
路由器是什么样子

Cisco GSR 12416



Capacity: 160Gb/s
Power: 4.2kW

Cisco CRS1



Juniper M160



Capacity: 80Gb/s
Power: 2.6kW



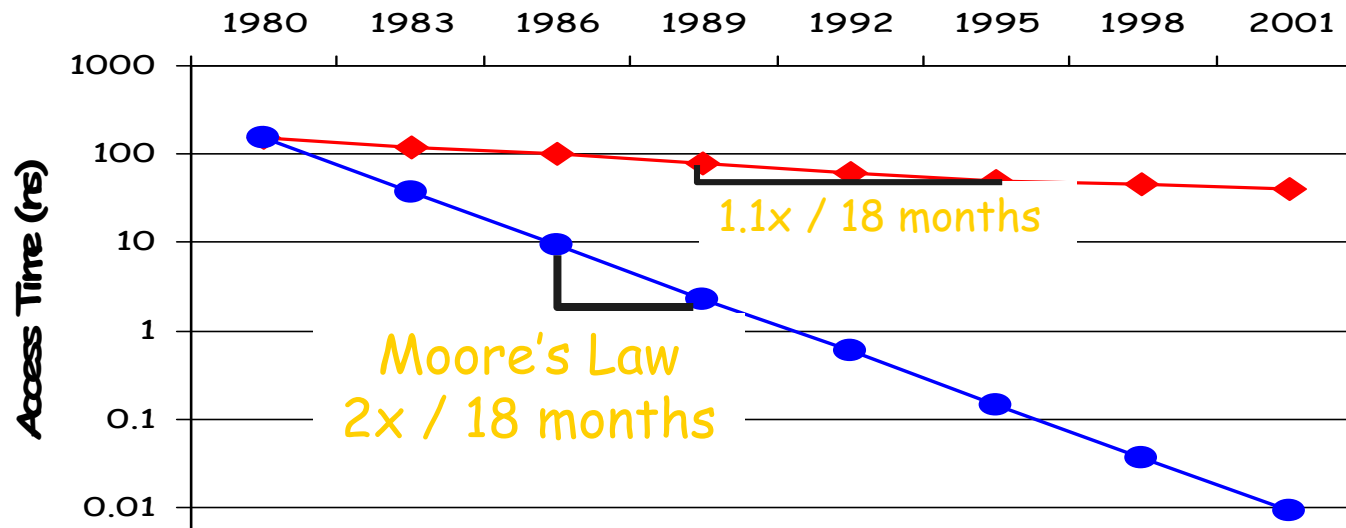
为什么设计制造高速路由器有很大的难度？

1. 内存速度是瓶颈，没法跟上摩尔定律
2. 对路由器性能要求的提升速度超过了摩尔定律，带宽增加速度超过了处理能力增加的速度



为什么设计制造高速路由器有很大的难度？（续）

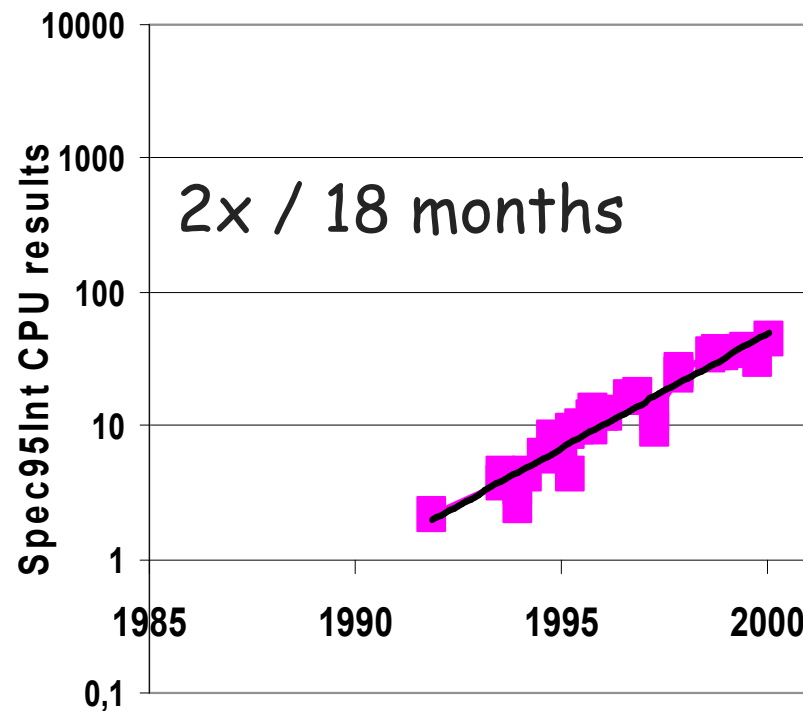
1. **It's hard to keep up with Moore's Law:**
 - The bottleneck is memory speed.
 - Memory speed is not keeping up with Moore's Law



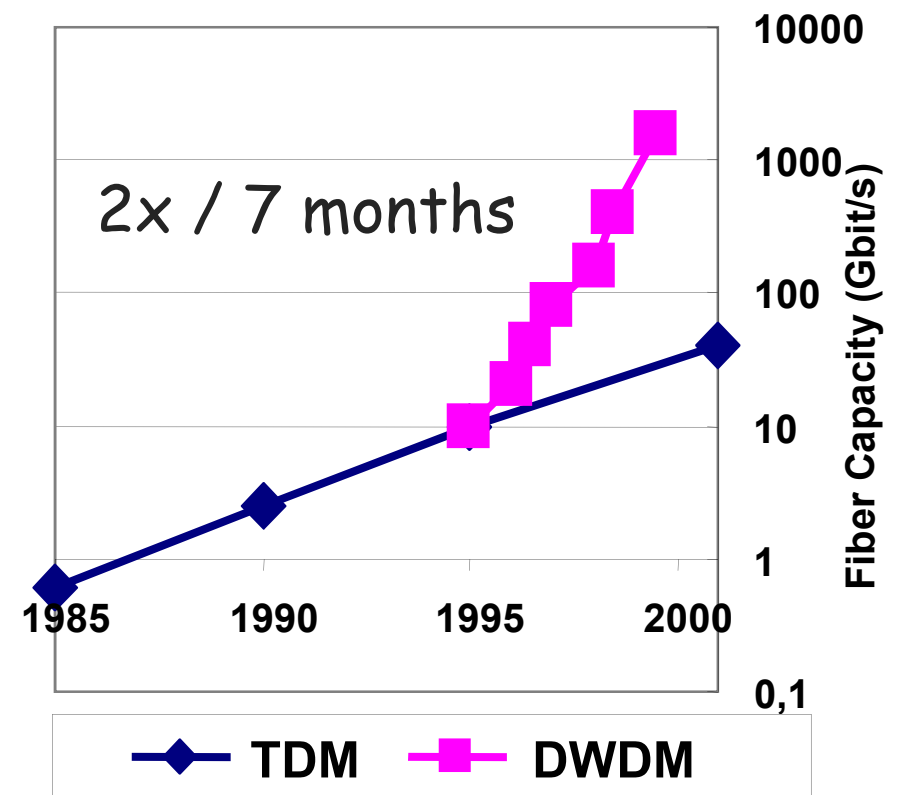


摩尔定律和光纤定律

分组处理能力



链路速度



Source: SPEC95Int & David Miller, Stanford.



路由器的性能增长超过了摩尔定律

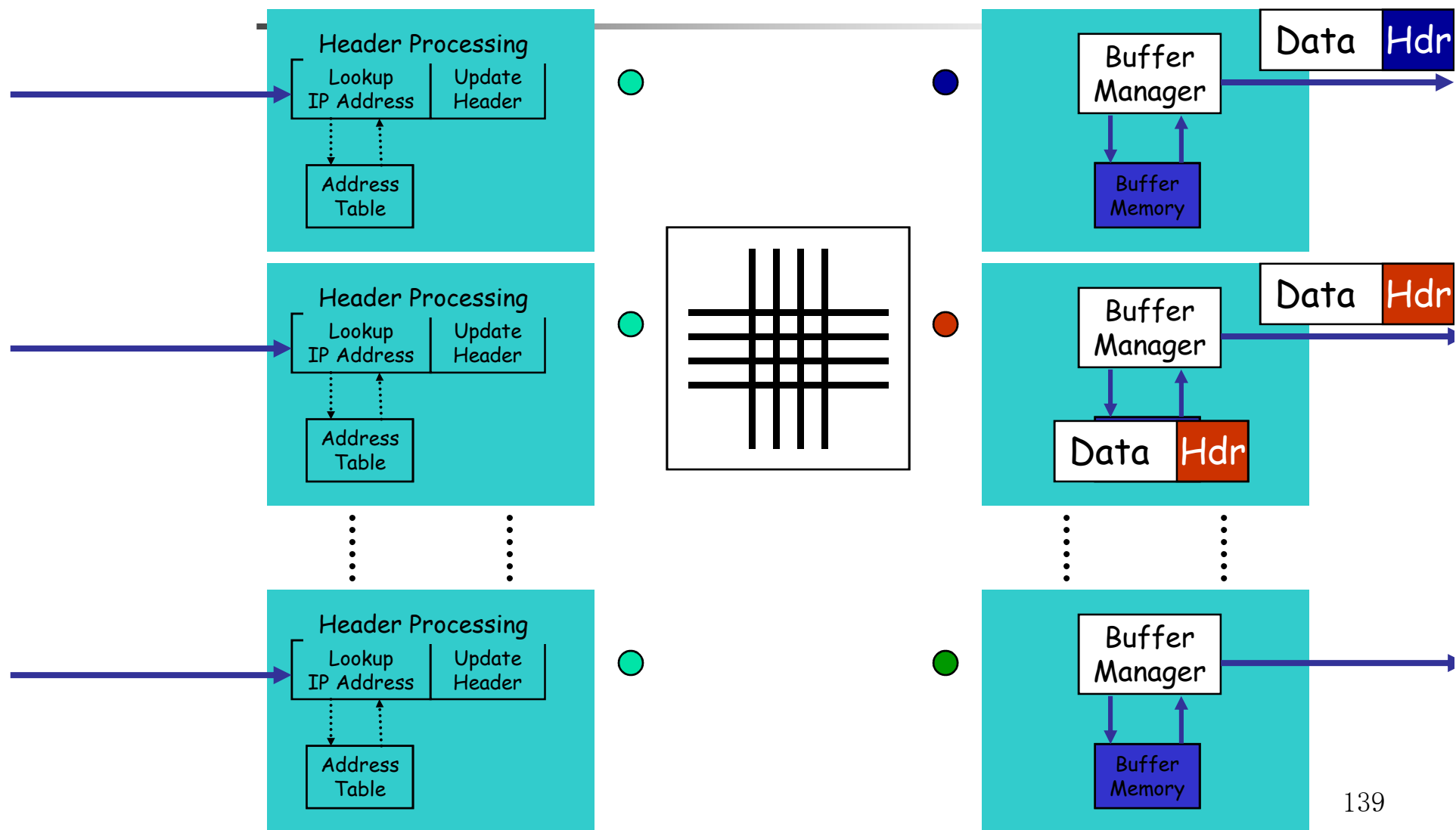
商业路由器的容量增长

- **Capacity 1992 ~ 2Gb/s**
- **Capacity 1995 ~ 10Gb/s**
- **Capacity 1998 ~ 40Gb/s**
- **Capacity 2001 ~ 160Gb/s**
- **Capacity 2003 ~ 640Gb/s**

平均增长率: 2.2x / 18 months.



通用路由器框架



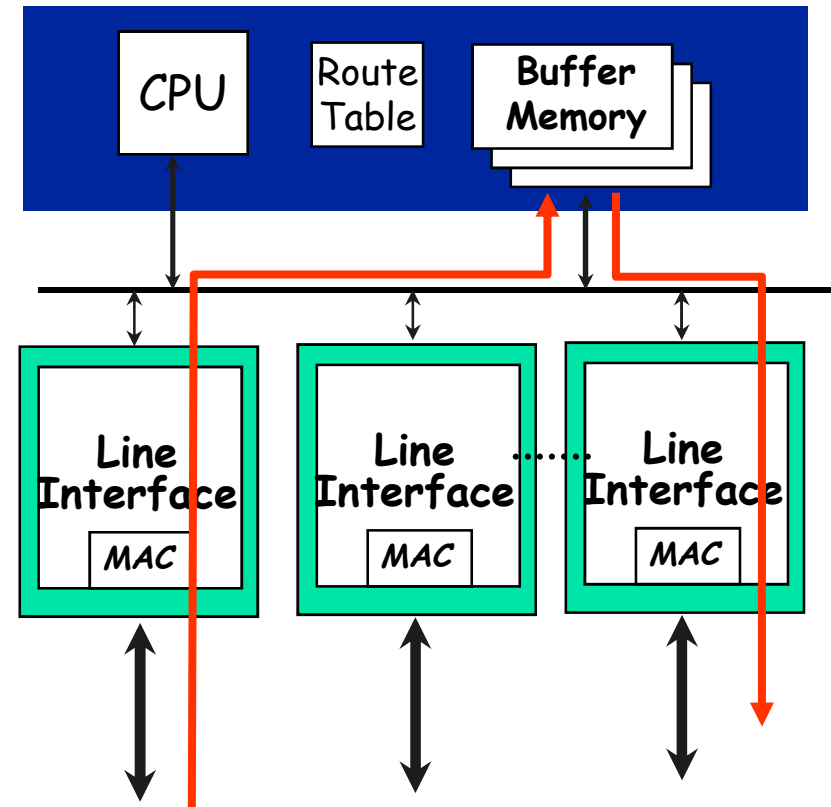
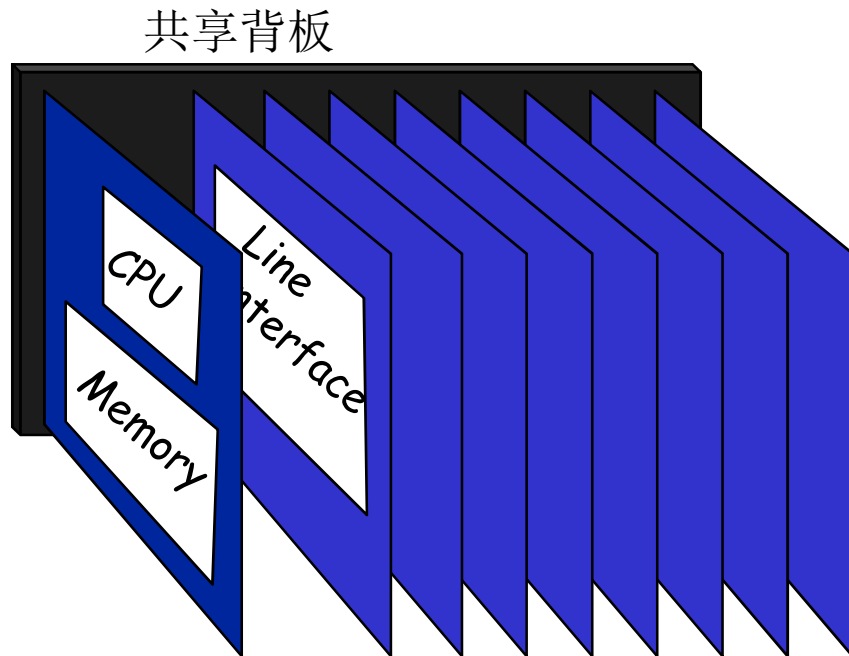


路由器体系结构的发展

- 单总线，单处理器
 - 传统的计算机结构，处理器成为处理瓶颈
- 单总线，多处理器，每个接口卡上有一个处理器，主处理器负责协调。
 - 分组转发由本地处理器完成，减少总线负担
- 交换结构 + 专用硬件 (**ASIC, FPGA, NP**)
 - 交换结构实现无阻塞转发
 - 硬件实现对**IP**分组的处理和路由查找
- 可扩展路由器 (**Internet Routing Matrix**)
 - 多结点点级连，类似并行计算机



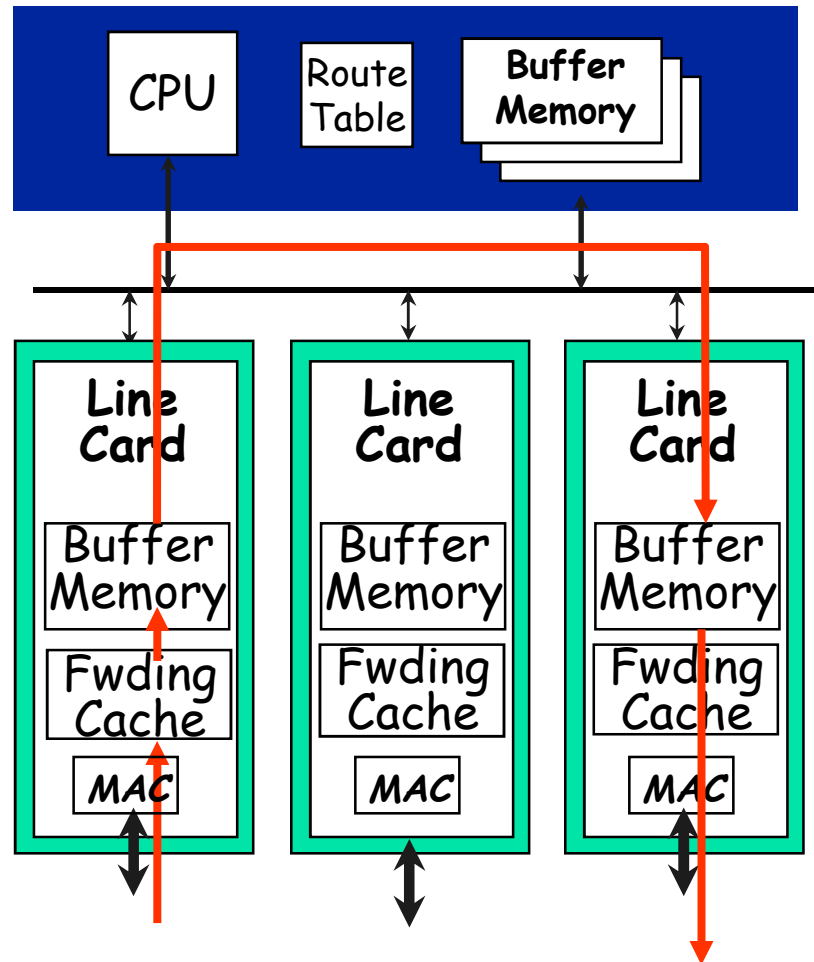
第一代路由器



Typically $<0.5\text{Gb/s}$ aggregate capacity



第二代路由器

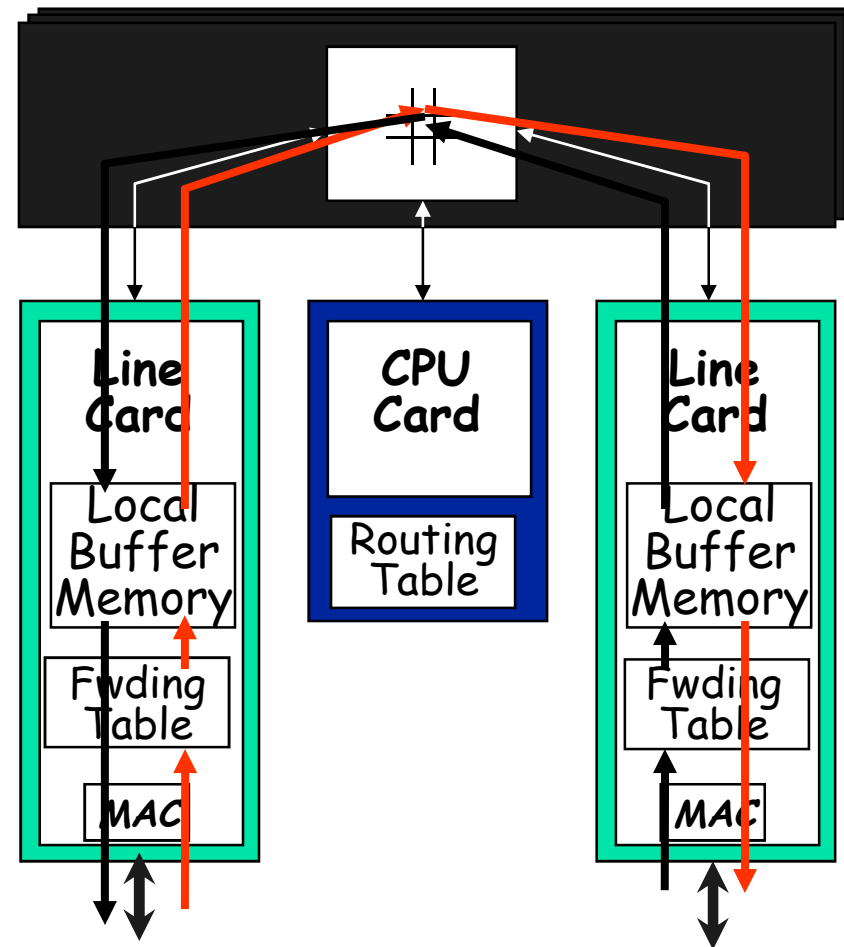
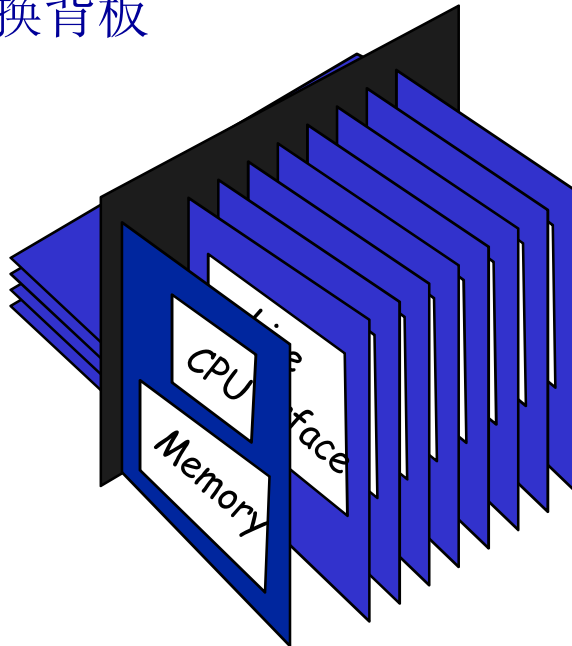


Typically <5Gb/s aggregate capacity



第三代路由器

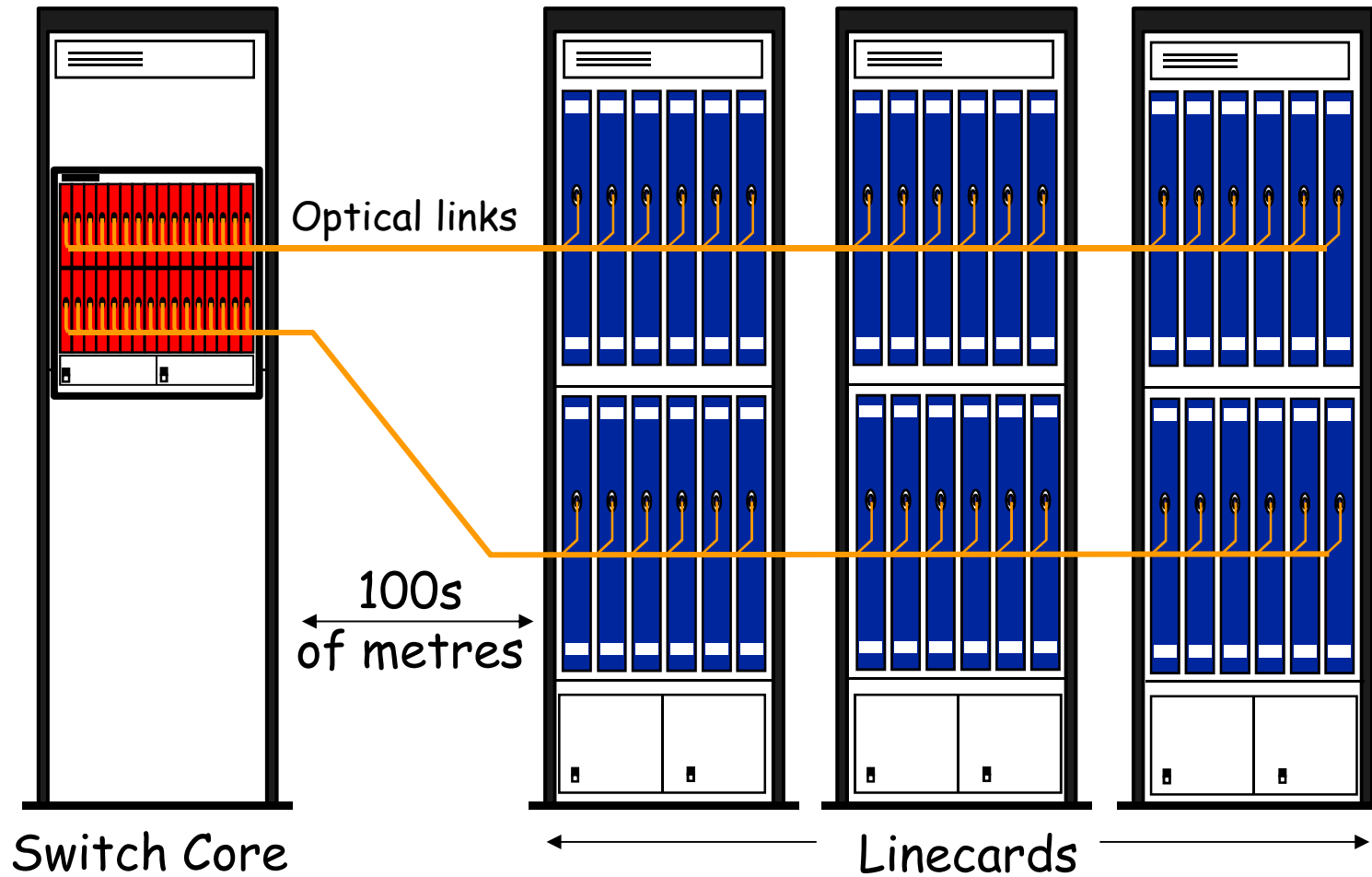
交换背板



Typically <1Tb/s aggregate capacity



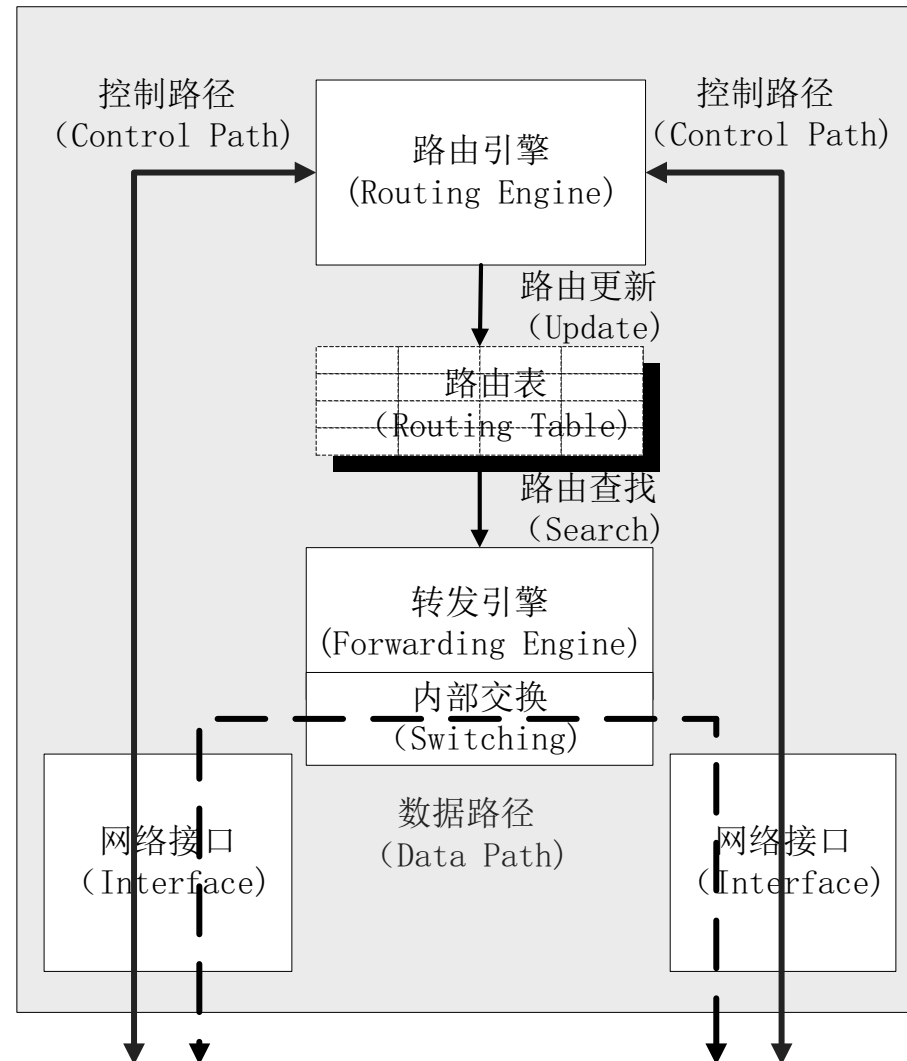
第四代路由器



1 - 几百Tb/s routers in development



路由器基本结构



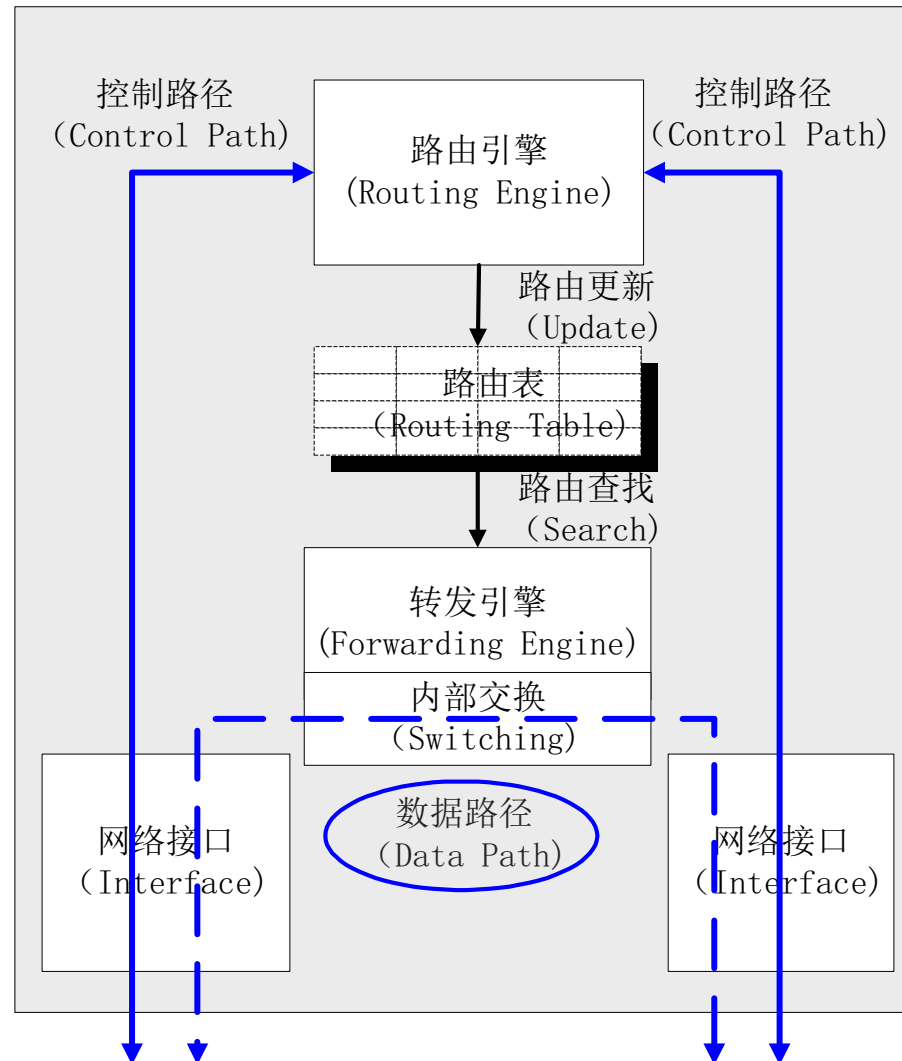


路由器基本结构（续）

- 网络接口
 - 完成网络报文的接收和发送
- 转发引擎
 - 负责决定报文的转发路径
- 内部交换
 - 为多个网络接口以及路由引擎模块之间的报文数据传送提供高速的数据通路
- 路由引擎
 - 由运行高层协议（特别是路由协议）的内部处理模块组成
- 路由表
 - 路由表分组含了能够完成网络报文正确转发的所有路由信息，它在整个路由器系统中起着承上启下的作用



分组处理路径



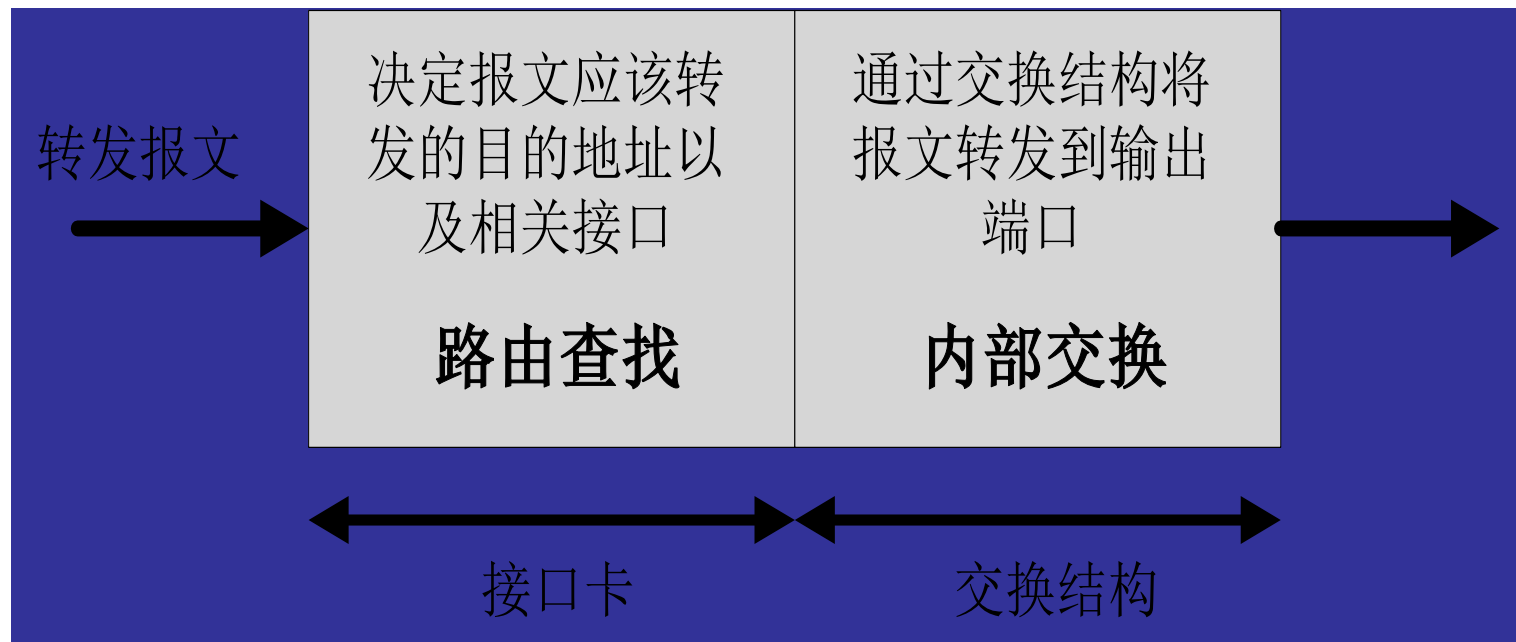


分组处理路径（续）

- 路由器提供了两种不同的报文处理路径：
 - 数据路径：处理目的地址不是本路由器而需要转发的报文，因此数据路径是整个路由器的关键路径，它的实现好坏直接影响着路由器的整体性能
 - 控制路径：处理目的地址是本路由器的高层协议报文，特别是各种路由协议报文。虽然控制路径不是路由器的关键路径，但是它负责完成路由信息的交互，从而保证了数据路径上的报文沿着最优的路径转发。



数据路径的工作流程



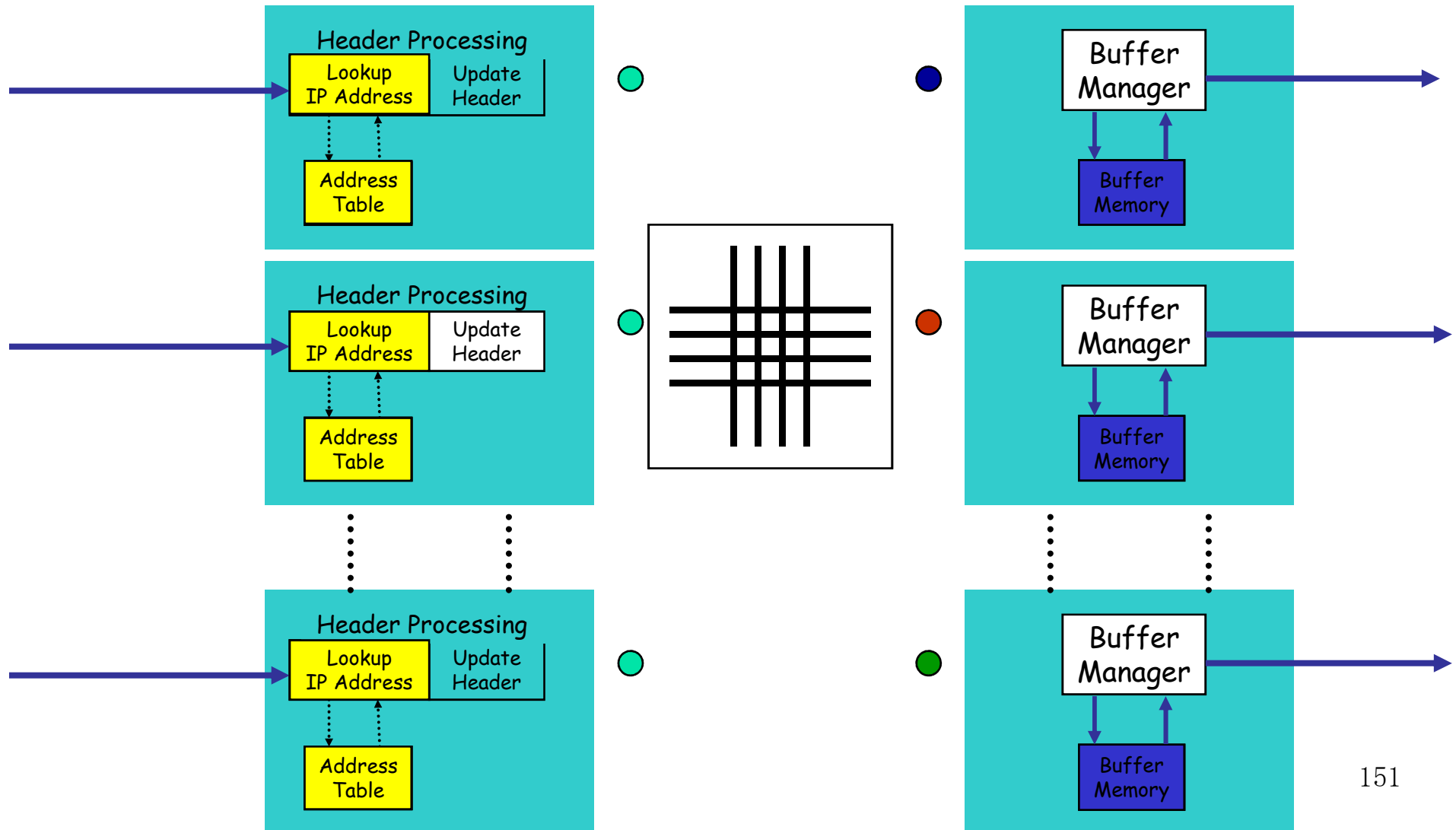


数据路径的工作流程

- **RFC1812**规定**IP**路由器必须完成两个基本功能
 - 首先路由器必须能够对每个到达本路由器的报文做出正确的转发决策，决定报文向哪一个下一跳路由器转发。为了进行正确的转发决策，路由器需要在转发表中查找能够与转发报文目的地址最佳匹配的表项，这个查找过程被称为路由查找（**Route Lookup**）
 - 其次路由器在得到了正确的转发决策之后必须能够将报文从输入接口向相应的输出接口传送，这个过程被称为内部交换过程（**Switching**）

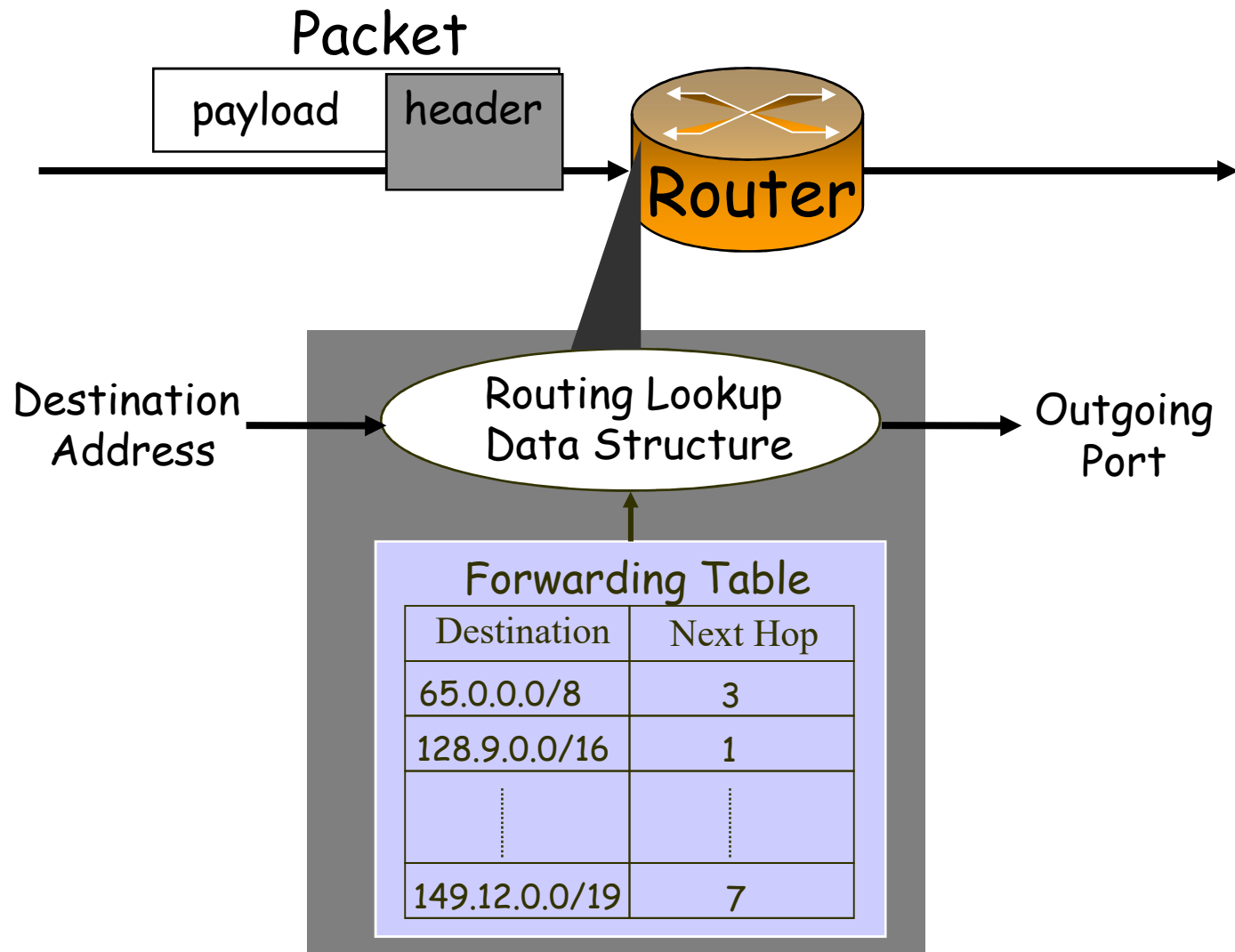


路由查找





路由查找过程示意图





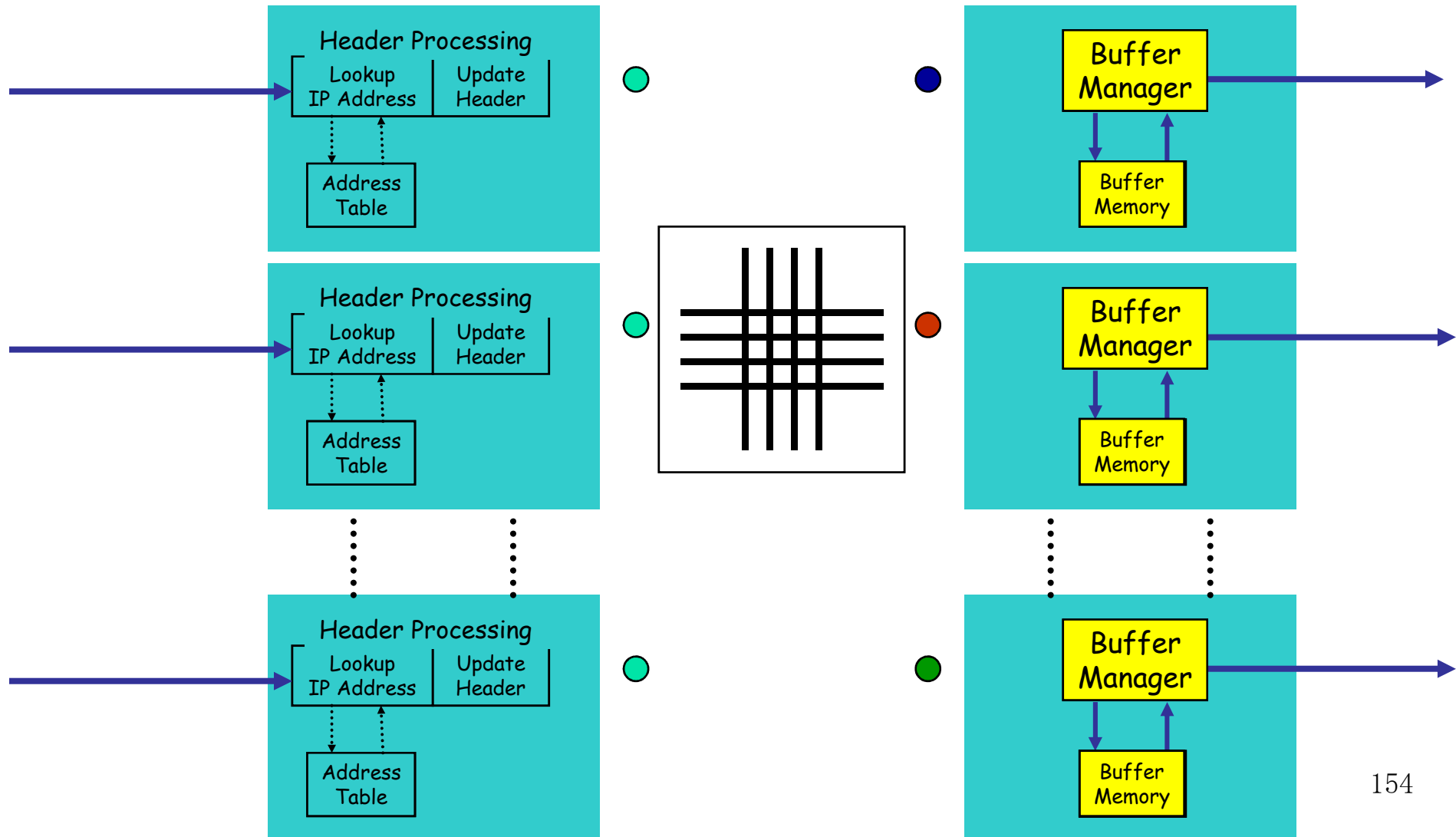
路由查找

路由查找的难点在于：

1. 不是精确匹配，是最长匹配。
2. 路由表很大，目前超过**40**万条表项，并且还在不断增长。
3. 路由查找必须很快：对应**10Gb/S**的链路，速度要求是**30ns**。



报文缓存

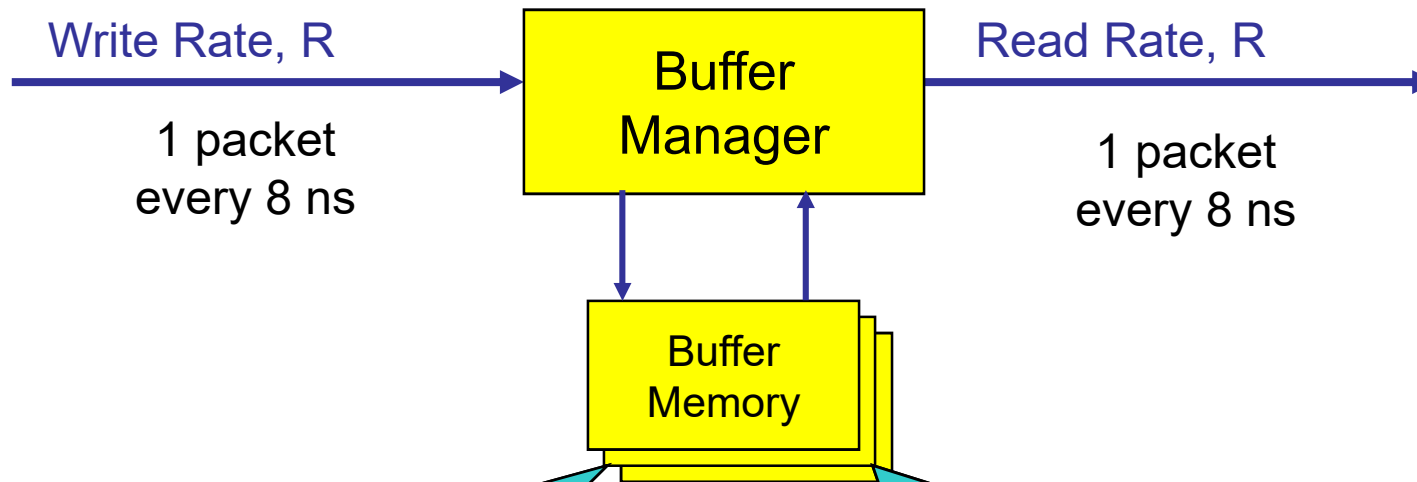




快速报文缓存

Example: 40Gb/s packet buffer

40 byte packets



使用SRAM?

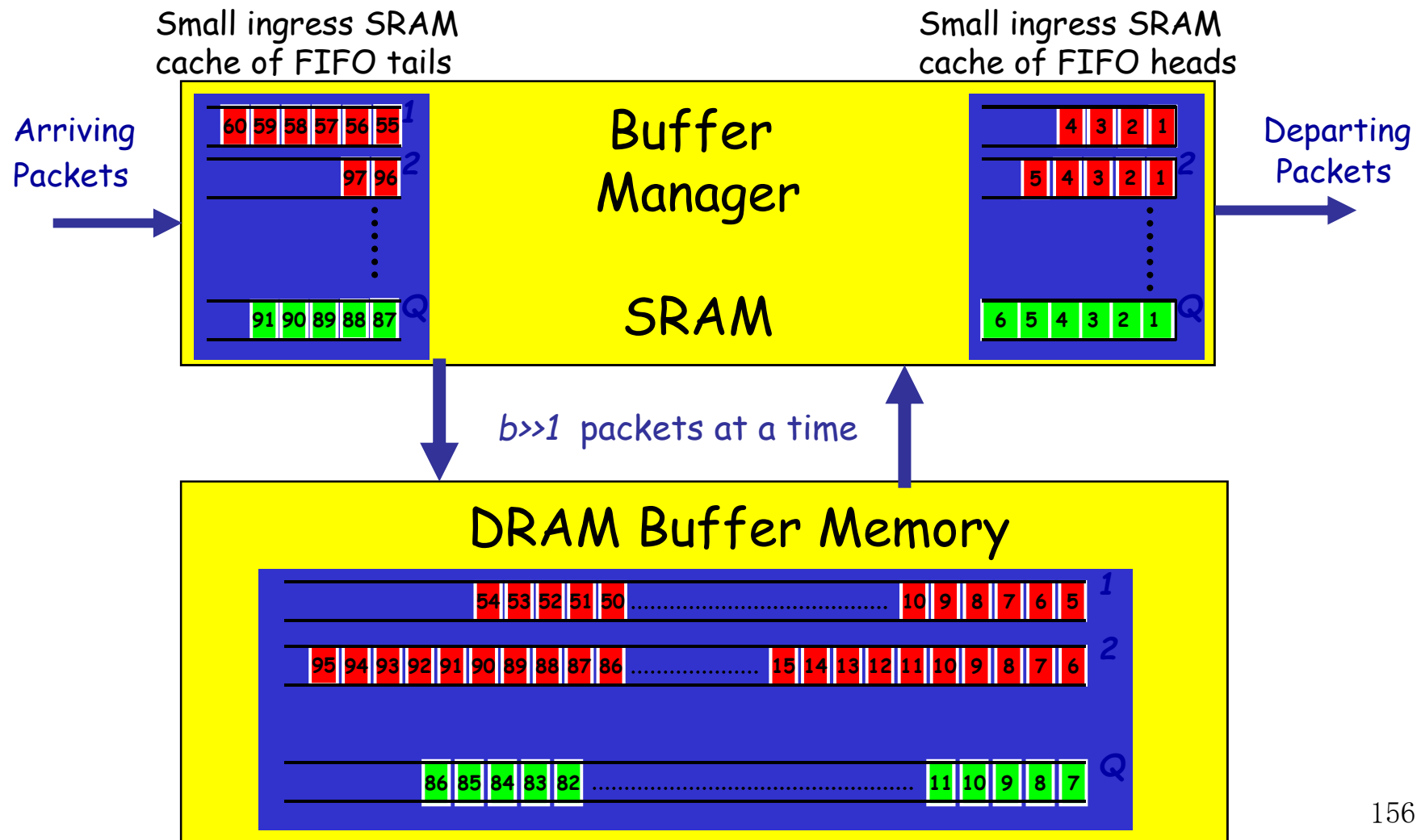
- + 速度够快
- 但容量不够.

使用DRAM?

- + 容量足够
- 但速度太慢.

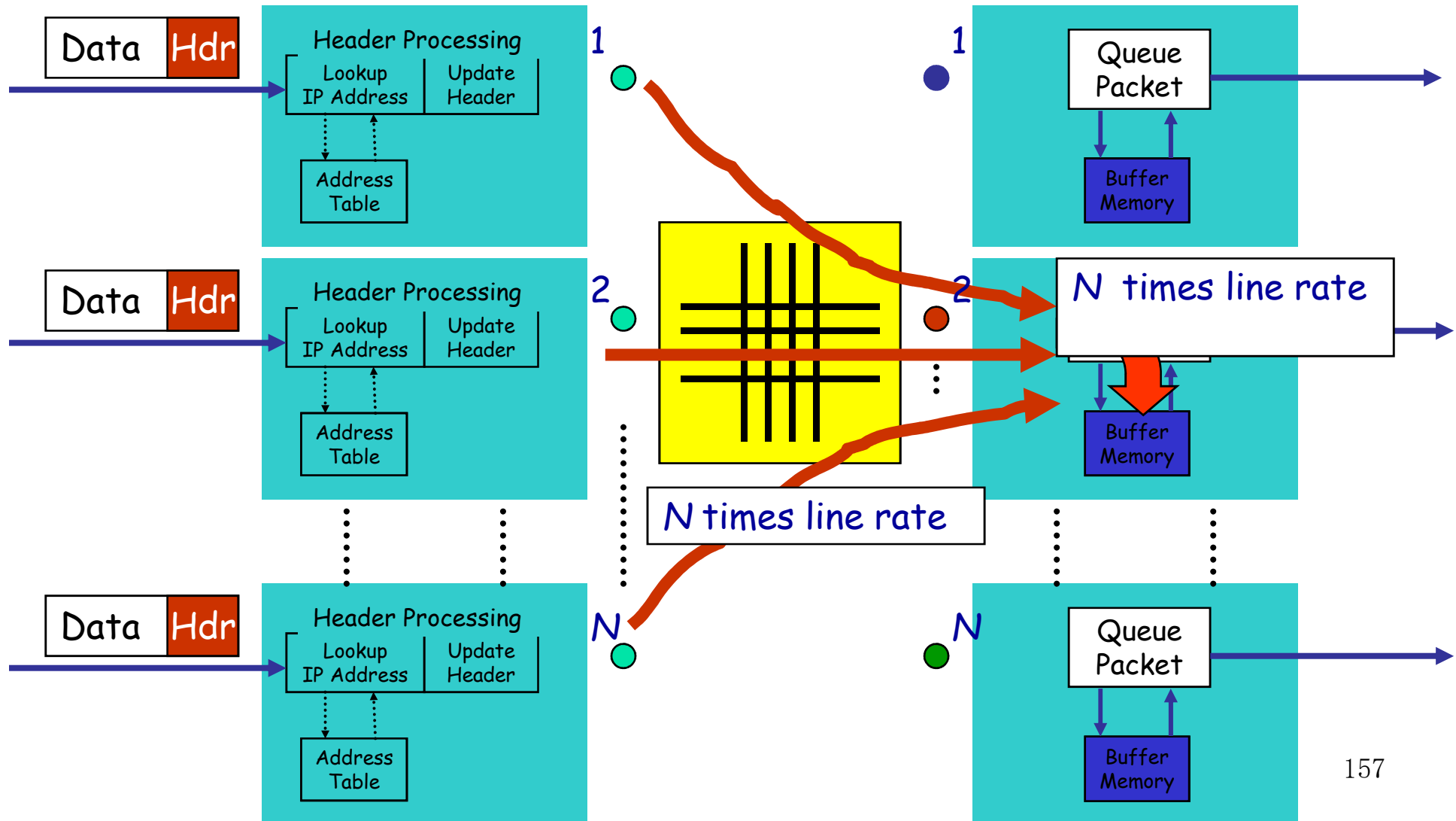


快速报文缓存（续）



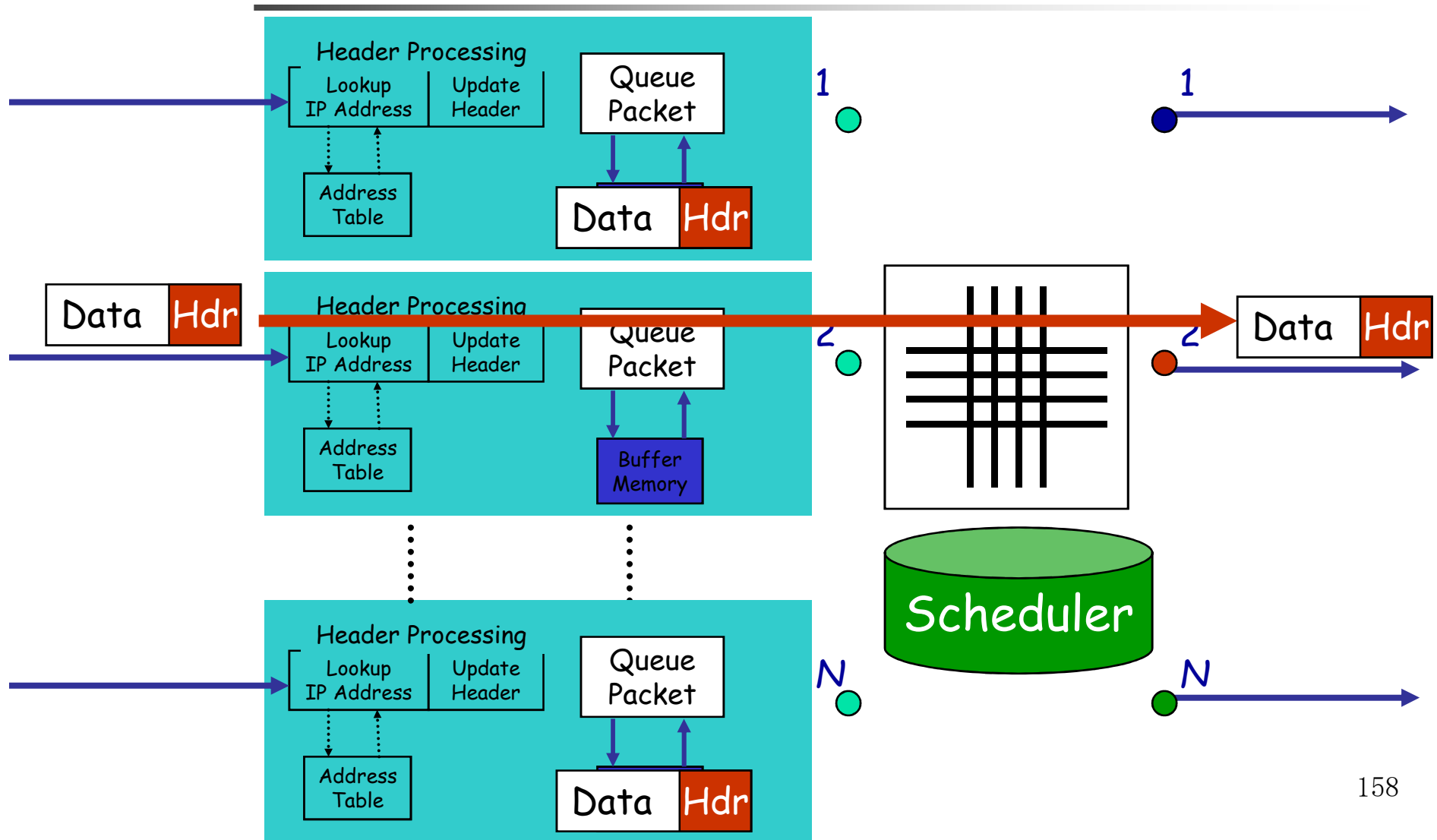


交换

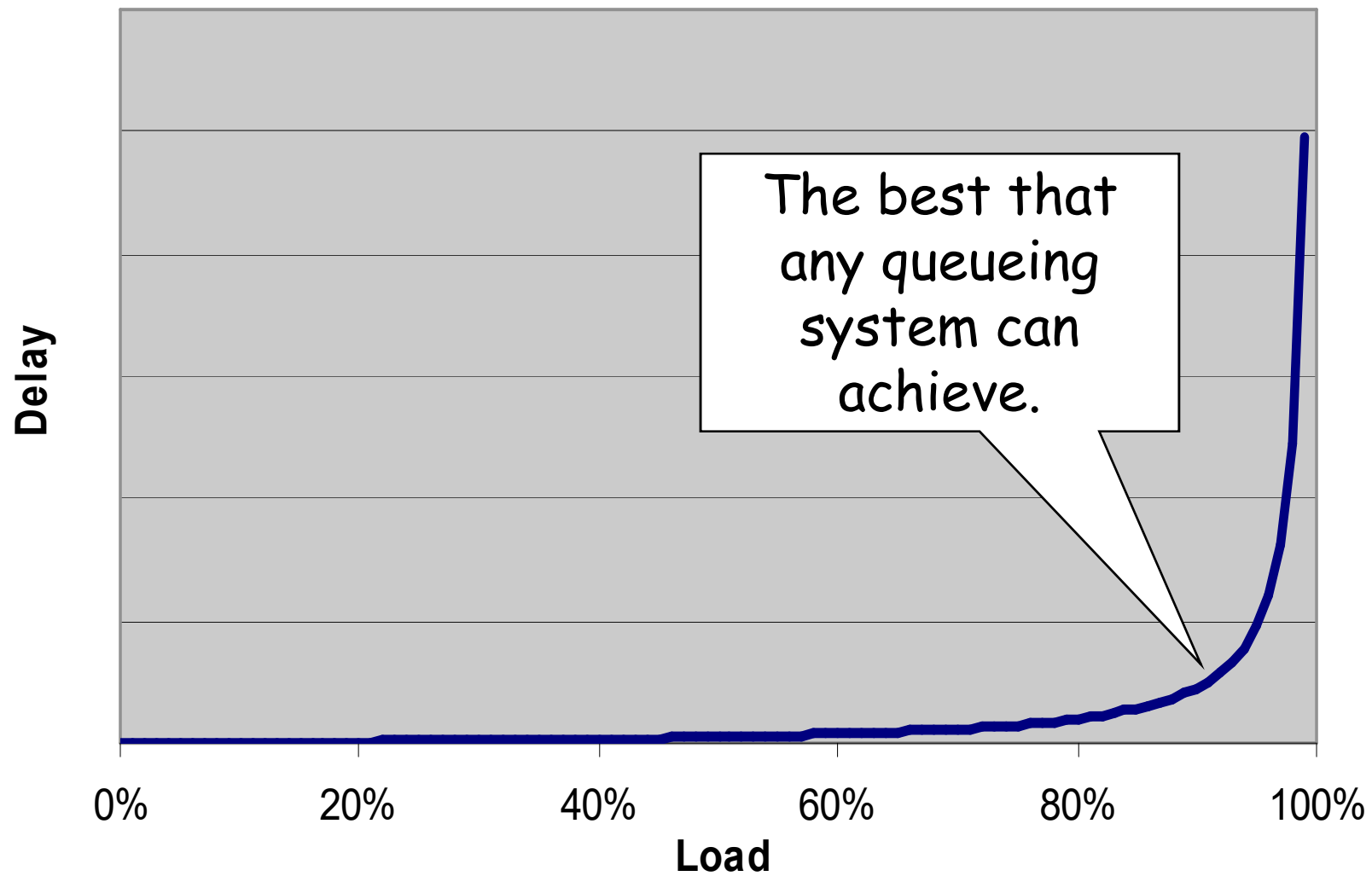




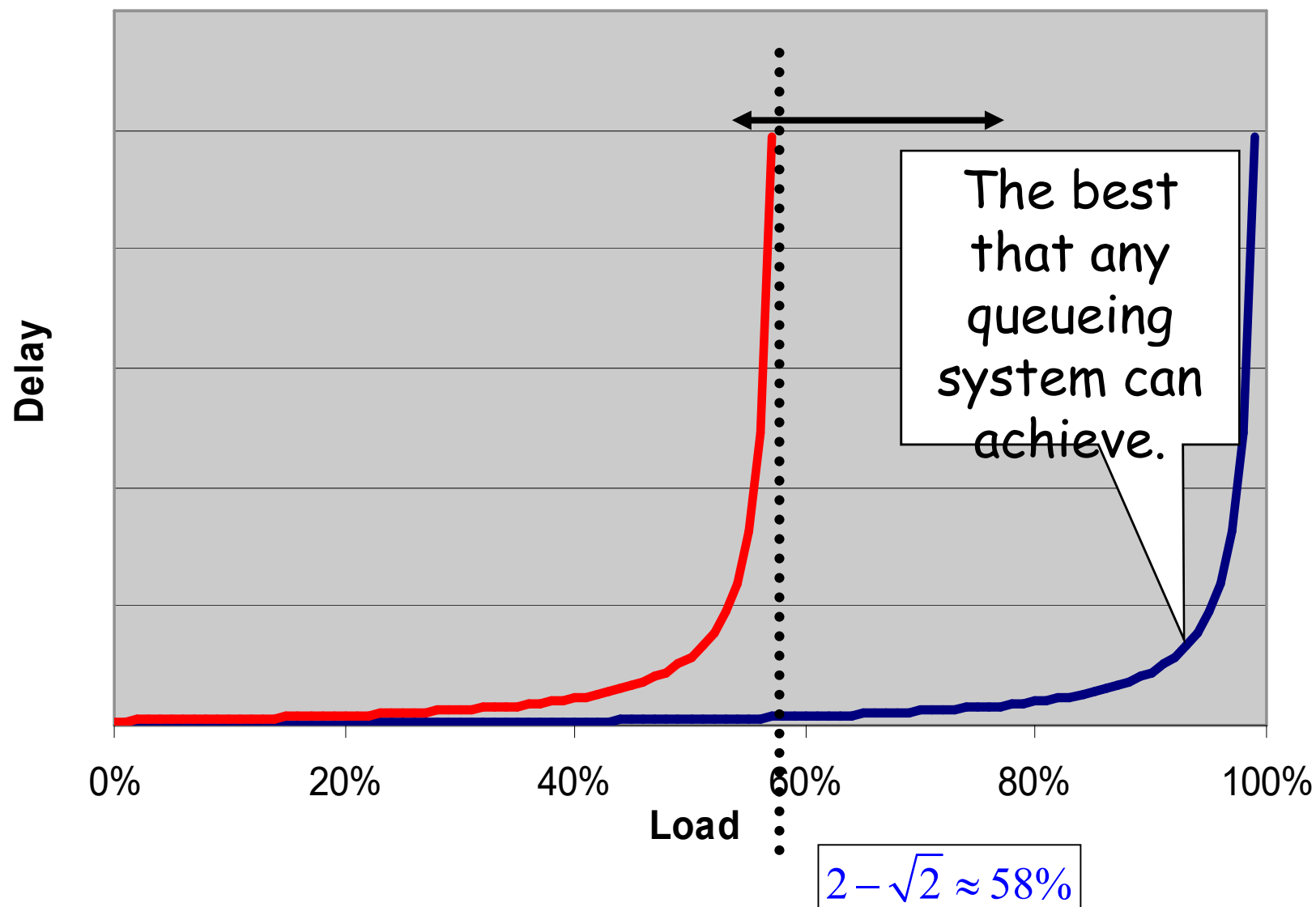
交换



使用输入队列的路由器

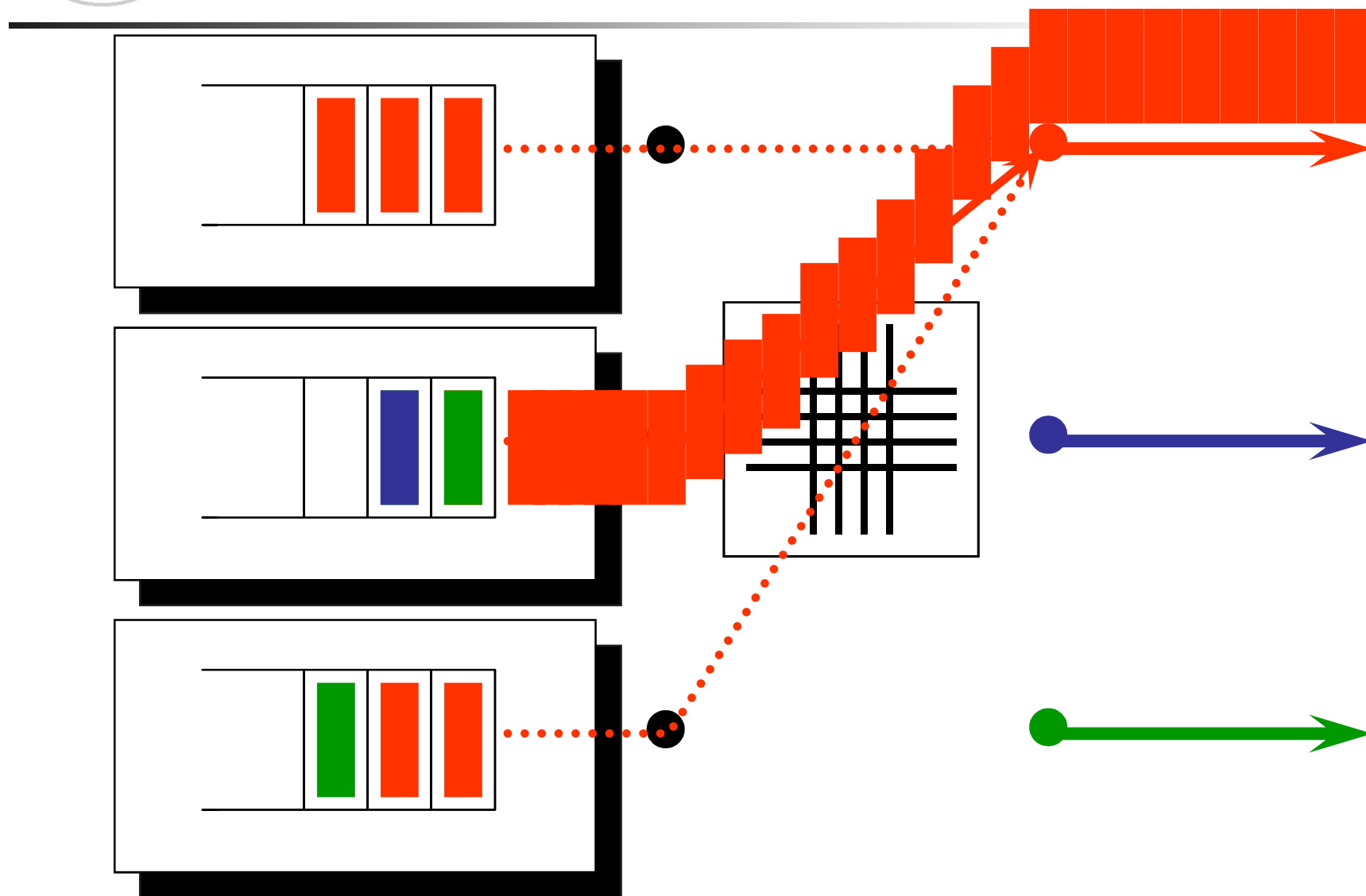


使用输入队列的路由器队头阻塞



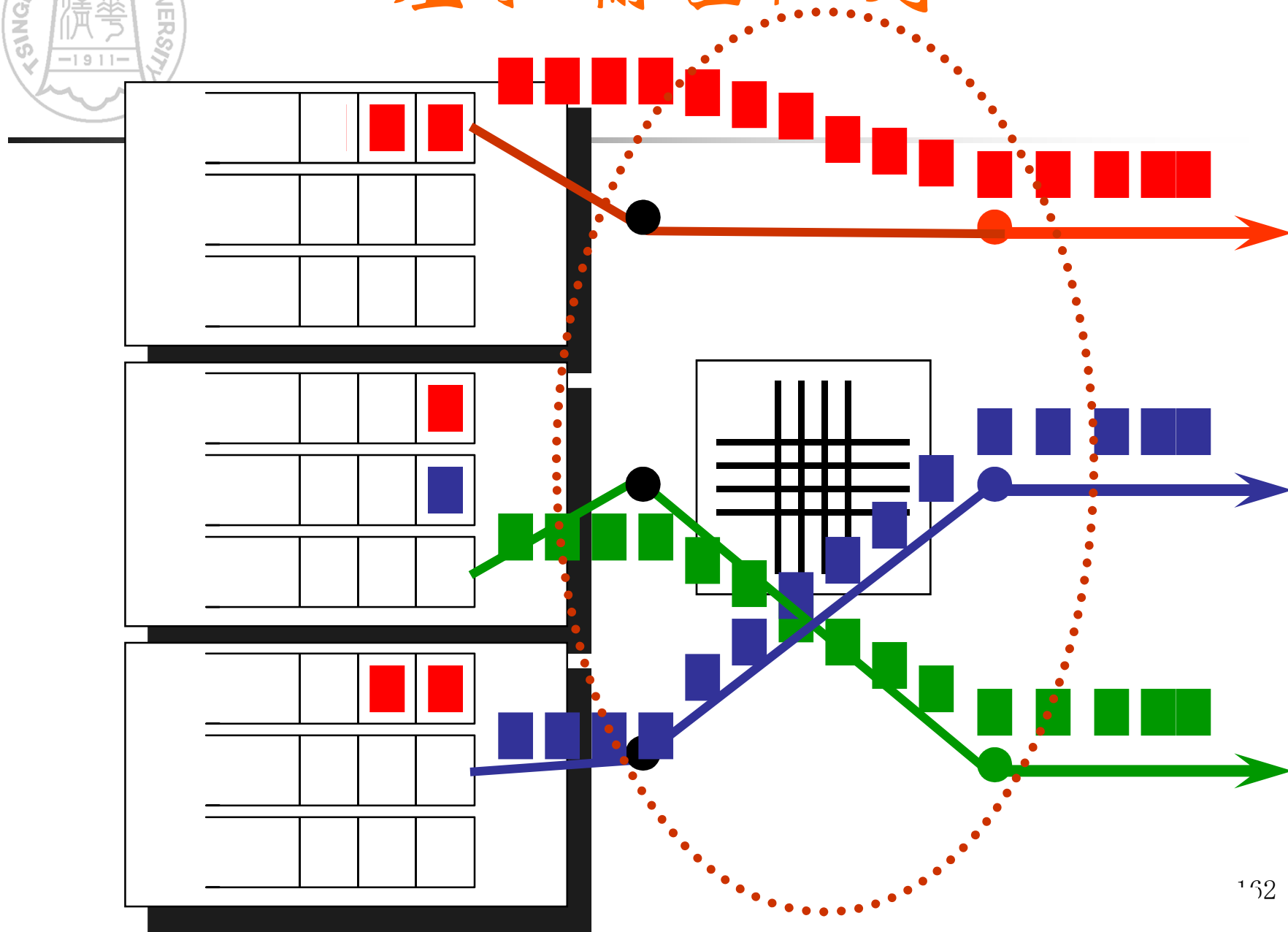


队头阻塞

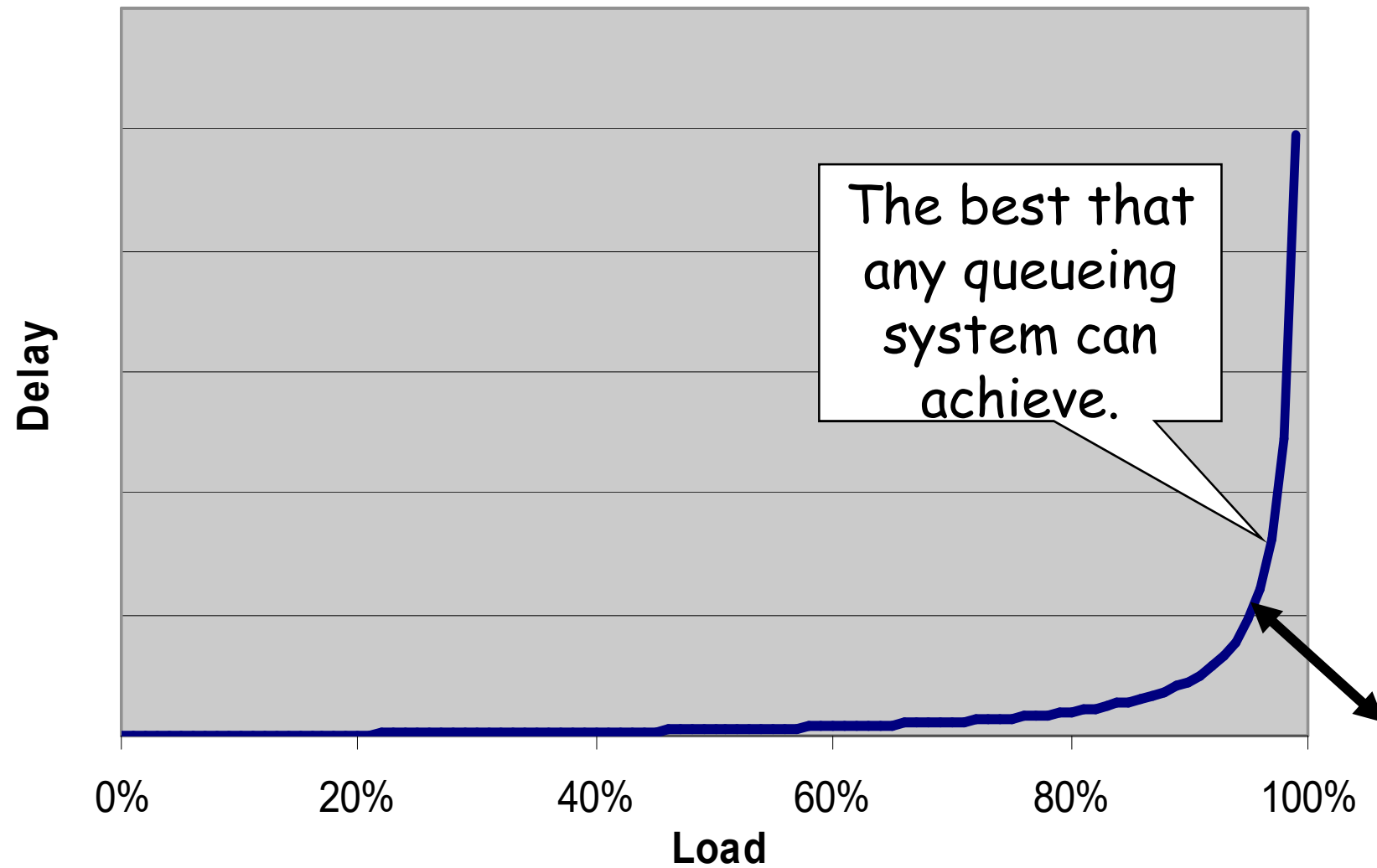




虚拟输出队列



使用虚拟输出队列的路由器



第八章

端到端访问与传输层





主要内容

- 传输服务
- 传输层寻址
- 连接管理
- 缓存和流控
- 互联网传输协议
 - **TCP**协议
 - **UDP**协议



传输服务

- 引入传输层的原因
 - 消除网络层的不可靠性
 - 提供从源端主机的进程到目的端主机的进程的可靠的、与实际使用的网络无关的数据传输
- 传输服务
 - 传输实体（**transport entity**）：完成传输层功能的硬软件
 - 传输层实体利用网络层提供的服务向上层提供有效、可靠的服务



传输服务（续）

- 传输层提供两种服务
 - 面向连接的传输服务：连接建立，数据传输，连接释放
 - 无连接的传输服务
- 1 ~ 4层称为传输服务提供者（**transport service provider**），4层以上称为传输服务用户（**transport service user**）

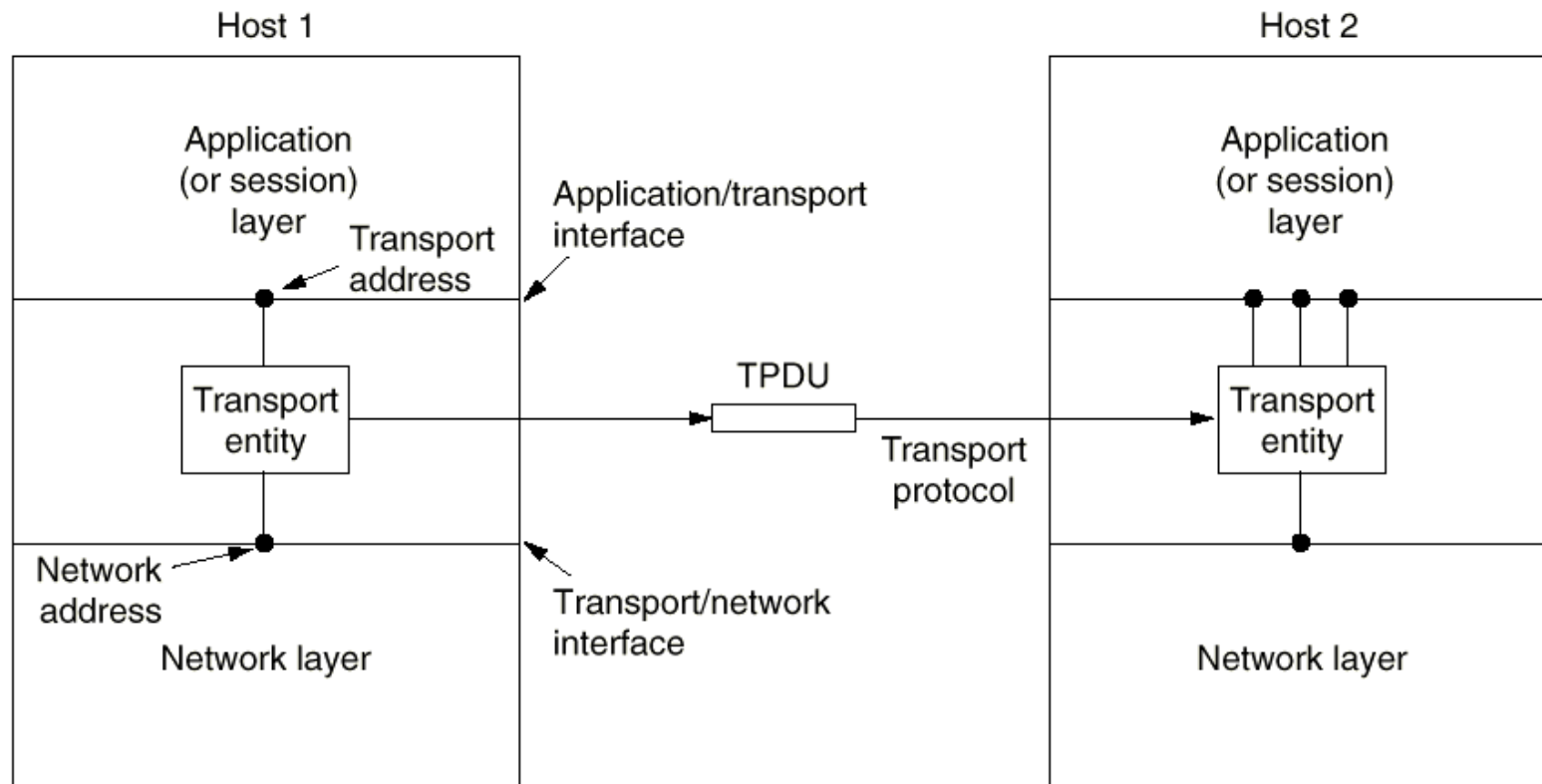


Fig. 6-1. The network, transport, and application layers.



传输服务原语

- 传输服务原语 (**Transport Service Primitives**)
 - 传输用户（应用程序）通过传输服务原语访问传输服务
 - 一个简单传输服务的原语
 - **Client/Server**模式

Primitive	TPDU sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA TPDU arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

Fig. 6-3. The primitives for a simple transport service.



TPDU

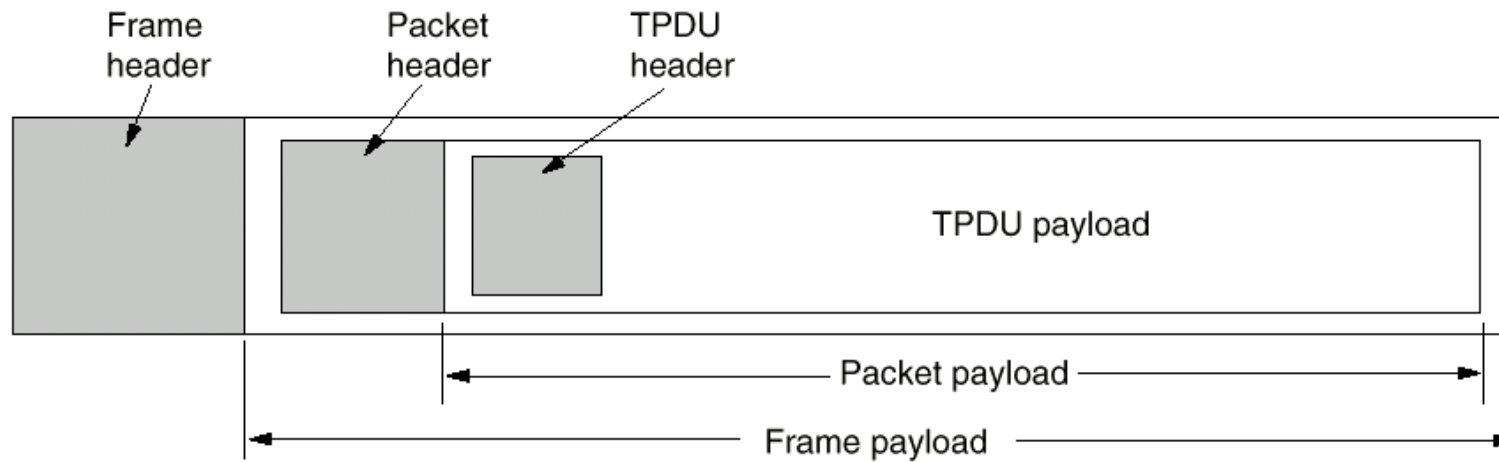


Fig. 6-4. Nesting of TPDU, packets, and frames.



简单连接管理

- 拆除连接方式有两种
 - 不对称方式：任何一方都可以关闭双向连接
 - 对称方式：每个方向的连接单独关闭，双方都执行**DISCONNECT**才能关闭整条连接
- 简单连接管理状态图

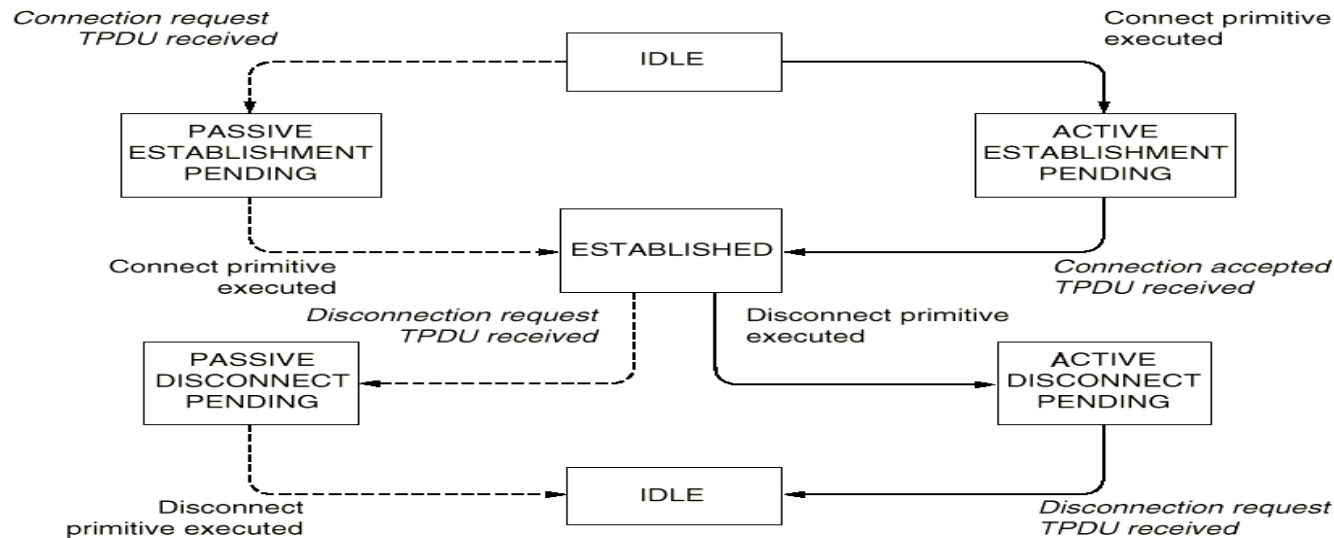


Fig. 6-5. A state diagram for a simple connection management scheme. Transitions labeled in italics are caused by packet arrivals. The solid lines show the client's state sequence. The dashed lines show the server's state sequence.



Berkeley Sockets

- **Berkeley Sockets**
 - 连接释放是对称的

Primitive	Meaning
SOCKET	Create a new communication end point
BIND	Attach a local address to a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Block the caller until a connection attempt arrives
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

Fig. 6-6. The socket primitives for TCP.



Berkeley Sockets (续)

■ 应用举例

- 一个服务程序和几个远程客户程序利用面向连接的传输层服务完成通信
- 建立连接
 - 服务程序
 - 调用`socket`创建一个新的套接字，并在传输层实体中分配表空间，返回一个文件描述符用于以后调用中使用该套接字
 - 调用`bind`将一个地址赋予该套接字，使得远程客户程序能访问该服务程序
 - 调用`listen`分配数据空间，以便存储多个用户的连接建立请求



Berkeley Sockets (续)

- 调用**accept**将服务程序阻塞起来，等待接收客户程序发来的连接请求。当传输层实体接收到建立连接的TPDU时，新建一个和原来的套接字相同属性的套接字并返回其文件描述符。服务程序创建一个子进程处理此次连接，然后继续等待发往原来套接字的连接请求

■ 客户程序

- 调用**socket**创建一个新的套接字，并在传输层实体中分配表空间，返回一个文件描述符用于在以后的调用中使用该套接字
- 调用**connect**阻塞客户程序，传输层实体开始建立连接，当连接建立完成时，取消阻塞



Berkeley Sockets (续)

- 数据传输
 - 双方使用**send**和**receive**完成数据的全双工发送
- 释放连接
 - 每一方使用**close**原语单独释放连接



传输层寻址

- 寻址 (**Addressing**)
- 方法：定义传输服务访问点**TSAP** (**Transport Service Access Point**)，将应用进程与这些**TSAP**相连。
 - 在互联网中，**TSAP**为(**IP address, local port**)
- 远方客户程序如何获得服务程序的**TSAP**?
 - 方法**1**：预先约定、广为人知的，例如**telnet**是 (**IP地址, 端口23**)
 - 方法**2**：从名称服务器 (**name server**) 或目录服务器 (**directory server**) 获得**TSAP**

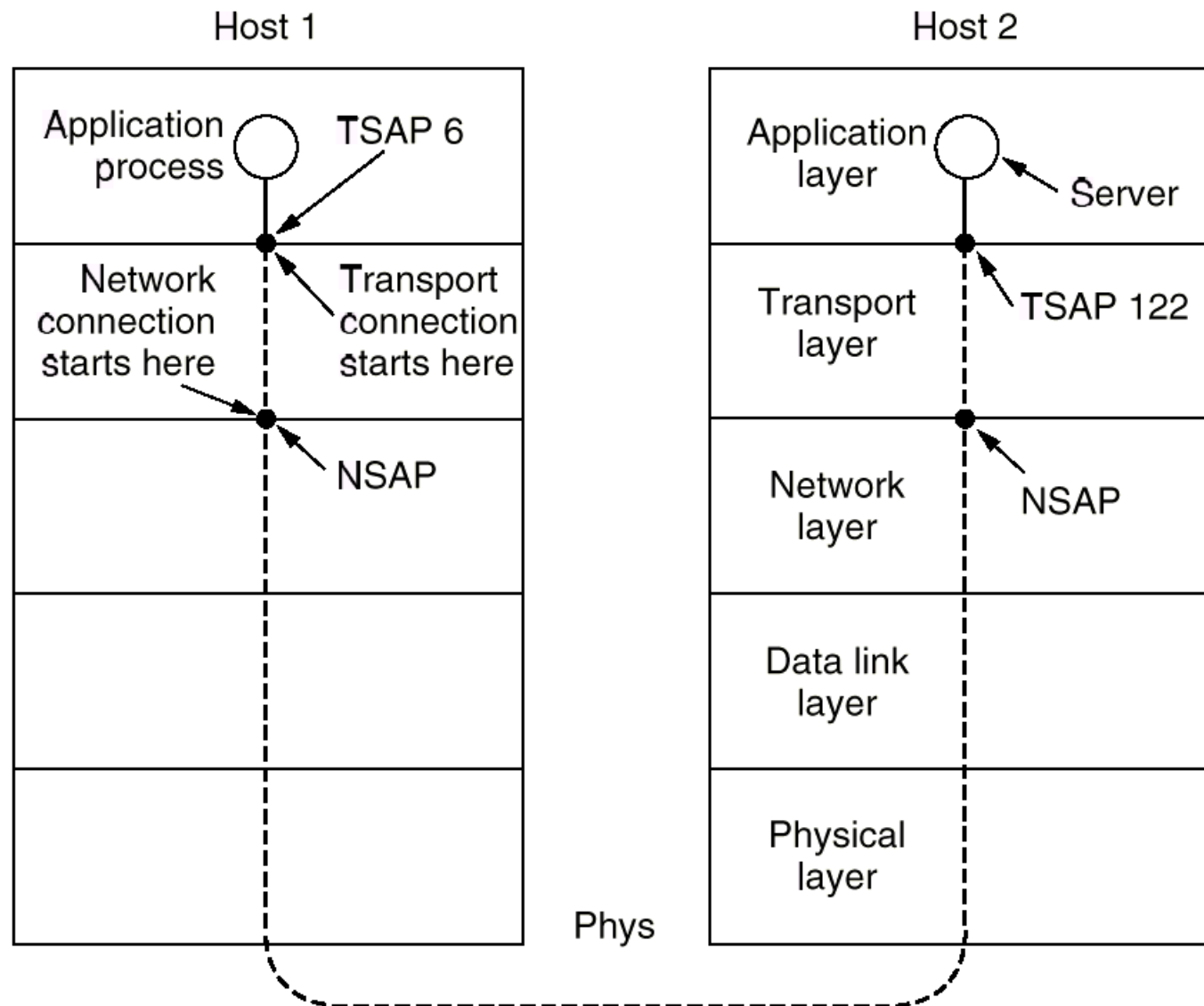


Fig. 6-8. TSAPs, NSAPs, and connections.



名称服务器

- 一个特殊的进程称为名称服务器或目录服务器（**TSAP**众所周知）
- 协议工作过程
 - 客户进程与名称服务器建立连接，发送服务名称，获得服务进程的**TSAP**，释放与名称服务器的连接
 - 客户进程与服务进程建立连接



进程服务器

- 当服务程序很多时，可以使用进程服务器（**process server**）
- 进程服务器是一个同时在多个端口上监听的进程（例如，**inetd**）
- 客户进程向它实际想访问的服务进程的**TSAP**发出连接建立请求
- 如果没有服务进程在此**TSAP**上监听，则客户进程和进程服务器建立连接
- 进程服务器产生所请求的服务进程，并使该进程继承和客户进程的连接
- 进程服务器返回继续监听
- 客户进程与所希望的服务进程进行数据传输

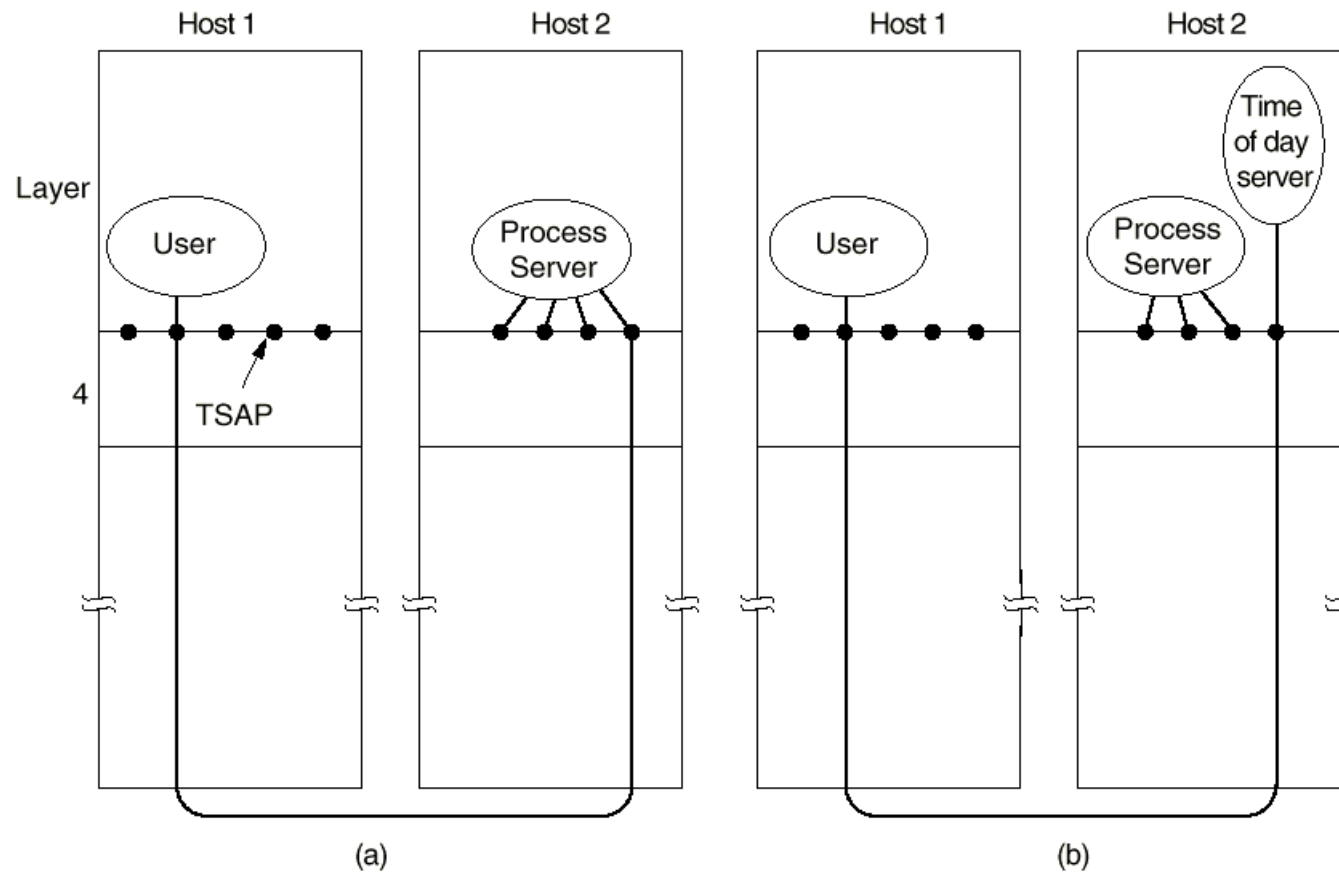


Fig. 6-9. How a user process in host 1 establishes a connection with a time-of-day server in host 2.



连接管理

■ 建立连接

- 网络可能丢失、重复包，特别是延迟重复包的存在，导致传输层建立连接的复杂性
- 两次握手不能满足要求
 - 两次握手：**A**发出连接请求**CR TPDU**，**B**发回连接确认**CC TPDU**
 - 失败的原因：网络层会丢失、存储和重复包
- 解决延迟重复包的关键是丢弃过时的包
- 采用三次握手



三次握手

- 三次握手 (**three-way handshake**)
 - **A** 发出序号为**X**的**CR TPDU**
 - **B** 发出序号为**Y**的**CC TPDU**，并确认**A**的序号为**X**的**CR TPDU**
 - **A** 发出序号为**X+1**的第一个数据**TPDU**，并确认**B**的序号为**Y**的**CR TPDU**
 - **Fig. 6-11** (**DATA**序号应为**X+1**)
- 三次握手方案解决了由于网络层会丢失、存储和重复包带来的问题

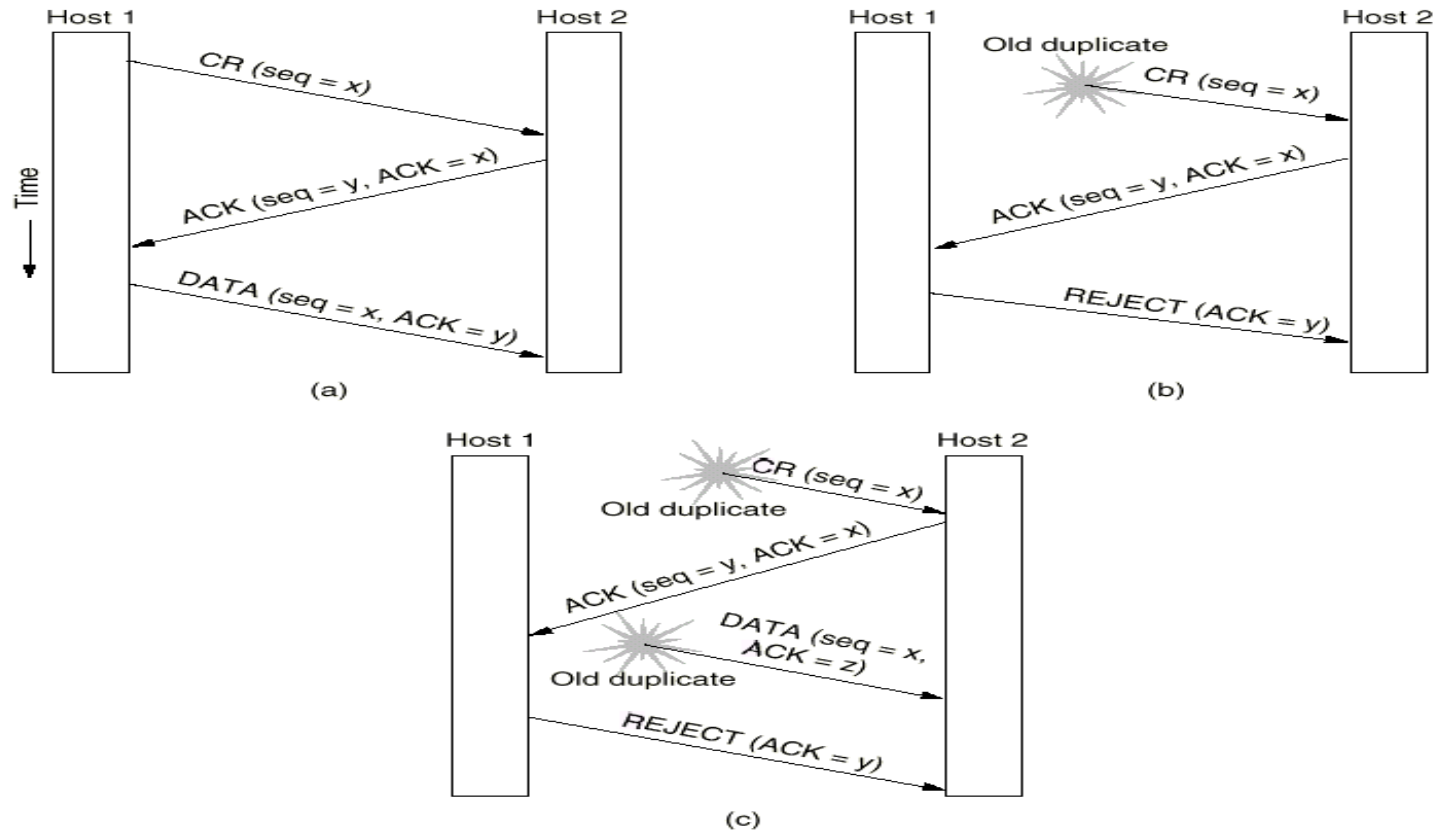


Fig. 6-11. Three protocol scenarios for establishing a connection using a three-way handshake. CR and ACC denote CONNECTION REQUEST and CONNECTION ACCEPTED, respectively. (a) Normal operation. (b) Old duplicate CONNECTION REQUEST appearing out of nowhere. (c) Duplicate CONNECTION REQUEST and duplicate ACK.



连接管理（续）

■ 释放连接

- 非对称式：一方释放连接，双向连接断开，存在丢失数据的危险
- 对称式：每个方向的连接单独关闭，不会丢失数据，但是存在两军问题

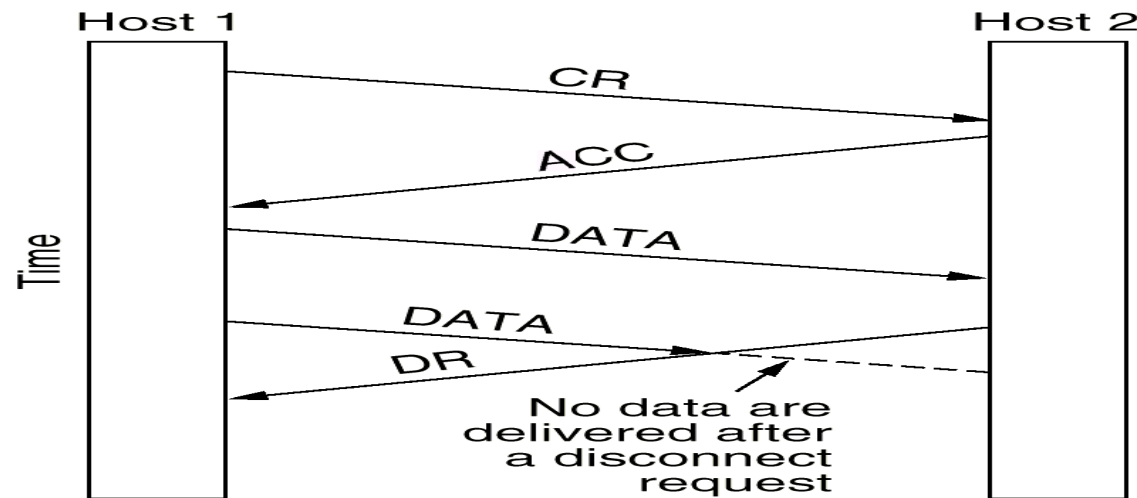


Fig. 6-12. Abrupt disconnection with loss of data.



两军问题

- 由于存在两军问题（**two-army problem**），可以证明不存在安全的通过**N**次握手实现对称式连接释放的方法

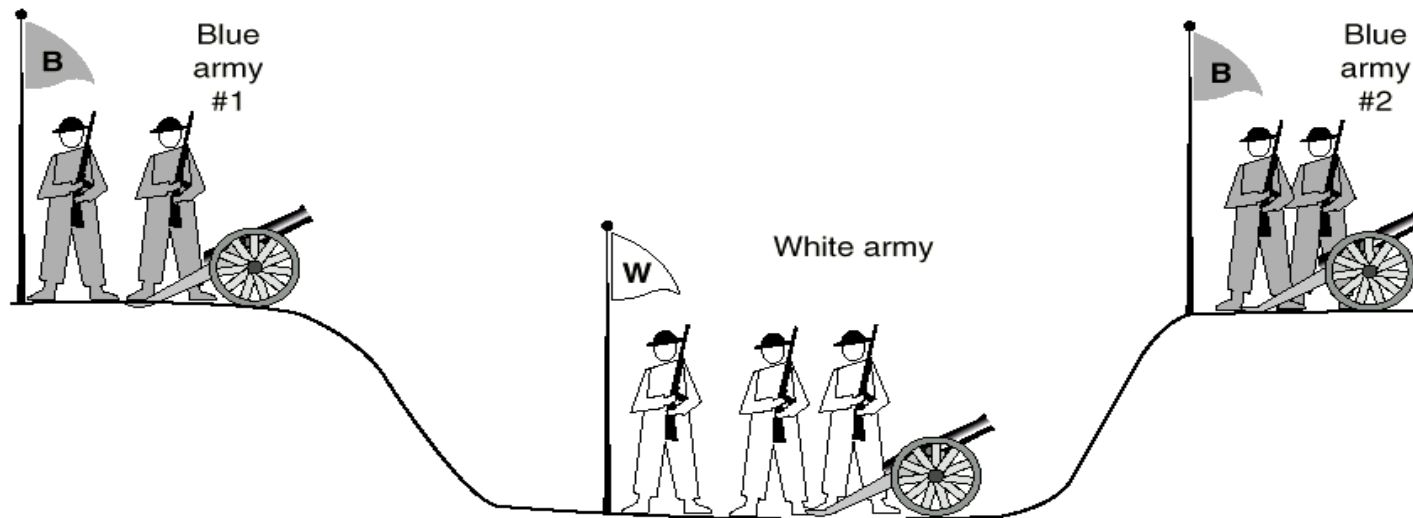


Fig. 6-13. The two-army problem.



连接管理（续）

- 但是在实际的通信过程中，使用三次握手 + 定时器的方法释放连接在绝大多数情况下是成功的。

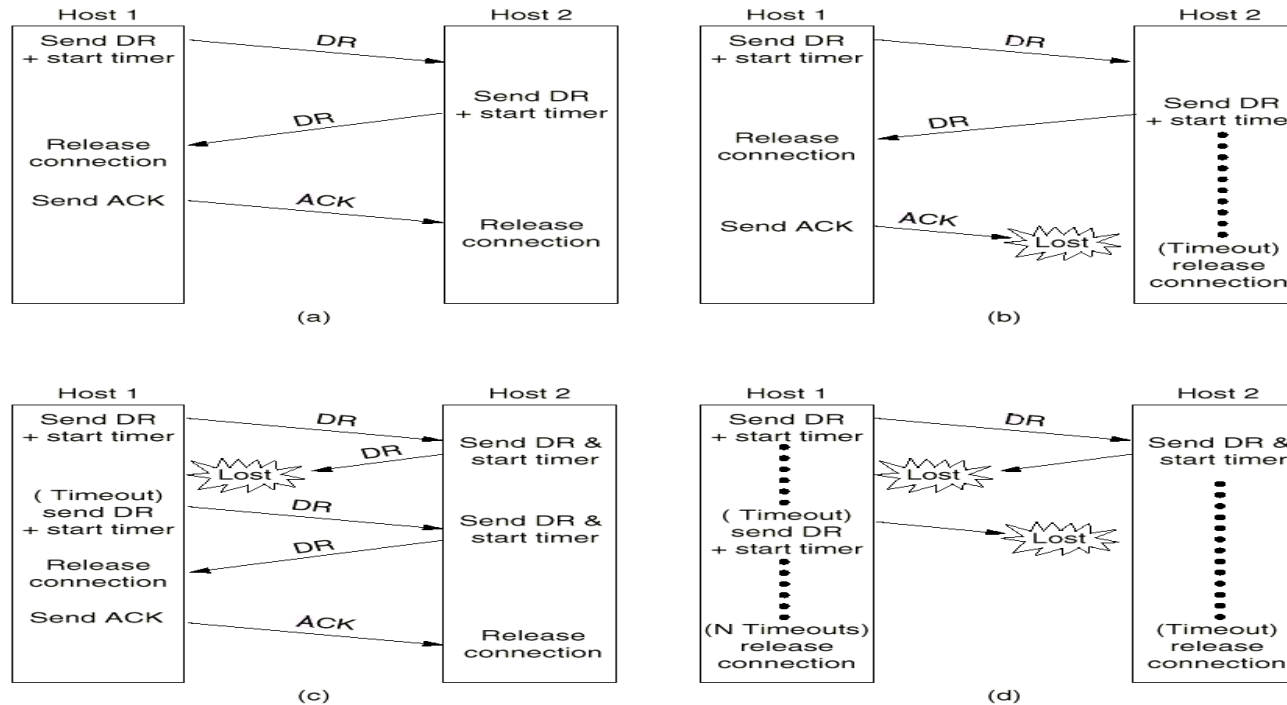


Fig. 6-14. Four protocol scenarios for releasing a connection. (a) Normal case of three-way handshake. (b) Final ACK lost. (c) Response lost. (d) Response lost and subsequent DRs lost.



缓存

- 由于网络层服务是不可靠的，传输层实体必须缓存所有连接发出的**TPDU**，而且为每个连接单独做缓存，以便用于错误情况下的重传
- 接收方的传输层实体可以做也可以不做缓存
- 缓存的设计有三种
 - 固定大小缓存
 - 可变大小缓存
 - 缓存池

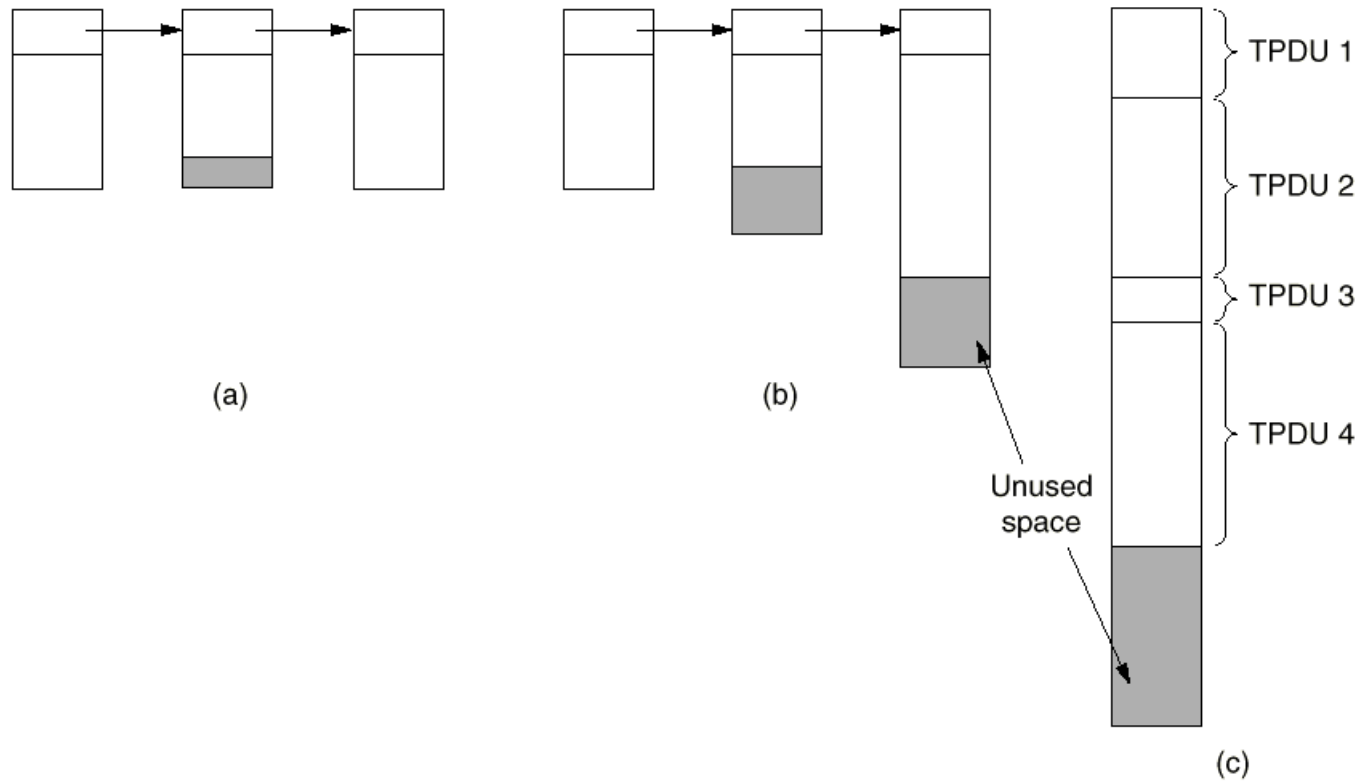


Fig. 6-15. (a) Chained fixed-size buffers. (b) Chained variable-size buffers. (c) One large circular buffer per connection.



流控

- 流控: **Flow Control**
- 传输层利用可变滑动窗口协议来实现流控
- 可变滑动窗口协议: 是指发送方的发送窗口大小是由接收方根据自己的实际缓存情况给出的
- 为了避免控制**TPDU**丢失导致死锁, 端系统应该周期性的发送**TPDU**



	<u>A</u>	<u>Message</u>	<u>B</u>	<u>Comments</u>
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	...	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	...	<ack = 6, buf = 4>	←	Potential deadlock

Fig. 6-16. Dynamic buffer allocation. The arrows show the direction of transmission. An ellipsis (...) indicates a lost TPDU.



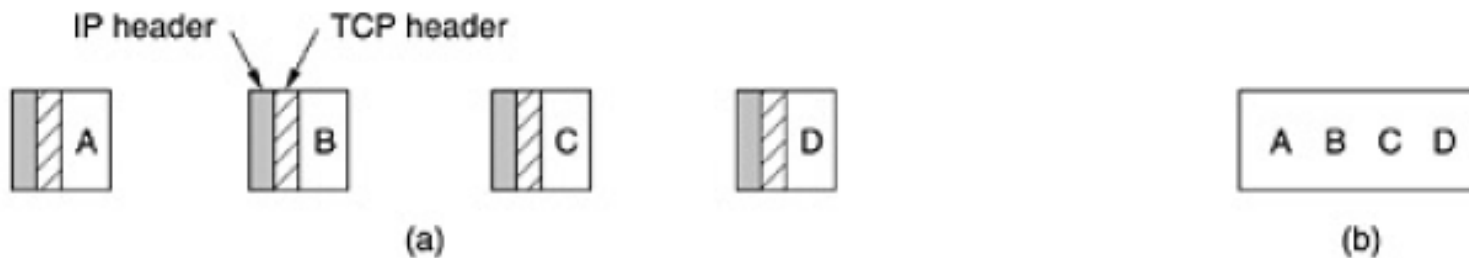
互联网传输协议

- **传输控制协议TCP (Transmission Control Protocol)**
 - 面向连接的、可靠的、端到端的、基于字节流的传输协议
 - **RFC 793, 1122, 1323, 2018, 2581等**
- **用户数据协议UDP (User Data Protocol)**
 - 无连接的端到端传输协议
 - **RFC 768**



TCP 协议

- 应用程序访问**TCP**服务是通过在收发双方创建套接字来实现的
- 套接字地址用（**IP**地址，端口号）来表示，**256**以下的端口号被标准服务保留，例如**FTP（21）**，**TELNET（23）**
- 每条连接用(套接字**1**,套接字**2**)来表示，是点到点的全双工通道
- **TCP**不支持组播（**multicast**）和广播（**broadcast**）
- **TCP**连接是基于字节流的，而非消息流，消息的边界在端到端的传输中不能得到保留





TCP 协议 (续)

- 对于应用程序发来的数据，**TCP**可以立即发送，也可以缓存一段时间以便一次发送更多的数据。为了强迫数据发送，可以使用**PUSH**标记
- 对于紧急数据，可以使用**URGENT**标记
- 按字节分配序号，每个字节有一个**32**位的序号
- 传输实体之间使用的**TPDU**称为段（**segment**）
- 每个段包含一个**20**字节的头（选项部分另加）和**0**个或多个数据字节。
- 段的大小必须满足**65535**字节的**IP**包数据净荷长度限制，还要满足数据链路层最大传输单元（**MTU**）的限制，例如以太网的**MTU**为**1500**字节
- **TCP**实体使用滑动窗口协议，确认序号等于接收方希望接收的下一个字节序号



TCP 协议 (续)

- TCP协议需要解决的主要问题
 - 连接管理
 - 建立连接：三次握手
 - 释放连接：三次握手 + 定时器
 - 可靠传输
 - 滑动窗口
 - 流控制和拥塞控制
 - 可变滑动窗口
 - 慢启动 (**slow start**)、拥塞避免 (**congestion avoidance**) ...



TCP头

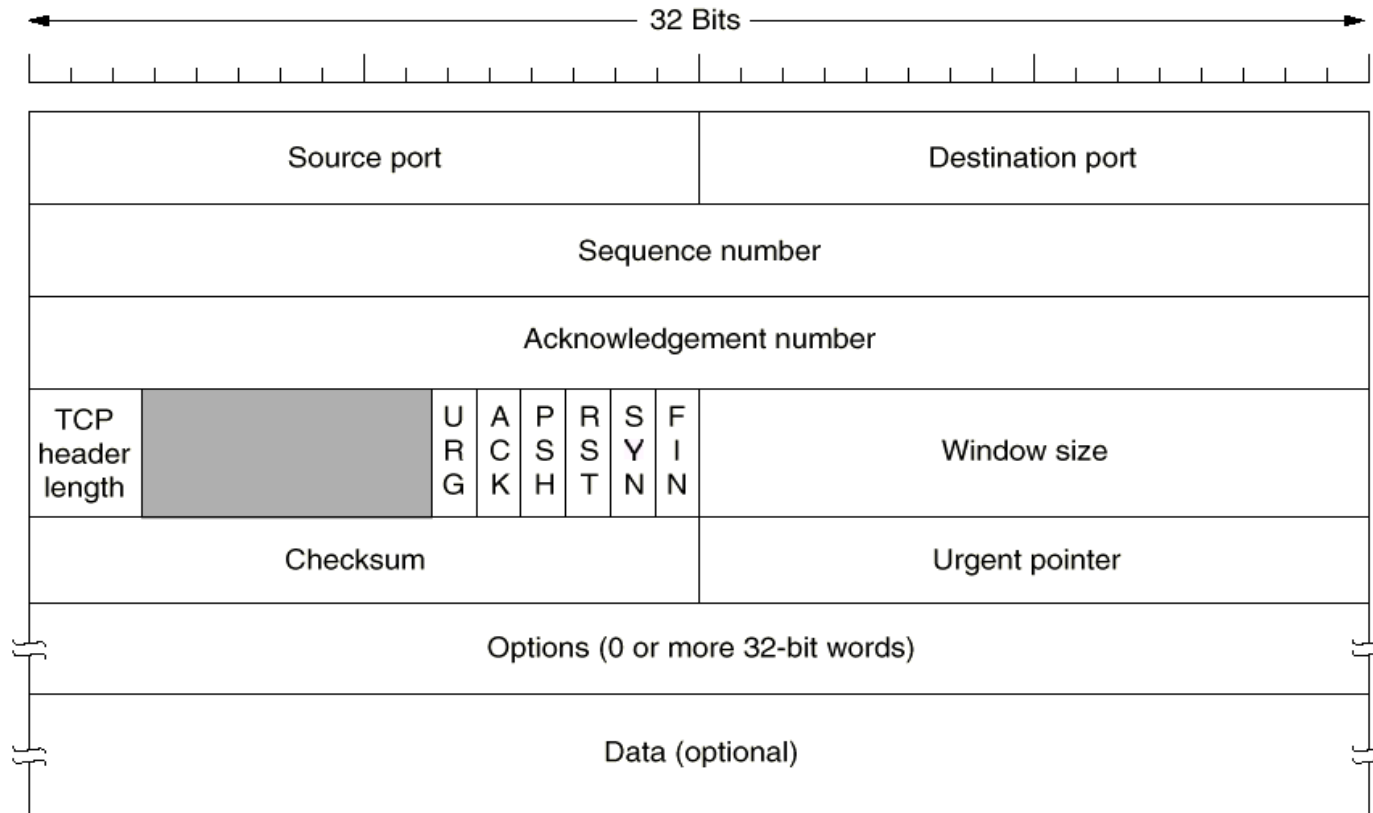


Fig. 6-24. The TCP header.



TCP头 (续)

- 源端口和目的端口：各**16**位
- 序号和确认号：以字节为单位编号，各**32**位
- **TCP**头的长度：**4**位，长度单位为**32**位字，包含选项域
- **6**位的保留域
- **6**位的标识位：置**1**表示有效
 - **URG**：和紧急指针配合使用，发送紧急数据
 - **ACK**：确认号是否有效
 - **PSH**：指示发送方和接收方将数据不做缓存，立刻发送或接收
 - **RST**：由于不可恢复的错误重置连接
 - **SYN**：用于连接建立指示
 - **FIN**：用于连接释放指示



TCP 头 (续)

- 窗口大小：用于基于可变滑动窗口的流控，指示发送方从确认号开始可以再发送窗口大小的字节流
- 校验和：为增加可靠性，对**TCP**头，数据和伪头计算校验和
- 选项域

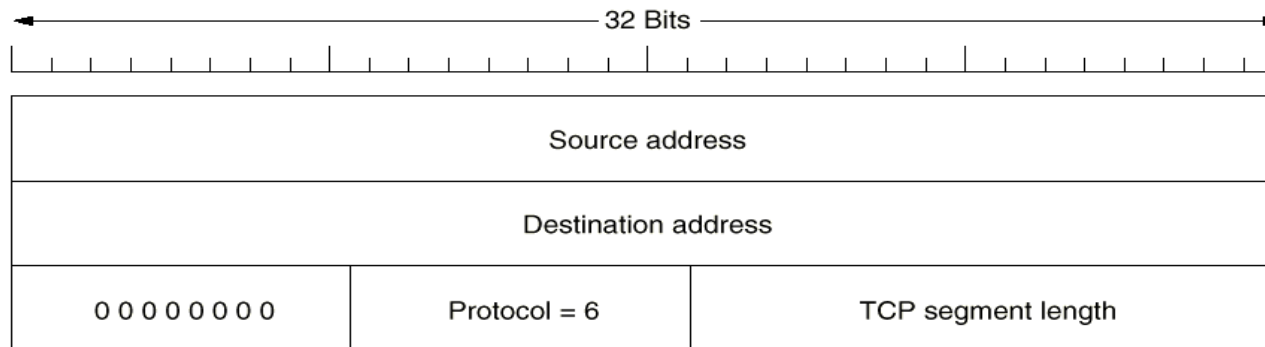


Fig. 6-25. The pseudoheader included in the TCP checksum.



TCP 连接管理

■ 三次握手建立连接

- 服务器方执行**LISTEN**和**ACCEPT**原语，被动监听
- 客户方执行**connect**原语，产生一个**SYN**为**1**和**ACK**为**0**的**TCP**段，表示连接请求
- 服务器方的传输实体接收到这个**TCP**段后，首先检查是否有服务进程在所请求的端口上监听，若没有，回答**RST**置位的**TCP**段
- 若有服务进程在所请求的端口上监听，该服务进程可以决定是否接受该请求。在接受后，发出一个**SYN**置**1**和**ACK**置**1**的**TCP**段表示连接确认，并请求与对方的连接
- 客户方收到确认后，发出一个**SYN**置**0**和**ACK**置**1**的**TCP**段表示给对方的连接确认
- 若两个主机同时试图建立彼此间的连接，则只能建立一条连接



TCP 连接管理 (续)

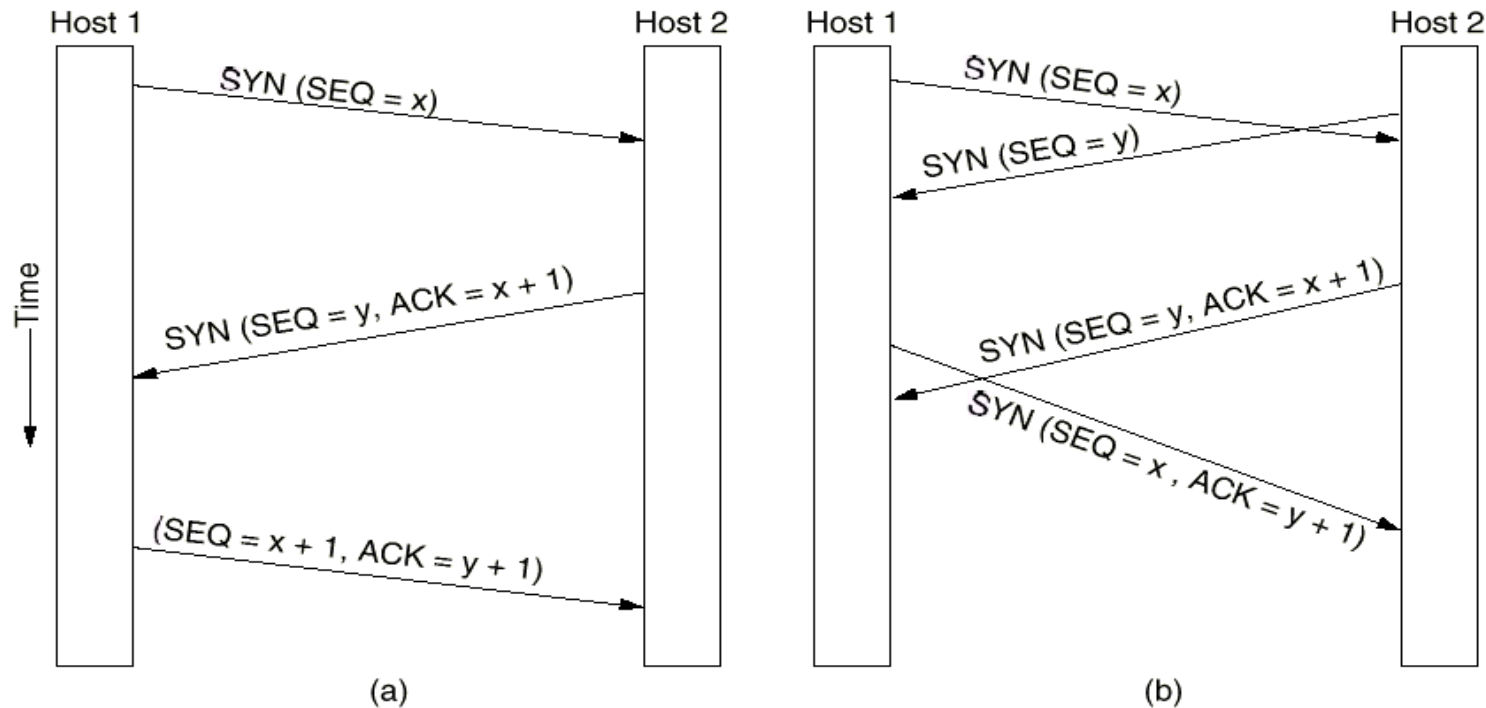


Fig. 6-26. (a) TCP connection establishment in the normal case. (b) Call collision.



TCP 连接管理 (续)

TCP A

1. CLOSED

2. SYN-SENT --> <SEQ=100><CTL=SYN> -->

3. ESTABLISHED <-- <SEQ=300><ACK=101><CTL=SYN,ACK> <--

4. ESTABLISHED --> <SEQ=101><ACK=301><CTL=ACK> -->

5. ESTABLISHED --> <SEQ=101><ACK=301><CTL=ACK><DATA> --> ESTABLISHED

TCP B

LISTEN

SYN-RECEIVED

SYN-RECEIVED

ESTABLISHED

Basic 3-Way Handshake for Connection Synchronization

Note that the ACK does not occupy sequence number space (if it did, we would wind up ACKing ACK's!).



TCP连接管理（续）

- 对称方式连接释放
- 释放连接时，发出**FIN**位置**1**的**TCP**段并启动定时器，在收到确认后关闭连接。若无确认并且超时，也关闭连接

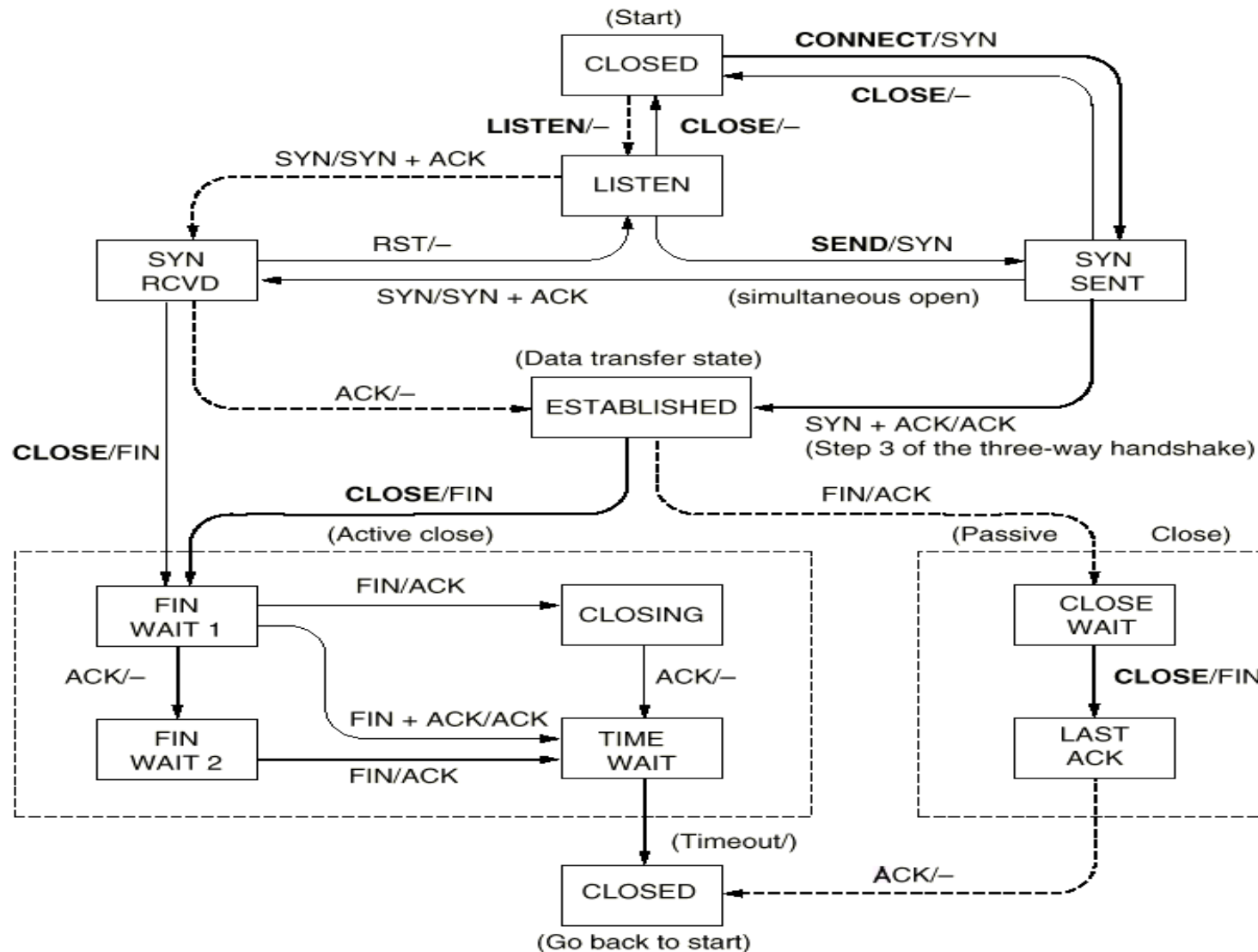


Fig. 6-28. TCP connection management finite state machine. The heavy solid line is the normal path for a client. The heavy dashed line is the normal path for a server. The light lines are unusual events.



TCP连接管理有限状态机

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIMED WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

Fig. 6-27. The states used in the TCP connection management finite state machine.



TCP传输策略

■ TCP的窗口管理机制

- 基于确认和可变窗口大小
- 窗口大小为0时，正常情况下，发送方不能再发TCP段，但有两个例外
 - 紧急数据可以发送
 - 为防止死锁，发送方可以发送1字节的TCP段，以便让接收方重新声明确认号和窗口大小

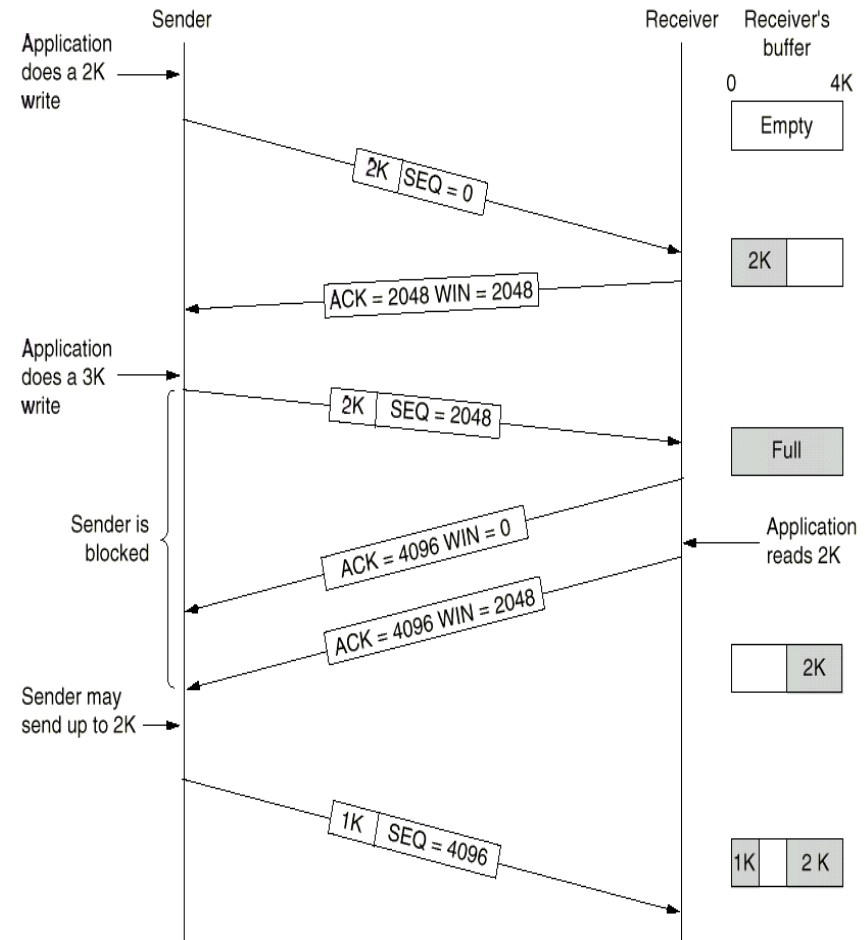


Fig. 6-29. Window management in TCP.



改进传输层的性能

- 策略1：发送方缓存应用程序的数据，等到形成一个比较大的段再发出
- 策略2：在有可能进行“捎带”的情况下，接收方延迟发送确认段
- 策略3：使用**Nagle**算法
 - 当应用程序每次向传输实体发出一个字节时，传输实体发出第一个字节并缓存所有其后的字节直至收到对第一个字节的确认
 - 然后将已缓存的所有字节组段发出并对再收到的字节缓存，直至收到下一个确认



改进传输层的性能（续）

■ 策略4：使用Clark算法解决傻窗口症状（silly window syndrome）

- 傻窗口症状：当应用程序一次从传输层实体读出一个字节时，传输层实体会产生一个一字节的窗口更新段，使得发送方只能发送一个字节
- 解决办法：限制收方只有在具备一半的空缓存或最大段长的空缓存时，才产生一个窗口更新段

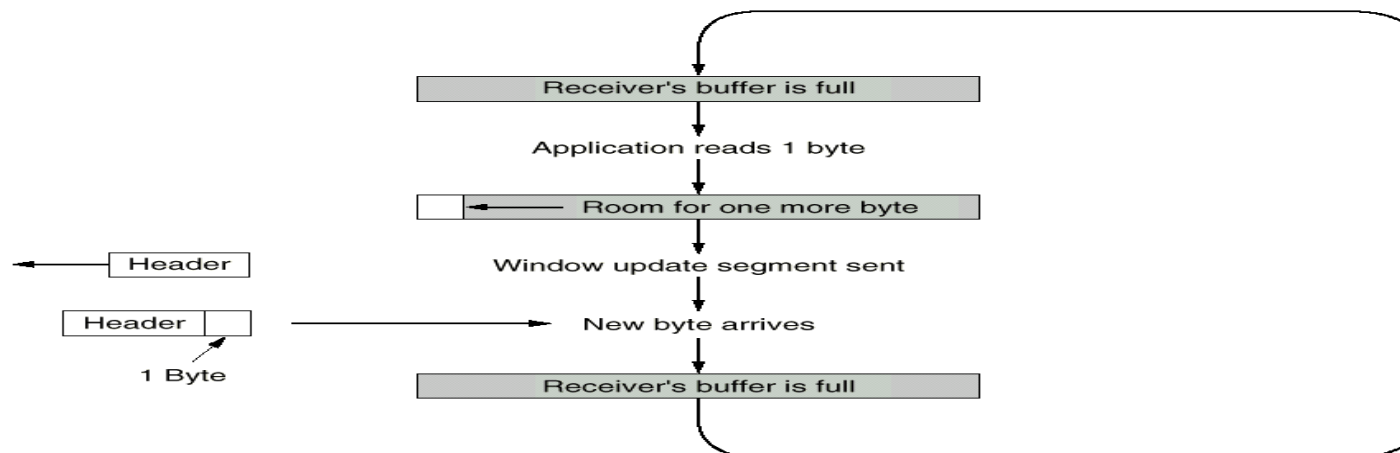


Fig. 6-30. Silly window syndrome.



TCP拥塞控制

- 导致网络拥塞的两个潜在因素是：网络转发能力和接收方接收能力
- 出现拥塞的两种情况
 - 快网络小缓存接收者
 - 慢网络大缓存接收者

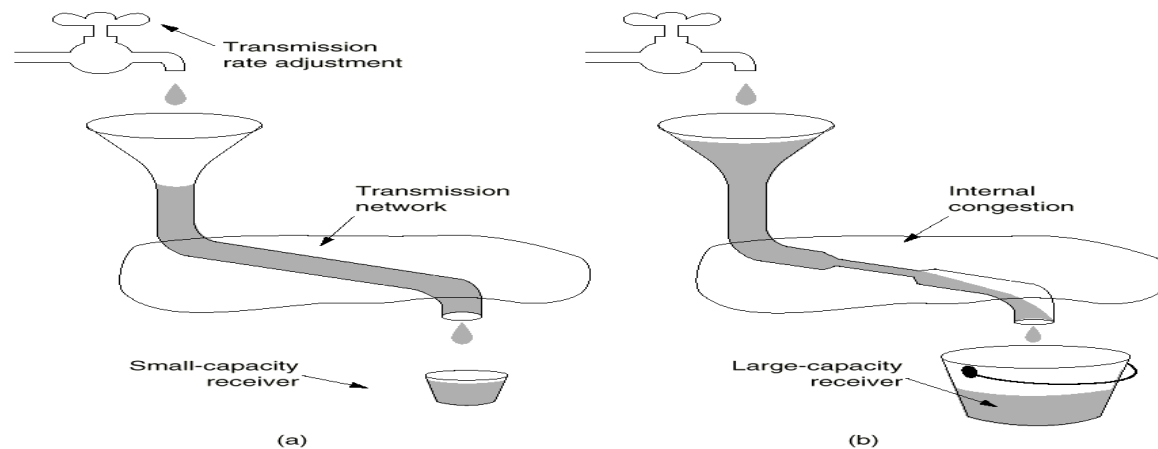


Fig. 6-31. (a) A fast network feeding a low-capacity receiver.
(b) A slow network feeding a high-capacity receiver.



TCP拥塞控制（续）

- **TCP处理第一种拥塞的措施**
 - 在连接建立时声明最大可接收段长度
 - 利用可变滑动窗口协议防止出现拥塞
- **TCP处理第二种拥塞的措施**
 - 发送方维护两个窗口：可变发送窗口和拥塞窗口，按两个窗口的最小值发送
 - 拥塞窗口按照慢启动（**slow start**）算法和拥塞避免（**congestion avoidance**）算法变化



慢启动算法

- 连接建立时拥塞窗口（**congwin**）初始值为该连接允许的最大发送段长**MSS**，阈值（**threshold**）为**64K**
- 发出一个最大段长的**TCP**段，若收到正确确认，则拥塞窗口变为两个最大段长
- 发出（拥塞窗口/最大段长）个最大长度的**TCP**段，若都得到确认，则拥塞窗口加倍
- 重复上一步，直至发生丢包产生的超时事件，或拥塞窗口大于阈值

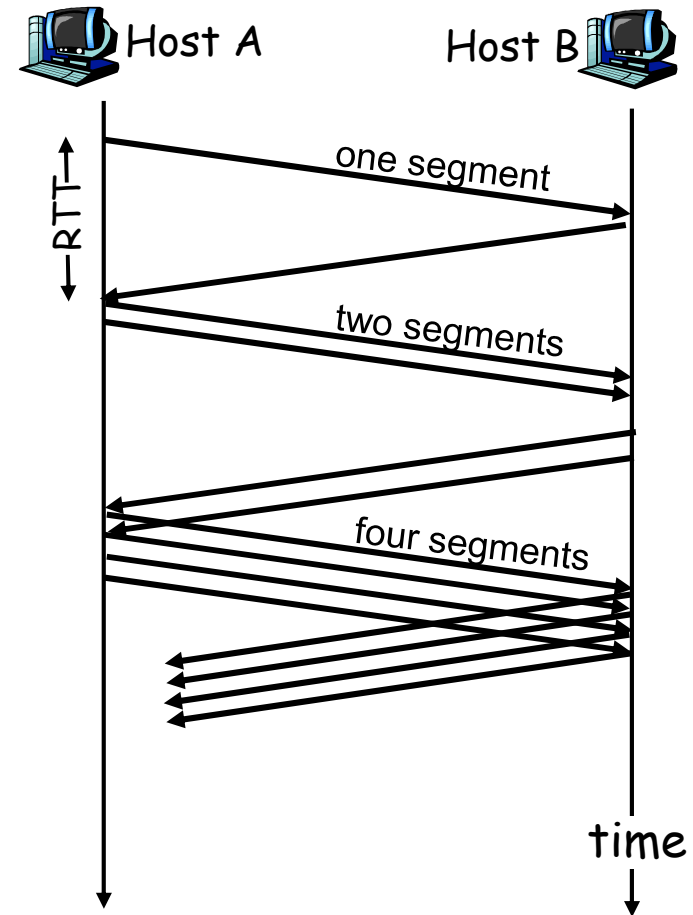


慢启动算法 (续)

慢启动算法

```
initialize: Congwin = 1  
for (each segment ACKed)  
    Congwin++  
until (loss event OR  
      CongWin  $\geq$  threshold)
```

- 窗口大小在每个**RTT**周期内指数增长





拥塞避免算法

- 当拥塞窗口大于阈值时，拥塞窗口开始线性增长，一个**RTT**周期增加一个最大段长，直至发生丢包产生的超时事件
- 超时事件发生后，阈值设置为当前拥塞窗口大小的一半，拥塞窗口重新设置为一个最大段长
- 执行慢启动算法



拥塞避免算法 (续)

Congestion avoidance

```
/* slowstart is over */  
/* Congwin > threshold */  
Until (loss event) {  
    every w segments ACKed:  
        Congwin++  
}  
threshold = Congwin/2  
Congwin = 1  
perform slowstart
```

w is the current value of the congestion window, and w is larger than threshold. After w acknowledgments have arrived, TCP replaces w with $w+1$.

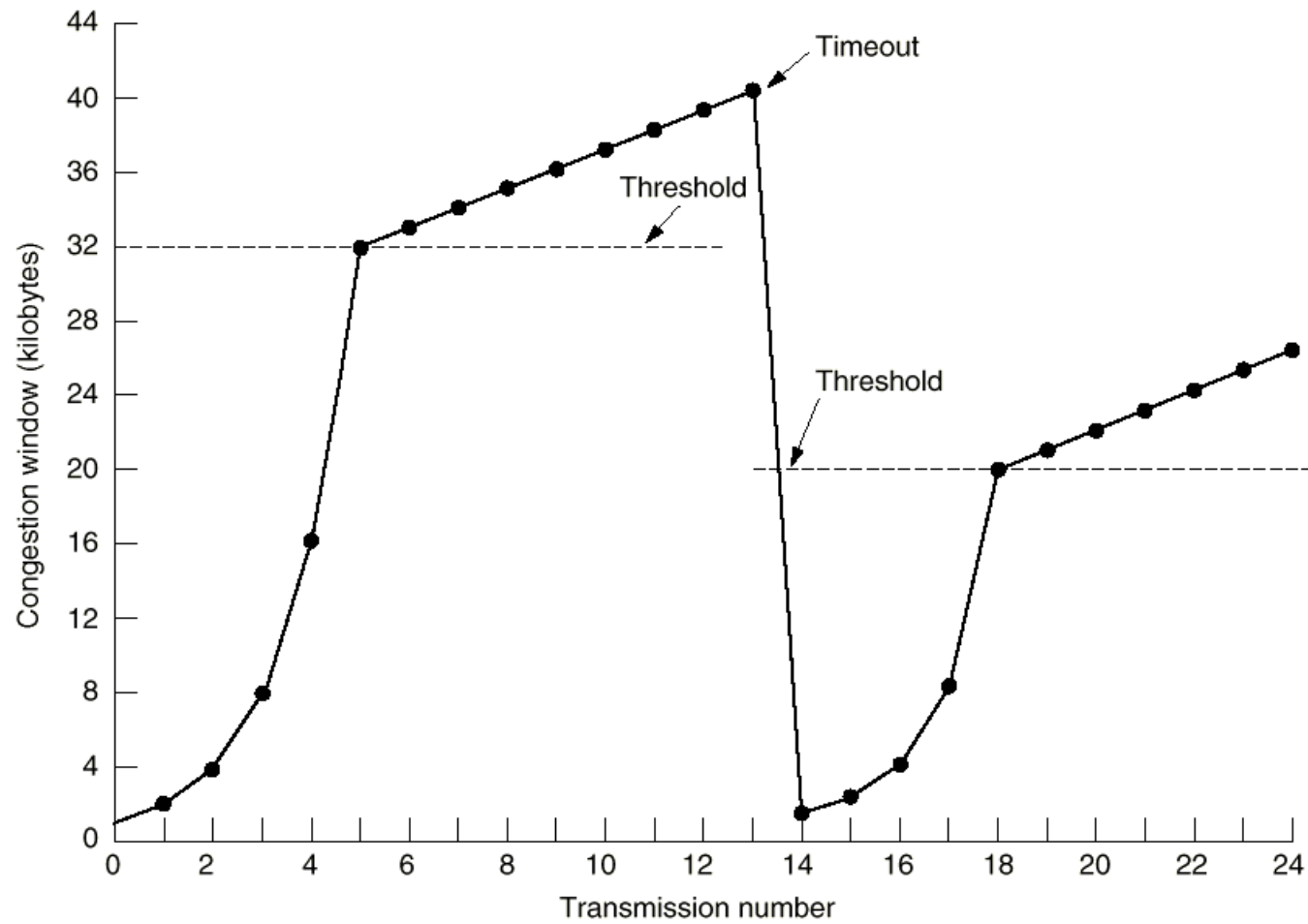


Fig. 6-32. An example of the Internet congestion algorithm.



例题

- **A、B双方已经建立了TCP连接，初始阈值为32K字节(1K = 1024)，最大发送段长MSS为1K字节。发送方向为A->B，B没有数据要发送，B每收到一个数据段都会发出一个应答段。在整个过程中上层一直有数据要发送，并且都以MSS大小的段发送。A的发送序列号从0开始。**
 - 在传输过程中，**A收到1个ACK为10240的数据段，收到这个应答段后，A处拥塞窗口的大小是多少？**
 - 当收到**ACK为32768**的数据段后，**A处拥塞窗口的大小是多少？**
 - 当阈值为**32K字节**、拥塞窗口为**40K字节**时，发送方发生了超时，求超时发生后拥塞窗口的大小和阈值的大小。



例题（续）

- 1.收到的是对第**10**个段的应答，变化后拥塞窗口的大小为**11**
- 2.收到的是对第**32**个段的应答，这时拥塞窗口已经超过阈值，应该使用线性增长，变化后的拥塞窗口大小为 **32 K**字节。
- 3.拥塞窗口 = **1 MSS = 1KB**，阈值 = **40 / 2 = 20 KB**



UDP 协议

- **UDP: User Datagram Protocol**
- **RFC 768**
- **为什么会有UDP**
 - 不需要建立连接，延迟小
 - 简单，没有连接状态
 - 报文头小
 - 没有拥塞控制，可以尽快的发送
- **UDP协议的特点**
 - 简单的传输协议
 - “**best effort**”服务，**UDP** 报文可能会丢失、乱序
 - 无连接
 - 收发双方不需要握手
 - 每个**UDP**报文的处理都独立于其他报文



UDP协议 (续)

- 经常用于流媒体应用
 - 可以容忍丢包
 - 速率敏感
- **UDP**的其他应用范围
 - **RIP**, 路由信息周期发送
 - **DNS**, 避免**TCP**连接建立延迟
 - **SNMP**, 当网络拥塞时, 网管也要运行。网管信息带内 (**in-band**) 传输, 用**UDP**比用可靠的、具有拥塞控制的**TCP**效果要好
 - 基于**UDP**的可靠传输: 应用程序自己定义错误恢复



UDP 头格式

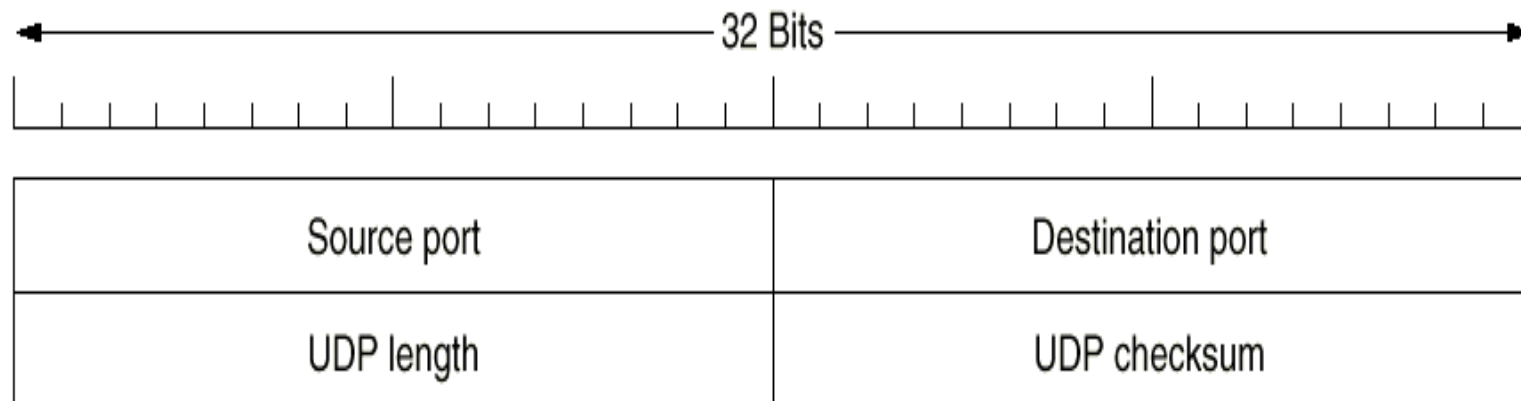


Fig. 6-34. The UDP header.



第九章

网络应用



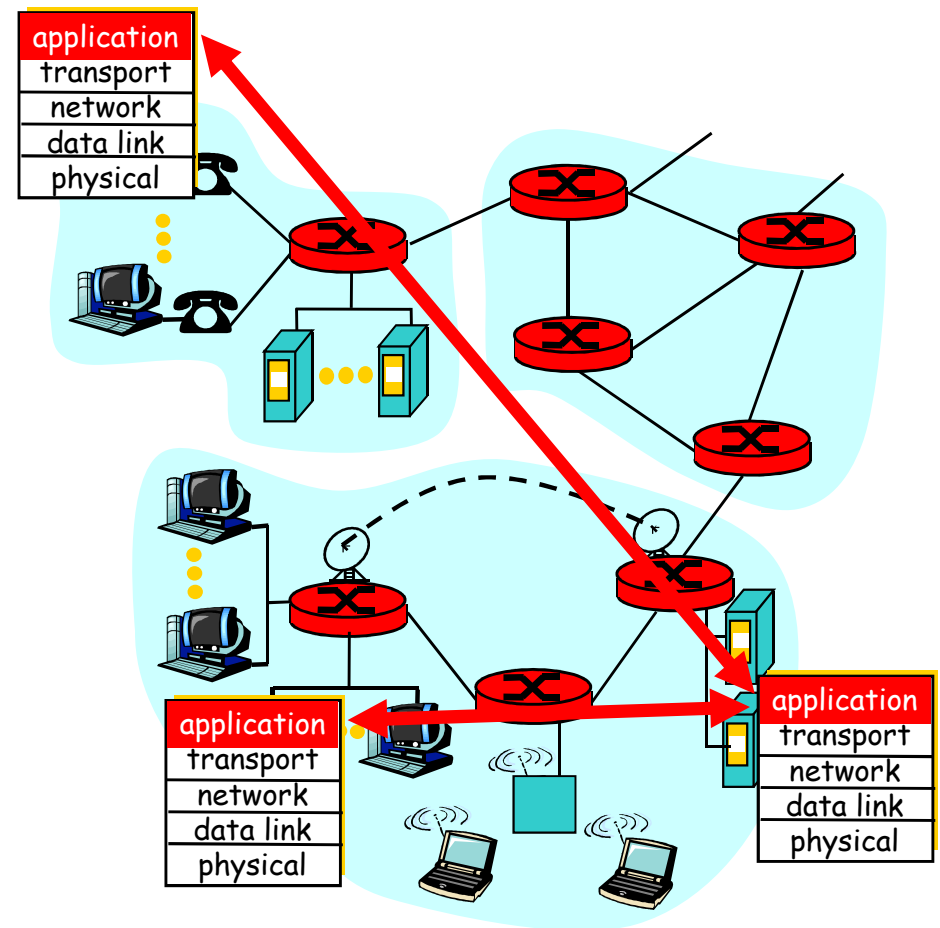
主要内容

- 应用层概述
- 客户/服务器模型
- 域名服务
- 简单网络管理协议
- 电子邮件
- **WWW**
- 文件传输协议



应用层概述

- 网络应用程序：互相通信的分布式进程
 - 在网络主机上的用户空间运行
 - 互相交换消息
 - 比如**email**、**ftp**和**web**
- 应用层协议
 - 应用程序的一部分
 - 定义应用程序之间交换的信息以及相应的动作
 - 利用底层协议提供的服务





应用层术语

- 一个进程（线程）是运行于主机上的一个程序
- 在同一主机上的进程（线程）利用操作系统提供的 **IPC (interprocess communication)** 进行通信
- 在不同主机上运行的进程利用应用层协议进行通信
- 用户代理 (**user agent**) 是指用户和网络应用程序间的接口。比如**web**浏览器，流媒体播放器等



应用程序编程接口

- 应用程序编程接口**API** (**application programming interface**)
 - 定义应用程序和传输层之间的接口
 - **socket: Internet API**
 - 两个进程通过向**socket**写数据和读数据来通信
- **Q:** 进程如何指明要与之通信的另一个进程
 - **IP**地址指明该进程所在的主机
 - 端口号指明该主机应该把收到的数据交给哪个当地进程



应用程序所需的传输服务

■ 数据丢失容忍度

- 有的应用程序可以容忍一定程度的数据丢失，例如音频应用
- 有的应用程序要求**100%**的可靠传输，例如文件传输

■ 带宽容忍度

- 有的程序需要一定的带宽才能工作，例如多媒体
- 有的程序则使用它所能得到的全部带宽，例如文件传输

■ 延迟容忍度

- 有的程序要求低延迟，例如**IP**电话和交互游戏
- 有的程序可以容忍较大延迟，例如电子邮件



应用程序所需的传输服务（续）

应用	数据丢失	带宽	时间敏感
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kb-1Mb video: 10Kb-5Mb	yes, 100's msec
stored audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	few Kbps up	yes, 100's msec
financial apps	no loss	elastic	yes and no



互联网传输协议提供的服务

TCP服务：

- 面向连接：用户端和服务端需要建立连接
- 接收和发送进程间的可靠传输
- 流量控制：发送方不会淹没接收方
- 拥塞控制：网络拥塞时限制发送方发送
- 不提供：延迟保证，最小带宽保证

UDP服务：

- 接收和发送进程间的不可靠传输
- 不提供：连接建立，可靠性、流量控制、拥塞控制和带宽保证



互联网应用和使用的相应协议

应用	应用层协议	底层传输层协议
e-mail	smtp [RFC 821]	TCP
remote terminal access	telnet [RFC 854]	TCP
Web	http [RFC 2068]	TCP
file transfer	ftp [RFC 959]	TCP
streaming multimedia	proprietary (e.g. RealNetworks)	TCP or UDP
remote file server	NFS	TCP or UDP
Internet telephony	proprietary (e.g., Skype)	typically UDP



主要内容

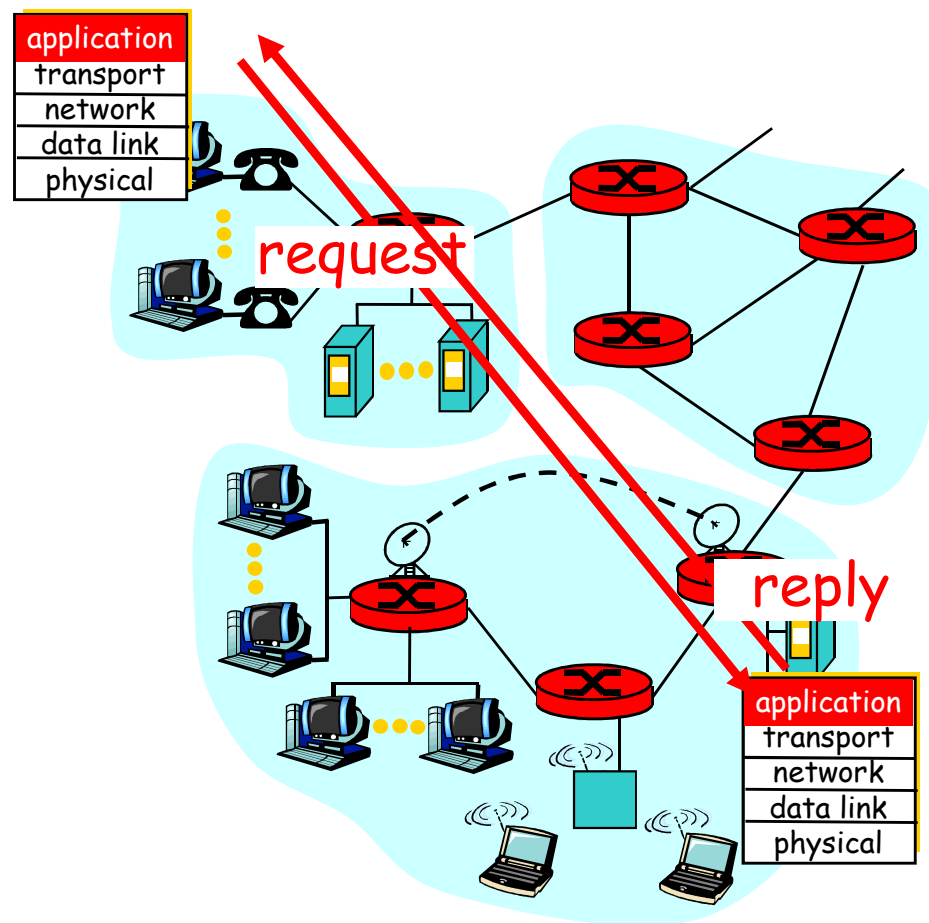
- 应用层概述
- 客户/服务器模型
- 域名服务
- 简单网络管理协议
- 电子邮件
- **WWW**
- 文件传输协议



客户/服务器模型

■ 基本概念

- 客户/服务器模型是网络应用的基础
- 客户/服务器分别指参与一次通信的两个应用实体，客户方主动地发起通信请求，服务器方被动地等待通信的建立，并提供服务





客户/服务器模型（续）

■ 客户软件

- 任何一个应用程序当需要进行远程访问时成为客户，这个应用程序也要完成一些本地的计算
- 一般运行于用户的个人计算机上
- 向服务器主动发起通信请求
- 不需要特殊的硬件和复杂的操作系统

■ 服务器软件

- 是专用的提供某种服务的特权程序，可以同时处理多个远程客户的请求
- 一般在系统启动时被执行，并连续运行以处理多次会话
- 被动地等待远程客户发起通信
- 需要特殊的硬件和复杂的操作系统



客户/服务器模型（续）

- 数据在客户和服务器之间是双向流动的，一般是客户发出请求，服务器给出响应
- 服务器软件的并发性
 - 由于服务器软件要支持多个客户的同时访问，必须具备并发性
 - 服务器软件为每个新到的客户创建一个进程或线程来处理与这个客户的通信
- 服务器软件的组成
 - 服务器软件一般分为两部分：一部分用于接受请求并创建新的进程或线程，另一部分用于处理实际的通信过程



客户/服务器模型（续）

- 客户/服务器之间使用的传送层协议
 - 基于连接的**TCP**协议
 - 要求建立和释放连接，适用于可靠的交互过程
 - 无连接的**UDP**协议
 - 适用于可靠性要求不高的或实时的交互过程
 - 同时使用**TCP**和**UDP**的服务
 - 有两种服务器软件的实现或服务器软件同时和**TCP**、**UDP**协议交互，不对客户做限制



主要内容

- 应用层概述
- 客户/服务器模型
- 域名服务
- 简单网络管理协议
- 电子邮件
- **WWW**
- 文件传输协议



域名服务

■ 产生原因

- **32比特的IP地址**难于记忆，符号地址便于记忆，因此需要一个完成二者之间相互转换的机制。比如用 **netlab.cs.tsinghua.edu.cn**表示**166.111.69.241**
- 当网络规模比较小时，例如**ARPANET**，每台主机只需查找一个文件（**UNIX的host**），该文件中列出了主机与**IP地址**的对应关系
- 当网络规模很大时，上述方法就不适用了，因此产生了域名系统**DNS**（**Domain Name System**）



DNS概述

- 域名系统是一个典型的客户/服务器交互系统
- 域名系统是一个多层次的、基于域的命名系统，并使用分布式数据库实现这种命名机制
- 操作过程
 - 当应用程序需要进行域名解析时，它成为域名系统的一个客户。它向本地域名服务器发出请求，请求以**UDP**段格式发出
 - 本地域名服务器找到对应的**IP**地址后，给出响应
 - 当本地域名服务器无法完成域名解析时，它临时变成其上级域名服务器的客户，继续解析，直到该域名解析完成
- **RFC 1034, 1035**



域名的结构

- 互联网的顶级域名分为组织结构和地理结构两种。每个域对它下面的子域和机器进行管理
- **DNS**中，域名是由“.”所分开的字符、数字串组成的，例如**www.tsinghua.edu.cn**
- 域名是大小写无关的，“**edu**”和“**EDU**”相同。域名最长**255**个字符，每部分最长**63**个字符

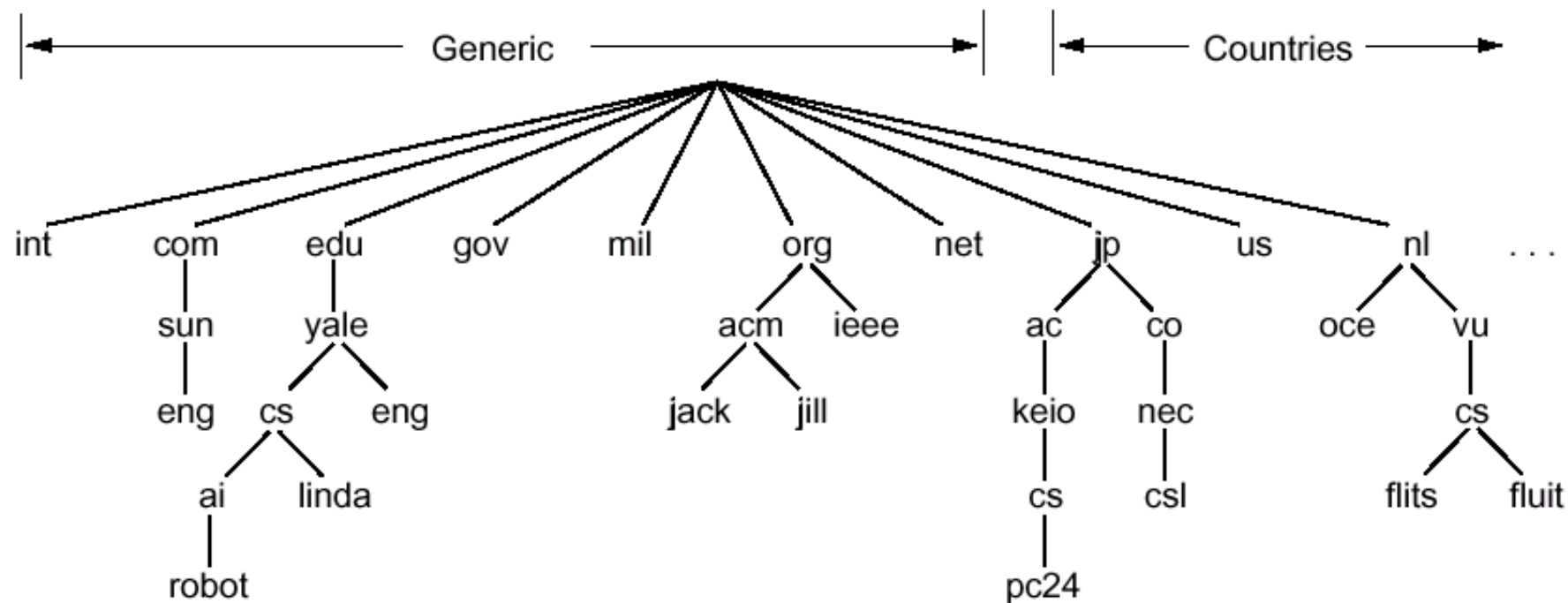


Fig. 7-25. A portion of the Internet domain name space.



资源记录

- 在**DNS**的数据库中用资源记录来表示主机和子域的信息，当应用程序进行域名解析时，得到的便是域名所对应的资源记录
- 资源记录是一个五元式
 - **Domain_name Time_to_live Type Class Value**

Type	Meaning	Value
SOA	Start of Authority	Parameters for this zone
A	IP address of a host	32-Bit integer
MX	Mail exchange	Priority, domain willing to accept email
NS	Name Server	Name of a server for this domain
CNAME	Canonical name	Domain name
PTR	Pointer	Alias for an IP address
HINFO	Host description	CPU and OS in ASCII
TXT	Text	Uninterpreted ASCII text

Fig. 7-26. The principal DNS resource record types.



资源记录 (续)

- **Type=A**
 - **Name:** hostname
 - **Value:** IP地址
- **Type=MX**
 - **Value:** 与**name**对应的邮件服务器的主机名 (hostname)
- **Type=NS**
 - **Name:** 域名 (例如, **edu.cn**)
 - **Value:** 该域权威域名服务器的**IP**地址
- **Type=CNAME**
 - **Name:** 规范名称 (**canonical name**) 的别名
 - **Value:** 规范名称



; Authoritative data for cs.vu.nl

cs.vu.nl.	86400	IN SOA	star boss (952771,7200,7200,2419200,86400)
cs.vu.nl.	86400	IN TXT	"Faculteit Wiskunde en Informatica."
cs.vu.nl.	86400	IN TXT	"Vrije Universiteit Amsterdam."
cs.vu.nl.	86400	IN MX	1 zephyr.cs.vu.nl.
cs.vu.nl.	86400	IN MX	2 top.cs.vu.nl.

flits.cs.vu.nl.86400	IN HINFO	Sun Unix
flits.cs.vu.nl.86400	IN A	130.37.16.112
flits.cs.vu.nl.86400	IN A	192.31.231.165
flits.cs.vu.nl.86400	IN MX	1 flits.cs.vu.nl.
flits.cs.vu.nl.86400	IN MX	2 zephyr.cs.vu.nl.
flits.cs.vu.nl.86400	IN MX	3 top.cs.vu.nl.
www.cs.vu.nl.86400	IN CNAME	Estar.cs.vu.nl
ftp.cs.vu.nl. 86400	IN CNAME	zephyr.cs.vu.nl

rowboat	IN A	130.37.56.201
	IN MX	1 rowboat
	IN MX	2 zephyr
	IN HINFO	Sun Unix

little-sister	IN A	130.37.62.23
	IN HINFO	Mac MacOS

laserjet	IN A	192.31.231.216
	IN HINFO	"HP Laserjet IIISi" Proprietary

Fig. 7-27. A portion of a possible DNS database for *cs.vu.nl*



域名服务器

■ 区域划分

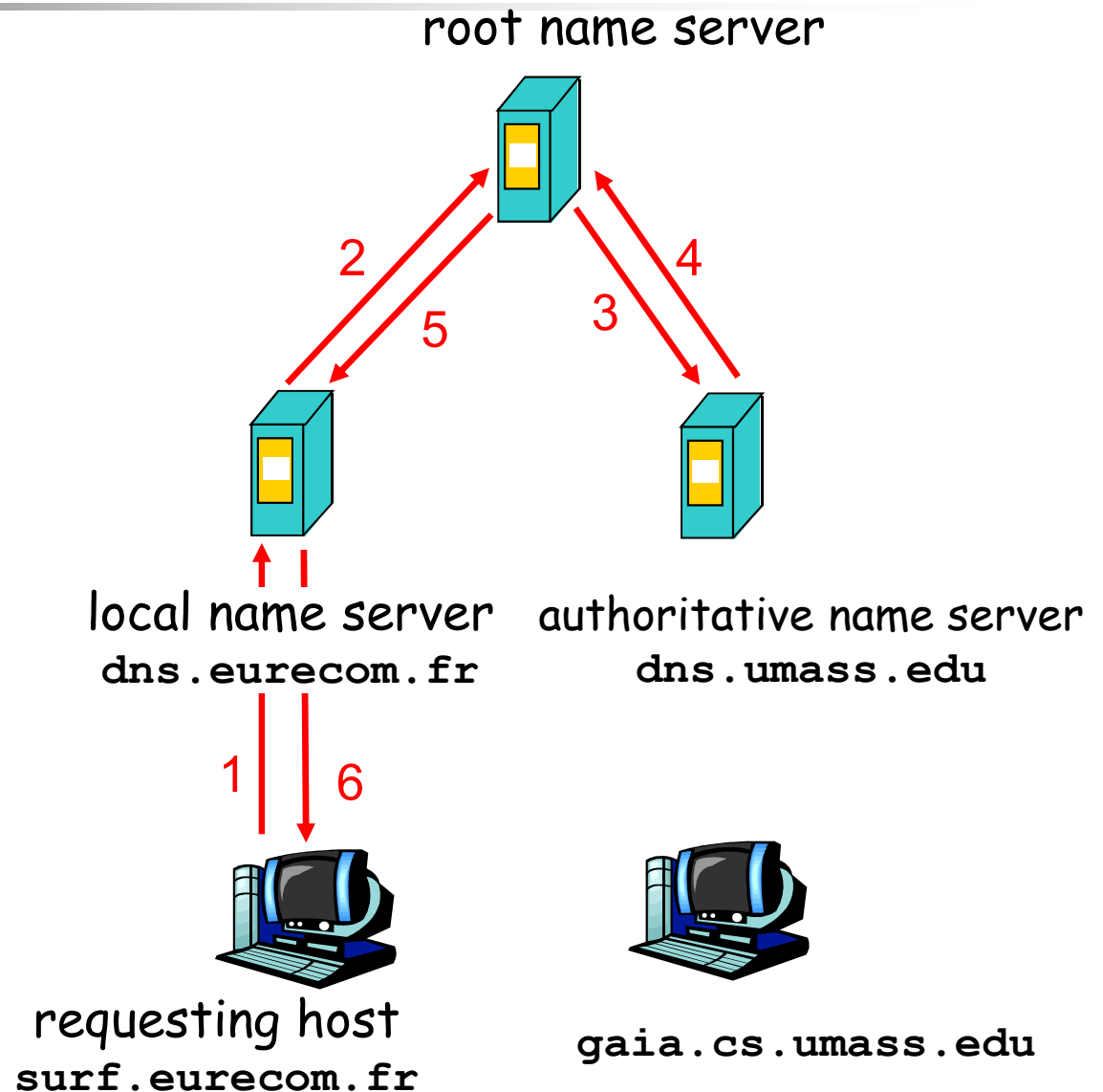
- **DNS**将域名空间划分为许多区域(**zone**)，每个区域覆盖了域名空间的一部分
- 区域的边界划分是人工设置的，比如：**edu.cn**和**tsinghua.edu.cn**不同的区域，分别有各自的域名服务器
- 每个区域有一个主域名服务器和若干个备份域名服务器



Simple DNS example

主机 **surf.eurecom.fr** 需要 **gaia.cs.umass.edu** 的 IP 地址

1. 与本地DNS服务器 **dns.eurecom.fr** 联系
2. 如有必要 **dns.eurecom.fr** 与根域名服务器联系
3. 如有必要，根域名服务器联系 authoritative 域名服务器 **dns.umass.edu**



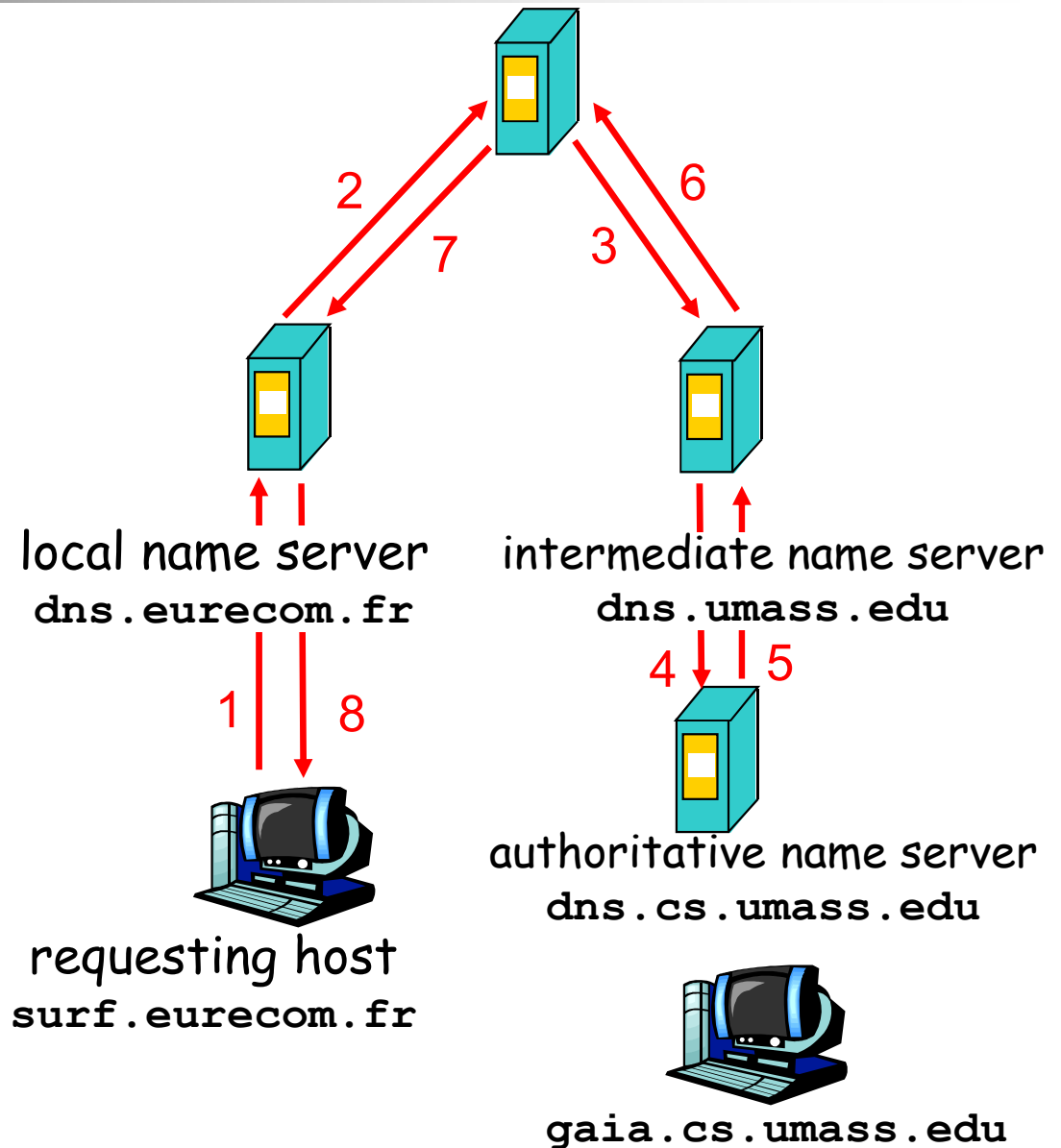


DNS example

root name server

根域名服务器:

- 可能不知道 authoritative 域名服务器
- 可能知道中间的域名服务器，中间域名服务器知道如何与 authoritative 域名服务器联系





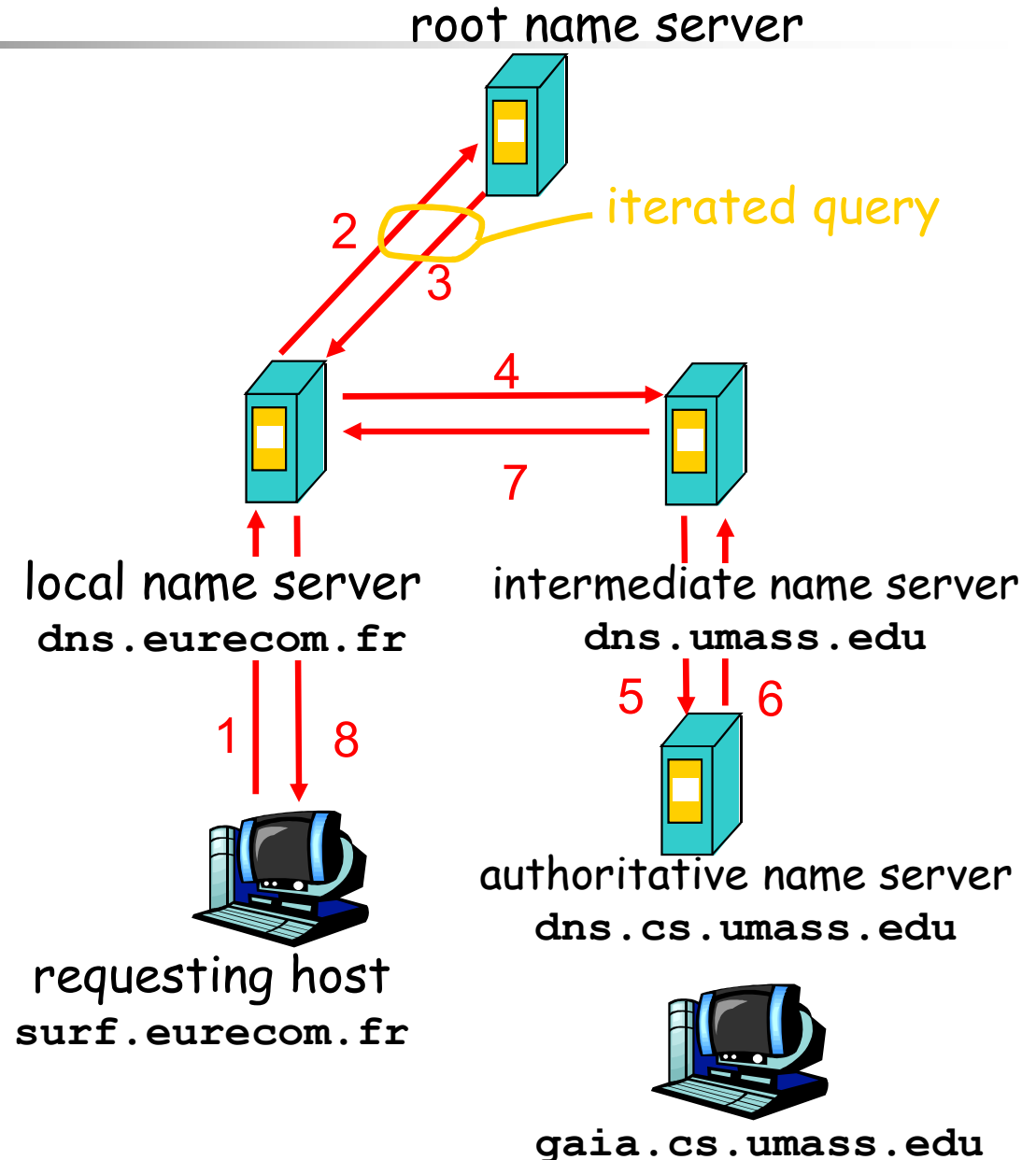
DNS: iterated queries

recursive query:

- puts burden of name resolution on contacted name server
- heavy load?

iterated query:

- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"





主要内容

- 应用层概述
- 客户/服务器模型
- 域名服务
- 简单网络管理协议
- 电子邮件
- **WWW**
- 文件传输协议



简单网络管理协议

- **SNMP (Simple Network Management Protocol) 的产生**
 - 早期网络，如**ARPANET**，规模很小，可以通过执行“**PING**”等命令来发现网络故障
 - 网络规模变大，需要一个好的工具来管理网络。**1990年**发布**RFC 1157**，定义了**SNMP v1**
 - **SNMP v2, RFC 1441 ~ 1452**
- 网络管理的五个基本管理功能：性能管理、故障管理、配置管理、记账管理和安全管理
- **SNMP**是基于**UDP**的



SNMP 模型

- 被管理节点
 - 运行**SNMP**代理程序 (**SNMP agent**)，维护一个本地数据库，描述节点的状态和历史，并影响节点的运行
- 管理工作站
 - 运行专门的网络管理软件 (**manager**)，使用管理协议与被管理节点上的**SNMP**代理通信，维护管理信息库
- 管理信息
 - 每个站点使用一个或多个变量描述自己的状态，这些变量称为“对象 (**objects**)”，所有的对象组成管理信息库**MIB** (**Management Information Base**)。
- 管理协议 (**SNMP**)
 - 管理协议用于管理工作站查询和修改被管理节点的状态，被管理节点可以使用管理协议向管理站点产生“陷阱 (**trap**)”报告

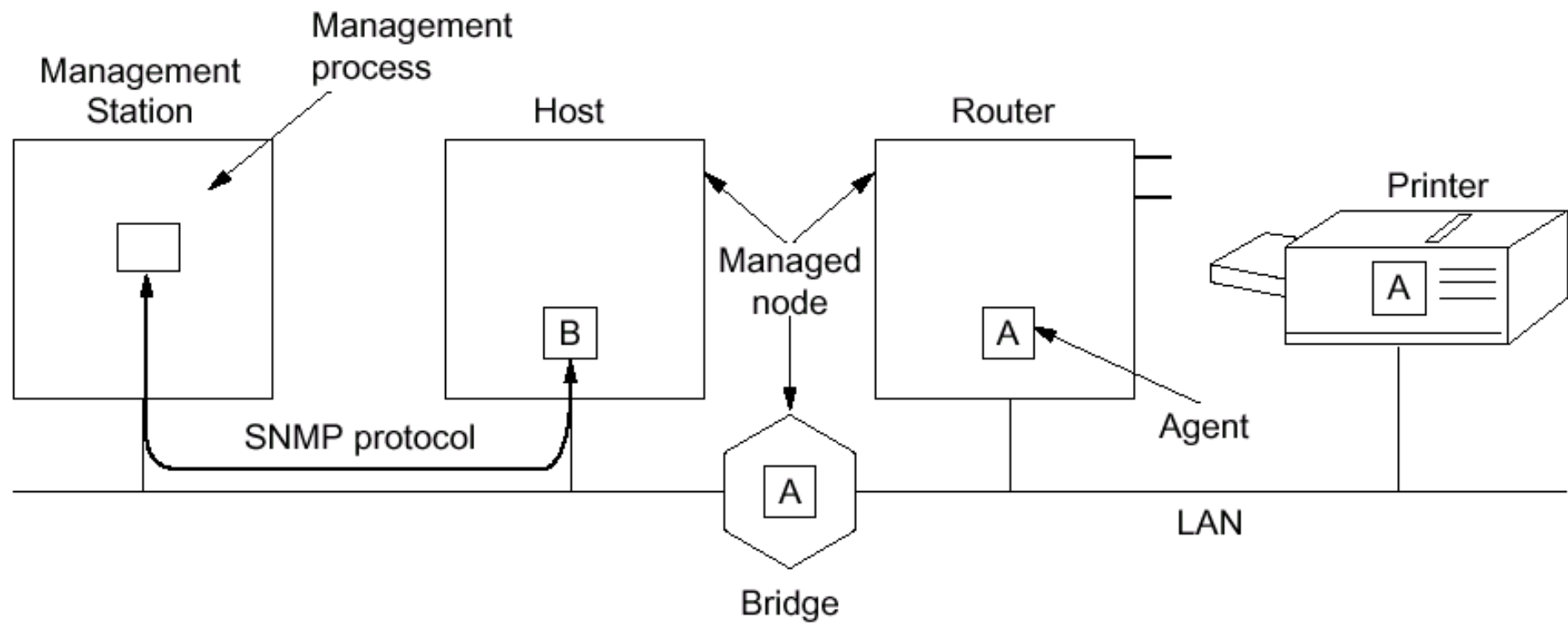


Fig. 7-30. Components of the SNMP management model.



抽象语法表示法1

■ 抽象语法表示法1（ASN.1）

- 是一种标准的对象定义语言
- 分为数据描述定义（**ISO 8824**）和传输语法定义（**ISO 8825**）两部分
- 可以作为异种计算机设备之间“对象”描述和传输的表示方法

■ ASN.1的基本数据类型

Primitive type	Meaning	Code
INTEGER	Arbitrary length integer	2
BIT STRING	A string of 0 or more bits	3
OCTET STRING	A string of 0 or more unsigned bytes	4
NULL	A place holder	5
OBJECT IDENTIFIER	An officially defined data type	6

Fig. 7-31. The ASN.1 primitive data types permitted in SNMP.



抽象语法表示法1 (续)

■ 对象命名树

- 对象命名树使用编码，唯一地确定每个标准中的对象。基于对象命名树，任何标准中的任意对象都可以用如下的对象表示符表示
- **{iso(1) identified-organizations(3)
dod(6) internet(1) mgmt(2) mib-
2(1) ..tcp(6)..}**
- 或者是 **{1 3 6 1 2 1 6}**

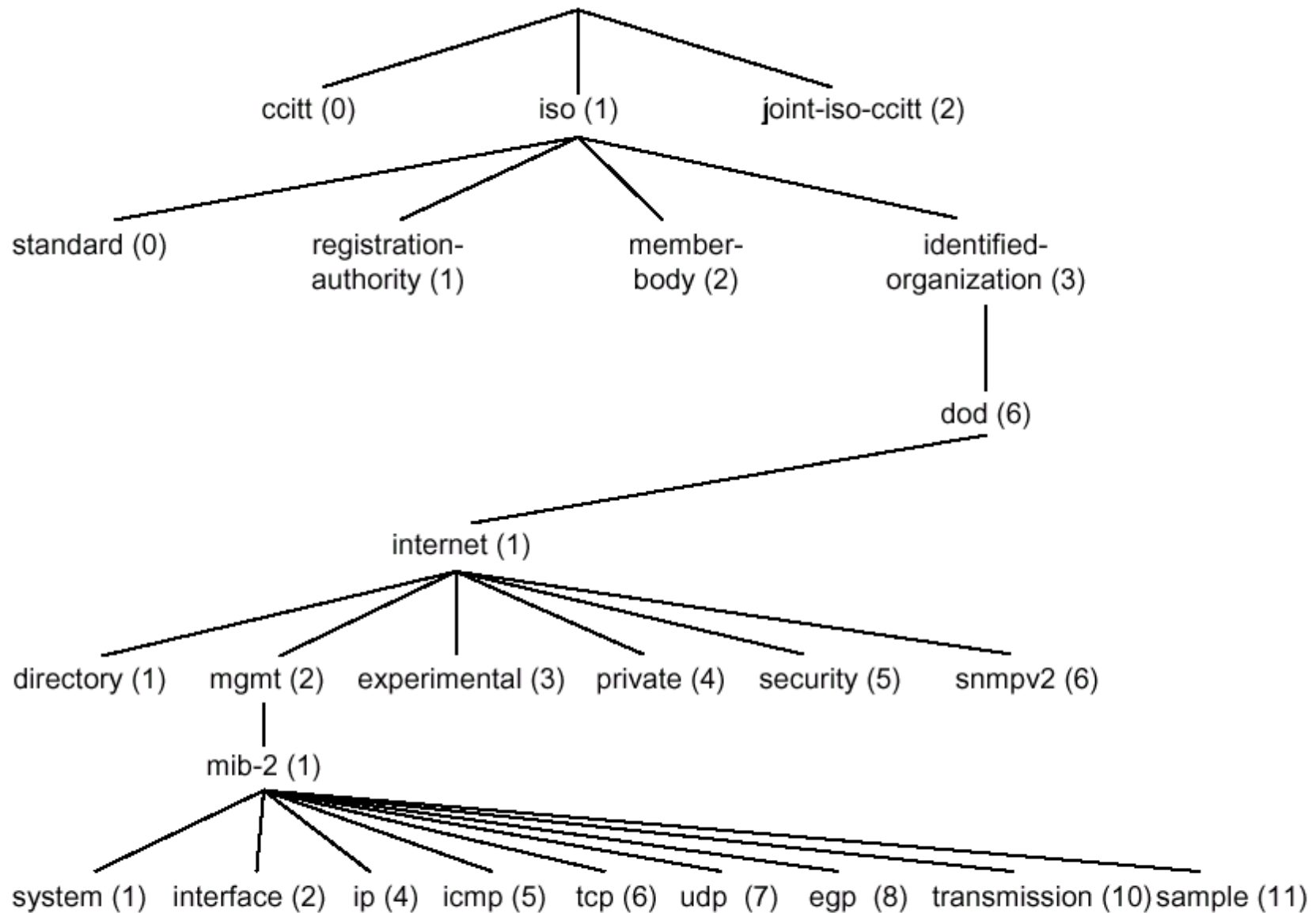


Fig. 7-32. Part of the ASN.1 object naming tree.



抽象语法表示法1 (续)

- **ASN.1**定义了**5**种方法构造新的类型
 - **SEQUENCE**: 多种类型的有序序列
 - **SEQUENCE OF**: 一种类型的一维有序序列
 - **SET**: 多种类型的无序集合
 - **SET OF**: 一种类型的无序集合
 - **CHOICE**: 创建一些类型的共同体 (**UNION**)
- 构造新类型的另一种方法是重新标记一个老的类型
 - 类似C语言中定义新的类型 (**#define ...**)
 - 标签有四类: **universal, application-wide, context-specific, private**
 - 例如, **Counter32 ::= [APPLICATION 1] INTEGER (0..4294967295)**



抽象语法表示法1 (续)

■ ASN.1的传输语法

- 基本编码规则**BER (Basic Encoding Rules)**定义了如何将**ASN.1**类型的值表示为无二义的字节序列
- 需要传输的内容
 - 标志符 (**type or tag**)
 - 数据长度域
 - 数据域



抽象语法表示法1（续）

- 标志符 (**type or tag**)
 - 包括三个子域
 - 当**tag**值在**0 ~ 30**之间时，用低**5**位表示
 - 当**tag**值大于**30**时，低**5**位为“**11111**”，用后面字节表示。每个标识字节包括**7**个数据位，最后一个字节高位为“**1**”，其它字节高位为“**0**”

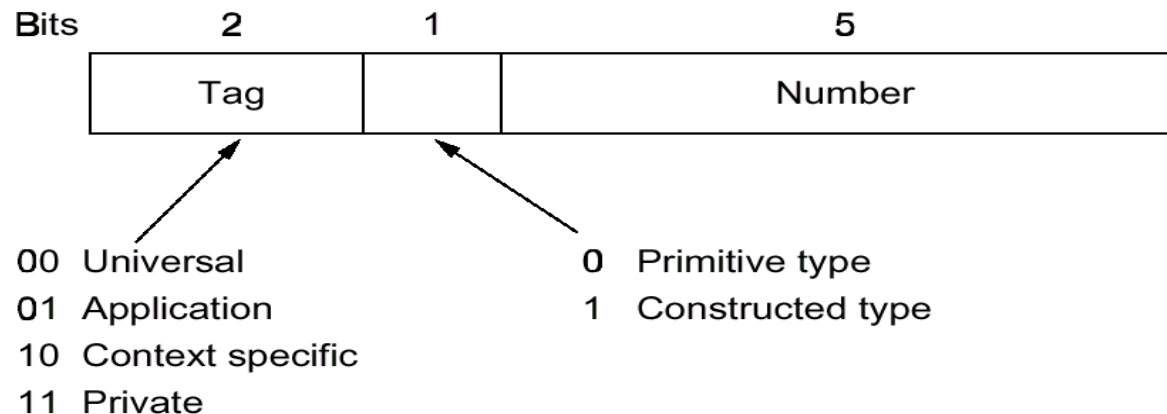


Fig. 7-33. The first byte of each data item sent in the ASN.1 transfer syntax.



抽象语法表示法1（续）

■ 数据域长度

- 当长度 < 128 字节时，用一个字节表示长度，高位为“0”
- 当长度 ≥ 128 字节时，第一个字节高位为“1”，低7位表示后面表示长度的字节个数，后面的若干个（ ≤ 127 ）字节表示长度
- 例，数据长度1000字节，则长度域包括3个字节，第一个字节为“10000010”，后两个字节为“00000011”和“11101000”



抽象语法表示法1 (续)

■ 数据域

- **INTEGER**: 二进制编码
- **BIT STRING**: 编码表示不变, 长度域表示字节个数, 并在传位串前先传一个字节表示位串最后一个字节不用的位数。
 - 例, 位串 “010011111” 传输时变为 “07 4f 80” (十六进制)
- **OCTET STRING**: 编码表示不变
- **NULL**: 长度域为0, 不传数据
- **OBJECT IDENTIFIER**: 按照命名树的编码整数序列编码, 前两个数 a, b 可用一个字节编码, 值为 $40a + b$
- 如果数据长度未知, 需要有结束标志



	Tag type	Tag Number	Length	Value	
Integer 49	000	000010	000000001	00110001	
Bit String '110'	000	000011	000000010	00000101	11000000
Octet String "xy"	000	000100	000000010	01111000	01111001
NULL	000	000101	000000000		
Internet object	000	000110	000000011	00101011	00000110 00000001
Gauge 32 14	010	000010	000000001	00001110	

Fig. 7-34. ASN.1 encoding of some example values.



管理信息结构和管理信息库

■ 管理信息结构**SMI**和管理信息库**MIB**

■ 定义

- **SNMP**在**ASN.1**的基础上，定义了四个宏，八个新数据类型来定义**SNMP**的数据结构，被称为管理信息结构**SMI**
- **SNMP**使用**SMI**首先将变量定义为“对象”（**object**），相关的对象被集成“组”（**group**），组最后被汇集成“模块”（**module**）

■ 管理信息库

- **SNMP**的**MIB**包含**10**个组。网络管理工作站通过使用**SNMP**协议，向被管理节点中的**SNMP**代理发出请求，查询这些对象的值



管理信息结构和管理信息库（续）

- 每个对象有以下四个属性：
 - 对象类型(object type): 定义了对象的名字
 - 语法(syntax): 指定了数据类型。
 - 存取(access): 表示了对象的存取级别，合法的值有只读、只写、读写和不可存取
 - 状态(status): 定义了对象的实现需要，
 - 必备的：被管理结点必须实现该对象
 - 可选的：被管理结点可能实现该对象
 - 已经废弃的：被管理结点不需要实现该对象



SNMP协议

- 定义了网络管理工作站和**SNMP**代理之间的通信过程和协议数据单元
- 网络管理工作站发往**SNMP**代理的数据请求
Get-request, Get-next-request, Get-bulk-request
- 网络管理工作站发往**SNMP**代理的数据更新请求
Set-request
- 网络管理工作站与网络管理工作站之间的**MIB**交换
Inform-request
- **SNMP**代理发往网络管理工作站的陷阱报告
SnmpV2-trap



主要内容

- 应用层概述
- 客户/服务器模型
- 域名服务
- 简单网络管理协议
- 电子邮件
- **WWW**
- 文件传输协议



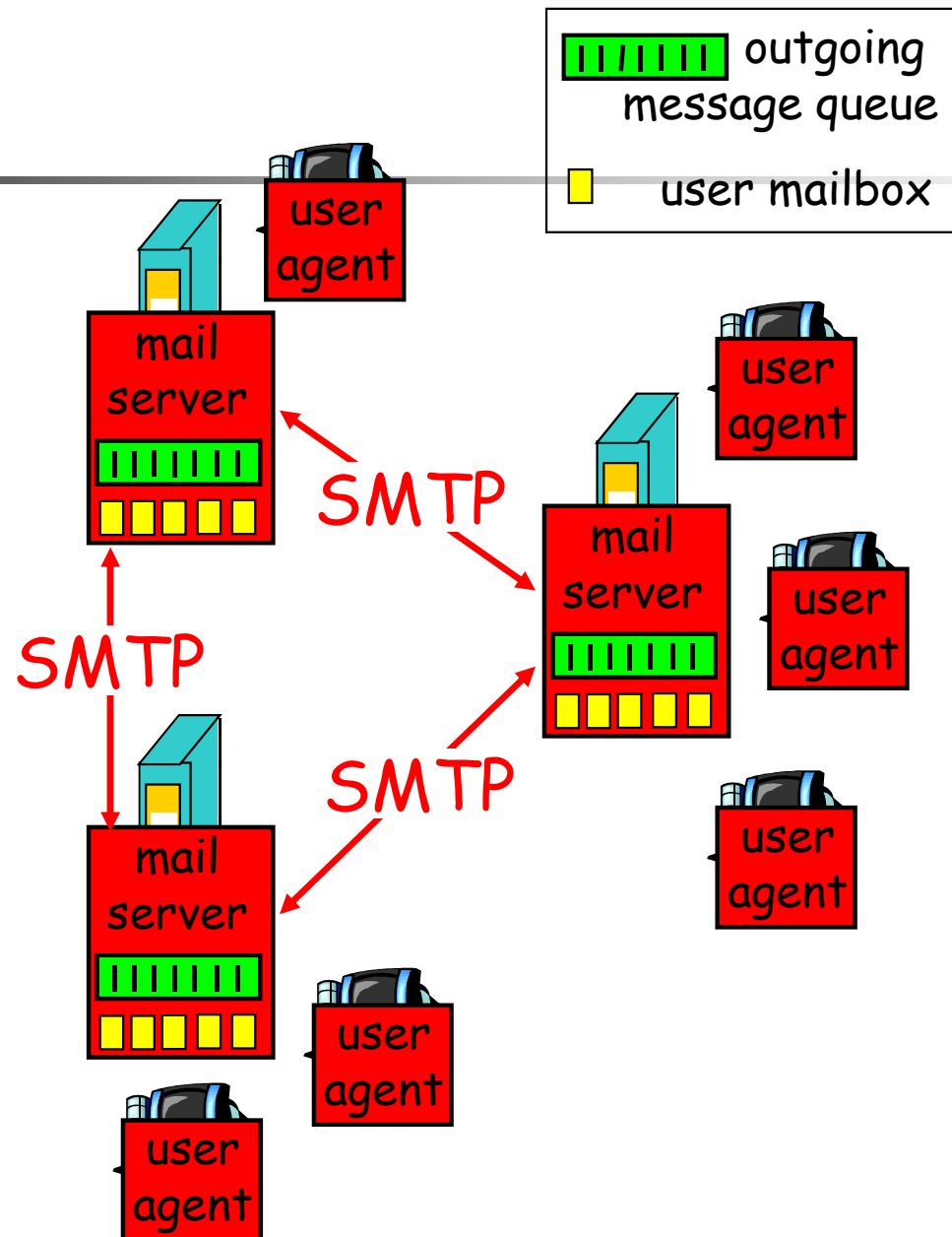
电子邮件

■ 相关协议标准

- **1982年ARPANET**提出了**RFC821(传输协议)**, **RFC822(消息格式)**作为电子邮件协议
- **1984年CCITT**提出了**X.400**建议, 但是没有得到普及

■ 体系结构

- **用户代理**: 允许用户阅读和发送电子邮件, 一般为用户进程
- **消息传输代理**: 将消息从源端发送至目的端, 一般为系统的后台进程
- **简单邮件传输协议SMTP(Simple Mail Transfer Protocol)**





电子邮件（续）

- 电子邮件系统提供的五大基本功能
 - 撰写：指创建消息或回答消息的过程
 - 传输：指将消息从发送者传出至接收者
 - 报告：将消息的发送情况报告给消息发送者
 - 显示：使用相应的工具软件将收到的消息显示给接收者
 - 处理：接收者对接收到的消息进行处理，存储/丢弃/转发等等
- 其它高级功能
 - 自动转发、自动回复
 - **mailbox**，创建邮箱存储邮件
 - **mailing list**
 - 抄送（**cc**）、高优先级、加密



电子邮件 (续)

■ 电子邮件的组成

- 信封：接收方的信息，如名字、地址、邮件的优先级和安全级别
- 信件内容：由信头和信体组成，信头包含了用户代理所需的控制信息，信体是真正的内容

■ 用户代理

■ 发送电子邮件

- **email地址**，例如，**webmaster@tsinghua.edu.cn**
- **mailing list**，例如，**students@cs.tsinghua.edu.cn**
- **X.400地址**，例如，**/C=US/SP=MASSACHUSETTS/L=CAMBRIDGE/PA=360 MEMORIAL DR./CN=KEN SMITH/**

■ 阅读电子邮件

- 用户代理在启动时检查用户的**mailbox**，通知用户是否有新邮件到来。并摘要性的显示邮件的主题、发送者及其邮件的状态



电子邮件（续）

■ 信件格式

■ RFC822

- 信件包括信封、若干信头域和信体

■ 电子邮件的扩展

■ MIME (Multipurpose Internet Mail

Extensions)，增加了对图像、声音、视频、可执行文件等的支持。使用不同的编码方法将信息转化为**ASCII**字符流

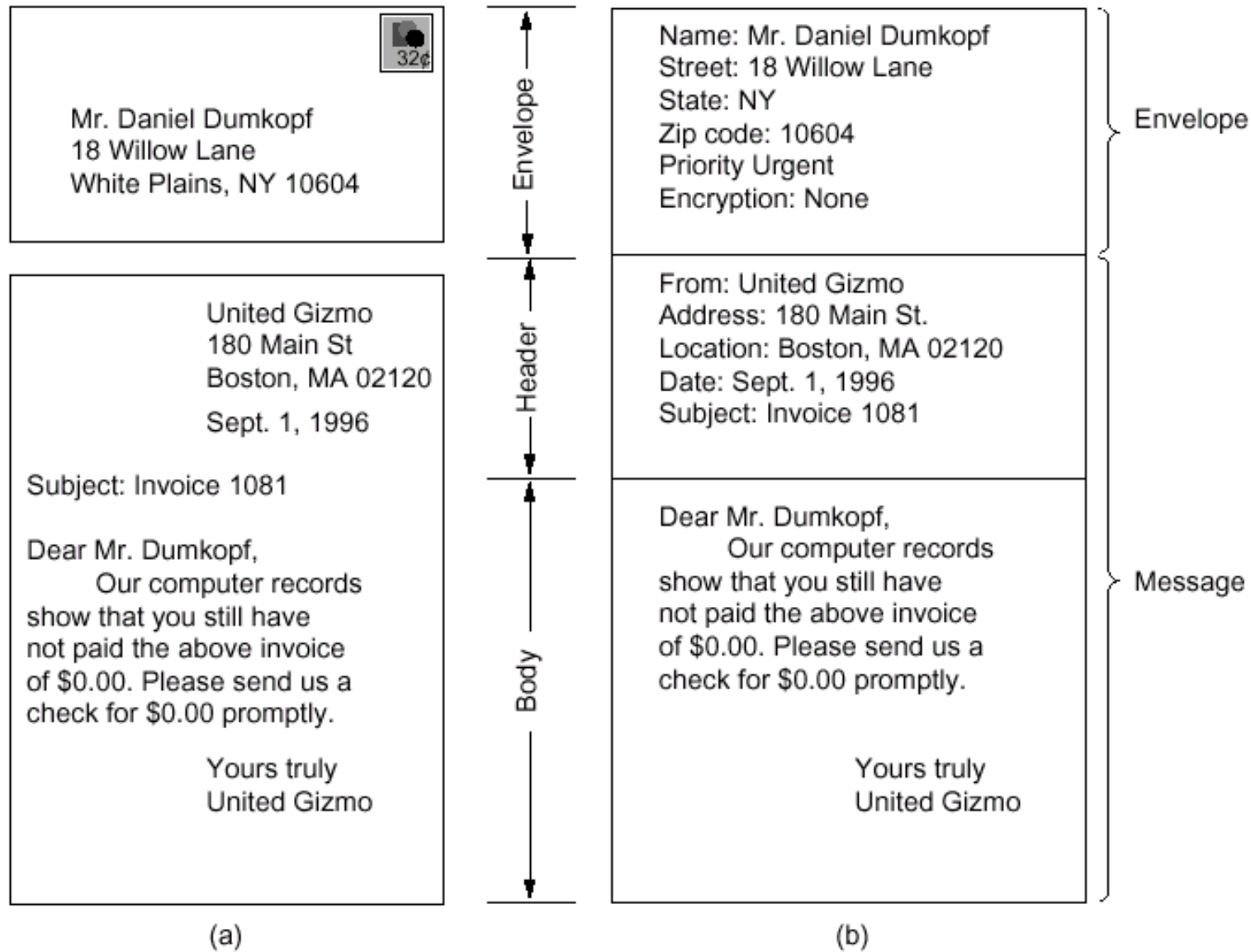


Fig. 7-39. Envelopes and messages. (a) Postal email. (b) Electronic mail.

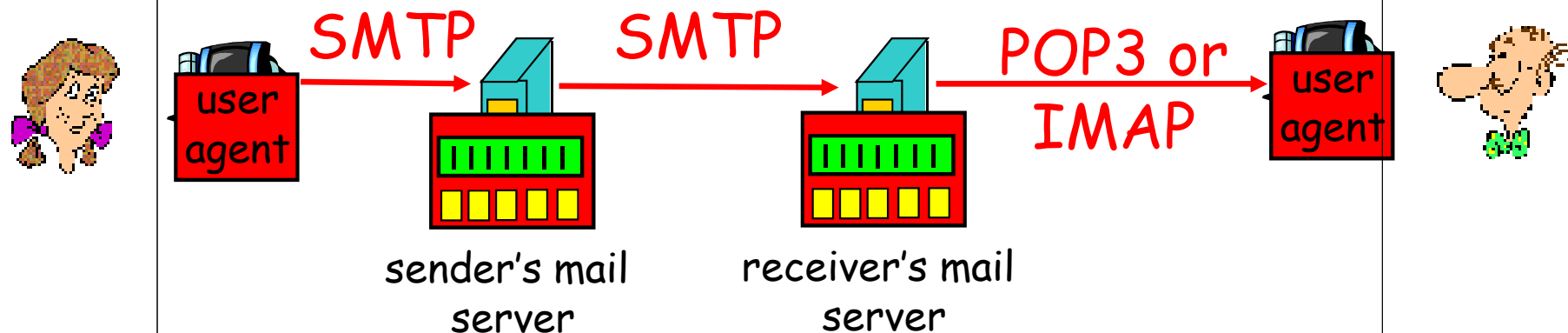


Header	Meaning
To:	Email address(es) of primary recipient(s)
Cc:	Email address(es) of secondary recipient(s)
Bcc:	Email address(es) for blind carbon copies
From:	Person or people who created the message
Sender:	Email address of the actual sender
Received:	Line added by each transfer agent along the route
Return-Path:	Can be used to identify a path back to the sender

Fig. 7-42. RFC 822 header fields related to message transport.



电子邮件（续）



- 消息传送协议

- INTERNET使用简单邮件传输协议SMTP完成电子邮件的交换



电子邮件（续）

■ 过程如下

- 消息传输代理在源端主机和目的主机的**25**号端口之间建立一条**TCP**连接，使用简单邮件传输协议**SMTP**协议进行通信
- 在**TCP**连接建立好之后，作为客户的邮件发送方等待作为服务器的邮件接收方首先传输信息
- 服务器首先发出准备接收的**SMTP**消息，客户向服务器发出**HELO**消息，服务器回答以**HELO**消息，双方进入邮件传输状态
- 邮件传输过程：客户首先发出邮件的发信人地址(**MAIL FROM**)，然后发出收信人的地址(**RCPT TO**)，服务器确认收信人存在后，发出可以继续发送的指示，客户发送真正的消息(**DATA**)，以“**CRLF.CRLF**”作为结束
- 当客户方邮件发送完后，服务器开始发送邮件至客户，过程同上
- 两个方向的发送完成后，释放**TCP**连接(**QUIT**)
- 持久（**Persistent**）方式



电子邮件（续）

■ 注意

- 消息以**7-比特ASCII**码为单位
- 某些特殊字符串(如**CRLF.CRLF**)不允许在消息中出现，需要编码(例如，**base64**)

■ 其它协议

- **POP3 (Post Office Protocol)**，**RFC 1939**，用户代理和邮箱不在同一机器上，用户代理使用此协议将邮箱中的信件取回本地
- **IMAP (Internet Mail Access Protocol)**，**RFC 1730**，收信人使用多个用户代理访问同一邮箱，邮件始终保持在邮箱中
- 加密电子邮件协议：**PGP**与**PEM**协议



Try SMTP

- **try smtp interaction for yourself**
 - **telnet servername 25**
 - **see 220 reply from server**
 - **enter HELO, MAIL FROM, RCPT TO, DATA, QUIT commands**
 - **above lets you send email without using email client (reader)**



Try SMTP (续)

■ Smtplib交互实例

S: 220 hamburger.edu

C: HELO crepes.fr

S: 250 Hello crepes.fr, pleased to meet you

C: MAIL FROM: <alice@crepes.fr>

S: 250 alice@crepes.fr... Sender ok

C: RCPT TO: <bob@hamburger.edu>

S: 250 bob@hamburger.edu ... Recipient ok

C: DATA

S: 354 Enter mail, end with "." on a line by itself

C: Do you like ketchup?

C: How about pickles?

C: .

S: 250 Message accepted for delivery

C: QUIT

S: 221 hamburger.edu closing connection



主要内容

- 应用层概述
- 客户/服务器模型
- 域名服务
- 简单网络管理协议
- 电子邮件
- **WWW**
- 文件传输协议



WWW

- **WWW(World Wide Web)**是用于访问遍布于互联网上的相互链接在一起的超文本的一种结构框架
- 一方面，用户可以按需获取信息；另一方面为信息发布提供方便途径
- 历史
 - **1989年**，设计**WWW**的思想产生于欧洲核研究中心**CERN**
 - **1991年**，第一个原型在美国的**Hypertext '91**会议上展示
 - **1993年**，第一个图形化浏览器，**Mosaic**
 - **1994年**，**Andreessen**创建**NETSCAPE**公司，开发**WEB**的客户和服务端软件
 - 同年，**CERN**和**MIT**共同创建**WWW**论坛，制定相关的协议标准，**<http://www.w3.org>**



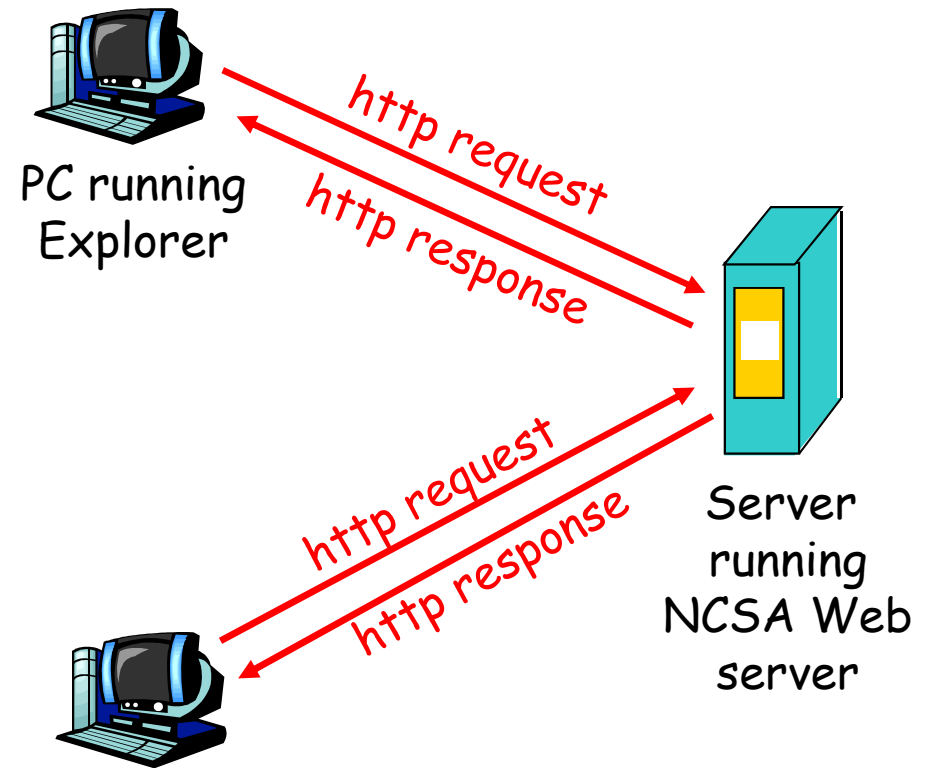
WWW 中的术语

- **Web**页面（网页）
 - 由对象（**object**）组成
 - 用**URL**标示地址
 - 协议类型（**HTTP**、**FTP**、**TELNET**等）
 - 对象所在服务器的地址（域名或**IP**地址）
 - 包含对象的路径名
 - 大部分网页包括基本的**HTML**页面和引用的对象
- 浏览器(**browser**): 用户访问网页的客户端
 - **MS Internet Explorer**
 - **Netscape Navigator**
- **Web**服务器: 存储**Web**对象
 - **Apache**
 - **Microsoft Internet Information Server**
- 超文本传输协议**http**



http

- **http: hypertext transfer protocol**
- **Web**的应用层协议
- 使用**TCP**, **80**端口
- 客户/服务器模型
 - 客户: 浏览器, 发送请求, 接收、显示**Web**对象
 - 服务器: **Web**服务器, 接收请求, 发送**Web**对象
- 无状态协议: **Web**服务器不保存客户信息
- **http1.0: RFC 1945**
- **http1.1: RFC 2068**





http example

Suppose user enters URL

`www.someSchool.edu/someDepartment/home.index`

(contains text,
references to 10
jpeg images)

1a. http client initiates TCP connection
to http server (process) at
`www.someSchool.edu`. Port 80 is
default for http server.

1b. http server at host
`www.someSchool.edu` waiting for TCP
connection at port 80. "accepts"
connection, notifying client

2. http client sends http *request message*
(containing URL) into TCP connection
socket

3. http server receives request message,
forms *response message* containing
requested object
(`someDepartment/home.index`), sends
message into socket

time

A yellow arrow pointing downwards, indicating the progression of time.



http example (cont.)

4. http server closes TCP connection.

5. http client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

time
↓



非持久连接和持久连接

非持久连接 (Non-persistent)

- HTTP/1.0
- 服务器解析请求，发出响应报文后关闭连接
- 每个object的取得都需要两个RTT
- 每个object的传输都要经历慢启动

But most 1.0 browsers use parallel TCP connections.

持久连接 (Persistent)

- default for HTTP/1.1
- 在同一个TCP连接上: 服务器解析请求并响应，再解析新的请求...
- 客户端一旦得到基本的HTML文件就发出请求索取全部object
- 较少的RTT和慢启动时间



Trying out http

1. Telnet to your favorite Web server:

```
telnet www.eurecom.fr 80
```

Opens TCP connection to port 80
(default http server port) at www.eurecom.fr.
Anything typed in sent
to port 80 at www.eurecom.fr

2. Type in a GET http request:

```
GET /~ross/index.html HTTP/1.0
```

By typing this in (hit carriage
return twice), you send
this minimal (but complete)
GET request to http server

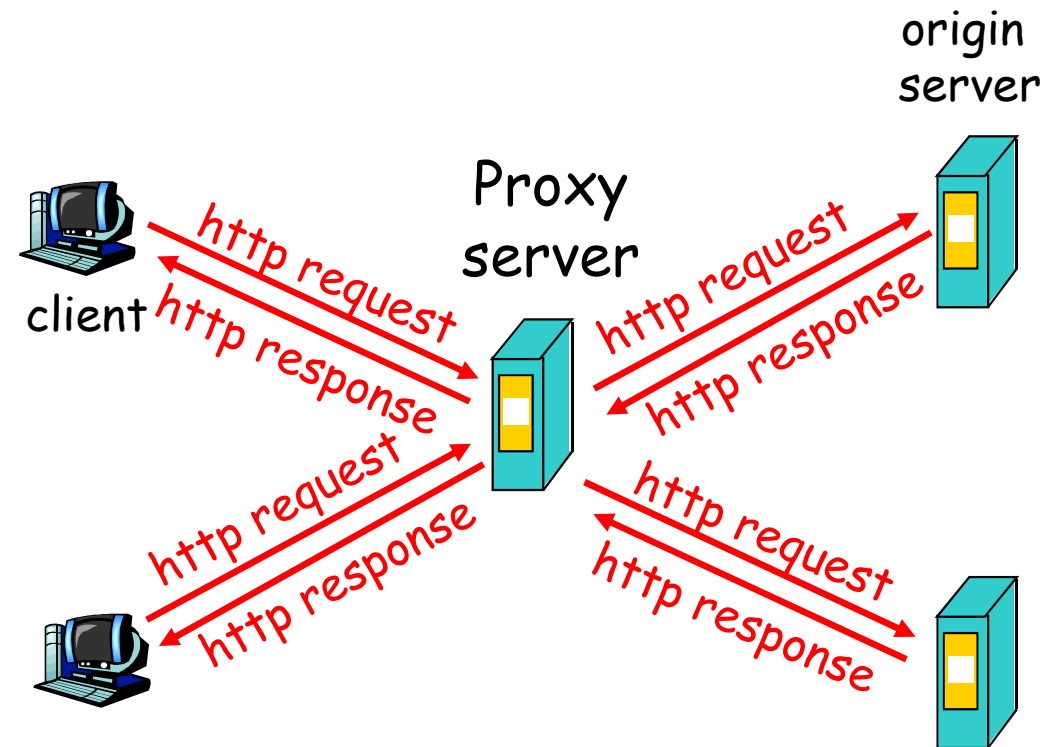
3. Look at response message sent by http server!



Web Caches (proxy server)

- 用户设置浏览器通过Web缓存来访问Web
- 浏览器将所有http请求发给Web缓存
 - 如果所请求object在缓存中，该object被立即通过http回答返回
 - 否则Web缓存向原服务器发请求获得该object，再响应浏览器的请求

目标：不访问原服务器而满足用户的请求

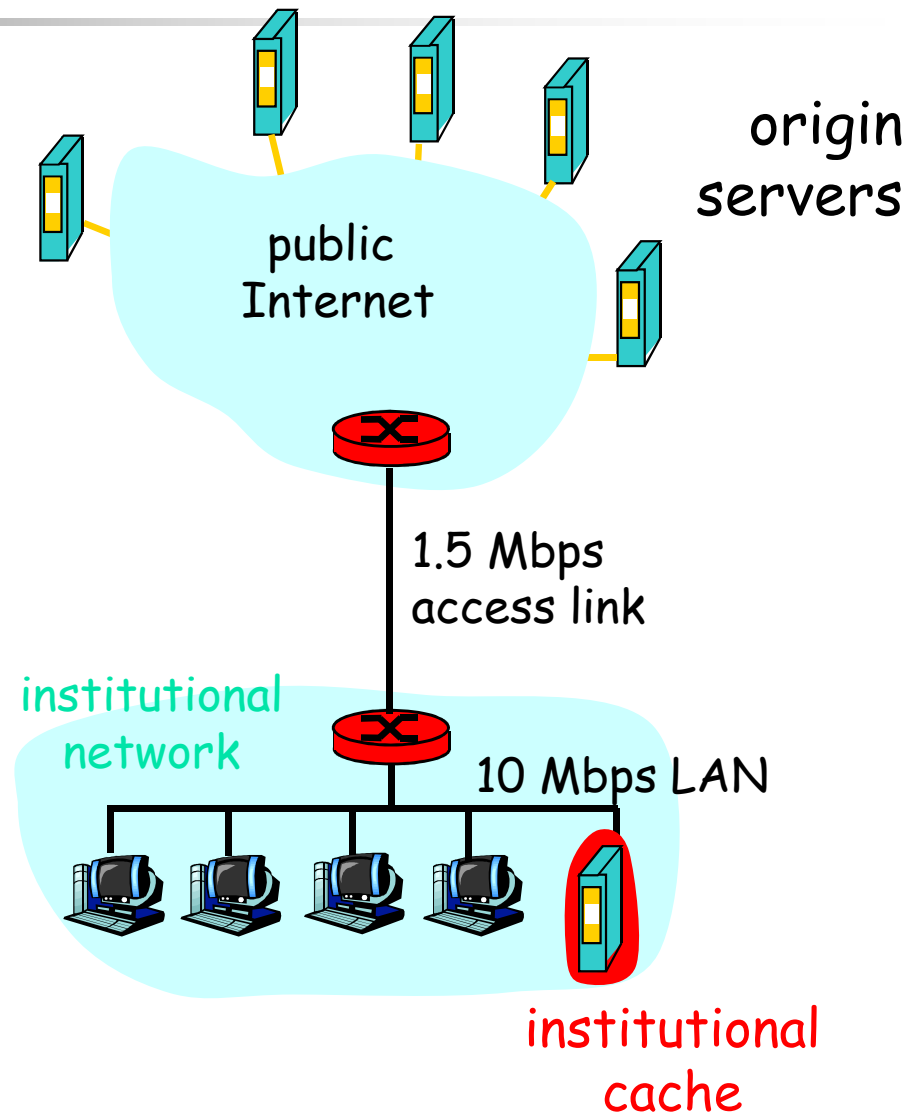




为什么需要Web缓存

假定：缓存离用户端近，
比如在同一网络

- 较小的响应时间
- 减轻通往远端服务器的连接的流量



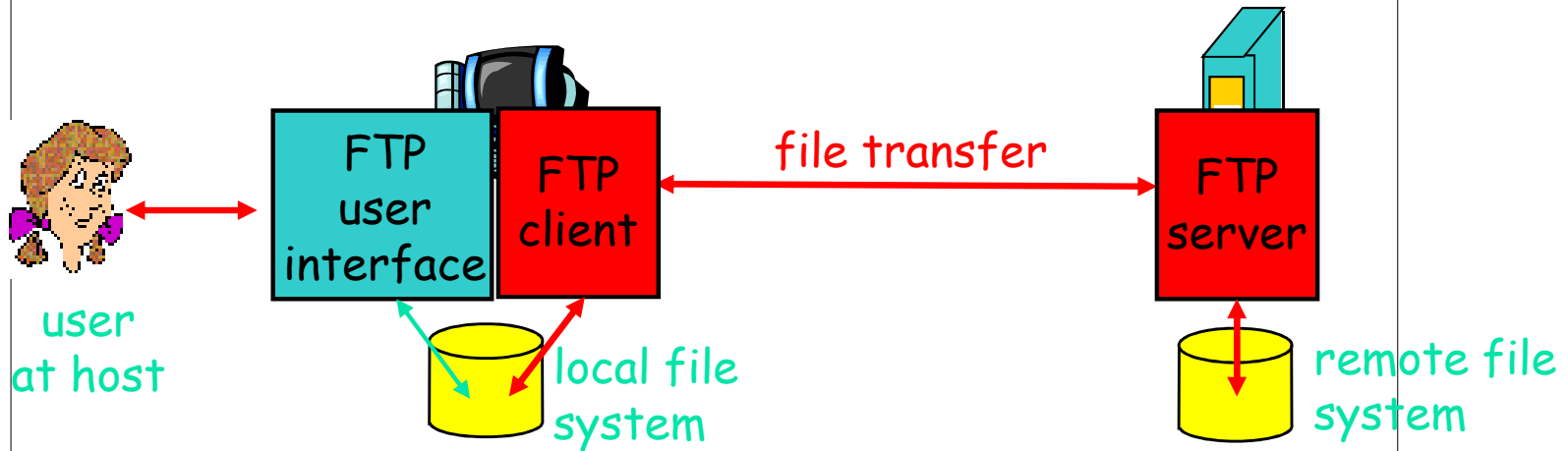


主要内容

- 应用层概述
- 客户/服务器模型
- 域名服务
- 简单网络管理协议
- 电子邮件
- **WWW**
- 文件传输协议



文件传输协议FTP

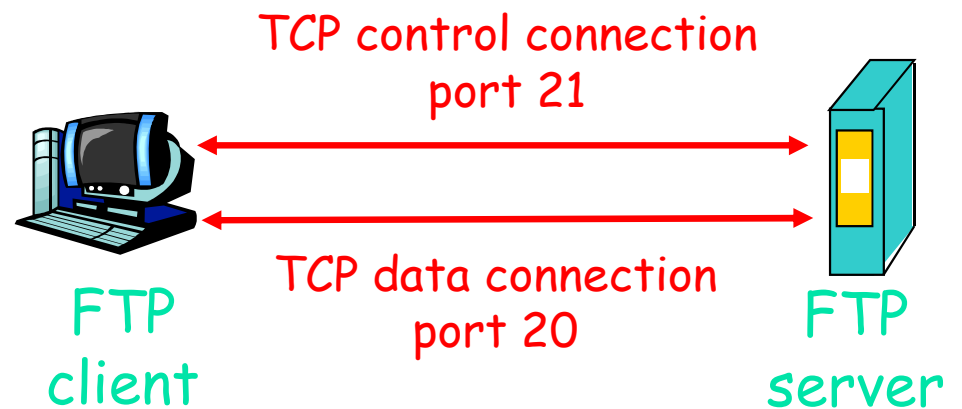


- 在两个主机之间传输文件
- 客户/服务器模式：由客户端发起文件传输(上传或下载)
- ftp: RFC 959
- ftp server: port 21



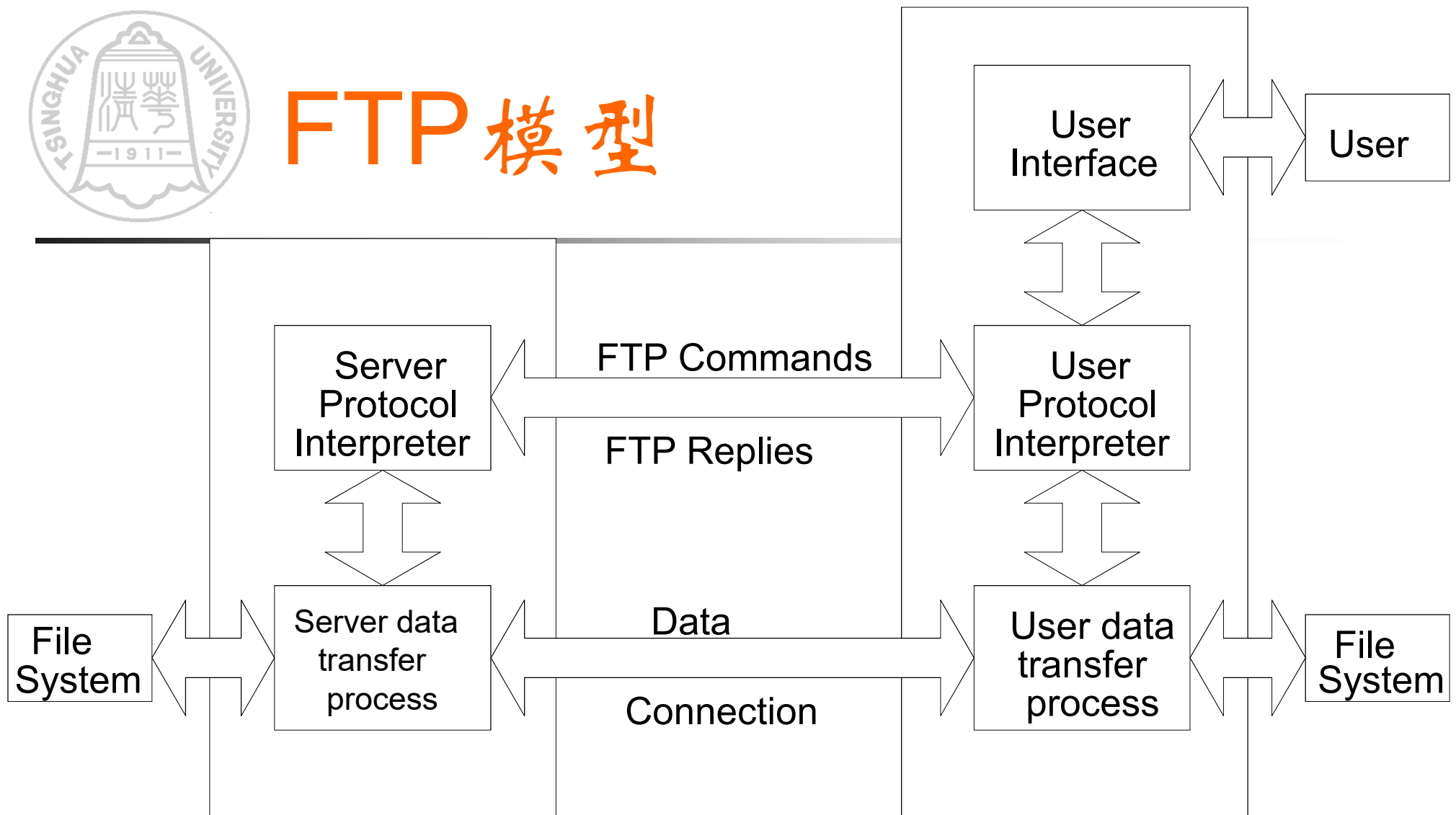
文件传输协议FTP（续）

- 客户端连接到ftp服务器TCP的21号端口
- 建立两个并行的TCP连接：
 - **控制**: 在客户端和服务端之间交换命令、响应
 - **数据**: 传递文件数据
- Ftp服务器维护状态：当前目录，身份认证





FTP 模型



- 注意:
1. 数据连接可以双向使用.
 2. 数据连接不必始终存在.
 3. 控制连接采用的是telnet.